WaveView™ User Guide

Version X-2025.06, June 2025

SYNOPSYS®

Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Feedback

Contents

New in This Release 28
Related Products, Publications, and Trademarks 28
Conventions 29
Customer Support 30
Statement on Inclusivity and Diversity 30

1. **Getting Started** 31

Setting Up 31
Starting the Tool 32
Invoking the WaveView Tool With a Design or Netlist 32
WaveView ADV License Features 32
Setting Environment Variables 33
Setting up the MATLAB Interface to WaveView 35
MATLAB Interface Components 36
Licensing Requirements 36
APIs Supported in MATLAB Interface 36
Limitations 40
Customizing File Browser Filters 41
Configuring "Send To" in the Windows Environment 42
Configuring the WaveView Tool Manually 42
Basic WaveView Usage 43
Supported Command-Line Options 43
WaveView Command-Line Options 43
WaveView ADV Command-Line Options 46
Loading and Saving Sessions 47
Save Session 47
Load Session 48
Viewing Recent Files and File Paths 49
Log Files and Command History 49
Viewing and Saving Log Files 49
Changing the Default Log File Directory 49
Replaying Log Files 50
Viewing Command History 50
Basic GUI Usage 50

Supported GUI Values 50
Viewing the GUI 51
Viewing the File Tab 53
Viewing the Waveform Tab 54
Viewing the Tools Tab 55
Viewing the Configuration Tab 56
Viewing the Waveform Browser 56
Running ACE Scripts 56

2. Importing Files 58
Supported File Formats 58
Additional File Format Information 59
Special Note for the ELDO COU Format 59
Special Note for the ADMS/ELDO WDB Format 60
Special Note for the PSF Format 60
Special Note for the WDF, FSDB, Binary HSPICE tr0, and PSF Formats . . 60
Opening a Waveform File 61
Opening Waveform Files in Case-Insensitive Mode 61
Opening Multiple Files 61
Opening PWL Files 62
Tabulated Data in Real Numbers 63
Tabulated Data in Complex Numbers 64
2-Dimension Sweep Data in Real Numbers 64
2-Dimension Sweep Data in Complex Numbers 65
Measured Data Without Name Header 66
SPICE PWL Sources 66
Opening Err Files 67
Opening RC Files 67
Reading Compressed Waveform Files (Linux Only) 67
Working With WDF Files 67
Converting Waveform Files to WDF 68
Converting Existing Output Files to WDF 68
Lossless Compression 68
Lossy Compression 68
Reducing WDF Data 68
Importing Multiple Files in Virtuoso 69
Virtuoso dcOp Parametric Analysis 69

3. **Working With Waveforms** 71

Using the Waveform Browser 71
Viewing the Waveform Hierarchy Browser 71
Analog Waveform Icon Reference 72
Digital Waveform Icon Reference 73
Non-Digital/Analog Waveform Icon Reference 74
Displaying Signals 74
Pinning a Filter String 74
Viewing Filter History 75
Highlighting Waveforms 75
Deleting Highlighted Waveforms 75
Grouping and Ungrouping Highlighted Waveforms 75
Copying Waveform Names 76
Sending a Waveform to Results Analyzer 76
Waveform Files and Data 76
Grouping Waveform Files 76
Bookmarking Waveform Files 77
Setting the Active Waveform File 78
Updating Waveform Files 78
Printing Waveform Files 78
PostScript Print Layout 78
Printing on Linux Platforms 79
Printing on Windows Platforms 79
Exporting Waveform Data 79
Marching Waveforms 81
Saving and Loading SigLists 81
Dumping the Screen 82
Dump All Screens 83
Setting Waveform Attributes 83
Working With Waveviews 84
The Active Waveview 85
Adding a Waveview 85
Deleting Waveviews 86
Displaying WaveView Windows 86
Stack Mode 86
Row and Column Mode 88
Vertical Row and Column Mode 88
Horizontal Row and Column Mode 88
Tiled Row and Column Mode 89

TableView Mode 89
Displaying All Open Waveviews 89
Displaying Waveview Titles 89
Displaying the Hierarchy and Signal Browsers 89
Displaying the Console Window 90
Undoing and Redoing a Waveview Operation 90
Refreshing Waveviews 90
Renaming Waveviews 90
Clearing Waveview Contents 90
Dumping the Waveview Contents 90
Docking and Undocking Waveviews 91
Toggling the Tab Background Color 91
Changing the Order of Tabbed Waveviews 91
Synchronizing Waveviews 91
Toggling Sync Mode 91
Setting Sync Axes 92
Adding Panels to a Sync Group 92
Deleting All Open Waveviews 92
Using the Drawing Tool 92
Working With Tableviews 103
Working With All Types of Tabular Data 104
Hiding and Showing Rows 104
Sorting Tabular Column Data by Ascending or Descending Order 104
Filtering Tabular Data 105
Working With Tabular Equation Data 106
Modifying Tabular Equations 107
Adding or Removing Tabular Statistics Results 107
Marking Annotating Equation Information From Equation Results 108
Exporting Tables 108
Working With Tabular Signal Data 108
Viewing Waveform Data 109
Working With Tabular Waveform Comparison Data 109
Viewing Tabular Waveform Check Details 110
Viewing Tabular Waveform Check Signal Details 110
Comparing and Checking Waveforms 110
Sampling and Converting Waveform Data 111
Creating a Waveform Compare Control File 111
Defining Aliases 112
Defining Rules 112
Defining Parameters for Comparison and Checks 113

a2d_threshold 114
batchsize 115
bus 116
delay 116
full_compare_log 116
ignorex 117
ignorez 117
len_reltol 117
length_check 117
map 117
mapfile, map_hier 117
master 118
match_type 118
mean_range 118
mean_step 118
mean_tol 119
pattern_check 119
ref_clock 119
report_style 119
report_type 119
report_vio_sig_only 120
show_diff 120
show_maxmin 120
signal 121
start, stop, step 121
t_tol 121
use_a2d_always 122
use_sim_steps 122
v_abstol, i_abstol, v_reltol, i_reltol 122
when 123
x_absmgn 123
x_chkrange 123
x_values 123
xval_precision 124
Controlling X- and Z-Level Constraints 124
Checking Waveforms 125
Waveform Compare Utility 125
Waveform Check Example 126
Checking Waveform Monotonicity 126
Checking Waveform Bounds 126
Checking the Waveform Envelope 127
Checking the Waveform Pattern 127
Rules Section Example 127
Saving Waveform Comparison Results 128

Using the Waveform Compare Tool 128
Specifying Global Settings 129
Adding Rules (Golden Settings) 130
Adding Checks (Target Settings) 133
Running Comparisons 134
Comparing Logic Signals 134
Setting User-Specified Colors for Diff Markers 135

4. Plotting Signals 136
Using Panels 136
Panel Actions 137
Adding Panels 137
Deleting Panels 137
Grouping Panels 137
Creating a Panel Set 139
Selecting Panels 139
Displaying Previous and Next Loaded Signals 139
Adding Labels to a Panel 140
Moving Panels 140
Copying Panels 140
Specifying Panel Attributes 142
Setting Panel Display Options 143
Grid 143
Zoom 144
XAxis 147
YAxis 147
Font 149
Full Scale 149
X Variable 149
Axis Attributes 150
Setting Signal Display Options 151
Full Logic Value 152
Short Logic Value 152
VCD Logic 152
RTF Logic 152
PWL Waveform 152
PWC Waveform 152
Bar Chart 152
Data Point Only 152
Setting Radix Options 153
Setting Signal Name Options 153

Sort . . . 153
Leafname . . . 154
Fullname . . . 154
Fullname+file . . . 154
Fullname+path+file . . . 154
Netname . . . 154
Netname+file . . . 155
Netname+path+file . . . 155
Alias . . . 155
Adjusting Panel Height . . . 156
Fit . . . 157
Default . . . 157
Fit All . . . 157
Default All . . . 158
Manual Adjust . . . 158
Panel Types . . . 158
X-Y Panels . . . 159
Creating an X-Y Panel . . . 159
Plotting X-Y Panel Values in Tabular Format . . . 159
FFT/DFT Conversion . . . 159
A to D Conversion . . . 163
Eye Diagrams . . . 165
Showing Eye Diagram Data Points . . . 166
Displaying the Envelope Center . . . 166
Unfolding Eye Diagrams . . . 166
Tracing Waveform Points in Eye Diagrams . . . 166
Configuring Eye Diagrams . . . 167
Generating Jitter Histograms . . . 172
Displaying the Eye Height Distribution . . . 172
Adding User-Defined Masks . . . 172
Logic Waveforms . . . 175
Mnemonic Mapping for Bus Signals . . . 175
D to A Conversion . . . 176
Tabular Panels . . . 177
Histogram Panels . . . 177
Configuring Histograms . . . 178
Smith Chart . . . 179
Polar Plot Panels . . . 180
2-D Sweeps . . . 180
Displaying Multi-trace Waveforms . . . 181
3-D Sweeps . . . 182
Pair Plot . . . 182

Constell Plot 183
Panel Set 185
Renaming a Panel Set 185
Expanding a Panel Set 185
Creating a Panel Set From Selected Panels 185
Rearranging Panel Set Panels 186
Parametric Plots 186
Built-in Parametric Tool for PrimeSim HSPICE .ALTER Simulations 186
Plotting Clock Jitter 186
Selecting Reference Signals 187
Specifying the Target Signal 187
Selecting Reference Edges 187
Target Signal Edges 188
Jitter Types 188
Jitter Output Options 188
PrimeSim HSPICE QQ Plot 189
Searching Signals 190
Scanning Signals 191
Scan Settings 191
Data Reduction 192
Direct Plot Mode 192

5. Measuring Waveforms 194
Using Cursors 194
Adding a Cursor 195
Selecting With the Cursor 196
Active Cursor and Reference Cursor 196
Moving a Cursor 197
Showing and Hiding Distance Lines 197
Setting Distance Between Two Cursors 198
Locking Pairs of Cursors 198
Locking Cursor Position 198
Snapping the Cursor to an X/Y Value 198
Snapping the Cursor to a Jump Type 198
Setting the Cursor Alias Name 199
Showing and Hiding Cursor Crossing Values 199
Cursors in Smith Charts and Polar Plots 199
Cursors in 2-D Sweep Panels 200
Cursors in 3-D Sweep Panels 200
Adding a Horizontal Cursor 200

Annotating Horizontal Cursors 200
Hiding Horizontal Cursors 201
Anchoring Horizontal Cursors 201
Exporting Horizontal Cursor Values 201
Adding a Cross-Cursor 201
Jumping Forward and Backward 202
Jump Settings 203
Deleting Cursors 203
Deleting All Cursors 203
Assigning Cursors to Monitors 203
Using Cursor De-Select Mode 204
Using Monitors 204
Adding Monitors 205
Copying Monitor Values 206
Reconfiguring Monitors 206
Deleting Monitors 206
Linking Monitors to Cursors 206
Exporting Monitors 206
Using the Measurement Tool 206
Setting the Measurement Font and Color 207
Adding or Removing Measurement Favorites 207
Viewing Measurement Results 208
Setting the Precision of Measurements 208
Exporting Measurements 208
Locking and Unlocking Measurements 209
Supported Measurements 209
General Measurements 212
Data(X,Y) 212
Difference 213
Length 214
Width 215
Y Diff 215
Y Range 215
Time Domain Measurements 216
Delay 216
Duty Cycle 217
F VS T 218
Frequency 219
Jitter 220
Overshoot 221

Rise/Fall Time 222
Settle Time 223
Slew Rate 224
Undershoot 224
Frequency Domain Measurements 225
Highpass 225
Lowpass 226
Bandpass 227
Stopband 228
Gain Margin 229
Phase Margin 229
Statistical Measurements 229
Cpk 230
Dpu 230
Mean 231
Mean+3std_dev 231
Mean-3std_dev 231
Std_dev 231
Yield 232
Level Measurements 232
AC Coupled RMS 233
Amplitude 233
Average 233
Baseline 233
Maximum 233
Minimum 233
Peak to Peak 233
RMS 234
Topline 234
X at Maximum 234
X at Minimum 234
S Domain Measurements 234
Damping Ratio 234
Frequency Value 234
Natural Frequency 235
Quality Factor 235
RF Measurements 235
P1dB 235
IP2 235
IP3/SFDR 235
Measuring Eye Diagrams 236
Measuring Eye Size 238
Eye Measurement Methodology 239

Measuring Aperture 240
Measuring DDR Read 241
Measuring DDR Write 242
Measuring Overshoot 242
Measuring Diamond Openings 242
Measuring Rectangle Openings 243
Measuring Hexagon Openings 243
Measuring DDR4 243
Measuring DDR5 245
Measuring GDDR5 246
Measuring PAM-N 246
Measuring Mask 251
Measuring Stateyes 251
Margins vs Mask Measurements 254
Using the Setup/Hold Results Viewer 256
Creating ACE Scripts From DDR Measurements 257
Clearing Eye Measurements 257
Copying Eye Diagram Settings 257
Pasting Eye Diagram Settings 258
Measuring DDR3 Eye Diagrams 258
Measuring Derate-Based Setup and Hold Times 258
Using the tVAC Measurement 263
Measuring Ringback 265
Sample Derating Table File 268
Measuring DDR4 Eye Diagrams 269
Taking DDR4 Measurements 269
Taking TdIVW and VdIVW Margin Measurements 275
Aligning the Aperture for LPDDR4 Measurements 277
Shifting the Trigger Clock Delay and Delay Clock to the Aperture Center 278
Sample DDR4 Specification File 279
RF Measurements 279
Measuring Circles 280
Measuring Noise Circles 281
Measuring Stability Circles 281
Measuring Operating Power Gain Circles 282
Measuring Available Power Gain Circles 283
Measuring VSWR Circles 283
Exporting Equations From RF Tool Setup 284
PrimeSim HSPICE Measure Tool 284

Pareto Analysis 284

6. **Using the Equation Builder** 288

Viewing the Equation Builder 289
Viewing the Equation Builder Keypad 290
Using the Text Editor 291

Adding an Equation 291
Importing PrimeSim HSPICE .meas Statements as Equations 292
Duplicating an Equation 292
Modifying an Equation 292
Removing an Equation 292
Registering a Tcl Process as an Equation 293

Building an Equation 293
Adding Signals to an Equation 293
Adding Multiple Signals From Different Waveform Files to an Equation 294
Adding Functions to an Equation 295
Viewing Function Arguments 295
Function Nesting 296
Adding User-Defined Functions 296
Adding User-Favorite Functions 296
Adding .MEASURE Statements to an Equation 296
Specifying Min/Max for an Equation to Test for TableView Failures 296
Naming an Equation 297

Setting the Target File of an Equation 297
Changing the Target File of an Equation 298
Changing the Index Number of a Target File 298

Evaluating Equations 299
Evaluating Sub-Equations 299

Plotting Results 299
Plotting All Results to TableView 300
Plotting Selective Signals to WaveView 300

Marking Annotation Information From Equation Results 300

Setting Up Macros 301

Saving DDR Equations as ACE Scripts 302

Calculating Multi-trace Waveforms 302

7. Supported Functions 303

Differences Between PrimeWave Expression Syntax and Standalone WaveView Expression Syntax 303

Signal name Syntax 303

Add Signals From Different Waveform Files 307

Operators, Constants, and Logic Operations 309

Supported Operators 310

Supported Constants 310

Supported Logic Operations 311

Mathematic Functions 311

abs 313

acos 314

acosh 315

asin 315

asinh 316

atan 316

atanh 317

complex 317

conjugate 318

cos 318

cosh 319

db10 319

db20 320

dbm 321

exp 321

floor 322

imag 322

int 323

ln 323

log10 324

mag 325

max 325

min 326

phase 327

pwr 328

real 328

round 329

sgn 330

sin . . . 330
sinh . . . 331
sqrt . . . 331
tan . . . 332
tanh . . . 333
Measurement Functions . . . 333
acrmsval . . . 337
avgdev . . . 338
avgfreq . . . 338
bw . . . 339
bw3db . . . 340
correlation . . . 341
cross . . . 342
dBCP . . . 344
dBCPh . . . 346
dBCPhVRI . . . 347
dcgain . . . 348
dcgamos . . . 350
dcswing . . . 352
delay . . . 353
direct_ipn . . . 357
dutycycle . . . 358
enb . . . 360
eye_mask . . . 362
falltime . . . 363
falltrig . . . 365
flatten . . . 367
gain1_f . . . 368
gain_m . . . 369
gbp . . . 370
get_harmonic_freq . . . 371
globalmax . . . 372
globalmean . . . 373
globalmin . . . 373
globalstddev . . . 374
integ . . . 374
ipn . . . 375
ipnh . . . 377

ipnhVRI 378
mean 380
measure_aperture 381
measure_eyejitter 382
measure_eyeopen 384
measure_eyesize 385
median_val 387
overshoot 387
p2p 389
pam4_ser 390
pamn_level_at_eye_center 397
pamn_level_at_user_ref 398
pamn_open_at_eye_center 399
pamn_open_at_user_ref 401
paramvalue 402
phase_m 403
risetime 403
risetrig 405
rmsval 407
root 408
settling_t 409
sfdr 413
signalPower 415
slewrate 417
slope 419
snb 420
sndr 422
snr 425
snr_f 427
snrh 428
stddev 430
sweepValueDiff 431
thd 432
variance 434
x_at_ymax 435
x_at_ymin 436
xmax 437
xmin 438
yield 439

ymax 440
ymin 441
yvalue 442
Waveform Functions 443
a2d 448
average 449
bestfit 450
clip 451
concat 452
cPwrContour 453
create_bus 454
create_eye 455
crossings 456
d2a 457
dBc2V2 458
dBCPVRICurves 459
delayvst 460
derivative 461
dft 463
dftbb 465
dnl 468
dumpPWLF 470
duty_vs_t 471
fallvst 473
fft 474
flip 476
fourEval 477
func 477
fvst 478
gdelay 479
histo 480
hold_times 482
hold_times2 484
ift 485
inl 486
integral 488
ipnhVRICurves 489
join 490

jvst_freq 491
jvst_ideal 493
jvst_llse 495
jvst_refsig 497
loadpull 499
max_window_average 500
median 501
optical_fft 502
parametric 504
pareto_r 505
pareto_r2 508
pareto_s 510
pass 513
pavg 514
peak 516
phasenoise 517
prms 518
psd 520
psdbb 522
pstddev 523
pwc 526
pzFreqRealFilter 527
reflectionContour 528
repeat 529
risevst 530
sample 531
setup_times 532
setup_times2 533
signalAtFreq 535
signalcompare 535
slewratevst 536
smooth 537
spectralPower 537
sPowerDiffVI 538
sPowerDiffVR 539
swapsweep 539
sweepavg 541
sweepmax 542
sweepmin 542

tangent . . . 543
text2wave . . . 543
trace . . . 544
v2todBc . . . 545
window_average . . . 546
x_at_y . . . 546
xscale . . . 547
xshift . . . 548
xval . . . 548
yvsi . . . 549
RF Functions . . . 549
apgc . . . 552
b1f . . . 554
ga . . . 555
gac_freq . . . 556
gac_gain . . . 557
gmax . . . 558
gmin . . . 559
gmsg . . . 559
gp . . . 560
gpc_freq . . . 561
gpc_gain . . . 562
gt . . . 563
gumx . . . 564
harmonic . . . 564
hp . . . 566
imProd . . . 567
kf . . . 567
lsb . . . 568
lq . . . 569
mu1 . . . 570
mu2 . . . 570
nc_freq . . . 570
nc_gain . . . 571
nf . . . 572
nfmin . . . 572
noisec . . . 572
opgc . . . 574

rn 576
Rn 577
sp 577
spm 578
ssb 579
stabc_in 580
stabc_out 582
vswr 584
vswrc_in 585
vswrc_out 587
ym 589
yp 590
zm 591
zp 592

8. Working With Designs 594

Importing a Design 594
Importing Scripts for VCS 3 Step Flow 596
Updating a Design 597
Removing a Design 597
Applying RC Reduction to a Design 597
Using DesignView 597
Opening DesignView 598
Traversing Designs 598
Viewing the Design Browser 598
Design Display Settings 599
Viewing Design Hierarchy 600
Viewing Design Cells 602
Viewing Design Files 602
Filtering Signals, Elements, and Files 602
Searching Designs 602
Generating Reports 604
Param Report 604
W/L Report 605
Generating a Flattened Netlist 607
Extracting Measure Statements 607
Extracting PWL Source 609
Converting Stimuli 610

Using the Current Debug Tool 610
Invoking the Current Debug Tool 611
Viewing the Current Debug Dialog Box 611
Single Time Point Current vs. Average Current 611
Setting Time 612
Additional Options 612
Using SourceView 612
Using Netlists 613
Using Bookmarks in your Netlist 613
Searching Netlists 614
Hiding Netlist Line Numbers 614
Finding Signal Connections 614
Traversing Connections 615
Tracing Connections 615
Displaying Trace Connections 615
Suppressing Out of Scope Warnings 616
Using ConnectionView 616
Selecting and Deselecting Items 617
Zooming In and Out 618
Viewing Full and Current Hierarchy Connections 618
Viewing the Total Load for Nets 618
Hiding and Showing Instances, Nodes, or Terminals 618
Showing Full and All Sub-level Full Connections 618
Expanding All Sub-Level Instances 619
Showing Single Connections 619
Showing Connections Through All Scopes 619
Showing Lumped Capacitance 619
Working With Hierarchical Labels for Instances or Nodes 619
Adding Labels 620
Moving Labels 620
Deleting Labels 620
Calculating Resistance 620
Setting Global VDD and GND Signals 621
Setting the Maximum Displayed String Length 622
Setting Parasitic RC Filters 622
Enabling Warnings for Traced VDD or GND Signals 623
Enabling Large Resistors to Be Opened 623
Setting Highlighting Style for Selected Instances or Nodes 624
Merging Parallel Instances 624

Changing ConnectionView Display Settings 625
Changing Subcircuit Display 625
Changing Instance Visibility Display 626
Displaying Key Bindings 626
Using SPICE Lint 626
Selecting Linter Rules and Running the SPICELinter 627
Viewing Linter Options 627
SPICELint Syntax Options 628
SPICELint Connectivity Options 630
SPICELint ERC Checks Options 631
SPICELint Parameter Options 631
Viewing Linter Results 633
Saving SPICELint Results 634
Displaying the LintView on Startup 634
Annotating a Netlist 634
On/Off 634
Setup 636
Binding Waveforms 636
Enabling and Disabling Linked Waveform Data 637
Removing Linked Waveform Data 638
Cosim 638
Config 638
Link A/D Report 638

9. Using WaveView Tools 639
Using the PWL Editor 639
File Operations 639
Source Operations 640
Point Operations 640
Using the ADC Toolbox 641
Generic Versus Coherent ADC Toolbox 642
Overview of the ADC ToolBox 643
Input Signal of ADC Toolbox 643
DC DNL/INL Analysis 645
Sine Input 645
Ramp Input 646
Number of Sample Points and INL/DNL 646
Sampling Frequency and INL/DNL 646

AC Analysis: Coherent Sampling Versus Window Sampling 646
Coherent Sampling 646
Window Sampling 648
Exporting the DNL/INL/FFT Results as Waveform Data 649
Saving or Loading an Analysis Setup 649
Dumping the Contents of the ADC Toolbox 649
ADC Toolbox Display Controls 649
Sampled Signal Window 649
Histogram, DNL, and INL Windows 650
Power Spectrum (Sine Input Only) 650
AC Dynamic Characteristics 651
DC Static Characteristics 651
Running Batch-mode ADC Analyses 651
Using the Coherent Sample Only (CSO) ADC Toolbox 652
Input Signal Requirements 653
Preparing the Input Waveform 653
Sampling the Input Waveform 654
Preparing Test Benches for the CSO ADC Toolbox 655
Input Signal Strength 655
Input Sine Wave Frequency 655
CSO ADC Toolbox User Interface 655
Input Parameters 655
Selecting Sampling Parameters 657
Sine Input 657
Ramp Input 658
Exporting DNL/INL/FFT Results as Waveform Data 658
Saving or Loading an Analysis Setup 658
ADC Toolbox Display Controls 658
Sampled Signal Window 658
Histogram, DNL, and INL Windows 659
Power Spectrum (Sine Input Only) 659
AC Dynamic Characteristics 660
DC Static Characteristics 660
Using the CCK Viewer 660
Loading CircuitCheck Error Reports 661
Exporting CircuitCheck Error Reports 662
Using CSA (Channel Simulation Analysis) 662
Getting Started With the CSA Environment 662

Customize 666
Setting the Initial Design Bit Rate 667
Setting the Initial Resistance for Terminating Pins 667
Saving Testbench Stages When Exiting CSA 667
Setting Up CS Testbenches 668
Working With Testbench Stages 668
Setting Up Input Element Options 671
Setting Up TxPkg Options 673
Setting Up PCB Options 675
Setting Up RxPkg Options 677
Setting Up Buffer Options 679
Setting Up Output Element Options 681
Working With the CS Connection View 682
Setting Up CS Simulations 685
Specifying General CS Simulation Settings 685
Adding CS Model Files 686
Adding CS Include Files 687
Adding CS VEC Files 688
Adding CS Design Variables 688
Adding CS Analyses 689
Adding CS Sweep Data 690
Adding CS Simulation Options 694
Adding Additional CS Commands 695
Adding CS Probes and Equations 695
Adding CS Probes 696
Adding CS Equations 697
Plotting Result Lists 699
Adding CS Corners 700
Generating CS Corners 701
Converting CS Corners to CS Sweeps 702
Show Netlist 702
SPUTIL 702
Adding and Editing S-Parameter Commands 703
Working With Runsets 704
Setting S-Parameter Simulation Options 705
Running and Stopping S-Parameter Simulations 705
Saving and Loading SPUTIL Files 706
Working With Results 706
Run 707
Job Control 708
Setting Run Options 708
Using ADE Post-Process Integration 709

Using the Results Viewer 710
Viewing the Results Viewer GUI 710
Configuring and Filtering Results and Table Content 710
Exporting Results 711
Basic Analysis Results 711
Summary Table 711
Detailed Table 711
Advanced Analysis Results 712
Corner and Sweep Analysis 712
Monte Carlo Analysis 713
Device Aging Results 714
Aging Time 715
Device Degradation Results 715
Advanced Plots 715
Histogram 715
Scatter 716
Pair Plot 717
QQ Plot 717

10. Customizing the WaveView Tool 718

Setting Key Bindings (Shortcut Keys) 718
Customizing Shortcut Keys 718
Reading the Key Binding File 719
Top Menu-Operated Shortcut Key Functions 720
WaveView Toolbar Shortcut Key Functions 722
Other Shortcut Key Functions 723
Setting Preferences 727
General Settings 727
Readers Settings 730
Waveview Settings 732
Panel Settings 736
Signals Settings 738
Axis Settings 741
Color Settings 742
Threshold Settings 743
CDS Link Settings 744
Saving Preference Files 745
Loading Preference Files 746
Mapping Bus Values 746

11. Troubleshooting 748

Linux Platforms 748

Screen Refresh 748

Alternative Methods for Backingstore Setup 749

Locate the XServers File 749

Enable Backing-store for Your Xserver 749

Restart Your Xserver to Enable the Changes 749

With GNOME 749

Program Fails During Startup on Linux Platforms 750

X-Window Font Warnings 750

XmTextField Font Warning 751

Cannot Change Flexlm License File 752

About This User Guide

This guide describes the WaveView waveform viewer. The WaveView tool is a graphical waveform viewing and analysis program for analog/mixed-signal IC design simulations, which can help you to use simulation tools more effectively.

This preface includes the following sections:

- New in This Release
- Related Products, Publications, and Trademarks
- Conventions
- Customer Support
- Statement on Inclusivity and Diversity

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the WaveView Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the WaveView tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

https://solvnetplus.synopsys.com

You might also want to see the documentation for the following related Synopsys products:

Synopsys® WaveView™

Synopsys® PrimeWave™ Design Environment

PrimeSim™ XA

PrimeSim™ HSPICE®

PrimeSim™ Pro

PrimeSim™ SPICE

VCS® PrimeSim™ AMS

Feedback

Synopsys® CustomSim™

Synopsys® FineSim®

Synopsys® HSPICE®

Synopsys® VCS®

Synopsys® Verdi®

Cadence® Spectre® Circuit Simulator

MathWorks® MATLAB

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
`Courier`	Indicates syntax, such as `write_file`.
`Courier italic`	Indicates a user-defined value in syntax, such as `write_file` *`design_list`*
`Courier bold`	Indicates user input—text you type verbatim—in examples, such as `prompt> write_file top`
`Purple`	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as `include_enclosing = true | false`
[ ]	Denotes optional arguments in syntax, such as `write_file [-format fmt]`
...	Indicates that arguments can be repeated as many times as needed, such as *`pin1 pin2 ... pinN`*.
|	Indicates a choice among alternatives, such as `low | medium | high`
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.

Convention	Description
Edit > Copy	Indicates a path to a menu command, such as opening the **Edit** menu and choosing **Copy**.
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

https://solvnetplus.synopsys.com

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to https://solvnetplus.synopsys.com.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Getting Started

This chapter details getting started with the WaveView tool.

The WaveView tool is a graphical waveform viewing and analysis program for analog/mixed-signal IC design simulations, which can help you to use simulation tools more effectively with:

- High-performance waveforms database I/O to access large amount of simulation data efficiently.
- Extensive mixed-signal display functions and analysis capabilities to extract design parameters from simulation result.

This chapter contains information on the following topics:

- Setting Up
- Basic WaveView Usage
- Basic GUI Usage
- Running ACE Scripts

Setting Up

This section contains information on the following topics:

- Starting the Tool
- Setting Environment Variables
- Setting up the MATLAB Interface to WaveView
- Customizing File Browser Filters
- Configuring "Send To" in the Windows Environment
- Configuring the WaveView Tool Manually

Feedback

Starting the Tool

Before starting the WaveView tool, consider any environment options you might want to set. See Setting Environment Variables for more information.

On Linux platforms, enter `wv` at the command line to start the WaveView tool.

Note:

If you have a WaveView ADV license and want to enable ADV features, enter `wv -adv` at the command line to start the tool.

See Supported Command-Line Options for a list of available command-line options you can use when starting the WaveView tool.

Invoking the WaveView Tool With a Design or Netlist

Note:

This feature requires a WaveView ADV license.

You can invoke the WaveView tool from the command line.

To invoke the WaveView with a netlist or design, enter the following line at the command line from the location where you have your design saved:

▶ `wv -adv <design_or_netlist>`

Your design opens in a design tab in the WaveView window.

All displayed sections are synchronized in a DesignView. For example, when you select a scope in the **Design Hierarchy**, the **Signal List** displays all signals in the current cell, the **SourceView** display jumps to the first line of the cell definition, and the **ConnectionView** displays the relative cells as well.

Note:

Unless you are opening a previously saved session, the LintView pane might not be visible. See Displaying the LintView on Startup for more information on making the LintView window visible whenever you invoke the WaveView tool.

WaveView ADV License Features

The following features are only available if you have a WaveView ADV license.

- Importing and working with designs
- Waveform compare
- DDR measurements

Feedback

- ACE scripting
- Channel Simulation Analyses (CSA)

Setting Environment Variables

When starting the WaveView tool, you might want to set one or more of the following environment variables.

- `SW_SX_QUEUE_LIC` (not supported on Windows platforms)

  Set to 1 to enable the license queuing feature for all WaveView licenses (including `CDS_link` licenses). Flexlm license queuing is not yet supported.

- `SW_SX_FAST_COU`

  Set to 1 to enable the fast COU file reader (the default is 0). Fast COU reader reads multi-run COU files much faster.

- `SW_SX_FAST_JWDB`

  Set to 1 to enable the fast WDB file reader (the default is 1). Fast WDB reader does not require the Java server.

- `SW_SX_HELP` (Linux only)

  Points to the directory that contains the WaveView online help.

- `SW_SX_INIT`

  Defines the initialization file (`.spxinit`) for the top-level customization. Ensure you set this variable to the `.spxinit` file name you use.

- `SW_SX_INIT_DIR`

  Defines the WaveView initial startup directory.

- `SW_SX_LOG_DIR` (Linux only)

  Defines the output directory for the `sxcmd.log` log file. The default location is the current working directory.

- `SW_SX_MASKFILE`

  Points to user-defined mask files for eye diagrams.

- `SW_SX_NO_ADV_LIC`

  When set to 1, WaveView does not check out an ADV license in any condition.

Feedback

When set to 2, WaveView does not check out an ADV license in WaveView-only mode (when invoked with command `wv`); WaveView does check out an ADV license if WaveView ADV mode (when invoked with commands `wv -adv`, `sx`, or `cx`).

When not set, WaveView does not block checking out a WaveView ADV license.

- `SW_SX_NO_APX_LIC`

  Set to 1 to indicate that no `sx_apx` license is present (the default is 0).

- `SW_SX_ORG_ADC`

  Set to 1 to use the old version of the ADC Toolbox.

- `SW_LIC_TIMEOUT`

  Sets the amount of time that the WaveView tool holds on to a license when idle. Once the set time is expired, a dialog box asks you to either reclaim the license or exit and save the session. The minimum value is 30 (minutes). Supported for all license checkouts, including `sx_cdslink`.

- `SX_WAIT_LICENSE_TIMEOUT`

  Sets the amount of time (in minutes) to wait for a license. When set, the WaveView tool exits when the license queuing timeout occurs.

- `SW_SX_TK_LIB`

  Points to the Tk runtime library for running ACE

- `SW_SX_TMP_DIR` (Linux only)

  Points to the temporary directory for reading a compressed (gzipped) waveform file. Default directory for temporary uncompressed files is the current working directory.

- `SW_SX_USE_AMAP` (Linux only)

  Set to 1 to use the map files in the Artist amap/ directory to map signal names.

- `SW_SX_USE_TK`

  Set to 1 to enable using Tk scripts in the WaveView tool.

- `SW_WLF_READER`

  Points to the wlf2sx executable for reading the ModelSim WLF format.

- `SX_HOME` (Linux only)

  Defines the WaveView home directory. The WaveView tool searches for the .spxrc file in $SX_HOME if SX_HOME is defined.

Feedback

- `SW_SX_64`

  Set to 1 to point to the 64-bit binaries.

- `CX_DRTAB_FILE`

  Specifies a DRTAB mask spec files, which includes the following units:

  - minimum target slew (in V/ns units)
  - minimum target VIHL_AC (in V units)
  - minimum target TdiVW_Total (in s units)
  - minimum Tdipw (in UI units)

  The following sample is a DRTAB mask spec file:

```
#ddr4_spec DDR4_1600
#ddr_ui  660p
#vref_set_tol 0.15
#min_aperture 320p
#min_dataslew 1
#min_tdipw_ui 0.58
#min_vihl_ac 186m
```

- `SW_SX_NTHREAD`

  Specifies the maximum thread numbers to use for plotting and calculations.

Setting up the MATLAB Interface to WaveView

The MATLAB interface to the WaveView tool includes features like reading in all WaveView supported file formats, including sweep signals (from group files as well), and creating parameter filters to read in only specified traces.

Note:

For more information on MATLAB, see the MATLAB documentation from MathWorks®.

This section contains information on the following topics:

- MATLAB Interface Components
- Licensing Requirements
- APIs Supported in MATLAB Interface
- Limitations

Feedback

MATLAB Interface Components

A shared library is included in the WaveView tool installation, which includes reader code for waveform file formats that the WaveView tool supports. The shared library works in compiled MATLAB, and can also be used in other C/C++ applications since it is a standard C library. The 64-bit RHEL5 and RHEL6 platforms are supported.

A set of APIs is also included, which is used to open files, read signal names, read data, handle sweep signals, and close files.

The header file for the interface (`matapi.h`) is located in the following directory:

`<installation_dir>/etc/include/`

The shared library for the interface (`libmatapi.so`) is located in the following directory:

`<installation_dir>/platforms/linux64/lib/`

Before loading the MATAPI library on Linux platforms, first include the reader libraries in `LD_LIBRARY_PATH`. The reader libraries are included in the WaveView tool installation package.

Licensing Requirements

WaveView license (`sx_wva`) is checked out when you use the APIs in the library.

If an `sx_wva` license is already checked out, the API is executed; otherwise, the `sx_wva` license is checked out and the API is executed.

APIs Supported in MATLAB Interface

Here are the APIs are supported.

API Name	Function
`HWRF wv_open_file(char *file_path)`	Opens a waveform file (including .grp files). Returns a void* type result, which is used as `file_handle` in the subsequent APIs.
`void wv_close_file(void* file_handle)`	Closes an opened waveform file.
`int wv_num_signals(void* file_handle)`	Gets the number of signals in a waveform file.

Feedback

API Name	Function
`char ** wv_list_signals(void* file_handle, char* pattern, INT4 *count)`	Lists the signals in the file. This API also performs the following functions: • Returns an array of signal names. • Saves the number of matching names in the INT4*count. • Returns only matched signals when a pattern is specified. Question marks (?) and asterisks (*) are supported in patterns. **Note:** This API might use a large amount of memory for a large list.
`HSIG wv_get_signal(HWRF file_handle,char* signal_name,INT4 signal_type,INT4 *pidx)`	Gets the signal object in the `file_handle` from the specified signal name and signal type. This API also performs the following functions: • Returns the signal object handle, which is used as `signal_handle` in subsequent APIs. • Defines `signal_type` in the `SIGTYPE` enumeration. This can be useful for multiple signals with the same name, but differing types. • Stores the return bit index in `pidx` if `psig` is a bit of a bus signal. For example, `A[2]. pidx` is set to `-1` if `psig` is an analog signal or if `psig` is a bus signal.
`int wv_load_signal(void* file_handle, HSIG signal_handle, struct wv_sig_data **data)`	Loads data for the given signal. Analog real, analog complex, digital, and bus signals can all be loaded. Single trace and multiple traces on analog signals are also supported. This API also performs the following functions: • Saves the signal data in `struct wv_sig_data **data`, which is an array of the `wv_sig_data` structure. • Saves the signal data in each element of data, which saves one trace of the data. • Includes the parameter name values if they exist, as well as the X and Y pairs for each trace. • Returns magnitude and phase Y values for complex data. • Returns the number of traces for sweep signals.
`void wv_unload_sig_data(HWRF file_handle,HSIG signal_handle)`	Unloads the signal data that are saved in the `wv_sig_data` structure array, which `wv_load_signal` returns.
`void wv_unload_signal(HWRF file_handle,HSIG signal_handle)`	Unloads the waveform data loaded into the memory by the waveview reader.

Feedback

API Name	Function
`int wv_num_sweeps(void* file_handle)`	Gets the number of sweeps of a file.
`int wv_signal_num_sweeps(void* signal_handle, int nofilter)`	Gets the number of sweeps of the specified signal. Gets the number of sweeps after a sweep filter when `nofilter` is specified. When set to `1`, the original sweep number of the signal is returned. If set to `0`, the sweep number after filter any filter applied through `wv_set_sweep_filter()` is returned.
`char ** wv_get_param_names(void* file_handle, int *nparams)`	Returns the sweep parameter names of the file in a string array. The number of parameters is set in `int *nparams`.
`char ** wv_get_signal_param_names(void* signal_handle, int *nparams)`	Returns the sweep parameter names of the specified signal in a string array.
`char** wv_get_param_values(HWRF file_handle,char *pname,INT4 *isStr,INT4 *nvals,double *dvals)`	Gets all the values for a specified parameter in a double array or string array. If the parameter has string values, the string array is returned as return value of the function. If the parameter has double values, the values are set in the `double *dvals` array. The number of values is set in `int nvals`.
`char** wv_get_signal_param_values(HSIG signal_handle,char *pname,INT4 *isStr,INT4 *nvals,double *dvals)`	Gets all the values for the specified parameter of a specified signal in a double array or string array. If a parameter has string values, the string array is returned as the return value of the function. If a parameter has double values, the values are set in the `double *dvals` array. The number of values is set in `int nvals`.
`int wv_sweep_conditions(void* file_handle, int tidx, struct param_cond **cond)`	Gets parameter values for the tidx trace. This API also performs the following functions: • Supports both string and double type parameter values. • Returns parameter conditions as array of the `wv_param_cond` structure. Each element of the array contains one parameter and its value. • Returns the number of parameters.

Feedback

API Name	Function
`int wv_signal_sweep_conditions(void* signal_handle, int tidx, struct param_cond **cond,int nofilter)`	Gets parameter values for the tidx trace of a specified signal. This API also performs the following functions: • Supports both string and double type parameter values. • Returns parameter conditions as array of the `wv_param_cond` structure. Each element of the array contains one parameter and its value. • Returns the number of parameters. • If `nofilter` is `0`, the `tidx` argument is considered as the trace index of the signal after the param filter if once exists (although it might be different trace than the original index).
`void wv_free_sweep_conditions(wv_param_cond *pcond,INT4 pcnt)`	Frees the memory that is allocated for saving parameter condition information in the `wv_param_cond` structure array.
`void wv_set_sweep_filter(void* file_handle, char* parm_name, char* values)`	Sets the sweep filter by issuing names and values to one parameter. One function call is for one parameter. It can be called multiple times if you want to set a filter for multiple parameters.The values can be a list of parameter values separated by semicolons (`;`), a range of parameter values `[<start>:<stop>]`, or `!value` for exclude of values (for example, `20:120;!60` or `40;80;100`. The range value filter is not supported for string values.For .grp files, `wv_load_signal` only loads filtered traces. `wv_load_signal` loads traces for all others and then filters them.
`int wv_is_analog(void* file_handle, char* signal_name)`	Checks if the signal is an analog signal.
`int wv_is_complex(void* file_handle, char* signal_name)`	Checks if the signal is a complex signal.
`int wv_is_logic(void* file_handle, char* signal_name)`	Checks if the signal is a logic signal.
`void wv_close_all(void)`	Closes all opened waveform files that are not closed by `wv_close_file`.

Feedback

Using the WV_SIG_DATA Structure

The `WV_SIG_DATA` structure saves the signal data read from waveform files by the wv_load_signal function. For example:

```
typedef struct wv_sig_data {
     char *paramcond;
         // param and values for the trace, in format of
 "A=1:B=slow:C=-20"
     int tidx;
         // trace index in full load (without filter)int npts;
     // number of points
     double *xval;
     double *yreal;
     double *yimag;
     char *pVec;
         // digital/bus values, each value is separated by " "
 } WV_SIG_DATA;
```

You need to declare a pointer and then call the wv_load_signal function to load data. For example:

```
% open waveform file
 strcpy(filepath,"test.wdf");
 pfile = wv_open_file(filepath);
% Get signal object in file_handle
 psig = wv_get_signal(wfile,"v(out)",WV_SIGTYPE_ALL,&pidx);
% load signal data
 sigdata = libpointer('wv_sig_data');
 calllib('libmatapi', 'wv_load_signal',pfile,psig,sigdata);
% number of points
 npts = sigdata.value.npts
% get x and y value array
 xval = sigdata.value.xval;
 setdatatype(xval,'doublePtr',npts);
 xdata = xval.value
 yreal = sigdata.value.yreal;
 setdatatype(yreal,'doublePtr',npts);
 ydata = yreal.value
% plot(x,y)
 plot(xval.value,yreal.value);
% clear the pointer if you do not need the data anymore
 clear sigdata;
```

Limitations

These limitations apply to the MATLAB interface.

- **Structure Limitations**

  Nested structures or structures containing a pointer to a structures are not supported. However, MATLAB has access to an array of structures created in an external library.

Feedback

- **Pointer Limitations**

  Support is limited for multi-level pointers and structures that contain pointers. Using inputs and outputs and structure members declared with more then two levels of indirection is also not supported. For example, `double ***outp` translated to `doublePtrPtrPtr` is not supported.

Customizing File Browser Filters

The file filter option menu in the file loading dialog box can be customized to add your own filter patterns.

To customize the menu:

1. Click **Preferences** in the **Configuration** tab.

   The **Preference Settings** window opens.

2. Select the **General** category.

3. Specify a waveform filter from the **Waveform File Filter** preference menu.

   You can choose to filter **All Files** or **Waveform Files**.

4. (Optional) Click **New** to create a new filter pattern, then enter your own filter pattern using the following format:

   ```
   pattern_label:filter_pattern
   ```

   The `pattern_label` is a character string, and `filter_pattern` is the actual string pattern used for name filtering. Both parts are displayed in the filter option menu. The `pattern_label` can contain spaces.

   - Multiple filter patterns are delimited by vertical bar (|):

     ```
     Spice3 output:*.raw|*.out
     ```

   - Multiple filter option menu pattern entries are delimited by a semicolon (;):

     ```
     Spice3 output:*.raw ; Spice listing:*.out
     ```

   - Multiple filter option menu pattern entries are delimited by a semicolon (;):

     ```
     Spice3 output:*.raw ; Spice listing:*.out
     ```

   - Globbing expressions are supported through wildcards:

     ```
     Hspice:*.tr[0-9]|*.ac[0-9|*.dc[0-9]
     ```

Feedback

If the `pattern_label` field is omitted in an entry, the `filter_pattern` is used as the display string in the filter option menu.

5. Click **OK** or **Apply-save** to save your changes.

To delete a custom filter, select the filter from the Waveform File Filter menu and click **Delete**.

Configuring "Send To" in the Windows Environment

In the Windows OS environment, the WaveView tool can be added to the Send To application list to support direct import of waveform files or the WaveView session files using the **Send To** context menu.

To add the WaveView tool to the Send To application list:

► Choose from the main desktop screen and enter `sendtoStart > Run` to open the SendTo folder.

To support reading waveform files using the **Send To** menu:

► Add a shortcut in the `SendTo` folder to the WaveView executable. The executable can be found at `C:/Program Files/Sandwork Design/WaveView version/wv.exe`.

To support reading session files using the **Send To** menu:

1. Add a short cut in the `SendTo` folder to the WaveView executable.

   Make sure that the `-x` option is appended to the end in the Target field in the shortcut property.

2. Rename the shortcut to reflect the purpose of the Send To shortcut ("WV Session", for example).

Configuring the WaveView Tool Manually

You can configure the WaveView tool by modifying the .spxrc configuration file for Linux platforms or modifying the Windows Registry for MS-Windows platforms.

On the Linux platform, the configuration settings are used to initialize the WaveView tool.

► The configuration file must be named `.spxrc`. The WaveView tool reads the configuration file from four different locations in the following order:

   a. `-spxrc switch on command line`

   b. `current_working_directory/.spxrc`

Feedback

c. `$(HOME)/.spxrc (if HOME is defined)`

d. `$(SX_HOME)/.spxrc (if SX_HOME is defined)`

`SX_HOME` is a user-defined environment variable; `HOME` is a shell variable which defaults to a user's home directory.

Configuration settings from `current_working_directory/.spxrc` have the highest priority. Settings read first are overridden by those read later in the sequence.

► On a Windows platforms, preference settings are automatically updated into Windows Registry when you click **Save** in the **Preference Settings** dialog box (See Setting Preferences).

If manual editing is necessary, run `regedit` from a MS-DOS prompt and modify settings in the following location:

`HKEY_CURRENT_USER\Software\Sandwork Design\WaveView`

Basic WaveView Usage

This section contains information on the following topics:

- Supported Command-Line Options
- Loading and Saving Sessions
- Viewing Recent Files and File Paths
- Log Files and Command History

Supported Command-Line Options

This section contains information on the following topics:

- WaveView Command-Line Options
- WaveView ADV Command-Line Options

WaveView Command-Line Options

The following WaveView command-line options are available.

Option	Action
`-ace_gui`	Starts an ACE Tcl script in GUI mode.
`-ace_no_gui`	Starts an ACE Tcl script in batch mode.

Feedback

Option	Action
`-ace_perl_gui`	Starts ACE Perl in GUI mode.
`-ace_perl_no_gui`	Starts ACE Perl in batch mode.
`-ace_cmd_arg arg`	Pass a command line argument to a Tcl ACE script.
`-adcdump [file_name]`	Dumps the contents of the ADC Toolbox display to a PNG file with the specified file name.
`-c`	Converts to WDF in batch mode (see `-r`, `-ri`, `-rv`, `-w`).
`-compare rule_file [out_file] [-x sx_file]`	Compares waveforms in batch mode.
`-console`	Show the console window.
`-convert2fsdb input_file <output_file> [-pskip_hier <hierarchy_string>]`	Converts the `input_file` waveform file to a `output_file` FSDB file. If the `output_file` is not set, an `input_file.fsdb` file is automatically created. Skips the hierarchy specified by `-pskip_hier`.
`-convert2out input_file {[-s signal_filename] [-o output_filename] [-compress z|gz|0] [-minstep min_time_step]} [-pskip_hier <hierarchy_string>]`	Converts the `input_file` waveform file to a `output_filename` .OUT file. The following options are available: • `-s filename`: The name of the input waveform file. • `-o output_filename`: The resulting waveform file. • `-compress z|gz|0`: Compresses the .OUT file to a zip, gzip, or no compression (0). • `-minstep min_time_step`: The minimum time step of the output OUT waveform file • `-pskip_hier`: Skips the hierarchy specified.
`-display host:screen`	Starts 'wv' by displaying the window to host.
`-geometry wxh+x+y`	Specifies the application window starting location and size.

Feedback

Option	Action
-group	Groups the opened waveform files passed at the command line. For example: wv *.tr0 -group
-h(elp)	Displays the command-line help message.
-hide_gui	Hides the display and automatically exits when finishing Tcl.
-k	Starts the WaveView tool more quickly without the greeting window.
-load	Preloads waveform data to memory. FSDB and NPX-SDIF files cannot be preloaded. Just the sweep results from tr0 and NW files can be preloaded.
-log file_name	Specifies a log file name.
-manual	Open the online Help.
-r mode	Performs a reduction for WDF conversion. 0:default, loss-less, 1:medium reduction, 2:high reduction (see -c).
-replay_delay time	Sets the delay time in milliseconds when replaying a log file.
-ri itol	Reduction current tolerance (see -c, -r).
-rv vtol	Reduction voltage tolerance (see -c, -r).
-spxrc pref_file	Starts the WaveView tool using the preference settings specified in the pref_file.
-text	Import a text table data file.
-text_nox	Text import option: no x column.
-text_noh	Text import option: no name header.
-v(ersion)	Reports software revision.
-x session_file	Loads a session file.
-y session_file	Applies a session file to existing waveform files.

Feedback

Option	Action
`file1, file2, …`	Loads waveform files.

WaveView ADV Command-Line Options

The following WaveView ADV command-line options are available.

Option	Action
`-b`	Enables SPICELint batch mode. This option is backward compatible with the SPICELint batch mode and is equivalent to `-nogui -spicelint`.
`-fast_parsing`	Reduces design import time and memory consumption by skipping parsing device terminals and parameters. If you later need to trace connectivity in the DesignView, the related devices are parsed dynamically and the connection is created.
`-init file_name`	Specifies an initialization file, which takes precedence over the .cxinit file in your working directory or home directory.
`-nogui`	Starts the WaveView tool in non-GUI mode.
`-sim_options "option strings"`	Specifies simulator command-line options.
`-sim_run_file file_name`	Specifies a simulation run script.
`-simulator=simulator_name`	Specifies the simulator from which you want to import a design. For example: `wv -simulator=nanosim -spice t.sp`
`-spice | spectre | eldo | lvs2`	Specifies the netlist format.

Feedback

Option	Action
`-spicelint [-syntax | -connectivity | -source | -erc]`	Enables SPICELint. • -syntax: Enables syntax rules check. • -connectivity: Enables connectivity rules check. • -source: Checks check "floated source/loop of source/loose global node" • -erc: Enables ERC rule check. For example, `-spicelint -syntax` checks the syntax rules only.
`-suppress_ew`	Suppresses error and warning messages.
`-tcl file_name`	Opens the specified Tcl command file.
`-tcl_arguments "argument strings"`	Specifies Tcl file input arguments.
`-verilog`	Specifies the file import type as Verilog.
`-wf file_names`	Loads the specified waveform file(s).
`-zip_output`	Generates .gzip files for all output files.
`-inst_device_mapping_file <map file path>`	Specifies the instance name pattern and device type mapping file.

Loading and Saving Sessions

The WaveView tool provides save and restore functions for users to save or retrieve a job session to or from a session file. Session files have the `.sx` default file extension.

- Save Session
- Load Session

Save Session

Use this procedure to save your current session.

1. Before saving your session, you might consider changing the save directory from the default.
2. Click **Save Session** on the **File** tab.

   The **Save Session** dialog box opens.

Feedback

3. Select or enter the file in which to save your current job session.
4. (Optional) Click the **Use Relative Path in Session File** check box to use a relative path for your session file.
5. (Optional) Click the **Save Current Waveview Only** check box to save only the current waveview tab in your saved session file.
6. (Optional) Click the **Save a Copy of Loaded Signals** check box to save loaded signals to a new file named `<session_file_name>.wdf`.

   For example, if a session is saved to a file called `foo.sx`, the data file name is `foo.sx.wdf`. When loading the session file, only `foo.sx.wdf` is loaded.
7. Click **OK** to save your job session.

Load Session

Use this procedure to load from a session file.

1. Click **Load Session** on the **File** tab.

   The **Load Session** dialog box appears.
2. Browse to and select the session file you want to open.

   The browser opens to the last location from which a session file was loaded.
3. (Optional) To apply the saved signal selection/layout to other loaded waveform data files in the application, ensure the **Apply to existing waveform files** option is enabled (the default). Select the preferred waveform data file for each restore target using the Waveform Source dialog box.

   If the **Apply to existing waveform files** option is not enabled, the session loader restores signal selection/layout using the waveform data files specified in the session file.
4. (Optional) Click the **Suppress popup warning messages** option check box to disable session loading warning messages (missing signals, for example).
5. Click **OK** to load the selected session file.

   The session file is loaded into the WaveView tool.

   Note:

   When loading a session file and applying it to multiple target files, an index number is appended to waveview tab names to avoid duplicate names. For example, for name `waveview1` in session file, the displayed tab names become `waveview 1_1`, `waveview1_2`, and so on. The index starts from 1.

Feedback

You can also load a session file from the command line with the `-x` option. For example:

```
wv -x setup_file
```

Add the `-y` option to apply the session setup to different waveform data files. For example:

```
wv -y setup_file file1 file2 … filen
```

Viewing Recent Files and File Paths

Use these procedures to view recent files and file paths.

► Click **Recent Files** in the **File** tab to display a list of recently opened files from the WaveView window.

► Click **Recent Paths** in the **File** tab to display a list of recently used file paths.

Log Files and Command History

The Console window displays the log files and command history.

► To toggle the display of the **Console** window, click **Console** on the **Configuration** tab.

This section contains information on the following topics:

- Viewing and Saving Log Files
- Changing the Default Log File Directory
- Replaying Log Files
- Viewing Command History

Viewing and Saving Log Files

Log files are saved for every session.

► Click the **Log** tab at the bottom of the **Console** window to view the contents of the log file for your current session.

You can also view log files from previous sessions, which are saved in your working directory as sequentially numbered files (for example, `cx.log.1`, `cx.log.2`, `cx.log.3`).

Changing the Default Log File Directory

By default, the WaveView tool outputs log files in the working directory. However, there is an option to change the default log file directory.

Feedback

► Use the `SW_SX_LOG_DIR` environment variable to save the log file to a different directory.

Replaying Log Files

All actions saved in log files can be replayed in the tool.

► To replay a log file, click **Run ACE** from the **Script** section of the **File** tab, and open a log file.

You can also replay a log file at the command line. To replay a saved log file, enter the following line at the command line:

```
cx -tcl <log_file>
```

As the log file is read, all actions saved in the log file are performed in real time in the GUI.

Viewing Command History

After you exit the tool, the underlying Tcl commands for all actions you take are saved to a log file.

Click the **History** tab at the bottom of the Console window to view a log of the underlying Tcl commands that are called for all actions performed in the tool, .

See Viewing and Saving Log Files for more information.

Basic GUI Usage

The following topics describe basic GUI usage:

- Supported GUI Values
- Viewing the GUI
- Viewing the Waveform Browser

Supported GUI Values

All GUI text boxes accept values in the scientific number format (1.00E+02) or numbers with scale unit such as nano (n) and micro (u).

The following scale units are supported:

- T - 1E12
- G - 1E9
- M (uppercase) - 1E6

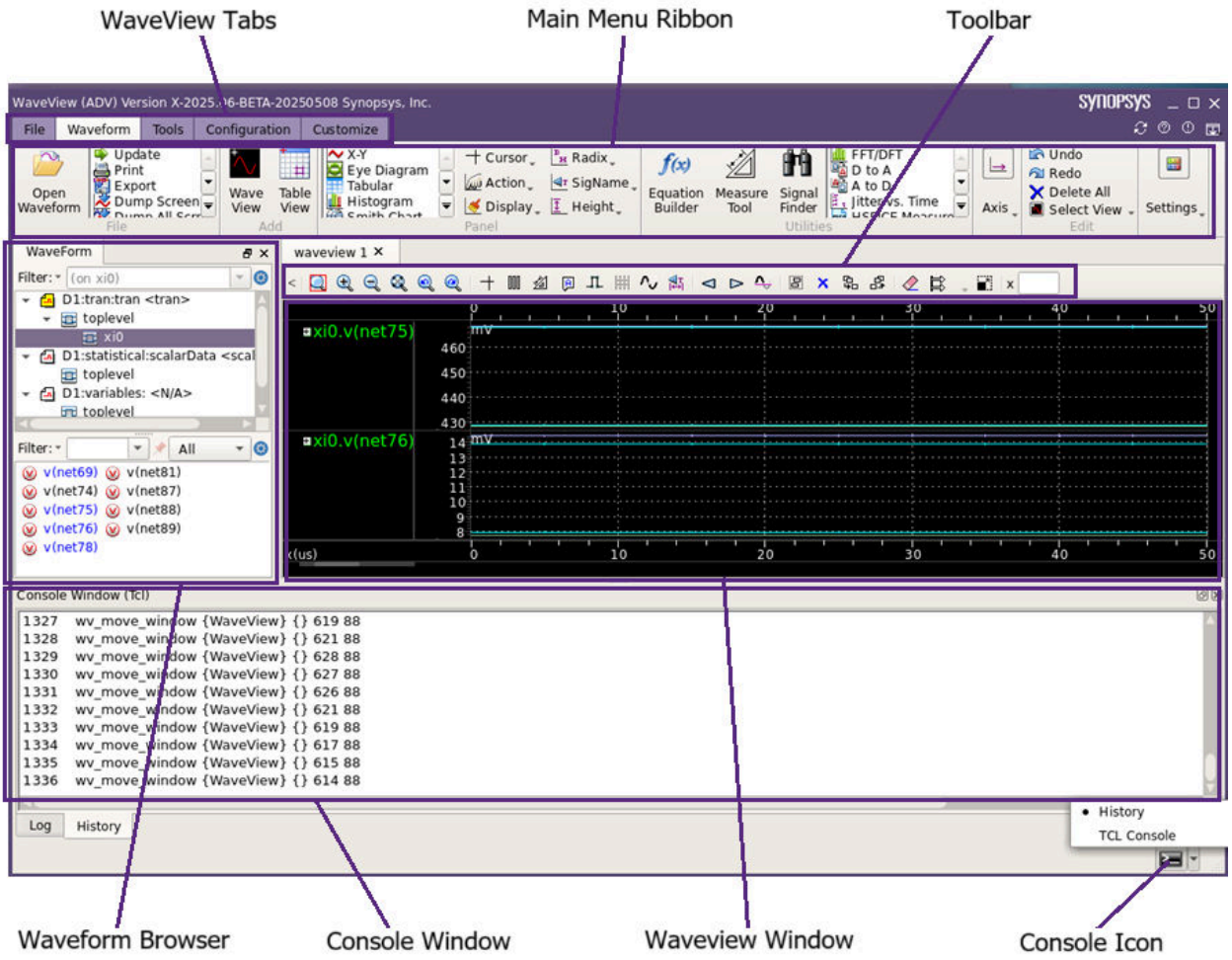
- MEG - 1E6
- x - 1E6
- k - 1E3
- m (lowercase) - 1E-3
- u - 1E-6
- n - 1E-9
- p - 1E-12
- f - 1E-15

Viewing the GUI

After opening a waveform file in WaveView tool, you can view the GUI.

Feedback

The following figure shows the WaveView GUI:



The main window is comprised of the following parts and associated functions:

- WaveView tabs — switches between the different tabs.
- Main menu ribbon — the top application menu bar.
- Toolbar — shortcuts for the WaveView window.
- Waveform Browser — contains the Waveform Browser including the top hierarchy browser and the bottom signal list window.
- Waveview window — the waveform viewing area. Multiple waveviews can be opened inside the waveview window.

  The width of the browser area and the waveview window can be adjusted using the vertical pane bar in between the Browser and the waveview area.

Feedback

- Console window — the console window.

  The width of the console window can be adjusted with the browser area and the waveview window using the horizontal pane bar in between the console window and waveview area.

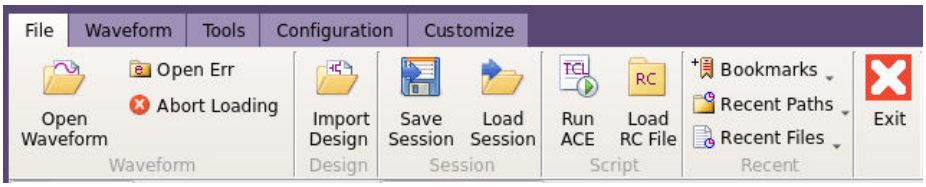
- Console icon — the icon to display the console window.

The following sections describe the WaveView GUI in detail:

- Viewing the File Tab
- Viewing the Waveform Tab
- Viewing the Tools Tab
- Viewing the Configuration Tab

Viewing the File Tab

The File tab appears as follows:



The following actions are available:

- **Open Waveform** — imports files into the tool. See Opening a Waveform File.
- **Open Err** — open an .err file. See Opening Err Files.
- **Abort Loading** — abort the loading of a file.
- **Import Design** — imports the design.
- **Save Session** — saves the current session. See Save Session.
- **Load Session** — loads a saved session. See Load Session.
- **Run ACE** — runs an ACE script. See Running ACE Scripts.
- **Load RC File** — loads an RC file. See Opening RC Files.
- **Bookmarks** — manage waveform bookmarks. Bookmarking Waveform Files
- **Recent Paths** — view recently used file paths.
- **Recent Files** — view recently used files.
- **Exit** — exit the WaveView tool.

Feedback

Viewing the Waveform Tab

The Waveform tab appears as follows:



The following actions are available:

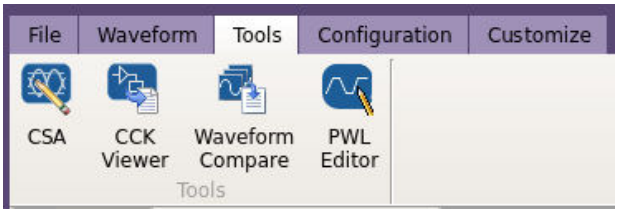
- **Open Waveform** — imports files into the tool. See Opening a Waveform File.
- File options:
  - **Update** — updates the waveform file. See Updating Waveform Files.
  - **Print** — prints the waveform file. See Printing Waveform Files.
  - **Export** — exports waveform data. See Exporting Waveform Data.
  - **Dump Screen** — dumps the current screen. See Dumping the Screen.
  - **Dump All Screens** — dumps all screens. See Dump All Screens.
  - **Marching** — automatically updates waveforms from a running simulation job. See Marching Waveforms.
  - **Save SigList** — saves the signal list. See Saving and Loading SigLists.
  - **Load SigList** — see above.
- **Wave View** — adds a new waveview. See Working With Waveviews.
- **Table View** — adds a new tableview. See Working With Tableviews.
- Adding panels: adds a panel type. See Using Panels.
- **Cursor** options: adds or configures cursors. See Using Cursors.
- **Display** options: sets panel display options. See Setting Panel Display Options.
- **Radix** options: sets radix options. See Setting Radix Options.
- **SigName** options: sets signal name options. See Setting Signal Name Options.
- **Height** options: adjusts panel height. See Adjusting Panel Height.
- Utilities:
  - **Equation Builder** — opens the Equation Builder. See Using the Equation Builder.
  - **Measure Tool** — opens the Measurement Tool dialog. See Using the Measurement Tool.

Feedback

    - **Signal Finder** — opens the Signal Finder. See Searching Signals.
    - Other utilities — see Using WaveView Tools.
- **Axis** options: sets axis attributes. See Axis Attributes.
- **Undo** — undoes an action.
- **Delete All** — deletes all waveview panels.
- **Select View** — selects a waveview panel.
- Settings:
    - **Waveform Attributes** — set waveform attributes. See Setting Waveform Attributes.
    - **Scan Settings** — set scan settings. See Scanning Signals.
    - **Key Bindings** — configure key bindings. See Setting Key Bindings (Shortcut Keys).
    - **Bus Mnemonic Map** — map bus values. See Mapping Bus Values.

Viewing the Tools Tab

The Tools tab appears as follows:



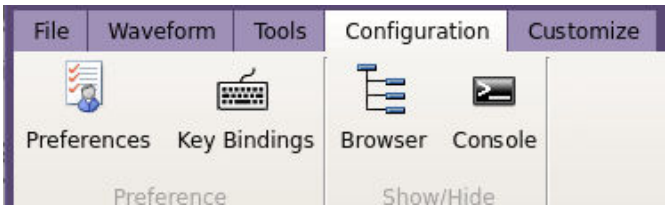
The following options are available:

- **CSA** — opens the Channel Simulation Analysis (CSA) environment. See Using CSA (Channel Simulation Analysis).
- **CCK Viewer** — opens the CircuitCheck (CCK) viewer. See Using the CCK Viewer.
- **Waveform Compare** — opens the Waveform Compare tool. See Using the Waveform Compare Tool.
- **PWL Editor** — opens the PWL Editor. See Using the PWL Editor.

Feedback

Viewing the Configuration Tab

The Configuration tab appears as follows:



The following options are available:

- **Preferences** — open Preferences dialog. See Setting Preferences.
- **Key Bindings** — configure key bindings. See Setting Key Bindings (Shortcut Keys).
- **Browser** — show/hide Waveform Browser.
- **Console** — show/hide console.

Viewing the Waveform Browser

See Using the Waveform Browser for information on viewing and using the Waveform Browser.

Running ACE Scripts

The Analysis Command Environment (ACE) is a Tcl-based programming environment comprised of a set of high-level waveform data operating commands evolved from the old Synopsys APX (Application Programming Extension) API functions. While the old APX API functions are still supported in ACE, the new high-level commands in ACE significantly increase the ease of use to construct scripts for complex analysis needs.

To run an ACE script, click **Run ACE** on the **File** tab.

The ACE commands support:

- Tcl list and Perl array variables for handling a collection of objects.
- Wild card name pattern for referencing multiple objects.
- Direct use of Tcl/Perl variables in waveform equations. All calculator functions are available for the `sx_equation` command.
- Execution in both GUI mode and non-GUI batch mode.

Feedback

See Also

- WaveView™ ACE Command Reference

2

Importing Files

This chapter details how to import files into the WaveView tool.

This chapter contains information on the following topics:

- Supported File Formats
- Opening a Waveform File
- Opening Multiple Files
- Opening PWL Files
- Opening RC Files
- Reading Compressed Waveform Files (Linux Only)
- Working With WDF Files
- Importing Multiple Files in Virtuoso

Supported File Formats

The following waveform formats are supported:

- Synopsys FSDB files (binary, v6.5 Linux platforms; v6.5 Windows platforms)
- Synopsys HSPICE® tr0, sw0, ac0, mt0, hb0, nw0, mc0, mct0, rlgc, wzo, annotate, listing file (Binary/ASCII)
- Synopsys StarSim™ wdb, wv, xp (binary)
- Synopsys HSIM®/NanoSim®/CustomSim™ wdb (binary)
- Synopsys HSIM/NanoSim out ,vector, .mt file (ASCII)
- Synopsys Saber AI/PL (binary/ASCII)
- Synopsys VCS®/Magellan™/DVE VPD (binary)
- Synopsys CustomSim .err, .errz, and .errt files

Feedback

- Synopsys Galaxy Custom Compiler® Simulation and Analysis Environment (SAE) `resultsMap.xml` and `results.xml` files
- Cadence® Spectre® PSF, PSFXL (support only in Tran section for corners, MC, and sweeps), WSF (binary/ASCII), runObjFile
- Cadence PSPICE® DAT (binary), Allegro® SIM (ASCII)
- Verilog VCD (ASCII), ModelSim® WLF (binary)
- Mentor Graphics® ELDO®/ADMS™ COU, WDB (binary), AdiT TR0/TB0.
- SPICE3/SmartSpice raw format (binary/ASCII)
- Agilent® ADS .ds format (binary), TouchStone® S-parameters (ASCII)
- CSDF (Common Simulation Data Format) (ASCII)
- Text file, CSV (comma separated values) format
- Other in-house simulator formats (SDIF, MICS, TI-punch)
- Synopsys WDF format (binary)
- Binary scope data (Tektronix®, Agilent, Keysight, Teledyne Lecroy®)
- IBIS models, Synopsys Liberty® files

Additional File Format Information

Additional information is available for the following file formats:

- Special Note for the ELDO COU Format
- Special Note for the ADMS/ELDO WDB Format
- Special Note for the PSF Format
- Special Note for the WDF, FSDB, Binary HSPICE tr0, and PSF Formats

Special Note for the ELDO COU Format

The COU format from ELDO simulator can contain multiple simulation runs. Before loading a COU file, COU reader in the WaveView tool needs to scan the entire file to determine how many runs the COU file contains.

For very large COU files, the detection process might take a very long time because the reader has to sequentially examine all data in the COU files.

To speed up this COU file loading process, the WaveView tool has an alternative built-in search algorithm that can cut the file loading time significantly. To invoke the fast read

Feedback

option, set the SW_SX_FAST_COU environment variable to any value in your shell (for example, `setenv SW_SX_FAST_COU`). Due to the nature of the COU format, the fast search algorithm has a trade-off that might potentially determine the number of runs of a COU file incorrectly. Although the possibility is very low, you can always disable the fast search algorithm by unsetting the environment variable (for example, `unsetenv SW_SX_FAST_COU`).

Special Note for the ADMS/ELDO WDB Format

By default, a built-in WDB file reader is used to load data from WDB files generated by ADMS and ELDO. However, for small amount of simulation waveform data, the WDB writer in these simulators might buffer waveform data in the WDB Java server without writing them to a disk file. To work around this problem, you can take one of the following actions:

1. For ELDO, set the `-isaving` option to a smaller value to force more frequent incremental saving to disk files. The file size specified in the -isaving option is in MB. For ADMS, set the JwdbSpillThreshold option in `$HOME/.vams_setup` to a smaller value to lower the buffer size.
2. Enter `setenv SW_SX_JWDB_API 1` at the command line to use the vendor-provided WDB API to load data directly from the Java WDB server. The API + Java WDB server operation, however, can be very slow for large WDB files.

Special Note for the PSF Format

PSF format assigns a unit for each signal in a simulation output file. For noise analysis, to display different waveforms using proper scales, signals with V^2/Hz unit are automatically converted to V/sqrt(Hz) by taking the square root of the loaded value during the waveform loading process.

PrimeSim HSPICE splits large output PSF format data (> 2GB) into multiple 2GB PSF files on 32-bit platform. Since 32-bit platform is obsolete and the simulator binary is always 64-bit, there is no 2GB file size limitation for PSF file.

Special Note for the WDF, FSDB, Binary HSPICE tr0, and PSF Formats

Split files are supported for the WDF, FSDB, binary HSPICE tr0, and PSF files. When these files are opened in the WaveView tool, a search is performed for the subsequent files in the same directory. For example, if the loaded file is `output.wdf`, the WaveView tool automatically looks for `output.wdf.1`. If the output file is `output.fsdb.1`, the WaveView tool looks for `output.fsdb.2`. If you do not want to load the subsequent split files, disable the **Automatically connect to subsequent split files** option in the **Loading Waveform Files** dialog box.

Feedback

Opening a Waveform File

To open a waveform, click **Open Waveform** in the Waveform tab.

To abort loading a waveform file that is taking too long to load, click **Abort Loading** on the **File** tab.

Opening Waveform Files in Case-Insensitive Mode

You can open waveform files in case-insensitive mode as follows:

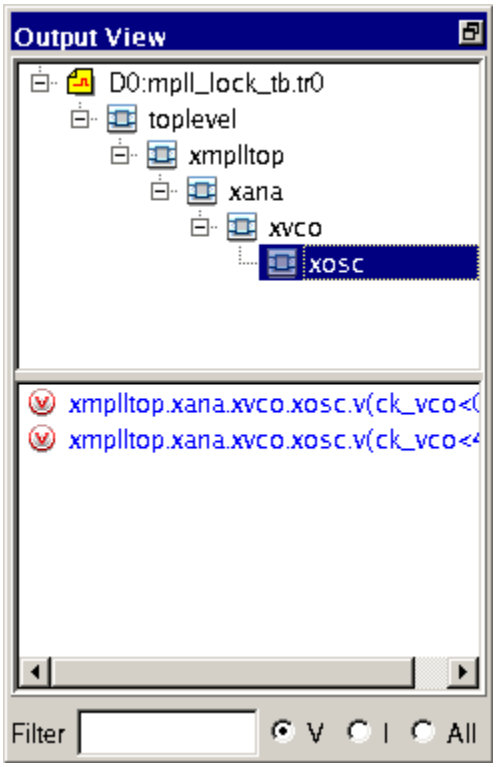
- Using the command-line option `-case_insensitive_mode`
- Using the option `-case_insensitive` with the ACE command `sx_open_sim_file_read`

For FSDB files, the tool does not perform a linear search, which provides significant performance improvement with the ACE command `sx_signal`.

Opening Multiple Files

If multiple files are loaded, to allow simultaneous signal browsing on these multiple target files, the **Open New Browser** context menu function opens multiple signal browsers for each waveform file respectively.

Feedback



Opening PWL Files

To open textual or PWL data, click **Open Waveform** on the **File** tab and expand the **Additional PWL/Table Format Options** section in the opened dialog box.

The WaveView tool supports data input in regular text format. Scaling factors (for example, 1ns) and scientific formats (for example, 1.0e-9) are both supported in the textual data files.

To distinguish WaveView text data files from regular text files, the first line of a text data file is reserved for the format specification and must appear in the following form, where format-type is the format type keyword:

```
#format format-type
```

Lines beginning with a number sign (#) are treated as comments.

Note:

If the file cannot be recognized, use the option **Force Open Waveform as** to open the file as a SPICE PWL or text file.

Feedback

The following sections describe how to prepare textual input for:

- Tabulated Data in Real Numbers
- Tabulated Data in Complex Numbers
- 2-Dimension Sweep Data in Real Numbers
- 2-Dimension Sweep Data in Complex Numbers
- Measured Data Without Name Header
- SPICE PWL Sources
- Opening Err Files

Tabulated Data in Real Numbers

Syntax:

```
#format table
xvar signal1 signal2 ....
valuex value1 value2 ...
...
```

xvar and signaln are variable names and valuen are numeric values. As a special-case exception, the first line can be omitted if the name of `xvar` is `TIME`.

Example 1:

```
#format table
VGATE Ichannel
0.0 0.0
1.0 1u
2.0 5u
3.0 5.2u
...
```

Example 2:

The following data file can be recognized without the #format line:

```
TIME V(1) V(2)
0.0 5.0 2.5
1n 4.8 2.6
...
```

If the data file does not contain the x-value column, (commonly seen in measurement data), add keyword #dataonly to the #format line. The reader automatically assigns a 0-based row index as the x-axis value.

Feedback

If the signal names are embedded in the last '#' comment line before the actual waveform data values, add keyword #cmt_name to the format line.

Tabulated Data in Complex Numbers

Format for complex numbers in (real, imaginary) form.

Syntax:

```
#format complex-ri
xvar signal1 signal2 ....
valuex value1-real value1-imag value2-real value2-imag ...
...
```

Format for complex numbers in (magnitude, angle [0-360]) form.

Syntax:

```
#format complex-ma
xvar signal1 signal2 ....
valuex value1-mag value1-angle value2-mag value2-angle ...
...
```

Format for complex numbers in (magnitude, phase [0-2]) form:

Syntax:

```
#format complex-mp
xvar signal1 signal2 ....
valuex value1-mag value1-phase value2-mag value2-phase ...
...
```

Example:

```
#format complex-ri
Freq vout
100meg 0.0 0.0
200meg 1.0 0.1
300meg 1.1 0.2
400meg 1.4 0.4
...
```

2-Dimension Sweep Data in Real Numbers

Syntax:

```
#format 2dsweep xvar2
xvar signal1 signal2 ....
#sweep value1-x2
valuex value1 value2 ...
```

Feedback

```
...
#sweep value2-x2
valuex value1 value2 ...
...
```

where xvar2 is the name of the second sweep variable.

Example:

```
#format 2dsweep vgs
VDS Ichannel
#sweep 0.0
0.0 0.0
1.0 1u
2.0 5u
3.0 5.2u
#sweep 0.1
0.0 0.0
1.0 2u
2.0 6u
3.0 6.2u
...
```

2-Dimension Sweep Data in Complex Numbers

Format for complex numbers in (real, imaginary) form

Syntax:

```
#format 2dsweep-ri xvar2
#sweep value1-x2
xvar signal1 signal2 ....
valuex
value1-real value1-imag value2-real value2-imag ...
...
#sweep value2-x2
xvar signal1 signal2 ....
valuex value1-real value1-imag value2-real value2-imag ...
...
```

Format for complex numbers in (magnitude, angle [0-360]) form.

Syntax:

```
#format 2dsweep-ma xvar2
#sweep value1-x2
xvar signal1 signal2 ....
valuex value1-real value1-imag value2-real value2-imag ...
...
#sweep value2-x2
xvar signal1 signal2 ....
```

Feedback

```
valuex value1-real value1-imag value2-real value2-imag ...
...
```

Format for complex numbers in (magnitude, phase [0-2]) form.

Syntax:

```
#format 2dsweep-mp xvar2
#sweep value1-x2
xvar signal1 signal2 ....
valuex value1-real value1-imag value2-real value2-imag ...
...
#sweep value2-x2
xvar signal1 signal2 ....
valuex value1-real value1-imag value2-real value2-imag ...
...
```

Example:

```
#format 2dsweep-ma temp
freq gain
#sweep 50
100meg 1.0 0
200meg 1.0 90
300meg 1.1 120
400meg 1.4 150
#sweep 75
100meg 1.0 0
200meg 1.0 80
300meg 1.2 110
400meg 1.3 140
...
```

Measured Data Without Name Header

To import text measure data that does not have name header line or the x-axis value column, click **Open Waveform** and expand the **Additional PWL/Table Format Options** section.

Select the required options and click **Ok** to load the data file.

SPICE PWL Sources

To import SPICE PWL sources into the WaveView tool, click **Open Waveform** and expand the **Additional PWL/Table Format Options** section.

Choose the **SPICE PWL Source Data** option and click **Ok** to load the data file. The SPICE PWL sources must not use any variable parameters.

Feedback

Opening Err Files

To open an .err file, click **Open Err** on the **File** tab. The errors are displayed in the .err File viewer dialog box.

Opening RC Files

Setup files can be loaded from nWave (rc files) and Xelga (swd files). To load a setup file, click **Load RC File** in the **File** tab.

Reading Compressed Waveform Files (Linux Only)

On Linux platforms only, compressed (.Z) or gzipped (.gz) waveform files can be directly opened in the WaveView tool. The tar.gz waveform files are not readable directly since a tar file can potentially contain multiple files.

By default the current working directory is used as the temporary directory to store the decompressed files. Decompressed files are automatically cleaned up when a the compressed file is closed in the WaveView tool. Set the `SW_SX_TMP_DIR` environment variable to redirect the temporary directory to a different location.

Working With WDF Files

The WDF (Waveform Data File) format is a Synopsys proprietary waveform storage format. The WDF format not only compresses analog and logic waveform data, it also facilitates fast waveform access for large data files. The compression scheme can be lossy or loss-less (default). Waveform data files in other formats can be converted into WDF files on demand.

The WDF format is directly supported by a number of simulators including HSIM, NanoSim, UltraSim, Spectre, and AMS-Designer.

You can extract signal files as a group. For example, you can create the following Tcl script file to extract signal files:

```
set f1 [sx_open_sim_file_read ./group.grp]
sx_current_sim_file $f1
set sig [sx_signal x1]
set sig2 [sx_signal x2]
set f2 [sx_open_sim_file_write ./save2d.wdf]
sx_save_signal $f2 "$sig $sig2"
sx_close_sim_file $f2
sx_close_sim_file $f1
```

Converting Waveform Files to WDF

To convert an open waveform file to WDF format, right-click the name of a waveform file name, and choose **Convert to WDF** from the menu that opens. The whole waveform file is saved into a new WDF file with X range you specify.

Converting Existing Output Files to WDF

One or more output files can be converted (in batch mode) to Synopsys WDF files using the `-c` command-line option.

Lossless Compression

With lossless compression, all original simulation output data points are preserved.

```
wv -c file1 file2 … filen
```

Lossy Compression

You can use the `-r` command-line option to turn on data reduction for the compression process. Use `-r 1` for moderate reduction or `-r 2` for a more aggressive reduction.

```
wv -c -r (1 or 2) <-rv 0.001> <-ri 10n>
```

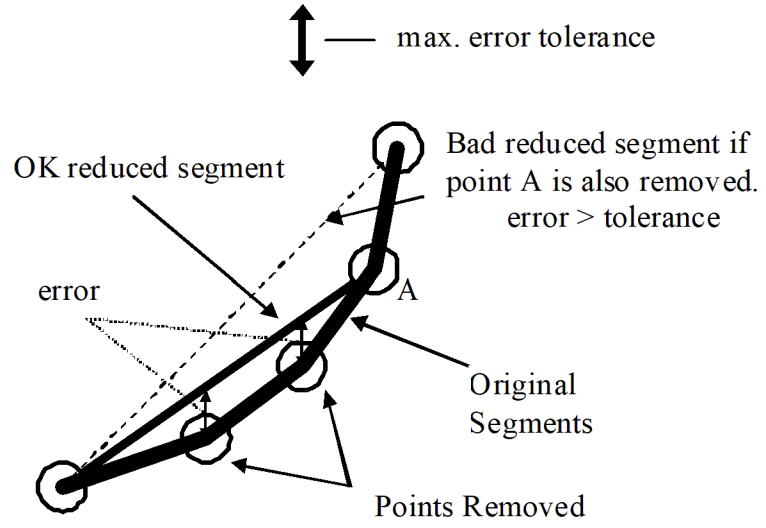
Reducing WDF Data

The data reduction option removes redundant data points from straight waveform segments based on user-specified error tolerances. Data points are removed if the error between the original data points and the reduced PWL segments are smaller than the user-specified tolerance.

Feedback

Figure 1 depicts the scheme used in WDF data reduction.

Figure 1 WDF data reduction scheme



The moderate reduction setting keeps at least 1 point out of four (4) original data points, while the more aggressive reduction mode keeps 1 out of ten (10) points. The `-rv` and `-ri` options are used to specify the reduction tolerance for voltage (or generic) waveforms and current waveforms respectively. The default setting is 5 mV for voltage waveform and 1 nA for current waveforms.

Importing Multiple Files in Virtuoso

Parametric simulations in the Virtuoso environment can generate multiple output files in multiple directories. The locations and associated sweeping variables of these files can be found in the runObjFile log file.

The runObjFile can be opened directly in the WaveView tool to access these waveform files.

Virtuoso dcOp Parametric Analysis

A dcOp analysis from a Virtuoso parametric run can also generate multiple result files. Each of these corner files contains a single-point operating point result for each device parameter. A special function is added in the Output View signal browser context menu to display or export these single-point values together with their sweeping variable

conditions. Right-click a signal in the signal browser, and choose **Signal 'NAME' > Display Values** or **Signal 'NAME' > Export Values**.

3

Working With Waveforms

This chapter details working with waveforms in the WaveView tool.

Note:

A WaveView ADV license is required for some of these features.

The section contains information on the following topics:

- Using the Waveform Browser
- Waveform Files and Data
- Setting Waveform Attributes
- Working With Waveviews
- Working With Tableviews
- Comparing and Checking Waveforms
- Comparing Logic Signals

Using the Waveform Browser

The following topics are detailed in this section:

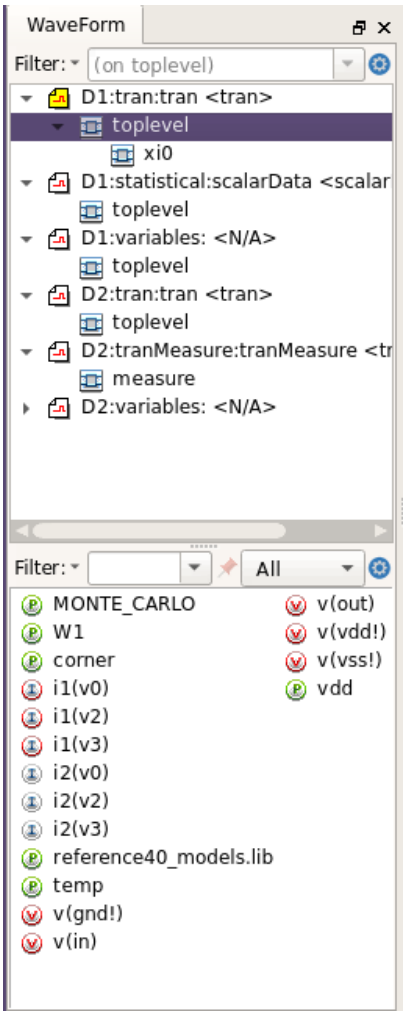
- Viewing the Waveform Hierarchy Browser
- Displaying Signals
- Highlighting Waveforms

Viewing the Waveform Hierarchy Browser

The Waveform Browser consists of an upper hierarchy browser and a lower signal list window. The lower list window displays signals under the selected hierarchy in the upper browser. Only one item can be selected in the upper hierarchy browser, while multiple signals can be selected in the signal windows for drag-and-drop operations.

Feedback

To distinguish waveform data with the same file names from different directories, in the root entries of the upper hierarchy browser, directory paths of loaded waveform files are displayed using directory prefix `DX` where X is the sequential index of different directories. To find out the original full paths of directory entries, select from the main menu **File > Show Directory Table** to display the table that maps directory identifiers to full file paths.



Analog Waveform Icon Reference

The following table describes what each analog waveform type icon signifies in the Waveform Browser:

Icon	Analog Waveform Type	Icon	Analog Waveform Type
	Real Voltage		(Real,Imaginary) Complex Generic

Feedback

Icon	Analog Waveform Type	Icon	Analog Waveform Type
	Real Current		(Magnitude,Degree) Complex Generic
	Real Generic		(Magnitude,Phase) Complex Generic
	(Real,Imaginary) Complex Voltage		Real Voltage Alias
	(Magnitude,Degree) Complex Voltage		Real Current Alias
	(Magnitude,Phase) Complex Voltage		Real Analog Alias
	(Real,Imaginary) Complex Current		Sweeping Parameter
	(Magnitude,Degree) Complex Current		Derived Data Top Level
	(Magnitude,Phase) Complex Current		Wire Type

Digital Waveform Icon Reference

The following table describes what each digital waveform type icon signifies in the Waveform Browser:

Icon	Digital Waveform Type	Icon	Digital Waveform Type
	Logic Integer		Logic Register
	Logic Supply		Logic Alias
	Logic Parameter		Logic Variable
	Logic Wire		(Magnitude,Phase) Complex Generic

Feedback

Non-Digital/Analog Waveform Icon Reference

The following table describes what each icon signifies besides those used for analog and digital waveform types in the Waveform Browser:

Icon	Icon Type	Icon	Icon Type
	Waveform Data File (no signal displayed, not linked)		Generic signal alias name
	Linked Waveform Data File (linked to netlist)		Voltage signal alias name
	Data file with some of its signals displayed		Current signal alias name
	An old data file that has been overridden by new result		Logic signal alias name
	One or more equations.		Waveform comparison results.

Displaying Signals

Signals from the Waveform Browser can be loaded into a waveview with the drag-and-drop operation. If a hierarchy (or file) entry is dragged from the upper hierarchy browser, all signals in the selected hierarchy level (or a file) are loaded. If a signal (or multiple signals) is dragged from the signal list window, only the selected signals are loaded. You can also double-click in the lower signal window to add an individual signal.

Depending on the signals being dragged, drop hints appear when signals are dragged over a waveview area. If the waveview is empty, hint boxes indicate available initial waveview modes. If the waveview contains existing panels, an insertion hint or drop-in box indicates the drop location.

If any signal from a waveform file is being displayed in a waveview window, the file icon changes, and the signal name is displayed in blue.

Alternatively, to plot all the signals of an instance, in the Waveform Browser, right-click an instance and select the **Plot Waveform to** option from the context menu.

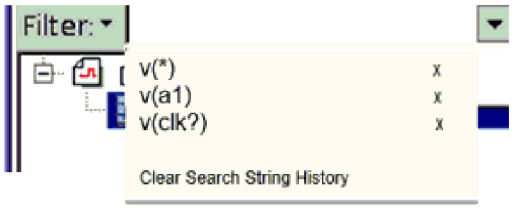
Pinning a Filter String

The toggle button  pins or unpins a filter string to the current hierarchy. Once pinned (grey), the filter string is restored whenever selecting this hierarchy. The filter string is not editable while pinned. When unpinned (red), the filter string follows the preference of **Keep Search String in the Signal List Filter**.

Feedback

Viewing Filter History

Filter history allows you to select previous strings entered into the filter line editor. The history dynamically updates as text is entered, and appears as a drop-down when the line editor is focused. For example:



The history list includes the last ten strings. Click **X** to remove an individual string, click **Clear Search String History** to remove all strings, and press Esc to close the history list.

Highlighting Waveforms

To highlight a waveform, click the signal check control button, and click the name of a signal to toggle its highlight state on and off.

You can also toggle a signal highlight by clicking the waveforms, which toggles the highlight states of all signals that intersect the mouse pointer.

To toggle which waveform is highlighted, press J to move down the waveform list and K to scroll up.

Deleting Highlighted Waveforms

Right-click a highlighted signal and choose **Cut** from the signal context menu to delete all highlighted signals.

You can also move highlighted signals in a panel with a drag-and-drop operation. With the signal mode control button armed, follow the steps in Moving Panels or Copying Panels to move highlighted signals between panels or waveviews.

Grouping and Ungrouping Highlighted Waveforms

Right-click a highlighted signal, and choose **Group** from the signal context menu to group all highlighted signals into a new panel.

You can also group signals by name by right-clicking signals and choosing **Group By Name** from the menu that opens.

To ungroup highlighted waveforms, right-click a highlighted signal, and choose **Ungroup** from the signal context menu to split each of the highlighted signals into a new panel.

Feedback

Copying Waveform Names

Right-click a highlighted signal, and choose **Copy Label** from the signal context menu to copy the names of all highlighted signals. This enables you to select and copy multiple signal names at one time. Note that the buffer size is limited to 8192.

Sending a Waveform to Results Analyzer

Right-click a signal in the Waveform Browser or in the WaveView panel and choose **Send to Results Analyzer** to add signal expression/waveform from WaveView to Results Analyzer.

The RA calculator opens where you can construct or modify an expression.

Waveform Files and Data

The section contains information on the following topics:

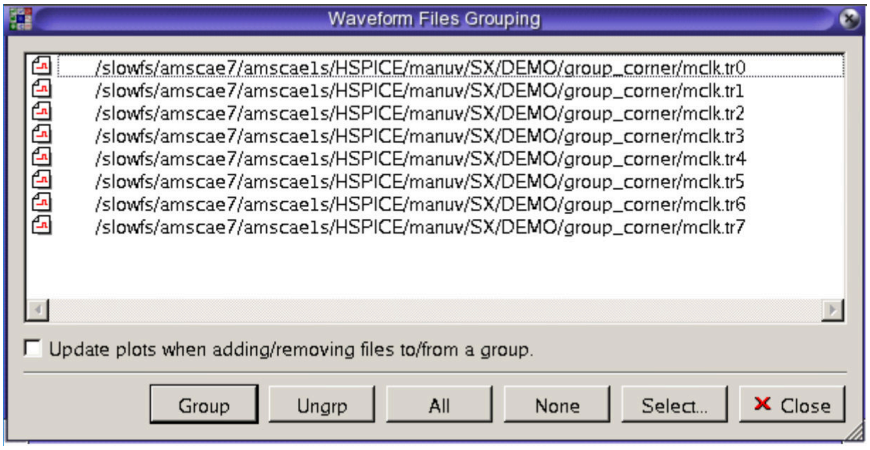
- Grouping Waveform Files
- Bookmarking Waveform Files
- Setting the Active Waveform File
- Updating Waveform Files
- Printing Waveform Files
- Exporting Waveform Data
- Marching Waveforms
- Saving and Loading SigLists
- Dumping the Screen

Grouping Waveform Files

Multiple waveform files can be collected into a group. When signals are loaded from a member in the grouped waveform files, the WaveView tool automatically loads the same signals from the other waveform files in the same group.

The grouping function is useful for loading signals from corner/alter simulations that usually involve multiple output files. To group multiple waveform files, right-click a waveform file, and choose **File Grouping** from the menu that opens. The **Waveform Files Grouping** window appears.

Feedback



Select waveform files in the list to group/ungroup them. For waveform formats that have associated sweeping parameters (such as the COU and the PSF format), click **Select** to filter files based on their parameter value conditions or by name of the `.fsdb` files.

The **File Grouping** function only ties multiple files into a group. It does not assign any parameter condition to each individual file. As a result, parametric analysis is not possible on these grouped files. To add corner parameter conditions to files, the files must be grouped into a file set using a link file (similar to the runObjFile approach). The link file not only specifies which files to be grouped, it also defines the associated parameter conditions for each file in the group. Grouped files using the link file are processed as a single file as if the result is from a sweep analysis. Since the link file provides sweeping parameter conditions, parametric analysis is also supported.

To create the link file, right-click a file name, and choose **Create File Set** from the menu that opens. To add parameter conditions to a file, enter the value in the (name=value) format, multiple values are delimited by comma (for example, temp=0,vdd=3.3). Click **Add** to add the parameter values to a file and the new values are indicated in the file list. Note that each of the files in a file set must have the same number of the same parameters. Finally, enter the path to the link file and click **Ok** to create the link file. Once a link file is generated, it can be opened directly as a regular waveform file.

If you want to update the plotted waveforms after files are added to or removed from a group, click the **Update plots when adding/removing files to/from a group** check box before clicking the **Group** or **Ungrp** buttons.

Bindkey for the **File Grouping** is available in the menu command.

Bookmarking Waveform Files

A waveform file can be added to the bookmark list. Click **Bookmarks** and choose **Manage Bookmarks** in the **File** tab.

Feedback

The **Add/Manage Bookmarks** window opens.

You can also reload a waveform file from the bookmark list from this window.

Setting the Active Waveform File

To have all plotted signals come from a specified waveform file, right-click a waveform file name in the WaveForm Browser and select **Set as Source of Plotted Signals**.

Updating Waveform Files

The WaveView tool can read growing waveform data files during a simulation job. Choose **Update** from the menu in the **File** section of the **Waveform** tab to update all displayed and derived signals.

If you restart a new simulation to override the old waveform data file, the update function automatically detects re-created data files and prompts you to select between keeping the old result or updating to new result.

If you keep the old data file, only the signals that are loaded are available from the old data file (because the old file is overwritten). An old data file is indicated by its blue file icon. To update an old data file to the new result, right-click the file in the Waveform Browser and choose **Update WDF** from the context menu.

To display signals from both the new and the old data file, choose to keep the old result. The old result file is saved and automatically opens the new data file.

If you need to update the WDF signal list, you can right-click the name of a file in the Waveform Browser and choose **Reload WDF Signal List** from the menu that appears.

Printing Waveform Files

On Linux platforms, the WaveView tool supports printout in the PostScript format only. To submit a print job, choose **Print** from the menu in the **File** section of the **Waveform** tab open the **Print Setup** window.

The print target is defaulted to the current active waveview, which can be changed by clicking **Change**. Select your preferred settings and click **Print** to submit the print job.

PostScript Print Layout

Multiple print options are supported for the PostScript printout:

- Print only the active waveview to a single page as shown on the screen.
- Print all signals in the active waveview using the display height. Multiple pages might be needed if the waveview has signals that are scrolled out of the view.

Feedback

- Print all waveviews in a single page, multi-column layout. The user can choose the number of columns preferred.
- Print all waveviews, each in a single page, as shown on the screen display.

Select the **Color PostScript** option if your printer supports color PostScript.

The default print setup is black/white in landscape orientation. The default printer device and user-defined print command can be predefined in the .spxrc file.

Printing on Linux Platforms

On Linux platforms, the Print Setup dialog box supports print out to different paper sizes and printers. Printer names can be predefined in the Printer Devices field of the Preference Settings dialog box. Multiple printer names are delimited by semicolon (;). Selected paper size and printer device are saved in `$HOME/.spxlast` when the application is closed. The stored values are automatically used as the default when the application starts again. If the `PRINTER` environment variable is defined in your Linux shell, it is always used as the default printer.

Printing on Windows Platforms

On MS Windows platforms, in additional to the PostScript print-to-file option, the WaveView tool also supports printout to all installed printers except line printers. PostScript format is supported as a file output option only. Choose **Print** from the menu in the **File** section of the **Waveform** tab to open the MS Windows version of the Print Setup dialog box. Select preferred options and click **Continue** to select a printer device, or select the path for the PostScript output file.

Exporting Waveform Data

To export waveform data, choose **Export** from the menu in the **File** section of the **Waveform** tab. Enter your output path and other settings in the Waveform Export Parameters dialog.

Confirm your output file path to open the **Waveform Export Parameters** dialog box.

Choose the panels or waveviews you want to export in the **Target** section: **Selected Panels**, **All Panels**, **Selected Waveviews**, or **All Waveviews**. If you choose a selected panel or waveform option, you are prompted to choose those panels or waveviews once you click the OK button.

Also, choose which signal names you want to export in the **Exported Signal Names** section: **Full Hierarchy** or **Net Name Only**. Click **Append File Name** if you want to append the file name to the associated exported signals.

Feedback

Waveform data in the active waveview can be saved using several different formats. The following formats are supported:

Format	Description
Text Table	Tabulated format. You can specify output options in the Text Table Output Option section of the Waveform Export Parameters dialog box. Selecting the **Suppress X-axis** column option hides the x-axis in the table output. Selecting the **Hex Logic Values** option changes the binary logic output values to hexadecimal values. Selecting the **Do not extend waveforms over the longer x-range of other waveforms** option prevents waveforms from being extended over an x-range of another waveform that is longer.
PWL Source	SPICE PWL (piecewise linear) format. You can specify PWL logic levels and slew values in the Logic Waveforms to PWL Sources section, as well as any netlist syntax options in the PWL Source Netlist Syntax Options section of the Waveform Export Parameters dialog box.
WDF	Synopsys WDF compressed format. You can specify the **Set Start X** and **Set Stop X** values in the Text Table Output Option section of the **Waveform Export Parameters** dialog box.
VCD	VCD file with analog support. You can specify the **VCD Timescale** in the lower right-hand corner of the **Waveform Export Parameters** dialog box.
M-File	file. Choose the panels or waveviews you want to export in the **Target** section: **Selected Panels**, **All Panels**, **Selected Waveviews**, or **All Waveviews**. If you choose a selected panel or waveform option, you are prompted to choose those panels or waveviews once you click the **OK** button. Also, choose which signal names you want to export in the **Exported Signal Names** section: **Full Hierarchy** or **Net Name Only**. Click **Append File Name** if you want to append the file name to the associated exported signals. You can specify the **Number of Valid Digits** in the lower-left corner of the **Waveform Export Parameters** dialog box.
FSDB	FSDB file. Choose the panels or waveviews you want to export in the **Target** section: **Selected Panels**, **All Panels**, **Selected Waveviews**, or **All Waveviews**. If you choose a selected panel or waveform option, you are prompted to choose those panels or waveviews once you click the **OK** button. Also, choose which signal names you want to export in the **Exported Signal Names** section: **Full Hierarchy** or **Net Name Only**. Click **Append File Name** if you want to append the file name to the associated exported signals. You can specify the starting and stopping X values by clicking the **Set X Start** and **Set Stop X** options in the **Text Table Output Options** section.

Feedback

Format	Description
OUT	.OUT file. Choose the panels or waveviews you want to export in the **Target** section: **Selected Panels**, **All Panels**, **Selected Waveviews**, or **All Waveviews**. If you choose a selected panel or waveform option, you are prompted to choose those panels or waveviews once you click the **OK** button. Also, choose which signal names you want to export in the **Exported Signal Names** section: **Full Hierarchy** or **Net Name Only**. Click **Append File Name** if you want to append the file name to the associated exported signals. You can specify the starting and stopping X values by clicking the **Set X Start** and **Set Stop X** options in the **Text Table Output Options** section.

By default, the WaveView tool only exports waveforms in selected panels using the x-axis step of the very first signal. Click **All Panels** to export waveforms in all panels.

Marching Waveforms

To automatically update waveforms from a running simulation job, choose **Marching** from the menu in the **File** section of the **Waveform** tab.

Enter the required update interval and the abort trial limit and click the **Start** button to start waveform marching. Automatic update is stopped if the waveform file is not growing after the retrial limit. The automatic update function can be manually stopped at any time by clicking the **Stop** button in the same dialog box.

If the automatic waveform marching is enabled, it is indicated with an icon in the main application toolbar.

Saving and Loading SigLists

Signal list files include all signal names that are currently plotted.

To save a signal list file, click **Save SigList** from the menu in the **File** section of the **Waveform** tab.

To load a signal list file, click **Load SigList** from the menu in the **File** section of the **Waveform** tab. Click the **Plot in current waveview check** box to plot all signals in the current waveview; otherwise, all signals are plotted in a new waveview.

Feedback

Dumping the Screen

All waveforms in a waveview can be included when performing a screen dump, even if the waveforms are scrolled above or below the currently visible portion of the waveview window.

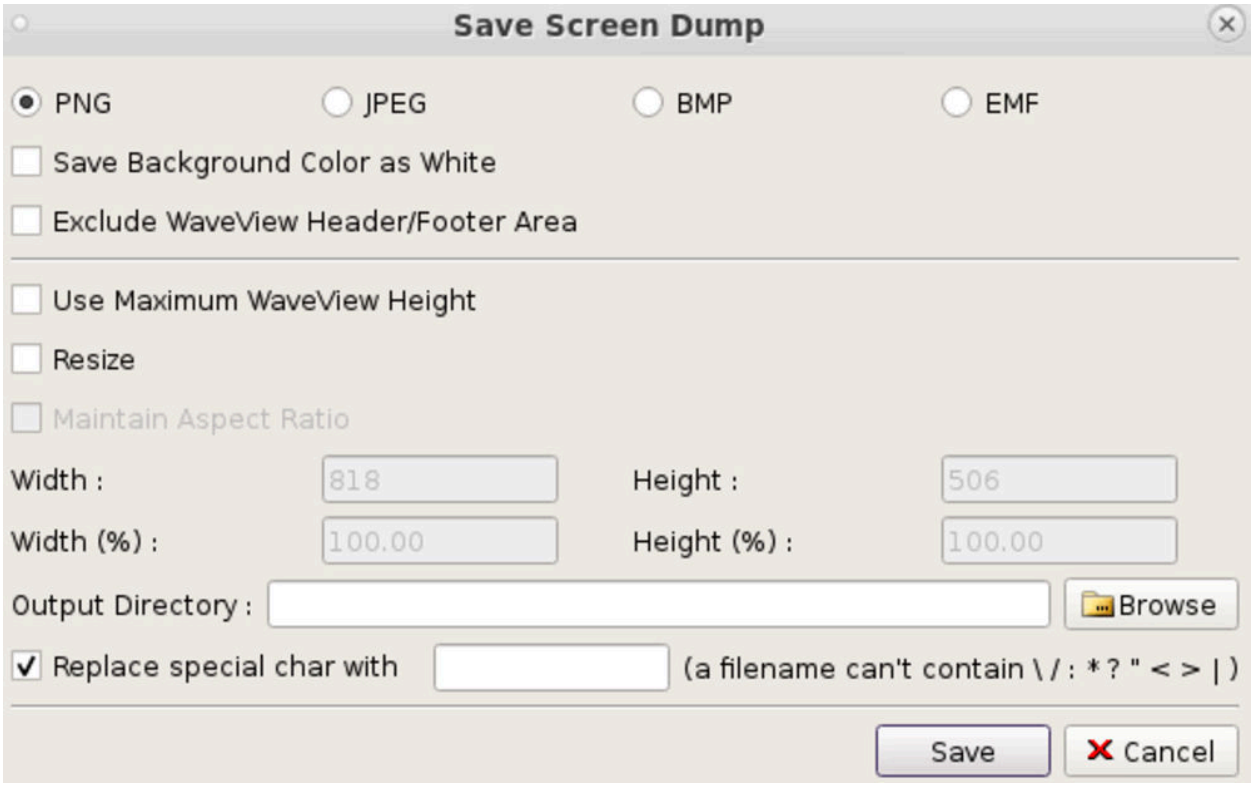
To include all off-screen waveforms, choose **Dump Screen** from the menu located in the **File** section of the **Waveform** tab. The **Save Screen Dump** dialog box opens. Click the **Use Maximum WaveView Height** check box, and click **OK**.

On Microsoft platforms, you can copy and paste the waveview contents to any application that has access to the clipboard. To copy the screen bitmap of a waveview window to the clipboard, choose **Copy to Clipboard** from the waveview context menu. To export a waveview in the vector-based Windows EMF (Enhanced Meta File) format, choose **Screen Dump to EMF**.

On Linux platforms, you can dump the waveview contents to either JPEG, PNG, BMP, or EMF formats. Choose **Dump Screen** from the waveview context menu to dump the display content.

To replace the special character that appears in the file name while dumping images, enable the **Replace special char with** check box, and enter the character of your choice. The special characters in the dumped file name are replaced with the entered character. For example, if the file name is `a?1:2<3`, and the number “5” is entered to replace the special characters, then the file name is updated as `a515253`.

Feedback



Dump All Screens

To dump all screens, choose **Dump All Screens** from the menu located in the **File** section of the **Waveform** tab. The **Save Screen Dump** dialog box opens.

Setting Waveform Attributes

To modify waveform attributes for a waveform:

1. Double-click the signal name of the waveform, or click **Settings** on the **Waveform** tab, and choose **Waveform Attributes** from the menu that opens.

   The **Waveform Attributes** dialog box opens with the attributes for the chosen signal.

2. Change any of the following waveform display attributes:

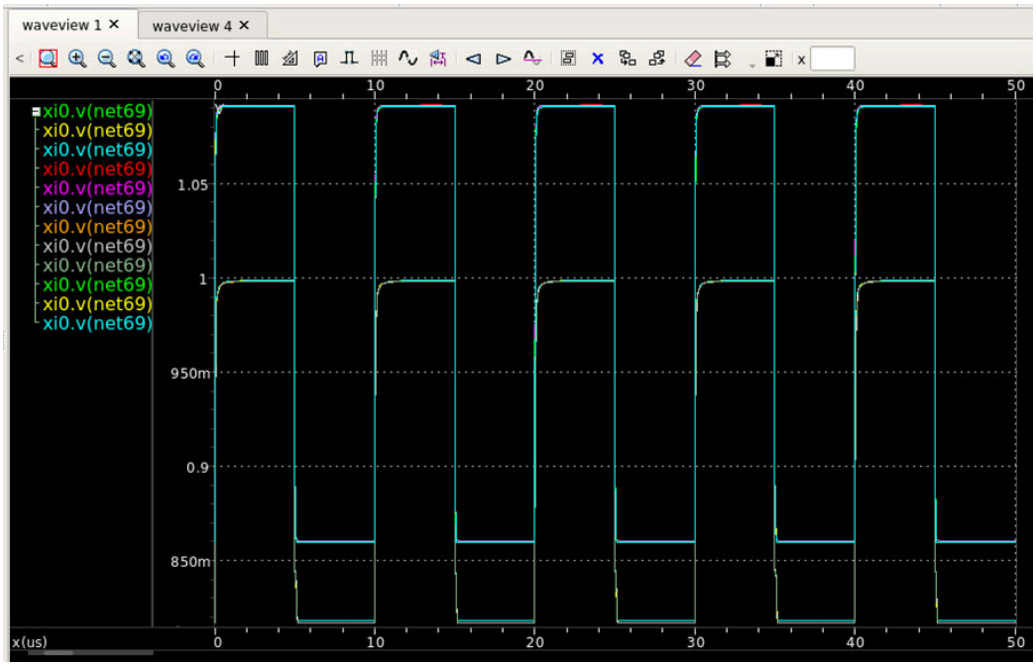
   - **Line Color**
   - **Line Width**
   - **Dash Type**
   - **Marker Shape**

Feedback

3. For analog signals, change the following attributes as necessary:
   - **Display Preference**
   - **Hide Signal**
   - **Axis** (left or right)
4. For digital signals, change the following attributes as necessary:
   - **Display Preference** and type
   - **Mnemonic On**
   - **Radix**
5. For analog or digital signals, enter the marker frequency, which changes the display frequency of the data point markers.
6. (Optional) Click the **Turn on Alias** check box to change the alias of a signal.
7. Click **Apply** to save your changes.

Working With Waveviews

The waveview window is the waveform display area in the WaveView tool. Click over the corresponding tab area of a waveview to bring it to the foreground.



Feedback

The following topics are covered in this section:

- The Active Waveview
- Adding a Waveview
- Deleting Waveviews
- Displaying WaveView Windows
- Displaying All Open Waveviews
- Displaying Waveview Titles
- Displaying the Hierarchy and Signal Browsers
- Displaying the Console Window
- Undoing and Redoing a Waveview Operation
- Refreshing Waveviews
- Renaming Waveviews
- Clearing Waveview Contents
- Dumping the Waveview Contents
- Docking and Undocking Waveviews
- Toggling the Tab Background Color
- Changing the Order of Tabbed Waveviews
- Synchronizing Waveviews
- Deleting All Open Waveviews
- Using the Drawing Tool

The Active Waveview

In the dock mode, the top waveview is considered the active waveview. In the undock mode, the top-most waveview or the selected waveview is the active waveview. Clicking the tab of a waveview window makes that waveview the active waveview.

Adding a Waveview

An empty waveview is initially opened when the WaveView tool starts. To add more waveviews, click **WaveView** in the **Add** section of the **Waveform** tab.

Feedback

The newly created waveview is placed on the top of the waveview stack.

Deleting Waveviews

To delete the active waveview, click the X button of a waveview tab.

Displaying WaveView Windows

A waveview window can have one or more non-overlapping Panels. Panels in a waveview can be arranged in two modes: vertical stack or independent row/column. In the vertical stack mode, panels stack top down and share a common horizontal axis. In the independent row/column mode, panels are arranged from left to right in a row. Panels in the row/column mode are independent of each other. A small icon at the upper-right corner of the waveview indicates the waveview orientation.

The following display modes are available:

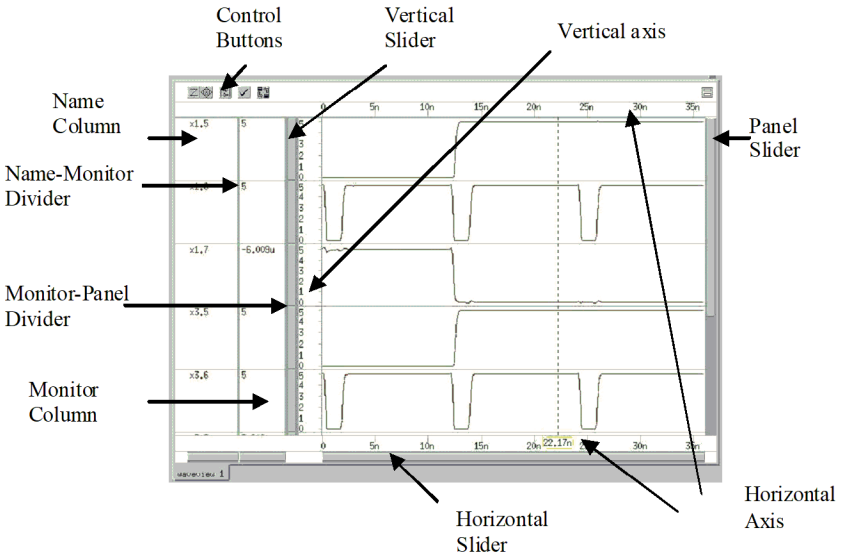
- Stack Mode
- Row and Column Mode
- Vertical Row and Column Mode
- Horizontal Row and Column Mode
- Tiled Row and Column Mode
- TableView Mode

Stack Mode

To view waveviews in vertical stack mode, right-click a waveview tab and choose **Stack Layout** from the menu that opens. Waveviews are displayed in this mode by default.

Feedback

The following figure shows the vertical layout display.



The vertical layout has the following display components:

- Control Buttons: Provides shortcuts to some of the commonly used waveview functions.
- Vertical Slider: Panes zoomed waveforms vertically. It is not displayed for logic panels.
- Vertical Axis: The vertical axis of a panel. For a logic panel, it displays the vector width and radix.
- Panel Slider: Panes the entire panel stack vertically when the waveview height is too small to show all panels. Mouse wheel can be used to scroll this slidebar.
- Horizontal Axis: The common x-axis of all panels, which appears at both the top and the bottom of a vertical waveview.
- Horizontal Slider: This slide bar is used to pane zoomed waves horizontally.
- Monitor Column: Displays waveform or cursor-related values. Multiple monitors can be added. A horizontal slide bar at the bottom scrolls long value strings horizontally.
- Monitor-Panel Divider: Defines the left boundary of the waveform plotting area. Click and drag over the divider to resize the width of monitor columns.
- Name-Monitor Divider: Defines the border between the name and monitor area. Click and drag the divider to resize the width of name and monitor columns.
- Name Column: Displays waveform names. A vertical slide bar scrolls name list vertically. A slide bar at the bottom scrolls long name strings horizontally.

Feedback

Row and Column Mode

To view waveviews in row and column mode, right-click a waveview tab and choose **Row/ Column Layout** from the menu that opens.

Vertical Row and Column Mode

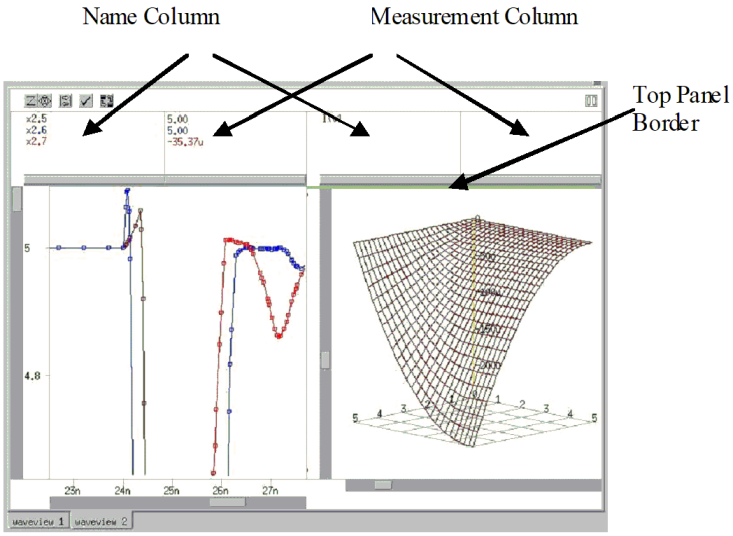
To view waveviews in vertical row and column mode, right-click a waveview tab and choose **Single Column Layout** from the menu that opens.

Horizontal Row and Column Mode

To view waveviews in horizontal row and column mode, right-click a waveview tab and choose **Single Row Layout** from the menu that opens.

You can use the mouse to change the height and width of the Signal Name panel in **Single Row Layout** by hovering the mouse over the edges of the panel until the cursor becomes a double arrow, then clicking and dragging to adjust the size.

The following figure shows the independent row and column display.



Feedback

Panels in the independent row/column layout are arranged in a similar way to the Vertical Stack mode, except for the following differences and additional display components:

- Name column: The name column is located at the top of each panel. The vertical and horizontal slide bars can be used to scroll long list or long name strings.
- Monitor column: The monitor column is at the top of each panel. A horizontal slide bar at the bottom scrolls long value strings horizontally.
- Top Panel border: The top border can be moved to redefine the top boundary of the waveform plotting area. Drag the divider and move it vertically to resize name/monitor height.

Each waveview window is associated with a waveview context menu that can be invoked from the upper right corner of the window, or from the waveview tab area. Choose an item from the waveview context menu to rename, delete or refresh a waveview, or edit the waveview title.

Tiled Row and Column Mode

To view waveviews in tiled row and column mode, right-click a waveview tab and choose **Tile Row/Column Layout** from the menu that opens.

TableView Mode

To view tabular data, click the  button in the button toolbar. Only tabular views are allowed in tableviews.

See Working With Tableviews for more information on tabular panels.

Displaying All Open Waveviews

To display a list of currently open waveviews from which you can choose, click **Select View** in the **Edit** section of the **Waveform** tab.

Displaying Waveview Titles

To hide or display the title of a waveview, right-click a waveview tab and choose either **Show Title** or **Hide Title** from the menu that opens.

Displaying the Hierarchy and Signal Browsers

To toggle the Hierarchy and Signal Browser displays, click **Browser** from the **Configuration** tab or press **Ctrl-H** on your keyboard.

Feedback

Displaying the Console Window

To toggle the display of the Console window, click **Console** on the **Configuration** tab. The Console window is displayed at the bottom of the WaveView tool main window.

Undoing and Redoing a Waveview Operation

To undo a waveview operation, click **Undo** in the **Edit** section of the **Waveform** tab. To redo a waveview operation, click **Redo**.

You can undo or redo panel, line, axis, cursor, monitor, measurement meter, drawing tool, and tabular operations.

Some waveview operations, such as adding or deleting a waveview, cannot be undone. The maximum undo and redo steps is limited to 100.

Refreshing Waveviews

Right-click the tab of a waveview, and choose **Refresh Waveview** from the menu that opens to refresh the display content of the active waveview.

Renaming Waveviews

Right-click a waveview tab, and choose **Rename Waveview** from the menu that opens to rename the active waveview.

Clearing Waveview Contents

To clear the contents of a waveview window, right-click inside a waveview and choose **Clear Waveview** from the menu that opens.

Dumping the Waveview Contents

All waveforms in a waveview can be included when performing a screen dump, even if the waveforms are scrolled above or below the currently visible portion of the waveview window.

To include all off-screen waveforms, right-click a waveview tab, and choose **Dump Screen** from the menu that opens. Click the **Use Maximum WaveView Height** check box, and click **OK**.

On Microsoft platforms, you can copy and paste the waveview contents to any application that has access to the clipboard. To copy the screen bitmap of a waveview window to the clipboard, right-click a waveview tab, and choose **Copy to Clipboard** from the menu that

Feedback

opens. To export a waveview in the vector-based Windows EMF (Enhanced Meta File) format, choose **Screen Dump to EMF**.

On Linux platforms, you can dump the waveview contents to the JPEG, PNG, or EMF formats. Right-click a waveview tab, and choose **Dump Screen** from the menu that opens to dump the display content.

Docking and Undocking Waveviews

Overlapping (docked) waveviews in the waveview window can be undocked into individual pop-up windows. Right-click a waveview tab, and choose **Dock/Undock** from the menu that opens to undock the waveview. To dock and undocked waveview, click the dock button  .

Toggling the Tab Background Color

To toggle the background color of individual waveform tabs between black and white, right-click on a tab and select **Black/White Background**.

Changing the Order of Tabbed Waveviews

If you have multiple waveviews open at the same time, you can change the order of those waveviews by dragging the tabs left or right.

Synchronizing Waveviews

A stack-mode waveview can be synchronized with other stack-mode waveviews. Cursors and the x-axis of a synchronized waveview are updated automatically when the x-axis display range or the active cursor location changes in other synchronized waveviews. Right-click a waveview and choose **Sync/Unsync** from the menu that opens to toggle the sync state of the active waveview, or choose **Sync/Unsync All** to toggle the sync states of all waveviews.

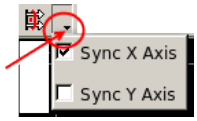
Toggling Sync Mode

The  toggle button in the WaveView toolbar switches the synchronization status of the current waveview. When in sync mode (), clicking the button removes all panels in the current waveview from the sync group. When in unsync (), clicking the button adds all panels to the sync group.

Feedback

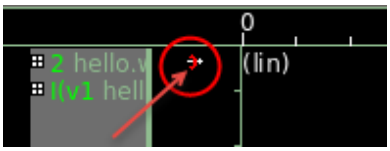
Setting Sync Axes

The option Sync Axes in the WaveView toolbar sets which axes to sync when in sync mode. This option is disabled in unsync mode. For example:



Adding Panels to a Sync Group

To add a panel to a sync group, **right-click** the panel and select **Add to Sync Group**. If the target panel is selected, all selected panels are added to the sync group. When in a sync group, the panel displays the following icon:



To remove a panel from a sync group, **right-click** and select **Remove from Sync Group**. If the target panel is selected, all selected panels are unsynced.

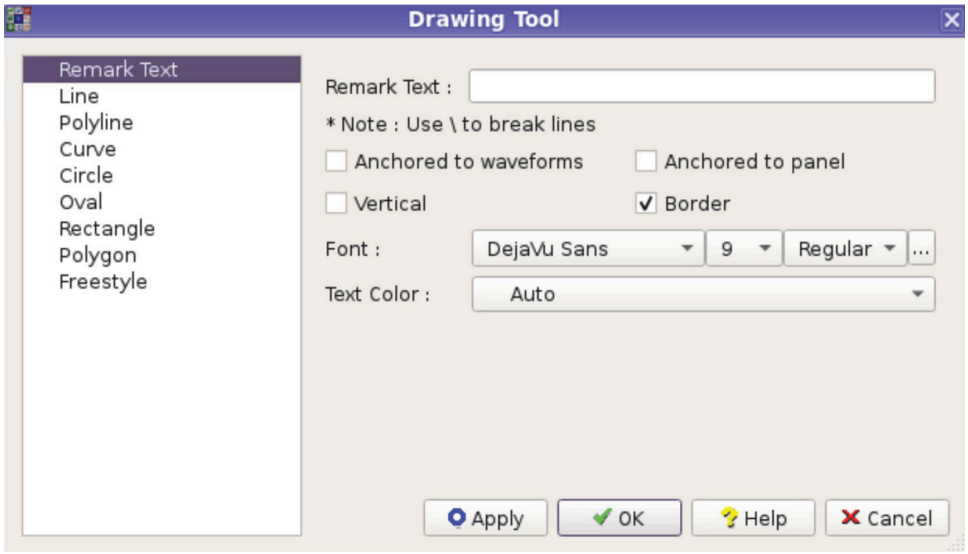
Deleting All Open Waveviews

To delete all currently open waveview windows, click **Delete All** in the **Edit** section of the **Waveform** tab.

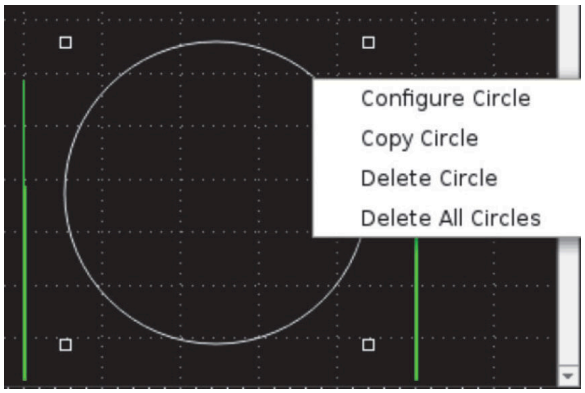
Using the Drawing Tool

To open the Drawing Tool:

1. Choose **Utilities > Drawing Tool** or click 
2. The Drawing Tool window is displayed as shown in the following image.

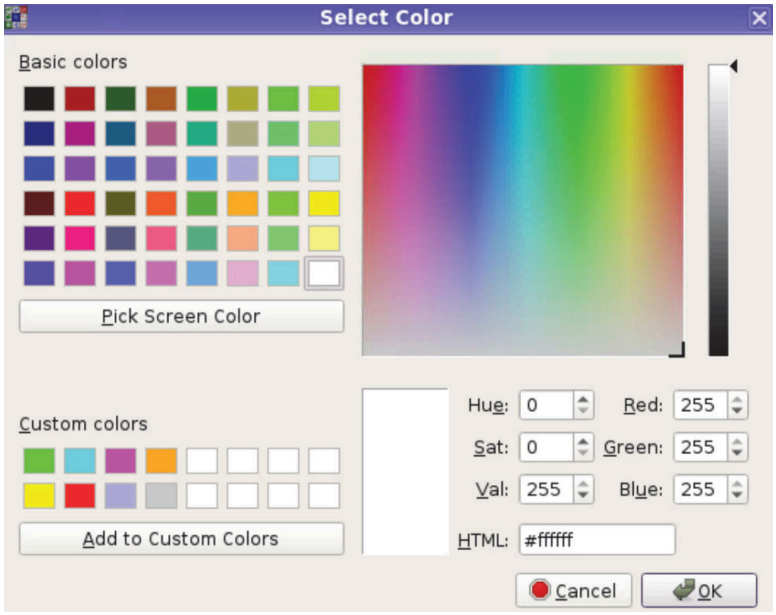


3. To draw a shape on a panel, select the required shape. Then, set the parameters like color, width, type, and so on.
4. Click on the panel and move the mouse to create the shape as per the selection.
5. Right click on the added shape to configure, copy, delete, or delete all sketches of the selected shape.



6. To add different colors to the shapes, select **More Colors** from the **Color** drop-down. The **Select Color** window is displayed.

Feedback



Select the required color and click **OK**. The **Drawing Tool** window is displayed.

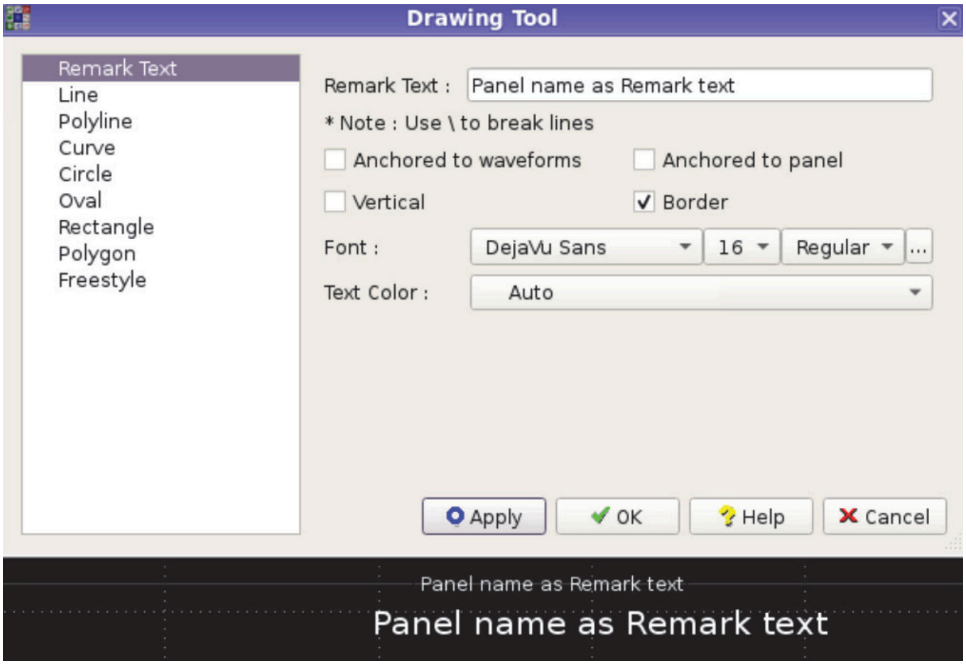
7. Click **Apply** and then click **OK** to complete the action.

The various shapes or options available in the **Drawing Tool** window are as follows.

Remark Text

Use this option to add a remark text to a panel.

Feedback



Enter the required text in the **Remark Text** field.

Select **Anchored to waveforms** to anchor the remark text to the waveform you choose with a visible line.

Select **Anchored to panel** to anchor the remark text to the panel. The remark text remains anchored even when you zoom in or zoom out..

Select **Vertical** to orient the remark text vertically on the panel.

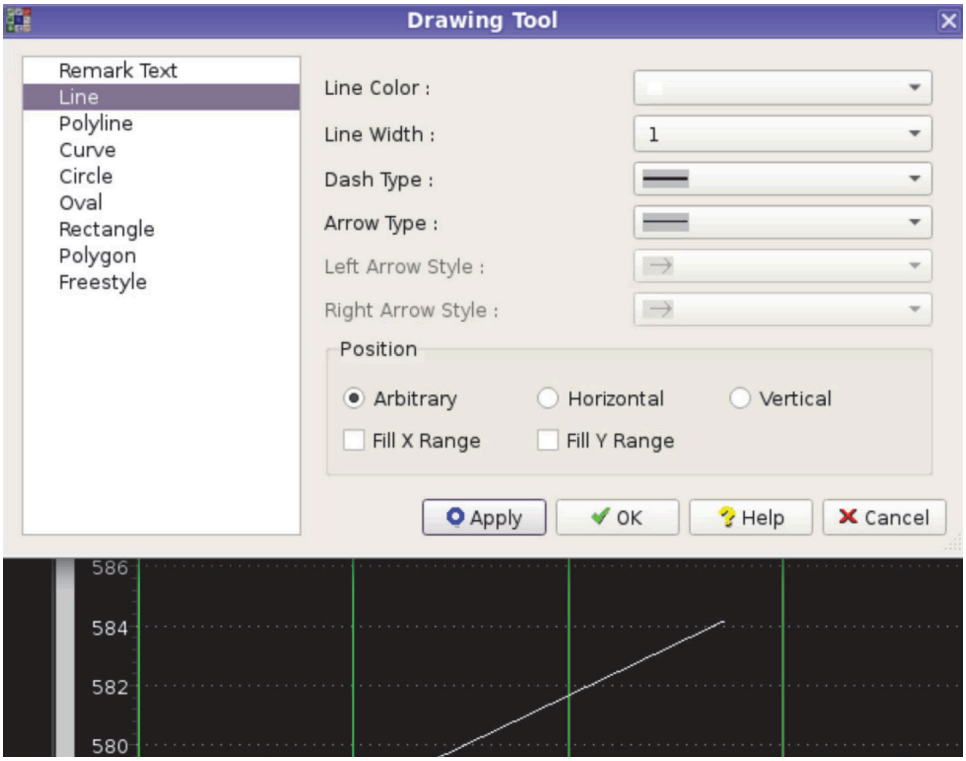
Select **Border** to place the remark text on the border of the panel.

Set the font and text color using the **Font** and **Text Color** options.

Line

Use this option to add a line on the panel.

Feedback



From the **Line Color** drop-down, select the required line color. The default is white.

From the **Line Width** drop-down, select the line width. The default is **1**.

From the **Dash Type** drop-down, select the line type. The default is a solid line.

From the **Arrow Type** drop-down, select the type of arrow to use. The available types are left, right, and double sided arrows.

Specify the x- and y-coordinate position of a line, which depends on the kind of line you want to draw. You can choose **Arbitrary**, **Horizontal**, or **Vertical**.

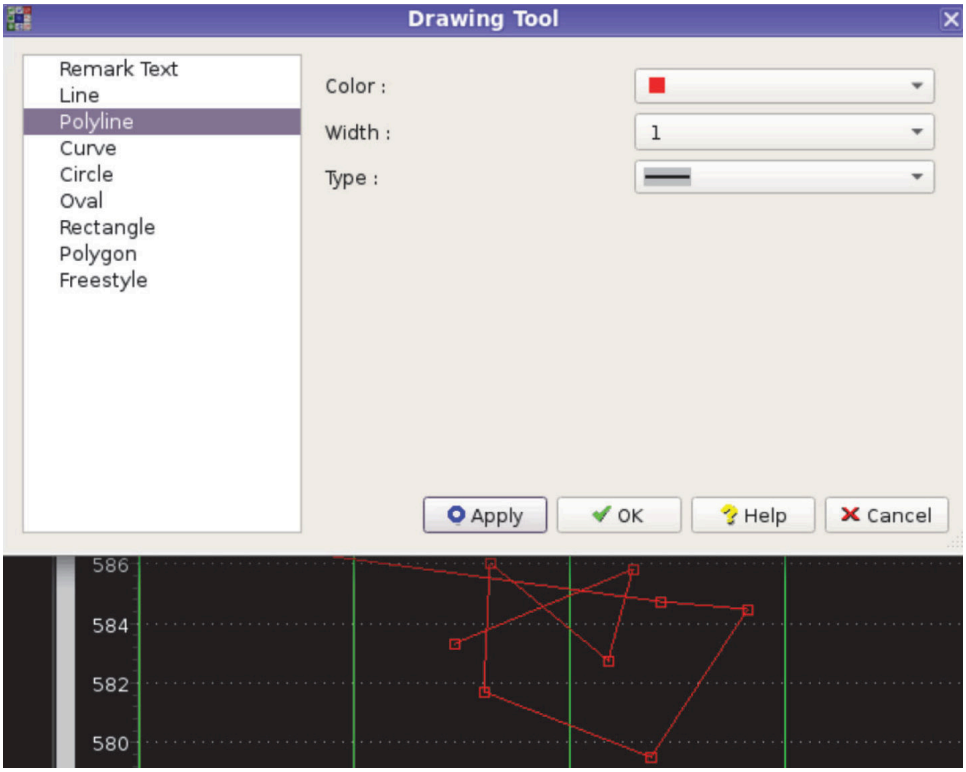
Select **Fill X Range** to draw a line that fills the entire range of the x-axis when you draw a horizontal line. This is enabled when either **Arbitrary** or **Horizontal** is selected.

Select **Fill Y Range** to draw a line that fills the entire range of the x-axis when you draw a vertical line. This is enabled when either **Arbitrary** or **Vertical** is selected.

Polyline

Use this option to add a group of connecting lines on the panel.

Feedback



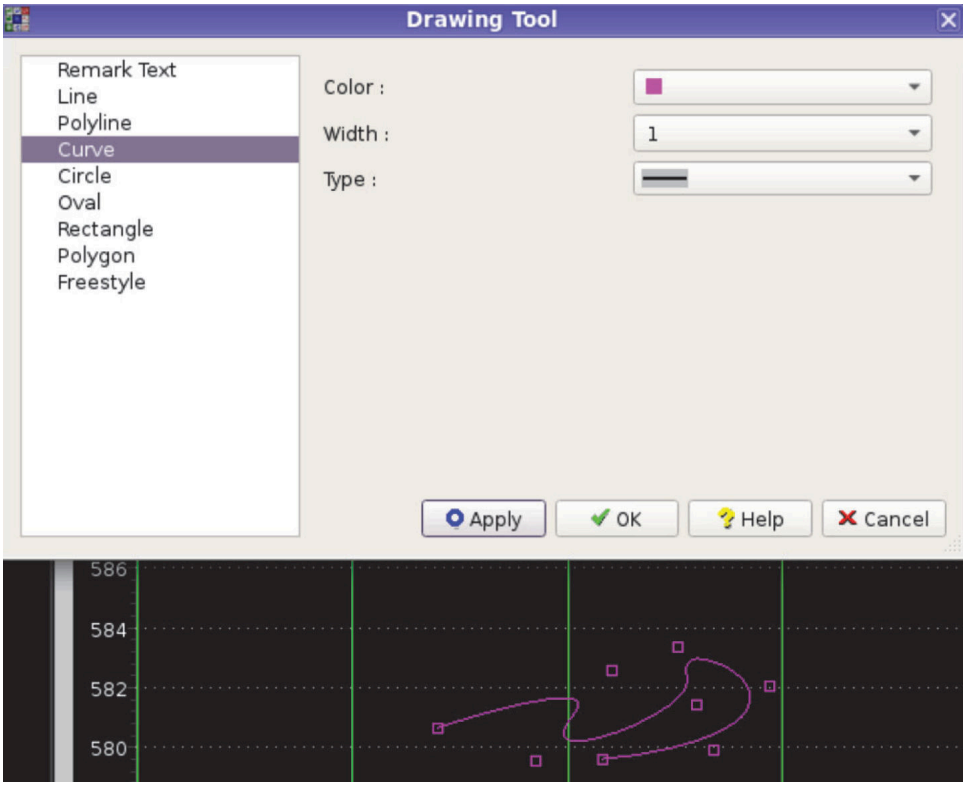
Specify the color, width, and type of the polyline.

Use the control points to drag and reshape the polyline as required.

Curve

Use this option to add a curved line on the panel.

Feedback



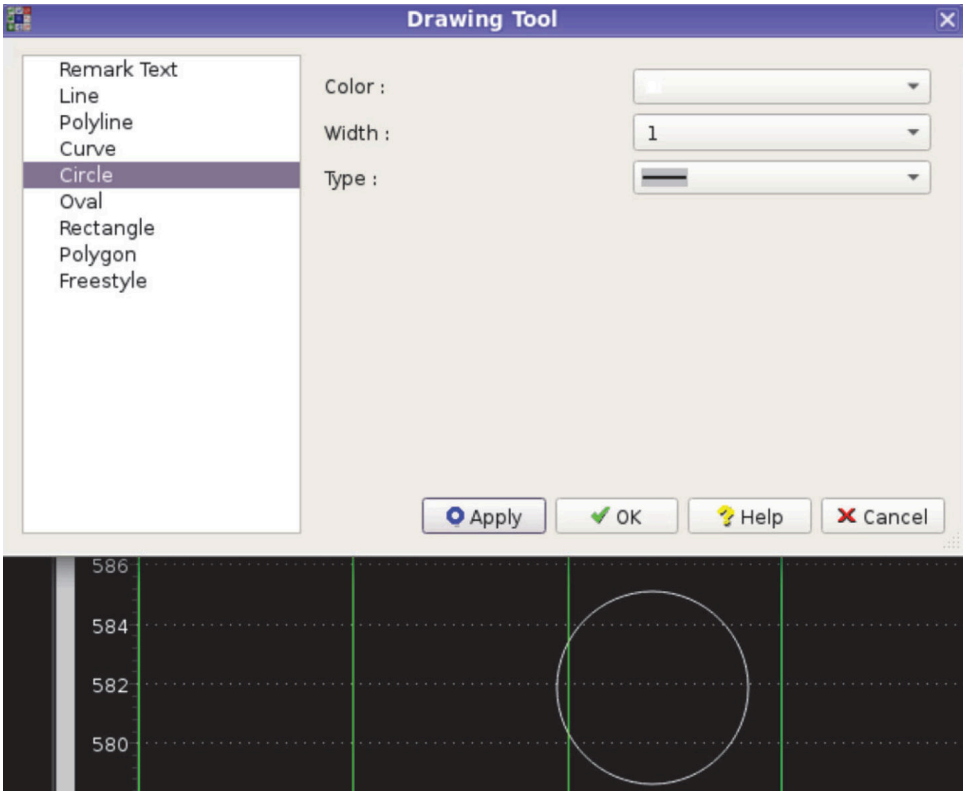
Specify the color, width, and type of the curve.

Use the control points to drag and reshape the curve as required.

Circle

Use this option to add a circle on the panel.

Feedback

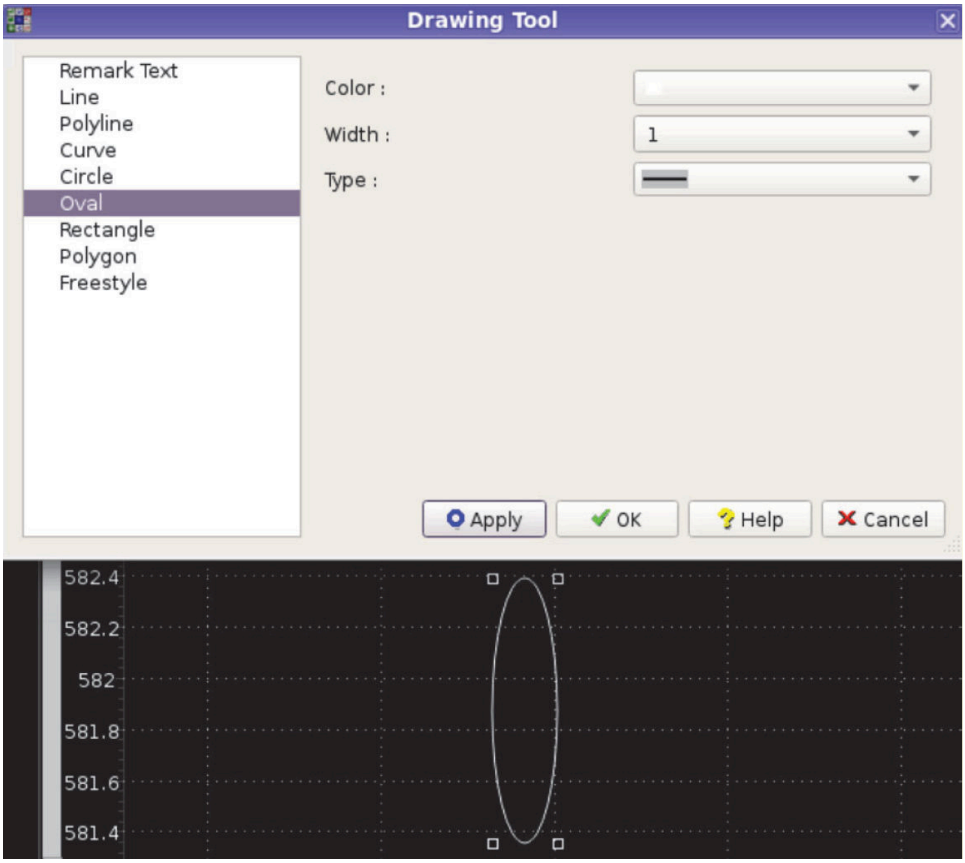


Specify the color, width, and type of the circle.

Oval

Use this option to add an oval on the panel.

Feedback



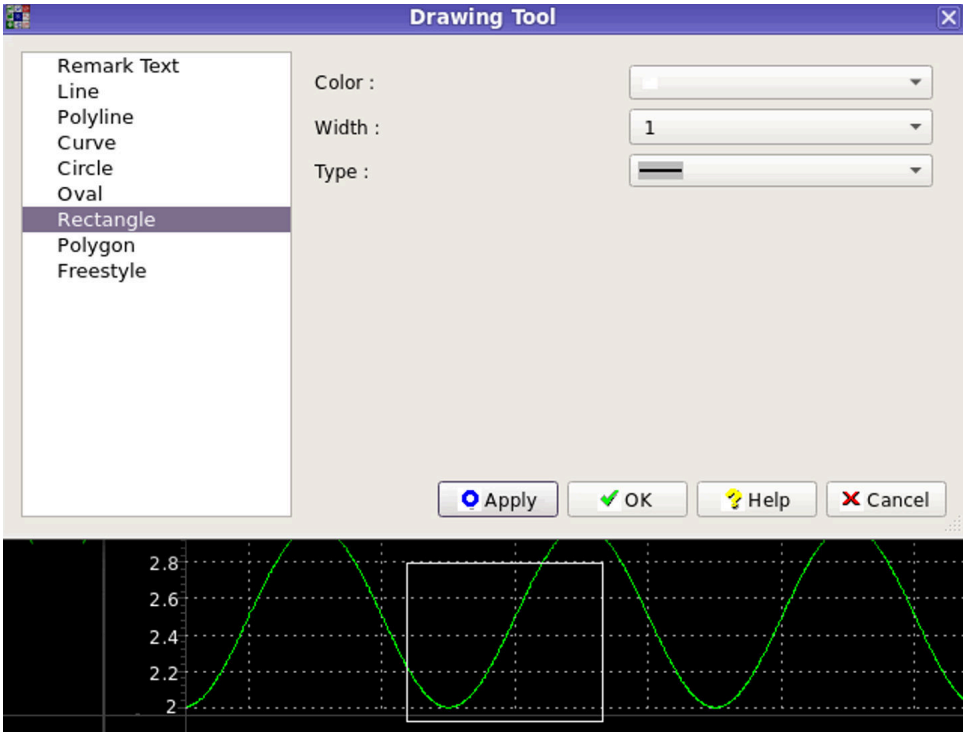
Specify the color, width, and type of the oval.

Use the control points to drag and reshape the oval as required.

Rectangle

Use this option to add a rectangle on the panel.

Feedback

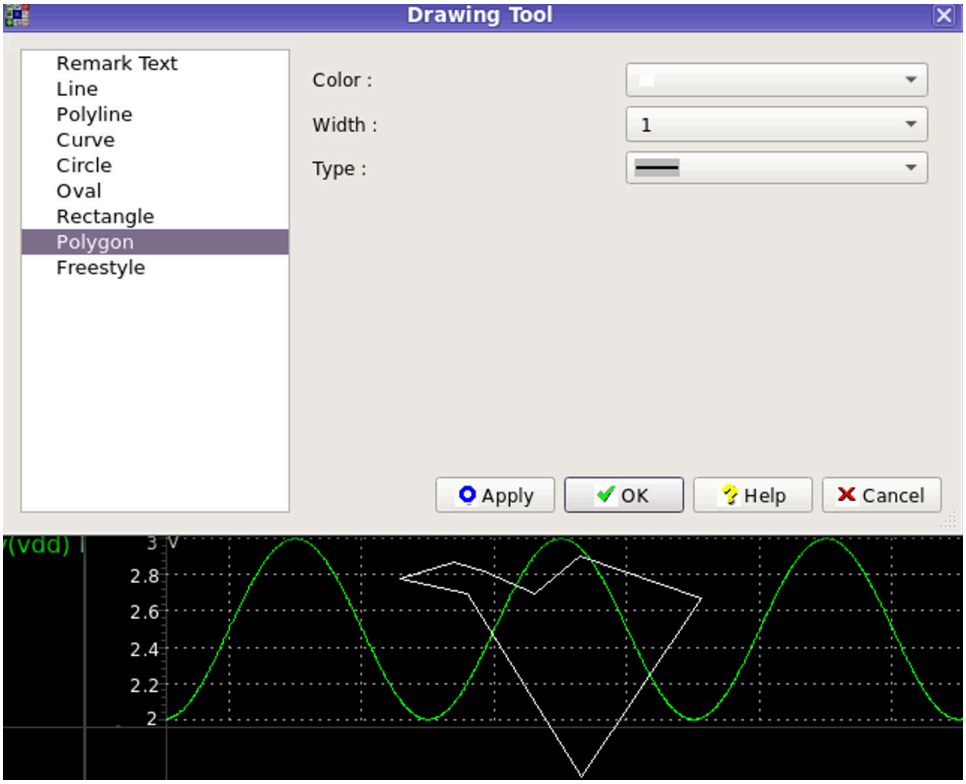


Specify the color, width, and type of the rectangle.

Polygon

Use this option to add a polygon on the panel.

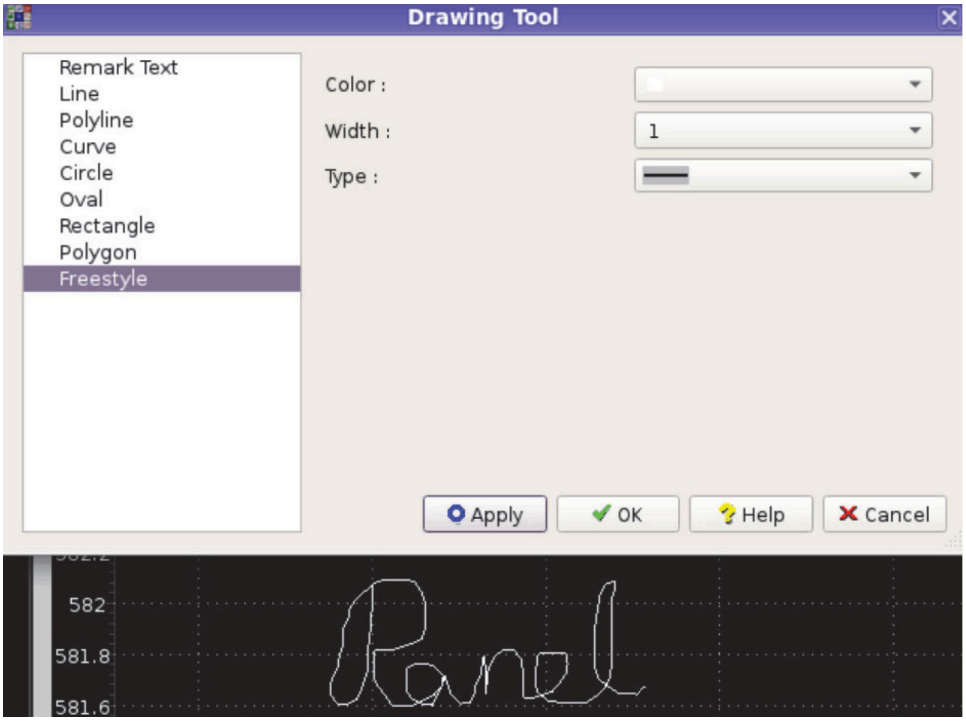
Feedback



Specify the color, width, and type of the polygon.

Freestyle

Use this option to add any shape of your choice on the panel.

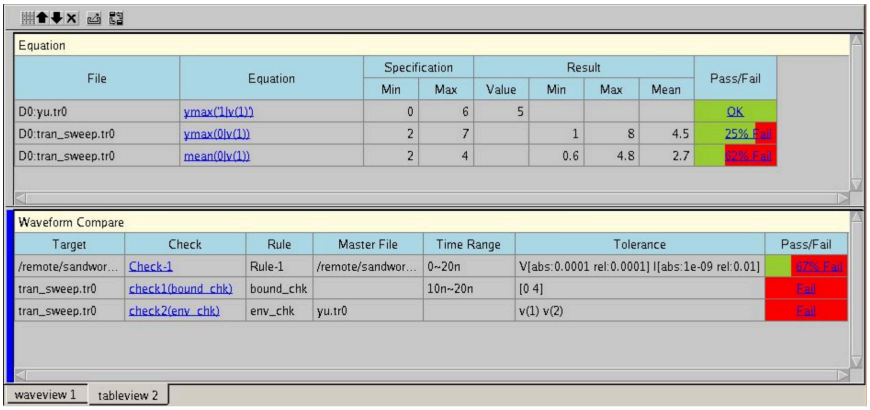


Specify the color, width, and type of the freestyle shape.

Working With Tableviews

The Tabular panel displays data such as waveform results and equations in a spreadsheet format. To create a Tabular panel, choose **Tabular** from the menu in the **Panel** section of the **Waveform** tab.

A new tabular panel is created on the current tableview. If the current view is not a tableview, a message is displayed that asks if you want to create a tableview.



Feedback

The Tabular panel can display the following types of information in separate table sections:

- equations
- signals
- waveform comparison results
- monitors

By default, only signals with scalar values are displayed in the tabular window (for example, when you double-click a signal from signal list). If you drag and drop a waveform result to tabular window, the signal is also displayed in the spreadsheet.

Note:

You can plot the value pairs from an X-Y panel into tabular format by right-clicking anywhere in an X-Y panel, and choose **Plot in Tabular** from the menu that opens. A new tableview is created with all the plotted value pairs from the X-Y panel.

Working With All Types of Tabular Data

This section contains the following information on how to sort and group all types of data, as well as how to hide and show table rows.

- Hiding and Showing Rows
- Sorting Tabular Column Data by Ascending or Descending Order
- Filtering Tabular Data

Hiding and Showing Rows

To hide a row in a Tabular window, right-click any row, and choose **Hide Row** from the menu that opens.

To show all rows that are hidden, right-click anywhere in the table, and choose **Unhide All Rows** from the menu that opens.

Sorting Tabular Column Data by Ascending or Descending Order

To sort column data by ascending or descending order, right-click a column title in the Tabular panel, and choose **Sort(Asc)** or **Sort(Desc)** to sort the data in ascending or descending order, respectively.

To sort the equations by signal names used in the equation strings, right-click the Equation column and select **Sort by Signal**.

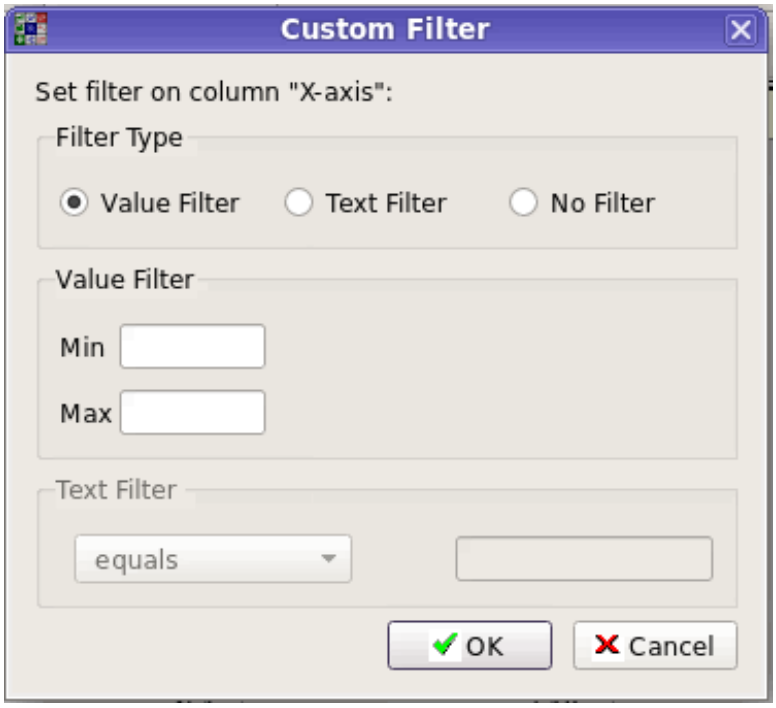
Feedback

Filtering Tabular Data

To filter tabular data:

1. Right-click a column heading in the Tabular panel that contains the data you want to filter, and choose **Value Filter** from the menu that opens.

   The **Custom Filter** dialog box opens.



2. Choose a Filter type: **Value Filter**, **Text Filter**, or **No Filter**.

   If you want to filter data by value, click **Value Filter**, and move on to the next step.

   If you want to filter data by a text string, click **Text Filter**, skip to Step 4.

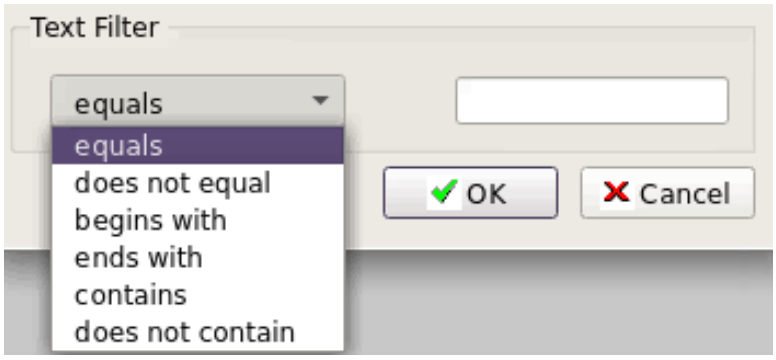
   If you click **No Filter**, skip to Step 6.

3. Provide the range of values to filter in the **Min** and **Max** fields.

   Skip to Step 6.

4. Choose a filter qualifier from the menu in the **Text Filter** section.

Feedback



The following qualifiers are available:

- **equals**
- **does not equal**
- **begins with**
- **ends with**
- **contains**
- **does not contain**

5. Enter a text string in the text box next to the filter qualifier menu to include in your filter.
6. Click **OK** to save your filter settings.

   The tabular data is filtered in the chosen column.

Working With Tabular Equation Data

The Equation section of a Tabular panel contains the following equation data:

Column Title	Description
File	File name.
Equation	Equation name (or equation string if no name is specified). If no corner or sweep results are available, the signal waveform is displayed instead of the equation.
Specification	Equation specifications.
Result	Equation results.
Pass/Fail	Pass or fail status. If no specifications exist, pass or fail result are not available.

Feedback

This section contains information on the following topics:

- Modifying Tabular Equations
- Adding or Removing Tabular Statistics Results
- Marking Annotating Equation Information From Equation Results
- Exporting Tables

Modifying Tabular Equations

To modify an equation, right-click the name of an equation in the **Equation** column, and choose **Modify** from the menu that opens.

See Using the Equation Builder for information on the Calculator.

Adding or Removing Tabular Statistics Results

To add or remove statistics in the Equation section of a Tabular panel:

1. Right-click the **Result** column, and choose **Statistics Setting** from the menu that opens.



   The **Statistics Setting** dialog box opens.

2. Click the check box next to each result type you want to display in the table.
3. Click **OK** to save your changes.

   The chosen result types are displayed in the table.

Feedback

Marking Annotating Equation Information From Equation Results

If there is annotation information, left-click the link in the Equation column or right-click and select **Show in WaveView** to annotate the information in the waveview. For sweep scalar results, first expand the details.



For example:

Exporting Tables

To export a table to an HTML or .csv file:

1. Click the button on the toolbar.

   The **Export Tables** dialog box opens.

2. Enter an **Output File** name, or click **Browse** to browse to and select one.
3. Choose an Output Format: **Web Page (*.html)** or **CSV (MS-DOS)(*.csv)**.
4. Choose a Target you want to export: **Selected Panel** or **All Panels**.
5. (Optional) Click the **Export Data With Unit Scale Factor** option to export the tabular data with the unit scale factor.

   This option is enabled by default.

6. Enter the number of valid digits in the **Valid Digits** text box.
7. Click **OK** to export your table data.

Working With Tabular Signal Data

The Equation section of a Tabular panel contains the following equation data:

Column Title	Description
File	File name.
Signal	Signal name.
Hierarchy	Hierarchy location of the signal.
Value	Value of the signal or waveform data.

Feedback

Column Title	Description
Unit	Unit of measure.

The Signal section of a Tabular panel contains the following signal data:

- File
- Signal
- Hierarchy
- Value
- Unit

Viewing Waveform Data

To view waveform data in the Signal section of a Tabular panel, click the icon in the Value column. A waveview opens with the waveform data displayed.

Working With Tabular Waveform Comparison Data

The Waveform Compare section of a Tabular panel contains the following waveform comparison data:

Column Title	Description
Target	Target file.
Check	Waveform check.
Rule	Rule name.
Master File	Master file name.
Time Range	Time range values.
Tolerance	Tolerance value.
Pass/Fail	Pass or fail status.

This section contains information on the following topics:

- Viewing Tabular Waveform Check Details
- Viewing Tabular Waveform Check Signal Details

Feedback

Viewing Tabular Waveform Check Details

To view waveform check details, click the name of a waveform check in the Check column in the Waveform Compare section. A new table opens with Signal and Difference Points/ Rate information for that waveform check.

Click the Waveform Compare link to return to the Waveform Compare section, or you can click a signal in the Signal column to display signal details. See Viewing Tabular Waveform Check Signal Details for more information.

Viewing Tabular Waveform Check Signal Details

To view waveform check signal details, click the name of a signal in the waveform check details table (see Viewing Tabular Waveform Check Details for more information). The check signal details table is displayed, which contains x-axis, Master (value), (Target) Value, Diff, Diff(%) information. Double-clicking on a row in the details table plots the related signals on a waveview displays a vertical line showing where the difference occurs.

Comparing and Checking Waveforms

This section contains information on the following topics:

- Sampling and Converting Waveform Data
- Creating a Waveform Compare Control File
- Defining Aliases
- Defining Rules
- Defining Parameters for Comparison and Checks
- Controlling X- and Z-Level Constraints
- Checking Waveforms
- Saving Waveform Comparison Results
- Using the Waveform Compare Tool

Feedback

Sampling and Converting Waveform Data

The following general rules apply when comparing waveforms:

- If a logic bus signal has different width (number of bits) between the master and the target file, only the less significant bits of the longer bus value are compared with the shorter bus value.
- Multiple signals are compared sequentially during the comparison process.
- Analog waveforms are compared based on the final V/I tolerance that is evaluated from user-defined tolerance values as:

  ```
  tolerance = MAX(abstol, reltol*master_waveform_value)
  ```

- A signal must exist in both the master and one of the target files for the comparison to take place.
- If one of the signals from the master file and the target file is a logic signal, logic comparison is used to compare the two signals.
- For analog waveforms, voltage tolerance is used if master and target signals are not both current type.

Creating a Waveform Compare Control File

Before the waveforms are compared, create a check rule file to compare signals and generate reports of any differences that exist.

The check rule file must include the following items:

- Path to the master file and the target files.
- Names of signals to compare and name mapping information if the signal names are different between the master file and the target files.
- Start, stop, delay, and step values for the comparison to take place.

Note:

When adding comments, ensure each line begins with a semicolon (;).

Feedback

Defining Aliases

Aliases define values that can be later referenced in the Rule and the Check sections of the control file. For example, you can define the following aliases to substitute a reference string in a control file:

```
alias small_step "step=200ps"
alias medium_step "step=.5ns"
alias initial_period "start=0ns, stop=5ns"
alias stable_period  "start=5ns, stop=16n"
alias end_period  "start=16ns, stop=20n"
```

Defining Rules

The Rule section of the Control File defines rules that you can use in the Check section to perform the actual checks. Multiple rules can be defined in the Rule section, and each rule must be defined using the following syntax format:

```
rule rule_name begin
 parm1=parm1_value
 parm2=parm2_value
end
```

The waveform compare function samples and compares waveform data at the steps you define. Each comparison rule can contain one master file and comparison parameters. You can specify the absolute and relative tolerances for voltage or current signals, respectively. Voltage tolerances are used if the signal type is unknown. Warnings are issued if the difference between the master signal and the target signal exceeds the following condition:

```
abstol + reltol * master_signal
```

To use a “one-direction” relative tolerance for comparison, add a plus (+) or minus (-) sign in front of the relative tolerance value.

You can define the following parameters in the Rule section:

- Type of check represented by a keyword. The following checks are supported:
  - Sample-based waveform compare function.

    See Checking Waveforms for more information.
  - Waveform monotonicity check.

    See Checking Waveform Monotonicity for more information.
  - Waveform absolute bound check.

    See Checking Waveform Bounds for more information.

Feedback

- Waveform envelop check.

  See Checking the Waveform Envelope for more information.

- Waveform Pattern Check

  See Checking the Waveform Pattern for more information.

- Comparison sampling step (overwrites global definition)
- Master file definition
- Check parameters such as tolerances and boundaries.

  For example, v_abstol, v_reltol, i_abstol, and i_reltol overwrites the defaults and Global definitions.

Note:

Aliases can be used in comparison sampling steps and tolerance or boundary check parameters.

Defining Parameters for Comparison and Checks

You can define parameters to control the waveform compare and checks. Any parameters defined outside of the Rule and Check sections are treated as global parameters. Any locally defined parameters inside of the Rule and Check sections supersede defined global parameters.

The following parameters are available:

- a2d_threshold
- batchsize
- bus
- delay
- full_compare_log
- ignorex
- ignorez
- len_reltol
- length_check
- map

Feedback

- mapfile, map_hier
- master
- match_type
- mean_range
- mean_step
- mean_tol
- pattern_check
- ref_clock
- report_style
- report_type
- report_vio_sig_only
- show_diff
- show_maxmin
- signal
- start, stop, step
- t_tol
- use_a2d_always
- use_sim_steps
- v_abstol, i_abstol, v_reltol, i_reltol
- when
- x_absmgn
- x_chkrange
- x_values
- xval_precision

a2d_threshold

The a2d_threshold statement applies to both the master and the target files. It describes the voltage threshold value for converting an analog signal to a digital signal or the

threshold range. This statement can be redefined in the check section to specify different thresholds for target signals. Default is 1.5. The syntax for the a2d_threshold value is:

```
a2d_threshold value
```
or

```
a2d_threshold low_value high_value
```

Note:

You can also use the a2d_threshold statement in the check section for target file signals using the same syntax. For example:

```
alias num1 2
alias num2 3

rule rule1 begin
    master yu.tr0
    step 1n
    use_a2d_always
    a2d_threshold num1 num2
    xval_precision 16
end

check begin
    target yu.tr0
    signal v(2)
    a2d_threshold 1 4
    rule rule1
end
```

batchsize

The batchsize statement controls the number of signals that are compared at a time, which can save memory and improve performance. The syntax for the batchsize statement is:

```
batchsize value
```

For FSDB and WDF files, set `value` to `1`. For PSF or *.tr0 files, use the following formula to calculate the appropriate `value`:

```
value <= host_RAM / (filesize * 2 / num_of_signals)
```

For example:

```
batchsize 200
```

Feedback

bus

The bus statement describes how a master logic bus value is interpreted when compared to an analog target signal. The following keywords are used with the bus statement:

- scale

  Specifies the scaling factor from the decimal bus value.

- signed/unsigned

  Specifies if the bus value is signed or unsigned.

- pwc/pwl

  Specifies if PWC or PWL interpolation is used for value in between two bus value transitions. The syntax of the bus statement is:

```
bus bus_signal_name scale=value unsigned pwc
```

delay

The delay value applies to both the master and the target files. It describes the shift amount of x-axis value the user prefers to add to the original waveforms. The value can be redefined in the master or target file section. Positive delay value implies shifting the waveform toward the positive x-axis direction. Default is zero. The syntax for the delay parameter is:

```
delay value
```

Note:

If you use delay with use_sim_step, the following occurs:

- The waveform shifts by the specified delay value.
- The time points from sim_steps (in the [start stop] time range when specifying time_range) is compared.

full_compare_log

The full_compare_log statement creates a log file using the `<rule_file_name>.log` file name syntax to record the comparison details for all points.

This log file records the following information:

- master values
- target values

Feedback

- difference values (target minus master values)
- tolerance

ignorex

The ignorex statement ignores x states during comparison.

ignorez

The ignorez statement ignores z states during comparison.

len_reltol

The len_reltol statement sets the relative length tolerance. The default is 0.01.

length_check

The length_check statement checks the length of a curve.

map

The map command is required when the compared signal names differ between the master and target files. One map statement is required for each mapped signal. The syntax of the map statement is:

```
map master_signal_name target_signal_name
```

mapfile, map_hier

The mapfile statement specifies a separate map file, which you can create to define master and target signals for comparison. The syntax of the mapfile statement is:

```
mapfile <mapfile_name>
```

The following example is a sample map file:

```
;mapfile1
;Master_SigName    Target_SigName
node_1    node_2
node_3
node_5    node_5
node_6    node_7
```

In this example, `node_1` and `node_2`, as well as `node_6` and `node_7`, are compared as the Master Signal and Target Signal for each pair, respectively. The `node_3` and `node_5` are compared as the same name.

Feedback

The map_hier statement maps the hierarchy of the defined signals. The syntax of the map_hier statement is:

```
map_hier <master_signal_hier> <target_signal_hier>
<master_signal_leaf_name> [<target_signal_leaf_name>]
```

The following example is a sample map file that uses the map_hier statement:

```
;commands inside the map file
map_hier testbench testbench.ifourphase
v(ck*)
end_hier
```

Note:

The mapfile statement can only be specified in the Check section, and map_hier can only be specified within a mapfile statement.

master

The master statement declares the file path to be used as the golden file when comparing waveforms.

match_type

The match_type statement forces target signals to be the same type as the master signals.

mean_range

The mean_range statement performs a mean comparison for a derived set of waveforms based on the defined starting and stopping range values. The syntax for the mean_range statement is:

```
mean_range start stop
```

When a mean check is enabled, the mean version waveforms are first created for the master and target signals. Using the mean check does not affect the time points used for the check. If the time value falls in one of the mean_range periods, the mean waveforms are used instead of the original waveforms. Multiple mean_range statements can exist in a single check section.

mean_step

The mean_step statement samples waveforms and performs mean comparisons based on the defined step value. The syntax for the mean_step statement is:

```
mean_step step_value
```

Default is 1ns. Mean comparisons are enabled as long as the mean_tol statement appears in the rule section.

Feedback

mean_tol

The mean_tol statement performs a mean comparison based on the defined tolerance value. The syntax for the mean_tol statement is:

```
mean_tol value
```

Default is 0.01.

pattern_check

The pattern_check statement searches for a pattern in logic signals. The syntax for the pattern_check statement is:

```
pattern_check <pattern>
```

For example:

```
pattern_check 101
```

ref_clock

A reference trigger clock signal can be used to determine the x-axis locations at which the comparison takes place. Two statements are required to define the reference trigger clock. Their syntax are:

```
ref_clock signal_name rise|fall|crossref_threshold value
```

The reference clock signal can be a logic or analog signal. For an analog reference signal, the ref_threshold value also needs to be defined. If the ref_threshold statement is omitted, the a2d_threshold value is used as the default. If ref_clock is defined in the rule file, the stop/start/step values are ignored.

report_style

The report_style statement defines the type of comparison report to produce. The syntax for the report_style statement is:

```
report_style value
```

For example, including the following line creates a report you can use with NanoSim:

```
report_style nanosim
```

report_type

The report_type statement defines what information is included in a comparison report. The syntax for the report_type statement is:

```
report_type value
```

You can specify the `value` as either `point`, `range`, or `.err` (`point` is the default value).

Feedback

report_vio_sig_only

When the report_vio_sig_only statement is specified, only signals with violations are reported in the output file. The report_vio_sig_only statement can be in the global or rule section, and It applies to all the following comparison types:

- waveform comparison
- monotonicity check
- length check
- bound check
- envelope check
- pattern searching

The syntax for the report_vio_sig_only statement is:

```
report_vio_sig_only
```

show_diff

When the show_diff statement is specified, the analog value differences in percentage are reported in the output file.

For example:

```
rule v_check begin
  master inv.mt0
  use_sim_steps
  v_abstol 0.0001
  show_diff
  show_maxmin
end

check begin
  target inv.mt1
  signal *
  rule "v_check"
end
```

show_maxmin

When the show_maxmin statement is specified, the max, min, and mean of the analog value differences are reported in the output file.

For example:

```
rule v_check begin
  master inv.mt0
  use_sim_steps
```

Feedback

```
  v_abstol 0.0001
  show_diff
  show_maxmin
end

check begin
  target inv.mt1
  signal *
  rule "v_check"
end
```

signal

The signal statement defines the signal to be compared and can be added in the Check section or used globally. Wildcards (* and ?) can be used in the name pattern. If the signal statement is omitted, all signals are compared. The syntax for the signal statement is:

```
signal name1 name2 … type=analog|logic|all -exclude
```

The `type` specifies the type of signal, and `-exclude` excludes the specified values from wildcard (* or ?) matching.

For example:

```
signal * -exclude i*
```

or

```
signal v* -exclude v(a1) v(a2)
```

Note that the signal line length is up to 1024. If the total length is greater than 1024, separate the signals with +.

For example:

```
signal sig1 sig2 ... sign

+ sigM1 sigM2 ... sigMn
```

start, stop, step

These values apply to the comparison process. The start and stop statements can be used globally, and the step statement can be defined in the Rule section or globally. All three can be redefined in the master file section. Default is 0. Syntax for these values are:

```
start | stop | step value
```

t_tol

The t_tol value specifies the point just before the continuous violation time range is exceeded. If the continuous violation time range is larger than t_tol, then the violation is reported; otherwise, it is considered as a passing result.

Feedback

The t_tol setting is only used when comparing one master versus one target, analog to analog comparison, and analog to digital comparison only. Setting t_tol to zero disables glitch filtering. The syntax for the t_tol statement is:

```
t_tol value
```

For example:

```
rule rule1 begin
    master target.tr0
    step 0.2n
    a2d_threshold 1.5
    t_tol 0.9n
    xval_precision 16
end

check begin
    target output_tb065.evcd
    signal v(a1)
    map v(a1) TESTBENCH.INST1.MEM1/MS_1T1C
    time_range 0 10n
    rule rule1
end
```

use_a2d_always

The use_a2d_always statement forces a comparison between two analog signals with converted logic values used for a2d_threshold.

use_sim_steps

The use_sim_steps statement uses the time steps saved in waveform files to determine where waveform comparisons occur on the x-axis.

v_abstol, i_abstol, v_reltol, i_reltol

The v_abstol, i_abstol, v_reltol, i_reltol values define the error tolerances for comparing analog waveforms. The values can be redefined in the target file section. Defaults are:

```
v_abstol=50uv, i_abstol=1nA, v_reltol=0.001, i_reltol=0.01
```

The syntax for setting the tolerance values is:

```
v_abstol | i_abstol | v_reltol | i_reltol value
```

The final error tolerance used internally for waveform comparison is evaluated using the following equation

```
MAX(ABS(abstol),reltol*ABS(master_v))
```

Feedback

Note:

`v_abstol` and `v_tolerance` are equivalent statements. `i_abstol` and `i_tolerance` are also equivalent statements.

when

The when statement executes when the specified condition is met. If when is used in the global or rule section, it applies to the master file. If the specified signal is not located in the master file, the tool tries to find the signal in the target file. If when is used in the check section, it applies to the target file.

Only simple conditions (!= and ==) are supported, and only one signal can be specified. For example:

```
when "v(b) != 1"
```

Arbitrary when statements are supported for logic signals.

x_absmgn

The x_absmgn value specifies the x-axis tolerance margin and is used to compare analog signals. The syntax for the x_absmgn statement is:

```
x_absmgn value
```

The `value` without a plus (+) or minus (-) sign specifies bidirectional tolerance. Adding a plus or minus sign specifies a unidirectional tolerance (`x_absmgn +5n` or `x_absmgn -3n`, for example).

x_chkrange

The x_chkrange statement performs logic transition waveform comparisons based on the value, which searches for a signal transition within the defined window. The syntax for the x_chkrange statement is:

```
x_chkrange value
```

The `value` without a plus (+) or minus (-) sign specifies bidirectional checking. Adding a plus or minus sign specifies unidirectional checking (`x_chkrange +5n` or `x_chkrange -5n`, for example).

Note:

The x_chkrange statement can be used in both rule and check sections when comparing digital signal transitions.

x_values

The x_values statement specifies the time points to compare.

Feedback

For example:

```
rule rule1 begin
   master comp_r.tr0
end

check check1 begin
   target comp_r.tr0
   signal v(rpin1)
   map v(rpin1) v(rpin2)
   x_values 1n 20n 45n 100n
   rule rule1
end
```

xval_precision

The xval_precision statement controls the x-value precision. The syntax for the xval_precision statement is:

```
xval_precision n
```

The `n` is the number of digits past the decimal point.

Controlling X- and Z-Level Constraints

X- and Z-level constraints change depending on the way you use some waveform comparison statements.

For example, when you use use_sim_steps and x_chkrange, the following X- and Z-level constraints are applied:

- The X and Z states remain as is and are compared. This is the default behavior.
- The default X and Z states change in the following manner if you use the following commands as described:
  - setx 0: Sets X-states to “Low” for comparison.
  - setx -1: Sets X-states to the previously known state for comparison.
  - setx 1: Sets X-states to “HIGH” for comparison.
  - setz 0: Sets Z-states to “Low” for comparison.
  - setz -1: Sets Z-states to the previously known state for comparison.
  - setz 1: Sets Z-states to “HIGH” for comparison.

Note:

Using the ignorex or ignorez statements with the x_chkrange statement might produce unwanted results.

Feedback

Checking Waveforms

This section contains information in the following topics:

- Waveform Compare Utility
- Waveform Check Example
- Checking Waveform Monotonicity
- Checking Waveform Bounds
- Checking the Waveform Envelope
- Checking the Waveform Pattern
- Rules Section Example

Waveform Compare Utility

The Waveform Compare Utility checks the rules specified in the check section sequentially on the specified targets, time range, and signal list. Checking parameters can be redefined to and apply to the subsequent checks in the check section. The check is performed sequentially on all rules specified in the Check section and reports the violations for each check.

You can specify the following parameters in the Check section:

- Target files.

  Multiple target files can be delimited by spaces or commas. For example:

  ```
  target "t1.dat t2.dat"
  ```

- Check start/stop time (x-axis) range.

  The time_range statement defines the check start and stop times.

  For example:

  ```
  time_range 10n 20n
  ```

- Target signals.

  Signal names can contain the asterisk (*) or question mark (?) wildcards. For example:

  ```
  signal "*"
  ```

- The rule to use.

  A predefined rule must be specified for each check. For example:

  ```
  rule "initial_v_check"
  ```

Feedback

Waveform Check Example

The following is a sample waveform check:

```
check begin
 ;; value can be a pre-defined alias
 time_range  "0ns, 6ns"
 ;; the rule must have been defined in the rule section
 rule  "initial_v_check"

 ;; missing target files are automatically removed from the

 ;; target file list
 target  "t1.dat, t2.dat, t3.dat, t4.dat"
 signal  "s1,s2,s3"
 time_range  stable_period
 rule  "v_check"

 ;; check different time range
 target  "t1.dat, t2.dat, t3.dat"
 signal  "s1,s2,s3"
 time_range  end_period
 rule  "last_master"

 target  "t1.dat"
 signal  "s1"
 time_range  "5n,15n"
 rule  "mono_chk"

 ;rule  "bound_chk"


 target  "t1.dat"
 signal  "s1"
 ; varcond_target  "VDD=5:TEMP=100"
 rule    "env_chk"
end
```

Checking Waveform Monotonicity

The monotonicity check function screens each of the specified signals in the target file for Y-value magnitude inversion. The monotonicity check does not require any input parameter. A warning is issued when the monotonicity is reversed and restored.

```
rule mono_chk begin
monotonicity_check
end
```

Checking Waveform Bounds

The bound check function screens every data point of the specified signals in the target file against the absolute lower and upper bounds defined in the `upper_bound` and

Feedback

`lower_bound` statements. A warning message is issued for each data point that exists outside of the bounds.

```
rule bound_chk begin
upper_bound 6.2
lower_bound 0.5
end
```

Checking the Waveform Envelope

The envelope check function screens every data point of the specified signals in the target file against the two 'bounding' signals defined in the `bound_sig1` and `bound_sig2` statements. A warning message is issued for each data point that exists outside of the bounds.

```
rule env_chk begin
 master gold1.dat
 bound_sig1 s1
 bound_sig2 s2
 ; master_sig1  sweep1.dat
 ; master_sig2  sweep2.dat
 ; varcond_sig1  "VDD=5:TEMP=0"
 ; varcond_sig2  "VDD=5:TEMP=100"
end
```

Checking the Waveform Pattern

The waveform pattern check searches for a specified pattern in logic signals. If the pattern is found, a message is issued with the x value where the pattern starts.

```
rule  pattern_chk begin
 pattern_check 101011
end

check begin
 target foo.vcd
 signal *
 time_range "0ns 40ns"
 rule "pattern_chk"
end
```

Rules Section Example

The following is a sample rules section:

```
rule initial_v_check begin
    master gold1.dat
    step   small_step
    v_tolerance 1mv
    v_reltol    +0.01
end
```

Feedback

```
rule v_check begin
    master gold2.dat
    step   .1ns
    v_tolerance 1mv
;    delay 1ns
    v_reltol    -0.01
end
```

If a waveform compare rule contains no master file, the previous master file in the check sequence is used. For example:

```
rule last_master begin
    step .2ns
    v_tolerance 2mv
end
```

Saving Waveform Comparison Results

Waveform comparison results can be saved in a session file with the violated intervals highlighted by adding the `-x session_file` option to the `wv -compare` command. The operating system and version information is also included in this file. You can load the session file into the WaveView tool for further analysis.

Using the Waveform Compare Tool

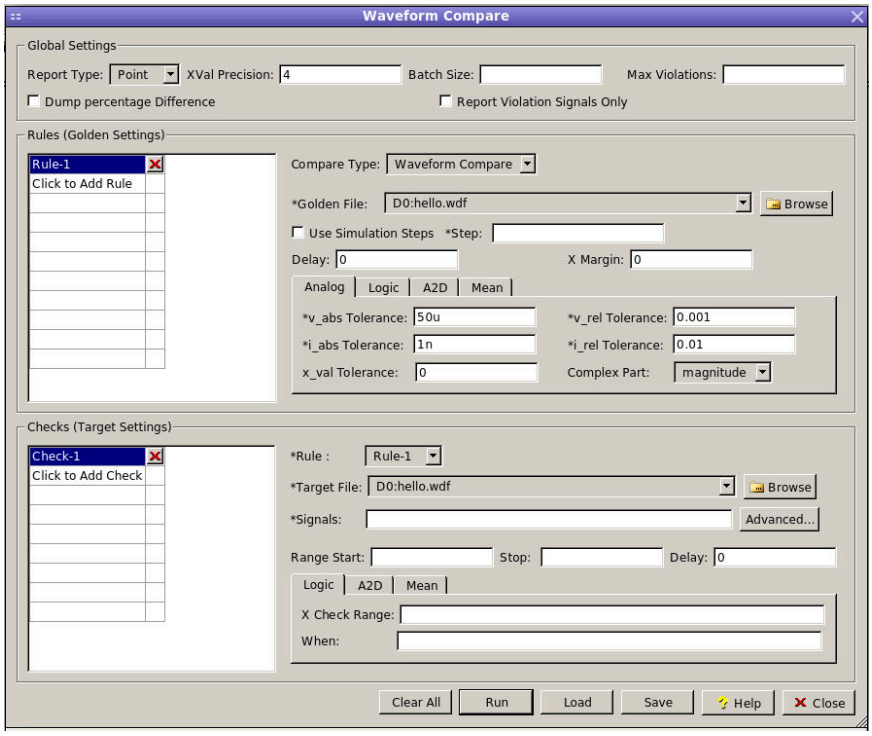
Note:

This feature requires a WaveView ADV license.

To compare waveforms in the GUI:

1. Click **Waveform Compare** in the **Tools** tab.

   The **Waveform Compare** window opens.



2. Specify Global Settings.

   See Specifying Global Settings.

3. Add one or more rules.

   See Adding Rules (Golden Settings).

4. Add one or more checks.

   See Adding Checks (Target Settings)

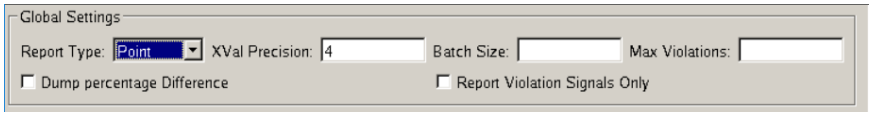
5. Run your comparison.

   See Running Comparisons.

Specifying Global Settings

To specify Global Settings for a waveform comparison:

1. Ensure you have the **Waveform Compare** window open (click **Waveform Compare** in the **Tools** tab).
2. Choose the type of report you want to generate from the **Report Type** menu in the Global Settings.



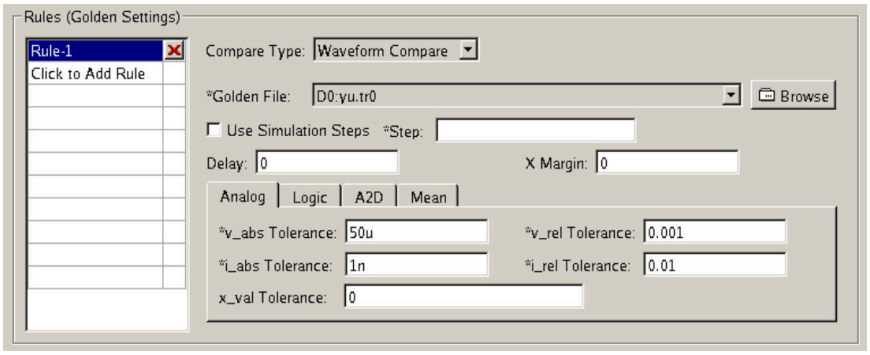
3. (Optional) Enter values for the **XVal Precision**, **Batch Size**, and **Max Violations**.
4. (Optional) Click the **Dump Percentage Difference** check box to include the percentage difference in the comparison report.
5. (Optional) Click the **Report Violation Signals Only** check box to include only the signals that have violations in your comparison report.
6. To add rules to your waveform comparison, continue to the next section, Adding Rules (Golden Settings).

Adding Rules (Golden Settings)

Note:

If you already have a rule file set up, click the **Load** button to load your rule file.

To add rules to your waveform comparison, choose what kind of comparison you want to run from the **Compare Type** menu in the **Waveform Compare** window. Double-clicking on rule name in the rule table enters editing mode where you can modify the rule name. No spaces are allowed in rule names.



The following comparison types are available. Any settings for each comparison type that has an asterisk (*) next to it are required:

Comparison Type	Description
Waveform Compare	Compares waveforms. Continue to the Adding Waveform Compare Rules section.
Envelope Check	Checks the specified envelope. Continue to the Adding Envelope Check Rules section.

Feedback

Comparison Type	Description
Monotonicity Check	Checks for monotonicity. No further rule setup in necessary—continue to the Adding Checks (Target Settings) section.
Bound Check	Checks the specified bounds. Continue to the Adding Bound Check Rules section.
Length Check	Checks the specified length. Continue to the Adding Length Check Rules section.
Pattern Search	Searches for the specified pattern. Continue to the Adding Pattern Search Rules section.

Adding Waveform Compare Rules

To add waveform compare rules:

1. Choose a golden file from the **Golden File** menu in the **Waveform Compare** window, or browse to and select the golden file you want to use.
2. Enter the step value you want to use for the waveform comparison in the **Step Value** text box, or click the **Use Simulation Steps** check box to use the steps from your simulation instead.
3. (Optional) Enter delay and X margin values into the **Delay** and **X Margin** text boxes.
4. Enter values for the following Analog tolerance values on the Analog tab:
   - **v_abs Tolerance**
   - **i_abs Tolerance**
   - **v_rel Tolerance**
   - **i_rel Tolerance**
   - **x_val Tolerance**
5. (Optional) Click the **Always Convert Analog signals to Logic then Compare** check box to always compare analog to digital.
6. Enter values for the **High** and **Low** A2D Threshold.
7. Click the Logic tab, and choose values from the **Treat as X** and **Treat as Z** menus.
8. Enter a value into the **X Check Range** text box for the X shift you allow when comparing a transition.

Feedback

This is enabled only when **Use Simulation Steps** is selected.

9. Enter a value into the **When** text box to specify a logical condition.

   The data is compared when the condition is true (for example, when "xclk/clkin==0") .

10. Click on **Mean** tab.
11. Click the **Enable Mean Check** check box to check the mean values of each step instead of comparing the original values.
12. Click **Mean Steps** to specify the steps used in a mean check.
13. Click **Mean Tolerance** specify the absolute tolerance for mean value comparisons.
14. To add checks to your waveform comparison, continue to the next part of the waveform setup, Adding Checks (Target Settings).

Adding Envelope Check Rules

To add envelope check rules:

1. Choose a signal file from the **Bound Signals File** menu in the **Waveform Compare** window, or browse to and select a file.
2. Enter signal names in the **Bound 1** and **Bound 2** text boxes.
3. To add checks to your waveform comparison, continue to the next part of the waveform setup, Adding Checks (Target Settings).

Adding Bound Check Rules

To add bound check rules, click the **Upper Bound** or **Lower Bound** check boxes in the **Waveform Compare** window, and enter values for the upper bound or lower bound limit, respectively.

To add checks to your waveform comparison, continue to the next part of the waveform setup, Adding Checks (Target Settings).

Adding Length Check Rules

To add length check rules:

1. Choose a golden file from the **Golden File** menu in the **Waveform Compare** window, or browse to and select the golden file you want to use.
2. Enter values for the **Absolute Tolerance** and **Relative Tolerance**.
3. To add checks to your waveform comparison, continue to the next part of the waveform setup, Adding Checks (Target Settings).

Feedback

Adding Pattern Search Rules

To add pattern search rules, enter a pattern in the **Pattern to Use** text box in the **Waveform Compare** window.

To add checks to your waveform comparison, continue to the next part of the waveform setup, Adding Checks (Target Settings).

Adding Checks (Target Settings)

You can add multiple checks for each rule you add. To add checks to your waveform comparison setup:

1. Choose a rule from the **Rule** menu in the **Waveform Compare** window, which you already set up in the Adding Rules (Golden Settings) section.

   The displayed check settings change depending on the type of rule you choose.

2. Choose a target file from the **Target File** menu, or browse to and select the target file you want to use.
3. Enter the names of one or more signals you want to check into the **Signals** text box.
4. Click **Advanced** to specify more signal related settings.
5. Enter a value for the **Max Hierarchy Level**.

   0 is the default.

6. Choose a type of signal from the **Signal Type** menu.
7. Click the **Match Type** check box to match the type of signal.

   If you are setting up a check for a rule that is Length Check comparison, no further setup for this check is available. Skip back to Step 1 to add more checks for rules, or if you are finished, skip to Step 16. Otherwise, continue to the next step.

8. Enter values for the **Range Start** and **Stop**.

   If you are setting up a check for a rule that is Monotonicity or Bound Check comparison, no further setup for this check is available. Skip back to Step 1 to add more checks for rules, or if you are finished, skip to Step 16. Otherwise, continue to the next step.

9. Enter a value for the **Delay**.
10. Specify the master to target signal name mappings in the Signal Mapping Table.
11. Click the **Logic** tab.
12. Enter a value into the **X Check Range** text box for the X shift on a target signal if different from the X Check Range value in Rule section.

This works only when you enable **Use Simulation Steps**.

13. Enter a value into the **When** text box to specify a logical condition using target file signals.

    The data is compared when the condition is true (for example, when xclk/clkin_b==0).

    Skip back to Step 1 to add more checks for rules, or if you are finished, continue to the next step.

14. Click on the **A2D** tab, and enter the **Low** and **High** A2D thresholds if they differ from the thresholds you set in Rule section.

15. Click the **Mean** tab, and enter the Mean check start and end values.

16. Your waveform comparison setup is complete. Continue to the final part of the waveform comparison, Running Comparisons.

Running Comparisons

To run a waveform comparison:

1. Ensure you have the waveform comparison setup complete.

   See Using the Waveform Compare Tool for more information.

2. Click the **Run** button.

   The Save Waveform Compare Result File dialog box opens.

3. Enter a name for your result file, and click **OK** to save it in the required location.

   The waveform comparison runs, and the results are saved in your result file. The result file is automatically opened in the WaveView tool, and the result is plotted to a tableview.

   See Working With Tableviews for more information on working with tabular waveform comparison data.

Comparing Logic Signals

Note:

A WaveView ADV licenses is required for this feature.

The batch mode sample-based waveform compare function compares mixed-signal waveforms between multiple target waveform files and a golden master waveform file. The comparison rules are defined in a check rule file that is used to control the comparison process.

Feedback

To compare waveforms in batch mode using the Waveform Compare utility:

1. Review the general rules that apply when comparing waveforms.

   See Sampling and Converting Waveform Data for more information.

2. Create a check rule file if you want to compare signals and generate reports of any differences that exist.

   See Creating a Waveform Compare Control File for more information.

3. Use the following syntax at the command line to start a comparison:

   ```
   wv -compare rule_file [output_file] [-x session_file]
   ```

   The `output_file` is the name of the output file where the comparison results are saved. If you do not specify a name, the comparison result is displayed directly in the standard output. If you specify `-x session_file`, a session file is created, which saves the comparison files, signals, and differences information. You can load this file into the WaveView tool to view the comparison results.

During comparison, if any errors occur due to unlocated or mismatched signals, a list of the mismatched signals are saved in a mismatch.txt file.

Setting User-Specified Colors for Diff Markers

You can specify colors for diff markers in the session file created by waveform compare, as follows:

```
wv -compare <rulefile> -x <session_file> -diff_color
 <color_or_color_list>
```

or:

```
wv -x <session_file> -diff_color <color_or_color_list>
```

where `-diff_color` specifies a color or list of colors from these supported color keywords: `black`, `white`, `grey`, `red`, `magenta`, `brown`, `blue`, `cyan`, `green`, `gold`, `darkgreen`, `yellow`, `orange`, `purple`, or `lightpurple`.

4

Plotting Signals

This chapter details plotting signals in the WaveView tool.

This section contains information on the following topics:

- Using Panels
- Panel Types
- Searching Signals
- Scanning Signals
- Data Reduction
- Direct Plot Mode

Using Panels

This section contains information on the following topics:

- Panel Actions
- Specifying Panel Attributes
- Setting Panel Display Options
- Setting Signal Display Options
- Setting Radix Options
- Setting Signal Name Options
- Adjusting Panel Height

Feedback

Panel Actions

The following panel actions are available:

- Adding Panels
- Deleting Panels
- Grouping Panels
- Creating a Panel Set
- Selecting Panels
- Displaying Previous and Next Loaded Signals
- Adding Labels to a Panel
- Moving Panels
- Copying Panels

Adding Panels

Panels are created automatically when signals are dropped into a waveview window. The default panel type is selected based on the signal type.

To manually add a specific type of panel into the active waveview window, choose one of the panel types from the menu in the **Panel** section of the **Waveform** tab.

See Panel Types for information on the different panel types.

Deleting Panels

To delete a panel, right-click a panel, and choose **Delete Pane** from the menu that opens. If the target panel is selected, all selected panels are deleted.

You can also select a panel and click **Action** in the **Panel** section of the **Waveform** tab, and choose **Delete** from the menu that opens.

Pasting Deleted Panels

Deleted panels can be pasted back to any waveviews by choosing **Paste** from the panel context menu. The deleted panels are always pasted after the current panel in which the mouse pointer resides.

Grouping Panels

Selected panels can be grouped into one panel. To group panels, select the panels you want to group, right-click one of the selected panels, and choose **Group** from the menu that opens.

Feedback

You can also select a panel and click **Action** in the **Panel** section of the **Waveform** tab, and choose **Group** from the menu that opens.

If the grouped panels are in a logic panel type, a new bus signal is formed. The Bus Configuration dialog box allows you to configure the bus signal.

Panel grouping action can also be invoked from the Panel Group control button.

Ungrouping Panels

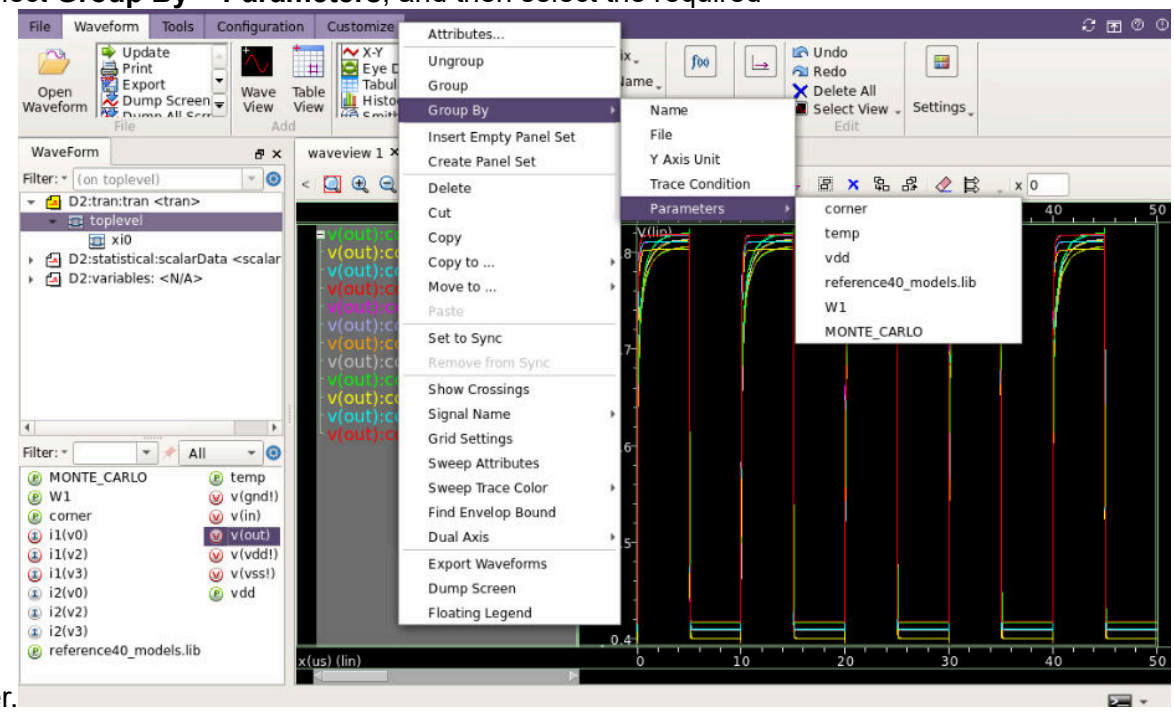
Signals inside a panel can be split so that each signal is placed into a new panel. To ungroup panels, select the panels you want to ungroup, right-click one of the selected panels, and choose **Ungroup** from the menu that opens.

You can also select a panel and click **Action** in the **Panel** section of the **Waveform** tab, and choose **Ungroup** from the menu that opens.

Group By

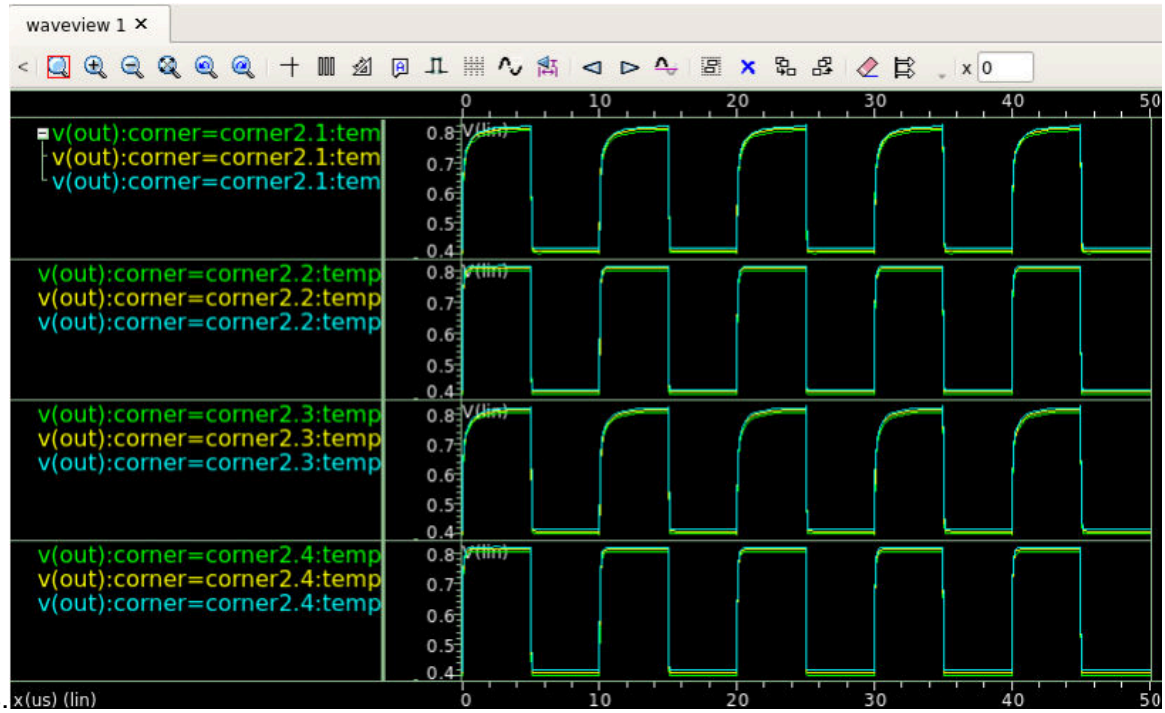
For a signal having corner and parametric sweep, you can group by corner and sweep parameters.

To perform this, right click on the signal panel of a multi-member signal, select **Group By > Parameters**, and then select the required parameter.



Feedback

In the image above, when you select a corner, the signals are grouped according to their corner names.



Creating a Panel Set

See Panel Set for information on using a panel set to group signals.

Selecting Panels

To select all panels in a waveview, click **Action** in the **Panel** section of the **Waveform** tab, and choose **Select All** from the menu that opens.

You can also select one of more individual panels by clicking the name or monitor area of a single panel or clicking multiple panels as you hold the **Shift** or **Ctrl** keys.

Deselecting All Panels

To deselect all panels, click **Action** in the **Panel** section of the **Waveform** tab, and choose **Select None** from the menu that opens.

You can also use the Panel Select control button to toggle between the Select-All and the Unselect-All operations.

Displaying Previous and Next Loaded Signals

To display the previously loaded signal from the Signal Browser, click **Action** in the **Panel** section of the **Waveform** tab, and choose **Scan Previous** from the menu that opens.

Feedback

To display the next loaded signal from the Signal Browser, click **Action** in the **Panel** section of the **Waveform** tab, and choose **Scan Next** from the menu that opens.

Adding Labels to a Panel

You can use the Drawing Tool to add labels to a panel, as well as control the line color and width on a panel.

See Using the Drawing Tool for more information on the Drawing Tool.

Moving Panels

To move a panel, click and hold a name or value area of a panel. Drag the panel to a new location and release the mouse button. If the source panel is selected, all selected panels are moved.

During the dragging process, a drop hint appears to indicate the drop location.

Copying Panels

To copy a panel, click and hold a name or value area of a panel. Drag the panel to a new location and release the mouse button. If the source panel is selected, all selected panels are copied.

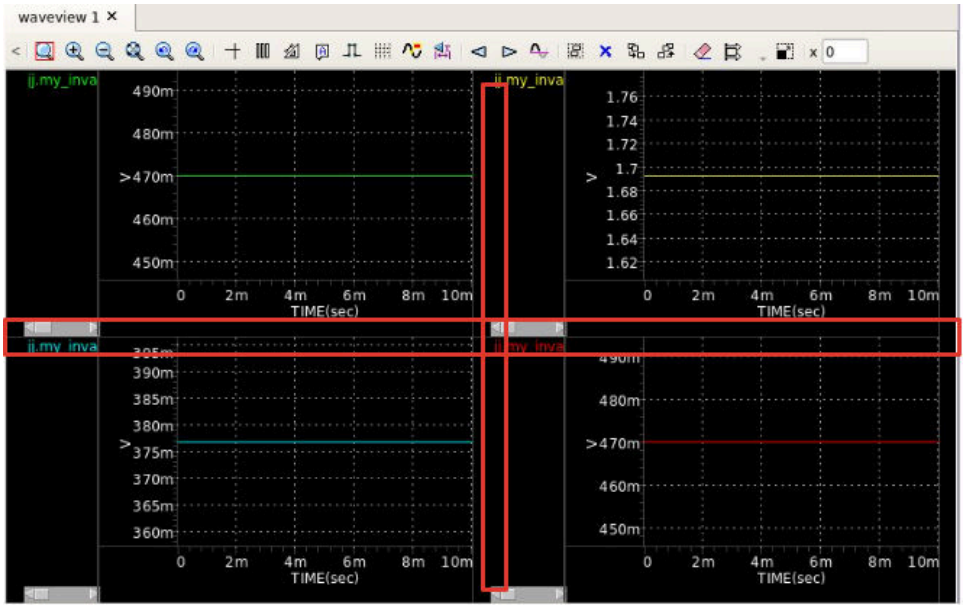
During the dragging process, a drop hint appears to indicate the drop location.

Panels can be copied from a waveview window to another waveview window. Use the preference settings to select between the **Move** and the **Copy** operations. Panels can also be copied and pasted using the corresponding menu entries in the panel context menu.

Resizing Panels

To resize a panel, hover the mouse pointer over the panel border and drag the border to resize the window size.

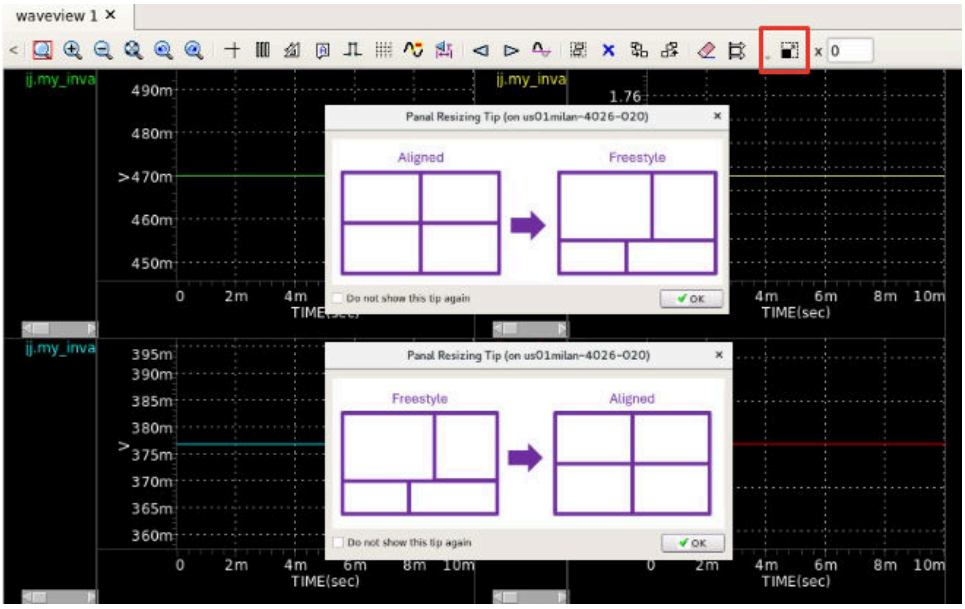
Feedback



Two modes resizing are supported:

- Aligned (default)
- Freestyle

To switch between the modes, click the toolbar resizing button as shown in the following image.



Feedback

Specifying Panel Attributes

To change the attributes of a panel, right-click anywhere inside a panel, and choose **Attributes** from the menu that opens.

The following attributes are available depending on the type of panel:

Panel Type	Attribute	Description
All Panels		
	Panel menu	Specifies the panel to which the attributes are applied.
X-Y, **Histogram**, **Polar Plot**, and **2D Sweep**		
	Fit to WaveView	Displays a waveview to fit the height.
	Default Height	Displays a waveview at the default height.
	Signal Name Display menu	Specifies which signal name to display.
	Signal Name Type menu	Specifies the displayed signal name type.
	Sort Names	Sorts the names of signals in alphabetical order.
	Display Preference menu	Specifies the display preference of a panel.
	Show Right Axis	Displays the right-hand y-axis.
	I-Signals to Right Axis	Uses the right-hand y-axis for the current signal.
StatEye		
	Zero Threshold	The zero threshold value (the default is `1.00001E-20`).
	Full Waveform Range	When enabled, the full waveform range of the Z display range is used. When disabled, you can enter **Min** and **Max** range values for the Z display range.
	Use Waveform Range	When enabled, the waveform range of the Z axis range is used. When disabled, you can enter **Min** and **Max** range values for the Z axis range.

Feedback

Panel Type	Attribute	Description
	Z Scale Type	Specifies the type of Z scale: **Linear** or **Log** scale (the default scale is **Log**). **Note:** When the scale type is **Log**, the **Min** and **Max** values for range setting need to be in the log10 scale.
	Set User Colors	Specifies colors for BER values. BER values can be a single value (for example, `1m`), or a range (for example, `2m-3m`).
Smith Chart		
	Impedance Chart or **Admittance Chart**	Specifies a Smith chart as an **Impedance Chart** or **Admittance Chart**.
	Impedance Data or **Admittance Data**	Specifies displayed data as **Impedance Data** or **Admittance Data**.
	Characteristic Z	Default value is `5.0000E+01`.

Setting Panel Display Options

This section contains information on the following topics:

- Grid
- Zoom
- XAxis
- YAxis
- Font
- Full Scale
- X Variable
- Axis Attributes

Grid

You can toggle the grids of selected panels on or off with the Grid control button.

Default bindkey: **G**.

Feedback

The following grid settings are available from the Grid menu:

- All Grid On
- H-Grid Only
- V-Grid Only
- All Grid Off
- User Settings

All Grid On

To display the full grid, click **Axis** on the **Waveform** tab, then click **Grid**, and choose **All Grid On** from the menu that opens.

H-Grid Only

To display only the horizontal grid, click **Axis** on the **Waveform** tab, then click **Grid**, and choose **H-Grid Only** from the menu that opens.

V-Grid Only

To display only the vertical grid, click **Axis** on the **Waveform** tab, then click **Grid**, and choose **V-Grid Only** from the menu that opens.

All Grid Off

To hide the grid display, click **Axis** on the **Waveform** tab, then click **Grid**, and choose **All Grid Off** from the menu that opens.

User Settings

To define the grid parameters of XY panels, click **Axis** on the **Waveform** tab, then click **Grid**, and choose **User Settings** from the menu that opens.

User-defined grids start from the Start value and repeat over the Step interval in both directions away from the startpoint.

Zoom

Zoom operations are supported in the following panel types:

- X/Y zoom in X-Y, 2D-Sweep and Eye-diagram panels
- X zoom in logic panels
- Radial zoom in Polar Coordinate panels

Feedback

Depending on the default action type of the left mouse button in a waveview (the default left mouse button action can be specified in **Left Button Default** in the **Preference Settings** window), the zoom operation is invoked with:

- Cursor mode: You can invoke the zoom operation by first arming a waveview into the zoom mode. Click the **zoom** control button to activate the zoom mode.
- Zoom mode: The zoom operation automatically starts when you hold down and drag the left mouse button inside a waveform.
- Default Bindkey: Press the **Z** key to arm the zoom mode and release it to unarm.

If the left mouse button is defaulted to cursor operations (see Setting Key Bindings (Shortcut Keys)), continue clicking the zoom control button, which cycles the zoom mode in the following order: X/Y zoom ( ), X zoom ( ), Y zoom ( ), no zoom ( ).

The current zoom mode is indicated with the following icons:



where:

- **Zoom Mode**  — Click this zoom mode icon to cycle through the three zoom modes:
  -  X/Y zoom

    Default Bindkey: Press the **Z** key to arm the X/Y zoom operation. X/Y zoom is indicated by the red square around the zoom mode icon.

    If you arm the X/Y-zoom operation of a waveview with the **Z** bindkey, releasing the **Z** bindkey without any zoom action results in a 2x zoom out. The same 2x zoom out action can also be invoked using the **Zoom Out** control button.
  -  X zoom

    Default Bindkey: Press the **H** key to arm the horizontal zoom operation. X zoom is indicated by red vertical lines around the zoom mode, zoom out, zoom in, and zoom to fit icons.

    If you arm the X-zoom operation of a waveview with the **H** bindkey, releasing the **H** bindkey without any zoom action results in a 2x x-axis zoom out.
  -  Y zoom

Feedback

Default Bindkey: Press the **V** key to arm the vertical zoom operation. Y zoom is indicated by red horizontal lines around the zoom mode, zoom out, zoom in, and zoom to fit icons.

If you arm the Y-zoom operation of a waveview with the **V** bindkey, releasing the **V** bindkey without any zoom action results in a 2x y-axis zoom out.

- **Zoom In**  — Click to zoom in on all selected panels (or all panels, if none are selected). Default zoom in amount is 50%.
- **Zoom Out**  — Click to zoom out on all selected panels (or all panels, if none are selected). Default zoom out amount is 2x.
- **Zoom to fit**  — Click to restore full page plotting on selected panels (or all panels, if none are selected). The zoom to fit is applied to the selected panel axes. If no panel is selected, the zoom to fit is applied to all panels.
- **Undo Zoom**  and **Redo Zoom**  — Click to traverse up and down a zoom sequence, undoing or redoing a zoom step, respectively. You can **right-click** and select **Remove Icon** to remove these icons from the toolbar.

  Use the undo-zoom function to undo a zoom operation. Default Bindkey: **U**

  Use the redo zoom function to redo a zoom operation. Default Bindkey: **R**

Setting Zoom Ratio

The preference **Zoom In Ratio %** allows you to select the % for zooming in and out (out being the inverse).

Set Zoom Area

To specify the the x-axis and y-axis zoom range operations manually, click **Axis** on the **Waveform** tab, then click **Zoom**, and choose **Set Zoom Area** from the menu that opens. Enter the zoom range parameters, and click **Ok** to set the zoom level on all panels or selected panels of the active waveview.

Note:

Y ranges are in db (or db10) units if the target panel y-axis is in the db (or db10) scale

Full UnZoom

To unzoom the display, click **Axis** on the **Waveform** tab, then click **Zoom**, and choose **Full Unzoom** from the menu that opens.

Default bindkey: **X**

Feedback

Using Sliders in Zoomed Panels

In a zoomed panel (X-Y, logic or 2D-Sweep), the zoomed area can be interactively shifted using the horizontal and vertical slide bars. The waveview is redrawn at real time when the slide bar is moved. This graphic intensive operation can become noticeably slow especially when you work from a remote host.

The real-time screen redraw can be disabled for the slide bar to avoid the slow graphic operation. To disable real-time redraw, disable the **Real-time waveview scroll** option in the **Preference Settings** window (click **Settings** on the **Configuration** tab). With the real-time waveview scroll disabled, only the grid rulers are interactively redrawn during slide bar movement.

XAxis

The following x-axis types are available from the **XAxis** menu:

- Linear
- Log10
- dB20

Linear

To set a linear x-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **XAxis**, and choose **Linear** from the menu that opens.

Log10

To set a log10 x-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **XAxis**, and choose **Log10** from the menu that opens.

dB20

To set a dB20 x-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **XAxis**, and choose **dB20** from the menu that opens.

YAxis

The following y-axis types are available from the YAxis menu:

- Linear
- Log10
- dB20
- dB10
- Y2 - Linear
- Y2 - Log10

Feedback

- Y2 - dB20
- Y2 - dB10
- Y2 - Phase

Linear

To set a linear y-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **YAxis**, and choose **Linear** from the menu that opens.

Log10

To set a log10 y-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **YAxis**, and choose **Log10** from the menu that opens.

dB20

To set a dB20 y-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **YAxis**, and choose **dB20** from the menu that opens.

dB10

To set a dB10 y-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **YAxis**, and choose **dB10** from the menu that opens.

Y2 - Linear

To set a Y2 - linear y-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **YAxis**, and choose **Y2 - Linear** from the menu that opens.

Y2 - Log10

To set a Y2 - log10 y-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **YAxis**, and choose **Y2 - Log10** from the menu that opens.

Y2 - dB20

To set a Y2 - dB20 y-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **YAxis**, and choose **Y2 - dB20** from the menu that opens.

Y2 - dB10

To set a Y2 - dB10 y-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **YAxis**, and choose **Y2 - dB10** from the menu that opens.

Y2 - Phase

To set a Y2 - Phase y-axis for a plotted signal, click **Axis** on the **Waveform** tab, then click **YAxis**, and choose **Y2 - Phase** from the menu that opens.

Feedback

Font

The following font sizes are available from the Font menu:

- Normal
- Large

Normal

To set the axis font size to the default normal size, click **Axis** on the **Waveform** tab, then click **Font**, and choose **Normal** from the menu that opens.

Large

To set the axis font size to a larger size, click **Axis** on the **Waveform** tab, then click **Font**, and choose **Large** from the menu that opens.

Full Scale

The following axis scales are available from the Full Scale menu:

- Set X Full Scale
- Set Y Full Scale
- Default Y Full Scale

Set X Full Scale

To set a fixed plotting range for the x-axis, click **Axis** on the **Waveform** tab, then click **Full Scale**, and choose **Set X Full Scale** from the menu that opens.

Set Y Full Scale

To set a fixed plotting range for the y-axis, click **Axis** on the **Waveform** tab, then click **Full Scale**, and choose **Set Y Full Scale** from the menu that opens.

Default Y Full Scale

Fixed plotting ranges can be assigned to the x-axis of X-Y panels. Click **Axis** on the **Waveform** tab, then click **Full Scale**, and choose **Default Y Full Scale** from the menu that opens. The **Default Y Axis Range** dialog box opens. Click **Ok** to apply the change to selected panels of the active waveview.

X Variable

To set an X variable:

1. Click **Axis** on the **Waveform** tab, and click **X Variable**.

   The **Setting X Variable** dialog box opens withe the default X variable is enabled by default.

Feedback

2. With the dialog box open, right-click a signal in the **Waveform Browser** signal list window, and choose **Use as X Variable**. In case you do not want to use default X variable, deselect the **Use Default X Variable** option.

   Dragging a signal to the **X Variable** field, resets the **Parameter Variable** field to empty. Selecting a **Parameter Variable** from the menu, resets the **X Variable** field to empty. So, you can set only one field.

   When the **Change X Variable** dialog box is opened, right-click on signal name in the signal browser, the **Use a X Variable** menu appears.

   If you drag a new signal, the **X Variable** shows the new signal name replacing the old one.

   Note:

   The field is not cleared if **Use Default X Variable** is selected. This is so that if you switch back you do not have to set it again.

3. Click **Apply** to apply the change to an active vertical waveview or all the selected panels in an active horizontal waveview.

By default, signals in X-Y panels are plotted with respect to their internal X variables defined in the original waveform data file. The X variable can be replaced with a signal so that other signals are plotted as functions of the selected signal.

In a vertical waveview, if the waveview contains no logic panel, you can assign a signal as the common X variable so that all signals are plotted against the selected X variable. In a horizontal waveview, the same operation can be applied individually to all selected X-Y panels.

You can set the sweep parameter as the X variable on the X-Y panel. Envelopes are enabled on the X-Y panel for sweep signals.

To change the X variable, from the signal filter of the waveform browser (**Configuration** > **Browser**), select any signal. Right-click the mouse button and select **Use as X Variable** from the displayed options. This changes the X variable of the plotted waveform to the selected signal. Alternatively, the X variable can be changed by dragging and dropping the required signal to the x-axis of the waveview window.

Axis Attributes

To set axis attributes:

1. Double-click on an axis.

   The **Axis Attributes** window opens.

2. Choose an axis from the **Axis** menu.

   All axes in the active waveview window are available.

Feedback

3. Click one of the zoom level buttons to change the display:
   - **Zoom In** increases magnification to show detail.
   - **Zoom to Fit** displays the maximum number of data points to show the entire range of a signal. The entire range is determined by the minimum and maximum values entered in the **Full Scale Range** text boxes at the bottom of the **Axis Attributes** window.
4. Enter values for the zoom level in the **Min** and **Max** text boxes.

   These values specify the range you want to display.
5. (Optional) Click the **Use Waveform Range** check box to specify the range of a signal, and enter the Min and Max values for that range.
6. (Optional) Click the **Grid On** check box to display the grid for the selected axes.

   You can click the **Auto Grid** check box and enter **Start** and **Stop** values for the gridding. You can only turn this option off for linear axes.
7. Choose the axis label font size from the **Font** menu.

   The following font sizes and styles are available:
   - **Normal**
   - **Large**
   - **Gothic**
   - **Times**
   - **Roman**
   - **Courier**
8. Click **Apply** to save your changes.

Setting Signal Display Options

This section contains information on signal display settings for viewing panels:

- Full Logic Value
- Short Logic Value
- VCD Logic
- RTF Logic
- PWL Waveform

Feedback

- PWC Waveform
- Bar Chart
- Data Point Only

Full Logic Value

Logic vector values by default are displayed in their shortest possible form. To display vector values in full length, click **Display** in the **Panel** section of the **Waveform** tab, and choose **Full Logic Value** from the menu that opens.

Short Logic Value

To display vector values in short length, click **Display** in the **Panel** section of the **Waveform** tab, and choose **Short Logic Value** from the menu that opens.

VCD Logic

To plot a signal as a VCD logic waveform, click **Display** in the **Panel** section of the **Waveform** tab, and choose **VCD Logic** from the menu that opens.

RTF Logic

To plot a signal as an RTF logic waveform, click **Display** in the **Panel** section of the **Waveform** tab, and choose **RTF Logic** from the menu that opens.

PWL Waveform

To plot a signal in piecewise linear format, click **Display** in the **Panel** section of the **Waveform** tab, and choose **PWL Waveform** from the menu that opens.

PWC Waveform

To plot a signal in piecewise-constant format, click **Display** in the **Panel** section of the **Waveform** tab, and choose **PWC Waveform** from the menu that opens.

Bar Chart

To plot a signal as a bar chart, click **Display** in the **Panel** section of the **Waveform** tab, and choose **Bar Chart** from the menu that opens.

Data Point Only

Waveforms are displayed by plotting piece-wise-linear (PWL) lines between data points. To display the actual data points, click **Display** in the **Panel** section of the **Waveform** tab, and choose **Data Point Only** from the menu that opens to toggle the data point markers on or off.

Default bindkey: **D**.

Feedback

Setting Radix Options

To select a radix setting for all panels, click **Radix** in the **Panel** section of the **Waveform** tab, and choose one of the following supported radix settings:

- **Binary**
- **Octal**
- **Decimal**
- **Signed**
- **Hexadecimal**
- **ASCII**
- **Mnemonic on/off**

Logic vector values by default are displayed in binary. You can also choose **Radix** from the panel context menu to select a radix setting for the panel in action.

Setting Signal Name Options

This section contains information on items listed in the **SigName** menu, which is located in the **Panel** section on the **Waveform** tab:

- Sort
- Leafname
- Fullname
- Fullname+file
- Fullname+path+file
- Netname
- Netname+file
- Netname+path+file
- Alias

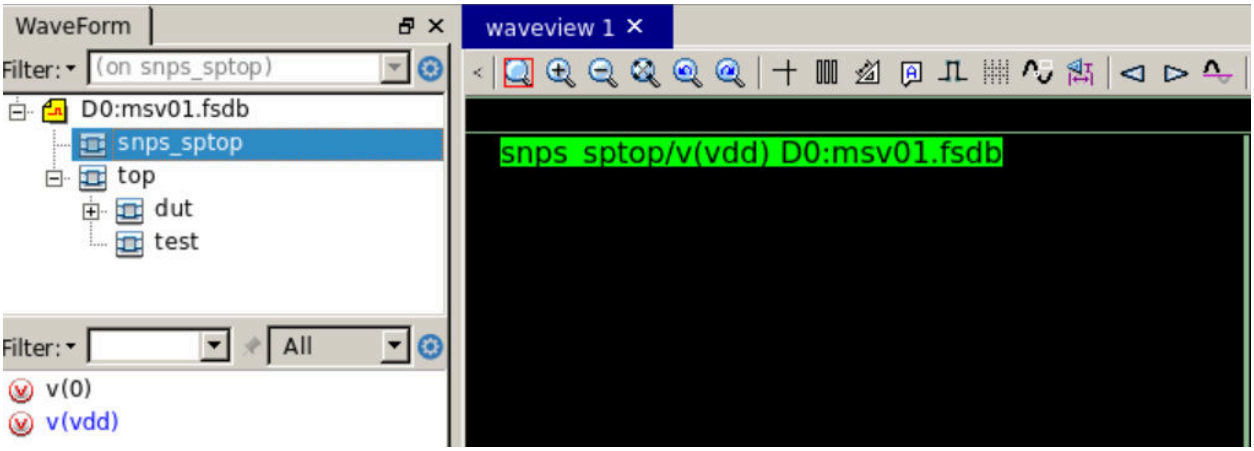
Sort

To sort signal names alphabetically, choose **Sort** from the **SigName** menu in the **Panel** section of the **Waveform** tab.

Feedback

Leafname

Leafname is the netname along with the signal type. For example, in the following image, if the full name of a signal is snps_sptop/v(vdd), its leaf name is v(vdd).



To display the leaf names for signals, choose **Leafname** from the **SigName** menu in the **Panel** section of the **Waveform** tab.

Fullname

To display the full names for signals, choose **Fullname** from the **SigName** menu in the **Panel** section of the **Waveform** tab.

For example, snps_stop/v(vdd).

Fullname+file

To display the full names and file names for signals, choose **Fullname+file** from the **SigName** menu in the **Panel** section of the **Waveform** tab.

For example, snps_stop/v(vdd) mvs01.fsdb.

Fullname+path+file

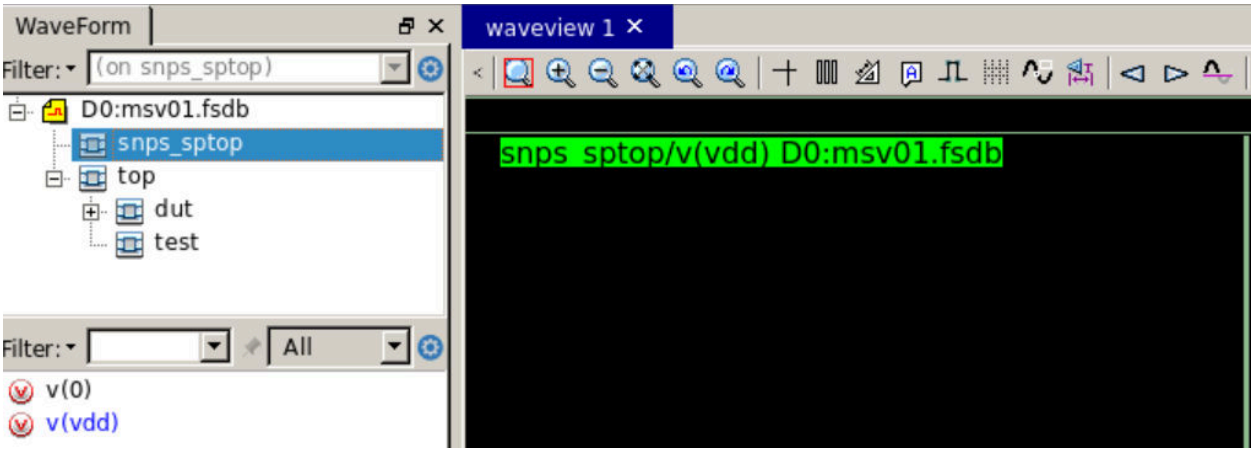
To display the full names, file names, and paths for signals, choose **Fullname+path+file** from the **SigName** menu in the **Panel** section of the **Waveform** tab.

For example, snps_stop/v(vdd) D0:mvs01.fsdb.

Netname

Netname is the name of the wire or net in the schematic. For example, in the following image, if the full name of a signal is snps_sptop/v(vdd), its net name is vdd.

Feedback



To display the net names for signals, choose **Netname** from the **SigName** menu in the **Panel** section of the **Waveform** tab.

Netname+file

To display the nets names and file names for signals, choose **Netname+file** from the **SigName** menu in the **Panel** section of the **Waveform** tab.

For example, vdd mvs01.fsdb.

Netname+path+file

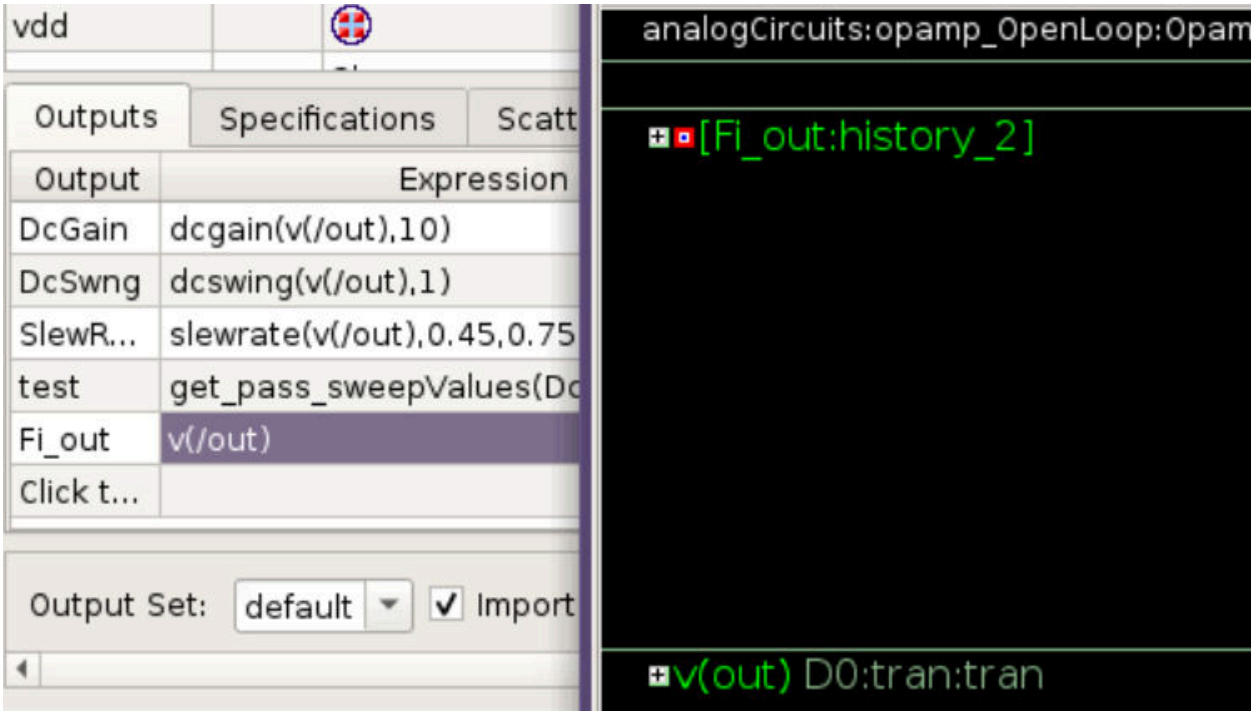
To display the net names, file names, and paths for signals, choose **Netname+path+file** from the **SigName** menu in the **Panel** section of the **Waveform** tab.

For example, vdd D0:mvs01.fsdb.

Alias

In PrimeWave, the expression name in the **Outputs** tab is taken as the signal alias name. For example, in the following image, if Fi_out is the expression name entered for a signal v(/out) in the PrimeWave **Outputs** tab, then the alias name for the v(/out) signal is taken as Fi_out. The alias name can also be edited and changed as required.

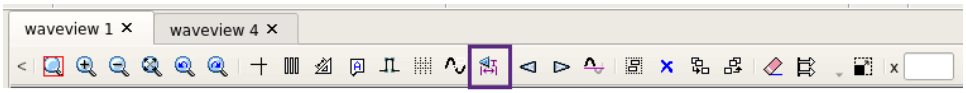
Feedback



To display the aliases for signals, choose **Alias** from the **SigName** menu in the **Panel** section of the **Waveform** tab.

Fit Longest Label

To fit the signal name width to the longest signal label, right-click on the canvas, choose **Signal Name > Fit Longest Label**. A shortcut icon for this is available in the toolbar



Fit Longest Label

Adjusting Panel Height

This section contains information on items listed in the **Height** menu, which is located in the **Panel** section on the **Waveform** tab:

- Fit
- Default
- Fit All

Feedback

- Default All
- Manual Adjust

Fit

In a vertical stack waveview, the height of selected panels can be automatically adjusted to fit to the full window height. To fit the full window height, click **Height** in the **Panel** section of the **Waveform** tab, and choose **Fit** from the menu that opens.

You can also use the default bindkey **T** to toggle selected panels between fit and unfit modes.

If the selected panels are separated into multiple groups, each group is fitted respectively. If there are too many panels in a group and the panel height becomes too small, the height is limited to the user-defined minimum height.

Default

To restore the default window height, click **Height** in the **Panel** section of the **Waveform** tab, and choose **Default** from the menu that opens.

Panel heights in a vertical stack waveview are calculated automatically based on the following preference settings:

- Maximum number of visible panels
- Minimum X-Y panel height

You can adjust the height of a panel by dragging the bottom edge of the left y-axis area. If the panel in action is a selected panel, the height of all selected panels is changed.

Individual panel height adjustment is available for analog panels only. To adjust the height of all logic panels, change the value of Logic Panel Height in the preference settings.

Fit All

In a vertical stack waveview, the height of selected panels can be automatically adjusted to fit to the full window height. To fit the full window height for all windows, click **Height** in the **Panel** section of the **Waveform** tab, and choose **Fit All** from the menu that opens.

You can also use the default bindkey **T** to toggle selected panels between fit and unfit modes.

If the selected panels are separated into multiple groups, each group is fitted respectively. If there are too many panels in a group and the panel height becomes too small, the height is limited to the user-defined minimum height.

Feedback

Default All

To restore the default window height for all windows, click **Height** in the **Panel** section of the **Waveform** tab, and choose **Default All** from the menu that opens.

Panel heights in a vertical stack waveview are calculated automatically based on the following preference settings:

- Maximum number of visible panels
- Minimum X-Y panel height

You can adjust the height of a panel by dragging the bottom edge of the left y-axis area. If the panel in action is a selected panel, the height of all selected panels is changed.

Individual panel height adjustment is available for analog panels only. To adjust the height of all logic panels, change the value of Logic Panel Height in the preference settings.

Manual Adjust

To manually adjust the height for a windows, click **Height** in the **Panel** section of the **Waveform** tab, and choose **Manual Adjust** from the menu that opens.

Panel Types

This section contains information on the following topics:

- X-Y Panels
- Eye Diagrams
- Logic Waveforms
- Tabular Panels
- Histogram Panels
- Smith Chart
- Polar Plot Panels
- 2-D Sweeps
- 3-D Sweeps
- Pair Plot
- Constell Plot
- Panel Set

Feedback

- Parametric Plots
- Plotting Clock Jitter
- PrimeSim HSPICE QQ Plot

X-Y Panels

This section contains information on the following topics:

- Creating an X-Y Panel
- Plotting X-Y Panel Values in Tabular Format
- FFT/DFT Conversion
- A to D Conversion

Creating an X-Y Panel

To create a new X-Y panel, choose **X-Y** from the menu in the **Panel** section of the **Waveform** tab.

An X-Y panel is designed to display analog waveforms. It is the only panel type that is allowed in both vertical stack and row/column waveviews. The X-Y panel accepts signals that are represented in (valuex,valuey) pairs, ((x,y) data points, for example), where valuex can be the default sweeping variable of a signal, or the signal value of a regular signal; valuey can be either the value of a real- signal or the magnitude/phase/real/imaginary part of a complex signal. The zoom operation is supported along both axes, and the logarithmic scale is also supported along both axes.

Waveforms in linear X-Y panels can be displayed with different plotting options including PWL (piece-wise-linear) line-type, PWC (piece-wise-constant) line-type, bar chart and data-point only mode. The default line type is determined by the type of waveform.

Plotting X-Y Panel Values in Tabular Format

To plot the value pairs from an X-Y panel into tabular format, right-click anywhere in an X-Y panel, and choose **Plot in Tabular** from the menu that opens. A new tableview is created with all the plotted value pairs from the X-Y panel.

FFT/DFT Conversion

To open the FFT/DFT Parameters window, choose **FFT/DFT** from the menu located in the **Utilities** section of the **Waveform** tab.

FFT and DFT operations are used to convert time-domain signals into their frequency-domain spectrum representation. The FFT/DFT operations can be performed on waveforms in the X-Y panels only.

Feedback

FFT/DFT operations are supported when using various windowing functions. The FFT/DFT dialog box accepts input parameters and applies DFT/FFT to all signals in selected panels of the active waveview. If no panel is selected in the active waveview, all signals in the active waveview become the FFT/DFT target. If the active waveview has only one panel, the FFT/DFT operation is by default applied to all waveforms in the panel. If the active waveview has more than one panel, you need to select the FFT target panels.

The FFT/DFT algorithm requires the following parameters:

- The number of sample points: Determines the number of time-domain sample points used for FFT/DFT operation. This number is internally rounded to the next power of two that is greater than the user input for FFT. A slow run time warning is displayed for DFT operations over 1024 points.

  OR

  The sampling frequency: The frequency of the sampling signal for the FFT/DFT operation. The sampling rate must be greater than 1 hz.

  Note:

  If you have a defined time range, you need to specify either the number points or sampling rate. If you do not have a time range specified, the number of points and the sampling rate are used to determine the start and stop times.

- The sampling window start time: Determines the start time of the time-domain window for FFT/DFT conversion.
- The sampling window end time: Determines the end time of the time-domain window for FFT/DFT conversion. The specified window end time must be greater than its start time.

You must choose which three input parameters are in use. Deselect an input parameter using the toggle switch in front of the parameter. If the **Enable zero-padding after stop time** option is selected, all four parameters need to be entered, the sampling stop time is adjusted based on the other three input parameters. For example:

```
window_end_time < window_start_time + number_of_samples *
(1/sampling_frequency)
```

Evenly spaced zeros are padded between the user specified end time and the one calculated by the waveview.

Click the **Check** button to check the input parameters before a FFT/DFT operation. It automatically fills the unspecific value and modifies the end time if necessary when four sampling parameters are specified. Use the **Save** and **Load** buttons to save and load FFT input parameters respectively.

Feedback

To enter the start and stop time based on the frequency of the input signal, enter the input signal frequency in the Input Signal Frequency field and click **Apply** to calculate the start/stop time.

Window functions can be applied for the FFT/DFT operation. If a window function is selected, the waveform data is multiplied with the selected windowing function before the FFT/DFT operation. No window function is used if the **Rectangle** window type is selected. The gaussian and the kaiser-bessel window types require the additional **Alpha** parameter.

The FFT/DFT output can be normalized against the waveform DC component or a user-defined value in Magnitude Normalization. All harmonic components are displayed in reference to the reference.

If the **Suppress DC before FFT/DFT** option is selected, the dc component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

The Output Frequency range settings specify the range of FFT/DFT spectrums for display or text output. The fmin and fmax value define the output spectrum range of interest. Specifying **frequency of interest** only displays or outputs the multiple of this frequency harmonic; all other harmonics are omitted.

The FFT/DFT results can be displayed in a regular X-Y panel or in a spectrum panel. User can also send the results to either a text window or text file for further processing. Y-axis scale is automatically set as db for the FFT/DFT plot.

Select the **Display Sampled Waveform** option to display the time-domain sampled waveform, including the effect of the windowing function.

The FFT result can be displayed in a regular X-Y panel, or a special spectrum panel, or into text window or file. See Viewing the Spectrum Panel for more information.

This section contains information on the following topics:

- Viewing the Spectrum Panel
- Calculating SNR/THD Using FFT
- Calculating SNR/ENOB With Respect to Full-Scale
- Calculating the FFT of Complex Signals
- Changing the FFT Axes Scale

Viewing the Spectrum Panel

The spectrum panel displays spectrum magnitude of harmonic components at different frequencies. Choose **Plot Complex** from the context menu to change the value to the phase, real, or imaginary part of the complex data.

Feedback

The Spectrum panel uses bar chart by default to display waveforms. You can change it to a line chart by choosing **Configure Spectrum** from the panel context menu.

Cursors in the Spectrum panel report waveform values based on the current complex plot mode. Instead of using piece-wise-linear (PWL) interpolation, the piece-wise-constant (PWC) value is used within each spectrum interval.

Calculating SNR/THD Using FFT

To calculate and display the total harmonic distortion (THD), signal to noise ratio (SNR), signal to noise and distortion ratio (SNDR), and effective number of bits (ENB) of the target signals, select the **Evaluate SNR/THD/SNDR/ENB** option. The result is displayed in the text window or the text output file.

For SNR/THD/SNDR/ENOB calculation, windowing function is usually used if the sample range does not cover integer cycles of the input signal. However, the time-domain windowing functions also spread the original harmonics of the input signal spectrum into multiple spectrum components. To accommodate the spreading effect, an additional parameter (**Bin Size**) is required to describe how many spectral components on each side of a harmonic frequency need to be included as the power of the harmonic.

Specify the cut-off frequency for the noise components in the Band Limit field. This value affects the SNR/SNDR/ENOB calculation. Set to 0 if no band limit is used.

Note:

To include counting floor components when calculating SNR/SNDR, enable the **Count floor noise components in SNR/SNDR calculation** option, which is available in the Preferences (**Settings** on the **Configuration** tab).

To calculate SNR/THD/ENOB with respect to the ADC's full-scale range(dBFS), enable the **Calculate SNR/THD/ENOB with Respect to Full-Scale, Compatible with Analog Devices** option. See Calculating SNR/ENOB With Respect to Full-Scale for more information.

Calculating SNR/ENOB With Respect to Full-Scale

To calculate SNR/ENOB/SFDR/THD compatible with Analog Devices, harmonics of input signal fa fall at frequencies equal to |±Kfs ± nfa|, where n is the order of the harmonic,

K = 0, 1, 2, 3, ..., and max harmonic index is set to 5 by default.

The magnitude of fa is scaled to ADC full scale (1.0) when calculating SNR/SFDR, if not normalized. The formula of ENOB is changed to (SNR-1.76)/6.02. And SFDR is calculated with largest noise or harmonic distortion component.

Calculating the FFT of Complex Signals

If the input signal of FFT/DFT is a complex signal, a non-symmetric spectrum is generated. The **Suppress DC before FFT/DFT** option is ignored if the input signal is in

Feedback

complex values. To prepare a complex input signal from transient analysis (for RF-mixer applications), use the `complex()` equation builder function to combine two real signals into a complex signal.

Changing the FFT Axes Scale

To change the scale of the x- and y-axis:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.

2. Click the **Signals** category, and then select the desired scales in the **Default FFT Signal** for the **X-Scale** and **Y-Scale**

3. Click **OK** to save the preferences.

   These preferences are saved in the `.spxrc` preference file.

A to D Conversion

Analog waveforms can be converted to logic waveforms based on user-specified threshold values. The conversion procedure can be applied only to waveforms in the X-Y panels.

If the active waveview has only one panel, the A/D conversion operation is by default applied to all waveforms in this panel. If the active waveview has more than one panel, you need to select panels for the A/D conversion operation.

To open the **A/D Conversion Parameters** window, choose **A to D** from the menu located in the **Utilities** section of the **Waveform** tab.

You can enable the **Suffix of Result Signal Name** option, which adds a suffix to the name of the converted signal name for any time of A to D conversion.

This section contains information on the following topics:

- Single-Bit A/D Conversion
- Multi-Bit A/D Conversion
- Creating Bus Signals for Analog Signals

Single-Bit A/D Conversion

Analog waveforms can be converted to single-bit logic waveforms in one of two ways:

- With a single threshold defined, data points above the threshold level are converted to logic H, and those below to logic L.
- With two thresholds defined, all points above the high threshold level are converted to logic H, while all points below the low threshold level are converted to logic L. All other points in between are converted to logic X.

Feedback

The converted waveforms are displayed in logic panels. Since logic panels are allowed only in vertical waveviews, a new vertical waveview is automatically created if the current active waveview is in horizontal mode.

The two-threshold conversion requires that a signal stays long enough in the X region (region between the high and low thresholds) to generate an X state in the converted logic waveform. The minimum duration requirement is defined by the rise (fall) slew if the signal level moves from below (above) the low (high) threshold level. The rise and fall slews are defined using the d2a_rise_slew and d2a_fall_slew configuration options.

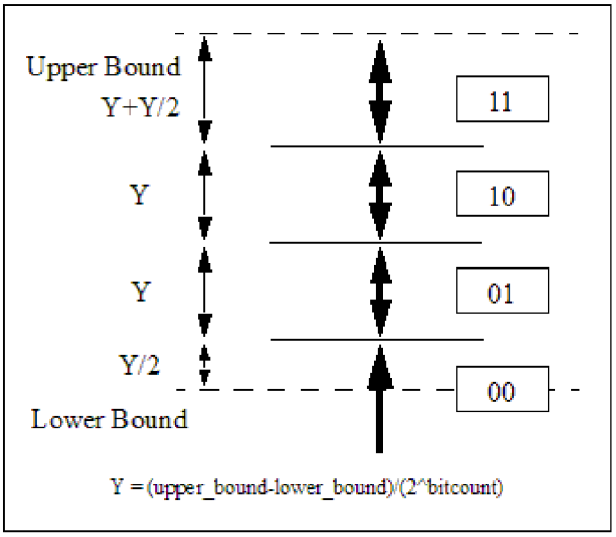
You can specify the default settings for A to D conversions in the **Threshold** category of the **Setting Preferences** window (click **Settings** in the **Configuration** tab).

Multi-Bit A/D Conversion

With multi-bit A/D conversion, analog waveforms are converted to multi-bit logic signals. The quantization thresholds are determined automatically based on the number of bits and user-specified lower/upper bounds.

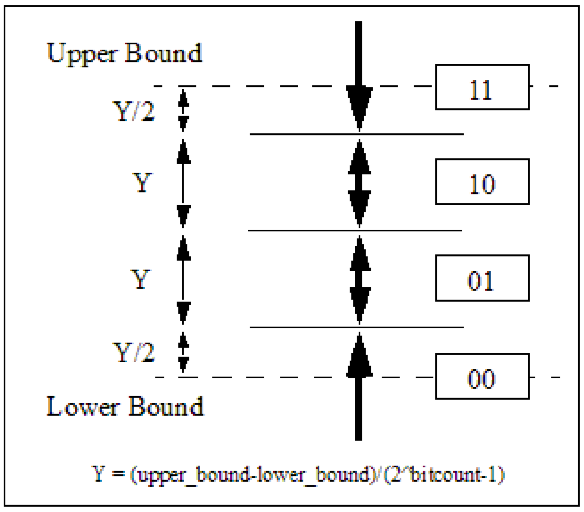
Two quantization threshold schemes are available:

- Y = (upper_bound - lower_bound)/(2^bit_counnt): Maintains asymmetric partitions as (Y+Y/2) for the largest codeword and Y/2 for the smallest codeword.



Feedback

- Y = (upper_bound - lower_bound)/(2^bit_count -1): Maintains a symmetric partition as Y/2 for both the largest and smallest codewords. This option is selected by default.



Creating Bus Signals for Analog Signals

To create bus signals for analog signals when converting from analog to digital, click the **Create Bus with Selected Analog Signals** check box at the bottom of the A/D Conversion Parameters dialog box as part of your conversion setup.

Eye Diagrams

To create an empty Eye diagram panel, choose **Eye Diagram** from the menu in the **Panel** section of the **Waveform** tab.

Signals are then dragged from the Waveform Browser and dropped into the Eye diagram. The Eye diagram by default displays signals in the folded mode.

A folded Eye diagram partitions signals into continuous intervals that are then overlapped to form the Eye diagram. The interval width can be user-specified or automatically determined.

Zoom operations are supported in both folded and unfolded eye diagrams.

The shift parameter is used to control the alignment of a folded Eye diagram. A slider at the lower-left corner of the Eye diagram panel allows users to interactively change the shift setting.

This section contains information on the following topics:

- Showing Eye Diagram Data Points
- Displaying the Envelope Center

Feedback

- Unfolding Eye Diagrams
- Tracing Waveform Points in Eye Diagrams
- Configuring Eye Diagrams
- Generating Jitter Histograms
- Displaying the Eye Height Distribution
- Adding User-Defined Masks

Showing Eye Diagram Data Points

To display the data points of an eye diagram, right-click an eye diagram, and choose **Show Data Points** from the menu that opens.

Displaying the Envelope Center

To display the envelope center of an eye diagram, right-click an eye diagram, and choose **Show Envelope Center** from the menu that opens.

After it is selected, the center trace is displayed along with the eye diagram. To disable this option, choose it again.

Unfolding Eye Diagrams

An Eye diagram can be unfolded by right-clicking an eye and choosing **Unfold Eye** from the menu that opens. The unfolded Eye diagram displays the original signals and the corresponding intervals highlighted in the red box at the top.

Similar to an oscilloscope, the Eye diagram also accepts a signal as an external trigger to construct the folded Eye diagram. To specify the external trigger signal, drag the signal from the Waveform Browser and drop it into the lower-right corner of an Eye diagram panel. A hint box highlights the drop location for the external trigger signal.

Tracing Waveform Points in Eye Diagrams

You can use cursors in an eye diagram to trace a waveform point location in the unfolded eye. Right-click an unfolded eye, and choose **Trace** from the menu that opens for the corresponding eye diagram location. You can trace **Eye Fuss**, **upper outer**, **upper inner**, **lower outer**, **lower, inner**, **left outer**, **left inner**, **right outer**, **right inner**, **upper margin inner**, **lower margin inner**, **upper margin outer**, or and **lower margin outer**. For an eye diagram that shows an eye mask, additional trace options **upper mask margin inner**, **lower mask margin inner**, **upper mask margin outer** or **lower mask margin outer** are available. Cursors are jumped to the corresponding location when the eye is unfolded.

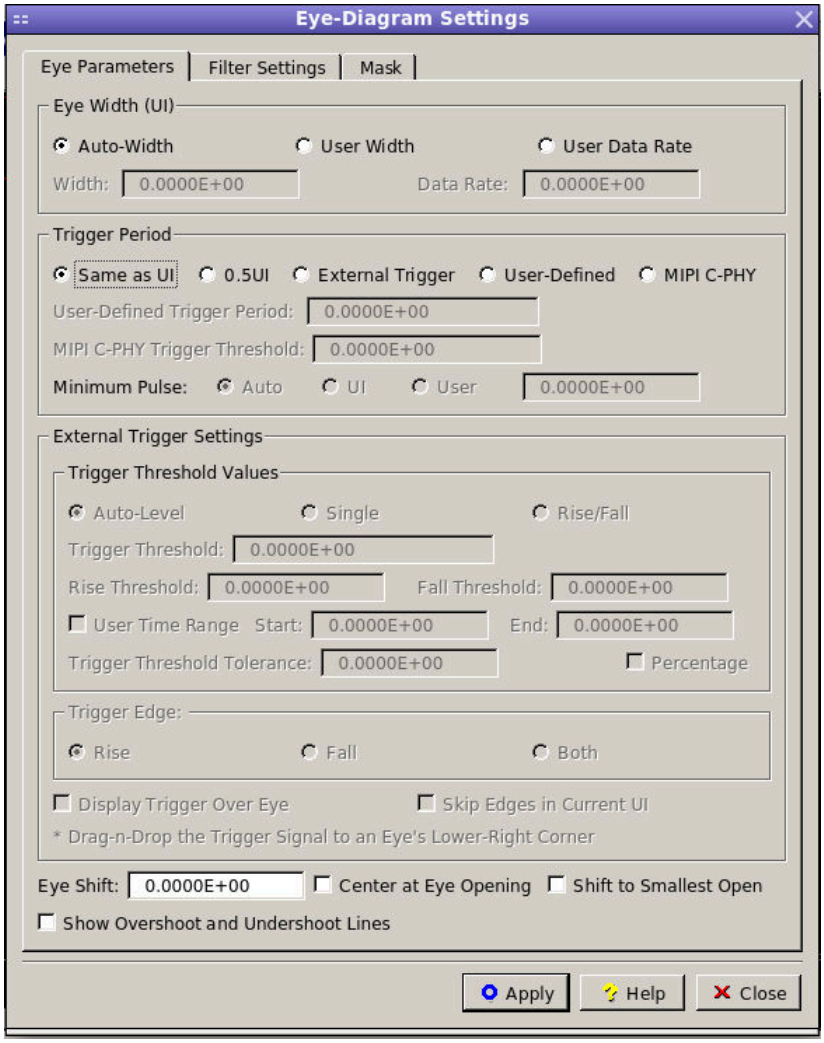
Feedback

Configuring Eye Diagrams

You can change eye diagram parameters from the **Eye Diagram Settings** dialog box. Right-click an eye diagram, and choose **Configure Eye** from the menu that opens. The **Eye Diagram Settings** window opens.

Note:

If you are configuring eye diagram settings for a stateye, only the settings on the Mask tab are visible. See Setting Mask Parameters for more information.



Eye diagram settings can be copied and pasted between multiple eye diagrams. Right-click an eye diagram, and choose either **Copy Settings** or **Paste Settings** from the menu that opens.

Feedback

The following sections explain the eye diagram settings that are available:

- Setting Eye Parameters
- Setting Filter Parameters
- Setting Mask Parameters
- Setting External Trigger Parameters

Setting Eye Parameters

The following parameters are set from the **Eye Parameters** tab in the **Eye Diagram Settings** window:

Parameter Name	Description
Auto-Width	Automatically determines the eye width or frequency. In Auto-Width mode, an eye diagram always uses the first signal in the panel to determine the eye width.
User Width	Specifies the eye width or frequency using the value you enter in the Width text box.
User Data Rate	Specifies the data rate using the value you enter in the User Data Rate text box.
Same as UI	Sets the eye trigger period to the same as the eye diagram UI width.
0.5UI	Sets the eye trigger period to 0.5UI.
External Trigger	Sets an external trigger using the settings in the External Trigger Settings section of the Eye Diagram Settings window. See the following table for the External Trigger Settings.
User-defined	Sets the eye trigger period to the value your enter in the **User-Defined Trigger Period** text box.

Feedback

Parameter Name	Description
MIPI C-PHY	Sets the MIPI C-PHY trigger period to the value your enter in **MIPI C-PHY Trigger Threshold** text box. Default: 0 **Minimum Pulse**: Select **Auto**, **UI (default)**, or **User**. If **Auto** is selected, it calculate minimum pulse among all three bits. Searching of the next trigger will skip 0.5*min_pulse after each trigger is found. The minimum pulse setting is saved in session file. The ACE command for the minimum pulse: `-min_pulse_mode=[AUTO | UI | USER]` and `-min_pulse=[user_min_pulse_value]` For example: `set eye_obj [ sx_create_eye $eye_sigs ui=2.22222e-10 phase=-1.11111e-10 trigger=CPHY cphy_vref=2.22222e-10 min_pulse_mode=USER min_pulse=2e-10 trig_level=auto trig_tolerance=0 ttpercentage=1 trig_edge=both start_t=4e-09 stop_t=7.6e-08 ]`
Eye Shift	Sets the eye shift value
Center at Eye Opening	Centers the eye diagram at the eye opening.
Show Overshoot and Undershoot Lines	Displays the overshoot and undershoot lines with eye diagrams.

The following Eye Parameter settings are available in the External Trigger Settings section when you choose the **External Trigger** option for the trigger period:

Parameter Name	Description
Auto-Level	Uses the average magnitude of the external trigger signal as the trigger level. The value in **Trigger Threshold Tolerance** field specifies the edge height tolerance (the default is `0.0000E+00`). Click the **Percentage** check box to change the tolerance to a percentage value. When enabled, this option calculates the percentage as `tolerance = value*0.01*(ymax - ymin)`. Edges with heights smaller than the tolerance are eliminated.
Single	Uses a single trigger value you enter in the **Trigger Threshold** text box. The value in **Trigger Threshold Tolerance** field specifies the edge height tolerance (the default is `0.0000E+00`). Click the **Percentage** check box to change the tolerance to a percentage value. When enabled, this option calculates the percentage as `tolerance = value*0.01*(ymax - ymin)`. Edges with heights smaller than the tolerance are eliminated.

Feedback

Parameter Name	Description
Rise/Fall	Uses the rising and falling trigger values you enter in the **Rise Threshold** and **Fall Threshold** text boxes, respectively. The value in **Trigger Threshold Tolerance** field specifies the edge height tolerance (the default is `0.0000E+00`). Click the **Percentage** check box to change the tolerance to a percentage value. When enabled, this option calculates the percentage as `tolerance = value*0.01*(ymax - ymin)`. Edges with heights smaller than the tolerance are eliminated.
Rise	Sets the trigger at the rising edge.
Fall	Sets the trigger at the falling edge.
Both	Sets the trigger at both the rising and falling edge.

Setting Filter Parameters

The following parameters are set from the Filter Settings tab in the Eye Diagram Settings window, which is used to exclude unwanted time periods when forming an eye diagram:

Parameter Name	Description
No Filter	Specifies that no filter is enabled.
Index Range	Enables a filter for a specified index range. Enter values for the **Starting Window Index** and **Number of Windows** to select the range.
Time Range	Enables a filter for a specified time range. Enter values for the **Range Starting Time** and **Range Stopping Time**.
Skip Frequency	Specifies the number of next trigger edges that are skipped. For example, if Skip Frequency is set to 2, the next 2 trigger edges will be skipped for a valid edge.
DDR 2/3T Mode	Enables or disables the DDR 2/3T mode. When enabled, if the trigger edge is bigger clock+0.5*period, the edge is skipped. For example: `set eye_obj [sx_create_eye $signal ui=auto trigger=ext trigsig=$strobe "option=ddr2t"]`
All Bit Patterns	Shows all bit patterns.
Remove 0->X	Filters out 0 to x transitions.
Remove 1->X	Filters out 1 to x transitions.

Feedback

Setting Mask Parameters

User-defined masks can be added using mask files.

The following parameters are set from the Mask tab in the Eye Diagram Settings window, which specifies a mask that can be superimposed over a folded eye diagram as a timing specification:

Parameter Name	Description
Calculate Mask	Select to calculate mask on eye diagram waveforms. Choose the mask type and color from the **Mask Type** and **Color** menus, respectively.
Mask Type	The mask type: **Standard** or **MIPI C-PHY**.
Color	The mask color.
Absolute Size Values	Enables absolute size values for the mask.
User-Defined Vertical Boundary Values	Uses the vertical boundary values you define
Show Mask on Top of Waveforms	Displays the mask on top of eye diagram waveforms.
Center Mask to Eye Opening	Centers a hexagon mask to the eye opening.
Show Worst Incursion (stateye only--requires a WaveView ADV license)	Displays the worst violation incursion on a stateye.
Show Voltage and Timing Margin (stateye only----requires a WaveView license)	Displays the voltage and timing margins at the mask corners.

Cursors in a folded eye diagram trace the eye fuss (eye height/width) or the envelops of the eye pattern. Select the required **Eye fuss** values and envelop measurements from the cursor context menu. If a cursor is selected to trace the eye envelope, the cursor is automatically moved to its corresponding unfolded position when the eye diagram is unfolded. For more details, see Tracing Waveform Points in Eye Diagrams.

Setting External Trigger Parameters

The **External Trigger** tab in the **Eye Diagram Settings** window specifies external trigger parameters only when external signals are set in the eye diagram panel.

Feedback

All signals and associated trigger values are displayed in the **Trigger Signals** table. You can change the trigger signal by dragging and dropping a new trigger into the table and clicking **Apply**.

Generating Jitter Histograms

Cursors are normally used in Eye Diagrams to trace the eye envelope or fuss. They can be also used to produce jitter histograms from the eye envelope pattern. To create the jitter histograms, move a cursor and position its horizontal hair at a preferred level. Right-click a cursor, and choose **View Histogram** from the menu that opens.

The time (the offset into a folded eye window) is automatically sampled for each cross-point between the horizontal hair and the eye waveform, and creates histograms for the two areas.

The generated histograms display the jitter distributions of the two cross section areas.

Displaying the Eye Height Distribution

To display the vertical distribution values of the height of an eye as a histogram over the eye diagram:

1. Right-click an eye diagram, and choose **Eye Height Distribution** from the menu that opens.

   The **Eye Height Distribution** dialog box opens.

2. Enter the **X Value**.

   The eye height distribution begins with this x-axis value.

3. Click **OK**.

   A histogram of vertical distribution values is displayed over the eye diagram.

Adding User-Defined Masks

A mask is an abstract reference overlaid in eye diagrams to qualify signals timing integrity. Two built-in masks are provided: a standard mask for common near-end timing specifications and the IEEE802.3 mask. You can add your own mask definitions using mask files as well.

Saving and Loading Masks

Use the **Save Mask** and **Load Mask** buttons of the **Mask** tab of the **Eye-Diagram Settings** dialog to save and load masks. This feature is supported for both standard and C-PHY masks.

Saved mask settings include all mask options shown on the **Mask** tab dialog. When saving a mask spec, you can choose to append the current mask setting to an existing file.

Feedback

Mask File Syntax

A mask file can contain one or multiple mask definitions. A mask consists of multiple segments that are lines plotted between adjacent points from a node list. Each mask definition is organized in the following format:

```
mask "maskname" direct|normalized

"nodename", X, Y

<newpath>

"nodename", X, Y

…
```

The `mask` keyword starts a new mask definition, which must be followed by a quoted mask name and then a type keyword (`direct` or `normalized`) that specifies the mask type. The words `mask`, `direct`, `normalized`, and `newpath` are reserved keywords.

Each node in the list consists of a quoted node name, the X value of the node, and the Y value of the node. For a direct type mask, `X` is the X value into an eye window, and `Y` is the Y value of the node. For a normalized type mask, the X and Y values denote a node position as fractions into the eye window. X=0 (Y=0) is on the left (bottom) window edge and X=1 (Y=1) is on the right (top) window edge.

A `newpath` keyword breaks line strokes and starts a new drawing stroke.

Incorporating Mask Files

The `SW_SX_MASKFILE` environment variable is used as a pointer to the user mask files.

To incorporate a mask file:

- On Linux Platforms

  ```
  setenv SW_SX_MASKFILE file1:file2:filen
  ```

  `file1…filen` are the path to the mask files. A column (:) is placed between file paths.

- On Windows 95/98 Platforms

  Open a MSDOS window and edit AUTOEXEC.BAT in the top directory of your boot hard-drive (C:\, for example). Add the following line to the AUTOEXEC.BAT file:

  ```
  SET SW_SX_MASKFILE=file1;file2;filen
  ```

  The file names are separated by semicolumn (;) instead of column (:) because Windows uses column (:) as a drive delimiter.

- On Windows NT Platforms

Feedback

Open the **Control Panel** and double-click the **System** icon. After you see the **System Properties** dialog box, select the **Environment Variables** tab. Add the `SW_SX_MASKFILE` environment variable.

- On Windows XP/2000 Platforms

  Open the **Control Panel** and double-click the **System** icon. After you see the **System Properties** dialog box, click the **Advanced** tab and click the **Environment Variables** button. Add the `SW_SX_MASKFILE` environment variable.

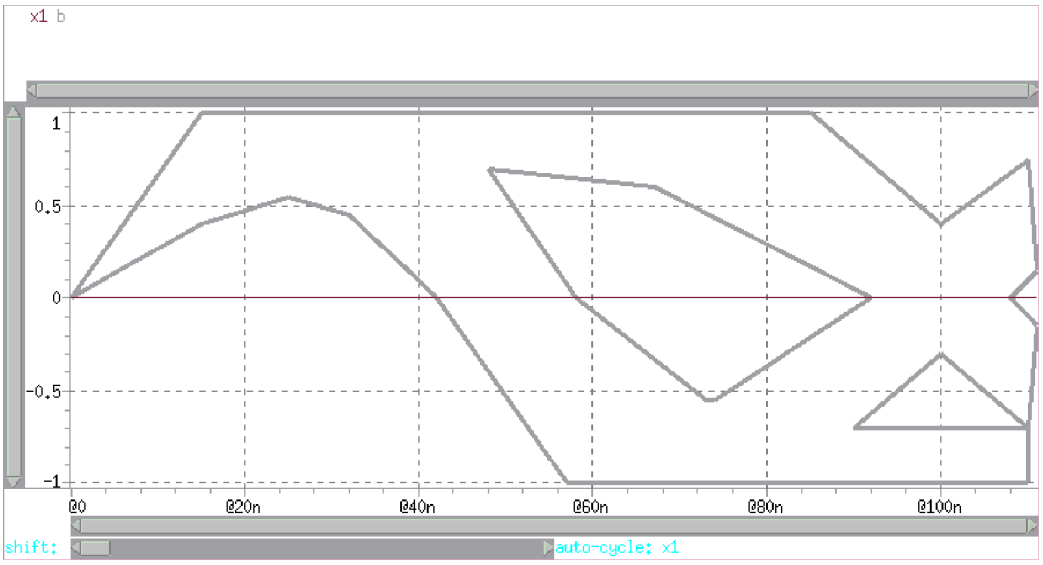
- On Windows Me Platforms

  Click **Start**, choose **Run**, then enter msconfig.exe and click **OK** to run the msconfig.exe program. Click the **Environment tab** and enter `SW_SX_MASKFILE` as a new environment variable. Make sure that the box before `SW_SX_MASKFILE` is checked to enable the environment variable.

  These mask files are read at initialization time and the mask menu is expanded in the Eye Configuration dialog box with the user-defined masks.

Mask File Example

The following mask example defines an IEEE802.3 mask as shown in the following figure.




```
mask "IEEE802.3" direct
"A",  0.0n, 0.0
"C", 15.0n, 0.4
"D", 25.0n, 0.55
"E", 32.0n, 0.45
"F", 42.0n, 0.0
"G", 57.0n,-1.0
"T",110.0n,-1.0
```

Feedback

```
"V",110.0n,-0.7
"S",111.0n,-0.15
"R",108.0n, 0.0
"Q",111.0n, 0.15
"P",110.0n, 0.75
"O",100.0n, 0.4
"N", 85.0n, 1.0
"B", 15.0n, 1.0
"A",  0.0n, 0.0
newpath
"H", 48.0n, 0.7
"I", 67.0n, 0.6
"J", 92.0n, 0.0
"K", 74.0n,-0.55
"L", 73.0n,-0.55
"M", 58.0n, 0.0
"H", 48.0n, 0.7
newpath
"U",100.0n,-0.3
"W", 90.0n,-0.7
"V",110.0n,-0.7
"U",100.0n,-0.3
```

Logic Waveforms

A logic panel displays signals using digital timing diagram. It is permitted only in vertical stack waveviews. Logic panels accept signals that are represented in (time, logic) pairs. Various logic states including logic strength levels are supported. Zoom operation is supported along the x-axis direction.

Logic panels cannot be manually added from the application top menu. They are automatically created when logic signals are dropped into a vertical waveview.

Current vector width (number of bits) and radix setting of a logic panel are indicated in the left column of each panel (B: binary, O: octal, D: decimal, H: Hex-decimal, A: ASCII). The width digits are red color-coded for bus signals formed from originally separated bits, or yellow color-coded for a single bit extracted from an originally grouped bus signal.

Logic panels can be arbitrarily mixed with linear X-Y panels in a vertical stack waveview.

This section contains information on the following topics:

- Mnemonic Mapping for Bus Signals
- D to A Conversion

Mnemonic Mapping for Bus Signals

Bus values can be mapped to predefined text strings. You can provide a translation table in an external file.

Feedback

Syntax:

```
# comment lines start with '#'
value1 name1
value2 name2
…
```

The values are the bus values to be mapped.

The following formats are supported:

- 2400
- 0xAF32
- 'hAF32
- 'b1001_0100_0101
- 'd2400
- 'b110110111100
- 'o371

The names are the text strings to be mapped to. A text string must be quoted if the name contains spaces.

Wildcards are supported in Bus Mnemonic expressions. The ? character is a single bit for wildcards. It works only on ’b, ’o, and ’h.

For example:

```
2'b1?
3'o1?1
4'h1?1?
```

You can load the translation table by clicking **Settings** in the **Waveform** tab and then choosing **Bus Mnemonic Map** from the menu that opens. Once a translation table is loaded, click Radix in the Panel section of the Waveform tab, and choose **Mnemonic On/Off** from the menu that opens to toggle the mapping settings.

The mapped values displayed in digital panel area is filled with color. This color is based on color information in map file.

D to A Conversion

Logic waveforms can be converted into analog waveforms based on user-specified analog levels. D/A conversion can be applied only to waveforms in the logic panels.

Feedback

If the active waveview has only one panel, the D/A conversion operation is by default applied to all waveforms in this panel. If the active waveview has more than one panel, you need to select panels for the D/A conversion operation.

To open the **D/A Conversion Parameters** window, choose **D to A** from the menu located in the **Utilities** section of the **Waveform** tab.

Both single-bit and multi-bit D/A conversions are supported. Analog levels for multi-bit D/A conversion are determined based on rules. Enter the analog voltage levels for the low and high logic states.

There are two modes for D2A sampling: fixed rate sampling and value transition sampling. With the value transition sampling, the D-to-A conversion is completed only at the time when the input digital signal changes its value.

X and Z states can be converted to low or high voltage levels based on user preferences.

Click the **Extend to Full Display Range** button to extend the x-axis range to the current full x-axis display range for each converted analog waveforms, even if the original logic data has an x-axis range that is shorter than the full range.

You can also enable the **Suffix of Result Signal Name** option, which adds a suffix to the name of the converted signal name.

Rising and falling slews of the converted analog waveforms can be specified in the **Preference Settings** window (click **Settings** in the **Configuration** tab), or using the **d2a_rise_slew** and **d2a_fall_slew** configuration options.

Tabular Panels

To create a new Tabular panel, choose **Tabular** from the menu in the **Panel** section of the **Waveform** tab.

See Working With Tableviews for more information on how to add a tableview and work with tabular panels.

Histogram Panels

To create a histogram panel, an empty histogram panel must be created first by choosing **Histogram** from the menu in the **Panel** section of the **Waveform** tab.

Signals can then be dragged from the **Waveform Browser** and dropped into the histogram. Histogram panels can be placed only in horizontal waveviews.

A histogram plots an individual histogram for each signal in the panel. The average value and standard deviation can be annotated on top of each histogram and the number of bins and data bounds can be reconfigured. The WaveView tool can also calculate the

Feedback

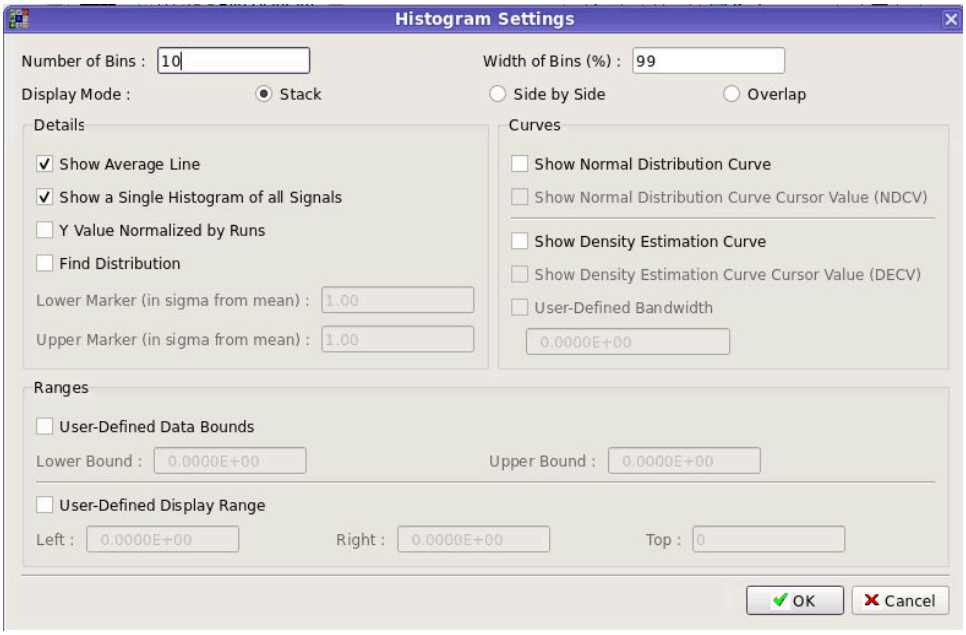
distribution percentages of regions divided by user-defined markers from the histogram mean center.

Configuring Histograms

To configure a histogram, right-click on the histogram panel and click **Configure Histogram** from the menu that opens. The **Histogram Settings** window opens.

Right-click on a histogram panel and click **Export Histogram Measures** to bring up a save file dialog for exporting the histogram measures. The m, s, RANGE, and BIN-Width values for the panel are saved in the specified file.

To change the maximum y-value in a histogram, open the **Histogram Settings** window and select the **User-Defined Display Range** check box. Enter the required maximum y-value in the **Display (Top)** field.



The following settings are available:

Setting	Description
Number of Bins	The number of bins to display in the histogram.
Width of Bins (%)	The width of bins that can be set from 1% to 100%. The default is 99%.

Feedback

Setting	Description
Display Mode	The modes in which the histogram can be displayed. The following three display modes are provided: • **Stack** - The data of signals is stacked and displayed in histogram bins. • **Side by Side** - The data of signals is arranged side by side in histogram bins. • **Overlap** - The data of signals is overlapped on each other in histogram bins.
Show Average Line	Enables the average bar. The histogram is plotted by default.
Show a Single Histogram of All Signals	Plots a single histogram of all signals in a panel.
Y Value Normalized by Runs	Normalizes the y-axis and sets it to the relative frequency (the number of occurrences divided by the number of Monte Carlo runs, for example).
Show Normal Distribution Curve	Displays the normal distribution curve over the histogram.
Show Density Estimation Curve	Displays the density estimation curve over the histogram.
Find Distribution	Displays the data distributions for the following specified range in percentage units: • [beginning, mean – lower_marker*sigma] • (mean – lower_marker*sigma, mean] • (mean, mean+upper_marker*sigma] • (mean+upper_marker*sigma, ending]
User-Defined Data Bounds	Specifies the data range to be displayed, which is displayed as the x-axis range in a histogram panel.
User-Defined Display Range	Specifies the display range of a histogram panel. It is defined by left, right, and top (max. y-value) ranges.

Smith Chart

Smith charts display complex normalized impedance, admittance, and scattering parameters (S-parameters). To create a Smith chart panel, choose **Smith Chart** from the menu in the **Panel** section of the **Waveform** tab.

Feedback

The Smith chart accepts signals that are represented in (real, complex) pairs. The normalized impedance is defaulted at 50 ohms and can be modified in Preference Settings (see the `smith_char_impedance` option in the `.spxrc` configuration file).

Smith charts are allowed only in horizontal waveviews.

You can configure a Smith chart by right-clicking a Smith chart and choosing **Attributes** from the menu that opens. The **Smith Chart Settings** dialog box opens.

Enter the required setting preferences and click **OK** to apply the changes.

Cursors (up to two) in a Smith chart can be used to trace the following value types:

- **Gamma** (the reflection coefficient)
- **SWR** (standing wave ratio)
- **Impedance** (Z)
- **Admittance** (Y)
- **Frequency**

You can change cursors by right-clicking on a Smith chart at the location where the cursor hairs cross, and one of these cursor types from the menu that opens

You can use the Data(X,Y) measurement to trace the frequency, magnitude, and phase values of a signal in a Smith chart.

The **Zoom in** and **Zoom out** buttons are used to zoom in and out a Smith chart. The **Full Unzoom** button displays a full-sized Smith chart.

Polar Plot Panels

To create a new polar plot panel, choose **Polar Plot** from the menu in the **Panel** section of the **Waveform** tab.

Polar plots display data on a complex plane. They are allowed only in horizontal waveviews. The Polar plot accepts signals that are represented in (real, complex) pairs. The zoom operation is supported only along the radial direction.

2-D Sweeps

To create a new 2-D sweep panel, choose **2-D Sweep** from the menu in the **Panel** section of the **Waveform** tab.

The 2-D Sweep panel displays 2-dimensional sweeping data on a flat X-Y plane. It is allowed only in horizontal waveviews. Similar to the 3-D Sweep panel, the 2-D Sweep panel accepts signals that are represented in ((x1, x2), value) pairs. Zoom operation is

Feedback

supported along both axes. Logarithmic scale is also supported along both axes (the x1 direction and the y direction, for example).

Both 2-D and 3-D sweep panels support sweep filters that display the selected sweep traces. Right-click a 2-D plot, and choose **Sweep Line Filter** from the menu that opens. You can highlight traces in the **Sweep Line Filter** dialog box that opens.

Enable the **Display Trace Value** option to display value label next to each sweep trace. If a 2D/3D sweep panel contains more than one signal, the first signal or the first highlighted signal is used as the filter target.

In 2-D sweep plot, the min/max Y-value bounds and the average Y-value of a multi-trace signal can be automatically extracted (for example, waveform envelop from statistical simulation result). To invoke this function, right-click a 2-D sweep panel, and choose **Find Envelope Bound** from the menu that opens. All data points are scanned from all traces, and three new waveforms representing the min/max bounds and the average waveform are created.

Displaying Multi-trace Waveforms

A multi-trace signal contains multiple waveform traces. Each trace has a set of associated sweeping parameters (or sub X-variables). For example:

```
Trace 1: Temp=0 Vdd=3.3
Trace 2: Temp=0 Vdd=5
Trace 3: Temp=100 Vdd=3.3
Trace 3: Temp=100 Vdd=5
```

By default, a multi-trace signal is plotted with respect to its main independent sweeping variable (time, for example). In this case, the sweep line filter function in a 2D-sweep panel can used to selectively display some of the traces.

The 2D-sweep panel also supports plotting a multi-trace signal with respect to one of its sweeping parameters. To change the default X-axis of a 2D-sweep panel to a sub X-variable, right-click a 2-D panel, and choose **Change X Variable** from the menu that opens. Drag and drop a sweeping parameter from the Waveform Browser to the **Setting X Variable** dialog box. Choose **Sweep Line Filter** from the panel context menu to open the Sub X-Variable Plotting Filter dialog box.

Note:

The filter setup is different between using the default x-axis and sub X-Variable x-axis. This is because in the sub X-Variable plot mode, based on the selected sub X-variable, the number of traces might vary depending on the number of unique combinations among other sweeping parameters. The number of traces are automatically detected and sorted in the order of the sub X-Variable values to display waveforms.

Feedback

A multi-trace signal must have equal number of points in each of its original traces in order to be plotted in the sub X-variable mode.

Plotting a multi-trace signal with respect to another multi-trace signal is not supported.

3-D Sweeps

To create a new 3-D sweep panel, choose **3-D Sweep** from the menu in the **Panel** section of the **Waveform** tab.

The 3-D Sweep panel displays 2-dimensional sweeping data on a 3-dimension coordinate. It is allowed only in horizontal waveviews. The 3-D Sweep panel accepts signals that are represented in ((x1, x2), value) pairs, where x1 and x2 are the first and second independent sweeping variable, respectively, and value can be either the value of a real signal or the magnitude/phase/real/imaginary part of a complex signal. The viewing angles of the 3-dimension coordinate can be interactively adjusted from the two sliders in the panel. The zoom operation is not supported. Logarithmic scale is supported only along the vertical axis.

Pair Plot

To create a new pair plot, choose **Pair Plot** from the menu in the **Panel** section of the **Waveform** tab.

The pair plot displays measurement results in pairs for viewing Monte Carlo data correlation. It is allowed only in row/column waveviews.

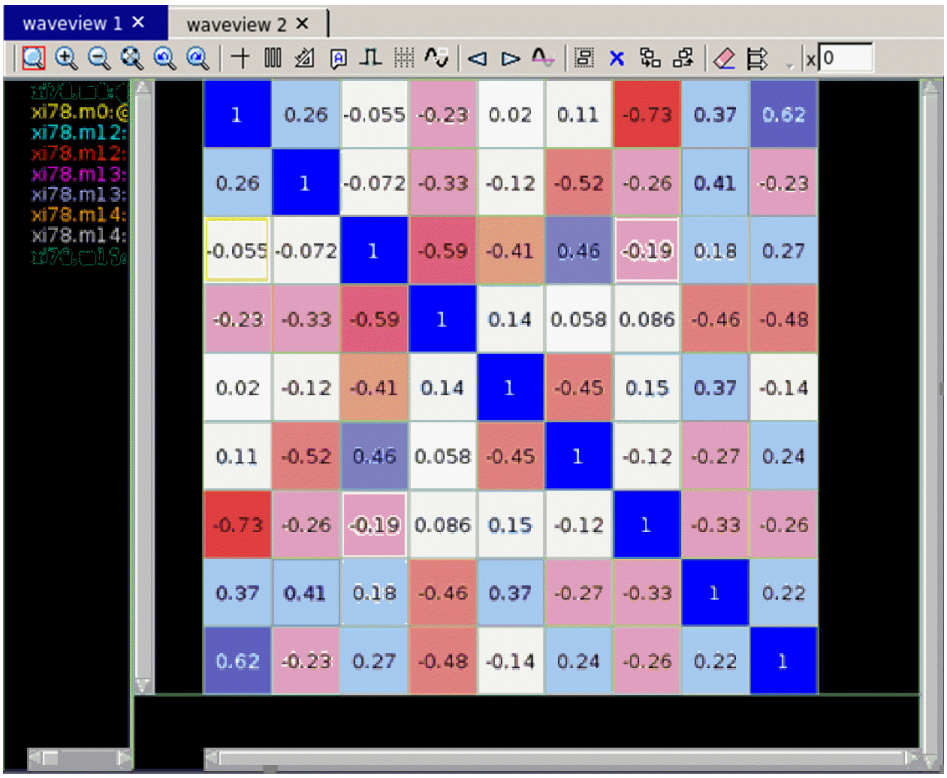
The WaveView tool creates a table with a number of rows and columns equal to the number of signals plotted. Each cell/square represents a pair of plotted signals. The signals start from top to bottom, left to right.

Correlation coefficient (R) values are calculated for each pair. Pairs with a value of 0 are shown as white squares. Pairs with a value of 1.0 are shown as blue squares, while values between 0 and 1.0 are a lighter blue, growing darker the closer it is to 1.0. Pairs with a value of -1.0 are shown as red squares, while values between 0 and -1.0 are a lighter red, growing darker the closer it is to -1.0.

If a lot of signals are plotted, the squares are smaller and only the color is shown. The zoom operation is supported.

Feedback

For example:



When hovering the mouse over signal name, a yellow rectangle highlights the row containing the plotted signal. When hovering over a square cell, the tool shows the R value in a box with canvas background color, as well as the signal names in the y-axis and x-axis areas. When clicking on a square cell, the tool shows a scatter or histogram plot in a separate panel for further analysis. A new panel is created on the first click and reused on each subsequent click.

You can also add a cursor on Pair Plot panel, the R values for each signal is shown in monitor area.

Constell Plot

This panel type supports optical and electrical constellations for OptSim simulations in OptoCompiler.

To create a new constellation, choose **Constell Plot** from the menu in the **Panel** section of the **Waveform** tab.

In this panel, you can plot regular, complex and sweep signals. The x-axis is the real part of the plotted signal and the y-axis is the imaginary part. You can also zoom in and zoom out in this panel.

Feedback

- Plotting regular signals

  For a regular non-complex signal, x-axis values are the signal values, and all y-axis values are equal to zero. The data points are drawn as markers in this panel.

- Plotting complex signals

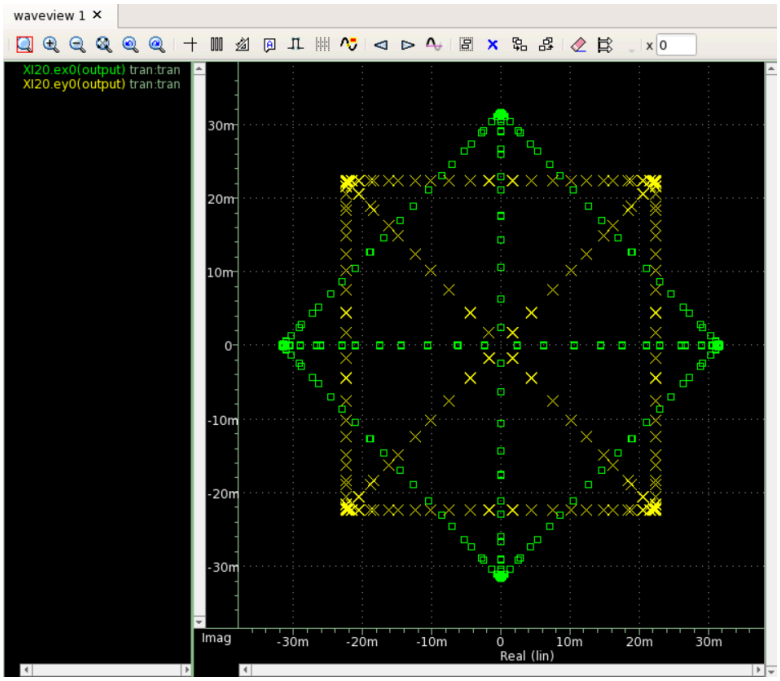
  For a complex signal, x-axis values are the real part of the signal and y-axis values are the imaginary part.

- Plotting sweep signals

  For a sweep signal, each trace of the sweep signal is plotted as a regular/complex signal in this panel.

Figure 2 shows an example of a constell plot.

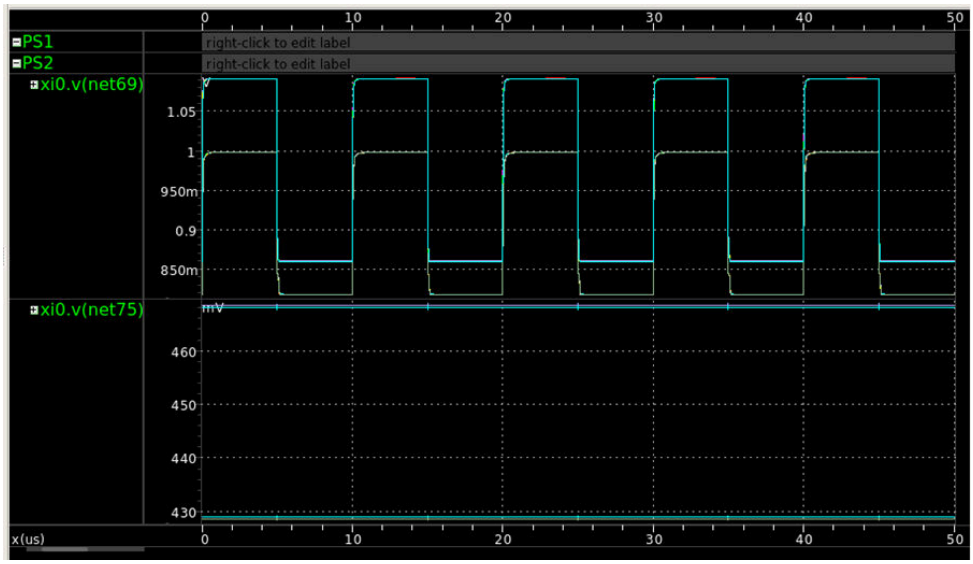
Figure 2 Constell plot



Feedback

Panel Set

A panel set is a named panel used for grouping signals from different panels. It can contain sublevel panel set panels. To create an empty panel set, choose **Panel Set** from the menu in the **Panel** section of the **Waveform** tab. It appears as follows:



When selecting to **Delete**, **Cut**, or **Copy** a panel set, the operation is applied to all panels contained in the panel set.

Renaming a Panel Set

The name of a panel set is automatically assigned when created. To rename a panel set, right-click the panel and select **Edit Label**.

Expanding a Panel Set

To expand or collapse all panels in a panel set, click the **+/-** icons before the panel set's name. When a panel is collapsed, meters located on that panel are hidden.

Creating a Panel Set From Selected Panels

To create a panel set for selected panels, right-click a panel and select **Create Panel Set**. To ungroup all panels under one single panel set, right-click the panel and choose **Ungroup Panel Set** from the menu it opens.

To insert an empty panel set before a selected panel, right-click the panel and choose Insert **Empty Panel Set**.

Feedback

Rearranging Panel Set Panels

To move a panel out of a panel set, drag and drop that panel to after the last panel of the panel set, outside of the yellow bounding box. To move a panel to the end of a panel set, drag and drop that panel above the bottom of last panel, inside of the yellow bounding box.

Parametric Plots

The following methods for generating parametric plots are available:

- Using the cursor in a 2D-sweep panel to plot the waveform value versus one of the sweeping parameters. Choose **Plot Y vs. X2** from the cursor context menu. To select a sweeping parameter, right-click the waveform file of the signal in the Waveform Browser and choose **X2 Parameter**. This method is limited to displaying the waveform value only.
- Use the parametric() function in the equation builder. The parametric() function takes two arguments. The first argument must be a measurement of a multi-trace waveform; the second argument is the sweeping parameter to be used as the x-axis in the parametric plot (for example, `parametric(mean("v(out)"),"TEMP")`). If the second argument is omitted, the default sweeping parameter is used.
- Use the built-in PrimeSim HSPICE parametric plot tool. The parametric tool only accepts HSPICE .MEASURE result files (.mtx/.msx/.max) as the input.

Built-in Parametric Tool for PrimeSim HSPICE .ALTER Simulations

Choose **Parametric Plot** from the menu located in the **Utilities** section of the **Waveform** tab. Parametric plots directly from PrimeSim HSPICE measurement results files (.mt0/.ms0/.ma0 files) are supported.

The Parametric Plot tool can be used to generate scatter plot with measure result annotation. Select the X/Y parameters, the **Scatter Plot** option, and click **Plot** to create the scatter plot.

Generic parametric plot functions can be realized using the parametric() function in the equation builder.

Plotting Clock Jitter

The built-in Jitter vs Time tool can be used to analyze clock jitter between any logic and analog target signal with respect to a reference signal. The reference signal can be an analog or logic signal from simulation output, or an idea signal defined using the width/

Feedback

period/delay parameters. The jitter can be measure against the rise and fall edges of a reference signal.

To invoke the Jitter vs Time tool, choose **Jitter vs. Time** from the menu located in the **Utilities** section of the **Waveform** tab.

This section contains information on the following topics:

- Selecting Reference Signals
- Specifying the Target Signal
- Selecting Reference Edges
- Target Signal Edges
- Jitter Types
- Jitter Output Options

Selecting Reference Signals

The reference signal can be selected from the following sources:

- User-defined Ideal Clock: An ideal clock signal defined by delay, period, and width.
- A Digital Signal: A digital simulation output waveform.
- An Analog Signal: An analog simulation output waveform.

The Digital or Analog Signal field requires an external waveform that can be specified in the External Signal field. Drag and drop the external signal from the Waveform Browser to the input field. If an analog reference signal is used, threshold value for the rise and fall edges also needs to be specified.

If the **Display Reference Clock Signal** option is selected, the reference signal is shown together with the jitter-versus-time output waveform.

Specifying the Target Signal

The target signal can be either a digital or an analog simulation output waveform. The analog target waveform requires the rise/fall threshold values to be specified.

Selecting Reference Edges

You can select the rise, fall, or both edges of the reference signal as the reference edge for the jitter evaluation.

Feedback

Target Signal Edges

The following options are available:

- **Same as Reference**: Finds rise-to-rise and fall-to-fall jitters.
- **Rise**: Finds jitter from reference edge to only the rising target edge.
- **Fall**: Finds jitter from reference edge to only the falling target edge.
- **Cross**: Finds jitter from reference edge to the next target switching edge (rise or fall).

The Target Signal Edge options are enabled only if the Jitter Type option is set to **Trigger Time**.

Jitter Types

The Jitter vs. Time tool can evaluate jitter for the following types:

- **Trigger Time**: Measures the edge differences between the trigger edge and the signal edge.
- **Pulse Width**: Measures the pulse differences between the reference signal and the target signal at the reference edge.
- **Period Width**: Measures the period differences between the reference signal and the target signal at the reference edge.

By default, the target signal edges are searched in both directions from a reference switch edge. Select the **Always Search Forward for Target Edge** option to force the target search in the forward direction only.

The default target edge search range is half of the clock period at the current reference clock cycle. To limit the maximum search time range, enter the limit in the **Max. Target Edge Search Range** field.

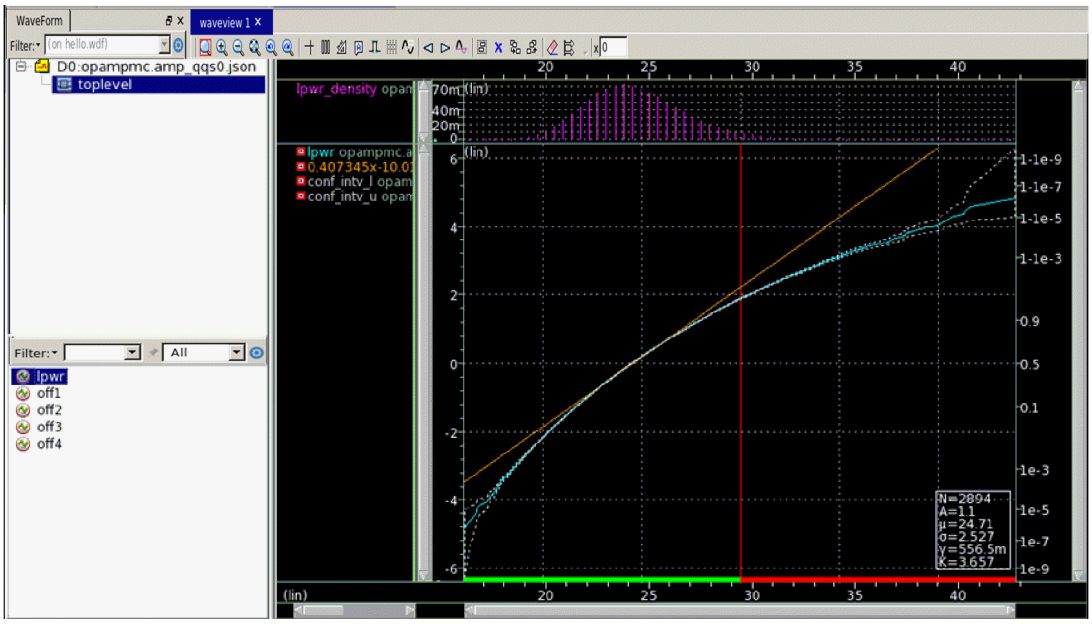
Jitter Output Options

The Jitter vs. Time result is displayed in a linear X-Y panel. PWL waveforms and bar charts are the available plot types. The jitter value sign can be selected between forward-positive or backward-positive. In the forward positive mode, jitter values are positive if the target edge appears later than the reference edge.

Feedback

PrimeSim HSPICE QQ Plot

The PrimeSim HSPICE QQ statistical plot appears as follows:



After opening files, the measures are listed as signals in the output view.

When plotting QQ measure data, the WaveView tool displays it in an XY panel with Y2 Axis showed. The data array for each measure is used as x-axis and the quantiles array is used as y-axis. The quantiles array is global, which means all measures have the same Y values, but different X values.

The Y2 Axis shows the PNorm values of the standard normal quantile values. The fit line is plotted in the same panel. The Confidence interval lower and upper lines are plotted in the panel as well, as dashed lines.

When plotting QQ measurements, the tool does not share panels with other signals. Each QQ measure is in a new panel, along with fit line and confidence interval lines. Density bars are displayed in a new panel above the QQ measure plot panel.

Statistical summary information is displayed in a box in the bottom right corner of the panel.

When adding a cursor, the tool shows both left Y and right y-axis values for QQ plot lines, separated by the ; character.

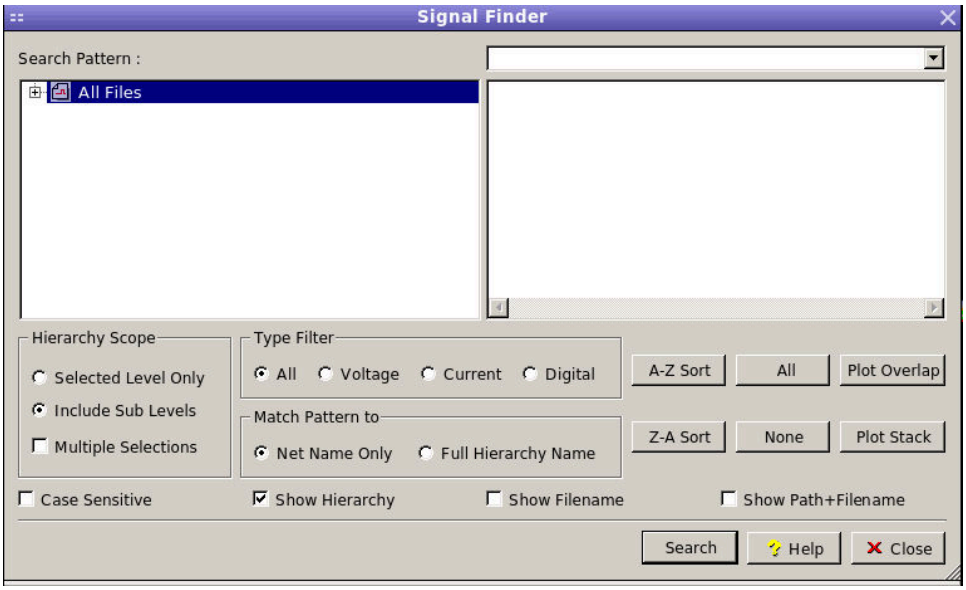
The lower and upper specification limits are plotted as vertical red lines. If both lower and upper specification limits values are -Inf or Inf, there is no specification line displayed. If one of them is not -Inf or Inf, there is one vertical line displayed. If both of them are not -Inf or Inf value, there are two vertical lines displayed.

Feedback

If lower limit < upper limit, a green bar is shown above the x-axis between lower limit and upper limit, with a red bar out side of [lower, upper] range. If lower limit > upper limit, the tool shows a red bar above the x-axis between lower limit and upper limit, with a green bar out side of [lower, upper] range. If lower limit == upper limit, or both of them are -Inf or Inf value, the tool does not show bars.

Searching Signals

The Signal Finder can be used to search signals from all (or selected) loaded waveform data files and hierarchies. Choose **Signal Finder** from the menu located in the **Utilities** section of the **Waveform** tab to open the **Signal Finder** dialog box.



The signal finder supports name patterns matching with the asterisk (*) and question mark (?) wildcards. Enter a search pattern and hit the return key or click **Search** button to commence a search. The search results are displayed in the right signal list window. Click a signal from the list window to highlight the signal source in the OutputView browser. Drag-and-drop operations are also supported from the signal finder result window into any waveview window. You can sort the results alphabetically by clicking the **A-Z Sort** and **Z-A Sort** buttons.

The search target signals can be refined to Voltage/Current signals only or All Signals by clicking one of the **Type Filter** option radio buttons. To limit the search to a file or a hierarchy level, select the corresponding entry from the left file/hierarchy browser. If the **Selected Level Only** option is selected, the target signals are those in the selected hierarchy/file only. If the **Include Sub Levels** option is selected, the target signals are those in and below the selected hierarchy/file.

Feedback

On Linux platforms, select the **Multiple Selection** option to select multiple hierarchy entries as the search target.

The **All** and the **None** buttons are used to select/de-select all items in the right signal list window for the drag-and-drop operations.

Click **Plot Overlap** or **Plot Stack** to plot selected signals from the result window in overlap or stack mode.

The **Case Sensitive** option controls the signal name case sensitivity, while the **Net Name Only** and the **Full Hier Name** options control which part of the signal names to be used in the search process.

The **Show Hierarchy**, **Show Filename**, and **Show Path+Filename** options can be used to control the name appearance in the search result browser.

Scanning Signals

Once signals are loaded onto a waveview, you can scan signals from the same files using the **Scan** control buttons. This feature can be used to compare signals from different data files.

Clicking the **right arrow** button advances signals from all selected panels to their corresponding next signals from the same waveform data files. Clicking the **left arrow** scans signals in the reverse direction.

Please note that scanning signals might require reloading data if the next/previous signals are not currently loaded. To speed up the scan process by avoiding data reloading, you might choose to load all waveform data to memory when waveform data files are initially opened (see Opening a Waveform File).

Default bindkey: **N** (next) or **P** (previous).

Scan Settings

To change the scan settings, click **Settings** on the **Waveform** tab, and choose **Scan Settings** from the menu that opens.

You can configure the scan function into two different modes:

- **Direct Scan**: With the Direct mode, the WaveView tool scans signals according to their order in the waveform data file.
- **Smart Scan**: In the Smart mode, with a master waveform data file selected, the WaveView tool scans signals from the master waveform data file according to their order in the file, while automatically searches other waveform data files for signals with matched names.

Feedback

Select a scan mode and a master file and click **Apply** to make the change. The **Lock Scope** option controls if the scan is limited within the current hierarchy scope.

Data Reduction

To open the Data Reduction tool, choose **Data Reduction** from the menu located in the **Utilities** section of the **Waveform** tab. The Data Reduction Parameters dialog box appears, and the parameters are acquired for the data reduction operation.

Piecewise-linear waveforms often contain redundant data points. The data point reduction function allows users to remove redundant data points based on user-specified error tolerance. The same data reduction scheme is also used in WDF data reduction.

If the active waveview has only one panel, the reduction operation is by default applied to all waveforms in this panel. If the active waveview has more than one panel, you need to first select panels for the reduction operation.

Click **Ok** to start the data reduction operation. Waveforms with reduced data points are created in new panels in the same waveview.

Direct Plot Mode

To enable direct plot mode, set `WV_DPLOT=1`, then click the following button at the top of the waveform browser:



Switching to direct plot mode changes the GUI as follows:

- The mini toolbar at the top of the Signal List is hidden.
- There is a new line editor for plotting:



- In the tree browser, the current file is selected and highlighted:



Feedback

The following features are supported for searching:

- Full scope matching is supported using the delimiter . or /, based on the current file (the end of the delimited part represents the signal).
- Both * and ? are supported for scope and signal (* represents only one level).
- Multiple string searching is supported, separated by whitespace (for example, “x1.x2.v* xx1.xx2.i*”).
- Search results are shown in the Signal List with full scope name.
- Currently, search is insensitive and does not support regular expressions.

5

Measuring Waveforms

This chapter details measuring waveforms in the WaveView tool.

This chapter contains information on the following topics:

- Using Cursors
- Using Monitors
- Using the Measurement Tool
- Supported Measurements
- Measuring Eye Diagrams
- RF Measurements
- PrimeSim HSPICE Measure Tool
- Pareto Analysis

Using Cursors

This section contains information on the following topics:

- Adding a Cursor
- Adding a Horizontal Cursor
- Adding a Cross-Cursor
- Jumping Forward and Backward
- Deleting Cursors
- Assigning Cursors to Monitors
- Using Cursor De-Select Mode

Feedback

Adding a Cursor

Cursors are used to trace waveform-related values. Multiple cursors can be added to a panel. The x-axis position of a cursor is indicated underneath the cursor hair.

To add a cursor, click **Cursor** in the **Panel** section of the **Waveform** tab, and choose **Add Cursor** from the menu that opens.

You can also click the cursor  button in the button toolbar. Cursors can be added for delta measurements by pressing the left- and right-bracket keys ([and]) while moving the cursor.

When multiple cursors are added to a panel, the last moved or added cursor becomes the active cursor.

Waveform values corresponding to the active cursor are displayed in the monitor. In an X-Y panel, distances from all non-active cursors to the active cursor are displayed next to the active cursor.

If you want vertical cursor crossing points with the existing settings, in the **Export V-Cursor Crossings** dialog box, right click a vertical cursor, and choose **Export V-Cursor Crossings With Existed Setting** from the menu that opens. It exports the values without popping up the setup dialog box.

If you want to export Y values at specified X-value crossings to a text file, right-click a vertical cursor, and choose **Export X-Values Crossings** from the menu that opens.

If you want to export some or all vertical cursor crossing points to a text file, right-click a vertical cursor, and choose **Export V-Cursor Crossings** from the menu that opens. In the **Target** field, choose **Selected Cursors** or **All Cursors**.

If you want to export the Y value and data for logic signals in time sequence for all selected cursors, select multiple cursors, right-click on one cursor and select **Export V-Cursor Crossings**. All the values are exported to a file or text window.

This section contains information on the following topics:

- Selecting With the Cursor
- Active Cursor and Reference Cursor
- Moving a Cursor
- Showing and Hiding Distance Lines
- Setting Distance Between Two Cursors
- Locking Pairs of Cursors
- Locking Cursor Position

Feedback

- Snapping the Cursor to an X/Y Value
- Snapping the Cursor to a Jump Type
- Setting the Cursor Alias Name
- Showing and Hiding Cursor Crossing Values
- Cursors in Smith Charts and Polar Plots
- Cursors in 2-D Sweep Panels
- Cursors in 3-D Sweep Panels

Selecting With the Cursor

The cursor is object-based and can select signals or panels, as follows:

- Mousing over a signal or panel highlights the object to be selected with a yellow box.
- **Left-click** selects the object. **Ctrl-click** and **Shift-click** can be used to select multiple objects at once. You cannot select both panels and signals at the same time.
- **Right-click** opens the context menu for the object (and select it if the object is not selected).
- When mousing over the naming area, the cursor appears as follows:
  - For objects that can only be selected, the cursor is an arrow.
  - For objects that can be expanded and collapsed, the cursor is a hand.

Active Cursor and Reference Cursor

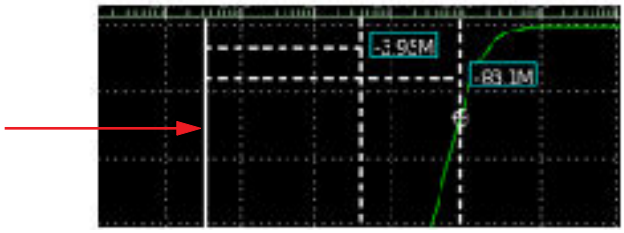
Distance lines are shown between a reference cursor and other cursors.

When you add, select, drag, or right-click a cursor, that cursor is set as the active cursor. The active cursor's value is displayed in the monitor.

The first added cursor is set as the reference cursor. The reference cursor can be different than the current cursor and does change when changing the current cursor. There can

Feedback

only be one reference cursor in an X-Y or logic panel. It is shown with a solid line. For example:



To set a cursor as the reference cursor, **right-click** the cursor and select **Set as Reference Cursor**. Or double-click a cursor and set the cursor as reference cursor in the **Cursor Attributes** dialog.

Moving a Cursor

Depending on the default action of the left mouse button in a waveview (default left mouse button action can be specified in **Left Button Default** preference setting), a cursor can be moved with:

- Cursor Mode: If the left mouse button is pressed with mouse pointer near a cursor, the cursor is grabbed. A grabbed cursor can be dragged to a new position. If the mouse pointer is not in the vicinity of any cursor when the left mouse button is pressed, the active cursor is grabbed regardless of its position.
- Zoom-First Mode: If the left mouse button is pressed and released without any mouse pointer movement. The active cursor is grabbed to the mouse pointer location when the button is released.

In a zoomed panel, cursors outside the plotting range are placed on the left (or right) boundary of the plotting area. They can be grabbed from the borders of the plotting area. The cursor that is outside but closest to the boundary is grabbed first.

Showing and Hiding Distance Lines

To show or hide the distance lines and delta value of a cursor, **right-click** the cursor and select **Show/Hide Distance**. Hiding distance on a reference cursor hides all distance lines.

When the preference **Hide All Distance between Cursors** is enabled, the tool hides all distance lines and value boxes for all cursors.

Feedback

Setting Distance Between Two Cursors

To set the distance between cursors, select two cursors, **right-click** on one of them and select **Set Distance**. Enter the required distance in the **Cursor Distance** dialog and click **OK**. The current cursor is moved to achieve the specified distance.

You can also open the **Cursor Distance** dialog by **double-clicking** on the distance value box or **right-clicking** on the distance value box and selecting **Set Distance**.

Locking Pairs of Cursors

To lock a pair of cursors, select two cursors, right-click on one of them and select **Lock distance**.

You can also lock distance by enabling **Lock Distance between Cursors** in the **Cursor Distance** dialog.

To unlock distance, **right-click** on a cursor and select **Unlock Distance**.

To unlock a pair of locked cursors, choose **Release Locked Pair** from the context menu.

Locking Cursor Position

To lock cursor position, enable **Lock Cursor** in the **Set X-value** and **Set Y-value** dialog boxes.

Snapping the Cursor to an X/Y Value

To snap a vertical/horizontal cursor to an X/Y value, respectively, **right-click** on the cursor and select **Set X/Y Value**. Enter the required value in the dialog and click **OK**.

Snapping the Cursor to a Jump Type

To snap the cursor to a jump type, perform the following steps:

1. **Right-click** on the added cursor, select **Attributes**. The **Cursor Attributes** dialog box opens.
2. Choose a **Jump Type**.

   If you choose **To**, the cursor jumps to the **X Value** you have set in this field.
3. Select the **Snap Cursor to Jump Type** check box.

   This snaps the cursor to the selected jump type.
4. Click **OK**

To enable or disable snapping the cursor to jump type, **right-click** on the added cursor, and select or deselect **Snap Cursor to Jump Type to this position** from the context-

Feedback

sensitive menu. When enabled, the cursor is snapped to the jump type position set in the earlier procedure.

Setting the Cursor Alias Name

To set an alias name for a cursor, right-click on the cursor value label box and select **Set Alias**. Enter the alias name in the dialog and click **OK**.

Showing and Hiding Cursor Crossing Values

Cursor value display has been enhanced as follows:

- **Showing and hiding V-value crossings:** To show/hide V-value crossing labels along a vertical cursor, **right-click** the cursor and select **Show/Hide V-Value Crossings**.
  - **Show/Hide X Value:** To show/hide X-axis values crossing labels along a vertical cursor, **right-click** the cursor and select **Show/Hide X Value**. To set **Hide X value** as default, choose **Configuration > Preferences > Waveview > Waveview**, select **Hide X Values on the Value Labels at V-Cursor Crossing Points by Default**.
- **Showing and hiding H-value crossings:** To show/hide H-value crossing labels along a horizontal cursor, **right-click** the cursor and select **Show/Hide H-Value Crossings**.

  You can also choose to show signal names and file names of all plotted signals or (select signals) as detailed in the **H-Cursor Attributes** dialog.
- **Showing P1/P2 and Delta Values:** The tool shows crossing point values and delta values (P2-P1) of two crossing points between the reference cursor (P1) and the target cursor (P2). By default, the P1 value is shown along with the reference cursor, and the P2 value and delta value are shown along with the target cursor.

  When distance lines are hidden, the delta value is not shown.
- When the preference **Display Value Labels at Crossing Points by Default** is enabled, the tool shows crossing labels along the cursor.
- The preference **Show Reference Cursor Crossing Value** can be set to the following:
  - **Individually** — the P1 value is shown along the reference cursor and the P2 value and delta value are shown along the target cursor.
  - **Along with Target Cursor** — the P1 value, P2 value, and delta value are all shown along with the target cursor.

Cursors in Smith Charts and Polar Plots

In Smith charts and Polar plots, a cursor consists of a circle hair and a radial hair. The cross point is the cursor location and its value is displayed. A cursor can be grabbed by clicking the mouse button when the mouse pointer is near any of its two hairs. The cursor values in the monitor column trace the data points that are closest to the hair cross.

Feedback

Cursors in 2-D Sweep Panels

You can use a cursor in a 2D-sweep panel to create a new 'Y vs. X2' plot. Move the cursor to a preferred position, right-click the cursor, and choose **Plot Y vs X2** from the menu that opens.

Cursors in 3-D Sweep Panels

In a 3D-sweep panel, a cursor consists of a vertical hair, and two crossing hairs lying in the base plane. To grab a cursor in a 3D-sweep panel, move the mouse pointer near the vertical hair base where the two crossing hairs meet.

Adding a Horizontal Cursor

Horizontal cursors are supported in analog X-Y panels. To add a horizontal cursor, click **Cursor** in the **Panel** section of the **Waveform** tab, and choose **Add H-Cursor** from the menu that opens.

You can also switch a vertical cursor to a horizontal cursor by right-clicking a vertical cursor and choosing **Switch to H-Cursor** from the menu that opens. The horizontal cursors trace the x-axis values of the nearest intersect point with waveforms. The x-values are displayed in the monitor column.

You can also click the **Add Cursor** control button to add an H-cursor by setting the **Add H-Cursor with Add Cursor Control Button** option in the preferences. To set this preference, click **Settings** in the Configuration tab, and click the **WaveView** tab. Click the **Add H-Cursor with Add Cursor Control Button** check box, and click **OK**.

This section contains information on the following topics:

- Annotating Horizontal Cursors
- Hiding Horizontal Cursors
- Anchoring Horizontal Cursors
- Exporting Horizontal Cursor Values

Annotating Horizontal Cursors

In an X-Y panel, labels can be annotated to all the signal crossing points of a horizontal cursor. To enable crossing value labels, right-click an H-cursor, and choose **Crossing Options** from the cursor context menu.

Note:

H-cursor label positions are recorded when the total number of H-crossing labels stay the same during H-cursor movement. The modified cursor label positions are saved in the session file.

Feedback

Hiding Horizontal Cursors

To hide the H-cursor hair, right-click an H-cursor, click the **Hide Cursor Hair** check box, and click **OK**.

Anchoring Horizontal Cursors

To anchor horizontal cursors to a single panel, right-click an H-cursor, and choose **Crossing Options** from the menu that opens. Click the **Cursor Anchor in Panel** check box, and click **OK**

Exporting Horizontal Cursor Values

If you want to export all horizontal cursor crossing points to a text file, right-click a vertical cursor, and choose **Export H-Cursor Crossings** from the menu that opens.

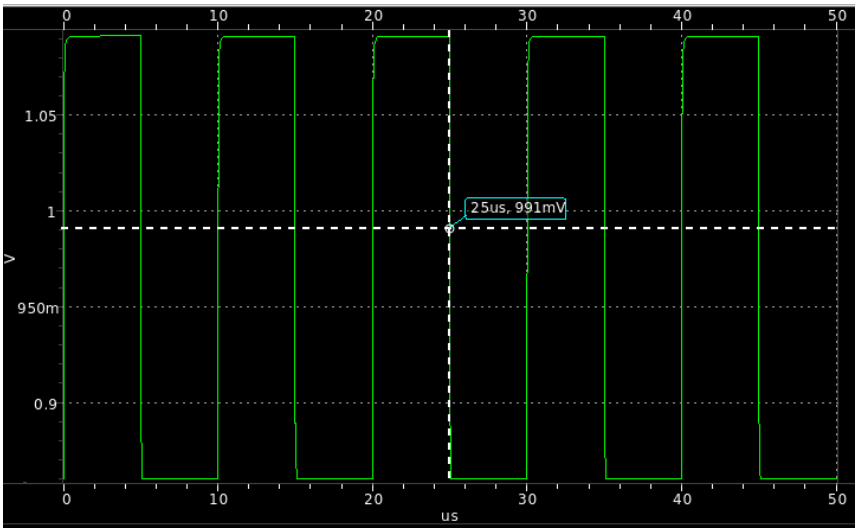
If you want to export X values at specified Y-value crossings to a text file, right-click a vertical cursor, and choose **Export Y-Values Crossings** from the menu that opens.

Adding a Cross-Cursor

Cross-cursors are supported in analog X-Y panels. To add a cross-cursor, click **Cursor** in the **Panel** section of the **Waveform** tab, and choose **Add Cross-Cursor** from the menu that opens.

No monitor is created since there is only one crossing point for all waveforms. The cross-cursor has one vertical line and one horizontal line, showing X and Y values at the crossing point. The crossing point can be in different panels.

It appears as follows:



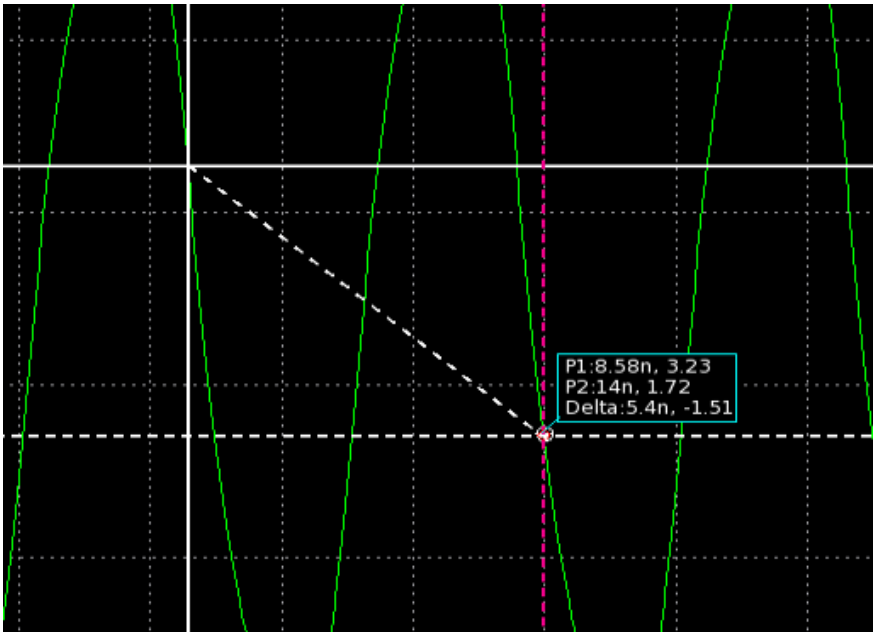
Feedback

A cross-cursor has two modes:

- Floating (default) — the cursor can be placed at any location, and the vertical and horizontal line can both be moved individually and freely.
- Snap to waveform — when moving the vertical or horizontal line, the other line may be moved to snap to the waveform. It can be snapped to different waveforms. This is enabled by default.

You can set the X or Y value through the right-click Attributes dialog, the icon bar, or using jump forward/backward.

You can pair a cross-cursor with another cross-cursor, which shows the Delta X and Delta Y. For example:



Jumping Forward and Backward

In an X-Y or logic panels, a cursor can be jumped forward to the next data point, peak, switching or crossing point using the cursor context menu.

To jump a cursor forward or backward, right-click near the cursor, and choose**Jump (F)orward "SIGNAL"** or Jump (B)ackward "SIGNAL" from the menu that opens to jump the cursor forward or backward on the target signal SIGNAL.

You can also select a cursor, then click **Cursor** in the **Panel** section of the **Waveform** tab, and choose **Jump Forward** or **Jump Backward** from the menu that opens.

Feedback

The target signal is indicated by a cross marker on the signal waveform at the intercept point with the cursor hair. To select a different target signal, move the cursor near a waveform (along the Y-direction) and the cross marker snaps to the new target waveform.

For digital panels, the cursor can be jumped to the next value change, the next match value, the next rise/fall edge, or the user-defined time.

To jump to a specific X value, right-click a V cursor, and choose **Goto X Value** from the menu that opens.

Default bindkeys: Press the **F** key to jump forward, and the **B** key to jump backward.

Jump Settings

To change the jump parameters, click **Cursor** in the **Panel** section of the **Waveform** tab, and choose **Jump Settings** from the menu that opens to invoke the **Cursor Jump Parameters** dialog box.

Enter a preferred jump type and associated parameters and click **Ok** to make the change. Click **Set as Default** to save the settings in ~/.spxlast as the default setting.

Deleting Cursors

To delete a single cursor, right-click near the cursor you want to delete, and choose **Delete** from the menu that opens to delete the cursor.

You can also select a cursor and click **Cursor** in the **Panel** section of the **Waveform** tab, and choose **Delete Cursor** from the menu that opens.

Deleting All Cursors

To delete all cursors, select a group of cursors, and right-click near the selected group. Choose **Delete All** from the menu that opens to delete all cursors.

You can also select a group of cursors and click **Cursor** in the **Panel** section of the **Waveform** tab, and choose **Delete All** from the menu that opens.

Assigning Cursors to Monitors

When you add a monitor of cursor value, if there is only one monitor, it links to the active cursor. Otherwise, the new added monitor links to a cursor that does not link to any monitor.

Selecting **Auto Link Monitors** from the monitor context menu links all monitors of cursor value to corresponding cursors.

Feedback

Selecting **Link to Monitor** from the cursor context menu links selected cursors to corresponding monitors. If the tool cannot find a monitor for a selected cursor, it adds a new monitor and link it to this cursor.

Using Cursor De-Select Mode

Disable the option **Move Cursor to Mouse Click Position** and the cursor operates in de-select mode and does not move.

Using Monitors

Monitors are used to trace waveform-related values. Multiple monitors can be added to vertical waveviews, or panels in horizontal. The following monitors are available for analog panels:

Monitor	Description
cursor value	(Y=) Traces waveform values that correspond to the current active cursor position. Logarithmic value is displayed if the corresponding panel has logarithmic Y-axis. To display the logarithmic value of a negative value x, the format `-<log(abs(x))>` is used.
derivative	(S=) Traces waveform derivative values that correspond to the current active cursor position. Derivative of logarithmic values is displayed if the corresponding panel has logarithmic Y-axis.
1/derivative	(1/S=) Traces the reciprocal of waveform derivative values that correspond to the current active cursor position.
Y delta	(D=) Displays difference between (1) waveform values that correspond to the active cursor position and (2) waveform values that correspond to the other cursor. Delta value is available only when a waveview (or panel) has two cursors.
cursor average	(A=) Displays waveform average between the cursors. Cursor average value is available only when a waveview (or panel) has two cursors.
maximum	(MAX=) Displays waveform maximum values over full or zoomed signal range.
minimum	(MIN=) Displays waveform minimum values over full or zoomed signal range.
peak-to-peak	(PP=) Displays waveform peak-to-peak values over full or zoomed signal range.
average	(AVG=) Displays waveform average values over full or zoomed signal range.

Feedback

Monitor	Description
RMS	(RMS=) Displays waveform root mean square values over full or zoomed signal range.

If you choose a **maximum**, **minimum**, **peak-to-peak**, **average**, or **RMS** monitor, you can also specify the following **X-axis Range** settings:

X-Axis Range Option	Description
full signal range	Measures across the full range of a signal.
zoomed view range	Measures just the zoomed portion of the signal that is currently displayed.
cursor range	Measures between the x-values of two cursors.
user-defined range	Measures between the two specified values.

This section contains information on the following topics:

- Adding Monitors
- Copying Monitor Values
- Reconfiguring Monitors
- Deleting Monitors
- Linking Monitors to Cursors
- Exporting Monitors

Adding Monitors

To add a monitor, click **Cursor** in the **Panel** section of the **Waveform** tab, and choose **Add Monitor** from the menu that opens.

If a vertical waveview (or a panel in a horizontal waveview) has no existing monitor in the cursor value type, a cursor value type monitor is automatically added when a cursor is created.

When you add a monitor of cursor value, if there is only one monitor, it links to the active cursor. Otherwise, the new added monitor links to a cursor that does not link to any monitor.

Feedback

To add a new monitor, click the control button to open the Monitor Settings dialog box. Select the preferred settings, and click **Ok** to create the monitor.

Copying Monitor Values

To copy the value of a monitor, right-click a monitor, and choose **Copy Value** from the menu that opens. The value is copied to the clipboard.

Reconfiguring Monitors

To reconfigure a monitor, right-click a monitor, and choose **Configure Monitor** from the menu that opens. In the **Monitor Settings** dialog box that opens, reconfigure the monitor, and click **Ok** to make the change.

Deleting Monitors

To delete a monitor, right-click a monitor, and choose **Delete Monitor** from the menu that opens.

Linking Monitors to Cursors

To link a monitor to a selected cursor, right-click a monitor and choose **Link Cursor** from the context menu that appears. In the **Cursor Selection** dialog box that opens, link any desired monitors to cursors.

Exporting Monitors

To export monitors to a text or CSV file, right-click a monitor and choose **Export Monitor** from the context menu that appears. Choose an **Output File** name, **Output Type**, and **Target** in the **Export Columns** dialog box that opens, and click **OK** to export the monitors.

Using the Measurement Tool

To access the Measurement Tool, click **Measure Tool** in the **Utilities** section of the **Waveform** tab.

Once one or more measurements are added to a waveview, you can drag those measurements to take the measurement interactively. If a panel has multiple signals and you want to perform a Width, Slew, Frequency, or Duty-Cycle measurement, the default measurement target is the first selected (highlighted) signal. The first signal is the default if no signal is selected.

Feedback

Note:

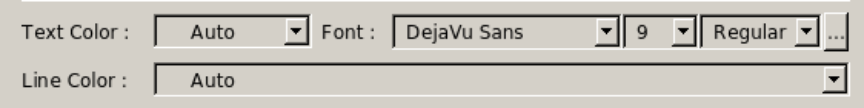
The names of any signals that are currently being measured are displayed at the bottom of measurement configuration dialog box.

This section contains information on the following topics:

- Setting the Measurement Font and Color
- Adding or Removing Measurement Favorites
- Viewing Measurement Results
- Setting the Precision of Measurements
- Exporting Measurements
- Locking and Unlocking Measurements

Setting the Measurement Font and Color

You can set the font and color in the **Measurement Tool** window to label individual measurements to differentiate between measurements.



Adding or Removing Measurement Favorites

In the Favorites tab of the **Measurement Tool** window, you can choose from several measurement you use most often.

To add a measurement to the list of favorites:

1. Click the **All** tab.

   The available measurement categories and corresponding measurement appear.

2. Click a measurement.

3. Click the **Favorite** button at the bottom of the **Measurement Tool** window.

   The selected measurement is saved as a favorite and is included as one of the measurements you can choose from the **Favorite** tab.

Feedback

To remove a measurement from the favorite measurement list:

1. Click the **Favorite** tab.

   The current measurement favorites are displayed at the top of the **Measurement Tool** window.

2. Click a measurement.

3. Click the **Remove** button at the bottom of the **Measurement Tool** window.

   The selected measurement is removed from the list of favorite measurements you can choose from the **Favorite** tab.

Viewing Measurement Results

To see existing Measurement Tool results from a waveview, right-click a measurement result in a waveview, and choose **Measure Results** from the menu that opens. The **Measure Results** window opens with a table of measurement result values.

Setting the Precision of Measurements

To change the precision of measurements, click **Settings** in the **Configuration** tab, and then click the **General** category in the **Settings Preferences** window that opens. Enter a value in the **Number of Valid Digits** text box.

Exporting Measurements

When you perform a measurement on a waveform, the measurement can be exported to the following equivalent forms:

- Equivalent PrimeSim HSPICE `.MEASURE` statements
- Tcl ACE `sx_equation` statements
- Equation builder equations

To directly export a measurement to the Equation Builder, right-click a measurement and choose **Measurement to Eqn** from the context menu. The equivalent measurement equation and the result are displayed in the Equation Builder.

To export all measurements on a waveview window, right-click a measurement and choose **Measurement Export**. Use the **Dynamic Measurement Export** dialog box to

Feedback

select the measurements to be exported and the preferred export type. Only the following types of measurements can be exported:

- Rise/Fall time
- Width/Frequency
- Difference
- Y-Difference
- Y-Range

In the **Dynamic Measurement Export** dialog box, you can define the export name of individual measurements, enable or disable a measurement in the list, control how to count the number of switch edges, and define the precision for the exported values.

For PrimeSim HSPICE Measure and ACE Tcl export modes, the path of the output file must be specified.

Locking and Unlocking Measurements

To lock or unlock a measurement, right-click the measurement and choose **Lock Measurement** or **Unlock Measurement**, respectively, from the menu that opens.

Supported Measurements

The following measurements are supported:

Measurement	Type	Description
AC Coupled RMS	Level	Calculates the RMS value of the AC component of a waveform.
Amplitude	Level	Calculates the difference between the topline and baseline reference levels.
Average	Level	Calculates the average level of a waveform.
Bandpass	Frequency Domain	Calculates the bandwidth of a bandpass-shaped waveform.
Baseline	Level	Calculates the baseline level of a waveform.
Cpk	Statistics	Calculates an indicator of the process capability for a waveform relative to specified upper and lower limits.

Feedback

Measurement	Type	Description
Damping Ratio	S Domain	Calculates the damping ratio of a waveform as -real/mag.
Data(X,Y)	General	Displays the X-axis and Y-axis values of a point on a waveform.
Delay	Time Domain	Calculates the delay between the edges of two waveforms relative to the default or specified topline and baseline levels for both the measured waveform and the reference waveform.
Difference	General	Displays the following information for two points on one or two waveforms: X1, Y1; X2, Y2; and Delta X, Del Y, Slope.
Dpu	Statistics	Calculates the total number of defects per unit based on the points in a waveform.
Duty Cycle	Time Domain	Displays a duty cycle of a periodic waveform.
F VS T	Time Domain	Calculates all frequency, period, or duty cycle values for the edges crossing the threshold, and then creates new Frequency vs. time, Period vs. time, or Duty Cycle vs. time waveforms using the specified X range and Y level.
Frequency	Time Domain	Displays the frequency and period of a periodic waveform.
Frequency Value	S Domain	Displays the frequency value at the point of a complex waveform on a Nyquist (or Nichols) plot.
Gain Margin	Frequency Domain	Displays the gain margin of a complex analog waveform in decibels (dB).
Highpass	Frequency Domain	Calculates the corner frequency of a waveform with a highpass shape.
IP2	RF	Calculates the second-order intercept point.
IP3/SFDR	RF	Calculates the third-order intercept point.
Jitter	Time Domain	Displays the histogram of crossing edges with a specified X range or Y range.
Length	General	Calculates the length of a straight line that connects two points on a waveform or two points on two waveforms.
Lowpass	Frequency Domain	Calculates the corner frequency of a waveform with a lowpass shape.

Feedback

Measurement	Type	Description
Maximum	Level	Calculates the maximum level of waveform.
Mean	Statistics	Calculates the mean value of a waveform.
Mean+3std_dev	Statistics	Calculates the (mean + 3 standard deviation) value of a waveform.
Mean-3std_dev	Statistics	Calculates the (mean - 3 standard deviation) value of a waveform.
Minimum	Level	Calculates the minimum level of a waveform.
Natural Frequency	S Domain	Calculates the natural frequency of a waveform as the absolute value of an argument.
Overshoot	Time Domain	Calculates the overshoot of a waveform relative to a default or specified topline.
P1dB	RF	Calculates the 1dB compression point.
Peak to Peak	Level	Calculates the peak to peak values of a waveform.
Phase Margin	Frequency Domain	Displays the phase margin of a complex analog waveform in degrees.
Quality Factor	S Domain	Calculates the quality factor of a waveform.
Rise/Fall Time	Time Domain	Calculates the risetime and falltime between specified upper and lower levels of a waveform.
RMS	Level	Calculates the RMS (Root Mean Square) value of a waveform.
Settle Time	Time Domain	Calculates the settle time of a waveform with respect to a default or specified settle level and a specified settle band.
Slew Rate	Time Domain	Calculates the slew rate of a waveform relative to the default or specified topline and baseline levels.
Std_dev	Statistics	Calculates the standard deviation of a waveform.
Stopband	Frequency Domain	Calculates the stopband, low, high, or center frequency level or the level at which the measurement is calculated for a stopband-shaped waveform.
Topline	Level	Calculates the topline level of a waveform.
Undershoot	Time Domain	Calculates the undershoot of a waveform relative to a default or specified baseline level.

Feedback

Measurement	Type	Description
Width	General	Calculates the pulse width of a waveform.
X at Maximum	Level	Displays the x-value corresponding to the maximum value of a waveform.
X at Minimum	Level	Displays the x-value at the minimum value of a waveform.
Y Diff	General	Displays the Delta Y at a X value between two points on two waveforms.
Y Range	General	Displays the following information for a waveform: Waveform name, peak-to-peak Y range; Minimum Y value, Maximum Y value; and Average Y value, RMS value.
Yield	Statistics	Calculates the ratio of the number of data points between the Y-axis levels Upper and Lower relative to the total number of data points.

General Measurements

The following general measurements are available:

- Data(X,Y)
- Difference
- Length
- Width
- Y Diff
- Y Range

Data(X,Y)

Displays the x-axis and y-axis values of a point on a waveform.

Note:

This measurement is supported only in Analog X-Y, 2-D Sweep, and Smith Chart waveview panels.

Feedback

The following parameters can be set for the Data(X,Y) measurement:

Parameter	Description
Locked Level	Locks the measurement Y level at the specified percentage or level (in Y-axis units).
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Range options	• Entire Waveform — (default) measures the entire waveform. • Visible Range — measures the visible range. • User Specified — measures the defined range.
Label Move Independently	Allows label and endpoints to move independently.
Label Move With Aim	Allows label to move with the anchor marker.
Show X-Axis Value	Displays the x-axis value.

Difference

Displays the following information for two points on one or two waveforms: X1, Y1; X2, Y2; and Delta X, Del Y, Slope. The capacitance and inductance values from the X and Y points are also calculated using the following functions:

$$C = \frac{x2\text{-}x1}{2\pi \cdot x1x2 \cdot (y1\text{-}y2)}$$

$$L = \frac{y1\text{-}y2}{2\pi(x1\text{-}x2)}$$

Note:

This measurement is supported only in Analog X-Y, Eye Diagram, and 2-D Sweep waveview panels.

Feedback

The following parameters can be set for the Difference measurement:

Parameter	Description
Lock Y Level	Locks the measurement Y level at the specified percentage or level (in y-axis units).
Lock X Level	Locks the measurement X level at the specified percentage or level (in x-axis units).
Indep P2 Lock Level	Enables the Lock 2nd Point at independent second Y-level point measurement option.
Lock 2nd Point at	Locks a second measurement Y-level point at the specified percentage or level (in y-axis units). This option is only available when the Indep. P2 Lock Level option is selected.
Lock P2 X Value	Locks a second measurement X-level point at the specified percentage or level (in x-axis units).
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Anchor First Point	Anchors the measurement at a single point; further dragging only moves the second point.
Gravity Snap	Snaps anchors to data points when moving.
Label Move Independently	Allows label and endpoints to move independently.
Label Move With Aim	Allows label to move with the anchor marker.
Range options	• Entire Waveform — (default) measures the entire waveform. • Visible Range — measures the visible range. • User Specified — measures the defined range.

Length

Calculates the length of a straight line that connects two points on a waveform or two points on two waveforms. If two waveforms are selected, the two waveforms do not need to be the same type.

Feedback

Width

Calculates the pulse width of a waveform.

Note:

This measurement is supported only in Analog X-Y and Eye Diagram waveview panels.

The following parameters can be set for the Width measurement:

Parameter	Description
Locked Level	Locks the measurement Y level at the specified percentage or level (in Y-axis units).
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Pulse Type	Specifies the pulse type as **Either**, **Up**, or **Down**.
Width Rate vs. X-axis Plot Option	Calculates the width value of an up or down pulse versus the X-axis.
Range options	• Entire Waveform — (default) measures the entire waveform. • Visible Range — measures the visible range. • User Specified — measures the defined range.

Y Diff

Displays the Delta Y at a X value between two points on two waveforms.

Note:

This measurement is supported only in Analog X-Y waveview panels.

Y Range

Displays the following information for a waveform:

- Waveform name
- Peak-to-Peak Y range

- Minimum Y value
- Maximum Y value
- Average Y value
- RMS value

Note:

This measurement is supported only in Analog X-Y waveview panels.

Specifying the **Measure over Zoomed X-Range** option measures the zoomed area that is currently displayed on screen. If this option is not specified, the whole waveform is measured even if the waveform is zoomed and some of the X-range is not visible.

Time Domain Measurements

The following General measurements are available:

- Delay
- Duty Cycle
- F VS T
- Frequency
- Jitter
- Overshoot
- Rise/Fall Time
- Settle Time
- Slew Rate
- Undershoot

Delay

Calculates the delay between the edges of two waveforms relative to the default or specified topline and baseline levels for both the measured waveform and the reference waveform. The rising or falling edge on the reference waveform is assumed to cause the corresponding (rising or falling) edge on the measured waveform so that the reference edge occurs before the measured edge.

All rising or falling edges for the measured waveform are determined based on the Trigger setting in the Measure dialog box. From each edge, the corresponding previously

Feedback

occurring edge on the reference waveform is determined. The difference between the two edges on the x-axis is the delay time.

The following parameters can be set for the Delay measurement:

Parameter	Description
Percentage	Locks the delay level at the specified percentages for the signal (L(%)) and reference signal (Ref. L(%)). In signal Y units.
Premise	Specifies the delay value as either (default), positive delay, or negative delay.
Signal Level	Locks the delay level at the specified levels for the signal (L(V)) and reference signal (Ref. L(V)). In signal Y units.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a delay level that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a delay level that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Trigger	Specifies the trigger type as **Either**, **Rising**, or **Falling** edge(s)
Ref. Trigger	Specifies the reference signal trigger type as **Either**, **Same**, or **Opposite**.
Measure Between Closest Edges by Default	Finds the closest edge between the `Signal` and `Ref. Signal`.
Delay Rate vs. X-axis Plot Option	Calculates the delay rate values of the **Rise**, **Fall**, or **Rise/Fall** edges vs. the x-axis.
Range options	• Entire Waveform — (default) measures the entire waveform. • Visible Range — measures the visible range. • User Specified — measures the defined range.

Duty Cycle

Displays a duty cycle of a periodic waveform.

The Duty Cycle is calculated as the percentage of:

$$\frac{t1}{t}$$

Feedback

Note:

This measurement is supported only in Analog X-Y waveview panels.

The following parameters can be set for the Duty Cycle measurement:

Parameter	Description
Locked Level	Locks the measurement Y level at the specified percentage or level (in Y-axis units).
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Duty Cycle vs. X-axis Plot Option	Calculates the duty cycle value vs. the x-axis.
Range options	• Entire Waveform — (default) measures the entire waveform. • Visible Range — measures the visible range. • User Specified — measures the defined range.

F VS T

Calculates all frequency, period, or duty cycle values for the edges crossing the threshold, and then create new Frequency vs. time, Period vs. time, or Duty Cycle vs. time waveforms using the specified X range and Y level.

Note:

This measurement is supported only in Analog X-Y waveview panels.

The following parameters can be set for the F VS T measurement:

Parameter	Description
Locked Level	Locks the measurement Y level at the specified percentage or level (in Y-axis units).
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a measurement Y level that is locked at a specified percentage.

Feedback

Parameter	Description
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Frequency, Period, Delta Period, Duty Cycle	The type of measurement versus time. You can choose one or more types.
Entire Waveform, Visible Range, User Specified	Uses the entire waveform, the visible range, or your specified values for the F vs T measurement.
Delta-P vs T Plot	Plots the delta period versus time. This option is available by right-clicking your mouse and choosing **Delta-P vs T Plot** from the menu that opens.

Frequency

Displays the frequency and period of a periodic waveform. The period is calculated as the difference in time between two consecutive edges of the same polarity of a waveform. The frequency is calculated as:

$$\frac{1}{period}$$

You can also measure waveform frequency or parameter versus parameter or index values by right-clicking a Frequency measurement on a multi-member waveform and choosing one the following menu items from the menu that opens:

- **Freq vs Param**
- **Period vs Param**
- **Freq vs Index**
- **Period vs Index**

Note:

This measurement is supported only in Analog X-Y and Logic waveview panels.

Feedback

The following parameters can be set for the Frequency measurement:

Parameter	Description
Locked Level	Locks the measurement Y level at the specified percentage or level (in Y-axis units).
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
User Specified	Uses the min/max Y values you specify in the **Min** and **Max** text fields.
Frequency	Calculates the frequency versus X-axis plot frequency.
Period	Calculates the period versus X-axis plot period.
Range options	• Entire Waveform — (default) measures the entire waveform. • Visible Range — measures the visible range. • User Specified — measures the defined range.
Tolerance Level	Controls the tolerance level for excluding unwanted small rise-fall triggers around the specified ylevel. The tool will ignore the small rise-fall triggers within the specified tolerance range.
Trigger Type	• With **Frequency vs x-axis option**, controls the versus x-axis plot with **Frequency** or **Period** as: • Either — plot with EITHER • Rise — plot with UP • Fall — plot with DOWN.

Jitter

Displays the histogram of crossing edges with a specified X range or Y range. You must click the right mouse button menu on the measurement to complete the measurement.

Note:

This measurement is supported only in Analog X-Y, Eye Diagram, and 2-D Sweep waveview panels.

Feedback

The following parameters can be set for the Jitter measurement:

Parameter	Description
Locked Level	Locks the measurement Y level at the specified percentage or level (in Y-axis units).
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a measurement Y level that is locked at a specified percentage.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Rising	Measures the rising edge jitter.
Falling	Measures the falling edge jitter.
H-Crossing	Measures edges that cross the horizontal range.
V-Crossing	Measures edges that cross the vertical range.
New WaveView	Plots the jitter histogram on a new waveview window.
Active WaveView	Plots the jitter histogram on the active waveview window.

Overshoot

Calculates the overshoot of a waveform relative to a default or specified topline. The overshoot is calculated as the difference between the maximum point on the waveform and the specified (or calculated) Topline value.

The following parameters can be set for the Overshoot measurement:

Parameter	Description
Percentage	Locks the rise/fall margin threshold at the specified high (H(%)) and low (L(%)) percentages. In signal Y units.
Signal Level	Locks the rise/fall margin threshold at the specified high (H(V)) and low (L(V)) signal levels. In signal Y units.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a rise/fall margin that is locked at a specified percentage range.

Feedback

Parameter	Description
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a rise/fall margin that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Absolute	Specifies the result as "maximum value-high level value."
Percentage	Specifies the result as a percentage of "absolute value/(high level-low level)."
Range options	• Entire Waveform — (default) measures the entire waveform. • Visible Range — measures the visible range. • User Specified — measures the defined range.

Rise/Fall Time

Calculates the risetime and falltime between specified upper and lower levels of a waveform.

Note:

This measurement is only supported in Analog X-Y waveview panels.

The following parameters can be set for the Rise/Fall Time measurement:

Parameter	Description
Percentage	Locks the rise/fall margin threshold at the specified high (H(%)) and low (L(%)) percentages. In signal Y units.
Y Range Options For Percentage Lock Level	• Target Signal Min/Max— Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a rise/fall margin that is locked at a specified percentage range. • Target Signal Topline/Baseline — Uses the topline/baseline from the signal you are measuring. • All Signals — Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a rise/fall margin that is locked at a specified percentage range. • User-specified — Uses the min/max Y values you specify in the Min and Max text fields. • Each Edge Min/Max — Uses the local edge range to determine the low and high thresholds.
Within X Range	Specifies the X range from which the low and high thresholds are determined.

Feedback

Parameter	Description
Show Range Levels	The range levels Min/Max or Topline/Baseline will be shown on the graph with the measurement results.
Trigger Type	Specifies trigger type as Rise, Fall, or Either.
Rise/Fall Time vs. X-axis Plot Option	Calculates the Rise, Fall, or Rise/Fall time versus the X-axis
X-Range options	• Entire Waveform — (default) measures the entire waveform. • Visible Range — measures the visible range. • User Specified — measures the defined range.

Settle Time

Calculates the settle time of a waveform with respect to a default or specified settle level and a specified settle band. The settle time is calculated by searching the waveform from right to left to find the first point that is outside of the settle band. The time that the waveform leaves the settle band is used as the settle time.

The following parameters can be set for the Settle Time measurement:

Parameter	Description
Percentage	Locks the settle level at the specified percentage level (Level(%)). In signal Y units.
Signal Level	Locks the settle level at the specified signal level (Level(V)). In signal Y units.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a settle level that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a settle level that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
% of SettleLevel	Calculates the band value as the specified band value percentage multiplied by the settle level (for example, 5*settle_level).
% of PeaktoPeak	Calculates the band value as the specified band value percentage multiplied by the peak to peak range of the signal (for example, 5*peak_to_peak_range).
Absolute	Calculates the band value as the specified value (in signal Y units).

Feedback

Slew Rate

Calculates the slew rate of a waveform relative to the default or specified topline and baseline levels.

Note:

This measurement is only supported in Analog X-Y waveview panels.

The slew rate is calculated using the difference between the upper and lower levels of a waveform divided by the risetime or falltime of the edge. You can select the upper and lower levels as a percentage of the topline or baseline.

The following parameters can be set for the Slew Rate measurement:

Parameter	Description
Percentage	Locks the rise/fall margin threshold at the specified high (H(%)) and low (L(%)) percentages. In signal Y units.
Signal Level	Locks the rise/fall margin threshold at the specified high (H(V)) and low (L(V)) signal levels. In signal Y units.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a rise/fall margin that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a rise/fall margin that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Slew Rate vs. X-axis Plot Option	Calculates the slew rate values of the **Rise**, **Fall**, or **Rise/Fall** edges vs. the x-axis.
Range options	• Entire Waveform — (default) measures the entire waveform. • Visible Range — measures the visible range. • User Specified — measures the defined range.

Undershoot

Calculates the undershoot of a waveform relative to a default or specified baseline level. The undershoot is calculated as the difference between the minimum point on the waveform and the specified (or calculated) Baseline value.

Feedback

The following parameters can be set for the Undershoot measurement:

Parameter	Description
Percentage	Locks the rise/fall margin threshold at the specified high (H(%)) and low (L(%)) percentages. In signal Y units.
Signal Level	Locks the rise/fall margin threshold at the specified high (H(V)) and low (L(V)) signal levels. In signal Y units.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a rise/fall margin that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a rise/fall margin that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Absolute	Specifies the result as "maximum value-high level value."
Percentage	Specifies the result as a percentage of "absolute value/(high level-low level)."
Range options	• Entire Waveform — (default) measures the entire waveform. • Visible Range — measures the visible range. • User Specified — measures the defined range.

Frequency Domain Measurements

The following Frequency Domain measurements are available:

- Highpass
- Lowpass
- Bandpass
- Stopband
- Gain Margin
- Phase Margin

Highpass

Calculates the corner frequency of a waveform with a highpass shape. The highpass measurement is calculated relative to a default offset or a specified topline and a specified offset.

Feedback

The corner frequency is calculated by searching from the right to the left until the waveform first falls below the measurement level, which is determined by the specified offset value (from the topline).

The following parameters can be set for the Highpass measurement:

Parameter	Description
Percentage	Locks the topline at the specified percentage (Topline(%)). In signal Y units.
Signal Level	Locks the topline at the specified signal level (Topline). In signal Y units.
Auto Detect	Calculates the topline and uses that value.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a topline that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a topline that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Offset	Specifies the offset in Y-axis units.
Show Topline	Displays the topline.
Show Topline+Offset Line	Shows the cross line in a panel.

Lowpass

Calculates the corner frequency of a waveform with a lowpass shape. The lowpass measurement is calculated relative to a default or specified topline and a specified offset.

The corner frequency is calculated by searching from the left to the right until the waveform first falls below the measurement level, which is determined by the specified offset value (from the topline).

The following parameters can be set for the Lowpass measurement:

Parameter	Description
Percentage	Locks the topline at the specified percentage (Topline(%)). In signal Y units.

Feedback

Parameter	Description
Signal Level	Locks the topline at the specified signal level (Topline). In signal Y units.
Auto Detect	Calculates the topline and uses that value.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a topline that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a topline that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Offset	Specifies the offset in Y-axis units.
Show Topline	Displays the topline.
Show Topline+Offset Line	Shows the cross line in a panel.

Bandpass

Calculates the bandwidth of a bandpass-shaped waveform.

The following parameters can be set for the Bandpass measurement:

Parameter	Description
Percentage	Locks the topline at the specified percentage (Topline(%)). In signal Y units.
Signal Level	Locks the topline at the specified signal level (Topline). In signal Y units.
Auto Detect	Calculates the topline and uses that value.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a topline that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a topline that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.

Feedback

Parameter	Description
Offset	Specifies the offset in Y-axis units.
Show Topline	Displays the topline.
Show Topline+Offset Line	Shows the cross line in a panel.

Stopband

Calculates the stopband, low, high, or center frequency level or the level at which the measurement is calculated for a stopband-shaped waveform. This measurement is calculated relative to a default or specified topline level and a specified offset:

```
fcenter = square root of (flow*fhigh)

quality factor = fcenter/bandwidth
```

The following parameters can be set for the Stopband measurement:

Parameter	Description
Percentage	Locks the topline at the specified percentage (Topline(%)). In signal Y units.
Signal Level	Locks the topline at the specified signal level (Topline). In signal Y units.
Auto Detect	Calculates the topline and uses that value.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying a topline that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying a topline that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.
Offset	Specifies the offset in Y-axis units.
Show Topline	Displays the topline.
Show Topline+Offset Line	Shows the cross line in a panel.

Gain Margin

Displays the gain margin of a complex analog waveform in decibels (dB). The gain margin is the difference between the gain of the measured waveform and 0 dB at the frequency where the phase shift is -180-degrees.

This measurement might fail if the phase of the measured waveform does not pass through -180-degrees or if the waveform is not complex.

Phase Margin

Displays the phase margin of a complex analog waveform in degrees. The phase margin is the difference between the phase of the measured waveform and the reference degree at the unity gain frequency. The reference can be either -180-degrees or 0-degree.

This measurement might fail if the magnitude of the waveform does not pass through 0 dB or if the waveform is not complex.

The following parameters can be set for the Phase Margin measurement:

Parameter	Description
-180	Specifies the reference as -180 degrees.
0	Specifies the reference as 0 degrees.

Statistical Measurements

The following Statistical measurements are available:

- Cpk
- Dpu
- Mean
- Mean+3std_dev
- Mean-3std_dev
- Std_dev
- Yield

Feedback

Cpk

Calculates an indicator of the process capability for a waveform relative to specified upper and lower limits. Cpk is calculated using the following equation:

$$\frac{\text{upper-mean}}{3(\text{std_dev})}$$

or

$$\frac{\text{mean-lower}}{3(\text{std_dev})}$$

In this calculation, `mean` represents the mean value of the scatter plot, `upper` and `lower` represent the specification limits you specify, and `std_dev` represents the standard deviations of the scatter plot. When both the `upper-mean` and `lower-mean` values are provided, the smaller result of these two calculations is displayed as the measurement.

The following parameters can be set for the Cpk measurement:

Parameter	Description
Percentage	Locks the upper/lower threshold at the specified high (H(%)) and low (L(%)) percentages. In signal Y units.
Signal Level	Locks the upper/lower threshold at the specified high (H(V)) and low (L(V)) signal levels. In signal Y units.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying an upper/lower threshold that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying an upper/lower threshold that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.

Dpu

Calculates the total number of defects per unit based on the points in a waveform. Given one or both of an upper and lower specification limit, the Dpu is calculated as the area under a normal distribution that falls outside the specification limits. The mean and standard deviation of the normal distribution are equal to the mean and standard deviation of the points in the waveform.

Feedback

The following parameters can be set for the Dpu measurement:

Parameter	Description
Percentage	Locks the upper/lower threshold at the specified high (H(%)) and low (L(%)) percentages. In signal Y units.
Signal Level	Locks the upper/lower threshold at the specified high (H(V)) and low (L(V)) signal levels. In signal Y units.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying an upper/lower threshold that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying an upper/lower threshold that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.

Mean

Calculates the mean value of a waveform, which is calculated using the following equation:

$$\frac{1}{N}\sum_{j=1}^{N} W_j$$

In this calculation, N is the number of points, and array W_j contains the individual points of the waveform.

Mean+3std_dev

Calculates the (mean + 3 standard deviation) value of a waveform.

Mean-3std_dev

Calculates the (mean - 3 standard deviation) value of a waveform.

Std_dev

Calculates the standard deviation of a waveform. This measurement is intended for statistical (discrete) data such as histograms and is calculated using the following equation:

$$\left[\frac{1}{N\text{-}1}\sum_{j=1}^{N}\left(W_j\text{-}\bar{w}\right)^2\right]^{\frac{1}{2}}$$

Feedback

Yield

Calculates the ratio of the number of data points between the y-axis levels Upper and Lower relative to the total number of data points.

The following parameters can be set for the Yield measurement:

Parameter	Description
Percentage	Locks the upper/lower threshold at the specified high (H(%)) and low (L(%)) percentages. In signal Y units.
Signal Level	Locks the upper/lower threshold at the specified high (H(V)) and low (L(V)) signal levels. In signal Y units.
Target Signal	Uses the min/max Y levels from the signal you are currently measuring. This option is only available when specifying an upper/lower threshold that is locked at a specified percentage range.
All Signals	Uses the min/max Y levels from all signals that are currently open. This option is only available when specifying an upper/lower threshold that is locked at a specified percentage range.
User Specified	Uses the min/max Y values you specify in the Min and Max text fields.

Level Measurements

Each Level measurement can be performed over one of the following ranges:

- Entire Waveform — (default) measures the entire waveform.
- Visible Range — measures the visible range.
- User Specified — measures the specified range.

The following Level measurements are available:

- AC Coupled RMS
- Amplitude
- Average
- Baseline
- Maximum
- Minimum
- Peak to Peak

Feedback

- RMS
- Topline
- X at Maximum
- X at Minimum

AC Coupled RMS

Calculates the RMS value of the AC component of a waveform. The AC coupled RMS value is calculated using the following equation:

$$\left[\frac{1}{(x2\text{-}x1)}\int_{x1}^{x2}(w\text{-}\bar{w})^2\,\mathrm{d}\,x\right]^{\frac{1}{2}}$$

In this calculation, `w` represents the waveform as well as its average value. The `x1` and `x2` values are the starting and ending points for the waveform. Add WV as an option for the AC Coupled measurement.

Amplitude

Calculates the difference between the topline and baseline reference levels.

Average

Calculates the average level of a waveform.

Baseline

Calculates the baseline level of a waveform.

Note:

This measurement is automatically calculated when enabled.

Maximum

Calculates the maximum level of a waveform.

Minimum

Calculates the minimum level of a waveform.

Peak to Peak

Calculates the peak to peak value of a waveform.

RMS

Calculates the RMS (Root Mean Square) value of a waveform using the following equation:

$$\left[\frac{1}{(x2\text{-}x1)}\int_{x1}^{x2}(W^2\,\mathrm{d}\,x)\right]^{\frac{1}{2}}$$

In this calculation, *W* represents the waveform, and *x1* and *x2* represent the starting and ending points.

Topline

Calculates the topline level of a waveform. The topline is automatically calculated using the probability density histogram method.

X at Maximum

Displays the x-value corresponding to the maximum value of a waveform.

X at Minimum

Displays the x-value at the minimum value of a waveform.

S Domain Measurements

The following S Domain measurements are available:

- Damping Ratio
- Frequency Value
- Natural Frequency
- Quality Factor

Damping Ratio

Calculates the damping ratio of a waveform as -real/mag.

Frequency Value

Displays the frequency value at the point of a complex waveform on a Nyquist (or Nichols) plot.

Feedback

Natural Frequency

Calculates the natural frequency of a waveform as the absolute value of an argument. The natural frequency is calculated using the following equation:

$$\text{natural frequency} = \sqrt{(\text{real}^2 + \text{imag}^2)}$$

Quality Factor

Calculates the quality factor of a waveform. This measurement is calculated as:

$$\frac{1}{2(\text{damping ratio})}$$

RF Measurements

The following RF measurements are available:

- P1dB
- IP2
- IP3/SFDR

P1dB

Calculates the 1 dB compression point.

IP2

Calculates the second-order intercept point.

IP3/SFDR

Calculates the third order intercept point.

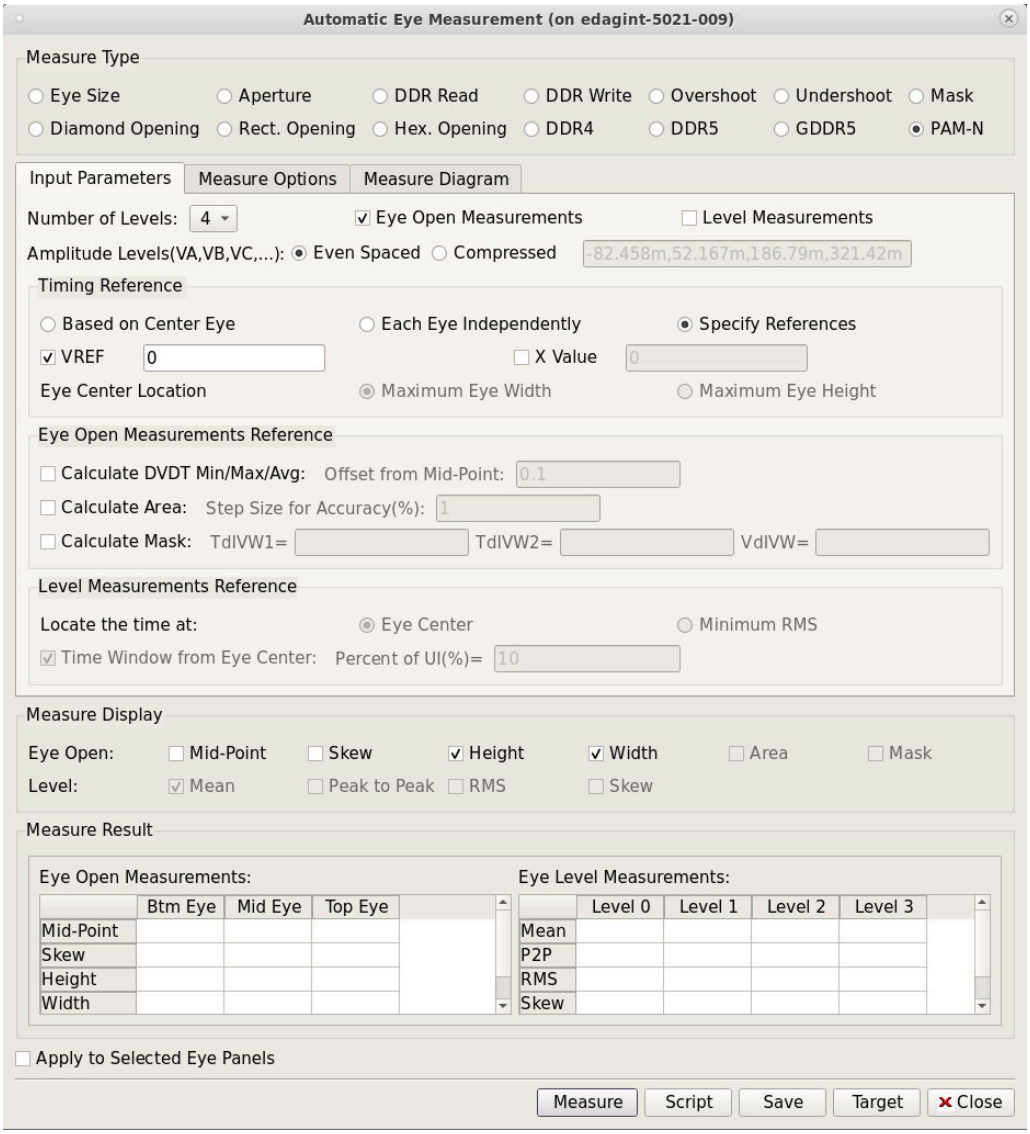
The following parameters can be set for the IP3/SFDR measurement:

Parameter	Description
Calculate SFDR	Specifies that IP3 is calculated as well as SFDR.
Use INTEGRATED_NF	Specifies the Integrated Noise Figure to be used for the SFDR calculation.
Noise Figure (dB)	Specifies the Noise Figure in decibels (dB).
SN ratio (dB)	Specifies the signal-to-noise ratio in decibels (dB).

Parameter	Description
Band width (dB)	Specifies the system Bandwidth in decibels (dB). This parameter is not available when "Use INTEGRATED_NF" is selected.
Integrated NF (dB)	Specifies the Integrated Noise Figure in decibels (dB). This parameter is not available when "Use INTEGRATED_NF" is selected.

Measuring Eye Diagrams

The **Automatic Eye Measurement** tool can take eye opening or aperture measurement on an eye diagram. Right-click an eye diagram, and choose **Measure Eye** from the menu. The **Automatic Eye Measurement** window opens.



The following sections contain information on the measurements available in the Automatic Eye Measurement window:

- Measuring Eye Size
- Measuring Aperture
- Measuring DDR Read
- Measuring DDR Write
- Measuring Overshoot
- Measuring Diamond Openings

- Measuring Rectangle Openings
- Measuring Hexagon Openings
- Measuring DDR4
- Measuring DDR5
- Measuring GDDR5
- Measuring PAM-N
- Measuring Mask
- Measuring Stateyes
- Using the Setup/Hold Results Viewer
- Creating ACE Scripts From DDR Measurements
- Clearing Eye Measurements
- Copying Eye Diagram Settings
- Pasting Eye Diagram Settings
- Measuring DDR3 Eye Diagrams
- Measuring DDR4 Eye Diagrams

The same eye measurement functions are also available as ACE functions for batch mode measurements. Refer to the *Analysis Command Environment Reference (ACE) Reference Manual* for details.

Measuring Eye Size

The following eye size measurement options are available:

Option	Description
VREF, **AC**, and **DC** Values	The VREF, AC, and DC values use for the eye measurement.
Use DVDT Levels	Defines the lower and upper voltage levels for the DVDT measurement.

Feedback

Option	Description
Use Time Window from Eye Center	Reports and plots measurement results in the Eye Size measurement naming convention using the specified percentage value, where: • `FUSS` is TJ_TX • `OH` is VDIF_AC_TX(max,upper) • `MH` is VDIF_AC_TX(min,upper) • `OL` is VDIF_AC_TX(max,lower) • `ML` is VDIF_AC_TX(min,lower)
Large Font	Sets the font size to large.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

Eye Measurement Methodology

The permanent default value for vref, vdc, and vac is 0.0. The vref, vdc, and vac values calculate the MH, ML, OH, OL, WH, and WL without checking which one is specified.

You can calculate MH and ML using the following method:

```
MH = (lower limit of the upper eye) - (vref+vdc)

ML = (vref - vdc) - (upper limit of the lower eye)
```

You can calculate OH and OL using the following method:

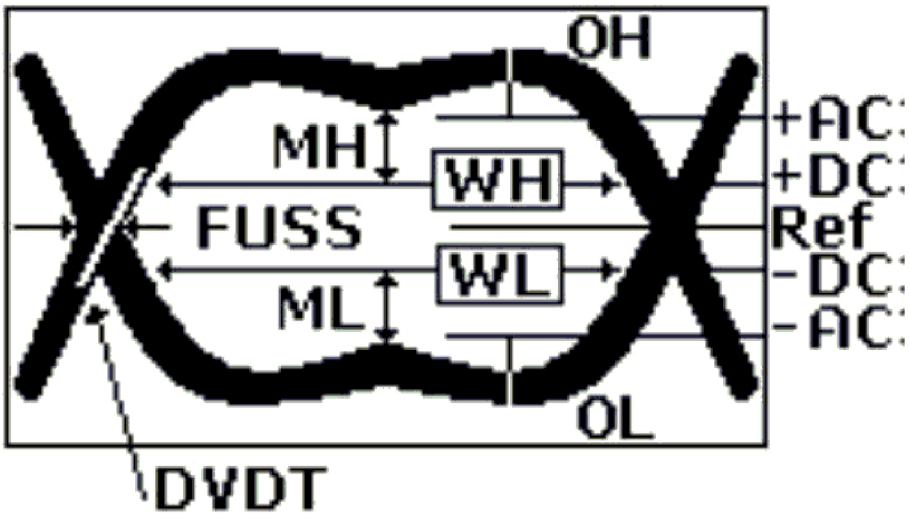
```
OH = (upper limit of the upper eye) - (vref+vac)

OL = (vref-vac) - (lower limit of the lower eye)
```

WH is the eye opening width at level (vref+vdc). WL is the eye opening width at level (vref-vdc).

Feedback

The following figure shows how the expressions mentioned are used to calculate eye diagram measurements:



Measuring Aperture

The following eye aperture measurement options are available:

Option	Description
Use Wider Aperture for VDC-Ringback Divided Eye	Evaluates the width for two openings on two sides of a glitch and uses the wider one as the measurement result. If not enabled, the aperture measurement always starts after any VDC crossing ringback.
Use 2-Box Aperture with no VAC Crossing Before	Sets the value you enter in the Tminac as the time for the width of the first box (the left box). The first box is free of VAC glitches, while the 2nd box (right box) is free of VDC glitches.
Use First Signal as T-Budget Setup/Hold Reference	Sets the first signal in the panel as the reference signal for setup/hold measurement.
VREF, **AC**, and **DC** Values	The VREF, AC, and DC values used for the eye measurement. You can enter multiple values for VREF by separating them with colons (:), semicolons (;), or commas (,).
Trigger Clock Delay	Sets the trigger clock delay value you enter in the Trigger Clock Delay text box. This option is disabled if you enable the **Delay Clock to Aperture Center** option.
Delay Clock to Aperture Center	Sets the trigger clock delay to the aperture center value.

Feedback

Option	Description
Min/Max Display	Displays the min and max delay from the eye.
Skew	Displays the skew measurement on the eye.
Aperture	Displays the aperture.
Aperture Violation	Displays aperture violations.
Jitter	Displays the point-to-point and RMS jitter as a histogram, which is plotted next to the Vref crossing area.
Timing Budget	Displays the setup and hold measurements.
Guide Grid	Displays the guide grid.
Large Font	Sets the font size to large.
Aper	Sets the line size and color for the aperture.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

Measuring DDR Read

The following DDR read measurement options are available:

Option	Description
VREF, **AC**, and **DC** Values	The VREF, AC, and DC values used for the DDR read measurement.
Guide Grid	Displays the guide grid.
Large Font	Sets the font size to large.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

Measuring DDR Write

The following DDR write measurement options are available:

Option	Description
VREF, **AC**, and **DC** values	The VREF, AC, and DC values used for the DDR write measurement.
Guide Grid	Displays the guide grid.
Large Font	Sets the font size to large.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

Measuring Overshoot

The following overshoot measurement options are available:

Option	Description
VREF value	The VREF value used for the overshoot measurement.
Large Font	Sets the font size to large.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

Measuring Diamond Openings

The following diamond opening measurement options are available:

Option	Description
Accuracy(%)	The accuracy percentage of the diamond opening measurement.
Large Font	Sets the font size to large.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

Feedback

Measuring Rectangle Openings

The following rectangle opening measurement options are available:

Option	Description
VREF Value	The VREF value used for the rectangle opening measurement.
Large Font	Sets the font size to large.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

Measuring Hexagon Openings

The following hexagon opening measurement options are available:

Option	Description
VREF Value	The VREF value used for the hexagon opening measurement.
Large Font	Sets the font size to large.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

Measuring DDR4

The following DRR4 measurement options are available:

Option	Description
Input Parameters	
VREF	The VREF value used for the DDR4 measurement.
AC	The AC value used for the DDR4 measurement.
DC	Sets the DC threshold for eye diagram measurements.
VIHL_AC(min)=	Select to set the VIHL_AC(min) value for calculating tr2 measurements. If a VIHL_AC(min) value is not specified, by default the WaveView tool uses the calculated VIHL_AC value.

Feedback

Option	Description
Clock Slew	Select to set the clock slew VIH and VIL values.
Enable	Find the Vcent from VRef sweeping using the **Start**, **Stop**, and **Step** values you enter. The VREF value is not used when you enable this option.
Align Bits By	Aligns all bits by **Aperture** or **Eye Open at Vcent**.
Report Margins vs. Mask	Measures the margins versus the mask defined by the `CX_DRTAB_FILE` environment variable.
DDR Spec	The a DDR specification from the drop-down to use for the measurement.
Detailed Results button	See Using the Setup/Hold Results Viewer for information on the Setup/Hold Results Viewer, which you can access via the **Detailed Results** button.
Measure Options	
Trigger Clock Delay	Sets the trigger clock delay value you enter in the Trigger Clock Delay text box. This option is disabled if you enable the **Delay Clock to Aperture Center** option.
Delay Clock to Aperture Center	Select to set the trigger clock delay to the aperture center value.
Measure Display	
Skew	This option is enabled by default and cannot be disabled.
Aperture	Displays the aperture.
Guide Grid	Displays the guide grid.
Large Font	Sets the font size to large.
Aper	Sets the line size and color for the aperture.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
VIHL_AC	Displays vertical bar and value.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

Feedback

Measuring DDR5

The following DRR5 measurement options are available:

Option	Description
Input Parameters	
VREF Value	The VREF value used for the DDR5 measurement.
AC Value	The AC value used for the DDR5 measurement.
Enable	Find the Vcent from VRef sweeping using the **Start**, **Stop**, and **Step** values you enter. The VREF value is not used when you enable this option.
Align Bits By	Aligns all bits by **Aperture** or **Eye Open at Vcent**.
Calculate Mask	Calculates mask using the **TdIVW1**, **TdIVW2**, and **VdIVW** values you enter.
Display Jitter on New Panel	Displays jitter on a new panel.
Measure Options	
Trigger Clock Delay	Sets the trigger clock delay value you enter in the Trigger Clock Delay text box. This option is disabled if you enable the **Delay Clock to Aperture Center** option.
Delay Clock to Aperture Center	Sets the trigger clock delay to the aperture center value.
Measure Display	
Min/Max delay	Displays the min and max delay from the eye.
Aperture	Displays the aperture.
Jitter	Displays the jitter.
Guide Grid	Displays the guide grid.
Large Font	Sets the font size to large.
Aper	Sets the line size and color for the aperture.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
VIHL_AC	Displays vertical bar and value.
Hex. Mask Spec	Displays the hexagonal spec mask.

Feedback

Option	Description
Max Hex. Mask	Displays the max hex spec mask.
Rect. Mask Spec	Displays the rectangular spec mask.
Max Rect. Mask	Displays the max rectangular spec mask.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

Measuring GDDR5

The following GDRR5 measurement options are available:

Option	Description
VREF, **DC**, and **AC** Values	The VREF, DC, and AC values used for the GDDR5 measurement.
Detailed Results button	See Using the Setup/Hold Results Viewer for information on the Setup/Hold Results Viewer, which you can access via the **Detailed Results** button.
Trigger Clock Delay	Sets the trigger clock delay value you enter in the Trigger Clock Delay text box. This option is disabled if you enable the **Delay Clock to Aperture Center** option.
Aperture	Displays the aperture.
Jitter	Displays the jitter.
Guide Grid	Displays the guide grid.
Large Font	Sets the font size to large.
Aper	Sets the line size and color for the aperture.
Time Unit	Sets the time unit to **Auto**, **ns**, or **ps**.
Apply to Selected Eye Panels	Applies measurements to all selected eye panels.

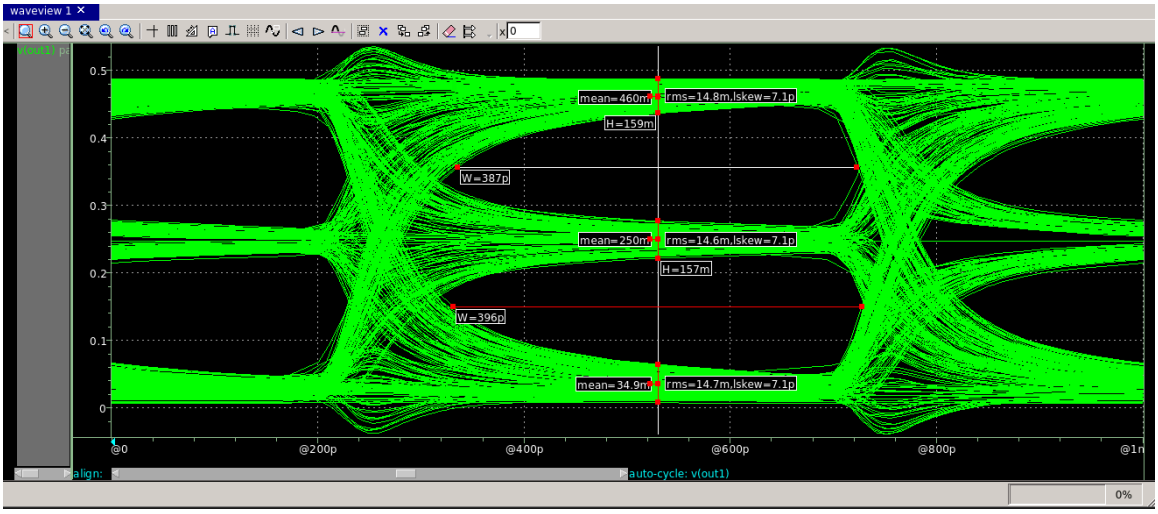
Measuring PAM-N

Select the PAM-N eye measurement in the **Measure Type** section of the **Automatic Eye Measurement** dialog of the GUI.

Feedback

The PAM-N measurement measures a pulse amplitude modulated signal with N amplitude levels (0, 1, 2, ..., n-1). Currently, PAM3, PAM4, and PAM6 measurements are supported. PAM3 measurement measures a pulse amplitude modulated signal with three amplitude levels (0, 1, 2) and two eye opens— Bottom Eye and Top Eye. PAM4 measurement measures a pulse amplitude modulated signal with four amplitude levels (0, 1, 2, 3) and three eye opens— Bottom Eye, Mid Eye, and Top Eye. PAM6 measurement measures a pulse amplitude modulated signal with six amplitude levels (0, 1, 2, 3, 4, 5) and five eye opens— Eye 0, Eye 1, Eye 2, Eye 3, and Eye 4.

The following figure shows an example for PAM3 measurement:



The options available in the **Automatic Eye Measurement** dialog are as follows:

Input Parameters:

The following measurements are supported for PAM-N:

- **Number of Levels** — specify the number of amplitude levels of the measured signal. Currently, measuring signals with 3, 4, and 6 levels are supported.
- **Eye Open Measurements** — for slicer level based receiver, eye height/width are important:
  - Mid-point — time and voltage level of the eye center of each eye
  - Skew — time differences between the individual eye centers and eye center (at maximum height) of the center eye
  - Height — vertical opening of each eye open at certain y level, x value, or eye center

  - Width — horizontal opening of each eye open at certain y level, x value or eye center
  - Area — outline of the eye area that is calculated

- **Level Measurements** — for ADC based receiver, level variation in constellation form is most important
  - Mean — amplitudes at specified x value, eye center or minimum thickness for each level
  - Thickness:
    - Peak-to-peak — Peak-to-peak values at specified x value, eye center, or minimum RMS for each level
    - RMS — RMS values at specified x value, eye center, or minimum RMS from each level
  - Skew — time differences at eye center or minimum thickness to the eye center of the center eye for each level
  - RLM — Level separation mismatch ratio
- **Amplitude Levels (VA,VB,VC,VD)**:
  - Even Spaced — voltage amplitude levels are even spaced. By default, amplitude levels are automatically detected assuming the levels are even spaced.
  - Compressed — voltage amplitude for various levels are too high or too low. In this case, you must specify the amplitude levels.

Timing Reference — reference values/measurements are based on:

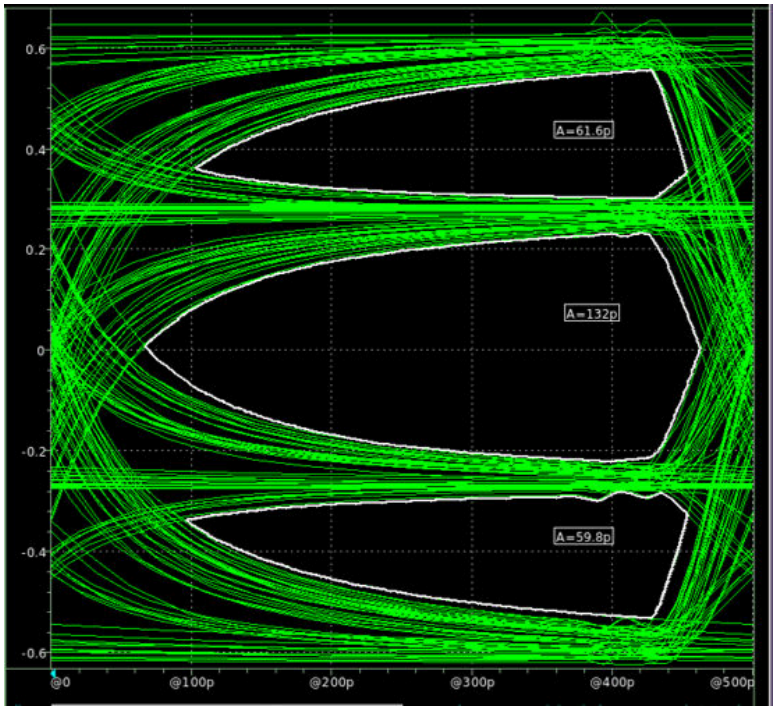
- Based on Center Eye — time reference for all eye opens is based on eye center of center eye
- Each Eye Independently — time reference for each eye open is based on eye center of each eye itself
- Specify References — time reference for eye open is based on VREF specified. If required, Select X value to specify

Eye Open Measurements Reference — reference values/measurements are based on:

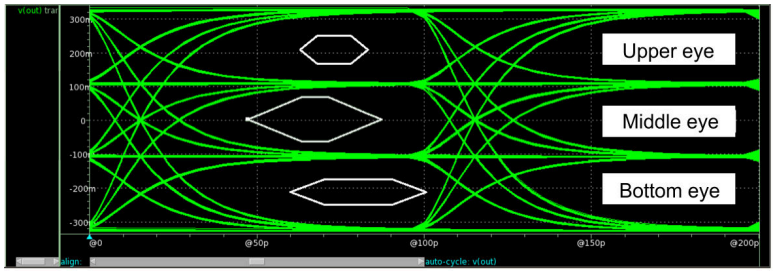
- Calculate DVDT Min/Max/Avg — select to automatically measure the DVDT values for minimum, maximum, and average slew rates of PAM4 eyes. Default- Unselected
- Offset from Mid-Point — specify the offset from mid-point to calculate DVDT values of PAM4 eyes.

Feedback

- Calculate Area — select to measure the area and display the outline of the area for each eye opening.

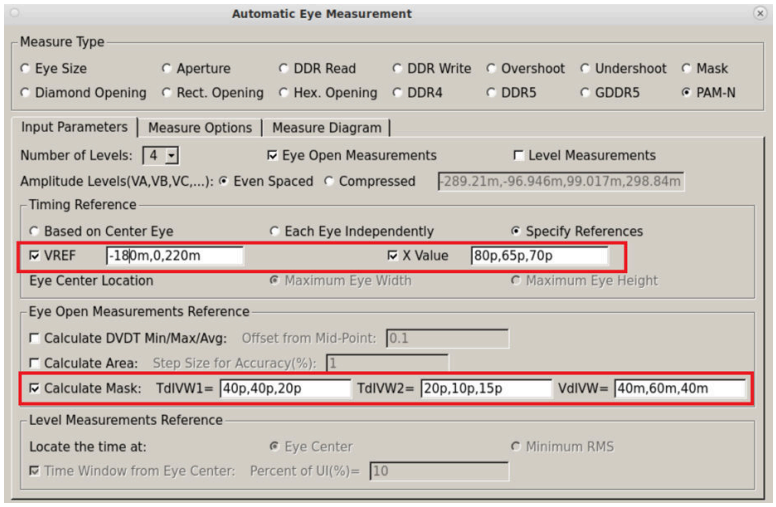


- Step Size for Accuracy (%) — specify the step size for accuracy to calculate the area of the eye diagram.
- Calculate Mask — select to specify the shape of the mask and reference X/Y values individually for each eye opening. The masks are shown for each eye opening.



Feedback

When setting up mask measurement, first specify the values for the bottom eye, then the middle eye, and then for the upper eye.



Eye Center Location — determine the center of each eye:

- Maximum Eye Width — center of the eye is at the mid point of maximum width or width at specified y level
- Maximum Eye Height — center of the eye is at the mid point of maximum height or height at specified x value

Level Measurements Reference — determine the time/x value for level measurements:

- Eye Center — time of the level is the average of the centers of the adjacent eyes
- Minimum RMS — time of the level is at the minimum level RMS
- Time window used to determine the mean of the level and minimum RMS, percentage of UI

Measure Display — displays the results for eye open or level measurements based on selected measurements.

Measure Result — results tables for eye open/level measurements, with RLM result shown on the top of the Eye Level Measurements results table. This table extends by 9 rows for Open Eye Measurement if DVDT is calculated.

Apply to Selected Eye Panels — applies the measurement to all selected eye panels.

Feedback

Measuring Mask

Select the **Mask** eye measurement in the **Measure Type** section of the **Automatic Eye Measurement** dialog of the GUI.

The **Mask** measurement measures the best VREF for the specified mask where the largest vertical margin is found. The measurement returns the best VREF, the largest Vertical Margin, the UI time value, and the voltage value on the eye diagram where the largest Vertical Margin is found.

The following mask measurement options are available:

Option	Description
VREF	The VREF value to place the mask.
Enable	Find the best VREF from vref sweeping using the Start, Stop, and Step values. The **VREF** setting is not used when you enable this option.
Accuracy(% of UI)	The percentage value used to find the largest Vertical Margin at a certain VREF level.
TdIVW1	Mask specification, hexagon mask center width.
TdIVW2	Mask specification, hexagon mask top/bottom width.

Measuring Stateyes

The following stateye measurement options are available:

Option Category	Option	Description
Bathtub/Jitter Curve	**BathtubV/JitterV VREF**	When enabled, uses the BathtubV/JitterV VREF value you specify and plots the BathtubV curve.
	BathtubT/JitterT TREF	When enabled, uses the BathtubT/JitterT TREF value you specify.
	Show BathtubT/JitterT next to BER on the right	When enabled, displays the BathtubT/JitterT measurement results to the right of the BER plot.

Feedback

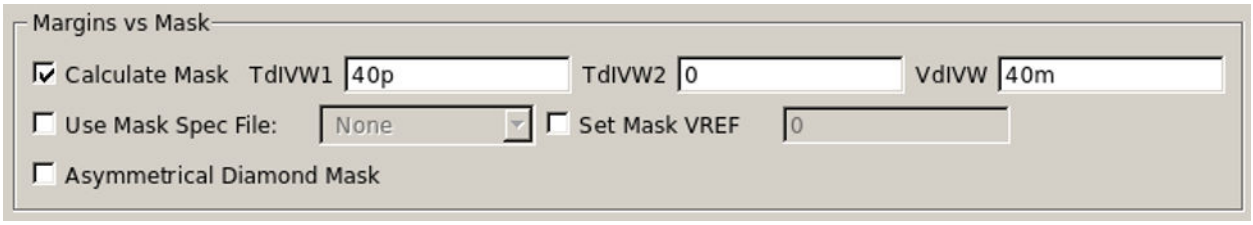
Option Category	Option	Description
BER Contour	**BER Levels**	Performs a BER contour plot based on the BER levels you specify. **Note:** BER contour can be displayed on the top of a StatEye eye diagram without changing the color grade. Therefore, making the BER contour visible.
	Show BER Contour on	Displays the BER contour on a **BER Eye** or in a **New Panel**.
DDR4/DDR5	**VREF** and **AC**	The VREF and AC DDR4 values.
	Find Vcent from VRef Sweeping	When you click the **Enable** check box, a Vcent measurement from a VRef sweep is performed based on the **Start**, **Stop**, and **Step** values you specify.
	VIHL_AC	Displays the VIHL_AC value in the measurement results, which is the biggest vertical open of the eye. The VIHL_AC measurement scans the eye opening of the average Y value from the left to the right with an accuracy of 1% to find the biggest vertical opening of an eye.
	TdiVW/TclVW_TOTAL	Displays the TdiVW/TclWV_TOTAL value in the measurement results, which is the biggest horizontal opening of an eye. The TdiVW/TclWV_TOTAL measurement scans the eye opening of the average Y value from the top to the bottom with an accuracy of 1% to find the biggest horizontal opening of an eye.
	TdiPW/TclPW	Displays the TdiPW/TclPW value in the measurement results, which is the aperture at Vref (where AC=0). You can specify the Vref value directly or use a Vref sweep (in which case Vref is also Vcent).
	Signal Offset	Displays the signal offset value in the measurement results.

Feedback

Option Category	Option	Description
	Margins vs Mask	**Calculate Mask**: The input mask configuration for diamond and hexagonal eye masks. **Use Mask Spec File**: You can load the mask specification file here. This is the same mask file used for a regular eye. Refer to Sample DDR4 Specification File. **Set Mask VREF**: This sets the vertical center of the mask at a certain VREF level. **Asymmetrical Diamond Mask**: Click the check box when using an asymmetrical diamond mask. For more information, refer to Margins vs Mask Measurements.
Jitter	**Jitter(P2)**	Measures the peak-to-peak jitter.
	Jitter(RMS)	Measures the root mean square jitter.
PAM-N Eye Open Measurements	**Mid-Point**	Measure the time and voltage level of the eye center of each eye.
	Skew	Measures the time differences between the individual eye centers and the eye center (at maximum height) of the center eye.
	Height	Measures the vertical opening of each eye open at a certain y level, x value, or eye center.
	Width	Measures the horizontal opening of each eye open at a certain y level, x value, or eye center.
PAM-N Eye Level Measurements	**Mean**	Measures the amplitudes at the specified x value, eye center, or minimum thickness for each level.
	Peak to Peak	Measures the peak-to-peak values at the specified x value, and eye center.
	RMS	Measures the RMS values at the specified x value, eye center, or minimum RMS from each level.
	Skew	Measures the time differences at an eye center, or the minimum thickness to the eye center of the center eye for each level.
	RLM	Measures the level separation mismatch ratio.

Feedback

Margins vs Mask Measurements



The following measurements are done in the **Margins vs Mask** section:

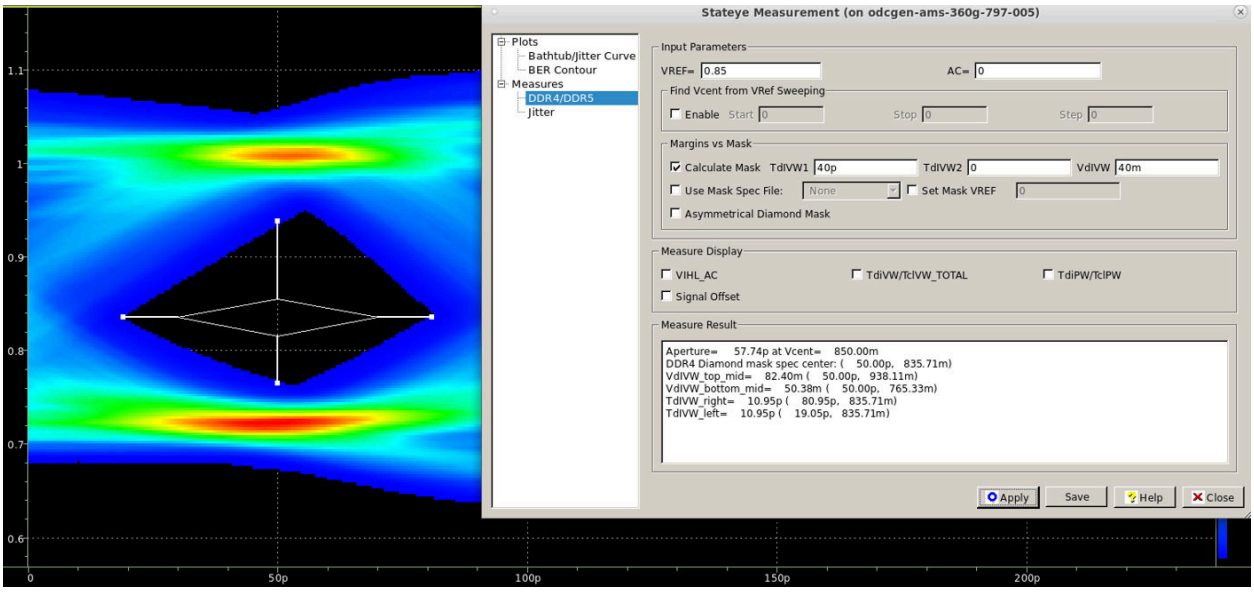
- Margins vs DQ and CA Diamond Mask
- Margins vs DQ Hexagonal Eye Mask

Margins vs DQ and CA Diamond Mask

This measurement is done in the following manner:

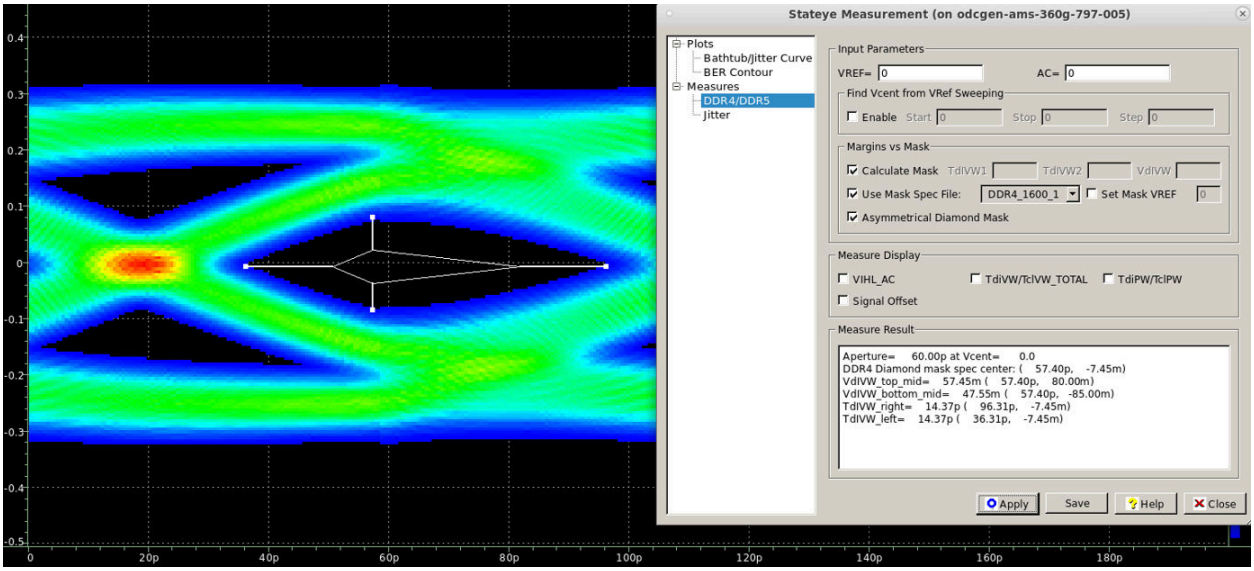
- Y center is the VREF level of the maximum horizontal eye open, or the specified VREF level (using **Set Mask VREF**).
- X center is:
  - The mid-point of the width at the VREF level for a symmetrical diamond mask.
  - The x location at the maximum vertical eye opening for an asymmetrical diamond mask.

The input mask configuration is as shown in the following image.



Feedback

You can load a specification file and use asymmetrical mask as shown in the following image.



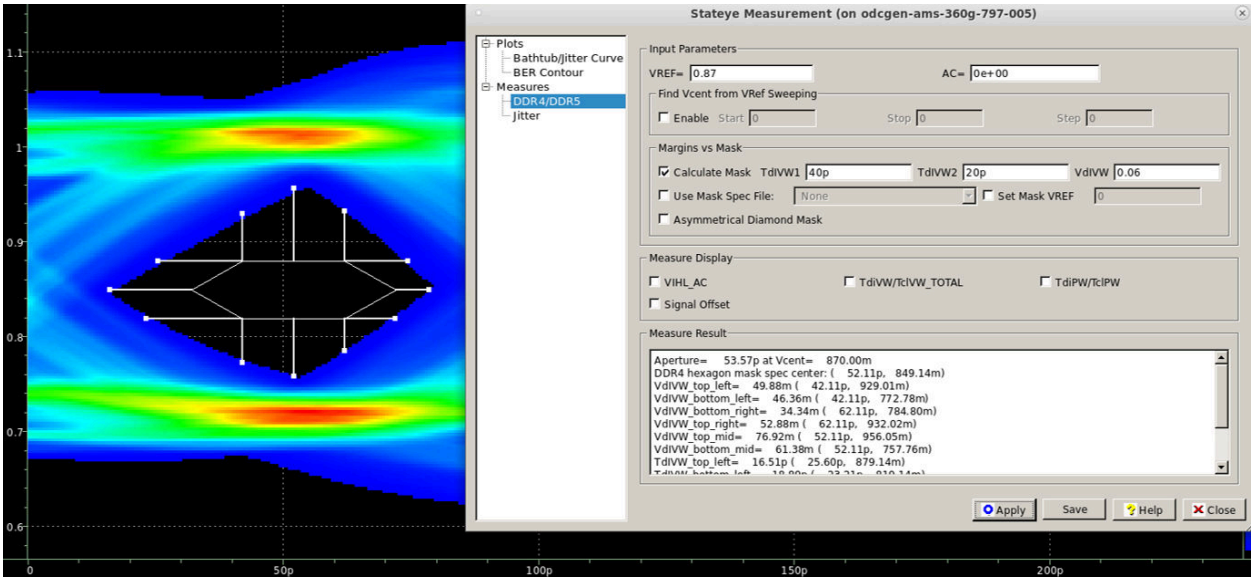
Margins vs DQ Hexagonal Eye Mask

This measurement is done in the following manner by training the Y and X positions to find the maximum summary of all the margin values of the six corners of the mask:

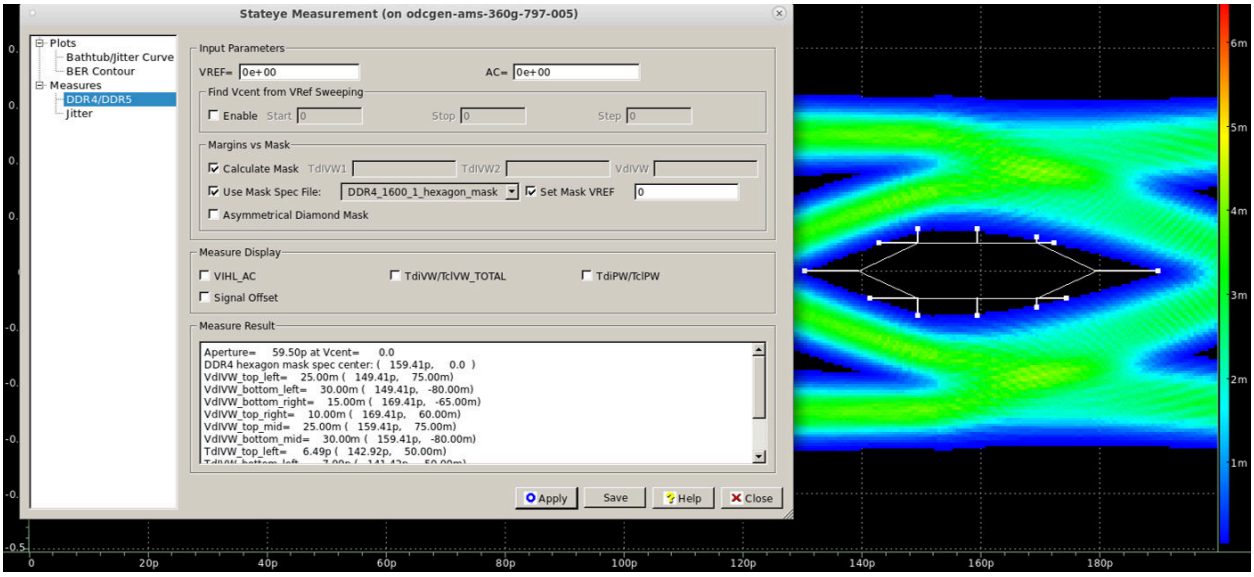
1. Find the maximum horizontal opening and VREF level.
2. Train the X position by placing the mask at the VREF level found earlier and move the mask along VREF horizontally to find the maximum sum of vertical margins.
3. Train the Y position by placing the mask at the X position found in step 2 and move the mask along the X position vertically to find the maximum sum of horizontal margins.

The input mask configuration is as shown in the following image.

Feedback



You can load a specification file as shown in the following image.



Using the Setup/Hold Results Viewer

To view the Setup/Hold Results Viewer:

1. Click the **Detailed Results** button in the Automatic Eye Measurement window.

   The Setup/Hold Results Viewer window opens with the setup and hold results displayed.

2. Click one of the following display category types to display the results you want to see:
   - **Setup/Hold**
   - **Setup+Hold**
   - **Setup**
   - **Hold**

   If you use a derate table, the margin, measured, and required values are reported, and the two corresponding rows that have the minimum setup and minimum hold margin are highlighted in yellow.

   If you do not use a derate table, the minimum setup or hold times are reported, and the two corresponding rows that have the minimum setup or hold times are highlighted in yellow.
3. (Optional) Click the **Data Slew Only** check box to limit the display to data slew values only.

   The worst slew values are reported at the bottom of the table. Exported results are also limited when this option is enabled.
4. (Optional) Click the **Export** button to export the selected results to a *.csv file.

Creating ACE Scripts From DDR Measurements

To save an ACE script from the a DDR measurement you currently have set up, click the **Script** button in the Automatic Eye Measurement window.

For each DDR evaluation, the equivalent ACE TCL script is generated in the saved file in lines that are commented out. You can uncomment them to use these lines.

Clearing Eye Measurements

Measurement results are displayed on top of the eye diagrams. Right-click an eye diagram, and choose **Clear Measure** from the menu that opens to clear the measure results.

Copying Eye Diagram Settings

Eye diagram settings can be copied and reused on another eye diagrams. **Right-click** an eye diagram, choose **Copy Settings** from the menu that opens to copy the settings.

Feedback

Pasting Eye Diagram Settings

Copied eye diagram settings can be applied on top of another eye diagram by performing the following steps:

1. Open the eye diagram you want to apply the copied settings.
2. **Right-click** on the eye diagram, choose **Paste Settings** from the menu. The **Copying Eye Configuration** dialog opens.
3. Choose an eye configuration setting to apply.
4. Select the **Measurement Settings** check box to apply measurements settings along with the selected eye configuration setting.
5. Click OK.

Measuring DDR3 Eye Diagrams

Note:

A WaveView ADV license is required for this feature.

The following DDR3 Eye Diagram panel measurements are available:

- Derate-based setup and hold measurements
- tVAC measurements
- Ringback measurements

You can use these measurements during the design phase to create specifications from a behavioral model or for postsimulation verification.

Measuring Derate-Based Setup and Hold Times

This section contains information on the following topics:

- Calculating Address, Command, Data Setup, and Hold Derating Values
- Setting Up DDR3 Measurements
- Viewing Derating Measurement Output

Calculating Address, Command, Data Setup, and Hold Derating Values

The total setup time (tIS) and hold time (tIH) required is calculated by adding the base value of the setup and hold to the derating value. For example:

```
tIS Total= tIS(Base) + tIS
tIH Total= tIH(Base) + tIH
```

Feedback

The base value of the setup or hold times is taken from the JEDEC standard. The following table shows sample setup or hold base values of several DDR3 specifications:

Symbol	Reference	DDR3-800	DDR3-1066	DDR3-1333	DDR3-1600	DDR3-1866	DDR3-2133	Units
tIS(base) AC175	$V_{IH/L(ac)}$	200	125	65	45	-	-	ps
tIS(base) AC150	$V_{IH/L(ac)}$	350	275	190	170	-	-	ps
tIS(base) AC135	$V_{IH/L(ac)}$	-	-	-	-	65	60	ps
tIS(base) AC125	$V_{IH/L(ac)}$	-	-	-	-	150	135	ps
tIH(base) DC100	$V_{IH/L(dc)}$	275	200	140	120	100	95	ps

The derating values of setup or hold times are calculated from the slew rate of the differential CLK and DQS signals, as well as the address, command, and data signals. In the JEDEC standard, the derating value lookup table corresponds to different slew rates of CLK or strobe and address, command, or data signals.

ΔtIS, ΔtIH derating in [ps] AC/DC based AC175 Threshold -> VIH(ac)=VREF(dc)+175mV, VIL(ac)=VREF(dc)-175mV																	
		CK,CK# Differential Slew Rate															
		4.0 V/ns		3.0 V/ns		2.0 V/ns		1.8 V/ns		1.6 V/ns		1.4 V/ns		1.2 V/ns		1.0 V/ns	
		ΔtIS	ΔtIH	ΔtIS	ΔtIH	ΔtIS	ΔtIH	ΔtIS	ΔtIH	ΔtIS	ΔtIH	ΔtIS	ΔtIH	ΔtIS	ΔtIH	ΔtIS	ΔtIH
CMD/ ADD Slew rate V/ns	2.0	88	50	88	50	88	50	96	58	104	66	112	74	120	84	128	100
	1.5	59	34	59	34	59	34	67	42	75	50	83	58	91	68	99	84
	1.0	0	0	0	0	0	0	8	8	16	16	24	24	32	34	40	50
	0.9	-2	-4	-2	-4	-2	-4	6	4	14	12	22	20	30	30	38	46
	0.8	-6	-10	-6	-10	-6	-10	2	-2	10	6	18	14	26	24	34	40
	0.7	-11	-16	-11	-16	-11	-16	-3	-8	5	0	13	8	21	18	29	34
	0.6	-17	-26	-17	-26	-17	-26	-9	-18	-1	-10	7	-2	15	8	23	24
	0.5	-35	-40	-35	-40	-35	-40	-27	-32	-19	-24	-11	-16	-2	-6	5	10
	0.4	-62	-60	-62	-60	-62	-60	-54	-52	-46	-44	-38	-36	-30	-26	-22	-10

Setting Up DDR3 Measurements

To set up a DDR3 measurement:

1. Load a “derating table” by setting the following environment variable before invoking the WaveView tool:

   ```
   setenv CX_DRTAB_FILE <derating_table_file>
   ```

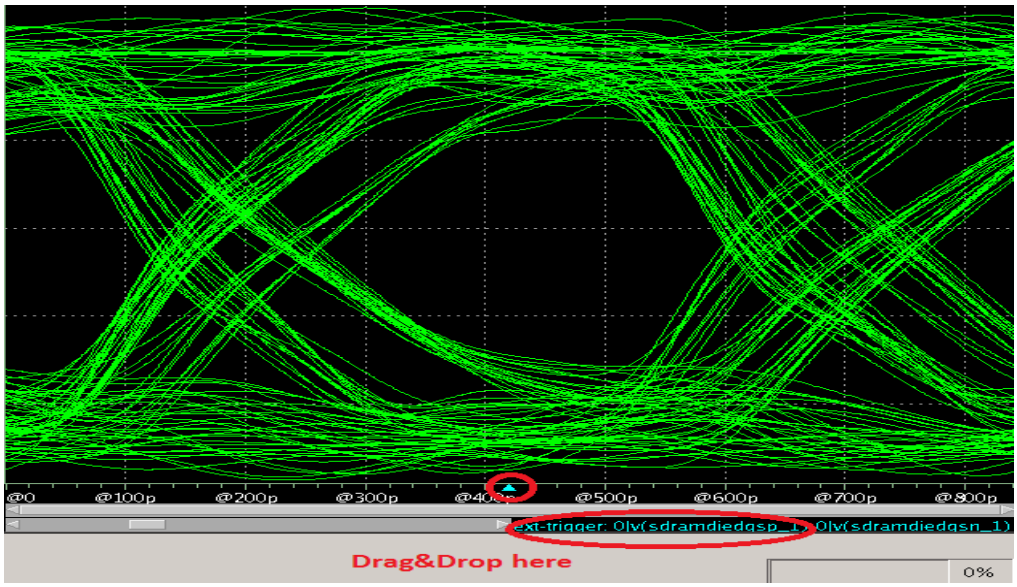
   This file contains the base values of setup andhold, as well aslookup tables for the derating valuesthat are from different DDR3 standards (DDR3-800/1066/ 1333/1600/1866/2133).

2. Launch the WaveView tool.

Feedback

3. Plot the eye diagram for the address, command, ordata signal and specify the CLK/DQS signal as the external trigger for the eye.

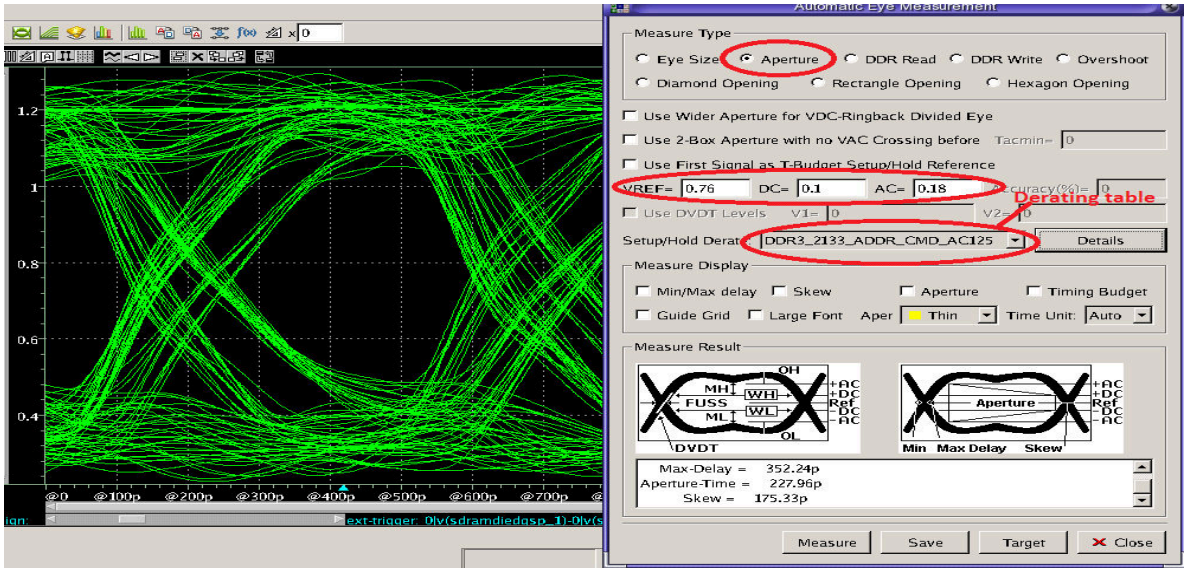
   To create an eye diagram with external trigger, plot the eye diagram of the signal, then drag and drop the external trigger signal in to the lower-right corner of the eye diagram panel.



4. Ensure the blue triangle (trigger point) on the x-axis is at the center of the eye.
5. Right-click on the plotted eye diagram, and choose **Measure Eye** from the menu that opens.

   The **Automatic Eye Measurement** window opens.
6. Specify theVref, DC, and AC values, and chooseaderatingtable from the Setup/HoldDerating menu.

Feedback



7. Click **Measure**.

   The measurement is performed. Click **Details** to display the measurement report.

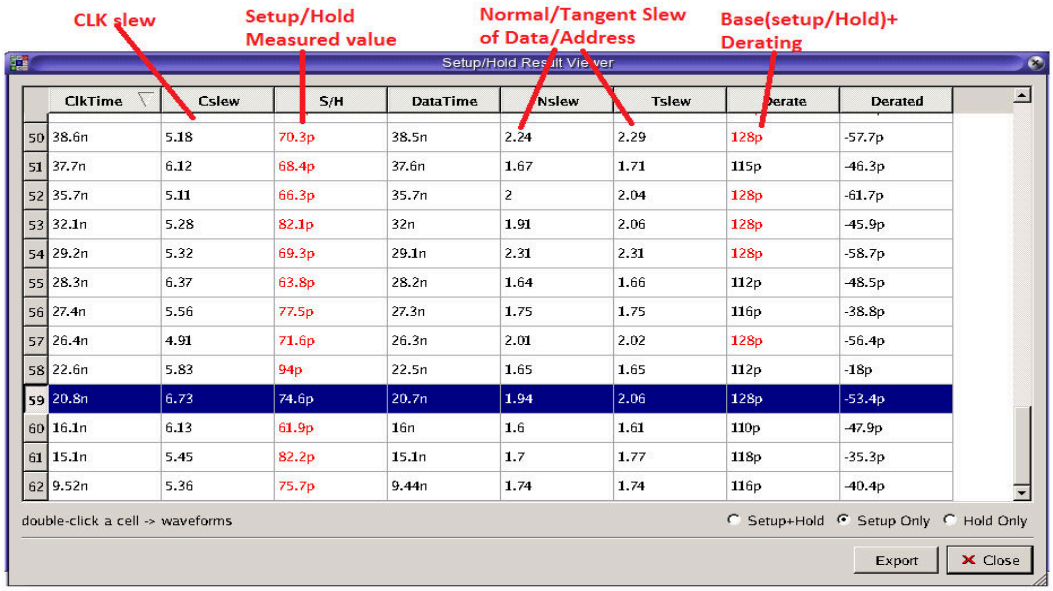
Viewing Derating Measurement Output

Two kinds of measurement results are calculated:

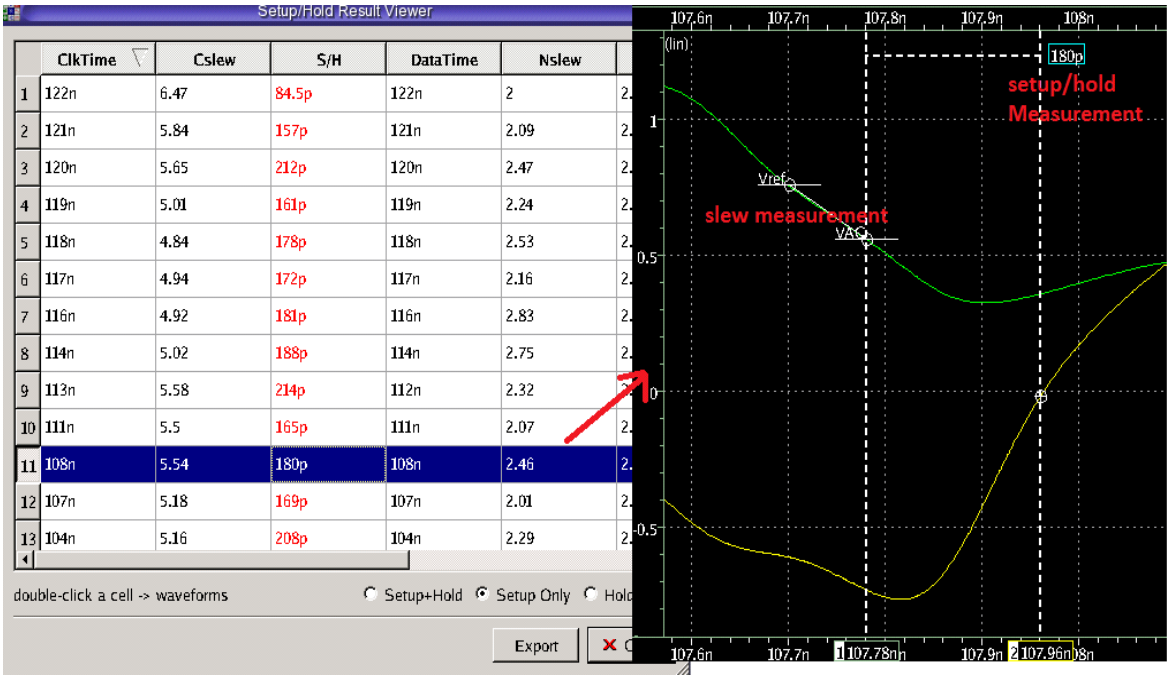
- The required setup or hold values, which are calculated by adding the base value to the derating value. The derating values are obtained after measuring the slew values.
- The actual setup or hold values from the waveform.

If the actual measured value is less than the required value, then it is a setup or hold violation. The output report displays the slew values of signals, as well as the required and actual measured values of setup or hold.

Feedback



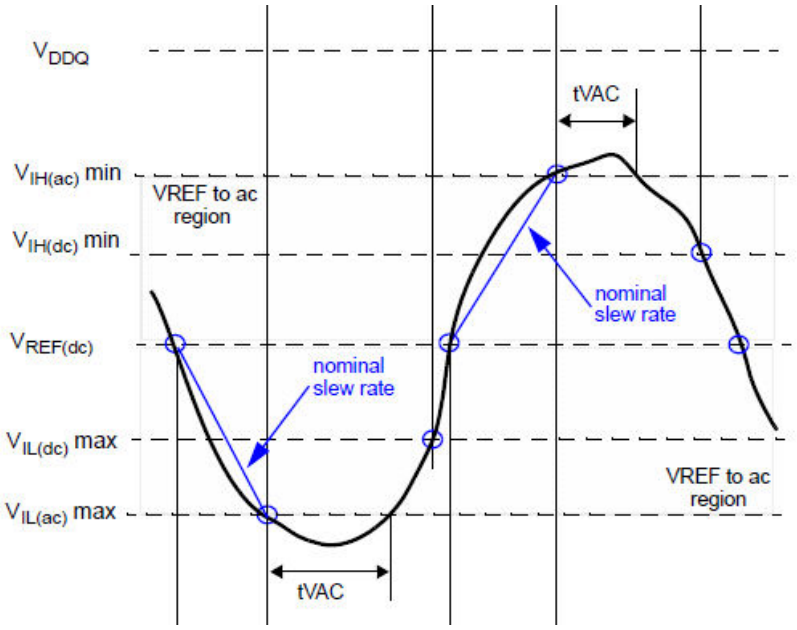
When you double-click a row, the measurements are displayed on a transient signal, which you can use for debugging the setup and hold violations. The transient portion of the signal that created the violation is displayed. The following figure shows the slew measurements and setup or hold measurements:



Feedback

Using the tVAC Measurement

tVAC is the required time above Vih (ac) or Vil (ac) for a valid transition. For a valid transition to occur, a signal has to remain above or below an AC threshold for a period of time. This time span is defined in the JEDEC standard. The tVAC constraint is slew-rate dependent similar to the setup or hold derating.



Setting Up tVAC Measurements

The setup for a tVAC measurement is similar to setup or hold derating. To set up a tVAC measurement:

1. Before starting the WaveView tool, enter the following line to load a file that contains the value of tVAC:

   ```
   setenv CX_DRTAB_FILE <tVAC_value_file>
   ```

   The tVAC values can specify inside derating table file along with setup/hold derate values.

   The measurement is performed after plotting the eye diagram with external trigger signal as in the case of setup or hold derating.

2. Open the WaveView tool.

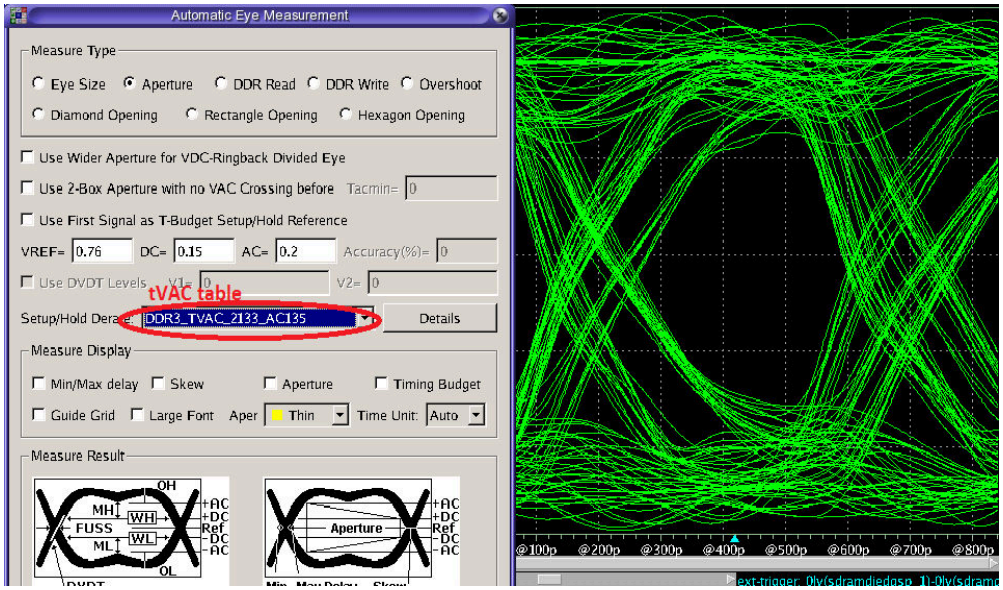
3. Right-click the eye diagram, and choose **Measure Eye** from the menu that opens.

   The **Automatic Eye Measurement** window opens.

Feedback

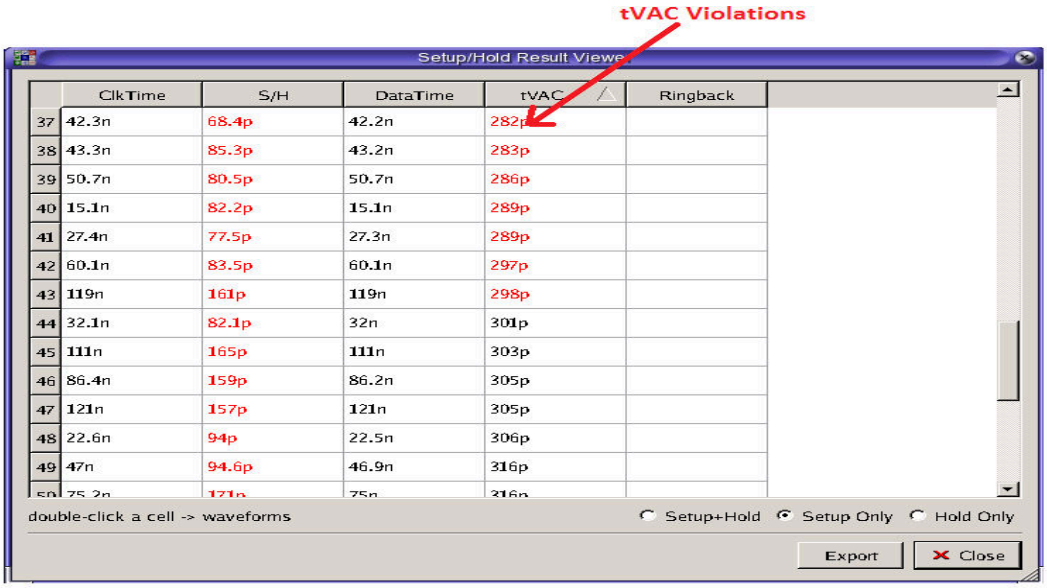
4. Select atVAC table from the **Setup/Hold Derate** menu.
5. Click **Measure**.

   The eye diagram is measured using the tVAC value.

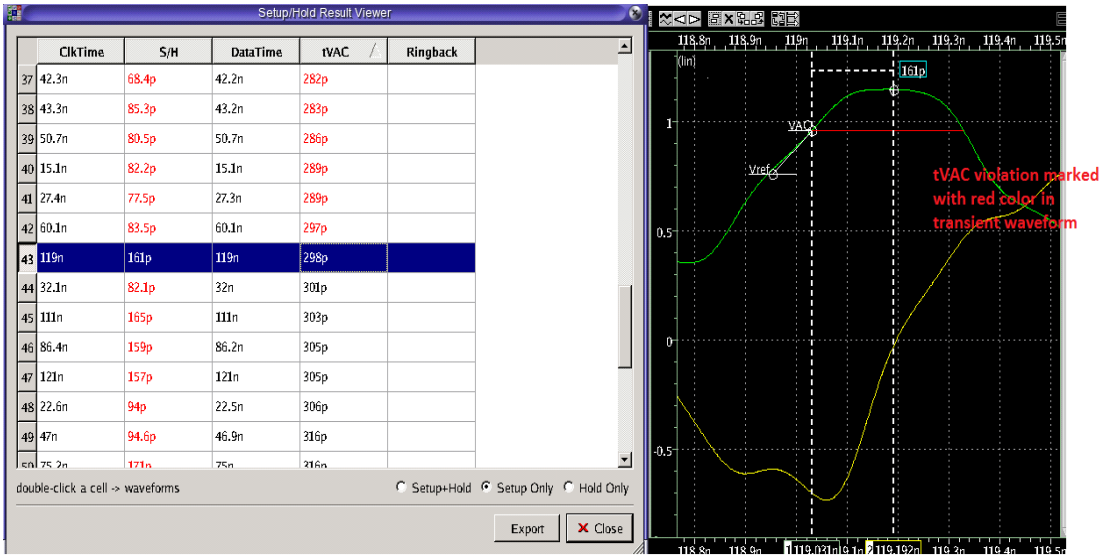


Viewing tVAC Measurement Output

If the measured tVACvalues are smaller than the required value, violations are flagged red in the report that is created after the tVAC measurement is performed.

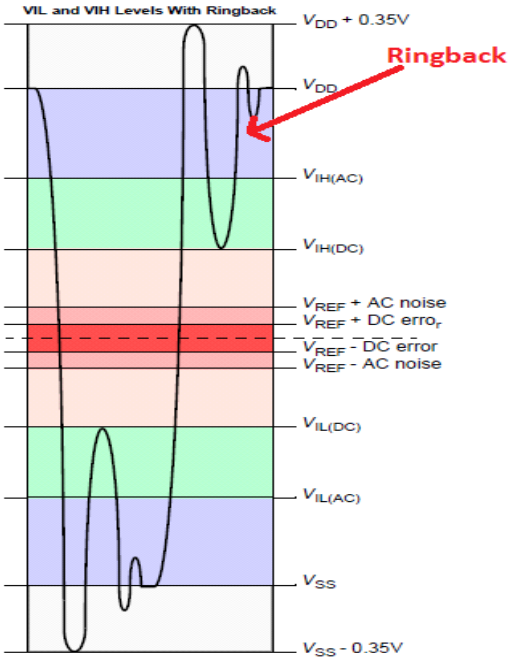


You can click on any of the rows in the report that shows the transient signal and tVAC measurement. In the following figure, the tVAC violations are displayed on a transient waveform:



Measuring Ringback

The ringback measurement calculates how far the top side of a waveform dips below the AC threshold while not crossing the DC threshold. If the waveform moves below the DC threshold, then a ringback violation occurs.



The ringback measurement setup is similar to the tVAC measurement setup in that you also need to provide a tVAC table. The ringback is measured when a signal goes above AC threshold and then goes below the AC threshold values.

Viewing Ringback Measurement Output

Ringback results can occur with or without violations. The output report displays the ringback voltage value. A ringback value without a violation is a signal that goes above the AC threshold, then goes below AC threshold, and finally stays within the AC and DC thresholds. A ringback value with a violationis a signal that goes above the AC threshold and then goes below the DC threshold.

Feedback

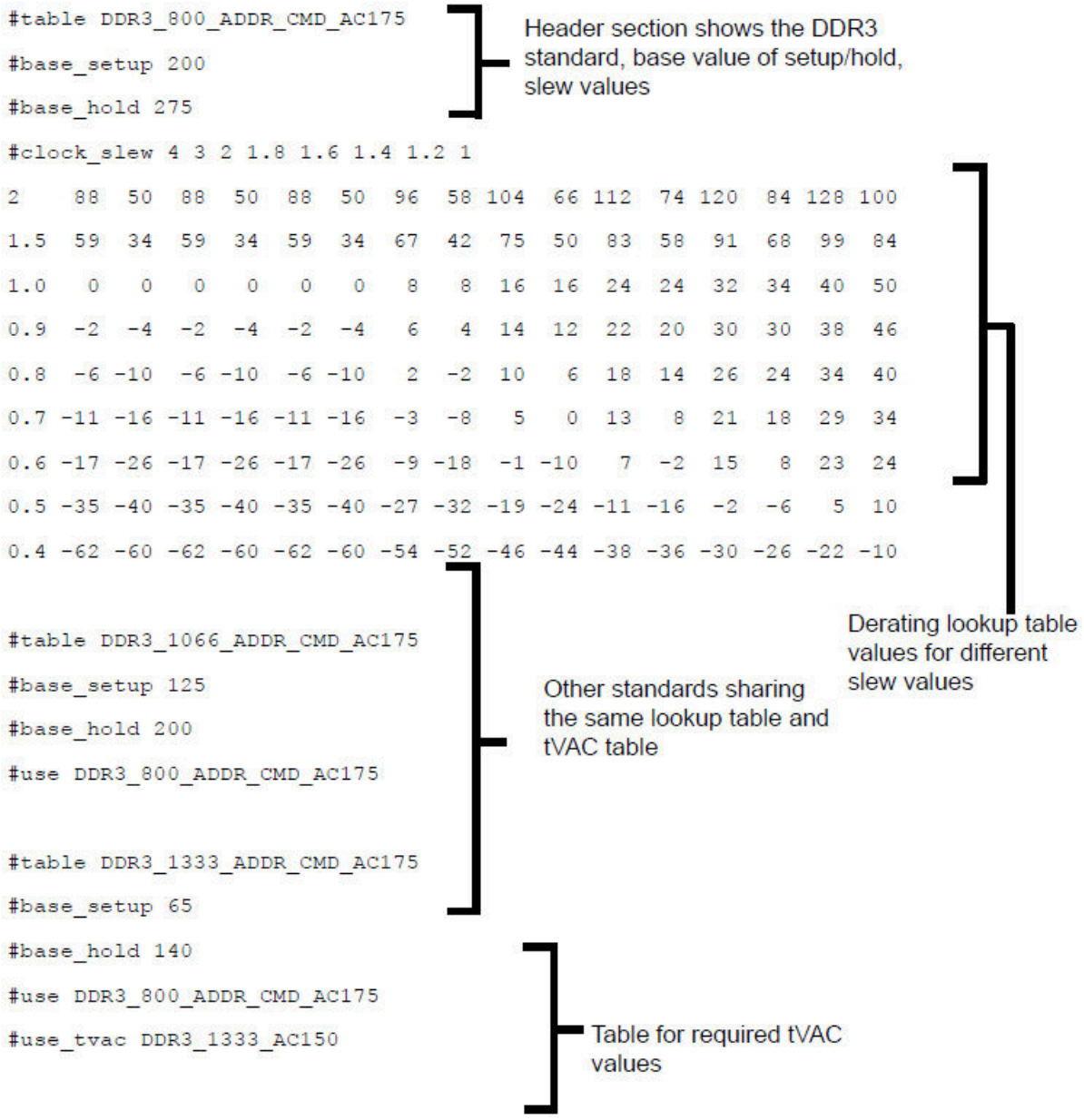
Ringback violations are highlighted in red in the report. When you select a row in the report, the ringback measurement is displayed on a transient signal. The following figure shows the ringback violation displayed on a transient signal in magenta:



Feedback

Sample Derating Table File

A derating table file includes the derating lookup table values for different DDR3 standards, as well as any tVAC required values. The following file shows a sample derating table:



The default units of derate table values are ps for time and V/ns for slew.

Feedback

Measuring DDR4 Eye Diagrams

Note:

A WaveView ADV license is required for this feature.

The following DDR4 measurements are supported in the WaveView tool:

- Vcent measurements from a Vref sweep
- Vcent_Dq (pin avg)
- TdiVW
- VIHL_AC
- TdiPW
- Tr&Tf based on VdIVW_Total
- SRIN_dIVW based on VdIVW_Total
- TdIVW & VdIVW margins
- Aperture alignment for LPDDR4

Please refer the *JEDEC Manual* (JEDEC standard no 79-4) for the details of these measurements.

This section contains information on the following topics:

- Taking DDR4 Measurements
- Taking TdIVW and VdIVW Margin Measurements
- Aligning the Aperture for LPDDR4 Measurements
- Shifting the Trigger Clock Delay and Delay Clock to the Aperture Center
- Sample DDR4 Specification File

Taking DDR4 Measurements

This section explains how to take the following DDR4 measurements:

- Taking Vcent Measurements
- Taking TdIVW_Total and VIHL_AC Measurements
- Taking TdIPW, Tr, Tf, and SRIN_diVW Measurements

Feedback

Taking Vcent Measurements

When a Vcent measurement is performed, the Vref value that corresponds to widest eye opening is calculated. If multiple eye diagrams exist on a single panel, the Vcent value of each eye is measured, and the average value is calcualted to get the Vcent_DQ pin average.

To take a Vcent measurement:

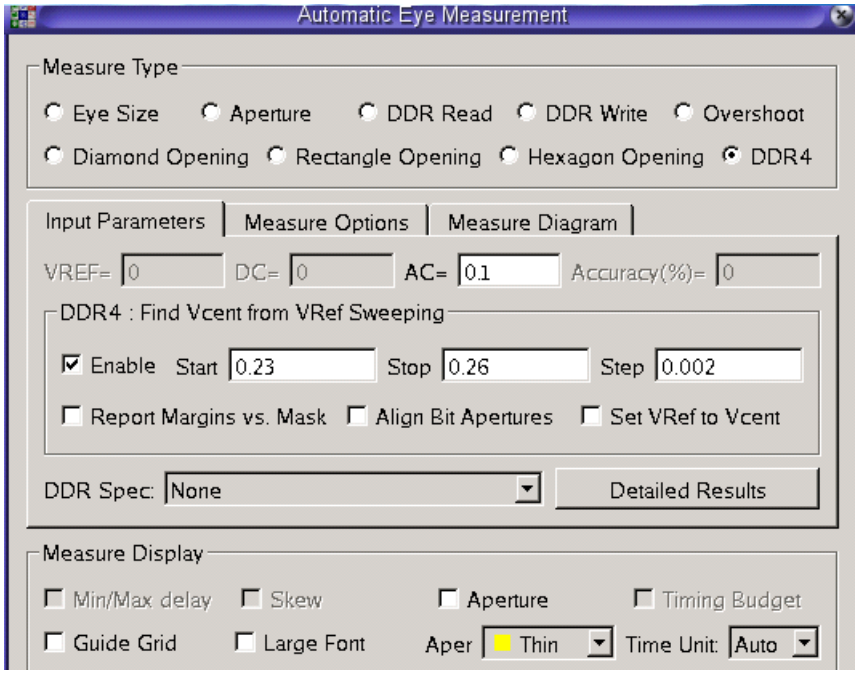
1. Right-click the eye diagram, and choose **Measure Eye** from the menu that opens.

   The **Automatic Eye Measurement** window opens.

2. Click the **DDR4** measurement.
3. Click the Input **Parameters** tab.
4. Click the **Enable** check box to enable a Vref sweep.

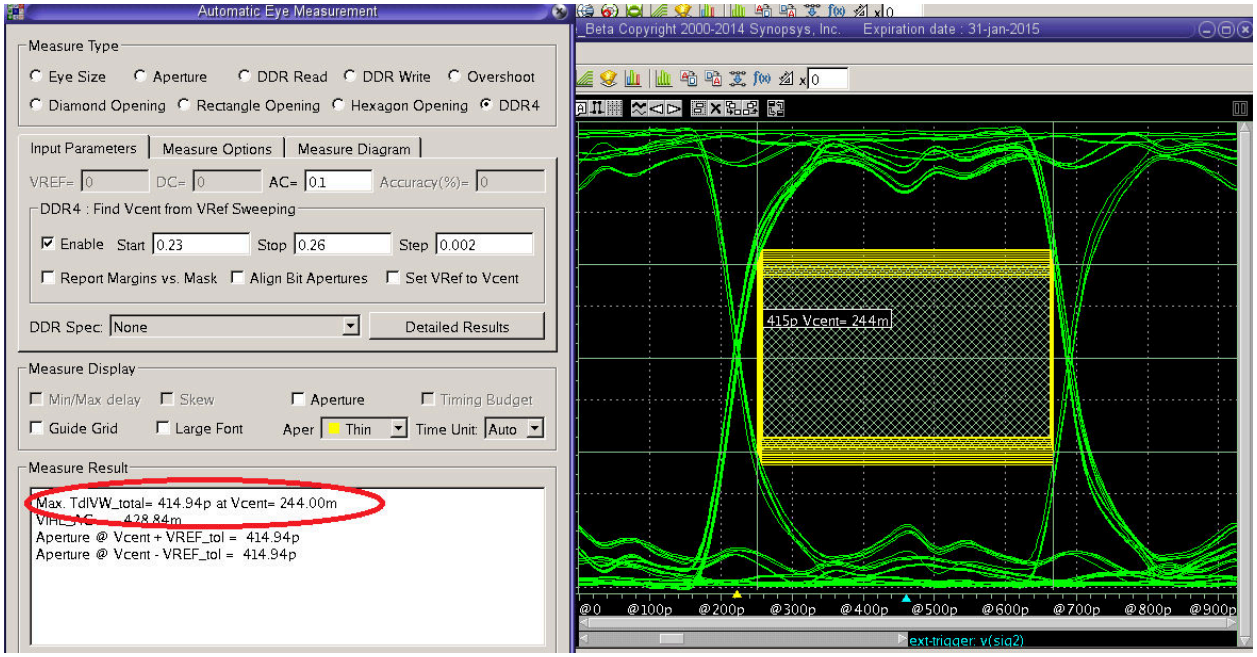
   The Vcent_DQ pin average for all the bits is calculated when this option is enabled.

5. Enter an **AC** value (VdIVW_Total/2), as well as the **Start**, **Stop**, and **Step** values.



6. Click the **Measure** button.

Feedback



The values for Vcent are calculated and displayed in the Measure Result field, as well as on the eye diagram.

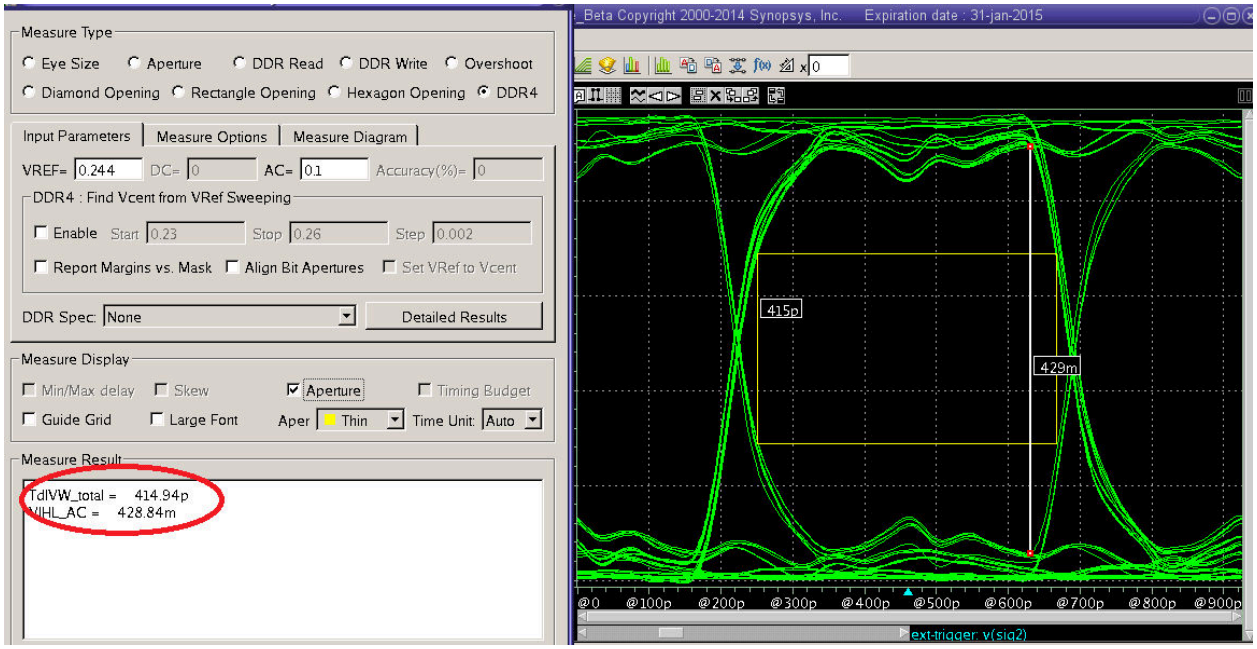
Taking TdIVW_Total and VIHL_AC Measurements

To take TdIVW_Total and VIHL_AC measurements:

1. Right-click the eye diagram, and choose **Measure Eye** from the menu that opens.

   The **Automatic Eye Measurement** window opens.

2. Click the **DDR4** measurement.
3. Click the **Input Parameters** tab.
4. Enter values for **Vref** and **AC**, and click the **Measure** button.



The values for TdIVW_Total and VIHL_AC are calculated and displayed in the Measure Result field, as well as on the eye diagram.

If you want to specify a target value for TdIVW_Total and VIHL_AC in a DDR4 specification file and load it into the WaveView tool, enter the `setenv CX_DRTAB_FILE <filename>` environment variable at the command line before invoking the WaveView tool. See the Sample DDR4 Specification File section for information on how to set up a DDR4 specification file.

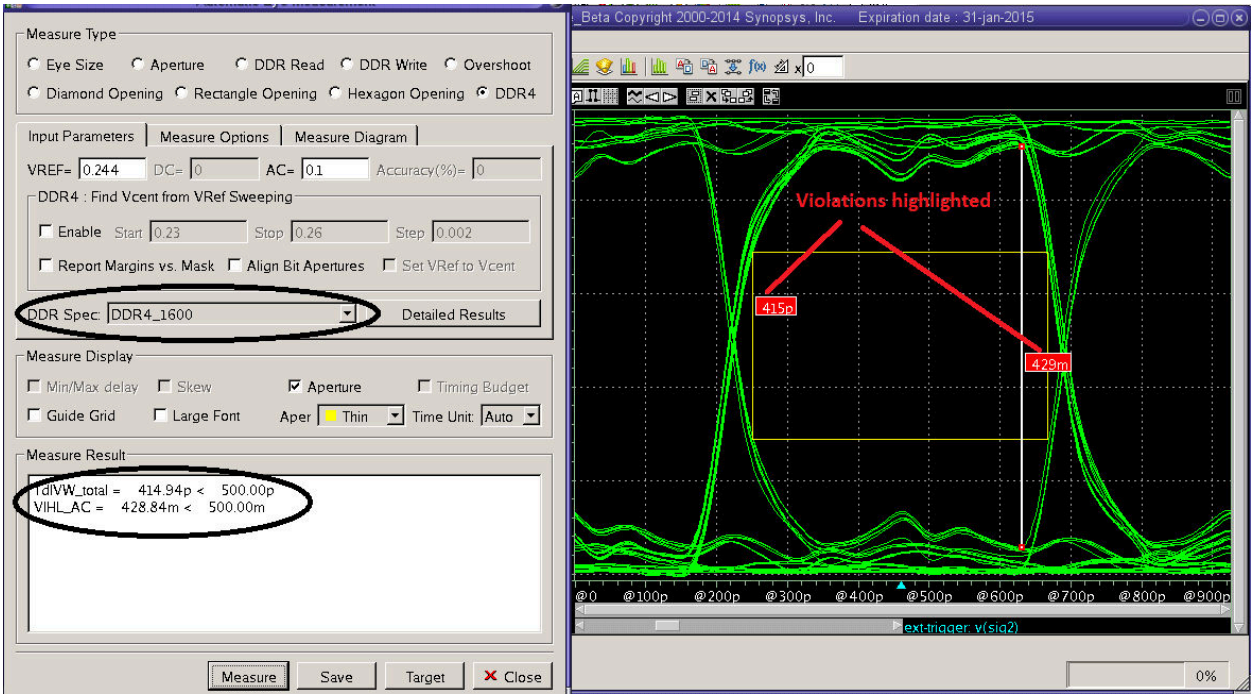
Include the following commands in your DDR4 specification file to set these values:

`#min_aperture <value>` or `#min_aperture_ui <value>`

`#min_vihl_ac <value>`

If a measured value does not meet the target, the violations are highlighted in red.

Feedback



Taking TdIPW, Tr, Tf, and SRIN_diVW Measurements

To take TdIPW, Tr, Tf, and SRIN_diVW measurements:

1. Right-click the eye diagram, and choose **Measure Eye** from the menu that opens.

   The **Automatic Eye Measurement** window opens.

2. Click the **DDR4** measurement.

3. Click the **Input Parameters** tab.

4. Enter values for **Vref** and **AC**, and click the **Measure** button.

   The values for TdIPW, Tr, Tf, and SRIN_diVW are calculated and displayed in the Measure Result field, as well as on the eye diagram.

5. Click the **Detailed Results** button.

Feedback



The **Setup/Hold Result Viewer** window opens with the TdIPW, Tr,Tf, and SRIN_diVW values displayed. You can double-click a row to show the corresponding measurements on a transient signal.

If you want to specify the target value for the TdIPW and slew, you can include the following commands in your DDR4 specification file:

```
#min_dataslew <value>

#min_tdipw_ui <value>
```

Measurements that do not meet the target value are highlighted.

Specifying VIHL_AC(min) for tr2 Measurements

You can specify the VIHL_AC(min) value for calculating tr2 measurements:

1. Select **DDR4** in the **Automatic Eye Measurement** window.
2. Check the VIHL_AC(min) box and specify a value.

If a VIHLAC(min) value is not specified, by default the WaveView tool uses the calculated VIHL_AC value.

If the specified VIHL_AC(min) is too high or too low for certain clk period and tr2 cannot be calculated for that period, the **Setup/Hold Result Viewer** shows a horizontal line at y=VIHL_AC(min).

Feedback

Taking TdIVW and VdIVW Margin Measurements

The following calculations occur when you place a mask inside of an eye and measure the voltage and timing margins:

- The Vcent value of each bit is calculated.
- The Vcent average of all the bits (Vcent_Dq pin_avg) is calculated.
- A mask based on the Vcent_Dq and Triggering point is placed.
- The margin values are measured. The ringback effect is also considered during this measurement.

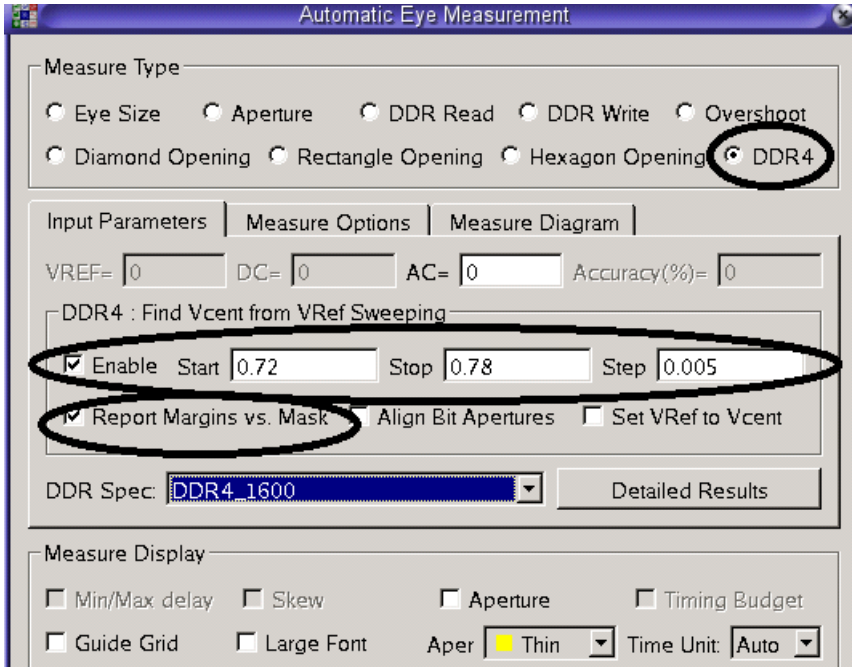
To measure the margin:

1. Right-click the eye diagram, and choose **Measure Eye** from the menu that opens.

   The **Automatic Eye Measurement** window opens.
2. Click the **DDR4** measurement.
3. Click the **Input Parameters** tab.
4. Click the **Enable** check box to enable a Vref sweep.

   The Vcent_Dq pin average for all the bits is calculated when this option is enabled.
5. Enter the **Start**, **Stop**, and **Step** values.

   If you enter the Vref value directly, the measurement works without a Vref sweep.
6. Click the **Report Margins vs Mask** check box.
7. Choose a DDR4 spec from the **DDR4 Spec** menu that contains mask specifications.

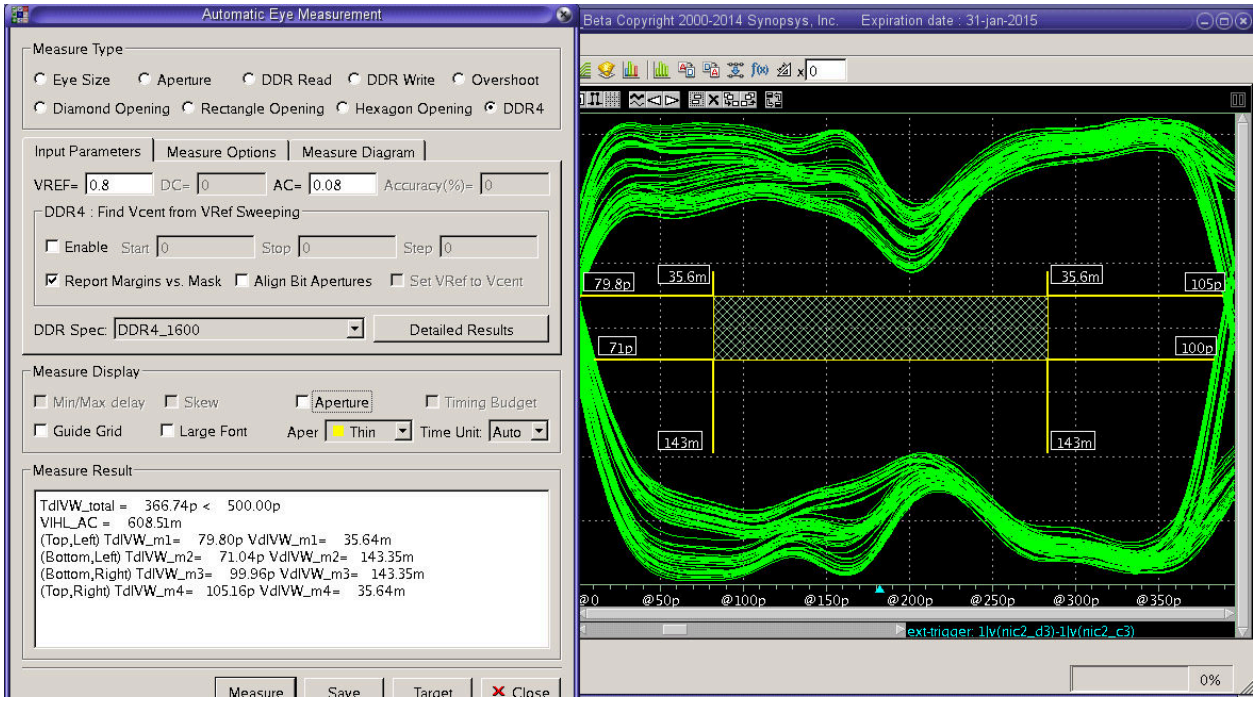


Use the following commands in your DDR4 specification file to set the width and height of mask:

```
#mask_tdivw_ui <width value>

#mask_vdivw <height value>
```

8. Click the **Measure** button.



The mask margin is calculated and displayed in the Measure Result field, as well as on the eye diagram.

Aligning the Aperture for LPDDR4 Measurements

Aperture alignment measures the mask offset values of each bit. You can plot the eye diagram of all the bits in a single panel and trigger it with a differential clock.

The first eye is used as the reference, and then the center point of the eye crossing is calculated. All other eye diagram center points are shifted so they overlap. Both the shift value of each bit and the aperture value are reported.

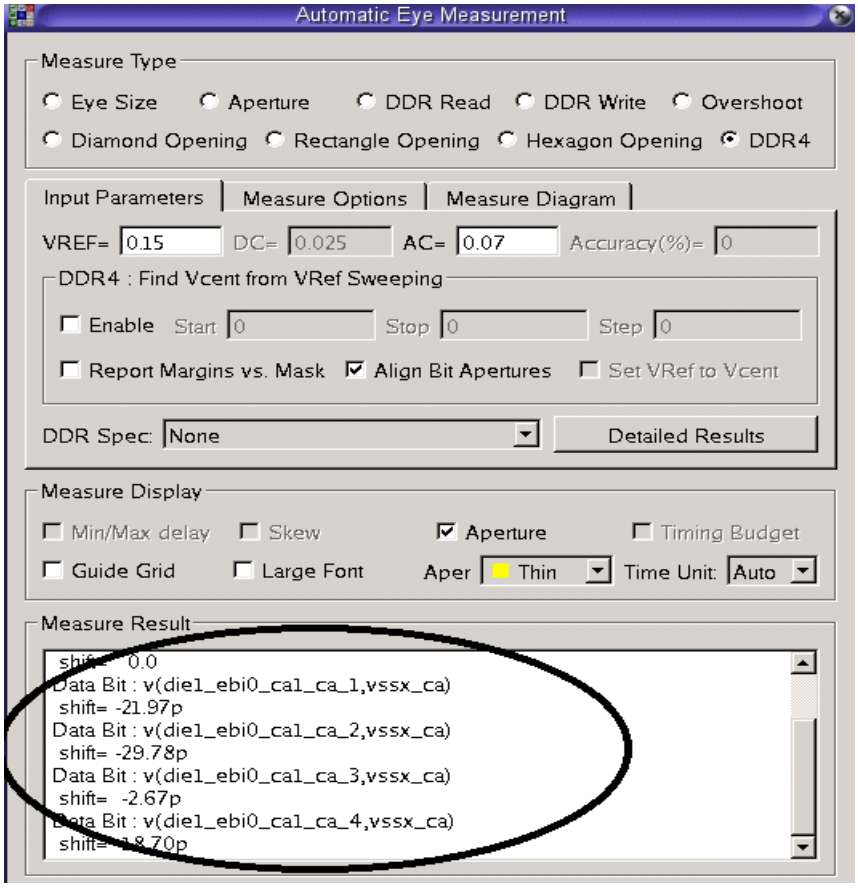
To align apertures:

1. Plot the eye diagrams of all the bits in a single panel, and use the same signal for triggering the eye.
2. Right-click the eye diagram, and choose **Measure Eye** from the menu that opens.

   The **Automatic Eye Measurement** window opens.
3. Click the **DDR4** measurement.
4. Click the **Input Parameters** tab, and enter **Vref** and **AC** values for the eye.
5. Click the **Align Bit Apertures** option.

Feedback

6. Click the **Measure** button.



The shifted values of each bit are calculated and displayed in the **Measure Result** field.

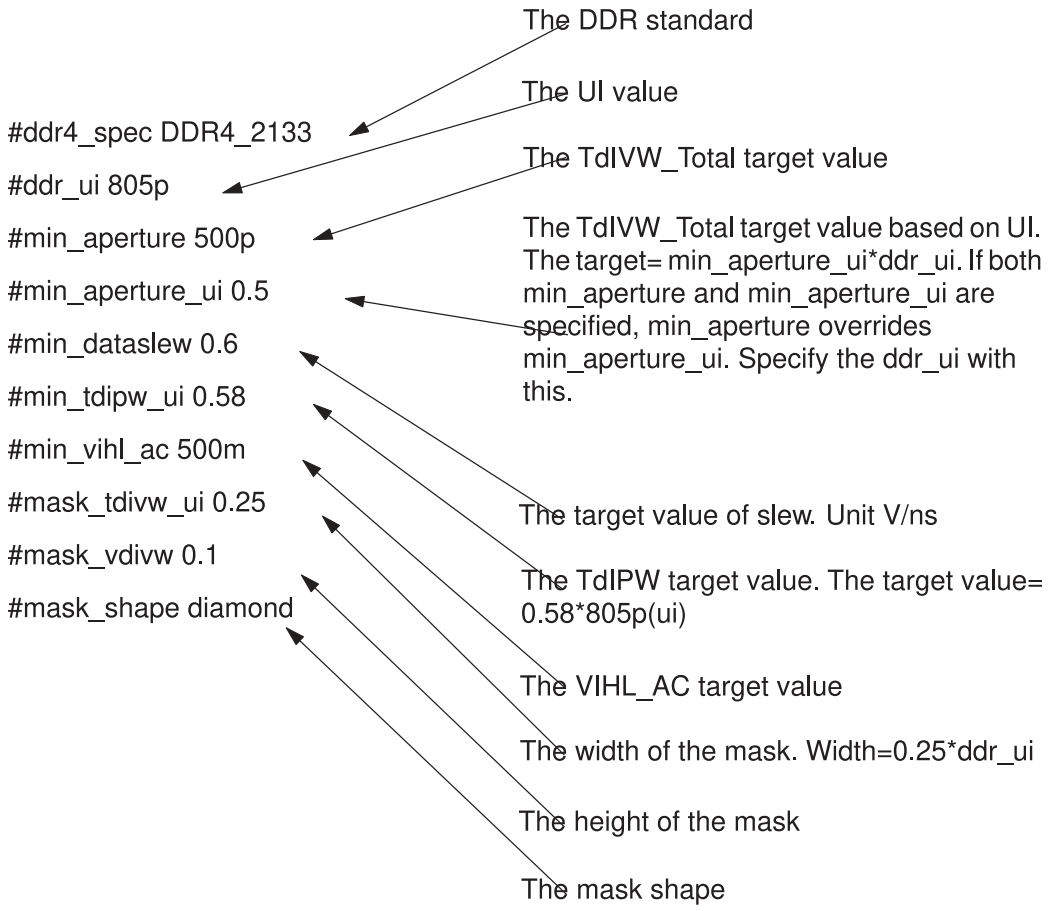
Shifting the Trigger Clock Delay and Delay Clock to the Aperture Center

Sometimes the simulation data might not have the trigger signal well centered, and you might want to center the trigger signal during the DDR4 measurement. The **Trigger Clock Delay** option specifies the delay value for the triggering signal, and the **Delay Clock to Aperture Center** option aligns the trigger signal to the center of the eye.

Feedback

Sample DDR4 Specification File

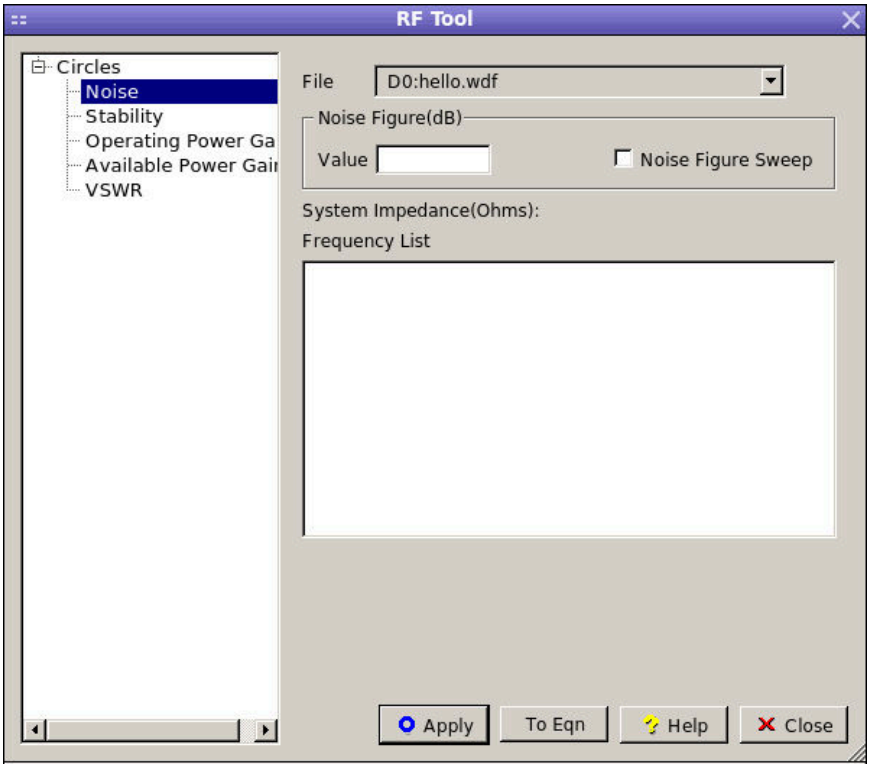
You can specify the target values for different measurements in a DDR4 specification file. For example:



RF Measurements

To open the RF Tool, choose **RF Tool** from the menu located in the **Utilities** section of the **Waveform** tab. The **RF Tool** window opens.

Feedback



The RF Tool takes special measurements or calculations for RF analyses. The RF analyses in the RF Tool are different from the measurements in WaveView Measurement Tool—they are not applied to a specific signal, but to a specific waveform file.

This section contains information on the following topics:

- Measuring Circles
- Measuring Noise Circles
- Measuring Stability Circles
- Measuring Operating Power Gain Circles
- Measuring Available Power Gain Circles
- Measuring VSWR Circles
- Exporting Equations From RF Tool Setup

Measuring Circles

An S-Parameter waveform file is required for all circle measurements. The **File** selection field at the top of the RF Tool window lists all currently open waveform files. If the specified

Feedback

file is an S-Parameter waveform file, the frequency values are extracted from the specified file and listed in the Frequency List.

The center point and radius values for each circle are calculated. 360 data points are created to make a circle with the center and radius.

The result circles are plotted on a Smith Chart. If a Smith Chart does not yet exist in any waveview, a new one is created, and the circles are plotted on it.

This section contains information on the following topics:

Measuring Noise Circles

To measure a noise circle in the RF Tool:

1. Click the **Noise** category on the left side of the RF Tool window.

   The Noise Circle measurement settings are displayed.

2. Choose a file from the **File** menu that contains the noise circle you want to measure.

3. Enter a dB noise figure value in the Noise Figure text box, or click the **Noise Figure Sweep** check box if you want to include a noise figure sweep as part of the measurement.

   Note:

   System Impedance value is not an input value, but is the value of signal `Zo.Zo(1)` or `Zo.Zod(1)` from the selected waveform file. The value is displayed for informational purposes.

4. Select one or more frequency values by holding the **Ctrl** key and clicking frequency values in the Frequency List.

   If you want to select all frequency values, pres **Ctrl + A** to select all.

5. Click **Apply** to take the noise circle measurements and plot the results.

Measuring Stability Circles

To measure a stability circle in the RF Tool:

1. Click the **Stability** category on the left side of the RF Tool window.

   The stability circle measurement settings are displayed.

2. Choose which S Parameter signals you want to include as part of the measurement in the S Parameters section.

Feedback

The associated signals and parameters are defaults from the selected waveform file. You can change the defaults signals by choosing different signals from the menus in the S Parameter section.

3. Select one or more frequency values by holding the **Ctrl** key and clicking frequency values in the Frequency List.

   If you want to select all frequency values, pres **Ctrl + A** to select all.

4. Choose the type of stability circle you want to plot. You can choose one or both types.

   Click the **Output Stability Circle** check box to plot an output stability circle waveform. Click the **Input Stability Circle** check box to plot an input stability circle waveform.

5. Click **Apply** to take the stability circle measurements and plot the results.

Measuring Operating Power Gain Circles

To measure an operating power gain circle in the RF Tool:

1. Click the **Operating Power Gain** category on the left side of the RF Tool window.

   The operating power gain circle measurement settings are displayed.

2. Choose which S Parameter signals you want to include as part of the measurement in the S Parameters section.

   The associated signals and parameters are defaults from the selected waveform file. You can change the defaults signals by choosing different signals from the menus in the S Parameter section.

3. Choose one of the following gain values:

   - **Value**

     Enter a gain value in the Value text box.

   - **From/To Sweep**

     Enter from and to values in the From and To text boxes, respectively. Enter a Step value as well.

   - **Gmax-based Sweep**

     Enter step and number values in the Step and Number text boxes, respectively.

4. Select one or more frequency values by holding the **Ctrl** key and clicking frequency values in the Frequency List.

Feedback

If you want to select all frequency values, pres **Ctrl + A** to select all.

5. Click **Apply** to take the operating power gain circle measurements and plot the results.

Measuring Available Power Gain Circles

To measure an available power gain circle in the RF Tool:

1. Click the **Available Power Gain** category on the left side of the RF Tool window.

   The available power gain circle measurement settings are displayed.

2. Choose which S Parameter signals you want to include as part of the measurement in the S Parameters section.

   The associated signals and parameters are defaults from the selected waveform file. You can change the defaults signals by choosing different signals from the menus in the S Parameter section.

3. Choose one of the following gain values:

   - **Value**

     Enter a gain value in the Value text box.

   - **From/To Sweep**

     Enter from and to values in the From and To text boxes, respectively. Enter a Step value as well.

   - **Gmax-based Sweep**

     Enter step and number values in the Step and Number text boxes, respectively.

4. Select one or more frequency values by holding the **Ctrl** key and clicking frequency values in the Frequency List.

   If you want to select all frequency values, pres **Ctrl + A** to select all.

5. Click **Apply** to take the available power gain circle measurements and plot the results.

Measuring VSWR Circles

To measure a VSWR circle in the RF Tool:

1. Click the **VSWR** category on the left side of the RF Tool window.

   The VSWR circle measurement settings are displayed.

2. Choose which S Parameter signals you want to include as part of the measurement in the S Parameters section.

   The associated signals and parameters are defaults from the selected waveform file. You can change the defaults signals by choosing different signals from the menus in the S Parameter section.

3. Choose the type of VSWR circle you want to plot. You can choose one or both types.

   Click the **Output VSWR Circle** check box to plot an output VSWR circle waveform, and enter a value for VSWROut. Click the **Input VSWR Circle** check box to plot an input VSWR circle waveform, and enter a value for VSWRIn.

4. Select one or more frequency values by holding the **Ctrl** key and clicking frequency values in the Frequency List.

   If you want to select all frequency values, pres **Ctrl + A** to select all.

5. Click **Apply** to take the VSWR circle measurements and plot the results.

Exporting Equations From RF Tool Setup

To export an equation of the setup from an RF measurement to the Equation Builder:

1. Ensure you set up all the necessary settings for an RF measurement.
2. Click the **To Eqn** button at the bottom of the RF Tool window.

   The Equation Builder opens with the created RF measurement equation. For example, the following equation is exported from the settings for a noise circle measurement:

   ```
   nc("0|gsmlna.sc0","50 60 10",50,"100M 200M 300M")
   ```

PrimeSim HSPICE Measure Tool

To open the **PrimeSim HSPICE Measure** tool, choose **HSPICE Measure** from the menu located in the **Utilities** section of the **Waveform** tab.

Pareto Analysis

The Pareto Analysis tool can import your data mining results from PrimeSim HSPICE, performs sensitivity, R, and R^2 calculations, then produces results in one of the following formats:

- HistBar plot
- scatter plot

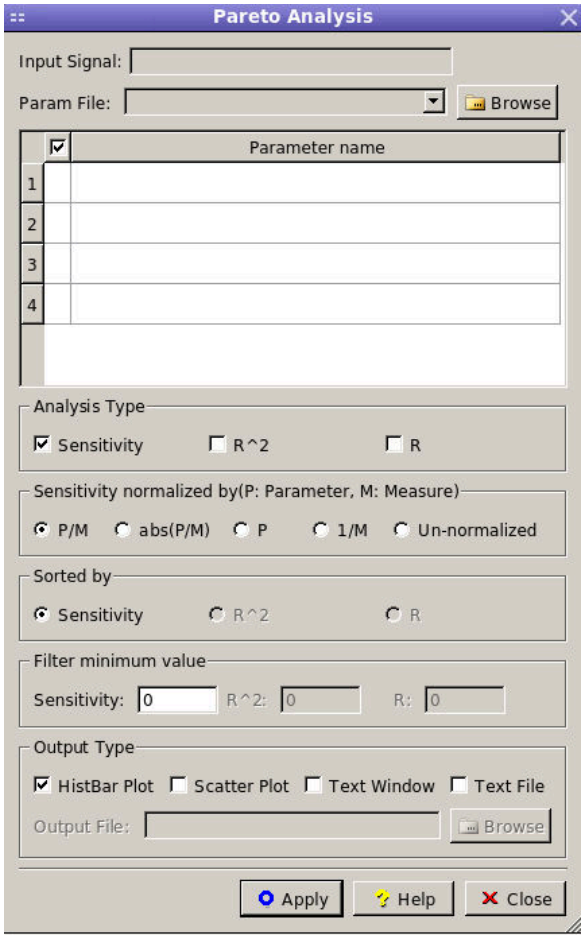
Feedback

- text window plot
- text file

To perform a Pareto analysis:

1. Choose **Pareto Analysis** from the menu located in the **Utilities** section of the **Waveform** tab.

   The **Pareto Analysis** window opens.



2. Select an Input Signal by dragging it from the signal list and dropping it in the text box.

   The input signal can be from an `*.mt` or `*.mc` file (which are files that contain parameter names and values for each trace you can load by clicking **Browse**), or from a measurement you apply to a sweep tr0 signal. The number of points in this signal need to be equal to the number of sweeps. Signals are not editable.

3. Ensure a Param File is selected.

Feedback

You can click the **Browse** button to browse to the location of a Param File, or enter the name in the Param File text box.

4. Choose which parameter signals you want to include by clicking the check box next to a signal name listed in the Parameter Name table.

   By default all signals are selected. You can deselect all signals by unchecking the check box in the heading row.

5. Choose one or more analysis types: **Sensitivity**, **R^2**, or **R**.

   Sensitivity analyses are calculated using the following function:

$$Sensitivity = \frac{\sum(x*y) - (\sum(x)*\sum(y)/n)}{\sum(x^2) - (\sum(x))^2/n} * \frac{\sum(x)}{\sum(y)}$$

   R^2 analyses are calculated using the following function:

$$Ajusted\,R-Square\ =\ 1-(1-R*R)*\frac{n-1}{n-p-1}$$

$$Ajusted\ R-Square\ = 1-\left(1-\frac{(\Sigma(x-\bar{x})*(y-\bar{y}))^2}{\Sigma((x-\bar{x})^2)*\Sigma((y-\bar{y})^2)}\right)*\frac{n-1}{n-p-1}$$

   R analyses are calculated using the following function:

$$R = \frac{\sum(x*y) - (\sum(x)*\sum(y)/n)}{\sqrt{(\sum(x^2)) - (\sum(x))^2/n)*(\sum(y^2) - (\sum(y))^2/n)}}$$

6. If you choose **Sensitivity** as the analysis type, select one of the following normalization factors:
   - **P/M**
   - **abs(P/M)**
   - **P**
   - **1/M**
   - **Un-normalized**

   Otherwise, skip to the next step.

7. If you selected more than one analysis, choose how you want your results sorted in the Sorted By section.

8. Enter values for the minimum filter value.

   If you select more than one analysis type, all the filter values are used together.

Feedback

9. Choose which type of output you want.

   If you choose **Text File**, enter a name for the text file.

10. Click **Apply** to run the Pareto analysis.

6

Using the Equation Builder

This chapter details using the Equation Builder in the WaveView tool.

The Equation Builder facilitates multi-file mixed-signal waveform operations and measurement. You can create and evaluate equations on waveform files to generate results as scalar measurements or new waveform data.

You can type input equations into the Equation Builder or construct them by dragging and dropping from the Waveform signal browser or from the WaveView tool. You can evaluate any or all equations at once, each on one or more specified target waveform files.

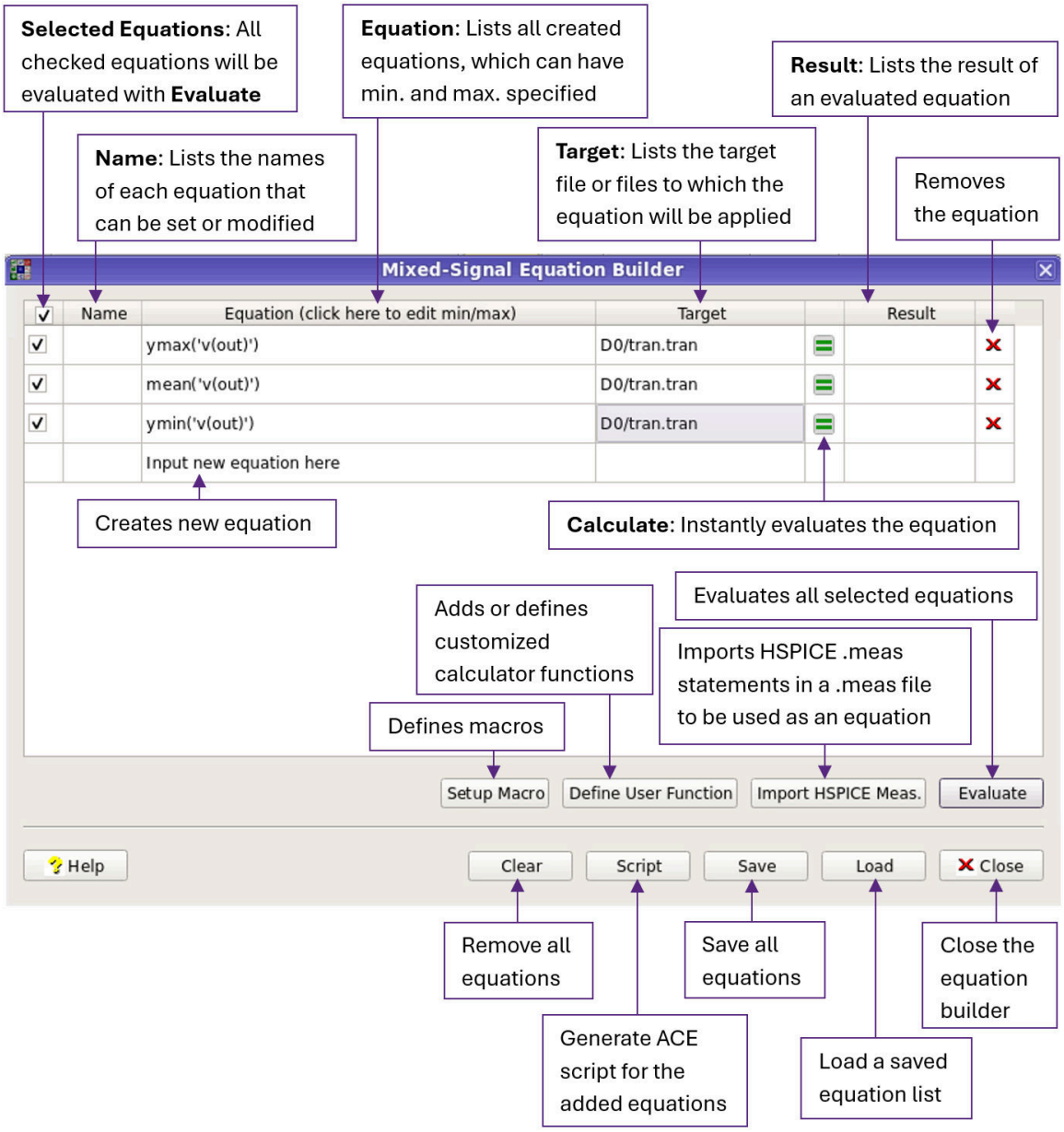
The following sections describe the Equation Builder:

- Viewing the Equation Builder
- Adding an Equation
- Building an Equation
- Setting the Target File of an Equation
- Evaluating Equations
- Plotting Results
- Marking Annotation Information From Equation Results
- Setting Up Macros
- Saving DDR Equations as ACE Scripts
- Calculating Multi-trace Waveforms

Feedback

Viewing the Equation Builder

Click **Equation Builder** in the **Utilities** section of the **Waveform** tab to open the Equation Builder. The following window appears.

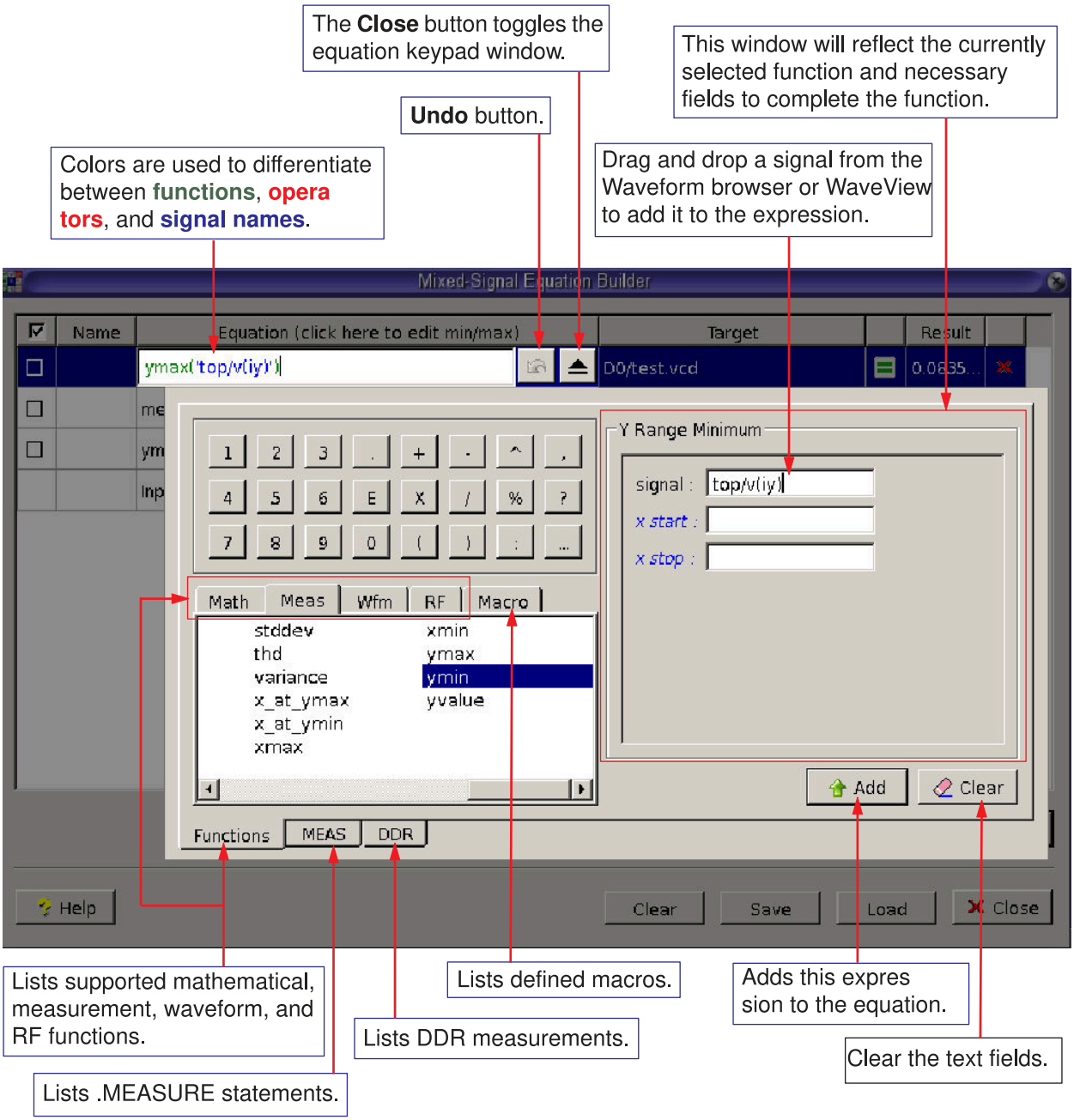


Feedback

Viewing the Equation Builder Keypad

When an equation is selected, the black **Close** button toggles the following equation keypad window to add or edit expressions to the equation.

Disabled areas of the image were explained on the previous page.

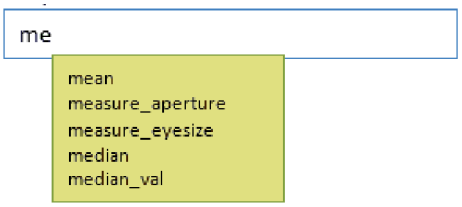


Feedback

Using the Text Editor

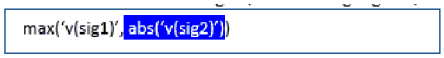
The equation builder text editing behaves as follows:

- **Bracket and quote completion:** When typing a left bracket or left quote, the tool completes the right side automatically.
- **Function completion:** When typing, the tool matches the input with existing functions and display them in a pop-up form as follows:



  Press **Tab** to select the first function in the pop-up list, press **Esc** to close the pop-up list, and use the preference **Hide function completion for calculator** to disable this feature.

- **Function selection:** You can quickly select an entire function and signal in the equation string with a single click, which highlights it as follows:



  Interaction with a highlighted function is as follows:

  - Typing anything replaces the entire highlighted string.
  - Dragging a signal onto the highlighted string replaces the entire string.
  - Clicking again on the highlighted string (or double-clicking) changes to type-in mode.
  - Clicking the white space outside of the function will enter type-in mode and place the cursor at the end of the string.

Adding an Equation

To add a new equation, click **Input new equation here** in the equation list. You can then type or use the equation keypad to build an equation, as detailed in Building an Equation.

- Importing PrimeSim HSPICE .meas Statements as Equations
- Duplicating an Equation
- Modifying an Equation

Feedback

- Removing an Equation
- Registering a Tcl Process as an Equation

Importing PrimeSim HSPICE .meas Statements as Equations

To import PrimeSim HSPICE .meas statements from a .meas file and use them as equations, click **Import HSPICE Meas** and navigate to your file. Each .meas statement is populated in the equation list with a name based on the measurement used.

For example, if your .meas file contains:

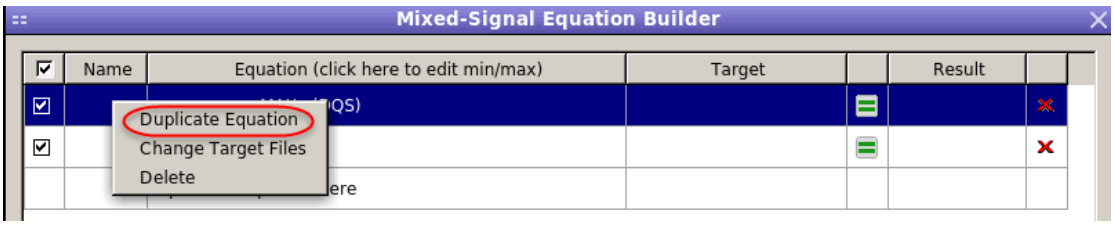
```
.meas vmax MAX v(DQS)
```

it is populated as follows:



Duplicating an Equation

To duplicate an equation, right-click the equation in the equation list and select **Duplicate Equation** from the menu:



Modifying an Equation

To modify an equation, click the equation in the equation list and follow the steps for Building an Equation, or select **Signal "Name" > Modify** from the Waveform context menu.

Removing an Equation

To remove an equation, click its red **X** on the right in the equation list or select any number of rows in the equation list, right-click, and select **Delete**.

Feedback

Registering a Tcl Process as an Equation

To learn how to register a user-defined Tcl process as an Equation Builder function, see sx_create_function in the *WaveView ACE Command Reference.*

Building an Equation

After Adding an Equation, you can build the content of the equation.

When selected, the functions are displayed in green, operators in red, and signal names in blue. Unmatched parenthesis are highlighted in red.



The Equation Builder accepts numbers in scientific or scale format (for example, `1.0e-12, 10n, 0.23m`).

To undo any typing, click the blue **Undo** arrow.

To expand or close the equation keypad, click the black **Close** button. Refer to Viewing the Equation Builder Keypad for details on the keypad buttons or continue for details on building an equation:

- Adding Signals to an Equation
- Adding Multiple Signals From Different Waveform Files to an Equation
- Adding Functions to an Equation
- Adding .MEASURE Statements to an Equation
- Specifying Min/Max for an Equation to Test for TableView Failures
- Naming an Equation

Adding Signals to an Equation

To add a signal to an equation, drag and drop the signal from the Waveform Browser or WaveView, or right-click the signal in the Waveform Browser and select **Signal "Name" > Add to Equation** from the context menu.

When dragging and dropping, you can drag the signal into an equation in the equation list, or into a **Signal** text field within the equation keypad.

Feedback

You can also type a signal name directly into an equation in the equation list. In this case, note that to add a signal using case-sensitive name search, you should use the `wave()` function to access the signal. For example:

```
wave('I(Vdd)')
```

If the target file for an equation has not been set, adding a signal automatically populates the **Target** column with the related waveform file.

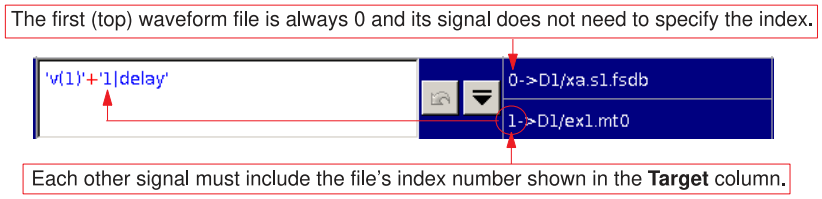
Adding Multiple Signals From Different Waveform Files to an Equation

When using multiple signals from different waveform files in a single equation, you need to specify the index for each signal after the first so that it corresponds to the correct waveform file.

To add these waveforms and assign the correct index:

1. Add each signal to the equation as shown in Adding Signals to an Equation.

   You see that multiple waveform files are automatically populated in the **Target** column for that equation:



   The index number for each signal corresponds to the order of the waveform files in the **Target** list, as shown above.

2. Format the signals with the corresponding index in the format of:

   file_index|*signal_name*

   Please note that the first (top) waveform file always has an index of 0 and does not need to be specified; this means that the second file has a `file_index` of 1, the third file has a `file_index` of 2, and so on.

See Changing the Index Number of a Target File to change index assignments.

Feedback

Adding Functions to an Equation

To add a function to an equation from the equation keypad:

1. Click the **Functions** tab and select a function from one of the subtabs.

   The right side of the keypad displays information for adding that function.

2. Add a signal to the function as shown in Adding Signals to an Equation.

3. Enter information for any needed options and click **Add**.

You can also type a function directly into an equation in the equation list.

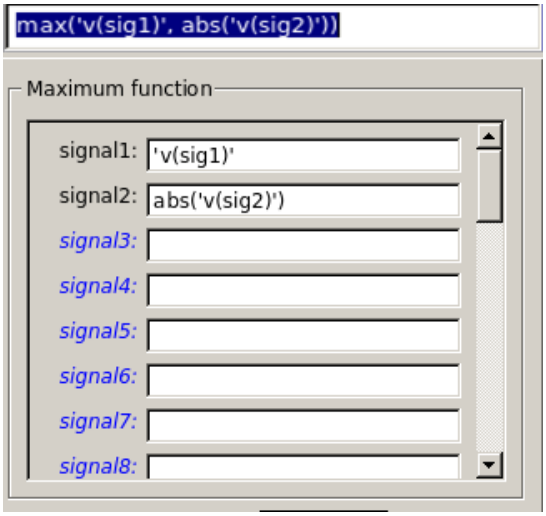
See Supported Functions for a list of all functions supported by the Equation Builder.

The following sections detail additional function options:

- Viewing Function Arguments
- Function Nesting
- Adding User-Defined Functions
- Adding User-Favorite Functions

Viewing Function Arguments

When a function is selected, the function dialog shows the arguments of the current function. To change an argument within the function, make changes in the text fields and select **Update**. For example:



Feedback

Function Nesting

When a function is selected and highlighted as described above, double-clicking a function in the function list of the keypad nests the currently highlighted string. Behavior changes depending on the number of arguments in the function, as follows:

- If the function has one argument, the function nests the current string and the new function is highlighted.
- If the function has more than one argument, the current string is set as the first argument and the cursor is placed into the line editor of the second argument.

Adding User-Defined Functions

You can define user functions through the GUI by clicking **Define User Function** in the Equation Builder window.

Once user-defined functions are loaded, they are displayed in the User category and the appropriate category based on return value (`Meas` or `Wfm`). This User category replaces the previous Macro category, and contains both macros and TCL functions. If there are no macros or TCL functions, the User category is hidden.

Adding User-Favorite Functions

To add a function to your favorites, **right-click** the function from one of the common categories and select **Add to favorites**. To remove a function from your favorites, **right-click** the function in the Favorites category and select **Remove from favorites**. If there are no user-favorites, the Favorites category is hidden.

Adding .MEASURE Statements to an Equation

To add a .MEASURE statement to an equation from the equation keypad:

1. Click the **Measures** tab.

   The .MEASURE options are displayed.

2. Enter information for any needed .MEASURE options and click **Add**.

   See the *HSPICE Reference Manual: Commands and Control Options* for further information on the .MEASURE statement.

Specifying Min/Max for an Equation to Test for TableView Failures

You can specify the min and max for an equation to have the WaveView tool judge the equation result for passes and fails in TableView.

Feedback

To specify the min and max for an equation, click the **Equation (click here to edit min/max** column title and enter your specifications in the newly added columns **Min** and **Max**.

Naming an Equation

Once an equation has been named with an alias, it can be referred to by this alias in subsequent operations.

To name an equation, either click the cell under the **Name** column for that equation and type a name, or assign a name in the equation itself with a prefix.

For example, entering the following equation will name the equation as `POWER`:

```
POWER='I(VDD)'*'VDD"
```

To rename an equation, repeat the above or select **Signal "Name" > Edit Alias** from the Waveform Browser context menu and enter a new name for it.

Setting the Target File of an Equation

For each equation, you can select the target file in the **Target** column. If you dragged the signal from the Waveform Browser or the WaveView tool, the target file is set automatically.

Feedback

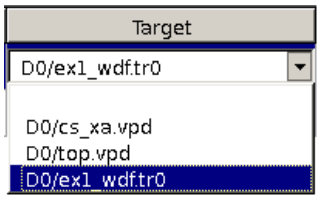
Changing the Target File of an Equation

You can change the target of an equation to any imported waveform file.

To change the target file of an equation:

1. In the equation list, click the dropdown arrow in the **Target** column.

   A dropdown menu appears with a list of all imported waveform files:



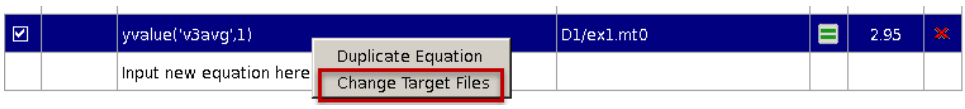
2. Click a file to set it as the target of the equation.

Changing the Index Number of a Target File

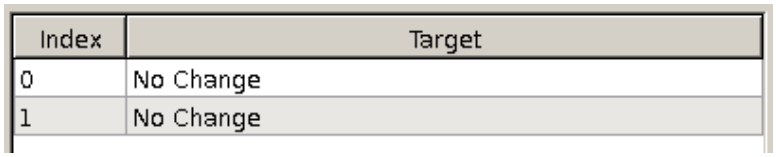
If there are multiple target files used in a single equation, each target file is assigned an index number. Each index number applies only to that equation.

To reassign the index number of a target file:

1. Highlight one or more equations in the equation list, then right-click and select **Change Target Files** from the menu:



2. In the **Change Targets** dialog, click **No Change** for the index number you want to reassign.:



   A box appears listing all imported waveform files.

3. Select the target file to be assigned with the index number and click **OK**.

See Adding Multiple Signals From Different Waveform Files to an Equation for more information on when these index numbers are used.

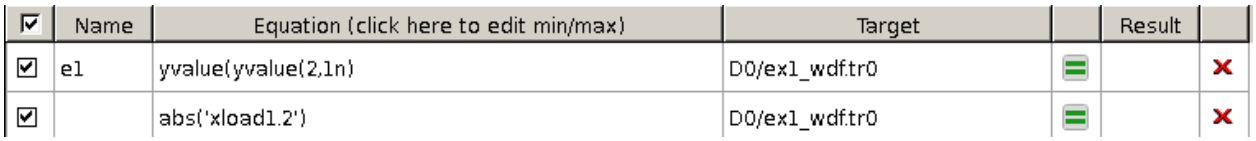
Feedback

Evaluating Equations

To evaluate an equation, click its green **Calculate** button  to the right in the equation list. You can use this to evaluate an equation as you build it.

To evaluate multiple equations at once:

1. In the equation list, check the box of each equation to evaluate:



	Name	Equation (click here to edit min/max)	Target		Result	
☑	e1	yvalue(yvalue(2,1n)	D0/ex1_wdf.tr0	=		✕
☑		abs('xload1.2')	D0/ex1_wdf.tr0	=		✕

2. Click the **Evaluate** button:



You can also right-click an equation in the equation list and select **Eval Current Equation with Sub-Equations** to evaluate the entire equation as well as its subequations. Any evaluated subequations are added to the equation list.

Evaluating Sub-Equations

To evaluate a single subequation, highlight the part of the equation you want to evaluate, right-click and select **Eval Selected Equation**. This will evaluate the highlighted subequation and add it to the equation list.

To evaluate all subequations, highlight the part of the equation you want to evaluate, right-click and select **Eval Selected Equation with Sub Equations**. This will evaluate the selected equation and add it as well as every contained subequation to the equation list.

If the selected equation has no subequations, these options are disabled.

Plotting Results

Once you have evaluated one or more equations, the results appear by default as follows:

- Scalar result (single or multiple scalars) — plotted to TableView.
- Waveform result — plotted to WaveView.

Feedback

Plotting All Results to TableView

To have all results plotted by default to TableView:

1. In the calculator keypad, click the **...** button.

   This brings up **Operators | Units | Constants | Controls** information.

2. In **Waveform Results**, check the **Plot to TableView by Default** box.

   This preference is saved to the file when quitting WaveView.

Plotting Selective Signals to WaveView

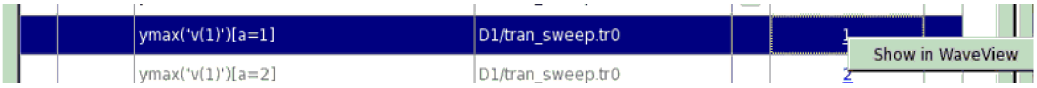
To plot all source signals in an equation, right-click an equation in the equation list and select **Plot Signals in Current Equation**.

To plot all source signals in a selected subequation, highlight part of an equation in the equation list, right-click, and select **Plot Signals in Selected Equation**.

To plot a single source signal, highlight a signal in an equation in the equation list right-click, and select **Plot**.

Marking Annotation Information From Equation Results

If there is annotation information, you can left-click the link in the Results column or right-click and select **Show in WaveView**. For sweep scalar results, first expand the details. For example:



The annotation information contains lines, markers, and labels, with the following behavior:

- The line and markers are not movable. The labels can be dragged. All are fixed to the panel in which the signal was drawn.
- Deleting the panel/line deletes the annotation information as well.
- Moving the panel/line moves the annotation information as well.
- Copying the line or panel/line does not copy the annotation information.
- Annotation information is supported when saving/loading sessions.

Feedback

Setting Up Macros

Macros are available to repeatedly take measurements with created equations.

To set up a macro from the Equation Builder:

1. Click the **Setup Macro** button.

   The **Setup Macro** dialog box is displayed.

2. Click **Input macro definition here** and enter the definition for the macro.

   Macro definitions with an arbitrary number of input arguments are supported. For example:

   ```
   FNEW(a,b) = a*b - 2 PI=3.1415
   ```

3. Enter a name in the **Name** (Args) column for the macro.

4. Press **Enter**.

Macro definitions can be imported or exported by clicking the **Load** and **Save** buttons.

Feedback

Saving DDR Equations as ACE Scripts

To save an ACE script for DDR-related equations you currently have set up, click the **Save** button in the Equation Builder window. Saved results can be restored using the **Load** button, or clicking **Run ACE** in the **Script** section of the **File** tab.

For each DDR evaluation, both the `sx_equation` command and the equivalent ACE TCL script are generated in the saved file in lines that are commented out. You can uncomment them to use these lines.

Calculating Multi-trace Waveforms

The Equation Builder supports operations on multi-trace sweep waveform data. Multi-trace waveforms are generated from parametric sweep analysis. An individual trace in a multi-trace waveform is usually associated with a set of parameters.

When applying an equation to a multi-trace waveform, the equation builder operation is automatically repeated to every trace. As a result, multiple waveforms are generated from the operation.

For scalar measurement, a new measure-versus-trace signal is generated. The `parametric()` function can then be applied to the new measure-versus-trace signal to plot the measure result versus a selected parameter. For example:

```
AVG=mean('I(VDD') parametric(AVG,TEMP)
```

or

```
parametric(mean('I(VDD)',TEMP)
```

7

Supported Functions

The following topics describe the different forms of syntax and the available Calculator functions:

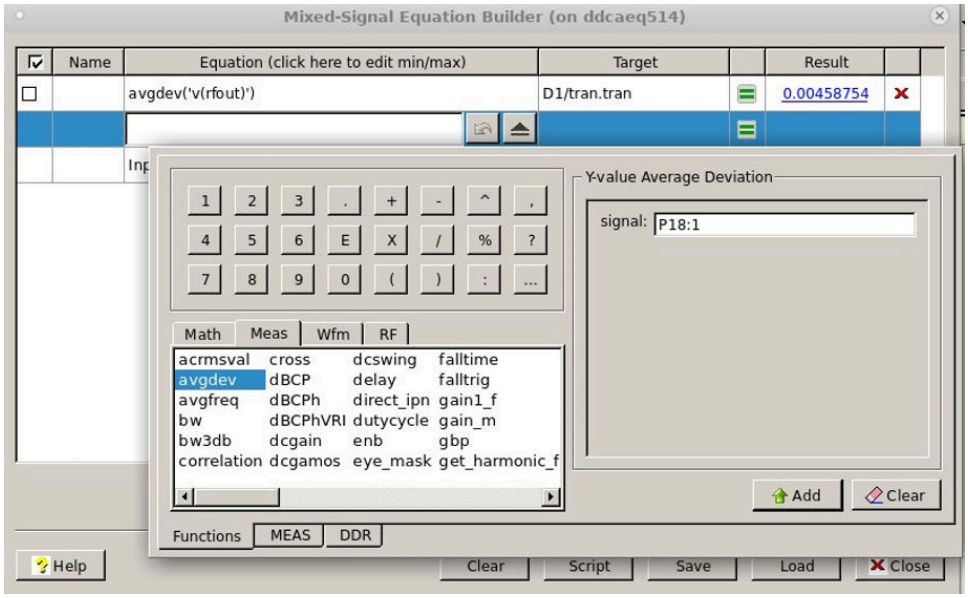
- Differences Between PrimeWave Expression Syntax and Standalone WaveView Expression Syntax
- Operators, Constants, and Logic Operations
- Mathematic Functions
- Measurement Functions
- Waveform Functions
- RF Functions

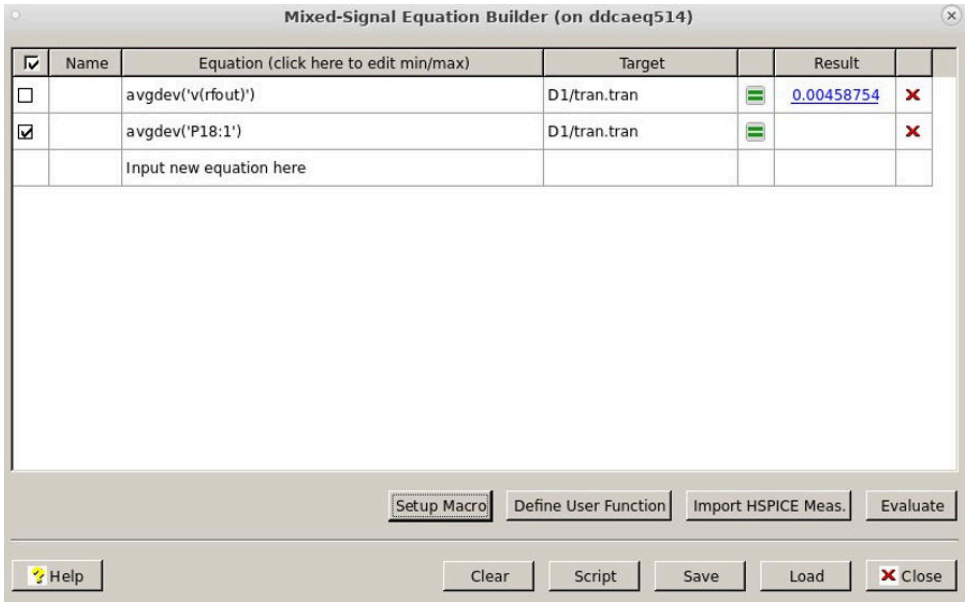
Differences Between PrimeWave Expression Syntax and Standalone WaveView Expression Syntax

When using the PrimeWave Calculator or the WaveView Equation Builder, the tools assist you in creating expressions with the correct syntax. If you enter expressions manually, note that there are two main syntax differences between these tools.

Signal name Syntax

- Standalone WaveView displays signal names as they appear in the waveform file. When referencing a signal name in a function, enclose the signal name in single quotes. If you use the Expression Builder to create an expression, single quotes are automatically added when you click the **Add** button.





For example:

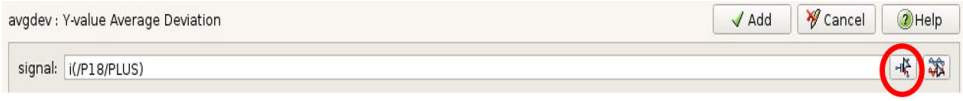
The current signal in waveform file is: *P18:1*

Using single quotes to wrap the signal name in a function: *avgdev('P18:1')*

A voltage signal name: *v(rfout)*

In a function avgdev: *avgdev('v(rfout)')*

- PrimeWave uses schematic signal names. The signal contains “/” before node names. When using the signal in a function, enter it directly, or click **Pick From Schematic** to select the nodes from the schematic.



  For example:

  A current signal name: *i(/P18/PLUS)*

  In a function avgdev: *avgdev(i(/P18/PLUS))*

  A voltage signal name: *v(/rfout)*

  In a function avgdev: *avgdev(v(/rfout))*

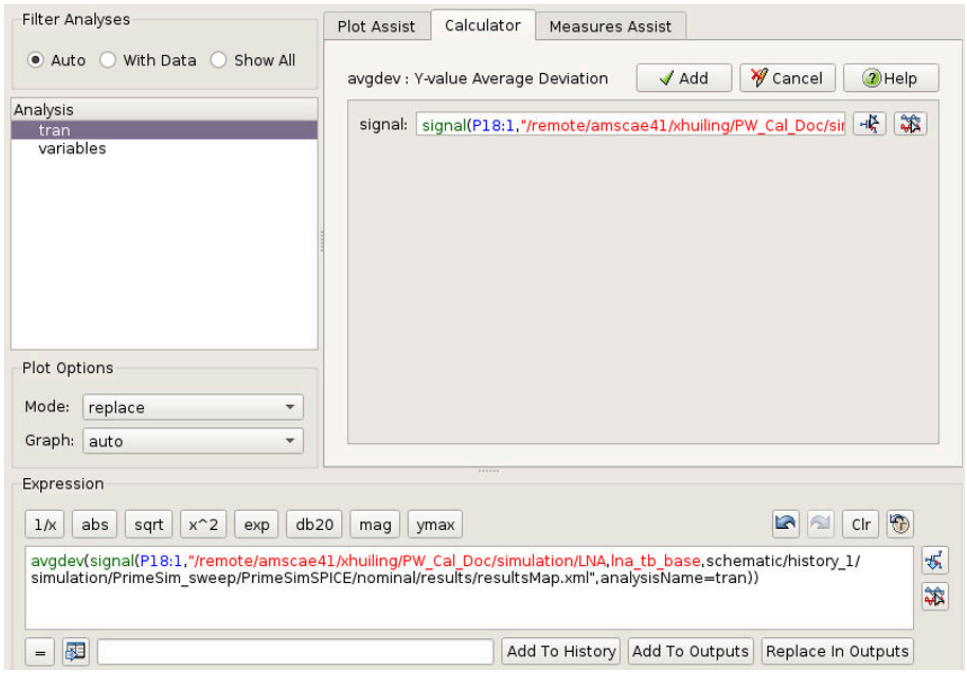
- There is one exception in PrimeWave functions: signal()

  In the PrimeWave Calculator function field, click **Pick from waveform viewer**, and select a waveform from WaveView. The signal is added with the `signal()` function.



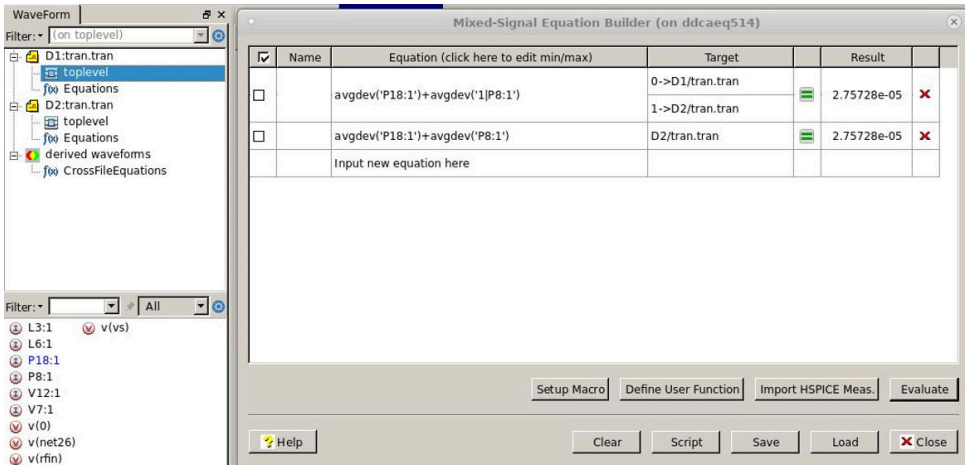
The function retrieves a signal from a waveform file. Use the waveform file signal name syntax, but omit quotation marks when specifying the signal name. For more information, see signal.

For example, add a waveform P18:1 from WaveView to PrimeWave Calculator function `avgdev`:



Add Signals From Different Waveform Files

- Standalone WaveView uses index numbers to add signals from different waveform files.



The first equation in the preceding image uses the signals from two files.

File `D1/tran.tran` is assigned to index 0.

File `D2/tran.tran` is assigned to index 1.

Create an `avgdev` equation, and select the signal P18:1 from `D1/tran.tran`.

Then add “+” with a second `avgdev` function. This time select P8:1 from `D2/tran.tran`.

The final equation is: *avgdev('P18:1')+avgdev('1|P18:1')*

The index of P18:1 is omitted because it is 0.

The second equation in the image uses the signals P18:1 and P8:1 from the same `D2/tran.tran` file. In this case, the file index can be omitted in the entire equation.

Summary of Standalone WaveView syntax with file index number:

- Syntax: `'file_index|signal_name'`

  `file_index` and `signal_name` pair are separated by |, and wrapped by single quotes.

  When file index is 0, the index can be omitted.

  When all signals in one equation are from the same file, the index can be omitted.

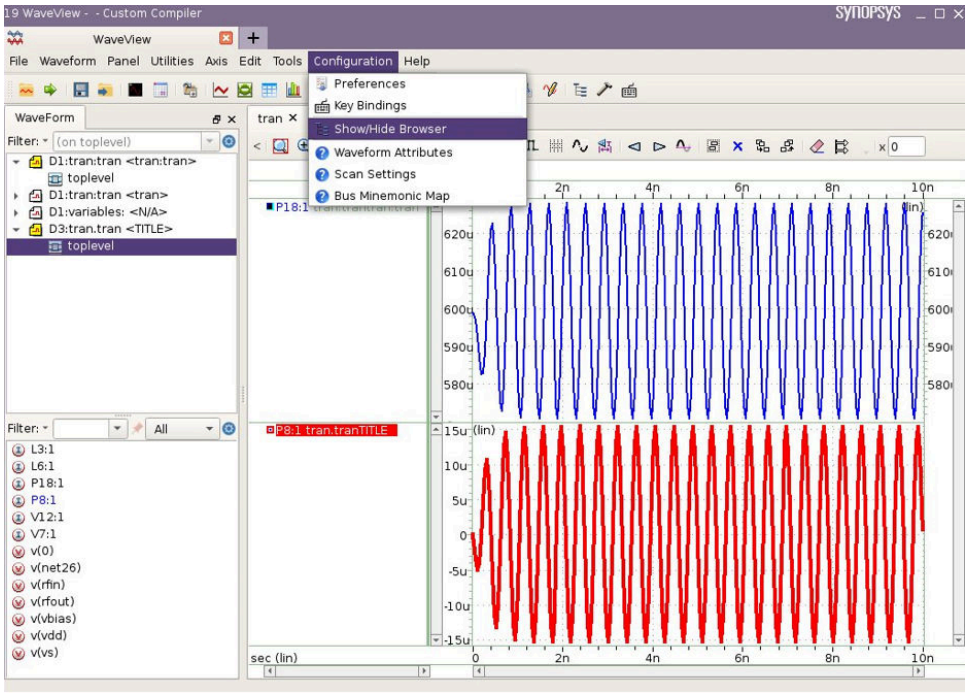
For more information, refer to *Adding Multiple Signals From Different Waveform Files to an Equation* in *WaveView User Guide*.

- PrimeWave uses the `signal()` function to add signals from different waveform files.

  Invoke WaveView from PrimeWave and click **Configuration > Show/Hide Browser** to show the WaveForm Browser.

  Then open two files: `D1:tran.tran` and `D3:tran.tran`

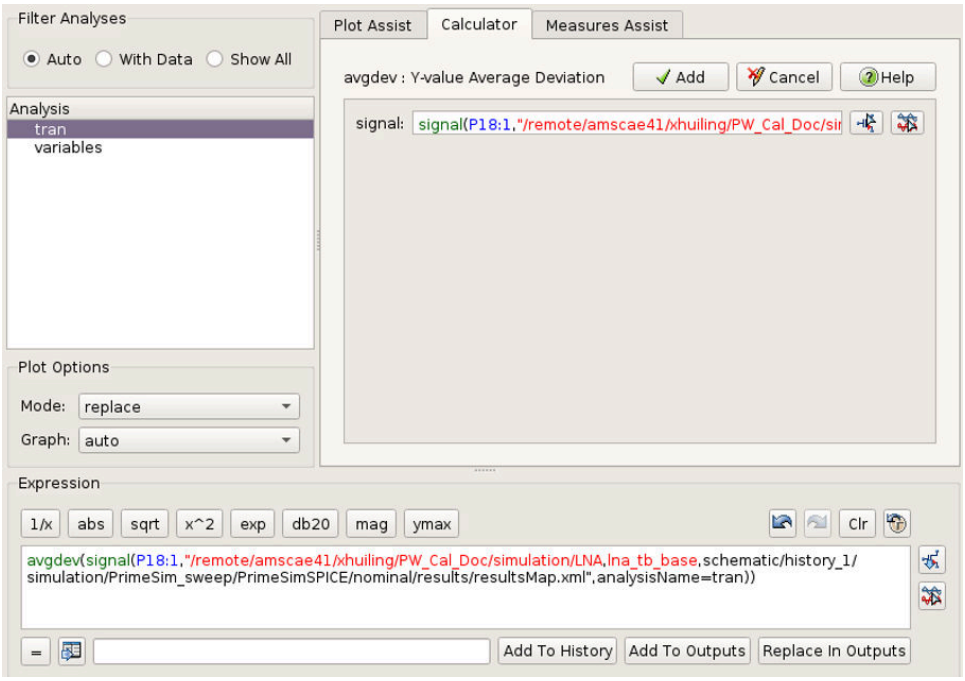
  Plot P18:1 from `D1:tran.tran` and P8:1 from `D3:tran.tran`.



In the PrimeWave Calculator function `avgdev` field, click **Pick from waveform viewer**.

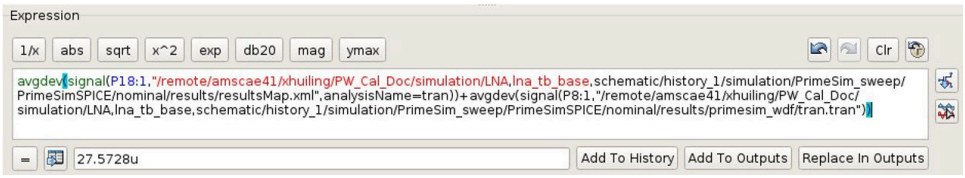


Select P18:1 waveform from WaveView.



In the **Expression** editing filed, append “+” after the first `avgdev()` function, then add the second `avgdev()` function. This time select the P8:1 waveform from WaveView.

The final expression is as shown in the following image.



There are other PrimeWave data functions that can specify the arguments of result type, analysis name, result directory and result file. They are in the category of Data Access Functions. Expressions with these functions can also specify signals from different results. For more information, see .

Operators, Constants, and Logic Operations

This section contains information on how to use the supported operators, constants, and logic operations.

Feedback

This chapter contains the following topics:

- Supported Operators
- Supported Constants
- Supported Logic Operations

Supported Operators

The following table summarizes the supported operators.

Operator/Function	Alias	Description
+		Addition operator
-		Subtraction operator
*		Multiplication operator
/		Division operator
^	**	Power operator
?		Ternary operator
%		Integer operator

Mathematic functions can operate on scalars or waveforms. The returned value can be a scalar (or a waveform) if its input argument is a scalar (or signal).

Supported Constants

The following table summarizes the supported constants.

Constant Name	Variable Name	Value
Pi (π)	c_pi	3.14159265358979
2*Pi	c_2pi	6.28318530717959
Elementary Charge (qe)	c_qe	1.60217662e-19
Euler's number (e)	c_e	2.71828182845905
K	c_k	1.380649e-23

Feedback

Constant Name	Variable Name	Value
sqrt(2)	c_sqrt2	1.41421356237310

Supported Logic Operations

The following table summarizes the supported logic operations.

Operator	Logic Function	Notes
&	AND	Bitwise or unary
|	OR	Bitwise or unary
^	XOR	Bitwise or unary
~	NOT	Bitwise only
>>	SHIFT right	
<<	SHIFT left	
~&	NAND	Unary or binary
~|	NOR	Unary or binary
~^	XNOR	Unary or binary

Note:

You can add the ’b ’o ’d or ’h prefix to a logic operator to specify binary, octal, decimal, or hexadecimal, respectively. For example:

```
6'b010=6'b000010
9'o077=9'b0001111111
4'd010=4'b1010
12'h0F0=12'b000011110000
```

Mathematic Functions

This topic contains information on how to use the supported mathematic functions.

The following topics summarize the mathematic functions available in the Calculator.

Feedback

Function	Description
abs	Measures the absolute value of a given signal or value.
acos	Measures the arc cosine value of a given signal or value.
acosh	Calculates the hyperbolic arch-cosine of the signal.
asin	Measures the arc sine value of a given signal or value.
asinh	Calculates the hyperbolic arch-sine of the signal.
atan	Measures the arc tangent value of a given signal or value.
atanh	Calculates the hyperbolic arch-tangent of the signal.
complex	Returns a complex scalar or waveform with x as the real part and y as the imaginary part.
conjugate	Returns the conjugate of a complex number.
cos	Measures the cosine value of a given signal or value (in radian units).
cosh	Measures the hyperbolic cosine value of a given signal.
db10	Measures the db10 value of a given signal.
db20	Measures the db20 value of a given signal.
dbm	Measures the dbm value of a given signal or value.
exp	Measures the exponential value of a given signal or value.
floor	Calculates the largest integer not greater than the value of given signal or value.
imag	Measures the imaginary part of a value of the given signal or value.
int	Calculates the rounded integer of the value of a given signal or value.
ln	Measures the natural logarithm value of given signal or value if the parameter of a signal is greater than 0, or 0 if the parameter is less than 0.
log10	Measures the base-10 logarithmic value of given signal or value if the parameter of a signal is greater than 0, or 0 if the parameter is less than 0.
mag	Measures the magnitude value of given signal or value.
max	Compares two signals or values and calculates the larger of the two signals or values.
min	Compares two signals or values and calculates the smaller of the signals or values.

Feedback

Function	Description
phase	Measures the phase value of given signal or value.
pwr	Calculates the power value of a given signal or value.
real	Measures the real part of a value of the given signal or value.
round	Rounds the given signal or value to an integer value and returns the nearest integer value of the float, double, or long double argument.
sgn	Calculates the sgn value of a given signal or value.
sin	Measures the sine value of the given signal or value (in radian units).
sinh	Measures the hyperbolic sine value of the given signal or value.
sqrt	Measures the square root value of the given signal or value (in radian units).
tan	Measures the tangent value of the given signal or value (in radian units).
tanh	Measures the hyperbolic tangent value of the given signal or value.

abs

Measures the absolute value of a given signal or value.

Syntax

```
abs(<signal>,[<type>])
```

Returns

A signal or scalar value.

Arguments

`<signal>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<type>`

(Optional) Specifies to perform the function in a discrete or continuous way. Default value is discrete.

Feedback

Errors

Possible error messages include:

- Signal not found

Examples

```
abs('0|v(out)')
abs(-5)=5
```

```
abs($sig,'discrete')
```

performs `abs` in a discrete way.

```
abs($sig,'continuous')
```

perform `abs` in a continuous way.

---

acos

Measures the arc cosine value of a given signal or value.

Syntax

```
acos(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error (>1, <-1)

Examples

```
acos('0|v(out)')
acos(0.6)=927m
```

Feedback

acosh

Description

Calculates the hyperbolic arch-cosine of the signal. If the value is out of domain [1,∞), a warning message will appear.

Syntax

```
acosh(<signal>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured. The value should be in the range of [1,∞).

asin

Measures the arc sine value of a given signal or value.

Syntax

```
asin(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error (>1, <-1)

Examples

```
asin('0|v(out)')
asin(0.5)=524m
```

Feedback

asinh

Description

Calculates the hyperbolic arch-sine of the signal.

Syntax

```
asinh(<signal>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

atan

Measures the arc tangent value of a given signal or value.

Syntax

```
atan(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
atan('0|v(out)')
atan(10)=1.47
```

Feedback

atanh

Description

Calculates the hyperbolic arch- tangent of the signal. If the value is out of domain (-1,1), then a warning will be given.

Syntax

```
atanh(<signal>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured. The value should be in the range of (-1,1).

complex

Returns a complex scalar or waveform with x as the real part and y as the imaginary part. Both x and y need to be real values or signals.

Syntax

```
complex(<real signal>|<value>,<imag signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<real_signal>|<value>`

The real signal or value.

`<imag_signal>|<value>`

The imaginary signal or value.

Errors

Possible error messages include:

- Signal not found(Real/Imag)

Feedback

Examples

```
complex('0|v(out)','0|v(out)')
complex(2,3)=0
```

---

conjugate

Description

Returns the conjugate of a complex number. The conjugate of a complex number is a number with opposite sign at the imaginary part.

Syntax

```
conjugate(<signal>)
```

Returns

A complex signal.

Arguments

`<signal>`

The signal to be calculated. Complex signals are supported.

---

cos

Measures the cosine value of a given signal or value (in radian units).

Syntax

```
cos(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Feedback

Examples

```
cos('0|v(out)')
cos(60)=-952m
```

---

cosh

Measures the hyperbolic cosine value of a given signal. The hyperbolic cosine function uses the following equation for calculations: $\cosh(x) = \frac{e^{x} + e^{-x}}{2}$

Syntax

```
cosh(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
cosh('0|v(out)')
cosh(60)=-952m
```

---

db10

Measures the db10 value of a given signal. The db10 function uses the following equation for calculations: $db10(x) = 10 \cdot \log(x)_{10}$

Syntax

```
db10(<signal>|<value>)
```

Returns

A signal or scalar value.

Feedback

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error(x<=0)

Examples

```
db10('0|v(out)')
db10(10)=10
```

---

db20

Measures the db20 value of a given signal. The db20 function uses the following equation for calculations: $db20(x) = 20 \cdot \log(x)_{10}$

Syntax

`db20(<signal>|<value>)`

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error(x<=0)

Examples

```
db20('0|v(out)')
db20(10)=20
```

Feedback

dbm

Measures the dbm value of a given signal or value. The dbm function uses the following equation for calculations:$dbm(x) = 30 + \log\ (x)_{10}$

Syntax

```
dbm(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error(x<=0)

Examples

```
dbm('0|v(out)')
dbm(10)= 40
```

exp

Measures the exponential value of a given signal or value.

Syntax

```
exp(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Feedback

Errors

Possible error messages include:

- Signal not found

Examples

```
exp('0|v(out)')
exp(2)=7.39
```

---

floor

Calculates the largest integer not greater than the value of given signal or value.

Syntax

```
floor(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
floor('0|v(out)')
floor(2.5)=2
```

---

imag

Measures the imaginary part of a value of the given signal or value.

Syntax

```
imag(<signal>|<value>)
```

Returns

A signal or scalar value.

Feedback

Arguments

`<signal>|<value>`

The imaginary part of a signal or value, or 0 if the signal or value is real.

Errors

Possible error messages include:

- Signal not found

Examples

```
imag('0|v(out)')
imag(23)=0
```

---

int

Calculates the rounded integer of the value of a given signal or value.

Syntax

`int(<signal>|<value>)`

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
int(-1.2)=-1
```

---

ln

Measures the natural logarithm value of given signal or value if the parameter of a signal is greater than 0, or 0 if the parameter is less than 0.

Feedback

Syntax

```
ln(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error(x<0)

Examples

```
ln('0|v(out)')
ln(2.5)=916m
```

---

log10

Measures the base-10 logarithmic value of given signal or value if the parameter of a signal is greater than 0, or 0 if the parameter is less than 0.

Syntax

```
log10(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Feedback

Errors

Possible error messages include:

- Signal not found
- Domain error(x<0)

Examples

```
log10('0|v(out)')
log10(10)=1
```

---

mag

Measures the magnitude value of given signal or value.

Syntax

```
mag(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
mag('0|v(out)')
mag(-2.5)=2.5
```

---

max

Compares at least two (and up to 32) signals or values and calculates the larger of the signals or values.

Syntax

```
max(<signal1>|<value1>,<signal2>|<value2>,[<signal3>|<value3> ...
 <signal32>|<value32>])
```

Feedback

Returns

A signal or scalar value.

Arguments

`<signal1>|<value1>`

The first signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<signal2>|<value2>`

The second signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<signal3>|<value3> ... <signal32>|<value32>`

(Optional) The third through thirty-second signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
max('0|v(out)','0|v(out)')
max(2,3)=3
```

---

min

Compares at least two (and up to 32) signals or values and calculates the smaller of the signals or values.

Syntax

```
min(<signal1>|<value1>,<signal2>|<value2>,[<signal3>|<value3> ...
 <signal32>|<value32>])
```

Returns

A signal or scalar value.

Arguments

`<signal1>|<value1>`

The first signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Feedback

`<signal2>|<value2>`

The second signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<signal3>|<value3> ... <signal32>|<value32>`

(Optional) The third through thirty-second signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
min('0|v(out)','0|v(out)')
min(1,2)=1
```

---

phase

Measures the phase value of given signal or value.

Syntax

```
phase(<signal>|<value>,[<type>],[<wrapped>])
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported.

`[<type>]`

(Optional) The return signal shows in degrees or radians.

`[<wrapped>]`

(Optional) The return signal range from -180(-ππ) to 180(π) or not.

Errors

Possible error messages include:

- Signal not found

Feedback

Examples

```
phase('0|v(out)')
phase(10)=0
```

---

pwr

Calculates the power value of a given signal or value using the following equation: $pwr(x) = \begin{cases} x^y & x \geq 0 \\ (-x)^y & x < 0 \end{cases}$

Syntax

```
pwr(<signal1>|<value1>,<signal2>|<value2>)
```

Returns

A signal or scalar value.

Arguments

`<signal1>|<value1>`

The first signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<signal2>|<value2>`

The second signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- (Sig1/Sig2)

Examples

```
pwr('0|v(out)','0|v(out)')
pwr(2,4)=16
```

---

real

Measures the real part of a value of the given signal or value.

Syntax

```
real(<signal>|<value>)
```

Feedback

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported.

Errors

Possible error messages include:

- Signal not found

Examples

```
real('0|v(out)')
real(2)=2
```

---

round

Rounds the given signal or value to an integer value and returns the nearest integer value of the float, double, or long double argument passed to this function. If a decimal value is between `.1` and `.5`, an integer value less than the argument is returned. If a decimal value is between `.6` and `.9`, the integer value is greater than the argument.

Syntax

`round(<signal>|<value>)`

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
round('0|v(out)')
round(2.6)=3
```

Feedback

sgn

Calculates the sgn value of a given signal or value using the following equation: $\mathrm{sgn}\ (x) = \begin{cases} 1 if\ x > 0 \\ -1 if\ x < 0 \end{cases}$

Syntax

```
sgn(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
sgn('0|v(out)')
sgn(2)=1
```

sin

Measures the sine value of the given signal or value (in radian units).

Syntax

```
sin(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Feedback

Errors

Possible error messages include:

- Signal not found

Examples

```
sin('0|v(out)')
sin(20)=913m
```

---

sinh

Measures the hyperbolic sine value of the given signal or value using the following equation: $\sin h(x) = \frac{e^{x}\text{-}e^{-x}}{2}$

Syntax

```
sinh(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
sinh('0|v(out)')
sinh(2)=3.63
```

---

sqrt

Measures the square root value of the given signal or value (in radian units).

Syntax

```
sqrt(<signal>|<value>)
```

Feedback

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Domain error(x<0)

Examples

```
sqrt('0|v(out)')
sqrt(5)=2.24
```

---

tan

Measures the tangent value of the given signal or value (in radian units).

Syntax

`tan(<signal>|<value>)`

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
tan('0|v(out)')
tan(30)=-6.41
```

Feedback

tanh

Measures the hyperbolic tangent value of the given signal or value using the following equation: $\tan h(x) = \frac{e^x - e^{-x}}{e^x + e^x}$

Syntax

```
tanh(<signal>|<value>)
```

Returns

A signal or scalar value.

Arguments

`<signal>|<value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found

Examples

```
tanh('0|v(out)')
tanh(2)=964m
```

Measurement Functions

This topic contains information on how to use the supported measurement functions.

The following sections summarize the measurement functions available in the Calculator.

Function	Description
acrmsval	Returns the accumulated Root Mean Square value for the waveform.
avgdev	Calculates the average deviation of a given signal.
avgfreq	Calculates the average frequency of a given signal.
bw	Returns the bandwidth of a given signal when the signal strength drops below a bound dB from its DC value.

Feedback

Function	Description
bw3db	Equivalent to specifying bw(<signal>, 3), where the dB threshold is fixed to 3dB.
correlation	Calculates the correlation coefficient of the Y values of given signal versus the X values of a given signal.
cross	Calculates the correlation coefficient of the Y values of given signal versus the X values of a given signal.
dBCP	Calculates a 1 dB Compression Point (dBCP).
dBCPh	Returns the nth order intercept point.
dBCPhVRI	dB compression point VRI.
dcgain	Calculates the X value of a given signal at the min gain.
dcgamos	Calculates the minimum value of a signal equivalent to the min(gain1,gain2) function.
dcswing	Calculates the Y value of a given signal at the min gain.
delay	Calculates the time difference between two signals by the edge index and at the specified switching edges.
direct_ipn	Calculates hbac/pac ipn.
dutycycle	Calculates the duty cycle of a given period signal at the specified edges of edge indexes.
enb	Calculates the effective number of bits of a given signal. The FFT is calculated first, then the SNDR.
eye_mask	Adds a mask on to the existing eye object.
falltime	Calculates the time difference of a given signal between the high y_threshold crossing and the low y_threshold crossing.
falltrig	Calculates the X value of the nth falling edge over threshold Y.
flatten	Flattens the given signal with the given parameter as the new x-axis.
gain1_f	Calculates the frequency of a given signal at the signal magnitude, which is 0db(1).
gain_m	Calculates the gain margin of a given signal, which is the amount of gain increase or decrease required to make the loop gain unity at the frequency.

Feedback

Function	Description
gbp	Calculates the gain margin of a given signal, which is the amount of gain increase or decrease required to make the loop gain unity at the frequency.
get_harmonic_freq	Calculates the frequency of the nth harmonic.
globalmax	Returns the global maximum Y value of all data points for signal.
globalmean	Returns the global discrete mean value of all data points for a signal.
globalmin	Returns the global minimum Y value of all data points for signal.
globalstddev	Returns the global standard deviation value of all data points for signal.
integ	Calculates the integral value of a given signal from x1 to x2.
ipn	Calculates the n-order intercept point between the target signal and reference signal.
ipnh	Returns the nth order intercept point.
ipnhVRI	N dB intercept point VRI.
mean	Calculates the mean value of a given signal or value from x1 to x2.
measure_aperture	Applies the Aperture measurements to the eye object returned from the create_eye function.
measure_eyejitter	Calculates Crossing Average, Crossing StdDev, Random Jitter (left), Random Jitter (right), and Deterministic Jitter of a signal using Eye Diagram.
measure_eyeopen	Measures the height and width of the eye opening.
measure_eyesize	Applies the Eye Size measurements to the eye object returned from the create_eye function.
median_val	Calculates the median value of a given signal or value.
overshoot	Calculates the overshoot value of a given signal.
p2p	Calculates peak-to-peak, the difference between the maximum and minimum Y values of the given signal.
pamn_level_at_eye_center	Measures the PAMN eye diagram level at eye center.
pamn_level_at_user_ref	Measures the PAMN eye diagram level at user reference.

Feedback

Function	Description
pamn_open_at_eye_center	Measures the PAMN eye diagram open at eye center.
pamn_open_at_user_ref	Measures the PAMN eye diagram open at user reference.
paramvalue	Calculates the value of a P parameter.
phase_m	Calculates the phase margin.
risetime	Calculates the time difference of given signal between low threshold crossing and high threshold crossing beginning at the X value.
risetrig	Measures the X value of the nth rising edge over the y_threshold starting from the x_start value.
rmsval	Measures the RMS value of given signal.
root	Computes the value of x at which f(x) equals the specified threshold.
settling_t	Measures the settling time of given signal.
sfdr	Measures the spurious free dynamic range.
signalPower	Measures the Signal Power in support of spectrum measurements.
slewrate	Measures the slew rate of given signal.
slope	Measures the slope value of the given signal at x value.
snb	Measures the Noise floor/Bin in support of spectrum measurements.
sndr	Measures the signal to noise-and-distortion ratio of a given signal.
snr	Measures the signal-to-noise ratio of given signal.
snr_f	Measures the signal-to-noise ratio of given signal, which is the resulting waveform of a Fast Fourier Transform (FFT) signal
snrh	Measures the Noise floor/rtHZ in support of spectrum measurements.
stddev	Measures the standard deviation of the Y values for given signal and the X values are discarded.
thd	Measures the total harmonic distortion of a given signal.
variance	Measures the variance of the Y values for given signal.
x_at_ymax	Measures the x value at the minimum y value of given signal from x_start to x_stop.

Feedback

Function	Description
x_at_ymin	Measures the x value at the minimum y value of given signal from x_start to x_stop.
xmax	Measures the maximum x value of a given signal.
xmin	Measures the minimum x value of a given signal.
Yield	Measures the count of the value specified by spec_min and spec_max and provides the value for (valid_count/total_count)*100.
ymax	Measures the maximum y value of a given signal.
ymin	Measures the minimum y value of a given signal.
yvalue	Measures the y value of given signal at x.

acrmsval

Returns the accumulated Root Mean Square value for the waveform. The following equation is for continuous waveform: $acrmsval = \sqrt{\frac{\int_{x_{start}}^{x_{stop}} [f(x)\text{-mean}\{f(x)\}]^2 dx}{ABS(x_{stop}\text{-}x_{start})}}$

Syntax

```
rmsval(<signal>,[<x_start>],[<x_stop>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`[<x_start>]`

(Optional) The starting value of X.

`[<x_stop>]`

(Optional) The ending value of X.

Feedback

Errors

Possible error messages include:

- Signal not found
- X value out of range

---

avgdev

Calculates the average deviation of a given signal.

Syntax

```
avgdev(<signal>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
avgdev('0|v(out)')
```

---

avgfreq

Calculates the average frequency of a given signal.

Syntax

```
avgfreq(<signal>,[<x_start>],[<x_stop>])
```

Returns

An analog scalar value.

Feedback

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`[<x_start>]`

(Optional) The starting x value. The default is the signal starting time.

`[<x_stop>]`

(Optional) The stopping x value. The default is the signal ending time.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- X value out of range

Examples

```
avgfreq('0|v(out)')
  avgfreq('0|v(out)',2n,8n)
  avgfreq(4)=201M
```

---

bw

Returns the bandwidth of a given signal when the signal strength drops below a bound dB from its DC value.

Syntax

`bw(<signal>,<db_band>,[<band_type>],[<topline>],[<percent>])`

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<db_band>`

The dB threshold.

Feedback

`[<band_type>]`

(Optional) The type of measurement (`lowpass`, `highpass`, `bandpass`, or `stopband`). The default is `lowpass`.

`[<topline>]`

(Optional) The top line reference (`firstpoint`, `detect`, a dB value, or a percentage value). The default is `firstpoint`.

`[<percent>]`

(Optional) When specified as 1, the <topline> is a percentage value.

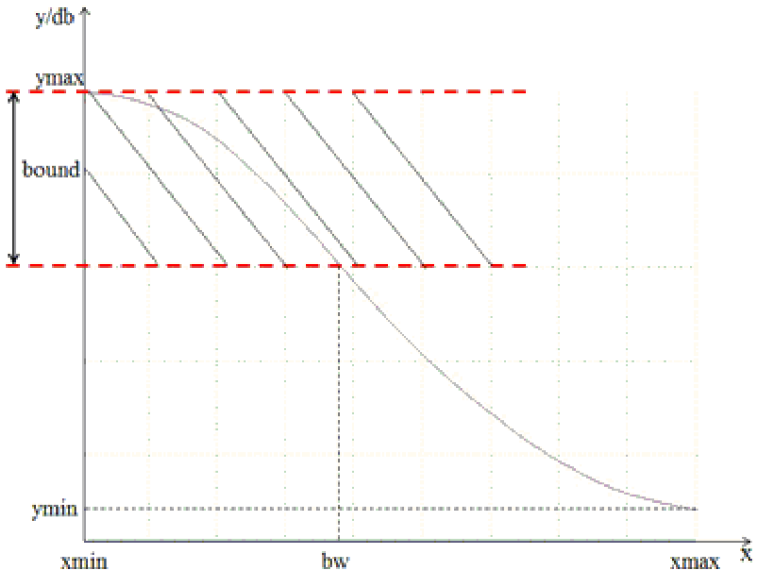
Errors

Possible error messages include:

- Signal not found Incorrect signal type
- Db value out of range

Examples

```
bw('0|v(out)','lowpass',topline=firstpoint,percent=1)
```



bw3db

Equivalent to specifying `bw(<signal>, 3)`, where the dB threshold is fixed to 3dB.

Feedback

Syntax

```
bw3db(<signal>,[<band_type>],[<topline>],[<percent>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`[<band_type>]`

(Optional) The type of measurement (`lowpass`, `highpass`, `bandpass`, or `stopband`). The default is `lowpass`.

`[<topline>]`

(Optional) The top line reference (`firstpoint`, `detect`, a dB value, or a percentage value). The default is `firstpoint`.

`[<percent>]`

(Optional) When specified as 1, the `<topline>` is a percentage value.

Errors

Possible error messages include:

- Signal not found.
- Incorrect signal type.
- DB value out of range.

Examples

```
bw3db('0|v(out)','lowpass',topline=firstpoint,percent=1)
```

---

correlation

Calculates the correlation coefficient of the Y values of given signal versus the X values of a given signal. The following functions represent the correlation coefficient: $corr(X,Y) = \frac{cov(X,Y)}{\sigma_X \sigma_Y}$ $cov(X,Y) = \sum_{i=1}^{n} (X_i - \bar{X})(Y_i - \bar{Y})$ σ_X, σ_Y are the standard deviations of X and Y, where: $\sigma X = \sqrt{\sum_{i=1}^{n} (X_i - \bar{X})^2}$ $\sigma Y = \sqrt{\sum_{i=1}^{n} (Y_i - \bar{Y})^2}$

Feedback

Syntax

```
correlation(<signal>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found.

Examples

```
correlation('0|v(out)')
correlation(1)=0
```

---

cross

Calculate the x-value of a given signal at y value crossing. Supports both positive and negative count indexes. If positive, it will find the cross from the beginning to the end. If negative, it will find the cross from the end to beginning.

Syntax

```
cross(<signal>,<y_level>,<edge_index>,[<x_start>],
[<percent>],[<trigger_type>],[<occurences>],
[<output_xAxisType>],[<tolerance>])
```

Returns

An analog scalar value. The `cross( )` function returns the x value of the nth crossing edge over threshold `y_level` beginning at `x_start`.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`<y_level>`

The threshold of the Y value.

Feedback

`<edge_index>`

An integer value, which can be `0`, `1`, …, `n`.

`[<x_start>]`

(Optional) The starting point of an X value of a signal.

`[<percent>]`

(Optional) The percentage value, which can only be `0` or `1`. If specified as `1`, the y_level is considered a percentage value; if not, Y is not be percentage value.

`[<trigger_type>]`

(Optional) The trigger type (`fall`, `rise`, or `either`). The `fall` argument specifies the wave crossing from above the y_level to below the y_threshold.

`[<occurences>]`

(Optional) Specifies to retrieve only one occurrence of an overshoot event for the given waveform (single), or all occurrences of overshoot for the given waveform which you can later plot or print (multiple).

`[<output_xAxisType>]`

(Optional) Specifies the X axis type of the result waveform. The value can be time or idx.

`[<tolerance>]`

(Optional) Determines the threshold crossings and filters crossings with amplitudes smaller than tolerance.

Errors

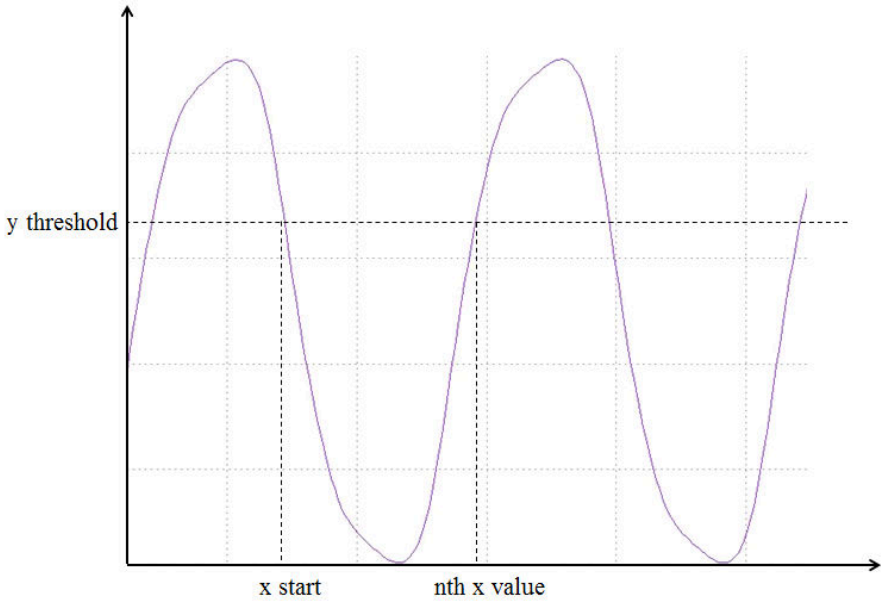
Possible error messages include:

- Signal not found. Y value out of range.

Examples

```
cross('0|v(out)',3,2,2n,1,trig='fall')
```

Feedback



Both positive and negative count indexes are supported. If the count index is positive, the cross is found from the beginning to the end. If the count index is negative, the cross is found from the end to the beginning.

In the following example, the x-value of the second edge from the beginning of the waveform when it crosses the 0.5 level is reported.

```
cross(v(/vin),0.5,2,trig=rise)
```

In the following example, the x-value of the second edge from the end of the waveform (or the second-last edge) when it crosses the 0.5 level is reported.

```
cross(v(/vin),0.5,-2,trig=rise)
```

---

dBCP

Description

Calculates a 1 dB Compression Point (dBCP). When the amplitude of the input signal is small enough, which is almost the same as (linear) AC analysis. When the input signal grows larger, the circuit response is saturated and the output amplitude does not linearly increase.

Feedback

You can calculate the 1dB compression point from HB SWEEP results. Sweep the input amplitude and measure the output amplitude. The algorithm includes:

- Calculating the value of gain by using extrapolation point and the waveform `y=f(x)`.
- Generating a linear function with `slope=1: y=x+gain` (where `gain` is known).
- Finding the point on the waveform that satisfies the following parameters: the difference between the Y value of the point (on the waveform) and the Y value of the point (on the linear function) equals `N`. `N` is gain decrease (for example, for `P1dB`, `N=1`; and for `P2dB`, `N=2`.

Syntax

```
dBCP(<signal>,<gain_decrease>,<extrapolation_point>,
[<input|output>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<gain_decrease>`

The difference of the Y value between a point on waveform and a point on the `y=x+gain` linear function.

`<extrapolation_point>`

The X value that needs to be input. According to the X value and the `y=f(x)` input signal, the system gets the Y value and then generates a `y=x+gain -> gain=y-x` linear function.

`[<input|output>]`

(Optional) Option to choose whether the results are input-based or output-based.

Errors

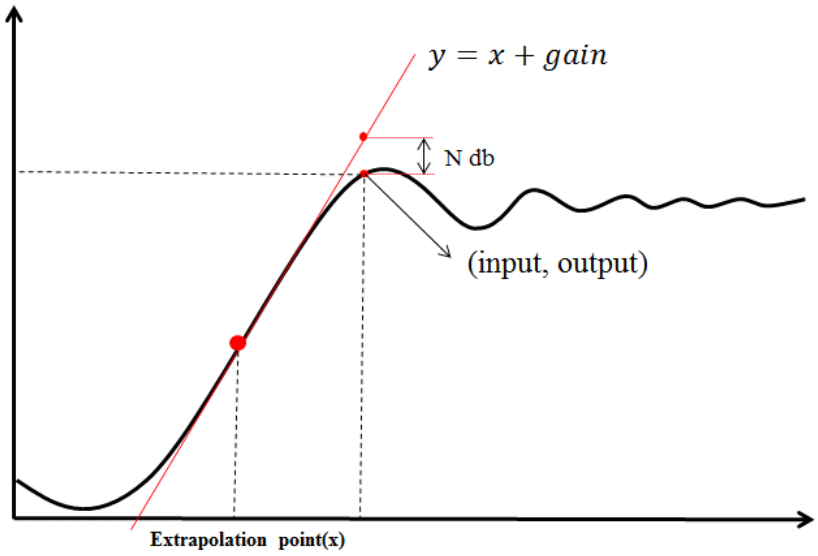
Possible error messages include:

- Signal not found. Extrapolation point X value out of range.

Examples

```
dBCP('0|v(out)',2n,4n,'input')
```

Feedback



dBCPh

Description

Returns the nth order intercept point.

Syntax

```
dBCPh (signal <string>,
     harmonicIndices <string>
     [, gainDecrease <string>]
     , extrapolationPoint <string>
     [, referred <enum>]
     [, sweepUnits <enum>]
     [, resultType <string>]
     [, resultsDir <string>]
     [, resultFile <string>]
     [, testbenchName <string>]
     [, analysisName <string>]
     [, params <string>])
```

Arguments

`signal`

The signal from which to extract the harmonic.

Feedback

`harmonicIndices`

The harmonic indices, space delimited.

`gainDecrease`

(Optional) The gain decrease. (Default: 1)

`extrapolationPoint`

Input signal extrapolation point.

`referred`

(Optional) The Input/Output referred measurement. Valid values: input, output. (Default: input)

`sweepUnits`

(Optional) The units of the input power sweep.

Valid values: W, dBm, and none. (Default: dBm)

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

`params`

(Optional) The parameter conditions.

---

dBCPhVRI

Description

dB compression point VRI.

Feedback

Syntax

```
dBCPhVRI(<voltage_signal>,<harmonic_index>,[<current_signal>],
 [<resistance>],[<gainDecrease>],[<extrapolationPoint>],[<sweepUnits>],[<
format>],[<referred>])
```

Returns

A multiple traces signal.

The first trace is the power or gain curve. The second trace is the extrapolation line.

Arguments

`<voltage_signal>`

The input voltage signal.

`<harmonic_index>`

Harmonic index.

`<current_signal>`

(Optional) The input current signal.

`<resistance>`

(Optional) Resistance.

`<gainDecrease>`

(Optional) The gain decrease in db. The default is 1.

`<extrapolationPoint>`

(Optional) The input signal extrapolation point.

`<sweepUnits>`

(Optional) Can be `dBm`, `W`, or `none`. Default is `dBm`.

`<format>`

(Optional) Can be `power` or `gain`. Default is `power`.

`<referred>`

(Optional) Input/Output referred measurement. Default is `input`.

---

dcgain

Calculates the dcgain value of a given signal at the min gain. The min gain is calculated using the `d(out)/d(in)` equation, where `d(out)` is the difference in Y value (`y2-y1`) and `d(in)` is the difference in X value (`x2-x1`). When this function is executed, a plot of

Feedback

`d(out)/d(in)` vs `d(in)` is displayed, which gets the values of `d(in)` that correspond to each min gain.

Syntax

```
dcgain(<signal>,<min_gain>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<min_gain>`

The value of `d(out)/d(in)`.

Errors

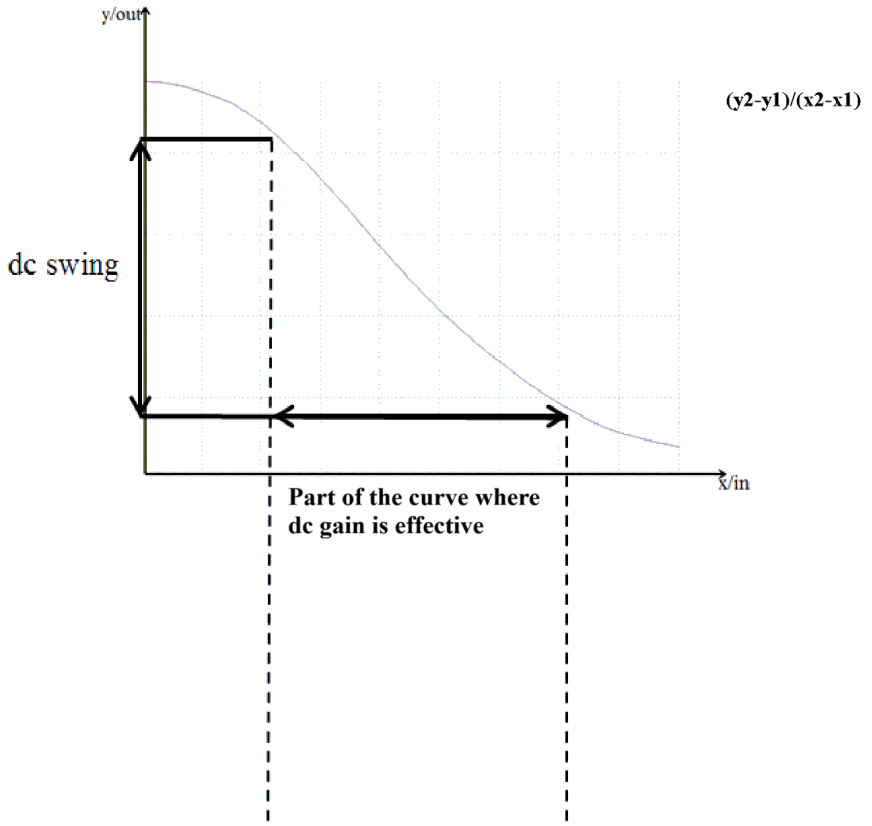
Possible error messages include:

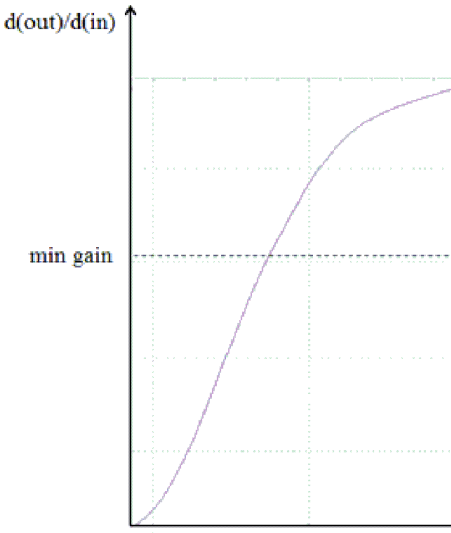
- Signal not found
- DC gain curve does not cross minimum gain level
- Ill-conditioned DC gain curve

Examples

```
dcgain('0|v(out)',4n)
```

Feedback





dcgamos

Calculates the minium value of a signal equivalent to the `min(gain1,gain2)` function. `gain1` is the value of `d(out)/d(in)` for the lower X value, `gain2` is the value of `d(out)/d(in)` for the higher X value— the DC gain of given signal at maximum output swing is returned.

Syntax

`dcgamos(<signal>,<output_swing>,[<percent>])`

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Feedback

`<output_swing>`

The Y value. Two corresponding X values exist for each output swing.

`[<percent>]`

(Optional) The percentage value, which can only be `0` or `1`. If specified as `1`, the y_level is considered a percentage value; if not, Y is not be percentage value.

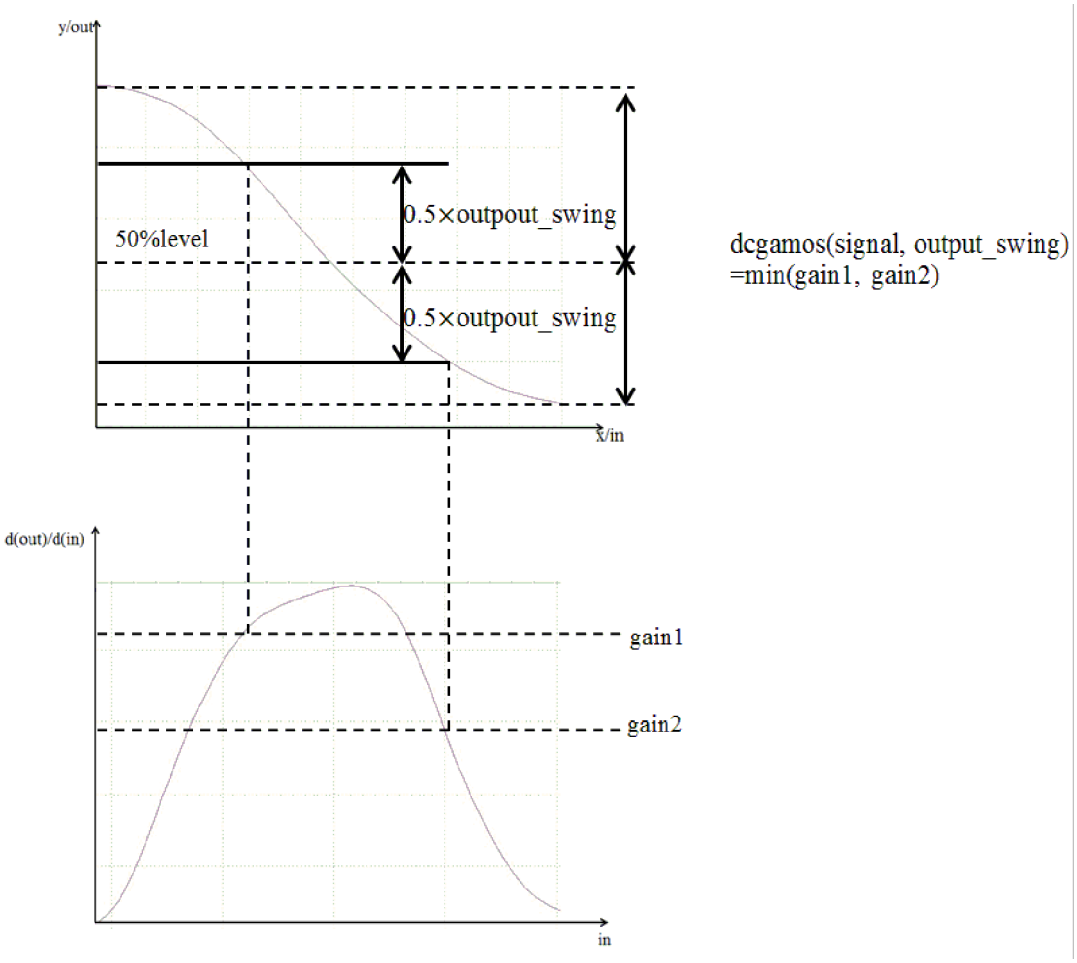
Errors

Possible error messages include:

- Signal not found
- Ill-conditioned DC gain curve

Examples

```
dcgamos('0|v(out)',4n,1)
```



Feedback

dcswing

Calculates the Y value of a given signal at the min gain. The min gain is calculated using the `d(out)/d(in)` equation, where `d(out)` is the Y value and `d(in)` is the X value. When this function is executed, a plot of `d(out)/d(in)` versus `d(in)` is displayed, which gets the values of `d(in)` that correspond to each min gain. The Y value is returned according to the value of `d(in)`.

Syntax

```
dcswing(<signal>,<min_gain>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`<min_gain>`

The value of `d(out)/d(in)`.

Errors

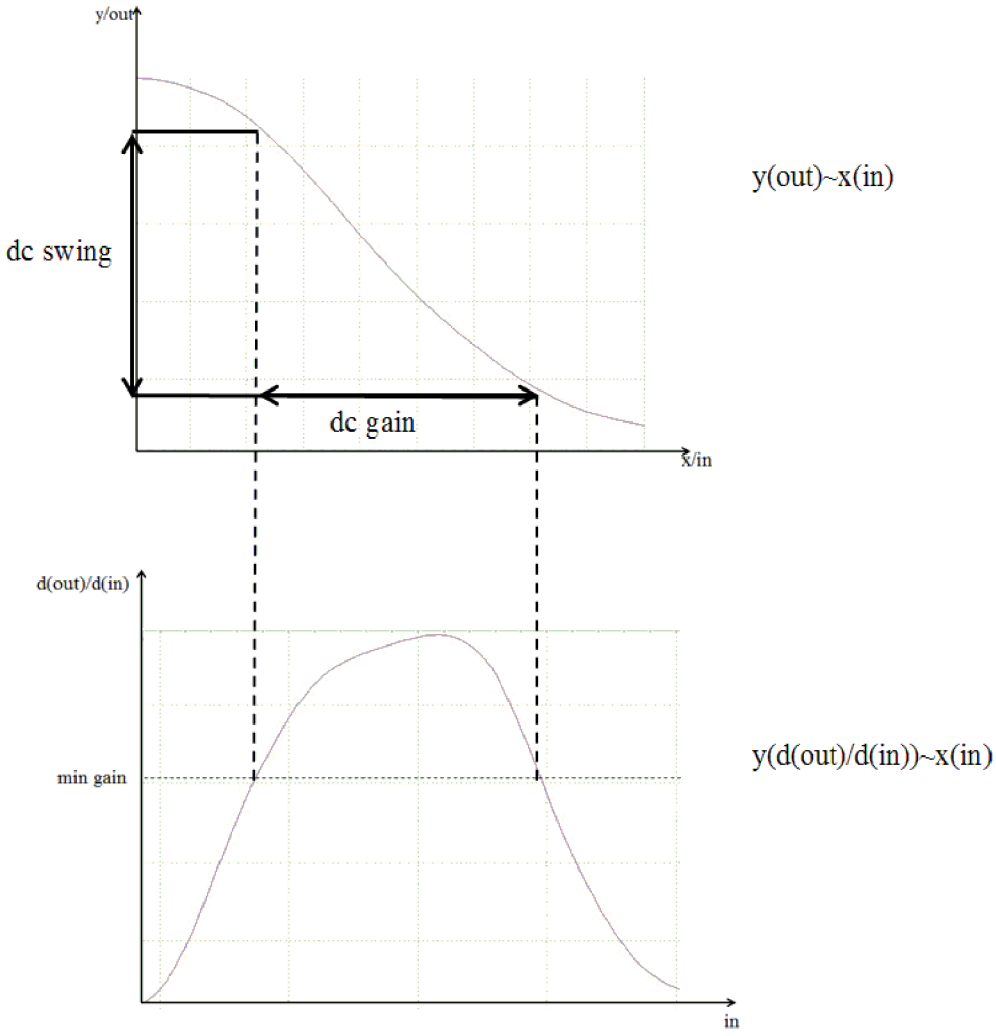
Possible error messages include:

- Signal not found
- DC gain curve does not cross minimum gain level
- Ill-conditioned DC gain curve

Examples

```
dcswing('0|v(out)',4n)
```

Feedback



delay

Calculates the time difference between two signals by the edge index and at the specified switching edges.

Syntax

```
delay(<trigger_signal>,<target_signal>,<trigger_y_level>,
<target_y_level>,<trigger_x_index>,<target_x_index>,
[<trigger_x_start>],[<percent>],[<target_x_start>],
[<target_start_relative_to>],[<x_stop>],
[<trigger_edge_type>],[<target_edge_type>],
[<trigger_periodicity>],[<target_periodicity>],
[<trigger_tolerance>],[<target_tolerance>],
[<occurrences>],[<output_xaxistype>],[<premise>])
```

Feedback

Returns

An analog scalar value.

Arguments

`<trigger_signal>`

The reference signal.

`<target_signal>`

The signal to be calculated. Analog signals are supported.

`<trigger_y_level>`

The y_threshold for the trigger signal.

`<target_y_level>`

The y_threshold for the target signal.

`<trigger_x_index>`

The index value for the trigger signal.

`<target_x_index>`

The index value for the target signal.

`[<trigger_x_start>]`

The time value on the x-axis (the start time for the trigger signal).

`[<percent>]`

(Optional) The percentage value, which can only be `0` or `1`. If specified as `1`, the y_level is considered a percentage value; if not, Y is `0`.

`[<target_x_start>]`

The start time for the target signal.

`[<target_start_relative_to>]`

Specifies whether the start time for the target signal is relative to the trigger signal start time or time.

`[<x_stop>]`

The stop time for the target signal.

`[<trigger_edge_type>]`

The edge type of the crossing for the trigger signal. There are three options: rising, falling, or either.

Feedback

`[<target_edge_type>]`

The edge type of the crossing for the target signal. There are three options: rising, falling, or either.

`[<trigger_periodicity>]`

The periodic interval for the trigger signal. The delay algorithm for multiple occurrences returns the difference between the X points for the specified edges on the target and trigger waveforms (with the specified periodicity).

For example, if the edge number specified for both waveforms is 1 and periodicity is 2, then it computes the difference between the 1,3,5,7... edges for both the waveforms.

`[<target_periodicity>]`

The periodic interval for the target signal.

`[<trigger_tolerance>]`

The tolerance value to detect the threshold crossings for the trigger signal.

`[<target_tolerance>]`

The tolerance value to detect the threshold crossings for the target signal.

`[<occurrences>]`

Specifies whether you want to retrieve only one occurrence of a delay time event (`single`), or all occurrences of delay time events (`multiple`). The default is `single`.

`[<output_xaxistype>]`

Specifies whether you want to output the delay time data against the trigger time, target time, or cycle. Cycle numbers refer to the nth occurrence of the delay time. This option is valid only when `[<occurrences>]` is set to `multiple`.

`[<premise>]`

Specifies the delay value as `positive`, `negative`, or `either`.

Errors

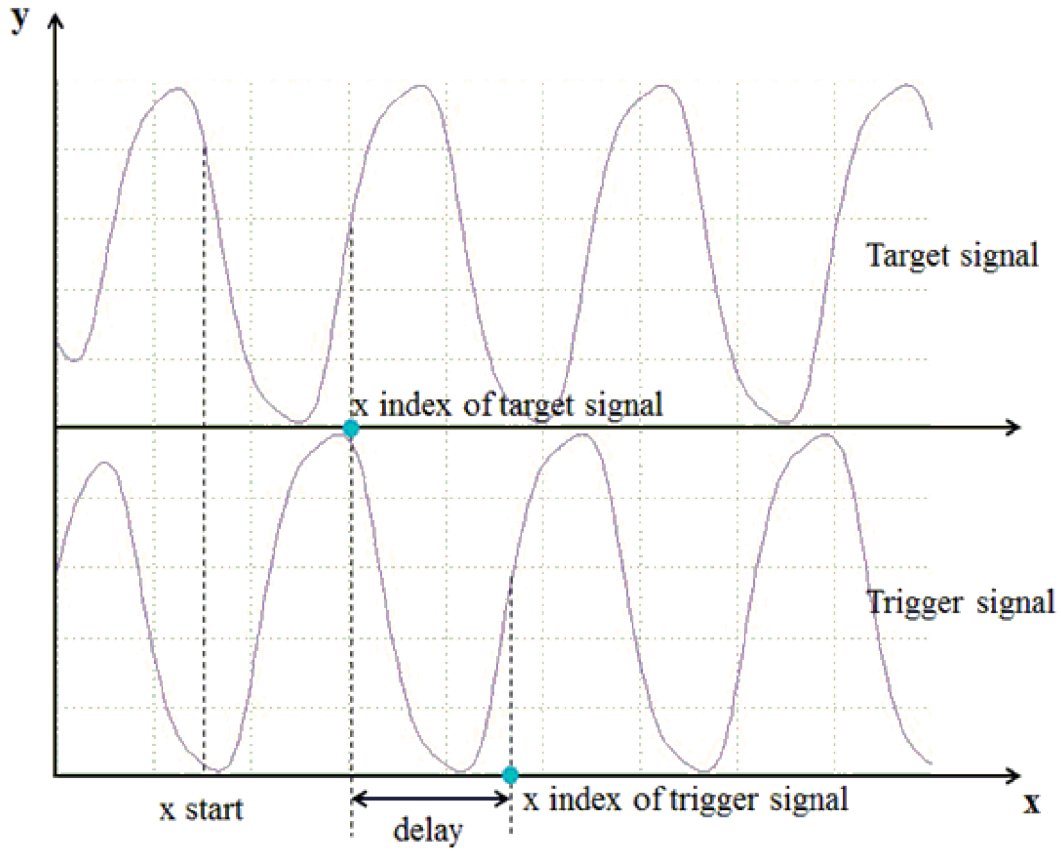
Possible error messages include:

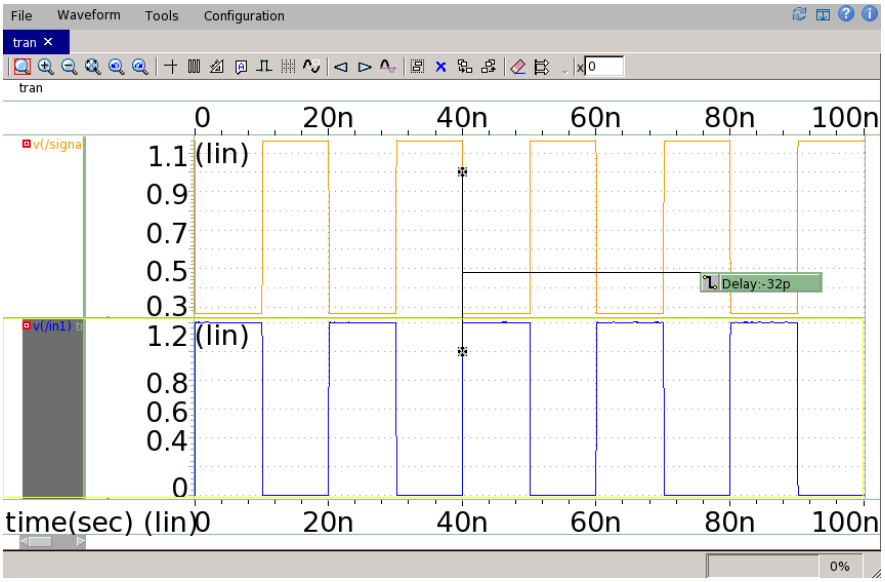
- Signal not found
- (Target/Trig)Y value out of range

Examples

```
delay('0|v(out)','0|v(out)',3,2,2,6,0)
delay('0|v(out)','0|v(out)',3,2,2,6,2n,0)
```

Feedback


y
Target signal
x index of target signal
Trigger signal
x start
delay
x index of trigger signal
x


File
Waveform
Tools
Configuration
tran
0
20n
40n
60n
80n
100n
1.1
(lin)
0.9
0.7
0.5
0.3
Delay:-32p
1.2
(lin)
0.8
0.6
0.4
0
time(sec) (lin)
0%

Feedback

The arguments are as follows:

- Trigger Signal - `0|v(in1)`
- Target Signal - `0|v(signal1)`
- Trigger Y level - Y level threshold of the trigger signal, equal to 1 in this case.
- Target Y level - Y level threshold of the target signal, equal to 1 in this case.
- Trigger X index - The edge index of the trigger signal. In this case, the delay is calculated on the second index of the trigger signal.
- Target X index - The edge index of the target signal. In this case, the delay is calculated between the second index of the target signal and the second index of the trigger signal.
- Trigger X start - The start x-value of the trigger signal.
- Target X start - The start x-value of the target signal.
- X stop - X-value stop times for both trigger and target signals.
- Trigger edge type - The edge type of the trigger signal. It is `rise` in this case.
- Target edge type - The edge type of the target signal. In this case, it is the opposite of the trigger signal edge type, and therefore it is false.

---

direct_ipn

Description

Calculates hbac/pac ipn.

Syntax

```
direct_ipn(file,[resistance],[input/output])
```

Arguments

`file`

The waveform file.

`resistance`

(Optional) The resistance.

`input/output`

(Optional) Input/output. (Default is input.)

Feedback

dutycycle

Calculates the duty cycle of a given period signal at the specified edges of edge indexes. This function uses the following algorithm, where `T` is the time the signal is active, and `P` is the total period of the signal:$D = \frac{T}{P} \cdot 100 percent$

Syntax

```
dutycycle(<signal>,[<y_value>],[<rise_edge>],[<nth_index>],
[<percent>],<vs_time>],[<output_xAxisType>],[<time_value>,[<average>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`[<y_value>]`

(Optional) Y threshold of the signal for duty cycle measurement.

`[<rise_edge>]`

(Optional) Sets the rising edge flag for the duty cycle. It has two values 0 and 1 (0=fall, 1=rise).

`[<nth_index>]`

(Optional) The index value, which can be `0`, `1`, `2`, `...`, or `n` for a period signal.

`[<percent>]`

(Optional) The percentage value, which can only be `0` or `1`. If specified as `1`, the y_level is considered a percentage value; if not, Y is `0`.

`[<vs_time>]`

(Optional) Specifies the output to be all of duty cycles.

`[<output_xAxisType>]`

(Optional) Specifies to retrieve value data against `time` or `cycle`. This feature depends on `vs_time`. Default is `time`.

`[<time_value>]`

(Optional) Specifies the time value, which is the cycle begin value or cycle end value. This feature depends on `vs_time`. Default is `cycle_end`.

Feedback

`[<average>]`

(Optional) Specifies the output value to be the average of duty cycle.

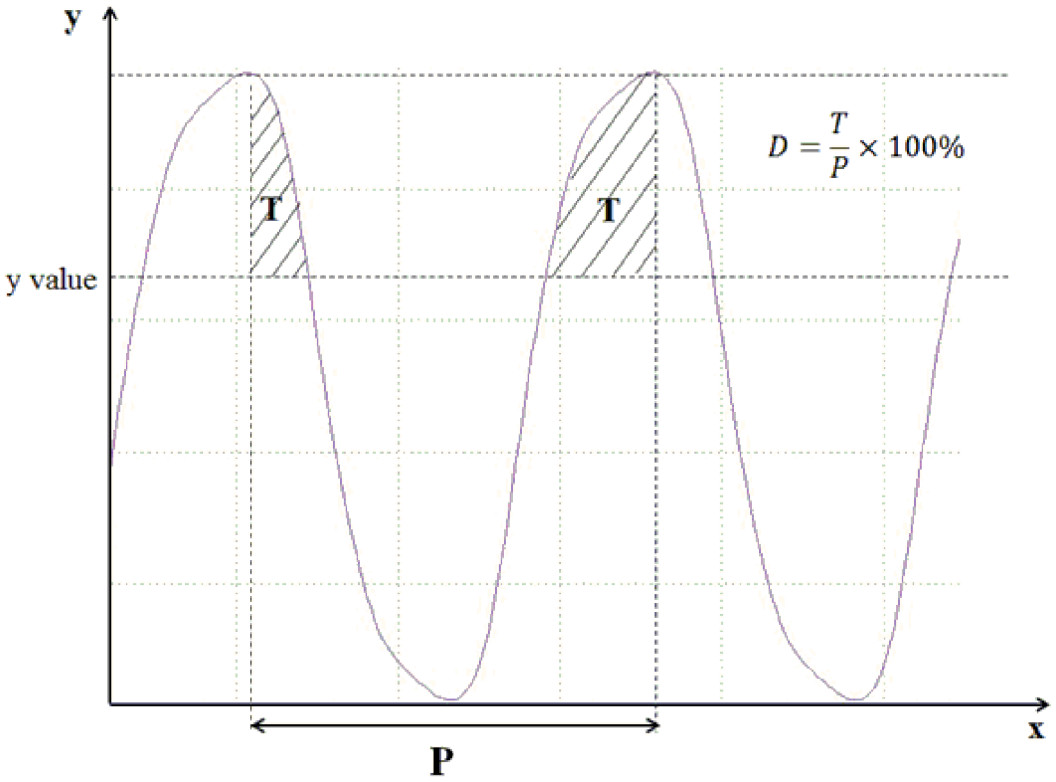
Errors

Possible error messages include:

- Signal not found
- Cannot find a compete pulse cycle
- Pulse period too small

Examples

```
dutycycle('0|v(out)',2,3,0)
dutycycle('0|v(out)',2,4n,3,0)
```



Feedback

```
dutycycle(v(/vin))
```

enb

Calculates the effective number of bits of a given signal. The FFT is calculated first, then the SNDR. The formula is ENB=(SNDR-1.76)/6.02.

Syntax

```
enb(<signal>,<x_start>,<x_stop>,<num_of_points>,
 <signal_freq>,[<band_limit>],[<windowing_type>], [<decimation_rate>]),
 [<bin_size>],[<window_alpha>],[<normalization>],[<user_defined_value>],[
<suppress_dc>],[<dual_sideband>],[<max_harmonic_index>],[include_floor_no
ise],[<full_scale>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated.

`<x_start>`

The starting point of X in the given signal.

`<x_stop>`

The stopping point of X in the given signal.

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The frequency of an input signal.

`[<band_limit>]`

(Optional) Sets the band limit frequency of the signal.

`[<windowing_type>]`

Specifies the windowing function. The default is `rectangle`.

`[<decimation_rate>]`

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. Normally, this argument is an integer or a rational fraction greater than one dividing the sampling rate.

Feedback

`[<bin_size>]`

(Optional) For window sampling, the bin_size parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

`[<window_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

`[<normalization>]`

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if [<normalization>] is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

`[<dual_sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<max_harmonic_index>]`

(Optional) The harmonic index of the maximum harmonic component.

`[<include_floor_noise>]`

(Optional) If set to true, floor noise components are counted in calculation.

`[<full_scale>]`

(Optional) If set to true, the magnitude of fundamental component is scaled to 1. And the formula is changed to ENB=(SNR-1.76)/6.02.

Feedback

Errors

Possible error messages include:

- Signal not found
- Invalid input value
- X value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2

Examples

```
enb('0|v(out)',0,8n,1024,100M,deci=20)
```

---

eye_mask

Adds a mask on to the existing eye object and returns 0 (fail) if the mask touches the eye; otherwise, this function returns 1 (pass).

Syntax

```
eye_mask(<eye_obj>,<width>,<height>,<x1>,
 [<absolute>],[<lower_bound>],[<upper_bound>],[<center>])
```

Returns

An analog scalar value.

Arguments

`<eye_object>`

The target eye object.

`<width>`

The width of the eye mask.

`<height>`

The height of the eye mask.

`<x1>`

The X value.

`[<absolute>]`

(Optional) Sets absolute size values.

Feedback

`[<lower_bound>]`

(Optional) Specifies the vertical lower boundary of the eye mask.

`[<upper_bound>]`

(Optional) Specifies the vertical upper boundary of the eye mask.

`[<center>]`

(Optional) Centers the mask to the eye opening.

Errors

Possible error messages include:

- Width is not specified
- Height is not specified
- x1 is not specified
- Signal must be eye object
- No valid eye object

Examples

```
eye_mask(eye,width=0.3,height=0.2,x1=0.1,absolute=0,center=1)
```

---

falltime

Calculates the fall time measured between theta1 (`percent_high`) to theta2 (`percent_low`) of the difference between the initial value and the final value.

Syntax

```
falltime(<signal>,<high_y_level>,<low_y_level>,
 [<x_start>],[<percent>],[edge_index>],[<min>],[<max>],[<min_value_type
>],[max_value_type>],[<occurrences>],[<output_xAxisType>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated.

`<high_y_level>`

Initial value at which the computation is to be started.

Feedback

`<low_y_level>`

Final value at which the computation is to be stopped.

`[<x_start>]`

(Optional) The starting point of the X time value.

`[<percent>]`

(Optional) The percentage value, which can only be `0` or `1`. If specified as `1`, the y_level is considered a percentage value; if not, Y is `0`.

`[<edge_index>]`

(Optional) The index of the edge. The default value is `1`.

`[<min>]`

(Optional) Min value for percentage y init/final. Default is xmin or ymin depending on type.

`[<max>]`

(Optional) Max value for percentage y init/final, default is xmax or ymax depending on type.

`[<min_value_type>]`

(Optional) Specifies whether the [<min>] value is an X-axis (x) or Y-axis value. (y). The default value is y.

`[<max_value_type>]`

(Optional) Specifies whether the [<max>] value is an X-axis (x) or Y-axis value (y). The default value is y.

`[<occurrences>]`

(Optional) Specifies whether you want to retrieve only one occurrence of a slewrate time event (single), or all occurrences of slewrate time events (multiple). The default value is single.

`[<output_xaxistype>]`

(Optional) Specifies whether you want to output slewrate data against time, or cycle. Cycle numbers refer to the nth occurrence of the slewrate event. This option is valid only when [<occurrences>] is set to “multiple”.

Feedback

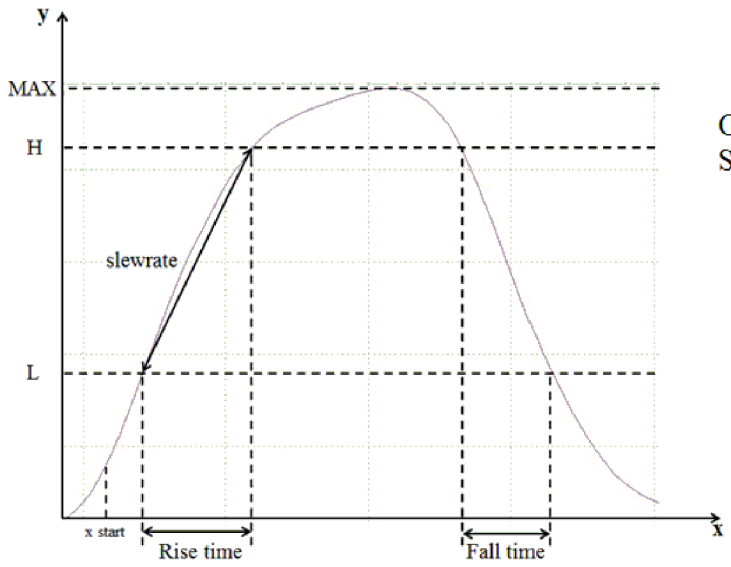
Errors

Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

Examples

```
falltime('0|v(out)',4,2)
falltime('0|v(out)',4,2,2n,1)
falltime('0|v(out)',4,2,2n,1,2)
```



Overshoot=(max-H)/(H-L),
Slewrate=max(dy/dx)

falltrig

Calculates the X value of the nth falling edge over threshold Y.

Syntax

```
falltrig(<signal>,<y_level>,<edge_index>,[<x_start>],[<percent>])
```

Returns

An analog scalar value.

Feedback

Arguments

`<signal>`

The signal to be calculated.

`<y_level>`

The y_threshold value.

`<edge_index>`

The number of the nth falling edge over the threshold.

`[<x_start>]`

(Optional) The starting point of the X time value.

`[<percent>]`

(Optional) The percentage threshold value, which can only be `0` or `1`. If specified as `1`, the y_level is considered a percentage value; if not, Y is `0`.

Errors

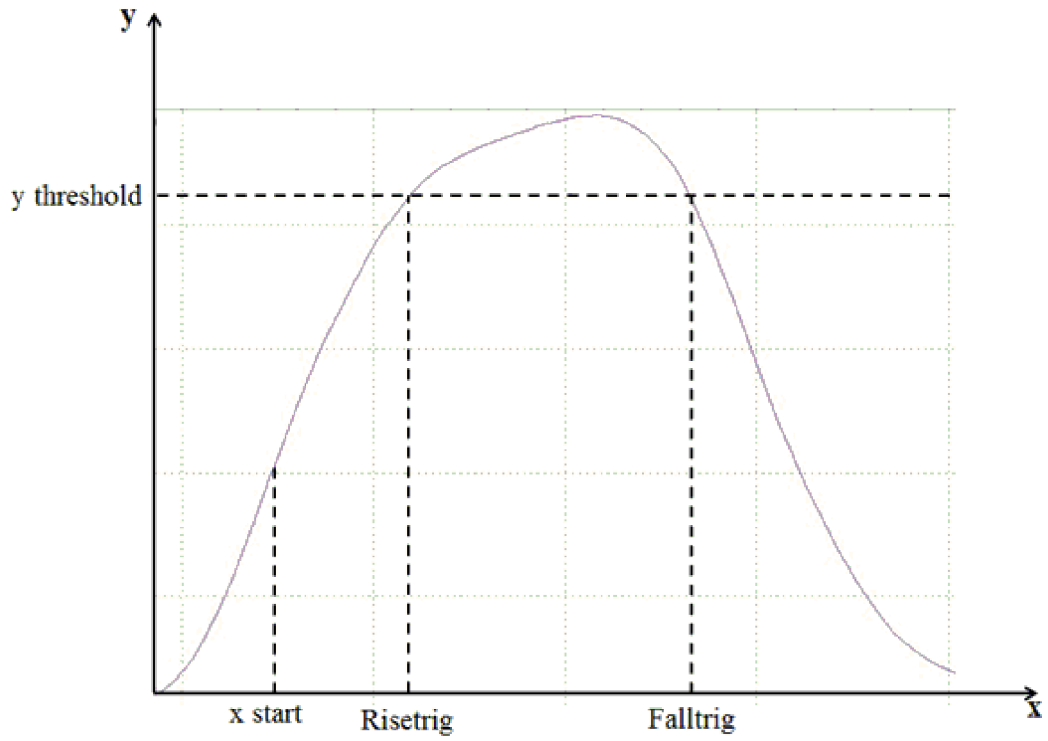
Possible error messages include:

- Signal not found
- Y value out of range

Examples

```
falltrig('0|v(out)',4,3)
falltrig('0|v(out)',4,2,4n,1)
```

Feedback



flatten

Description

Flattens the specified signal with a specified parameter as the new x-axis. The `flatten` function can be used if you have a set of sweep scalar values that you want to process further using calculator functions that take a signal as an argument. The `flatten` function is particularly useful when you have single point analyses (DC, AC) over a parametric sweep or corners and you want to convert the result into a suitable form for calculator function measurements.

Syntax

```
flatten (signal <string>, param name <string>)
```

Arguments

`signal`

The signal to flatten.

`param name`

(Optional) The parameter to become the x-axis.

Feedback

gain1_f

Calculates the frequency of a given signal at the signal magnitude, which is `0db(1)`.

Syntax

```
gain1_f(<signal>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

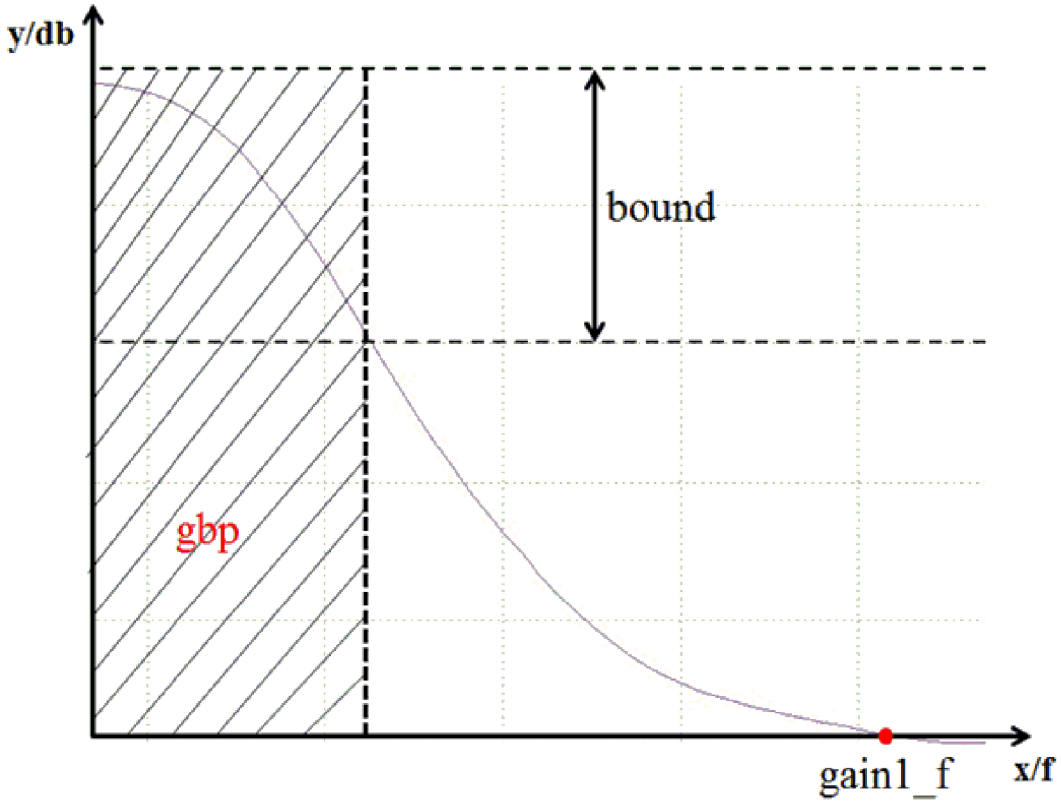
Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
gain1_f('0|v(out)')
```

Feedback



gain_m

Calculates the gain margin of a given signal, which is the amount of gain increase or decrease required to make the loop gain unity at the frequency. The phase angle is -180 degrees (modulate 360 degrees), and the gain margin is `1/g` if `g` is the gain at -180 degrees.

Syntax

```
gain1_m(<signal>,[<phase_degree>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

Feedback

`[<phase_degree>]`

(Optional) The phase degree, which can be `-180` or `0`. The default value is `-180`.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Waveform phase does not go through -180 degree

Examples

```
gain_m('0|v(out)')
gain_m('0|v(out)',0)
```

---

gbp

Calculates the gain margin of a given signal, which is the amount of gain increase or decrease required to make the loop gain unity at the frequency. The phase angle is -180 degrees (modulate 360 degrees), and the gain margin is `1/g` if `g` is the gain at -180 degrees.

Syntax

```
gbp(<signal>,[<db_bound>],[<gain>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`<db_bound>`

(Optional) The upper dB threshold.

`<gain>`

(Optional) User-defined gain value.

Feedback

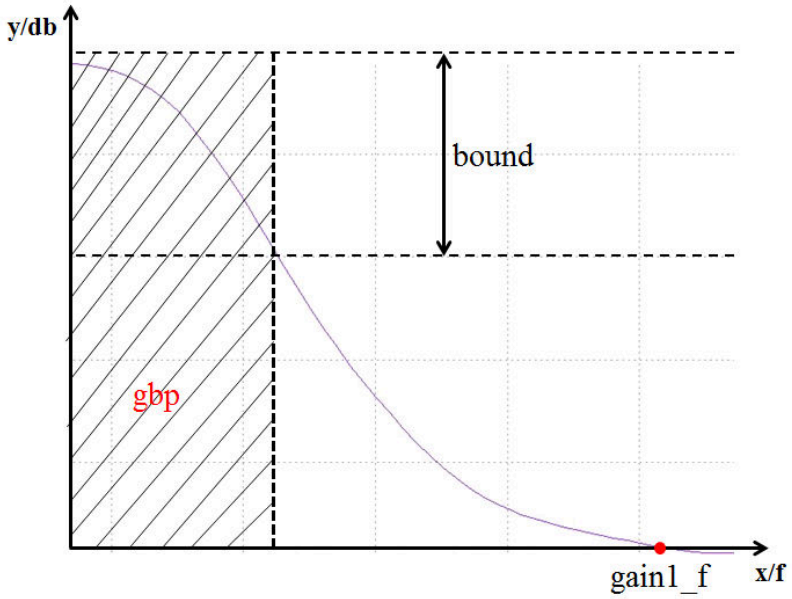
Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Cannot find the bandwidth

Examples

```
gbp('0|v(out)',4)
```



get_harmonic_freq

Calculates the frequency of the nth harmonic.

Syntax

```
get_harmonic_freq(<signal>,<0-based_index>)
```

Returns

An analog scalar value.

Feedback

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`<0-based_index>`

The index number.

Errors

Possible error messages include:

- Signal not found
- Invalid input index

Examples

```
get_harmonic_freq('0|v(out)',4)
```

---

globalmax

Returns the global maximum Y value of all data points for signal. (Also accepts scalar inputs.)

Syntax

```
globalmax(<signal>|<value>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
globalmax('0|v(out)')
```

Feedback

globalmean

Returns the global discrete mean value of all data points for a signal. (Also accepts scalar inputs.)

Syntax

```
globalmean(<signal>|<value>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

Errors

Possible error messages include:

- Signal not foundIncorrect signal type

Examples

```
globalmean('0|v(out)')
```

globalmin

Returns the global minimum Y value of all data points for signal. (Also accepts scalar inputs.)

Syntax

```
globalmin(<signal>|<value>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

Feedback

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
globalmin('0|v(out)')
```

---

globalstddev

Returns the global standard deviation value of all data points for signal. (Also accepts scalar inputs.)

Syntax

```
globalstddev(<signal>|<value> [, mc])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`mc`

(Optional) When `mc` is set to true, `globalstddev` flattens Monte Carlo only and calculates the stddev. (By default, `mc` is false.)

For example, given a signal `sig:temp=1~4,monte_carlo=1~4`

`globalstddev(sig) = 1` value (flatten temp and monte_carlo)

`globalstddev(sig, 0) = 1` value

`globalstddev(sig, 1) = 4` value (flatten monte_carlo only)

---

integ

Calculates the integral value of a given signal from x1 to x2.

Syntax

```
integ(<signal>,[<x_start>],[<x_stop>],[<extrapolate>])
```

Feedback

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`[<x_start>]`

(Optional) The starting point of the time X value.

`[<x_stop>]`

(Optional) The ending point of the time X value.

`[<extrapolate>]`

(Optional) When enabled, approximates a value if the x value is out of range. When disabled, the `x_start` and `x_stop` out of range values will be reset to xmin and xmax.

Errors

Possible error messages include:

- Signal not found
- X value out of range

Examples

```
integ('0|v(out)')
integ('0|v(out)',2n,4n)
```

---

ipn

Calculates the n-order intercept point between the target signal and reference signal.For a non-linear system with an output and two fundamental excitation `f1` and `f2` frequencies, the nth order Intercept Point is the point on a set of PowerOut (`PowerOut1` and `PowerOutn`) versus PowerIn curves at which a line extrapolated from PowerOut1 with a slope of `1` and a line with a slope of `n` extrapolated from the PowerOutn intersect. `PowerOut1` represents the first order term, and `PowerOutn` is the nth order term. Both lines must be extrapolated from a region with sufficiently low input power. `OIPn` is the value of `PowerOut` at which `IPn` occurs. `IIPn` is the value of the `PowerIn` value at which `IPn` occurs.

Feedback

Syntax

```
ipn(<signal>,<reference_signal>,<term_order>,
 <extrapolation_point>,[<input|output>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`<reference_signal>`

The reference signal.

`<term_order>`

An integer value, which can be `0`, `1`, `...`, or `n`. `n` is the slope of the linear function.

`<extrapolation_point>`

The extrapolated X value at which two signals start to intercept each other.

`[<input|output>]`

(Optional) The X and Y components on the waveform, where X is the `input` and Y is the `output`. The default is `output`.

Errors

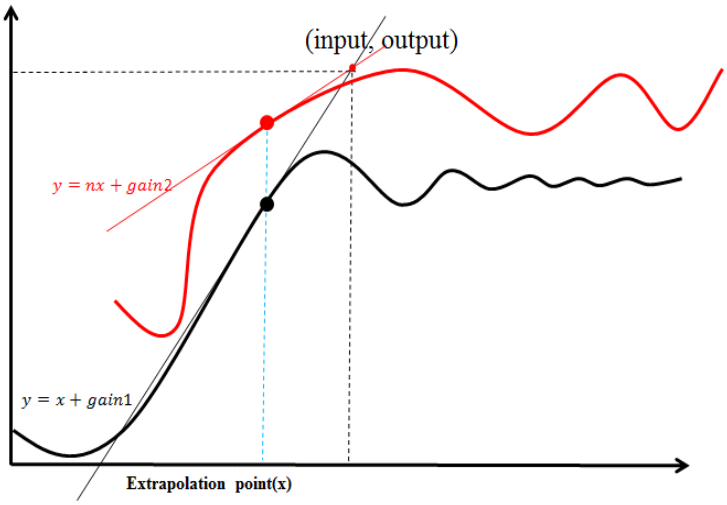
Possible error messages include:

- Signal not found
- X value out of range

Examples

```
ipn('0|v(out)','0|v(out)',3,4n)
ipn('0|v(out)','0|v(out)',3,4n,'input')
```

Feedback



ipnh

Description

Returns the nth-order intercept point.

Syntax

```
ipnh (signal <string>,
     firstOrderIndices <string>,
     nthOrderIndices <string>
     [, extrapolationPoint <string>]
     [, order <integer>]
     [, referred <enum>]
     [, sweepUnits <enum>]
     [, resultType <string>]
     [, resultsDir <string>]
     [, resultFile <string>]
     [, testbenchName <string>]
     [, analysisName <string>]
     [, params <string>])
```

Arguments

`signal`

The signal from which to extract the harmonics.

`firstOrderIndices`

The first order indices, space delimited.

Feedback

`nthOrderIndices`

The nth order indices, space delimited.

`extrapolationPoint`

(Optional) The input signal extrapolation point. If unspecified, defaults to lowest value.

`order`

(Optional) The intermodulation order. (Default: 3)

`referred`

(Optional) The Input/Output referred measurement.

Valid values: input, output (Default: input)

`sweepUnits`

(Optional) The units of the input power sweep.

Valid values: W, dBm, and none (Default: dBm)

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) Testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

`params`

(Optional) The parameter conditions.

---

ipnhVRI

Description

N dB intercept point VRI.

Feedback

Syntax

```
ipnhVRI (voltage signal <string>,
     harmonic indices 1 <string>,
     harmonic indices 2 <string>
     [, current signal]
     [, resistance]
     [, order 1 <integer>]
     [, order 2 <integer>]
     [, extrapolationPoint 1 <string>]
     [, extrapolationPoint 2 <string>]
     [, sweepUnits <enum>]
     [, referred <enum>])
```

Arguments

`voltage signal`

Input voltage signal.

`harmonic indices 1`

The first indices, space delimited.

`harmonic indices 2`

The second indices, space delimited.

`current signal`

(Optional) The input current signal.

`resistance`

(Optional) The resistance.

`order 1`

(Optional) `Harmonic indices 1` signal's order. (Default: 1)

`order 2`

(Optional) `Harmonic indices 2` signal's order. (Default: 3)

`extrapolationPoint 1`

(Optional) `Harmonic indices 1` signal extrapolation point. If unspecified, defaults to lowest value.

`extrapolationPoint 2`

(Optional) `Harmonic indices 2` l extrapolation point. If unspecified, defaults to lowest value.

`sweepUnits`

(Optional) The units of the input power sweep.

Feedback

Valid values: W, dBm, and none (Default: dBm)

`referred`

(Optional) The Input/Output referred measurement.

Valid values: input, output (Default: input)

---

mean

Calculates the mean value of a given signal or value from `x1` to `x2`.

Syntax

```
mean(<signal>|<value>,[<x_start>],[x_stop],[<type>])
```

Returns

An analog scalar value.

Arguments

`<signal|value>`

The signal or value to be calculated. Analog signals are supported.

`[<x_start>]`

(Optional) The starting point of the X value.

`[x_stop]`

(Optional) The ending point of the X value.

`[<type>]`

(Optional) The type of given signal, which can be `continuous` or `discrete`. The default is `continuous`.

Errors

Possible error messages include:

- Signal not found
- X value out of range

Examples

```
mean('0|v(out)')
mean('0|v(out)',2n,4n,'discrete')
```

Feedback

measure_aperture

Applies the aperture measurements to the eye object returned from the `create_eye` function.

Syntax

`measure_aperture(<eye_object>,<vref>,[<dc>],[<ac>],<return_type>)`

Returns

N/A

Arguments

`<eye_object>`

The output of the `create_eye` function.

`<vref>`

The central reference voltage level.

`[<dc>]`

(Optional) The DC voltage offset from the reference level.

`[<ac>]`

(Optional) The AC voltage offset from the reference level.

`[<return_type>]`

(Optional) The returned measurement type, which includes enumerated values in a list of all the measurements that the aperture returns.

The following values are included:

- Aperture_Time (the default)
- Min_Delay
- Max_Delay
- Skew
- Jitter_P2P
- Jitter_RMS
- Setup
- Hold

Feedback

- rho/rhi: The outer/inner most crossing time of the eye open at VREF+VAC on rising edge
- fto/fti: The inner/outer most crossing time of the eye open at VREF+VDC on falling edge
- fho/fhi: The outer/inner most crossing time of the eye open at VREF-VAC on falling edge
- rto/rti: The inner/outer most crossing time of the eye open at VREF-VDC on rising edge

Errors

Possible error messages include:

- Reference voltage is not specified
- Signal must be eye object
- No valid eye object

---

measure_eyejitter

Calculates Crossing Average, Crossing StdDev, Random Jitter (left), Random Jitter (right), and Deterministic Jitter of a signal using Eye Diagram.

Syntax

```
measure_eyejitter(<eye_object>,<vref>,<return_type>)
```

Returns

A scalar value

Arguments

`<eye_object>`

The output of the `create_eye` function.

`<vref>`

The threshold reference voltage level at which the crossing points are measured.

`[<return_type>]`

(Optional) The returned measurement type, which includes enumerated values in a list of all the measurements that the eye jitter measurement function returns.

Feedback

The following values are included:

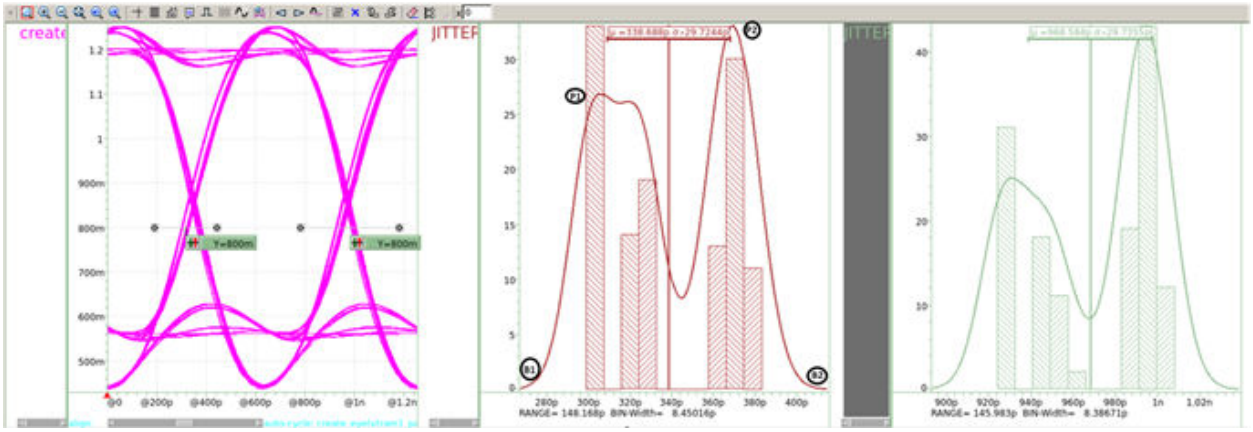
- CrossingAverage: Measures the average of the crossing times at the threshold level.
- CrossingStdDev: Measures StdDev of the crossing times at the threshold level.
- RandomJitter (left): Measures the Random Jitter at the left side of eye crossings.
- RandomJitter (right): Measures the Random Jitter at the right side of eye crossings.
- DeterministicJitter: Measures the time difference between the peaks of eye crossings on the left side and right side.

Deterministic and Random Jitter are calculated using the histograms of crossing times. If a deterministic jitter is present, each side of the distribution is expected to have two peaks. As shown in the following image, the crossing times on the left of the eye open are sampled into the histogram in red, and the crossing times on the right of the eye open are sampled into the histogram in green. The density estimation curves for both histograms are calculated. Deterministic Jitter is the distance between the two peeks of the curve. For the red histogram, it is P2-P1 and same for the green one. The Total Deterministic Jitter is calculated by the average of the Deterministic Jitter of red and green.

Random Jitter is the standard deviation of the distribution. It is calculated by:

$$\sqrt{\sum_i p_i (x_i - \mu)^2}$$

Where, μ is the mean, x is the x-value from B1 marker to P1 marker (in red histogram, tail of the distribution) and p is the corresponding height. The same calculation is repeated for the other tail from B2 to P2. Random Jitter (left) is the average stddev of the two tails. Random Jitter (right) is calculated in the same way on the green histogram.



Feedback

Errors

Possible error messages include:

- Reference voltage is not specified
- Signal must be eye object
- No valid eye object

---

measure_eyeopen

Measures the height and width of the eye opening.

Syntax

```
measure_eyeopen(<eye_object>,[<opening_type>],[<accuracy(%)>],[<vref>],
 [<return_type>])
```

Returns

The height or width of the eye object.

Arguments

`<eye_object>`

The output of the `create_eye` function.

`[<opening_type>]`

(Optional) The type of eye opening to be measured. Valid values are Diamond, Rectangle, Hexagon. Default: Diamond.

`[<accuracy(%)>]`

(Optional) The accuracy percentage of the diamond opening measurement. Default: 1.

`[<vref>]`

(Optional) The VREF value used for the rectangle and hexagon opening measurements.

`[<return_type>]`

(Optional) The returned measurement type. Valid values are Height and Width. Default: Width.

Feedback

Errors

Possible error messages include:

- Reference voltage is not specified
- Signal must be eye object
- No valid eye object
- Evaluation failed

---

measure_eyesize

Applies the eye size measurements to the eye object returned from the `create_eye` function.

Syntax

```
measure_eyesize(<eye_object>,<vref>,[<dc>],[<ac>],<return_type>)
```

Returns

N/A

Arguments

`<eye_object>`

The output of the `create_eye` function.

`<vref>`

The central reference voltage level.

`[<dc>]`

(Optional) The DC voltage offset from the reference level.

`[<ac>]`

(Optional) The AC voltage offset from the reference level.

`[<return_type>]`

(Optional) The returned measurement type, which includes enumerated values in a list of all the measurements that the Eye Size returns.

The following are values are included:

- WH (the default)
- WL

Feedback

- MH
- ML
- OH
- OL
- FUSS
- FUSS2
- DVDT
- DVDT_AVG
- DVDT_MIN
- DVDT_MAX
- Skew1
- Skew2
- CenterX
- IHEIGHT_AtCenterX
- OHEIGHT_AtCenterX

Errors

Possible error messages include:

- Reference voltage is not specified
- Signal must be eye object
- No valid eye object

Note that the `measure_eyesize` function requires an eye object. The picker for the eye object field in the Calculator tab opens the design editor, asking for a net, terminal or instance term.

You must provide a valid eye object, but do not use the picker. Manually enter the name of a `create_eye` expression in the eye object field in the Calculator. If your `create_eye` expression is unnamed, you must first enter a name for the expression in the Outputs pane.

Feedback

median_val

Calculates the median value of a given signal or value. The signals are sorted, where `n` is odd in the top equation and `n` is even in the bottom equation: $\begin{cases} Val\left(\frac{n+1}{2}\right) \\ Val\left(\frac{n}{2}\right) + Val\left(\frac{n}{2}+1\right) \end{cases}$

Syntax

```
median_val(<signal>|<value>)
```

Returns

An analog scalar value.

Arguments

`<signal|value>`

The signal or value to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

Errors

Possible error messages include:

- Signal not found
- Input signal has no data point
- Input signal has too many data point
- Out of memory

Examples

```
median_val('0|v(out)')
```

overshoot

Calculates the overshoot value of a given signal, which uses the following algorithm:
`(Max(signal)-high_y_level)/(high_y_level-low_y_level)`

Syntax

```
overshoot(<signal>,<low_y_level>,<high_y_level>,
[<percent>],[<low_value_type>],[<high_value_type>],
[<occurrences>],[<output_xAxisType>],[<output_type>])
```

Returns

An analog scalar value.

Feedback

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`<low_y_level>`

The low threshold of the Y signal value.

`<high_y_level>`

The high threshold of the Y signal value.

`[<percent>]`

(Optional) The percentage value, which can only be `0` or `1`. If specified as `1`, the y_level is considered a percentage value; if not, Y is `0`.

`[<low_value_type>]`

(Optional) Determines how to retrieve the low y value.

`[<high_value_type>]`

(Optional) Determines how to retrieve the high y value.

`[<occurrences>]`

(Optional) Specifies result to be only one occurrence of an overshoot event for the given waveform (`single`) or all occurrences of overshoot for the given waveform which you can later plot or print (`multiple`).

`[<output_xAxisType>]`

(Optional) Specifies the X axis type of the result waveform. The value can be time or idx.

`[<output_type>]`

(Optional) Default is `absolute`. If `percentage` is selected, the threshold values are in percentages, otherwise, absolute values.

Errors

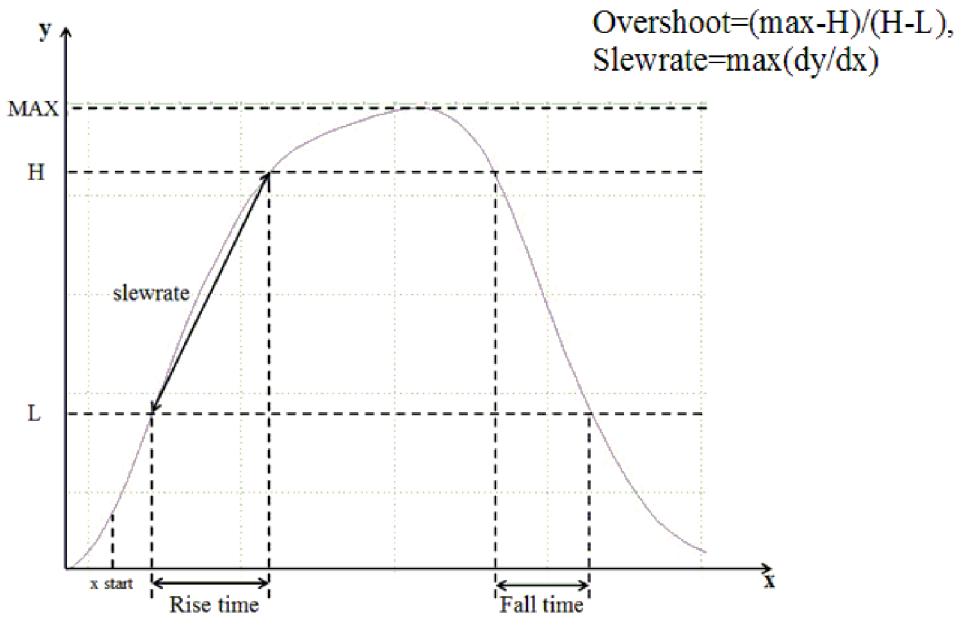
Possible error messages include:

- Signal not found
- Y value out of range

Examples

```
overshoot('0|v(out)',2,3)
overshoot('0|v(out)',2,4,1)
```

Feedback



p2p

Measures peak-to-peak, the difference between the maximum and minimum Y values of the given signal. When the argument `measEdge` is 1 (true), `p2p` measures peak-to-peak value of edges.

Syntax

```
p2p(<signal>,[x1=<x_start>],[x2=<x_stop>],[measEdge=0|1],
[n=<nth_index>],[minpp=<min_p2p>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`[x1=<x_start>]`

(Optional) The starting point of the X range.

`[x2=<x_stop>]`

(Optional) The stop point of the X range.

Feedback

`[<measEdge>=0|1]`

(Optional) Specifies whether to measure peak-to-peak value of edges. Default is 0 (false).

When `measEdge=0`, arguments `n` and `minpp` will be ignored even if they are specified.

`[n=<nth_index>]`

(Optional) The integer number. Specifies the index of edge to be measured. It takes effect only when `measEdge=1`. If unspecified and `measEdge=1`, the function finds the edge with biggest peak-to-peak and returns the value.

`[minpp=<min_p2p>]`

(Optional) The minimum peak-to-peak value that an edge must have. It takes effects only when `measEdge=1` and `nth_index` is specified. Its default value is 5% of the peak-to-peak value of the biggest edge.

If all edges' peak-to-peak value are smaller than min_p2p, the function will return `failed`.

Errors

Possible error messages include:

- Signal not found
- X value out of range

Examples

The following measures the peak-to-peak value of the biggest edge.

```
p2p('v(out)',measEdge=1)
```

The following measures the peak-to-peak value for the 4th edge after 10n.

```
p2p('v(out)',x1=10n,measEdge=1,n=4)
```

The following measures the peak-to-peak value for the 4th edge. The edges are counted only when their peak-to-peak value is greater than 1.5.

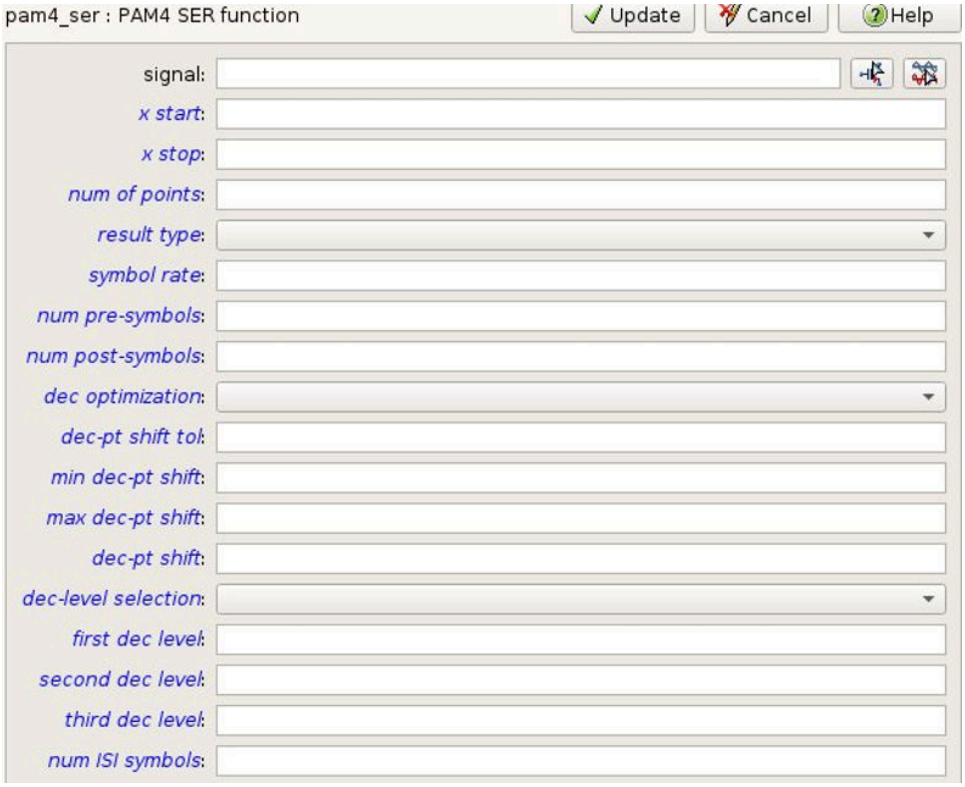
```
p2p('v(out)',measEdge=1,n=4,minpp=1.5)
```

---

pam4_ser

Measures the symbol-error rate (SER) for PAM4 electrical signals

Expression Field



Syntax

```
pam4_ser(<signal>,[<x_start>],[<x_stop>],
         [<num_of_points>],[<result_type>],
         [<symbol_rate>],[<num_pre-symbols>],
         [<num_post-symbols>],[<dec_optimization>],
         [<dec-pt_shift_tol>],[<min_dec-pt_shift>],
         [<max_dec-pt_shift>],[<dec-pt_shift>],
         [<dec-level_selection>],[<first_dec_level>],
         [<second_dec_level>],[<third_dec_level>],
         [<num_ISI_symbols>])
```

Note:

There is a difference between signal name specification in PrimeWave vs. standalone WaveView usage. The examples in the proceeding sections demonstrate the use of both the syntax without detailed explanation. For more information and examples, see Differences Between PrimeWave Expression Syntax and Standalone WaveView Expression Syntax.

Feedback

Description

This function measures the symbol-error rate (SER) for PAM4 electrical signals, taking an analog electrical PAM4 signal as input and estimating the rate of symbol errors from the signal's eye diagram.

You can define the number of symbols at the start and end of the input signal to be excluded from the SER algorithm. This approach is helpful when you want to avoid including numerical transient effects or discontinuities in the calculation.

The SER is calculated by constructing histograms for each of the four PAM4 symbol levels in the eye diagram, from which estimates are made for the mean and standard-deviation values for each symbol at a particular decision time relative to the center of the eye. An SER is then estimated in each case, assuming Gaussian noise distributions. A final SER is then calculated based on the SER for each symbol.

You can specify whether the decision time used for the SER calculation should be optimized by sweeping over a specified range of values, or whether a fixed value should be used. If a sweep is selected, you can adjust the tolerance for the search, as well as adjust the minimum and maximum shift values.

As for the three decision thresholds used to recognize the four PAM4 symbols, the user can specify that they be automatically set based on the optical modulation amplitude of the signal (OMS_{outer}) and the average signal level P_{ave} [1]:

$P_{th1} = P_{ave} - OMA_{outer}/3$

$P_{th2} = P_{ave}$

$P_{th3} = P_{ave} + OMA_{outer}/3$

Alternatively, you can choose to specify these values directly.

You can also account for the effects of inter-symbol interference by calculating SER values for different partial eyes representing different symbol patterns. You should specify a value for *num_ISI_Symbols* > 0, in which case SER values are calculated for the center symbol of each possible symbol pattern of length 2.*num_ISI_Symbols*+1. The final SER is calculated as a weighted average of these values.

Finally, the `pam4_ser()` function provides the facility to measure and report values for not just the SER, but also the final decision-time shift and OMA_{outer}.

Arguments

`<signal>`

- The analog PAM4 signal to be calculated.
- Valid signal types: analog single-trace waveform or multiple-trace waveform, expression.

Feedback

- Invalid signal types: digital signal, scalar value.
- When the y values of the trace are complex numbers, the magnitude is automatically applied.

`[<x_start>]`

(Optional) The starting point of x. The default is the beginning of the signal.

`[<x_stop>]`

(Optional) The ending point of x. The default is the end of the signal.

`[<num_of_points>]`

- (Optional) The number of points needing to be measured.
- Algorithm adjusts the signal sampling and time interval such that the number of data points is the closest power of 2 greater than or equal to <num_of_points>, and the number of symbols is the closest power of 2 less than or equal to the total number of symbols in the time interval.
- The default is the number of points in the requested time interval.

`[<result_type>]`

(Optional) Specifies the measurement value to produce. Default is `ser`.

- `ser` – the SER value
- `decisionPointShift` – the decision-time shift relative to the eye center used for the final SER value
- `omaOuter` – outer modulation amplitude (OMA)

`[<symbol_rate>]`

- (Optional) Symbol rate of signal.
- Must be positive. The default is 26.5625e9.

`[<num_pre-symbols>]`

- (Optional) Number of initial symbols to ignore.
- Must be greater than or equal to 0. The default is 0.

`[<num_post-symbols>]`

- (Optional) Number of final symbols to ignore.
- Must be greater than or equal to 0. The default is 0.

`[<dec_optimization>]`

(Optional) Specifies how to optimize the decision time. Default is `sweep`.

Feedback

- `sweep` – sweeps the decision-point shift relative to the eye center time to find the best SER
- `none` – uses the user-specified decision-point shift

`[<dec-pt_shift_tolerance>]`

- (Optional) Tolerance for decision-point time-shift optimization.
- Must be greater than 0. Default is 1e-3.

`[<min_dec-pt_shift>]`

- (Optional) Minimum decision point shift, specified as a fraction of a symbol period.
- Must be in the interval (-0.5,0.5). The default -0.05.

`[<max_dec-pt_shift>]`

(Optional) Maximum decision-point shift, specified as a fraction of a symbol period. Must be in the interval (-0.5,0.5). The default 0.05.

`[<dec-pt_shift>]`

- (Optional) User-specified decision-point shift, specified as a fraction of a symbol period.
- Must be in the interval (-0.5,0.5). The default is 0.

`[<dec-level_selection>]`

(Optional) Specifies how to determine decision thresholds. Default is `oma`.

- `oma` – based on OMA_{outer} calculated from an eye diagram
- `defined` – uses user-specified decision thresholds

`[<first_dec_level>]`

(Optional) User-specified first decision threshold. Default is 0.

`[<second_dec_level>]`

(Optional) User-specified second decision threshold. Default is 0.

`[<third_dec_level>]`

(Optional) User-specified third decision threshold. Default is 0.

`[<num_ISI_Symbols]>]`

(Optional) Specifies additional symbols to account for ISI.

Feedback

Returns

- When the given signal is a single-trace waveform, this function returns a scalar value of a specified result type.
- When the given signal is a multiple-trace waveform, this function returns a waveform of a specified result type.

Errors

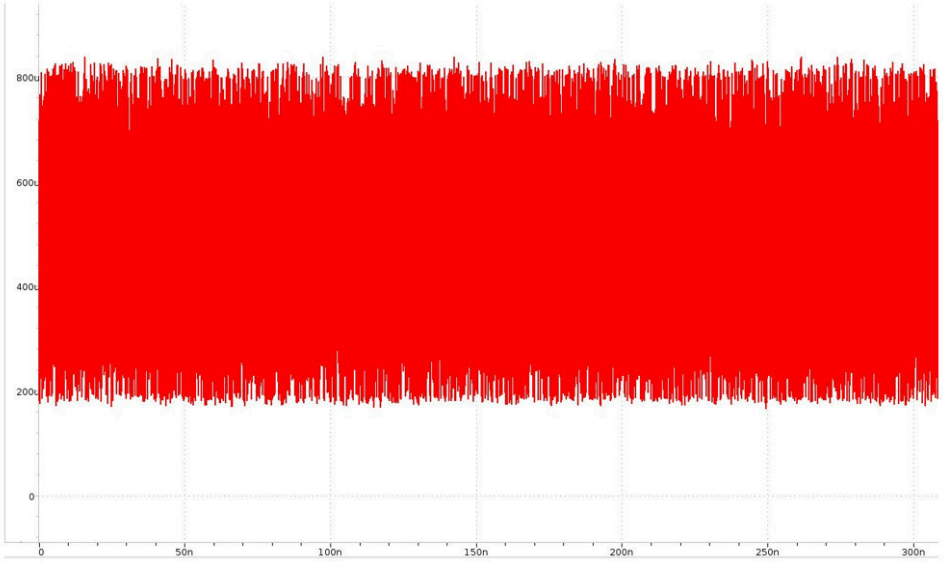
The possible error messages include:

- Signal not found
- Number of sample points is non-positive
- Number of symbols is non-positive
- Symbol rate is non-positive
- Number of pre-symbols is negative
- Number of post-symbols is positive
- Invalid decision-shift tolerance
- Invalid decision-shift minimum
- Invalid decision-shift maximum
- Logic signals not supported
- Complex signals not supported
- End time is outside waveform range
- Signal must be analog
- SER calculation failure

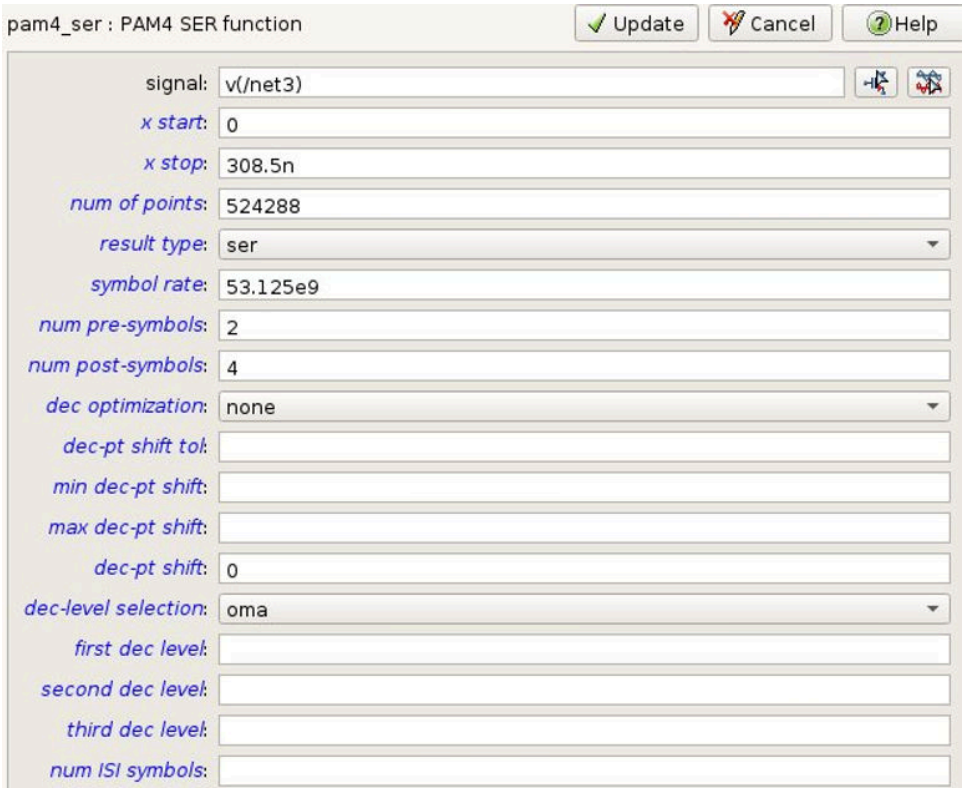
Feedback

Examples

Input Signal Waveform:



Expression Field:



Feedback

PrimeWave Expression:

```
pam4_ser(v(/net3),0,308.5n,524288,ser,53.125e9,2,4,none,
         decisionPointShift=0,decisionLevelSelection=oma)
```

Standalone WaveView Expression:

```
pam4_ser('v(net3)',0,308.5n,524288,ser,53.125e9,2,4,none,
         decisionPointShift=0,decisionLevelSelection=oma)
```

Return:

45.5783m

---

pamn_level_at_eye_center

Measures eye level at eye center.

Syntax

```
pamn_level_at_eye_center(<eye object>,[number of levels],[levels],[time
 reference],[eye center],[time location type],[time window %],[eye open],
 [return type])
```

Returns

An analog scalar value.

Plot Assist Expression

PAMN Eye Level Measurement

Arguments

`<eye object>`

(Required) The measurement PAMn eye object that is returned from the create_eye function.

`<number of levels>`

(Optional) The number of PAMn levels. Currently, 3, 4, 6 levels are supported. The default is 4.

`<levels>`

(Optional) The amplitude levels. The default is empty, which means eye openings are even spaced. PW calculates default levels, otherwise, use input levels as a string of numbers separated by “,”.

Feedback

`<time reference>`

(Optional) The default is the center eye. It is based on the center eye and each eye is independent.

`<eye center>`

(Optional) The location of the eye center for time reference. The default is maximum eye width. For **Maximum Eye Width**, the center of the maximum eye width of the Middle/Center eye is used as the time reference. For **Maximum Eye Height**, the maximum of the eye height of the Middle/Center eye is used as the time reference.

`<time location type>`

(Optional) The default is eye center. The available values are **Eye center** and **Minimum rms**.

`<time window %>`

(Optional) The time window from the eye center as a percent of UI. The default is 10.

`<eye open>`

(Optional) The eye level that is measured. Level 1 is the bottom eye opening. The default is level1.

The available values are level0, level1, level2, level3, level4, level5.

`<return type>`

(Optional) The returned measurement type. The default is mean.

The available values are mean, point to point, rms, skew, rlm, rlm, oif.

---

pamn_level_at_user_ref

Measures eye level at user reference.

Syntax

```
pamn_level_at_user_ref(<eye object>, [number of levels], [levels],
 [vref], [x value], [time location type], [time window %], [eye open],
 [return type]
```

Returns

An analog scalar value.

Plot Assist Expression

PAMN Eye Level Measurement

Feedback

Arguments

`<eye object>`

(Required) The measurement PAMn eye object that is returned from the create_eye function.

`<number of levels>`

(Optional) The number of PAMn levels. Currently, 3, 4, 6 levels are supported. The default is 4.

`<levels>`

(Optional) The amplitude levels. The default is empty, which means eye openings are even spaced. PW calculates default levels, otherwise, use input levels as a string of numbers separated by “,”.

`<vref>`

(Optional) The user specified VREF. Use middle point of VREF as the time reference if the X value is not given.

`<x value>`

(Optional) The user specified X value as the time reference.

`<time location type>`

(Optional) The default is eye center. The available values are **Eye center** and **Minimum rms**.

`<time window %>`

(Optional) The time window from the eye center as a percent of UI. The default is 10.

`<eye open>`

(Optional) The eye level that is measured. Level 1 is the bottom eye opening. The default is level1.

The available values are level0, level1, level2, level3, level4, level5.

`<return type>`

(Optional) The returned measurement type. The default is mean.

The available values are mean, point to point, rms, skew, rlm, rlm, oif.

---

pamn_open_at_eye_center

Measures eye open at eye center.

Feedback

Syntax

```
pamn_open_at_eye_center(<eye object>, [number of levels], [levels], [time
 reference], [eye center], [offset from midpoint], [eye open], [return
 type])
```

Returns

An analog scalar value.

Plot Assist Expression

PAMN Eye Open Measurement

Arguments

`<eye object>`

(Required) The measurement PAMn eye object that is returned from the create_eye function.

`<number of levels>`

(Optional) The number of PAMn levels. Currently, 3, 4, 6 levels are supported. The default is 4.

`<levels>`

(Optional) The amplitude levels. The default is empty, which means eye openings are even spaced. PW calculates default levels, otherwise, use input levels as a string of numbers separated by “,”.

`<time reference>`

(Optional) The default is the center eye. It is based on the center eye and each eye is independent.

`<eye center>`

(Optional) The location of the eye center for time reference. The default is maximum eye width. For **Maximum Eye Width**, the center of the maximum eye width of the Middle/Center eye is used as the time reference. For **Maximum Eye Height**, the maximum of the eye height of the Middle/Center eye is used as the time reference.

`<offset from midpoint>`

(Optional) The offset value from the midpoint for calculating DVDT min/max/avg. The default is 0.1.

`<eye open>`

(Optional) The eye opening that is measured. The default is eye1.

The available values are eye0, eye1, eye2, eye3, eye4.

Feedback

`<return type>`

(Optional) The returned measurement type. The default is midpoint.

The available values are midpoint, skew, height, width, dvdt_min_rise, dvdt_max_rise, dvdt_avg_rise, dvdt_min_fall, dvdt_max_fall, dvdt_avg_fall, dvdt_min_both, dvdt_max_both, dvdt_avg_both, area, mask.

---

pamn_open_at_user_ref

Measures eye open at user reference.

Syntax

```
pamn_open_at_user_ref(eye object, [number of levels], [levels], [vref],
 [x value], [offset from mid-point], [eye open], [return type])
```

Returns

An analog scalar value.

Plot Assist Expression

PAMN Eye Open Measurement

Arguments

`<eye object>`

(Required) The measurement PAMn eye object that is returned from the create_eye function.

`<number of levels>`

(Optional) The number of PAMn levels. Currently, 3, 4, 6 levels are supported. The default is 4.

`<levels>`

(Optional) The amplitude levels. The default is empty, which means eye openings are even spaced. PW calculates default levels, otherwise, use input levels as a string of numbers separated by “,”.

`<vref>`

(Optional) The user specified VREF. Use middle point of VREF as the time reference if the X value is not given.

`<x value>`

(Optional) The user specified X value as the time reference.

Feedback

`<offset from mid-point>`

(Optional) The offset value from the midpoint for calculating DVDT min/max/avg. The default is 0.1.

`<eye open>`

(Optional) The eye opening that is measured. The default is eye1.

The available values are eye0, eye1, eye2, eye3, eye4.

`<return type>`

(Optional) The returned measurement type. The default is midpoint.

The available values are midpoint, skew, height, width, dvdt_min_rise, dvdt_max_rise, dvdt_avg_rise, dvdt_min_fall, dvdt_max_fall, dvdt_avg_fall, dvdt_min_both, dvdt_max_both, dvdt_avg_both, area, mask.

---

paramvalue

Calculates the value of a sweeping parameter, which is mainly used with multi-trace waveforms to extract the values of the specified sweeping parameter.

Syntax

```
paramvalue(<sweep_variable>)
```

Returns

Several analog scalar values.

Arguments

`<sweep_variable>`

The multi-trace sweeping signal to be calculated.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
paramvalue('1|pin_dbm')
```

Feedback

phase_m

Calculates the phase margin, which is the difference between the phase of the response and `-180` degrees when the loop gain is `1.0`.

Syntax

```
phase_m(<signal>,[<base_degree>])
```

Returns

Several analog scalar values.

Arguments

`<signal>`

The signal to be measured.

`[<base_degree>]`

(Optional) The base degree.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Waveform phase does not go through -180 degree

Examples

```
phase_m('0|v(out)')
phase_m('0|v(out)',0)
```

risetime

Calculates the rise time for a signal, which is the time taken by a signal to change from a specified low value to a specified high value. Typically, these values are 10% and 90% of the step height.

Syntax

```
risetime(<signal>,<low_y_level>,<high_y_level>,
[<x_start>],[<percent>],[edge_index>],[<min>],[<max>],
[<min_value_type>],[max_value_type>],[<occurrences>],
[<output_xaxistype>])
```

Feedback

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated.

`<low_y_level>`

Initial value at which the computation is to be started.

`<high_y_level>`

Final value at which the computation is to be stopped.

`[<x_start>]`

(Optional) The starting point of the X time value.

`[<percent>]`

(Optional) The percentage threshold value, which can only be `0` or `1`. If specified as `1`, the y_level is considered a percentage value; if not, Y is `0`.

`[<edge_index>]`

(Optional) The index of the edge. The default value is `1`.

`[<min>]`

(Optional) Min value for percentage y init/final. Default is xmin or ymin depending on type.

`[<max>]`

(Optional) max value for percentage y init/final, default is xmax or ymax depending on type.

`[<min_value_type>]`

(Optional) Specifies whether the [<min>] value is an X-axis (x) or Y-axis value. (y). The default value is y.

`[<max_value_type>]`

(Optional) Specifies whether the [<max>] value is an X-axis (x) or Y-axis value (y). The default value is y.

`[<occurrences>]`

(Optional) Specifies whether you want to retrieve only one occurrence of a slewrate time event (single), or all occurrences of slewrate time events (multiple). The default value is single.

Feedback

`[<output_xaxistype>]`

(Optional) Specifies whether you want to output slewrate data against time, or cycle. Cycle numbers refer to the nth occurrence of the slewrate event. This option is valid only when [<occurrences>] is set to “multiple”.

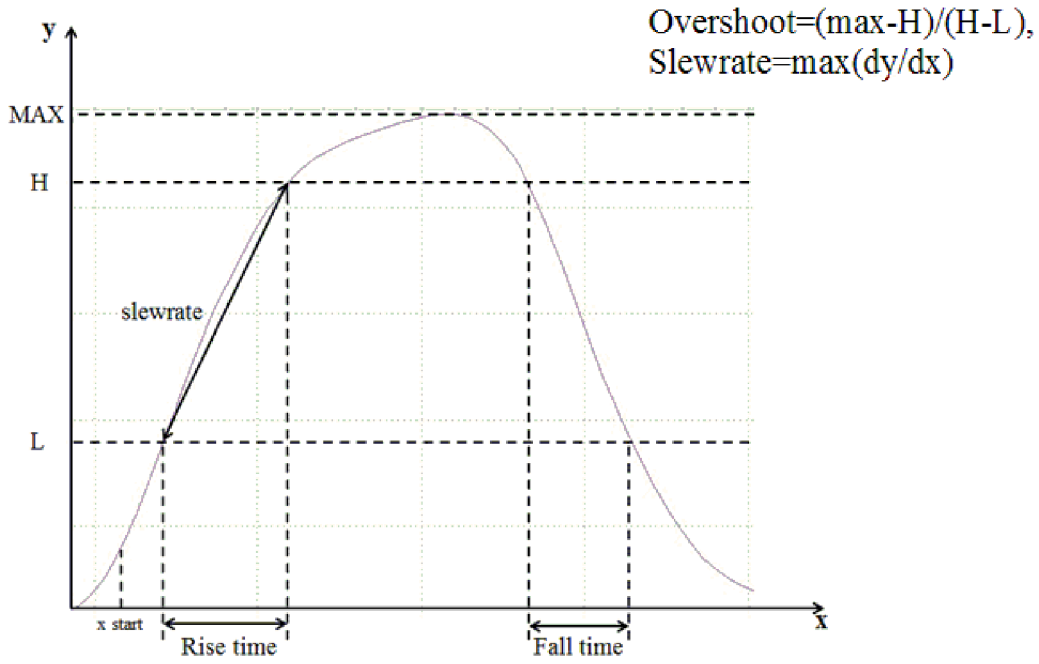
Errors

Possible error messages include:

- Signal not foundY value out of rangeX value out of range

Examples

```
risetime('0|v(out)',2,4)
risetime('0|v(out)',2,4,2n,1)
risetime('0|v(out)',2,4,2n,1,2)
```



risetrig

Measures the X value of the nth rising edge over the y_threshold starting from the x_start value.

Syntax

`risetrig(<signal>,<y_level>,<edge_index>,[<x_start>],[<percent>])`

Feedback

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<y_level>`

The threshold of the Y value.

`<edge_index>`

The nth rising edge.

`[<x_start>]`

(Optional) The starting point of the X value.

`[<percent>]`

(Optional) The percentage value, which can only be `0` or `1`. If specified as `1`, the y_level is considered a percentage value; if not, Y is `0`.

Errors

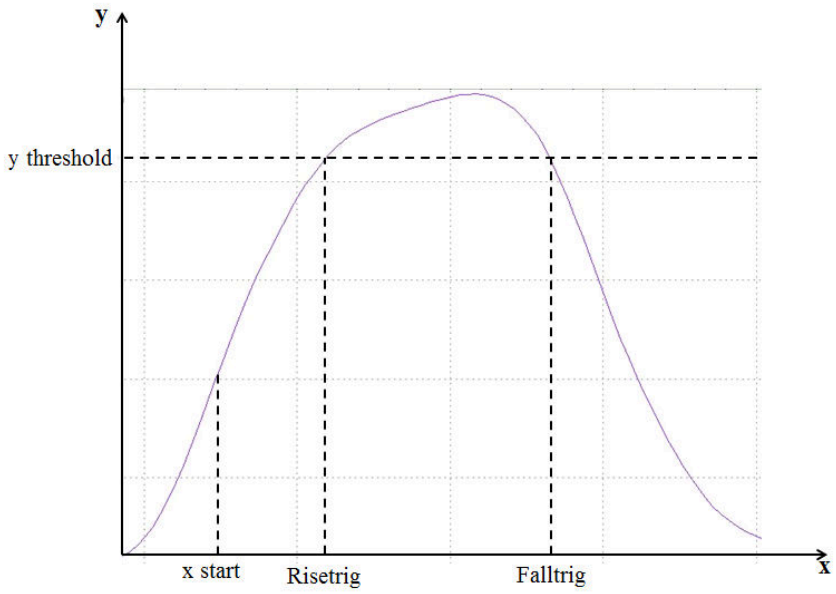
Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

Examples

```
risetrig('0|v(out)',2,3)
risetrig('0|v(out)',2,3,6n,1)
```

Feedback



rmsval

Measures the RMS value of given signal, where RMS is Root Mean Square. The following equation is for continuous waveform: $RMS = \sqrt{\frac{1}{x_{stop}\text{-}x_{start}} \int_{x_{start}}^{x_{stop}} [f(x)]^2 dx}$

Syntax

```
rmsval(<signal>,[<x_start>],[<x_stop>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`[<x_start>]`

(Optional) The starting value of X.

`[<x_stop>]`

(Optional) The ending value of X.

Feedback

Errors

Possible error messages include:

- Signal not found
- X value out of range

Examples

```
rmsval('0|v(out)')
rmsval('0|v(out)',2n,6n)
```

---

root

Description

Computes the value of x at which f(x) equals the specified threshold.

Syntax

```
root(<signal>,<threshold>,<nth_root>)
```

Returns

An analog scalar value or an analog signal.

Additionally:

- In case of `nth_root`=1, return value is the 1st value of x at which f(x) equals the specified threshold.
- In case of `nth_root`=2, return value is the second value of x at which f(x) equals the specified threshold.
- In case of `nth_root`<0, return value is the maximum value of x at which f(x) equals the specified threshold.
- In case of `nth_root`=0, return value is a waveform of x at which f(x) equals the specified threshold.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported.

`<threshold>`

Waveform value at which the root value is to be computed.

Feedback

`<nth_root>`

Root value you want to calculate.

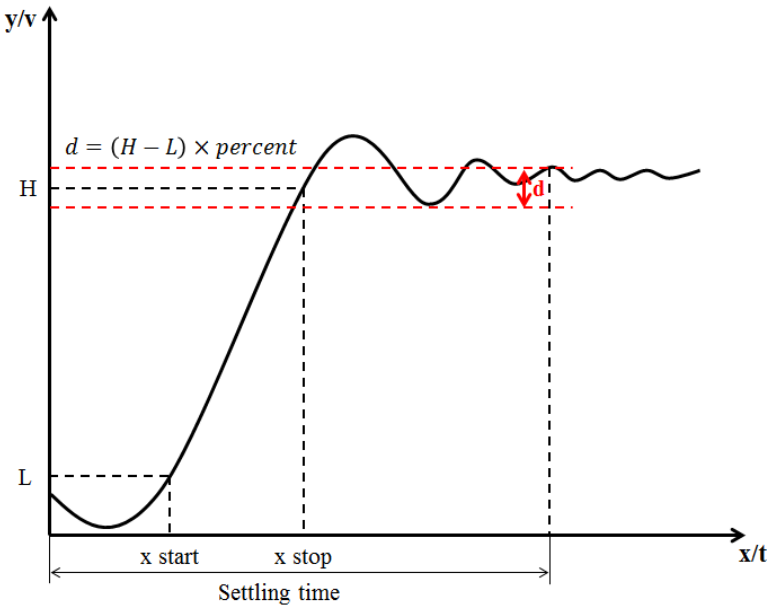
---

settling_t

Description

Measures the settling time of given signal. Settling time is the time required for the response curve to reach and stay within a range of certain percentage (usually 5% or 2% of the final value).

Figure 3 settling_t Example



To calculate the d value:

```
d=(yvalue at xstop - yvalue at xstart)*settle_band%
```

If `half_band_error=false`, the signal is considered settled when the y values of the signals are within `(yvalue at xstop+(d/2)` and `(yvalue at xstop-(d/2)`, and the tool reports the `xvalue(time)` at which this signal entered into this range of settling time.

If `half_band_error=true`, the signal is considered settled when the y values of the signals are within `(yvalue at xstop+(d)` and `(yvalue at xstop-(d)`, and tool reports the `xvalue(time)` at which this signal entered into this range of settling time.

Feedback

Syntax

```
settling_t(<signal>,<start>,<stop>,<settle_band>,[<x_end>],[start_by
=x/y],[stop_by=x/y],[<half_error_band>],
 [<occurrences>],[<output_xaxistype>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<start>`

The starting value.

`<stop>`

The stopping value.

`<settle_band>`

A percentage value. For example, to set the settle band to 3%, set this value to `3`. This is equivalent to the *d* value in Figure 3.

`[<x_end>]`

(Optional) The ending value of X. If `<x_end>` is specified, the function calculates the settle time in a sub-window waveform.

`[<start_by>]`

(Optional) The starting value, which can be X or Y. If you specify `start_by='x'`, the starting value is for the x-axis (the *x-start* value in Figure 3). If you specify `start_by='y'`, the starting value is for the y-axis (the *L* value in Figure 3). The default value is `x`.

`[<stop_by>]`

(Optional) The stopping value, which can be X or Y. If you specify `stop_by='x'`, the stopping value is for the x-axis (the *x-stop* value in Figure 3). If you specify `stop_by='y'`, the stopping value is for the y-axis (the *H* value in Figure 3). The default value is `'x'`.

`[<half_error_band>]`

(Optional) If `half_error_band` is `false` (the default), the signal should settle between *H*+(*d*/2) and *H*-(*d*/2). If `half_error_band` is true, the user-defined `settle_band` value is equal to half of the error band. In that case, the signal should settle between *H*+*d* & *H*-*d*.

Feedback

`[<occurrences>]`

Specifies whether you want to retrieve only one occurrence of a settling time event (single), or all occurrences of settling time (multiple). The default value is single.

`[<output_xaxistype>]`

Specifies whether you want to output settling time data against time or cycle. Cycle numbers is the nth occurrence of the setting time event in the input waveform. This option is valid only when [<occurrences>] is set to “multiple”.

Errors

Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

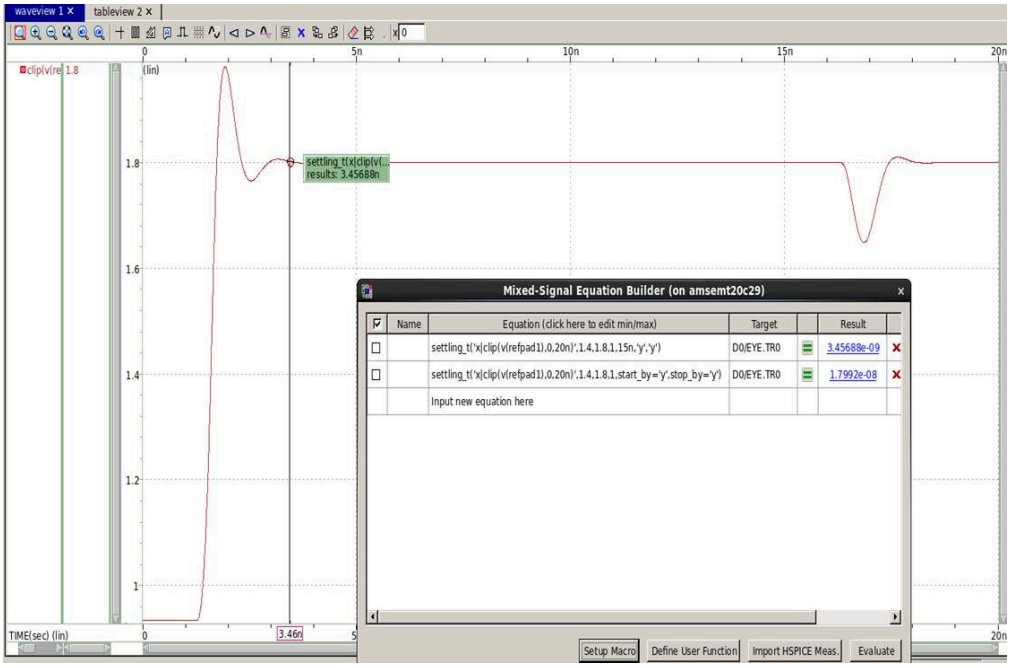
Examples

```
settling_t('x|clip(v(refpad1),0,20n)',1.4,1.8,settle_band=1,xend=15n,start_by='y',stop_by='y')
```

In this example, first `d` value is calculated as `(1.8-1.4)*1%=0.004`.

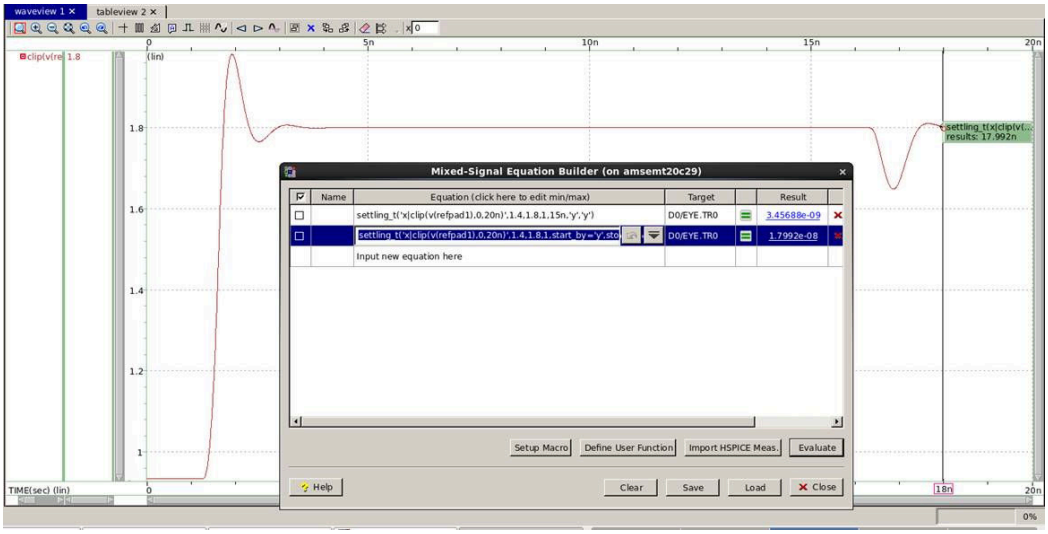
Therefore, the signal can be considered settled when the y values of the signals are within `1.8+0.004=1.8004 and 1.8-0.004=1.7996`, and tool will report the `xvalue(time)` at which this signal entered in to this range as settling time. Here, since `xend=15n`, the tool will not consider the second glitch because it appears after 15n.

Feedback




```
settling_t('x|clip(v(refpad1),0,20n)',1.4,1.8,1,start_by='y',stop_by='y')
```

In this example, there is no `xend` value defined, so the tool considers the entire signal (up to 20n) for the settling time. Because there is a glitch occurring around 17n, the settling happens after the glitch.



Feedback

sfdr

Description

Measures the spurious free dynamic range. SFDR is the strength ratio of the fundamental signal to the strongest spurious signal in the specified frequency band set by the Start and End frequency fields.

$$SFDR(dBc) = 20\log\left(\frac{V_1}{\sqrt{max(V_2^2, V_2^2, ..., V_n^2)}}\right)$$

or:

$$SFDR(dBFS) = 20\log\left(\frac{1}{\sqrt{max(V_2^2, V_2^2, ..., V_n^2)}}\right)$$

when the `full_scale` option is true.

Syntax

```
sfdr(<signal>,<x_start>,<x_stop>,<num_of_points>,
<signal_freq>,[<band_limit>],[<windowing_type>],
[<decimation_rate>],[<bin_size>],
[<window_alpha>],[<normalization>],
[<user_defined_value>],[<suppress_dc>],
[<dual_sideband>],[<max_harmonic_index>],[<include floor
 noise>],[<full_scale>])
```

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of x.

`<x_stop>`

The stopping value of x.

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

`[<band_limit>]`

(Optional) The limit band frequency to be set.

Feedback

`[<windowing_type>]`

(Optional) Specifies the windowing function. The default is rectangle.

`[<decimation_rate>]`

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. Normally, this argument is an integer or a rational fraction greater than one dividing the sampling rate.

`[<bin_size>]`

(Optional) For window sampling, the `bin_size` parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

`[<window_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types.

`[<normalization>]`

(Optional) There are three options for this argument:

- none — FFT results are not normalized.
- normalized — FFT results are normalized against the maximum magnitude value of the waveform.
- user-defined — FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if [<normalization>] is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

`[<dual_sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<max_harmonic_index>]`

(Optional) The harmonic index of the maximum harmonic component.

Feedback

`[<include floor noise>]`

(Optional) When enabled, the `sfdr` function takes spurs from harmonics and noise.

`[<full_scale>]`

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.

Errors

Possible error messages include:

- Signal not found
- Invalid input value
- X value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2

Examples

```
sfdr('0|v(out)',2n,4n,1024,500M)
sfdr('0|v(out)',2n,4n,1024,500M,550M,'rectangle',deci=2)
```

---

signalPower

Measures the Signal Power in support of spectrum measurements.

Syntax

```
signalPower(<signal>,<x_start>,<x_stop>,<num_of_points>,
<signal_freq>,[<band_limit>],[<windowing_type>],
[<decimation_rate>]),[<bin_size>],[<window_alpha>],
[<normalization>],[<user_defined_value>],[<suppress_dc>],
[<dual_sideband>],[<max_harmonic_index>],[include_floor_noise],
[<full_scale>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of x.

Feedback

`<x_stop>`

The stopping value of x.

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

`[<band_limit>]`

(Optional) The limit band frequency to be set.

`[<windowing_type>]`

Specifies the windowing function. The default is rectangle.

`[<decimation rate>]`

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. This argument is usually an integer or a rational fraction greater than one dividing the sample rate.

`[<bin_size>]`

(Optional) For window sampling, the bin_size parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

`[<window_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

`[<normalization>]`

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if `normalization` is set to user-defined.

Feedback

[<suppress_dc>]

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

[<dual_sideband>]

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

[<max_harmonic_index>]

(Optional) The harmonic index of the maximum harmonic component.

[<include_floor_noise>]

(Optional) If set to true, floor noise components are counted in SNR calculation.

[<full_scale>]

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.

---

slewrate

Measures the slew rate of given signal, where slew rate is defined as the maximum rate of change of the output value. The following equation represents the slew rate calculation:

$$SR = max\left(\left|\frac{dy}{dx}\right|\right)$$

Syntax

```
slewrate(<signal>,<y_init>,<y_final>,[<percent>],[<min>],[<max>],[<min_value_type>],[<max_value_type>],[<occurrences>],[<output_xaxistype>])
```

Returns

An analog scalar value.

Arguments

<signal>

The signal to be measured.

<y_init>

The initial crossing of the y_threshold.

<y_final>

The final crossing of the y_threshold.

Feedback

`[<percent>]`

(Optional) The percentage value, which can only be `0` or `1`. If specified as `1`, the threshold value is considered a percentage value; if not, the threshold value is `0`.

`[<min>]`

(Optional) Min value for percentage y init/final. Default is xmin or ymin depending on type.

`[<max>]`

(Optional) max value for percentage y init/final, default is xmax or ymax depending on type.

`[<min_value_type>]`

(Optional) Specifies whether the [<min>] value is an X-axis (x) or Y-axis value. (y). The default value is y.

`[<max_value_type>]`

(Optional) Specifies whether the [<max>] value is an X-axis (x) or Y-axis value (y). The default value is y.

`[<occurrences>]`

(Optional) Specifies whether you want to retrieve only one occurrence of a slewrate time event (single), or all occurrences of slewrate time events (multiple). The default value is single.

`[<output_xaxistype>]`

(Optional) Specifies whether you want to output slewrate data against time, or cycle. Cycle numbers refer to the nth occurrence of the slewrate event. This option is valid only when `[<occurrences>]` is set to `multiple`.

Errors

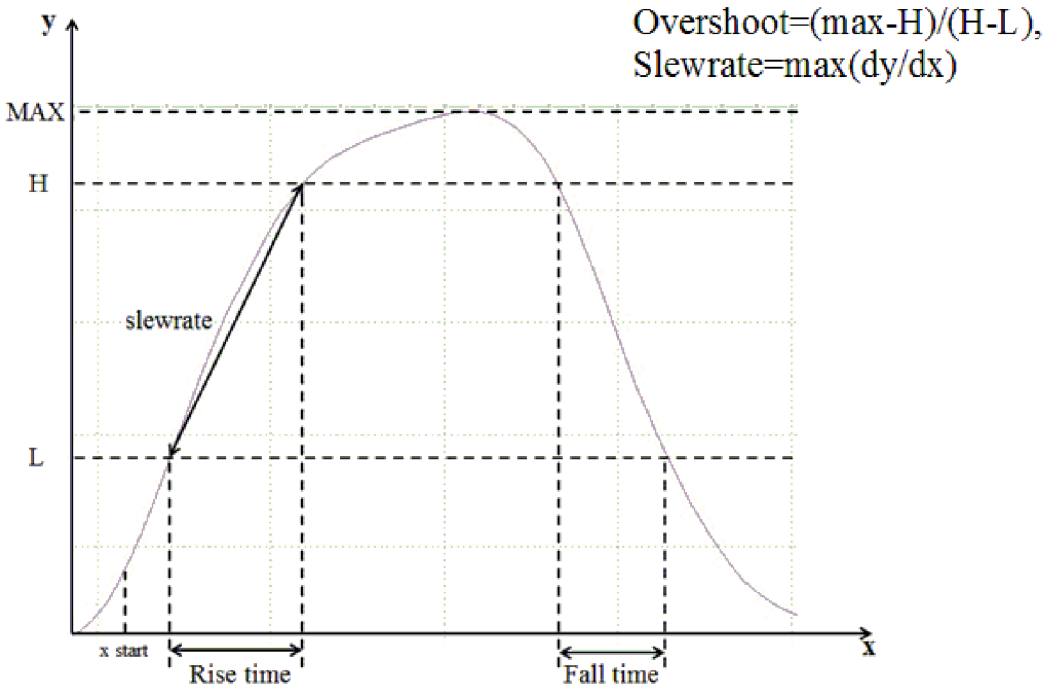
Possible error messages include:

- Signal not found
- Y value out of range

Examples

```
slewrate('0|v(out)',2,4)
slewrate('0|v(out)',2,4,0)
```

Feedback



slope

Measure the slope value of the given signal at x value.

Syntax

```
slope(<signal>,<x_value>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_value>`

The x value to be set.

Errors

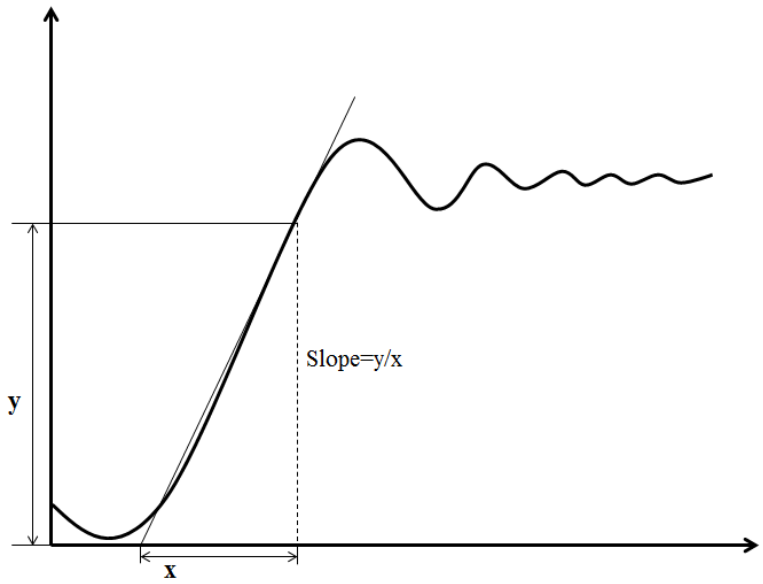
Possible error messages include:

- Signal not found
- Y value out of range

Feedback

Examples

```
slope('0|v(out)',2n)
```



snb

Measures the Noise floor/Bin in support of spectrum measurements.

Syntax

```
snb(<signal>,<x_start>,<x_stop>,<num_of_points>,
<signal_freq>,[<band_limit>],[<windowing_type>],
[<decimation_rate>]),[<bin_size>],[<window_alpha>],
[<normalization>],[<user_defined_value>],[<suppress_dc>],
[<dual_sideband>],[<max_harmonic_index>],[include_floor_noise],
[<full_scale>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of x.

Feedback

`<x_stop>`

The stopping value of x.

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

`[<band_limit>]`

(Optional) The limit band frequency to be set.

`[<windowing_type>]`

Specifies the windowing function. The default is rectangle.

`[<decimation rate>]`

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. This argument is usually an integer or a rational fraction greater than one dividing the sample rate.

`[<bin_size>]`

(Optional) For window sampling, the bin_size parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

`[<window_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

`[<normalization>]`

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if `normalization` is set to user-defined.

Feedback

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

`[<dual_sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<max_harmonic_index>]`

(Optional) The harmonic index of the maximum harmonic component.

`[<include_floor_noise>]`

(Optional) If set to true, floor noise components are counted in SNR calculation.

`[<full_scale>]`

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.

`[<window_sampling>]`

(Optional) Determines whether the window samples are computed in a periodic or symmetric manner. For example, when set to periodic, the window extends by 1 sample. The default is symmetric.

---

sndr

Description

Measure the signal to noise-and-distortion ratio of a given signal (in dB units). The Fast Fourier Transform (FFT) evenly Piece-Wise-Constant (PWC) sample signals from `x_start` to `x_stop` for n points. The algorithm of SNDR (Signal to Noise-and-Distortion Ratio) can be: $20\log\sqrt{\frac{V_1^2}{V_2^2+V_3^2+V_4^2+\ldots V_n^2}}$

V_1 is a fundamental component, here is signal frequency, within the range of `x_start` and `x_stop`, among which there are n points, so V_2 . . . V_n include harmonic frequency and distortion components.

Syntax

```
sndr(<signal>,<x_start>,<x_stop>,<num_of_points>,
<signal_freq>,[<band_limit>],[<windowing_type>],
[<decimation_rate>]),[<bin_size>],[<window_alpha>],
[<normalization>],[<user_defined_value>],[<suppress_dc>],
[<dual_sideband>],[include_floor_noise])
```

Feedback

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of x.

`<x_stop>`

The stopping value of x.

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

`[<band_limit>]`

(Optional) The limit band frequency to be set.

`[<windowing_type>]`

Specifies the windowing function. The default is rectangle.

`[<decimation_rate>]`

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. Usually this argument is an integer or a rational fraction greater than one dividing the sampling rate.

`[<bin_size>]`

(Optional) For window sampling, the `bin_size` parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

`[<window_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

Feedback

`[<normalization>]`

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if [<normalization>] is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

`[<dual_sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<include_floor_noise>]`

(Optional) If set to true, floor noise components are counted in SNDR calculation.

`[<window_sampling>]`

(Optional) Determines whether the window samples are computed in a periodic or symmetric manner. For example, when set to periodic, the window extends by 1 sample. The default is symmetric.

Errors

Possible error messages include:

- Signal not found
- Invalid input value
- X value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2

Feedback

Examples

```
sndr('0|v(out)',2n,40n,1024,500M)
sndr('0|v(out)',2n,40n,1024,500M,550M,'rectangle',deci=2)
```

---

snr

Description

Measures the signal-to-noise ratio of given signal in dB units. The Fast Fourier Transform (FFT) evenly Piece-Wise-Constant (PWC) samples signals from `x_start` to `x_stop` for `n` points. The SNR is the ratio of the remaining harmonics to the fundamental. The following function is the algorithm for the SNR (Signal to Noise Ratio):

$$20\log\left(\frac{V_1}{\sqrt{V_{i+1}^2+V_{i+2}^2+\ldots+V_n^2}}\right)$$

V_1 is the fundamental component, here is signal frequency, within the range of `x_start` and `x_stop`, among which there are n points. Where *i* is the value of `max_harmonic_index`, so V_{i+1}... V_n are remaining harmonic frequency.

Syntax

```
snr(<signal>,<x_start>,<x_stop>,<num_of_points>,
<signal_freq>,[<band_limit>],[<windowing_type>],
[<decimation_rate>]),[<bin_size>],[<window_alpha>],
[<normalization>],[<user_defined_value>],[<suppress_dc>],
[<dual_sideband>],[<max_harmonic_index>],[include_floor_noise],
[<full_scale>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of x.

`<x_stop>`

The stopping value of x.

`<num_of_points>`

The number of points needing to be measured.

Feedback

`<signal_freq>`

The input signal frequency.

`[<band_limit>]`

(Optional) The limit band frequency to be set.

`[<windowing_type>]`

Specifies the windowing function. The default is rectangle.

`[<decimation rate>]`

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. This argument is usually an integer or a rational fraction greater than one dividing the sample rate.

`[<bin_size>]`

(Optional) For window sampling, the bin_size parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

`[<window_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

`[<normalization>]`

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if `normalization` is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

Feedback

`[<dual_sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<max_harmonic_index>]`

(Optional) The harmonic index of the maximum harmonic component.

`[<include_floor_noise>]`

(Optional) If set to true, floor noise components are counted in SNR calculation.

`[<full_scale>]`

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.

`[<window_sampling>]`

(Optional) Determines whether the window samples are computed in a periodic or symmetric manner. For example, when set to periodic, the window extends by 1 sample. The default is symmetric.

Errors

Possible error messages include:

- Signal not found
- Invalid input value
- X value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2

Examples

```
snr('0|v(out)',4n,8n,1024,750M)
snr('0|v(out)',4n,8n,1024,750M,10G,deci=20)
```

---

snr_f

Measures the signal-to-noise ratio of a given signal (in dB units), which is the resulting waveform of a Fast Fourier Transform (FFT) signal. A similar algorithm with SNR.
V_1 is the signal component in the range between `signal_from` and `signal_to`;
V_2... V_n are the noise components in the range of `noise_from` and `noise_to`.

$$20\log\sqrt{\frac{V_1^2}{V_2^2+V_3^2+V_4^2...+V_n^2}}$$

Feedback

Syntax

```
snr_f(<signal>,<signal_from>,<signal_to>,<noise_from>, <noise_to>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<signal_from>`

The starting point of signal around harmonic.

`<signal_to>`

The stopping point of signal around harmonic.

`<noise_from>`

The point where noise begins.

`<noise_to>`

The point where noise stops.

`[<window_sampling>]`

(Optional) Determines whether the window samples are computed in a periodic or symmetric manner. For example, when set to periodic, the window extends by 1 sample. The default is symmetric.

Errors

Possible error messages include:

- Signal not found
- Invalid input value

Examples

```
snr_f('0|v(out)',75M,80M,100m,10G)
```

---

snrh

Measures the Noise floor/rtHZ in support of spectrum measurements.

Feedback

Syntax

```
snrh(<signal>,<x_start>,<x_stop>,<num_of_points>,
<signal_freq>,[<band_limit>],[<windowing_type>],
[<decimation_rate>]),[<bin_size>],[<window_alpha>],
[<normalization>],[<user_defined_value>],[<suppress_dc>],
[<dual_sideband>],[<max_harmonic_index>],[include_floor_noise],
[<full_scale>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of x.

`<x_stop>`

The stopping value of x.

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

`[<band_limit>]`

(Optional) The limit band frequency to be set.

`[<windowing_type>]`

Specifies the windowing function. The default is rectangle.

`[<decimation rate>]`

Specifies the decimation rate, which is the process of reducing the sampling rate a signal. This argument is usually an integer or a rational fraction greater than one dividing the sample rate.

`[<bin_size>]`

(Optional) For window sampling, the bin_size parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. Default bin size is 10.

Feedback

`[<window_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

`[<normalization>]`

(Optional) There are three options for this argument:

- None - FFT results are not normalized.
- Normalized - FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined - FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if `normalization` is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

`[<dual_sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<max_harmonic_index>]`

(Optional) The harmonic index of the maximum harmonic component.

`[<include_floor_noise>]`

(Optional) If set to true, floor noise components are counted in SNR calculation.

`[<full_scale>]`

(Optional) If set to true, the magnitude of fundamental component is scaled to 1.

---

stddev

Measures the standard deviation of the Y values for given signal and the X values are discarded. For discrete random variable, the standard deviation is $\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Y_i - \bar{Y})^2}$

Feedback

Syntax

```
stddev(<signal/scalar>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

Errors

Possible error messages include:

- Signal not found

Examples

```
stddev('0|v(out)')
```

---

sweepValueDiff

Calculates the value difference between values of sweep scalars.

Syntax

```
sweepValueDiff(<signal>)
```

Returns

A waveform.

Arguments

`<signal>`

The input sweep scalar signal.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Mixed logic/analog-function equation not supported

Feedback

Examples

For a usage example, see SolvNetPlus article #000042278, "How can I use the sweepValueDiff function to calculate differences in simulation results in the PrimeWave Design Environment?"

---

thd

Measures the total harmonic distortion of a given signal (in dB units). The Fast Fourier Transform (FFT) evenly Piece-Wise-Constant (PWC) sample signals from `x_start` to `x_stop` for n points. The following function is the algorithm for the THD (Total Harmonic Distortion). V_1 is the fundamental component, so V_2, …, V*i* include harmonic frequency where *i* is the value of `max_harmonic_index`. $20\log\left(\frac{\sqrt{V_2^2+V_3^2+\ldots+V_I^2}}{V_1}\right)$

Syntax

```
thd(<signal>,<x_start>,<x_stop>,<num_of_points>,<signal_freq>,
 [<band_limit>],[<windowing_type>],[<harmonic_in_index>],
[<decimation_rate>],[<bin_size>],[<window_alpha>],
[<normalization>],[<user_defined_value>],[<suppress_dc>],
[<dual_sideband>],[<max_harmonic_index>],[<full_scale>],
[<output_type>],[all_frequencies])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting value of x.

`<x_stop>`

The stopping value of x.

`<num_of_points>`

The number of points needing to be measured.

`<signal_freq>`

The input signal frequency.

`[<band_limit>]`

(Optional) The limit band frequency to be set.

Feedback

`[<windowing_type>]`

(Optional) Specifies the windowing function. The default is `rectangle`.

`[<harmonic_in_index>]`

(Optional) The maximum harmonic index flag, which can only be 0 or 1. When specified as 1, the `band_limit` value is considered as maximum harmonic index. This option is disabled if the `max_harmonic_index` value is set.

`[<decimation_rate>]`

(Optional) Specifies the decimation rate, which is the process of reducing the sampling rate a signal. Normally, this argument is an integer or a rational fraction greater than one dividing the sampling rate.

`[<bin_size>]`

(Optional) For window sampling, the `bin_size` parameter is required to include all neighboring components within +/- bin size spectral range as part of the original fundamental (or harmonic) components. The default bin size is 10.

`[<window_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

`[<normalization>]`

(Optional) There are three options for this argument:

- None — FFT results are not normalized.
- Normalized — FFT results are normalized against the maximum magnitude value of the waveform.
- User-defined — FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if `normalization` is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

Feedback

`[<dual-sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<max_harmonic_index>]`

(Optional) The harmonic index of the maximum harmonic component.

`[<full_scale>]`

(Optional) If set to true, the THD is calculated with respect to full-scale. Default is false.

`[<output_type>]`

(Optional) Specifies the output as dB or percentage.

`[all_frequencies]`

When enabled, considers all frequencies.

Errors

Possible error messages include:

- Signal not found
- Invalid input value
- Number of sample points is non-positive
- Number of sample points is not power of 2

Examples

```
thd('0|v(out)',2n,40n,1024,650M)
thd('0|v(out)',2n,40n,1024,650M,8,'rectangle',ish=1,deci=30)
```

---

variance

Measures the variance of the Y values for given signal (the X values are discarded). For discrete random variable, the variance is $Var(Y) = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \bar{Y})^2$

Syntax

`variance(<signal>)`

Returns

An analog scalar value.

Feedback

Arguments

`<signal>`

The signal to be measured.

Errors

Possible error messages include:

- Signal not found

Examples

```
variance('0|v(out)')
```

---

x_at_ymax

Measures the x value at the maximum y value of given signal from `x_start` to `x_stop`.

Syntax

```
x_at_ymax(<signal>,[<x_start>],[<x_stop>],[<nth_max>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`[<x_start>]`

(Optional) The starting point of x.

`[<x_stop>]`

(Optional) The stopping point of x.

`[<nth_max>]`

(Optional) The nth maximum of the given signal counting from `x_start` to `x_stop`. If you enter a negative integer, it will count from `x_stop` to `x_start`.

Feedback

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- X value out of range

Examples

```
x_at_ymax('0|v(out)')
x_at_ymax(0|v(out)',2n,4n)
```

---

x_at_ymin

Measures the x value at the minimum y value of given signal from `x_start` to `x_stop`.

Syntax

```
x_at_ymin(<signal>,[<x_start>],[<x_stop>],[<nth_min>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured.

`[<x_start>]`

(Optional) The starting point of x.

`[<x_stop>]`

(Optional) The stopping point of x.

`[<nth_min>]`

(Optional) The nth minimum of the given signal counting from `x_start` to `x_stop`. If you enter a negative integer, it will count from `x_stop` to `x_start`.

Feedback

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- X value out of range

Examples

```
x_at_ymin('0|v(out)')
x_at_ymin('0|v(out)',2n,4n)
```

---

xmax

Measures the maximum x value of a given signal.

Syntax

```
xmax(<signal>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured (analog signals are supported). For complex signals, the magnitude is automatically applied.

Errors

Possible error messages include:

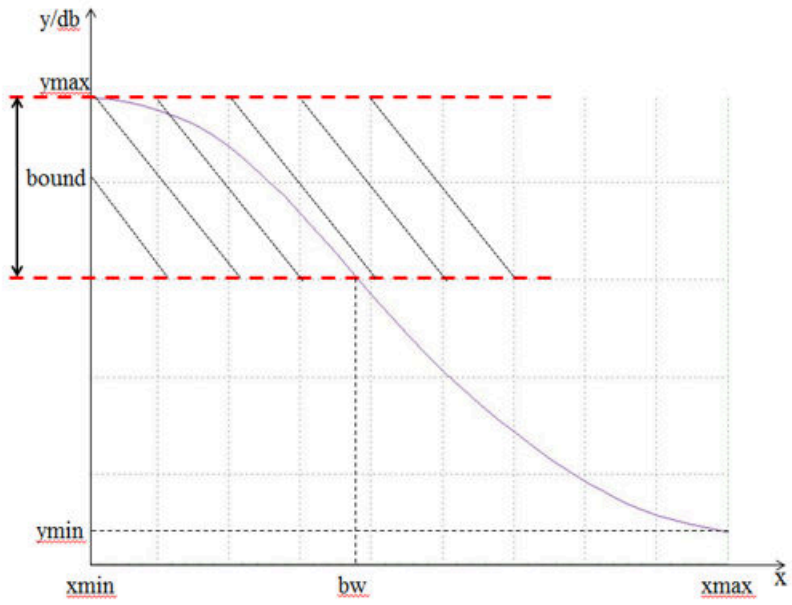
- Signal not found

Examples

```
xmax('0|v(out)')
```

```
xmax('0|v(out)')
```

Feedback



xmin

Measures the minimum x value of a given signal.

Syntax

```
xmin(<signal>)
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured (analog signals are supported). For complex signals, the magnitude is automatically applied.

Errors

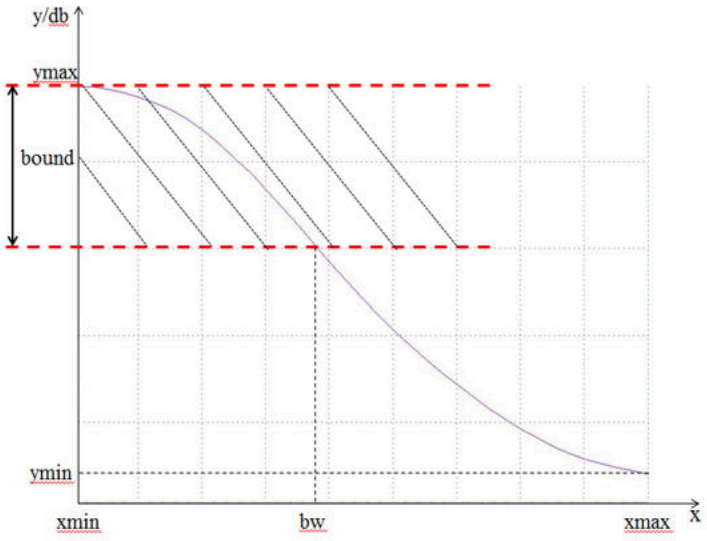
Possible error messages include:

- Signal not found

Examples

```
xmin('0|v(out)')
```

Feedback



yield

Measures the count of the value specified by spec_min and spec_max and returns the value for (valid_count/total_count)*100.

Where,

total_count = total number of input data point count

valid_count = total_count - invalid_count

If spec_min is defined and any one data point in the input (signal/measurement) < spec_min, or if spec_max is defined and the value > spec_max, then it is an invalid_count.

Also all NaN values and errors are viewed as invalid_count.

Syntax

```
yield(signal<string>,spec_min<string>,spec_max<string>)
```

Arguments

`signal`

The signal or measurement to be calculated.

`spec_min`

(Optional) The minimum spec value.

Feedback

`spec_max`

(Optional) The maximum spec value.

---

ymax

Measures the maximum y value of a given signal.

Syntax

```
ymax(<signal/scalar>,[<x_start>],[<x_stop>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured (analog signals are supported). For complex signals, the magnitude is automatically applied.

`[<x_start>]`

(Optional) Specifies the starting x value. The default is the signal start time.

`[<x_stop>]`

(Optional) Specifies the x value end time. The default is the signal end time.

Errors

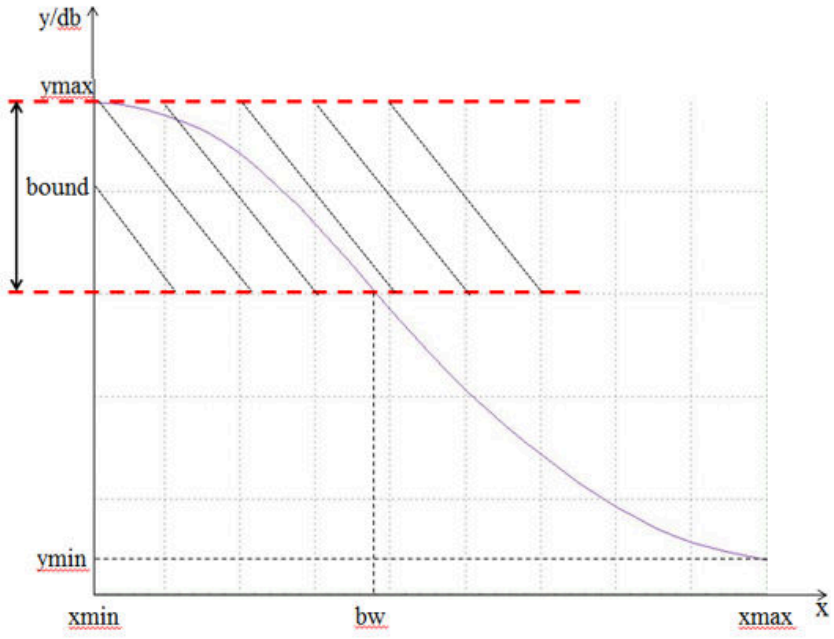
Possible error messages include:

- Signal not found
- Incorrect signal type
- X value out of range

Examples

```
ymax('0|v(out)')
ymax('0|v(out)',10M,100M)
```

Feedback



ymin

Measures the minimum y value of a given signal.

Syntax

```
ymin(<signal/scalar>,[<x_start>],[<x_stop>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be measured (analog signals are supported). For complex signals, the magnitude is automatically applied.

`[<x_start>]`

(Optional) Specifies the starting x value. The default is the signal start time.

`[<x_stop>]`

(Optional) Specifies the x value end time. The default is the signal end time.

Feedback

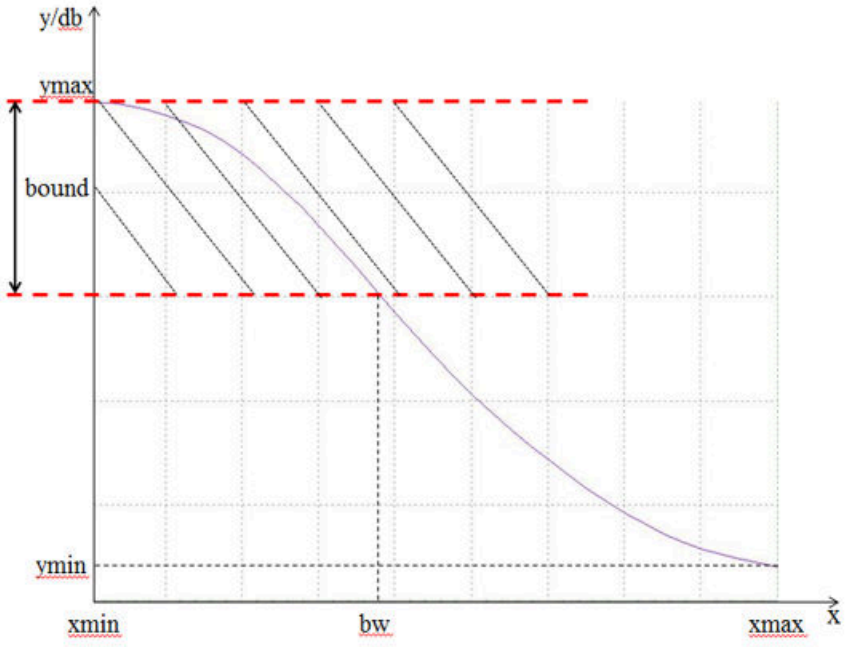
Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- X value out of range

Examples

```
ymin('0|v(out)')
ymin('0|v(out)',10M,100M)
```



---

yvalue

Measures the y value of a given signal at x.

Syntax

```
yvalue(<signal/scalar>,<x_value>,[<method>],
[<occurrences>],[<period>],[<output_xAxisType>],[<extrapolate>])
```

Returns

An analog scalar value.

Feedback

Arguments

`<signal>`

The signal to be measured.

`<x_value>`

The input x value.

`[<method>]`

(Optional) Specifies the interpolation method. The default is linear. If set to `round_down`, the point value is the lower bound of an interval that contained it.

`[<occurrences>]`

(Optional) Specifies the result to be only one occurrence of an interpolated value for the given waveform (`single`) or all interpolated values for the given waveform (`multiple`), which you can later plot or print. The default is `single`.

`[<period>]`

(Optional) Specifies the time interval to calculate the next Y value.

`[<output_xAxisType>]`

(Optional) Specifies the X axis type of the result waveform. The value can be `x` or `idx`.

`[<extrapolate>]`

(Optional) When enabled, ignores the boundary and always returns a value.

Errors

Possible error messages include:

- Signal not found
- X value out of range

Examples

```
yvalue('0|v(out)',2n)
```

Waveform Functions

This topic contains information on how to use the supported waveform functions.

Feedback

The following sections summarize the waveform functions available in the Calculator. Waveform functions must operate on waveforms (or derived waveforms). They always return new waveforms.

Function	Description
a2d	Converts the input signal from analog to digital.
average	Calculates the running average waveform of a given signal in the specified range.
bestfit	Measures the best fit line for a waveform.
clip	Generates a new waveform whose x range is limited to the given x_start and x_stop values. The y range is limited to the given y_min and y_max values.
concat	Concatenates up to 32 signals or scalar values into one signal. The result signal's x-axis values are data point indixes.
cPwrContour	Generates contours with the same power for Z-Smith.
create_bus	Creates a bus signal with given name and up to 31 digital signals.
create_eye	Creates an eye object which can be plotted as an eye diagram.
crossings	Calculates the crossing points between signal1 and signal2 or a given ylevel value.
d2a	Converts the input signal from digital to analog.
dBc2V2	Converts phase noise signal from dBc/Hz to V^2/Hz. Only works for waveform files that contain carrier magnitude information.
dBCPVRICurves	Creates a signal associated with an nth compression point measurement.
delayvst	Measures the time difference versus the time between two signals at the specified switching edges from x_start to x_stop.
derivative	Calculates the derivative waveform of given signal.
dft	Measures the Fast-Fourier-Transform (FFT) of a given signal. The signal is evenly Piece-Wise-Constant sampled from x_start to x_stop for n points.
dftbb	Computes the fast Fourier transform (FFT) of a complex signal.
dnl	Computes the differential non-linearity of a transient simple or parametric waveform.
dumpPWLF	Saves the signal to a text file for PWLF sources.
duty_vs_t	Measures the duty cycle versus time.

Feedback

Function	Description
fallvst	Measures the time difference versus the time of the given signal starting from high y_level to low y_level within the range of x_start and x_stop.
fft	Measures the Fast-Fourier-Transform (FFT) of a given signal. The signal is evenly Piece-Wise-Constant sampled from x_start to x_stop for n points.
flip	Reverses the signal along the y-axis.
fourEval	Evaluates the Fourier series represented by the buffer expression.
func	Plots the waveform of signal1 versus signal2 for the two specified signals.
fvst	Calculates the frequency versus time for the given signal.
gdelay	Measures the group delay waveform of a given signal.
histo	Generates a histogram waveform for the input signal.
hold_times	This is used to plot a waveform that contains all of the hold time data points.
hold_times2	Plots a waveform containing all of the hold time data points.
ift	Calculates the Inverse Fourier transform (IFT) time domain waveform of the input frequency domain signal.
inl	Computes the integral non-linearity of a transient simple or parametric waveform.
integral	Measures the running integral waveform of the input signal.
ipnhVRICurves	N dB intercept point VRI Curves.
join	Generates a joined waveform from signal1 and signal2, which depends on the starting x-axis value of the two signals.
jvst_freq	Calculates the jitter versus time given the frequency of an ideal clock.
jvst_ideal	Calculates the jitter versus time with an ideal clock.
jvst_llse	Calculates the jitter versus time with an estimated frequency of the ideal clock using LLSE (Linear Least Squares Estimator).
jvst_refsig	Calculates the jitter versus time using a reference signal as the clock.
loadpull	Generates load pull contour of waveform PSS analysis.
max_window_average	Calculates the maximum window average using step size between a start and a stop time.
median	Measures the smoothed waveform of the given signal using median value method.

Feedback

Function	Description
optical_fft	Converts time-domain optical signal waveforms into their equivalent complex optical spectra. The FFT of the optical signal is calculated over the full range of negative and positive frequencies.
parametric	Gets the parametric plots of measure versus sweeping param_name.
pareto_r	Calculates the Pareto regression result for the s signal and the parameter file.
pareto_r2	Calculates the Pareto regression squared result for the s signal and the pfile parameter file.
pareto_s	Calculates the Pareto sensitivity result for the s signal and the pfile parameter file.
pass	Returns 0 (specifying fail) or 1 (specifying pass) for a given expression defined in PrimeWave's output view. If the specification of the input expression name is pass, it returns 1, else it returns 0.
pavg	Calculates the average value of signal based on the given period.
peak	Calculates the peak of a given signal.
phasenoise	Calculates the phase noise of a given signal.
prms	Calculates the root-mean-square value of signal based on the given period.
psd	Power spectral density. Describes how the power (or variance) of a time series (signal) is distributed with frequency.
psdbb	Power spectral density baseband. Returns an estimate for the power spectral density of a waveform1+j * waveform2.
pstddev	Calculates the standard deviation value of signal based on the given period.
pwc	Measures the piece-wise constant value of a signal.
pzFreqRealFilter	Returns a filtered pole or zero signal from the input poles/zeros signal.
reflectionContour	Constant reflection coefficient contour.
repeat	Creates a new waveform by repeating the given period signal for a specified number of times.
risevst	Calculates the time difference versus the time of the given signal between x_start and x_stop from the low threshold crossing to the high threshold crossing.
sample	Creates a new waveform which is a sample of a given signal starting from x with period for a given number of times.

Feedback

Function	Description
setup_times	Generates a new waveform containing all of the setup time data points.
setup_times2	Generates a new waveform that contains all of the setup time data points.
signalAtFreq	Calculates the signal at frequency.
signalcompare	Compares the waveform outputs of the simulation.
slewratevst	Calculates the slew rate versus time for the given signal within the y_initial and y_final.
smooth	Calculates the smoothed waveform from the given signal.
spectralPower	Calculates the spectral power for the specified current waveform and voltage waveform.
sPowerDiffVI	Calculates spectral power with two nodes voltage and current signals.
sPowerDiffVR	Calculates spectral power with two nodes voltage and resistance.
swapsweep	Swaps the x-axis value with the specified sweep variable.
sweepavg	Calculates the average waveform of a multi-trace sweep waveform.
sweepmax	Calculates the maximum bound waveform of a multi-trace sweep.
sweepmin	Calculates the minimum bound waveform of a multi-trace sweep.
tangent	Calculates a line that passes through x and y coordinates and the specified slope.
text2wave	Reads in a text file and plots the waveform.
trace	Returns a trace from a multi-trace of signal s based on the parameter condition parm_con.
v2todBc	Converts phase noise from V^2/Hz to dBc/Hz.
window_average	Calculates the window average.
x_at_y	Plots the crossing points of a given signal on a specified y_threshold.
xscale	Plots a new waveform identical to the given signal with the x component multiplied by a scaling factor.
xshift	Plots a new waveform identical to the given signal but shifted to the right by x delay.
xval	Measures the x value of the given signal and creates a new waveform equivalent to y=x.

Feedback

Function	Description
yvsi	Creates a new waveform with a y value versus a data point index signal.

a2d

Converts the input signal from analog to digital.

Syntax

```
a2d(<signal>,<low_bounds>,<high_bounds>,[<num_of_bits>],
 [<time_step>],[<x_start>],[<x_stop>],
[<single_threshold>],[<delay_to_x>])
```

Returns

A logic signal.

Arguments

`<signal>`

The analog signal to input.

`<low_bounds>`

The low threshold of y.

`<high_bounds>`

The high threshold of y.

`[<num_of_bits>]`

(Optional) Number of bits to be set.

`[<time_step>]`

(Optional) The value of each time_step.

`[<x_start>]`

(Optional) The starting point of x.

`[<x_stop>]`

(Optional) The ending point of x.

`[<single_threshold>]`

(Optional) Specifies the center threshold of y. This argument conflicts with low_bounds and high_bounds.

Feedback

`[<delay_to_x>]`

(Optional) Specifies time transferred to x.

Errors

Possible error messages include:

- Signal not found
- Y value out of range
- Bit number can’t be greater than 32
- X value out of range

Examples

```
a2d('0|v(out)',2,4);
a2d('0|v(out)',2,4,5,2n,0,10n)
```

---

average

Calculates the running average waveform of a given signal in the specified range.

Syntax

```
average(<signal>,[<x_start>],[<x_stop>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`[<x_start>]`

(Optional) The starting value of x.

`[<x_stop>]`

(Optional) The stopping value of x.

Errors

Possible error messages include:

- Signal not found
- X value out of range

Feedback

Examples

```
average('0|v(out)')
average('0|v(out)', 2n,14n)
```

---

bestfit

Measures the best fit line for a waveform.

Syntax

```
bestfit(<signal>, [<type>], [<power>])
```

Returns

A waveform to fit the signal.

Arguments

`<signal>`

The signal or value to be calculated.

`<type>`

The method applied to calculate the fitted line. It can be 'Polynominal fitting'(default), 'Exponential fitting', 'Logarithmical fitting', or 'Ratio polynom fitting'.

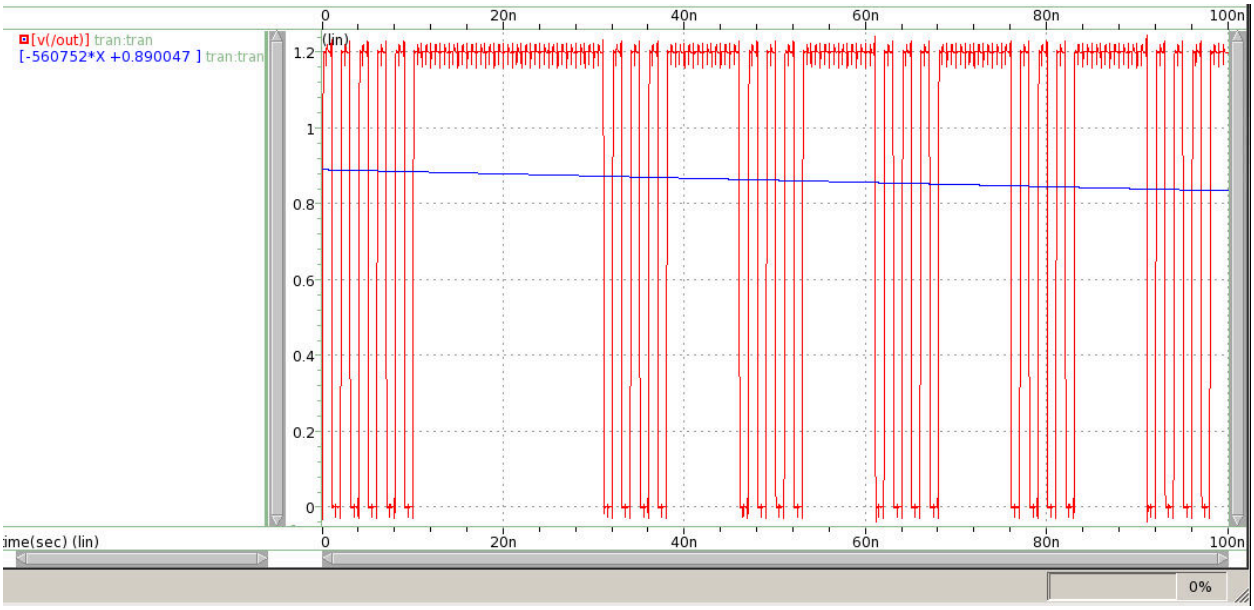
`<power>`

Power value for 'Exponential fitting'. Default value is 1.

Examples

```
bestfit('v(f6g)')
bestfit('v(f6g)','Exponential fitting','3')
bestfit('out')
```

Feedback



clip

Generates a new waveform whose x range is limited to the given x_start and x_stop values. The y range is limited to the given y_min and y_max values.

Syntax

```
clip(<signal>,[<x_start>],[<x_stop>],[<y_min>],[<y_max>],[<percent>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be clipped.

`[<x_start>]`

(Optional) The point of x where the clip starts.

`[<x_stop>]`

(Optional) The value of x where the clip stops.

`[<y_min>]`

(Optional) The low threshold of y.

Feedback

`[<y_max>]`

(Optional) The high threshold of y.

`[<percent>]`

(Optional) The value can be `0` or `1`. If the value is specified as `1`, the y_threshold is considered as a percentage value. If the value is specified as `0`, the y_threshold is not considered a percentage value.

Errors

Possible error messages include:

- Signal not found
- X value out of range
- Y value out of range

Examples

```
clip('0|v(out)');
clip('0|v(out)',2n,4n,1,3,1)
```

---

concat

Description

Concatenates up to 32 signals or scalar values into one signal. The result signal's X axis values are data point indices.

Syntax

```
concat <signal1> <signal2> [<signal3> … <signal32>]
```

Returns

A signal.

Arguments

`<signal1>`

The first signal to concatenate. It can be a scalar value.

`<signal2>`

The second signal to concatenate. It can be a scalar value.

`[<signa3> … <signal32>]`

(Optional) More signals to concatenate. They can be scalar values.

Feedback

cPwrContour

Description

Generates contours with the same power for Z-Smith.

Syntax

```
cPwrContour(<current_signal>, <voltage_signal>, <harmonic_index>,
 [<current_load>], [<voltage_load>], [<max_power>], [<min_power>],
 [<num_contour>], [<resistance>], [<close>], [<modifier>])
```

Returns

A waveform.

Arguments

`<current_signal>`

Signal used to calculate power as current.

`<voltage_signal>`

Signal used to calculate power as voltage.

`<harmonic_index>`

Harmonic index of waves in current_signal and voltage_signal.

`[<current_load>]`

(Optional) Signal used to calculate reflection coefficient as current.

`[<voltage_load>]`

(Optional) Signal used to calculate reflection coefficient as voltage.

`[<max_power>]`

(Optional) The maximum power in contours.

`[<min_power>]`

(Optional) The minimum power in contours.

`[<num_contour>]`

(Optional) The number of contours.

`[<resistance>]`

(Optional) Resistance used to calculate reflection coefficients.

`[<close>]`

(Optional) The contours are closed contours (ON or OFF).

Feedback

`[<modifier>]`

(Optional) Modifier function apply to power.

- mag
- dbm
- db10
- db20

---

create_bus

Description

Creates a bus signal with given name and up to 31 digital signals.

Syntax

```
create_bus(<name>,signal1,signal2,[signal3,...,signal31])
```

Returns

A bus signal.

Arguments

`<name>`

A string to be used as a result signal's name.

`signal1`

First digital signal.

`signal2`

Second digital signal.

`[signal3,...,signal31]`

Optional digital signals.

Examples

```
create_bus(tst,'CLK','EZ','DED')
```

will create a 3 bits bus signal from inputs signals CLK', 'EZ', and, 'DED'. The result signal name is tst.

Feedback

create_eye

Creates an eye object which can be plotted as an eye diagram.

Syntax

```
create_eye(<signal>,[<trigger_signal>],[eye_width],
[<user_width/data_rate>],[<trigger_period>],
[<start_i>],[<stop_i>],[<start_t>],[<stop_t>],[<phase>])
```

Returns

An eye object, which is a waveform object that contains eye parameter information.

Arguments

`<signal>`

A transient periodic signal.

`[<trigger_signal>]`

The external trigger signal.

`[eye_width]`

The eye width type: `auto`, `user`, or `data_rate`.

If `auto` is selected, the eye diagram width is calculated, which is the value to be used for folding the signal. The default is `auto`.

`[<user_width/data rate>]`

The width of the eye diagram, which is a scalar value used to folding the signal. If `user` is selected as the `[eye width]`, then the value input in this field is the eye width. If `data_rate` is selected as the `[eye width]`, then the value input in this field is the data rate value.

`[<trigger_period>]`

The trigger period.

`[<start_i>]`

(Optional) The starting edge, which is an index-based filter.

`[<stop_i>]`

(Optional) The last edge, which is an index-based filter.

`[<start_t>]`

(Optional) The starting time, which is a time-based filter.

Feedback

`[<stop_t>]`

(Optional) The ending time, which is a time-based filter.

`[<phase>]`

(Optional) The eye shift. Setting as `auto` centers the eye opening. .

Errors

Possible error messages are:

- Signal not found
- User width must be specified if not auto width
- Stop_i must be greater than start_i
- Stop_t must be greater than start_t
- Invalid user input shift value

---

crossings

Calculates the crossing points between `signal1` and `signal2` or a given `ylevel` value.

Syntax

```
crossings(<signal1>,<signal2/ylevel>,[<edge_type>],[<output_yAxisType>],[<output_xAxisType>])
```

Returns

A waveform

Arguments

`<signal1>`

The first signal to be measured.

`<signal2/ylevel>`

The second signal to be measured. The value can also be a specified threshold.

`<edge_type>`

(Optional) The edge type. Valid values are `either`, `rise`, and `fall`. Default is `either`.

`<output_yAxisType>`

(Optional) The output signal yAxisType. Can be either signal1's Y axis or signal1's X axis. Default is the Y axis.

Feedback

`<output_xAxisType>`

(Optional) The output signal xAxisType. Can be either signal1's X axis or index. Default is the X axis.

Errors

Possible error messages include:

- Signal not found

Examples

```
crossings('0|v(out)','0|v(out)');
crossings('0|v(out)',3)
```

---

d2a

Converts the input signal from digital to analog.

Syntax

```
d2a(<signal>,<low_bounds>,<high_bounds>,
 [<min_glitch_span>],[<time_step>],[<x_start>],
 [<x_stop>],[<x_value>],[<bus_output_type>],[<display>])
```

Returns

A waveform.

Arguments

`<signal>`

An analog signal.

`<low_bounds>`

The low threshold of y.

`<high_bounds>`

The high threshold of y.

`[<min_glitch_span>]`

(Optional) The minimum glitch span to be set.

`[<time_step>]`

(Optional) The value of each `time_step`.

`[<x_start>]`

(Optional) The starting point of x.

Feedback

`[<x_stop>]`

(Optional) The ending point of x.

`[<x_value>]`

(Optional) Determines state x of the digital waveform. This value can be a expression. For example, vl and vh can represent low_bounds and high_bounds.

`[<bus_output_type>]`

(Optional) Specifies output as an analog waveform or a list waveforms.

`[<display>]`

(Optional) Specifies whether display is pwl or pwc. Default is pwc.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Y value out of range
- X value out of range

Examples

```
d2a('0|v(out)',0,1);
d2a('0|v(out)',0,1,5,2n,0,10n)
```

---

dBc2V2

Description

Converts phase noise signal from dBc/Hz to V^2/Hz.

This only works for waveform files that contain carrier magnitude information. Otherwise, it will report: `Carrier magnitude is not defined`

Syntax

```
dBc2V2(<signal>)
```

Returns

An analog signal.

Feedback

Arguments

`<signal>`

The signal to be converted.

Examples

```
dBc2V2('PAC_L(f)')
```

---

dBCPVRICurves

Description

Creates a signal associated with an nth compression point measurement.

This function extracts the specified harmonic from the input signals, calculates a power waveform through the `spectralPower` function for given voltage signal and current signal or resistance.

This function then uses the power waveform to extrapolate a constant slope line.

The result signal is a multiple traces signal containing power waveform and the extrapolation line.

Syntax

```
dBCPVRICurves(<voltage_signal>,<harmonic_index>,[<current_signal>],
 [<resistance>],[<gainDecrease>],[<extrapolationPoint>],[<sweepUnits>],
 [<format>])
```

Returns

A multiple traces signal.

The first trace is the power or gain curve. The second trace is the extrapolation line.

Arguments

`<voltage_signal>`

The input voltage signal, which must be a multiple traces spectrum waveform.

`<harmonic_index>`

Harmonic index.

`<current_signal>`

(Optional) The input current signal, which must be a multiple traces spectrum waveform.

Feedback

`<resistance>`

(Optional) Resistance into the output port. Default value is 50 Ohm.

`<gainDecrease>`

(Optional) The delta between power waveform and ideal gain line.

`<extrapolationPoint>`

(Optional) The X value (in dBm) indicating the power waveform point that constant slope extrapolation line crosses. Default is the first point of the power waveform.

`<sweepUnits>`

(Optional) Can be `dBm`, `W`, or `none`. Default is `dBm`.

`<format>`

(Optional) Can be `power` or `gain`. Default is `power`.

---

delayvst

Measures the time difference versus the time between two signals at the specified switching edges from `x_start` to `x_stop`.

Syntax

```
delayvst(<signal>, <reference_signal>, <signal_y_level>,
<reference_signal_y_level>, [<percent>],[<trigger_type>],
[<reference_trigger_type>],[<x1_start>],[<x1_stop>],
[x2_start],[<x2_stop>], [<premise>]
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<reference_signal>`

The reference signal.

`<signal_y_level>`

Threshold level for trigger signal.

`<reference_signal y_level>`

Threshold level for reference signal.

Feedback

`[<percent>]`

(Optional) The value can be `1` or `0`. When specified as `1`, the y_threshold is a percentage value; when specified as `0`, the threshold is not a percentage value.

`[<trigger_type>]`

(Optional) The `trigger_type` of measured signal, which can be `fall`, `rise`, or `either`. The default value is `either`.

`[<reference_trigger_type>]`

(Optional) The trigger_type of reference signal, which can be fall, rise, same, opposite, and either. Default is either.

`[<x1_start>]`

The starting value of x for trigger signal.

`[<x1_stop>]`

(Optional) The stopping value of x for trigger signal.

`[<x2_start>]`

(Optional) The starting value of x for reference signal.

`[<x2_stop>]`

(Optional) The stopping value of x for reference signal.

`[<premise>]`

Specifies the delay value as `positive`, `negative`, or `either`.

Errors

Possible error messages include:

- Signal not found
- Y value out of range

Examples

```
delayvst('0|v(out)','0|v(out)',4,3);
delayvst('0|v(out)','0|v(out)',4,3,0,2,4,10n,'fall',
 reftrig='same',x1start=1n,x1stop=4n,x2start=2n,x2stop=5n)
```

derivative

Calculates the derivative waveform of given signal.

Feedback

Syntax

```
derivative(<signal>, [<display>], [<type>])
```

Returns

A waveform.

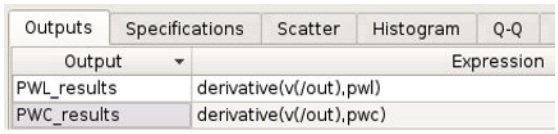
Arguments

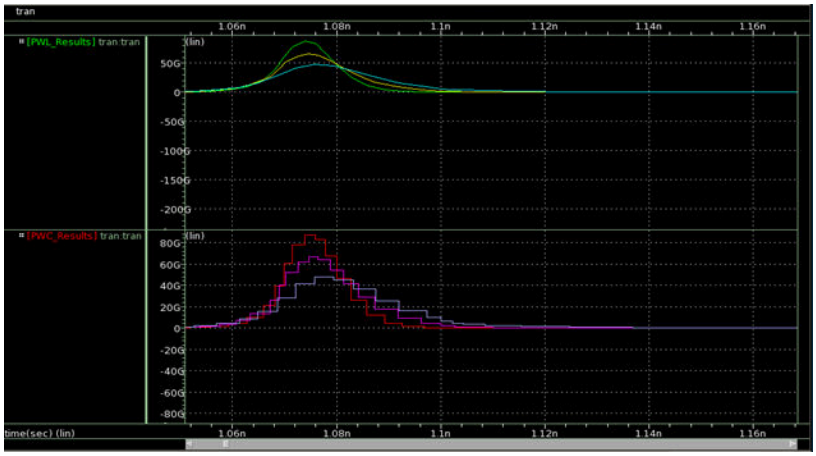
`<signal>`

The signal to be measured.

`[<display>]`

(Optional) Displays the resulting waveform in either PWC or PWL.





`[<type>]`

(Optional) The values are tangent (the default) or secant. The tangent value has the same behavior as the current derivative function. When set to the secant value, the value for a point is the slope of line after that point. See the examples for more information.

Errors

Possible error messages include:

- Signal not found

Feedback

Examples

```
derivative('0|v(out)')
```

---

dft

Measures the Discrete-Fourier-Transform (DFT) of a given signal. The signal is evenly Piece-Wise-Constant sampled from x_start to x_stop for n points.

Syntax

```
dft(<signal>,<x_start>,<x_stop>,<num_of_points>,
[<windowing_type>],[<window_alpha>],
[<normalization>],[<user_defined_value>],
[<suppress_DC>], [<dual_sideband>],
[<coherent_gain>],[<coherent_gain_factor>],
[<ADC_span>],[<window_sampling>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting point of x.

`<x_stop>`

The ending point of x.

`<num_of_points>`

The number of points needing to be measured.

`[<windowing_type>]`

(Optional) Specifies the windowing function. The default is a rectangle.

`[<windowing_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

Feedback

`[<normalization>]`

(Optional) There are three options for this argument:

- none — FFT results is not normalized.
- normalized — FFT results is normalized against the maximum magnitude value of the waveform.
- user-defined — FFT results is normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if [<normalization>] is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

`[<dual_sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<coherent_gain>]`

(Optional) This argument specifies coherent gain factor type (none, default, magnitude, db20, db10). The coherent gain factor value is used to scale the power spectral density results.

- none: no scaling the results
- default: scaling the results by 1, if the window type is a rectangle. Else, scaling the results to the signal's amplitude.
- magnitude: scaling with coherent_gain_factor
- dB20: scaling with pow(10,coherent_gain_fator/20)
- dB10: scaling with pow(10,coherent_gain_factor/10)

`[<coherent_gain_factor>]`

(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by 1/(Coherent Gain). Valid values are 0 < 1. Default is 0. You can use 1 if you do not want this scaling factor to be used.

Feedback

`[<ADC_span>]`

(Optional) Specifies the peak saturation level of the FFT waveform. When specified the magnitude of the input waveform is divided by ADC span value before computing FFT. This is full-scale span, ignoring any DC offsets. Valid values are any floating point number. Default value is 1.0.

`[<window_sampling>]`

(Optional) Determines whether the window samples are computed in a periodic or a symmetric manner. For example, when set to periodic, the window will be extended by 1 sample. The default is symmetric.

Errors

Possible error messages include:

- Signal not found
- Y value out of range
- Number of sample points is non-positive
- X value out of range

Examples

```
dft('0|v(out)',2n,4n,2);
dft('0|v(out)',2n,4n,2,'rectangle');
```

---

dftbb

Description

Computes the fast Fourier transform (FFT) of a complex signal $Z(n) = ReZ(n) + j \cdot ImZ(n)$ by reading in two signals, sampling evenly from `x start` to `x stop` for *n* points, and performs FFT separately.

Syntax

```
dftbb(<signal_1>,<signal_2>,<x_start>,<x_stop>,
<number_of_points>,[<windowing_type>],
[<window_alpha>],[<normalization>],
[<user_defined_value>],[<suppress DC>],[<dual_sideband>],
[<coherent gain>],[<coherent_gain_factor>],[<ADC_span>],
[<window_sampling>])
```

Returns

A waveform which has *N* complex values.

Feedback

Arguments

`<signal_1>`

The first signal.

`<signal_2>`

The second signal.

`<x_start>`

The starting point of x.

`<x_stop>`

The ending point of x.

`<number_of_points>`

The number of points needing to be measured.

`[<windowing_type>]`

(Optional) Specifies the windowing function (default is rectangle).

`[<windowing_alpha>]`

(Optional) Windowing alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

`[<normalization>]`

(Optional) There are three options for this argument:

- none — FFT results is not normalized.
- normalized — FFT results is normalized against the maximum magnitude value of the waveform.
- user-defined — FFT results is normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if [<normalization>] is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

Feedback

`[<dual_sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<coherent_gain>]`

(Optional) This argument specifies coherent gain factor type (none, default, magnitude, db20, db10). The coherent gain factor value is used to scale the power spectral density results.

- none: no scaling the results
- default: scaling the results by 1 if the window type is a rectangle. Else, scaling the results to the signal's amplitude.
- magnitude: scaling with coherent_gain_factor
- dB20: scaling with pow(10,coherent_gain_fator/20)
- dB10: scaling with pow(10,coherent_gain_factor/10)

`[<coherent_gain_factor>]`

(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by 1/(Coherent Gain). Valid values are 0 < 1. Default is 0. You can use 1 if you do not want this scaling factor to be used.

`[<ADC_span>]`

(Optional) Specifies the peak saturation level of the FFT waveform. When specified the magnitude of the input waveform is divided by ADC span value before computing FFT. This is full-scale span, ignoring any DC offsets. Valid values are any floating point number. Default value is 1.0.

`[<window_sampling>]`

(Optional) Determines whether the window samples are computed in a periodic or a symmetric manner. For example, when set to periodic, the window will be extended by 1 sample. The default is symmetric.

Examples

$$I(n) = ReI(n) + j \cdot ImI(n)$$

$$Q(n) = ReQ(n) + j \cdot ImQ(n)$$

and then computes:

$$ReZ(n) = ReI(n) \text{-} ImQ(n)$$

Feedback

$$ImZ(n) = ImI(n) + ReQ(n)$$

---

dnl

Description

Computes the differential non-linearity of a transient waveform. DNL is calculated as:

$$DNL(D+1)\Big|_{LSB} = \frac{V(D+1)\text{-}V(D)}{V_{LSB}}\text{-}1$$

where 0 < D < 2^N-2.

Syntax

```
dnl(<signal>,<sampling_signal|list|step>,[<cross_type>],
[<mode>],[<threshold>],[<delay>],[<method>],[<unit>],
[<no_of_samples>])
```

Returns

An analog signal.

Arguments

`<signal>`

The signal to be measured.

`<sampling_signal|list|step>`

This argument specifies three ways to sampling the waveform:

- sampling_signal: the measured the signal is sampled according to the input sampling signal. The sample points is obtained at which the waveform crosses the threshold (specified with [<mode>] and [<threshold>] options) at either rising or falling edge (specified with [<cross_type>] option). The sampling can start with a delay (specified with [<delay>] option).
- list: list of x values at which the sample points are obtained for the measured signal.
- step: the interval value specified for sampling the measured signal.

`[<cross_type>]`

(Optional) Specifies the edge types at which the sampling signal intersect with the threshold. There are two options: rising or falling.

Feedback

`[<mode>]`

(Optional) Specifies the mode used to calculate the threshold value. There are two options:

- user: Specify the threshold value with [<threshold>] option.
- auto: the threshold is calculated automatically by (integral_of_waveform / x_range).

`[<threshold>]`

(Optional) Specifies the threshold value used to sample the data. Only specified if `mode` is `user`.

`[<delay>]`

(Optional) Specifies time after which the sampling begins.

`[<method>]`

(Optional) Specifies whether the end-to-end (end) or straight line (fit) method is used.

`[<unit>]`

(Optional) Specifies whether the output waveform is to be output as an absolute value (abs) or multiples of least significant bit (lsb).

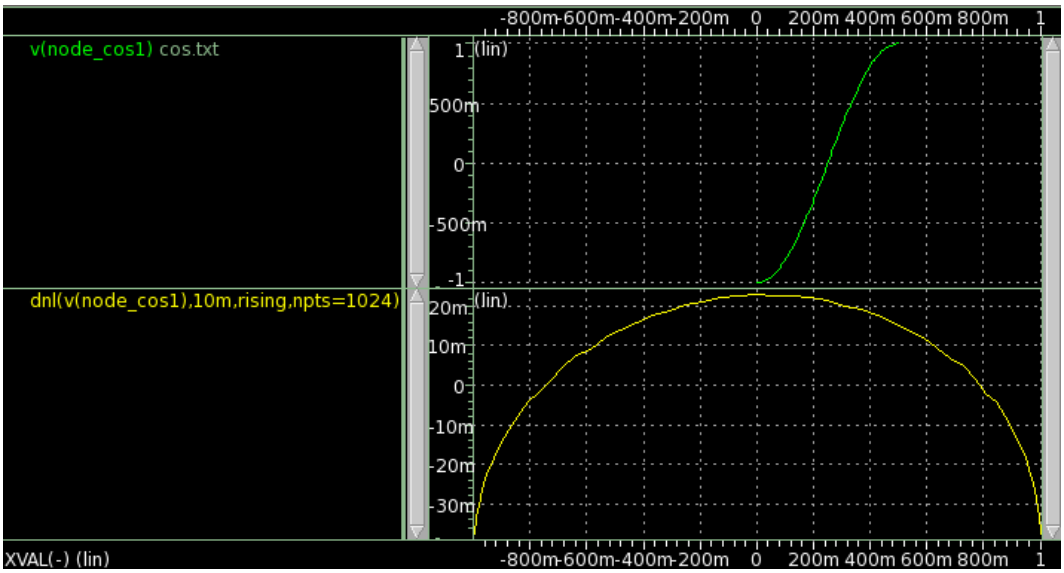
`[<no_of_samples>]`

(Optional) Specifies the samples used for calculating the non-linearity. If not specified, the samples are taken against the entire data window.

Feedback

Examples

The following example shows how the function works on the input waveform “v(node_cos1)”:



The example inputs of the arguments show as follows:

- signal: v(node_cos1)
- step: 10m
- npts: 1024

The function expression shows as follows:

```
dnl('v(node_cos1)',10m,'rising',npts=1024)
```

---

dumpPWLF

Saves the signal data to a text file for PWLF sources.

Syntax

```
dumpPWLF(<signal>,<fileName>,[<precision>],[<start>],[<stop>],[<step>])
```

Returns

`1` on success and `0` on failure.

Feedback

Arguments

`<signal>`

The signal to be saved to a text file.

`<fileName>`

The name of the output text file.

`[<precision>]`

(optional) The number of digits, with a default of 10.

`[<start>]`

(Optional) The starting point on the x-axis, Defaults to the initial x-axis value of the saved signal.

`[<stop>]`

(Optional) The endpoint on the x-axis. Defaults to the final x-axis value of the saved signal.

`[<step>]`

(Optional) The step size along the x-axis. Defaults to an empty value (`" "`).

Examples

The example inputs for the arguments are as follows:

- signal: `v(/node1)`
- file name: `outFile`
- Precision: `12`

The function expression is shown as follows:

```
dumpPWLF(v(/node1),outFile,12)
```

---

duty_vs_t

Measures the duty cycle versus time. The duty cycle is the percentage of one period in which a signal is active. A period is the total time for a signal to complete an on-and-off cycle. Duty cycle can be expressed as where T is the active time signal and P is the total period of the signal:$D = \frac{T}{P} \times 100$

Syntax

```
duty_vs_t(<signal>,<y_threshold>,<rise_edge>,[<x_start>],
 [<x_stop>],[<percent>],[<output_xAxisType>],[<time_value>])
```

Feedback

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<y_threshold>`

The threshold value of y.

`<rise_edge>`

This has two values 0 and 1 (0='fall', 1='rise').

`[<x_start>]`

(Optional) The starting point of x.

`[<x_stop>]`

(Optional) The ending point of x.

`[<percent>]`

(Optional) The value can be `1` or `0`. When the value is `1`, the `y_threshold` is a percentage value. When the value is `0`, the `y_threshold` is not a percentage value.

`[<output_xAxisType>]`

(Optional) Specifies the X axis type of the result waveform. The value can be time or idx.

`[<time_value>]`

(Optional) Specifies the time value, which is the cycle begin value or cycle end value. Default is cycle_end.

Errors

Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

Examples

```
duty_vs_t('0|v(out)',3,1);
duty_vs_t('0|v(out)',3,1,2n,4n,1);
```

Feedback

fallvst

Measures the time difference versus the time of the given signal starting from high `y_level` to low `y_level` within the range of `x_start` and `x_stop`.

Syntax

```
fallvst(<signal>,<high_y_level>,<low_y_level>,[<percent>],
 [<x_start>],[<x_stop>])
```

Arguments

`<signal>`

The signal to be measured.

`<high_y_level>`

The high threshold value of y

`<low_y_level>`

The low threshold value of y.

`[<percent>]`

(Optional) The value can be `1` or `0`. When the value is `1`, the y_threshold is a percentage value. If the value is `0`, the y_threshold is not a percentage value.

`[<x_start>]`

(Optional) The starting point of x.

`[<x_stop>]`

(Optional) The ending point of x.

Errors

Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

Examples

```
fallvst('0|v(out)',3,1);
fallvst('0|v(out)',3,1,0,2n,8n);
```

Feedback

fft

Measures the Fast-Fourier-Transform (FFT) of a given signal. The signal is evenly Piece-Wise-Constant sampled from x_start to x_stop for n points.

Syntax

```
fft(<signal>,<x_start>,<x_stop>,<num_of_points>,
[<windowing_type>],[<window_alpha>],
[<normalization>],[<user_defined_value>],
[<suppress_DC>], [<dual_sideband>],
[<coherent_gain>],[<coherent_gain_factor>],
[<ADC_span>],[<window_sampling>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The starting point of x.

`<x_stop>`

The ending point of x.

`<num_of_points>`

The number of points needing to be measured. Must be a power of 2.

`[<windowing_type>]`

(Optional) Specifies the windowing function. The default is a rectangle.

`[<window_alpha>]`

(Optional) Window alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

Feedback

`[<normalization>]`

(Optional) There are three options for this argument:

- none — FFT results are not normalized.
- normalized — FFT results are normalized against the maximum magnitude value of the waveform.
- user-defined — FFT results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if `normalization` is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

`[<dual_sideband>]`

(Optional) If this argument is set to false, the resultant waveform is only on one side of the Y-axis starting from 0 to N-1. When set to true, the resultant waveform is symmetric to the Y-axis from -N/2 to N/2.

`[<coherent_gain>]`

(Optional) This argument specifies coherent gain factor type (none, default, magnitude, db20, db10). The coherent gain factor value is used to scale the power spectral density results.

- none: no scaling the results
- default: scaling the results by 1 if the window type is a rectangle. Else, scaling the results to the signal's amplitude.
- magnitude: scaling with coherent_gain_factor
- dB20: scaling with pow(10, coherent_gain_factor/20)
- dB10: scaling with pow(10, coherent_gain_factor/10)

`[<coherent_gain_factor>]`

(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by 1/(Coherent Gain). Valid values are 0 < 1. Default is 0. You can use 1 if you do not want this scaling factor to be used.

Feedback

`[<ADC_span>]`

(Optional) Specifies the peak saturation level of the FFT waveform. When specified the magnitude of the input waveform is divided by ADC span value before computing FFT. This is full-scale span, ignoring any DC offsets. Valid values are any floating point number. Default value is 1.0.

`[<window_sampling>]`

(Optional) Determines whether the window samples are computed in a periodic or a symmetric manner. For example, when set to periodic, the window will be extended by 1 sample. Default is symmetric.

Errors

Possible error messages include:

- Signal not found
- Y value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2
- X value out of range

Examples

```
fft('0|v(out)',2n,4n,2);
fft('0|v(out)',2n,4n,2,'rectangle');
```

---

flip

Description

Reverses the signal along the Y-axis.

Syntax

`flip(<signal>)`

Returns

An analog signal.

Arguments

`<signal>`

The signal to be reversed.

Feedback

fourEval

Description

This function is an inverse Fourier transformation. It transforms the specified waveform from the frequency domain to the time domain.

Syntax

```
fourEval(<signal>,<x_start>,<x_stop>,<time_step>,
[<baseband>])
```

Returns

An analog signal.

Arguments

`<signal>`

The signal to be calculated.

`<x_start>`

Specifies time at which to begin evaluating the series.

`<x_stop>`

Specifies time at which to stop evaluating the series.

`<time_step>`

Specifies time increment.

`[<baseband>]`

(Optional) Evaluates the baseband version of the inverse of the DFT function.

func

Plots the waveform of `signal1` versus `signal2` for the two specified signals.

Syntax

```
func(<signal1>,<signal2>)
```

Returns

A waveform.

Feedback

Arguments

`<signal1>`

The first signal to be measured.

`<signal2>`

The second signal to be measured.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
func('0|v(out)','0|v(out)')
```

---

fvst

Calculates the frequency versus time for the given signal.

Syntax

```
fvst(<signal>,[<y_threshold>],<x_start>,<x_stop>,
[<percent>],[<edge_type>],
[output_xaxistype>],[<time_value>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`[<y_threshold>]`

(Optional) The threshold value of y. If not specified, the default value will be the integral of the waveform divided by the X range.

`<x_start>`

The starting value of x.

`<x_stop>`

The ending value of x.

Feedback

`[<percent>]`

(Optional). The value can be `1` or `0`. When the value is `1`, the `y_threshold` is a percentage value. If the value is `0`, the `y_threshold` is not a percentage value.

`[<edge_type>]`

(Optional) Specifies the trigger type (fall, rise, or either). The fall argument specifies the wave crossing from above the y_level to below the y_threshold.

`[<output_xaxistype>]`

(Optional) Specifies the X axis type of the result waveform. The value can be time or idx. Default is time.

`[<time_value>]`

Specifies whether X values in result waveform are equal to the start time or end time of each period.

Errors

Possible error messages include:

- Signal not found
- Y value out of range
- X value out of range

Examples

```
fvst('0|v(out)',2,2n,6n,1)
```

---

gdelay

This is an AC domain function which measures the group delay waveform of a given signal. The group delay can be defined as where is the phase and is the angular frequency. $\frac{d\varphi}{d\omega}$ $\frac{d\varphi}{d\omega}$ ω $(2\pi f)$

Syntax

```
gdelay(<signal>)
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

Feedback

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
gdelay('0|v(out)')
```

---

histo

Generates a histogram waveform for the input signal.

Syntax

```
histo(<signal>,[<nbins>],[<min>],[<max>],[<type>],
 [<annotations>],[<density_estimator>],[<bandwidth>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`[<nbins>]`

(Optional) The number of bins in the histogram.

`[<min>]`

(Optional) The minimum threshold value.

`[<max>]`

(Optional) The maximum threshold value.

`[<type>]`

(Optional) Specifies the type of histogram:

- standard
- cumulative line — connects center of cumulative boxes.
- cumulative box

Feedback

`[<annotations>]`

(Optional) Specifies cursor annotation type:

- none
- std dev lines — show where are average + (-3, -2, -1, 0, 1, 2, 3) times of standard deviation.
- % markers — show where are (0.3%, 4.5%, 32%, 50%, 68%, 95.5%, 99.7%) of the data.
- std dev lines and % markers

`[<density_estimator>]`

(Optional) Shows density estimation curve (on or off).

`[<bandwidth>]`

(Optional) Controls the bandwidth used in density estimation.

Errors

Possible error messages include:

- Signal not found
- Nbins must be greater than 0
- High threshold must be greater than low threshold
- It should be in range(0, Inf)

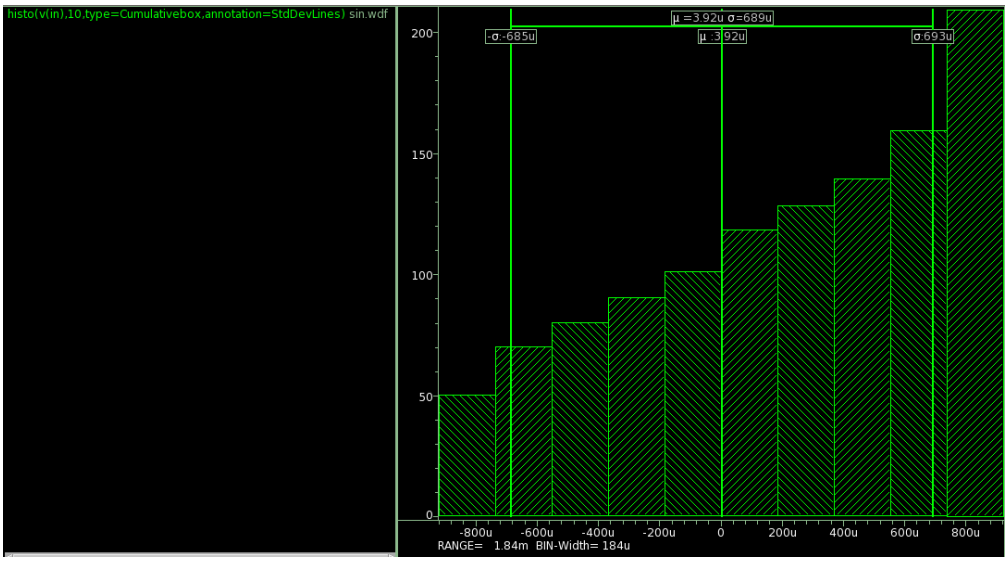
Examples

Example 1:

```
histo(v(in),10,type=Cumulativebox,annotation=StdDevLines)
```

Feedback

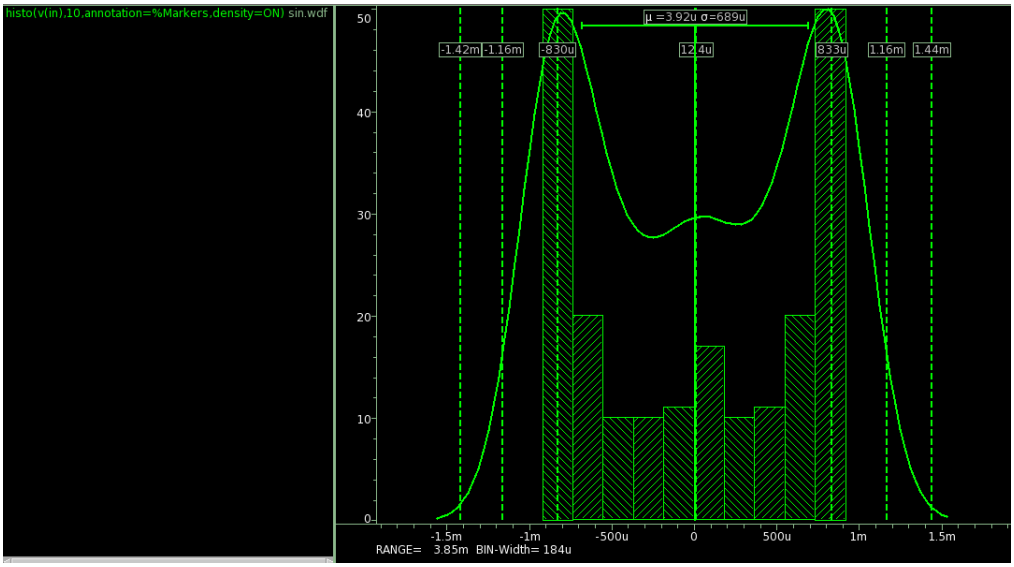
will appear as follows:



Example 2:

```
histo(v(in),10,annotation=%Markers,density=ON)
```

will appear as follows:



hold_times

Plots a waveform that contains all of the hold time data points. The x values in the waveform indicate the time at which the hold times occur.

Feedback

Syntax

```
hold_times(<signal>,<clock_signal>,<clock_period>,
<y_threshold>,<signal_edge>,<clock_edge>,[<x_start>],
[<x_stop>],[<percent>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured (logic/digital signals or complex data is not supported).

`<clock_signal>`

The reference clock signal.

`<clock_period>`

The period of the reference `clock_signal`.

If the input value for the `clock_signal` period is `0` or smaller than `0`, the `clock_period` is calculated using the `clock_signal`.

`<y_threshold>`

The threshold value of y.

`<signal_edge>`

The type of measured `signal_edge`. The valid `signal_edge` can be `rise`, `fall`, or `both`. The default `signal_edge` is `rise`.

`<clock_edge>`

The type of reference `clock_signal` edge. The valid `signal_edge` can be `rise`, `fall`, or `both`. The default `signal_edge` is `rise`.

`[<x_start>]`

(Optional) The starting value of x.

`[<x_stop>]`

(Optional) The ending value of x.

`[<percent>]`

(Optional) The value can be `1` or `0`. When the value is `1`, the `y_threshold` is a percentage value. When the value is `0`, the `y_threshold` is not a percentage value.

Feedback

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Y value out of range

Examples

```
hold_times('0|v(out)','0|v(out)'4n,4,'rise','rise')
hold_times('0|v(out)','0|v(out)'4n,4,'rise','rise',1n,4n,1)
```

---

hold_times2

Like `hold_times`, plots a waveform containing all of the hold time data points. The x values in the waveform indicate the time at which hold times occur. `hold_times2` differs from `hold_times` in that it gives you the ability to specify a different threshold for the clock signal as well as the measured signal, whereas `hold_times` allows selecting the threshold of only the measured signal.

Syntax

```
hold_times2(<signal>,<clock_signal>,<clock_period>,
 <data_threshold>,<clock_threshold>,<signal_edge>,
 <clock_edge>,[<x_start>],[<x_stop>],[<percent>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<clock_signal>`

The reference clock signal.

`<clock_period>`

The period of the reference `clock_signal`.

If the input value for the `clock_signal` period is `0` or smaller than `0`, the `clock_period` is calculated using the `clock_signal`.

`<data_threshold>`

The threshold value of y for the measured signal.

Feedback

`<clock_threshold>`

The threshold value of y for the `clock_signal`.

`<signal_edge>`

The type of measured `signal_edge`. The valid `signal_edge` can be `rise`, `fall`, or `both`. The default `signal_edge` is `rise`.

`<clock_edge>`

The type of reference `clock_signal` edge. The valid `signal_edge` can be `rise`, `fall`, or `both`. The default `signal_edge` is `rise`.

`[<x_start>]`

(Optional) The starting time of measurement.

`[<x_stop>]`

(Optional) The ending time of measurement.

`[<percent>]`

(Optional) The value can be `1` or `0`. When the value is `1`, the `y_threshold` is a percentage value. When the value is `0`, the `y_threshold` is not a percentage value.

Description

Possible error messages include:

- Signal not found
- Incorrect signal type
- Y value out of range

Examples

```
hold_times2('0|v(out)','0|v(out)',4n,2,3,'rise','rise' )
hold_times2('0|v(out)','0|v(out)',4n,2,3,'rise', 'rise',2n,6n,1)
```

---

ift

Calculates the Inverse Fourier transform (IFT) time domain waveform of the input frequency domain signal.

Syntax

```
ift(<signal>,<fundamental_freq>,<max_harmonic>)
```

Feedback

Returns

A waveform.

Arguments

`<signal>`

The input signal.

`<fundamental_freq>`

The fundamental frequency in the frequency domain spectrum.

`<max_harmonic>`

The maximum harmonic index to be included in the IFT. For DC, the value is `0`.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Number of harmonics is negative

Examples

```
ift('0|v(out)',50,4)
```

---

inl

Description

Computes the integral non-linearity of a transient simple or parametric waveform. INL is calculated as:

$$INL(D)\big|_{LSB} = \frac{V(D)\text{-}V(0)}{V_{LSB}}\text{-}D$$

where $0 < D < 2^N\text{-}1$.

Syntax

```
inl(<signal>,<sampling_signal|list|step>,[<cross_type>],
[<mode>],[<threshold>],[<delay>],[<method>],[<unit>],
[<no_of_samples>])
```

Returns

An analog signal.

Feedback

Arguments

`<signal>`

The signal to be measured.

`<sampling_signal/list/step>`

This argument specifies three ways to sampling the waveform:

- sampling_signal: the measured the signal is sampled according to the input sampling signal. The sample points is obtained at which the waveform crosses the threshold (specified with [<mode>] and [<threshold>] options) at either rising or falling edge (specified with [<cross_type>] option). The sampling can start with a delay (specified with [<delay>] option).
- list: list of x values at which the sample points are obtained for the measured signal.
- step: the interval value specified for sampling the measured signal.

`[<cross_type>]`

(Optional) Specifies the edge types at which the sampling signal intersect with the threshold. There are two options: rising or falling.

`[<mode>]`

(Optional) Specifies the mode used to calculate the threshold value. There are two options:

- user: Specify the threshold value with [<threshold>] option.
- auto: the threshold is calculated automatically by (integral_of_waveform / x_range).

`[<threshold>]`

(Optional) Specifies the threshold value used to sample the data. Only specified if `mode` is `user`.

`[<delay>]`

(Optional) Specifies time after which the sampling begins.

`[<method>]`

(Optional) Specifies whether the end-to-end (end) or straight line (fit) method is used.

`[<unit>]`

(Optional) Specifies whether the output waveform is to be output as an absolute value (abs) or multiples of least significant bit (lsb).

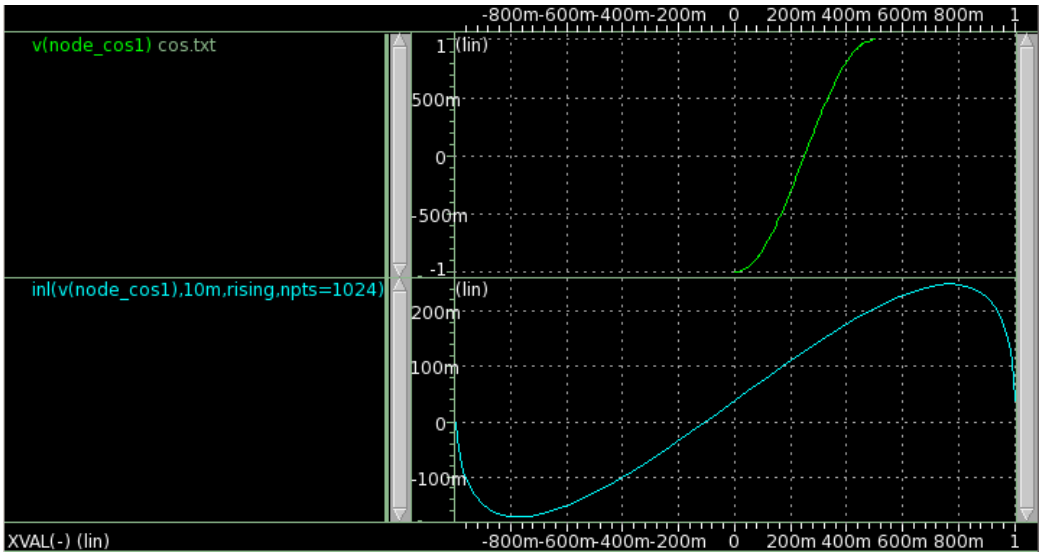
Feedback

`[<no_of_samples>]`

(Optional) Specifies the samples used for calculating the non-linearity. If not specified, the samples are taken against the entire data window.

Examples

The following example shows how the function works on the input waveform “v(node_cos1)”:



The example inputs of the arguments show as follows:

- signal: v(node_cos1)
- step: 10m
- npts: 1024

The function expression shows as follows:

```
inl('v(node_cos1)',10m,'rising',npts=1024)
```

---

integral

Measures the running integral waveform of the input signal.

Syntax

`integral(<signal>)`

Returns

A waveform

Feedback

Arguments

`<signal>`

The input signal.

Errors

Possible error messages include:

- Signal not found

Examples

```
integral('0|v(out)')
```

---

ipnhVRICurves

Description

N dB intercept point VRI Curves.

Syntax

```
ipnhVRICurves (voltage signal <string>,
     harmonic indices 1 <string>,
     harmonic indices 2 <string>
     [, current signal]
     [, resistance]
     [, order 1 <integer>]
     [, order 2 <integer>]
     [, extrapolationPoint 1 <string>]
     [, extrapolationPoint 2 <string>]
     [, sweepUnits <enum>])
```

Arguments

`voltage signal`

Input voltage signal.

`harmonic indices 1`

The first indices, space delimited.

`harmonic indices 2`

The second indices, space delimited.

`current signal`

(Optional) The input current signal.

Feedback

`resistance`

(Optional) The resistance.

`order 1`

(Optional) `Harmonic indices 1` signal's order. (Default: 1)

`order 2`

(Optional) `Harmonic indices 2` signal's order. (Default: 3)

`extrapolationPoint 1`

(Optional) `Harmonic indices 1` signal extrapolation point. If unspecified, defaults to lowest value.

`extrapolationPoint 2`

(Optional) `Harmonic indices 2`l extrapolation point. If unspecified, defaults to lowest value.

`sweepUnits`

(Optional) The units of the input power sweep.

Valid values: W, dBm, and none (Default: dBm)

---

join

Generates a joined waveform from `signal1` and `signal2`, which depends on the starting x-axis value of the two signals. The waveform that has the greater starting x value is always appended to the one with the smaller x starting value. If `signal1` and `signal2` overlap, the overlapped portion of the latter waveform is removed before joining. For the reversed sweep data whose x starting value >x stopping value from the two signals, `signal1` and `signal2` is reordered into a forward sweep in the joined waveform.

Syntax

```
join(<signal1>,<signal2>)
```

Returns

A waveform.

Arguments

`<signal1>`

The first input signal.

`<signal2>`

The second input signal.

Feedback

Errors

Possible error messages include:

- Signal not found

Examples

```
join('0|v(out)','0|v(out)')
```

---

jvst_freq

Description

Calculates the jitter versus time given the frequency of an ideal clock for the following jitter types:

- N-Cycle and N-Period Jitter

  N-Cycle jitter extends cycle-to-cycle jitter. Instead of measuring the difference between adjacent periods, a multiple grouping of N periods is measured. Where N = 3, the width of three periods is measured. N can be any value from 2 to n where only the available sample population limits n. Once again, after taking a significant sample of periods and measuring the differences in the multiple groupings, the standard deviation and peak values can be resolved. This statistical information is the N-Cycle “jitter.” There are two permutations of the N-Cycle jitter. The first reports statistics from the difference between adjacent groupings (best called N-Cycle jitter). The second reports statistics from the variation of the multiple periods (best called N-Period jitter).

- Pulse Width Jitter

  Measures the pulse differences between the reference signal and the target signal at the reference edge.

- Period Jitter

  Measures the period differences between the reference signal and the target signal at the reference edge.

- Cycle-to-cycle Jitter

  The application of simple arithmetic to the period measurements just taken. If the timing information for the two adjacent periods is known, the difference between them is the cycle-to-cycle change: period one minus period two. Again, after taking a significant sample of periods, and measuring the difference between them, the standard deviation and peak values can be resolved. This statistical information is the cycle-to-cycle “jitter”.

Feedback

Syntax

```
jvst_freq(<signal>,<jitter_type>,<clock_freq>,<y_at_rise>,
<y_at_fall>,<clock_edge>,[<signal_edge>],
[<number_of_periods>],[<search_forward>],
[<search_range>],[<value_sign>],[<percent>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<jitter_type>`

The jitter type can be `trigger`, `width`, `period`, `cycle`, `nperiod`, or `ncycle`.

`<clock_freq>`

Frequency for an ideal reference clock.

`<y_at_rise>`

The input signal threshold level at the rising edge.

`<y_at_fall>`

The input signal threshold level at the falling edge.

`<clock_edge>`

The reference `signal_edge` type. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<signal_edge>]`

The type of input `signal_edge`. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<number_of_periods>]`

(Optional) Only one `nperiod` and `ncycle` is specified for the `jitter_type`. The default is `1` for `n` period.

`[<search_forward>]`

(Optional) Always search forward for the target edge. Values can be `0` or `1`. The default value is `0`.

`[<search_range>]`

(Optional) The maximum target edge search range.

Feedback

`[<value_sign>]`

(Optional) The forward or backward positive value sign. Values can be `0` or `1`. The default value is `0`, which represents the forward positive `value_sign`.

`[<percent>]`

(Optional) The value can be `1` or `0`. When the value is `1`, the `y_threshold` is a percentage value. If the value is `0`, the `y_threshold` is not percentage value.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Invalid input value
- Frequency value can't be 0
- Unknown target edge type, should be same, rise, fall, or cross

Examples

```
jvst_freq('0|v(out)','width',2.5E8,3,2,'rise')
jvst_freq('0|v(out)','width',2.5E8,3,2,'rise','same',nperiod=1,forward=0,
range=10n,sign=0,percent=1)
```

---

jvst_ideal

Calculates the jitter versus time with an ideal clock.

Syntax

```
jvst_ideal(<signal>,<jitter_type>,<clock_delay>,
<clock_period>,<clock_width>,<y_at_rise>,<y_at_fall>,
<clock_edge>[,<signal_edge>][,<num_of_periods>][
<search_forward>][,<search_range>][,<value_sign>][,<percent>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<jitter_type>`

The jitter type can be `trigger`, `width`, `period`, `cycle`, `nperiod`, or `ncycle`.

Feedback

`<clock_delay>`

Delays the time value for an ideal reference clock with a period value.

`<clock_period>`

Period value for reference ideal clock.

`<clock_width>`

Width value for an ideal reference clock. Note that the value of `clock_width` should always be smaller than the value of `clock_period`.

`<y_at_rise>`

The input signal threshold level at the rising edge.

`<y_at_fall>`

The input signal threshold level at the falling edge.

`<clock_edge>`

The reference `signal_edge` type. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<signal_edge>]`

The type of input `signal_edge`. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<number_of_periods>]`

(Optional) Only one `nperiod` and `ncycle` is specified for the `jitter_type`. The default is `1` for `n` period.

`[<search_forward>]`

(Optional) Always search forward for the target edge. Values can be `0` or `1`. The default value is `0`.

`[<search_range>]`

(Optional) Maximum target edge search range.

`[<value_sign>]`

(Optional) Forward or backward positive value sign. The value can be `0` or `1`. The default value is `0`, which represents the forward positive `value_sign`.

`[<percent>]`

(Optional) The value can be `1` or `0`. When the value is `1`, the `y_threshold` is a percentage value. If the value is `0`, the `y_threshold` is not percentage value.

Feedback

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Invalid input value
- Frequency value can’t be 0
- Unknown jitter type, should be trigger, width, period, cycle, nperiod or ncycle
- Unknown ref edge type, should be rise, fall, or both

Examples

```
jvst_ideal('0|v(out)','width',2n,4n,2n,3,2,'rise')
jvst_ideal('0|v(out)','width',2n,4n,2n,'rise','same',nperiod=1,forward=0,
range=8n,sign=0, percent=1)
```

---

jvst_llse

Description

Calculates the jitter versus time with an estimated frequency of the ideal clock using LLSE (Linear Least Squares Estimator).

LLSE creates an X-Y plot, where X is edge number and Y is time of edge occurrence. For the ideal clock, the result is a linear line whose slope equals the period time. For the real clock, the result is a point scattered plot.

To find the closest frequency around which these points are scattered, use LLSE estimator to get the `y=aX+b` formula of this line. This is useful for the PLL jitter calculation, because sometimes the target frequency of the PLL is unknown, and the jitter can be calculated around the optimal frequency.

Syntax

```
jvst_llse(<signal>,<jitter_type>,<y_at_rise>, <y_at_fall>,
<clock_edge>,[<signal_edge>],
[<number_of_periods>],[<search_forward>],
[<search_range>],[<value_sign>],[<percent>])
```

Returns

A waveform.

Feedback

Arguments

`<signal>`

The signal to be measured.

`<jitter_type>`

The jitter type can be `trigger`, `width`, `period`, `cycle`, `nperiod`, or `ncycle`.

`<y_at_rise>`

The input signal threshold level at the rising edge.

`<y_at_fall>`

The input signal threshold level at the falling edge.

`<clock_edge>`

The reference `signal_edge` type. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<signal_edge>]`

(Optional) The type of input `signal_edge`. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<number_of_periods>]`

(Optional) Only one `nperiod` and `ncycle` is specified for the `jitter_type`. The default is `1` for `n` period.

`[<search_forward>]`

(Optional) Always search forward for the target edge. Values can be `0` or `1`. The default value is `0`.

`[<search_range>]`

(Optional) Maximum target edge search range.

`[<value_sign>]`

(Optional) Forward or backward positive value sign. The value can be `0` or `1`. The default value is `0`, which represents the forward positive `value_sign`.

`[<percent>]`

(Optional) The value can be `1` or `0`. When the value is `1`, the `y_threshold` is a percentage value. If the value is `0`, the `y_threshold` is not percentage value.

Feedback

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Invalid input value
- No valid frequency found by Linear Square Estimator
- Unknown jitter type, should be trigger, width, period, cycle, nperiod or ncycle
- Unknown ref edge type, should be rise, fall, or both

Examples

```
jvst_llse('0|v(out)','width',3,2,'rise','same',nperiod=1,
     forward=0,range=8n,sign=0 percent=1)
```

---

jvst_refsig

Calculates the jitter versus time using a reference signal as the clock.

Syntax

```
jvst_refsig(<signal>,<jitter_type>,<clock_signal>,
<y_at_rise>, <y_at_fall>, <clock_edge>, [<signal_edge>], [<clock
 y_at_rise>],[<clock y_at_fall>],
[<number_of_periods>],[<search_forward>],
[<search_range>],[<value_sign>],[<percent>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<jitter_type>`

The jitter type can be `trigger`, `width`, `period`, `cycle`, `nperiod`, or `ncycle`.

`<clock_signal>`

The reference clock signal.

Feedback

`<y_at_rise>`

The input signal threshold level at the rising edge.

`<y_at_fall>`

The input signal threshold level at the falling edge.

`<clock_edge>`

The reference `signal_edge` type. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<signal_edge>]`

The type of input `signal_edge`. The default type is `rise`, however the type can also be `same`, `fall`, or `both`.

`[<clock_at_y_rise>]`

(Optional) The reference signal threshold level at the rising edge.

`[<clock_at_y_fall>]`

(Optional) The reference signal threshold level at the falling age.

`[<number_of_periods>]`

(Optional) Only one `nperiod` and `ncycle` is specified for the `jitter_type`. The default is `1` for `n` period.

`[<search_forward>]`

(Optional) Always search forward for the target edge. Values can be `0` or `1`. The default value is `0`.

`[<search_range>]`

(Optional) Maximum target edge search range.

`[<value_sign>]`

(Optional) Forward or backward positive value sign. The value can be `0` or `1`. The default value is `0`, which represents the forward positive `value_sign`.

`[<percent>]`

(Optional) The value can be `1` or `0`. When the value is `1`, the `y_threshold` is a percentage value. If the value is `0`, the `y_threshold` is not percentage value.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Feedback

- Invalid input signal value
- No valid frequency found by Linear Square Estimator
- Unknown jitter type, should be trigger, width, period, cycle, nperiod or ncycle
- Unknown ref edge type, should be rise, fall, or both
- Unknown target edge type, should be same, rise, fall, or cross

Examples

```
jvst_refsig('0|v(out)','width','0|v(out)',2,3,'fall')
jvst_refsig('0|v(out)','width',2,3,'fall','fall',yrise_ref=1,yfall_ref=4,
nperiod=1,forward=0,range=5n,sign=1,percent=1)

jvst_refsig('0|v(out)','width','0|v(out)',2,3,'fall')
jvst_refsig('0|v(out)','width',2,3,'fall','fall',yrise_ref=1,
     yfall_ref=4,nperiod=1,forward=0,range=5n,sign=1,percent=1)
```

---

loadpull

Generates load pull contour of waveform PSS analysis.

Syntax

```
loadpull(<signal>, [<max_value>], [<min_value>], [<num_contour>],
 [<close>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`[<max_value>]`

(Optional) The maximum threshold value.

`[<min_value>]`

(Optional) The minimum threshold value.

`[<num_contour>]`

(Optional) The number of contours.

`[<close>]`

(Optional) The contours are closed contours (ON or OFF).

Feedback

max_window_average

Maximum Window Average: Calculates the maximum window average using step size between a start and a stop time.

Syntax

```
max_window_average(<signal>
 [<x_start>][<x_stop>][<time_step>][<window_size>]
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`x_start`

(Optional) The start time (`x1`). Default: `xmin`.

`x_stop`

(Optional) The stop time (`x2`). Default: `xmax`.

`time_step`

(Optional) The step size (`step`). Default: (`x_stop-x_start`).

`window_size`

The window size (`size`). Default: `time_step`

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Input signal has no data point

Examples

The following example shows the usage of the arguments for the window average function:

Feedback



median

Measures the smoothed waveform of the given signal using median value method. For every data point in the original signal, the median value method sorts the values of the original data point and w points window before and after the original point. The median value of the sorted result is then used to replace the original data value.

Syntax

```
median(<signal>,<points_window>)
```

Returns

A waveform.

Arguments

`<signal>`

The input signal.

`<points_window>`

The number of points in the window.

Errors

Possible error messages include:

- Signal not found
- Invalid input value

Examples

```
median('0|v(out)',20)
```

Feedback

optical_fft

Converts time-domain optical signal waveforms into their equivalent complex optical spectra. The FFT of the optical signal is calculated over the full range of negative and positive frequencies.

Syntax

```
fft(<optical signal>,<x_start>,<x_stop>,<num_of_points>,
[<windowing_type>],[<window_alpha>],
[<normalization>],[<user_defined_value>],
[<suppress_DC>],[<coherent_gain>],
[<coherent_gain_factor>],[<ADC_span>],
[<window_sampling>],[<output xAxisType>])
```

Returns

A waveform

Arguments

`<signal>`

The optical signal to be measured.

`<x_start>`

The starting point of x.

`<x_stop>`

The ending point of x.

`<num_of_points>`

The number of points needing to be measured. Must be a power of 2.

`[<windowing_type>]`

(Optional) Specifies the windowing function. The default is a rectangle.

`[<window_alpha>]`

(Optional) Window alpha is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 1.0 to 20.0. The default is 3.0.

Feedback

`[<normalization>]`

(Optional) There are three options for this argument:

- none - optical_fft results are not normalized.
- normalized - optical_fft results are normalized against the maximum magnitude value of the waveform.
- user-defined - optical_fft results are normalized against a user-defined magnitude value.

`[<user_defined_value>]`

(Optional) This argument is only applicable if `normalization` is set to user-defined.

`[<suppress_dc>]`

(Optional) If set to true, the DC component is excluded from the target waveforms in the FFT/DFT operation. This is to minimize the error introduced by those signals with large DC component and very small AC magnitudes.

`[<coherent_gain>]`

(Optional) This argument specifies coherent gain factor type (none, default, magnitude, db20, db10). The coherent gain factor value is used to scale the power spectral density results.

- none: no scaling the results
- default: scaling the results by 1 if the window type is a rectangle. Else, scaling the results to the signal's amplitude.
- magnitude: scaling with coherent_gain_factor
- dB20: scaling with pow (10, coherent_gain_factor/20)
- dB10: scaling with pow (10, coherent_gain_factor/10)

`[<coherent_gain_factor>]`

(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by 1/(Coherent Gain). Valid values are 0 < 1. Default is 0. You can use 1 if you do not want this scaling factor to be used.

`[<ADC_span>]`

(Optional) Specifies the peak saturation level of the FFT waveform. When specified the magnitude of the input waveform is divided by ADC span value before computing FFT. This is full-scale span, ignoring any DC offsets. Valid values are any floating point number. Default is 1.0.

Feedback

`[<window_sampling>]`

(Optional) Determines whether the window samples are computed in a periodic or a symmetric manner. For example, when set to periodic, the window will be extended by 1 sample. Default is symmetric.

`[<output xAxis Type>]`

(Optional) Determines the X Axis type of the result spectra, two options are available:

- opticalWavelength: Optical Wavelength, calculated by C/F, where C is the speed of light, F is the optical frequency.
- opticalFrequency: Optical Frequency, calculated by F=f+C/wavelength, where f is the frequency value.

Errors

Possible error messages include:

- Signal not found
- Y value out of range
- Number of sample points is non-positive
- Number of sample points is not power of 2
- X value out of range

---

parametric

Gets the parametric plots of `measure` versus sweeping `param_name`.

Syntax

`parametric(<measure>,<param_name>)`

Returns

A waveform.

Arguments

`<measure>`

The sweeping signal to be measured.

`<param_name>`

The parameter name in the sweeping signal set to be the x component.

Feedback

Errors

Possible error messages include:

- Signal not foundIncorrect signal type

Examples

```
parametric('0|v(out)','T')
```

---

pareto_r

Description

Calculates the Pareto regression result for the s signal and the parameter file. Pareto Analysis is a simple method for separating the major causes (the “vital few”) of a problem, from the minor ones (“trivial many”). Regression analysis is used to examine the relation of the problem (output/dependent variable) to the causes (parameters/independent variables). For the pareto calculation, the 1st order fit calculates the sensitivity values (Main Effect), R**2, and correlation coefficient R by fitting a straight line to the measured output versus each of the parameters. This method is a 1st order polynomial least square fitting. The straight line is: $f(x) = \beta_0 + \beta_1 x$

Where β_1 is the non-normalized sensitivity. $\frac{\sum(xy) - (\sum(x)\sum(y))/n)}{\sum x^2 - \frac{(\sum(x))^2}{n}}$

Adjusted: $R = \frac{\sum(xy) - \sum\frac{(y)}{n}}{\sqrt{\left(\sum(x^2) - \frac{(\sum(x))^2}{n}\right)\left(\sum(y^2) - \frac{(\sum(y))2}{n}\right)}}$ $R^2 = 1 - (1 - R \times R) \times \frac{n-1}{n-2}$

Where:

- x is the vector of parameter values.
- y is the vector of waveform values,
- n is the number of points of the vectors.

Syntax

```
pareto_r(<measure_signal>,<parameter_file>,
[<normalized_by>],[<sorted by>],[<filter_for_s>],
[<filter_for_r2>],[<filter_for_r>],[<histbar_plot>],
[scatter_plot],[<textWindow>],[<text_file>])
```

Feedback

Returns

A waveform value which can have the behavior of a histbar, plot (default), scatter plot, text window, or text file.

Arguments

`<measure_signal>`

The input signal to be measured (sweep signals are not supported).

`<parameter_file>`

The name of the file containing all of the parameters.

`[<normalized_by>]`

(Optional) For non-interactive regressions, the sensitivity values (effects) that the Pareto calculation results in can be normalized by multiplying a normalization factor. When the factor has a negative value, the sensitivity value signs are changed by the normalization. The normalization factor can be `Param/Meas`, `abs(Param/Meas)`, `Param`, `1/Meas`, or un-normalized.

The default normalization factor is `Param/Meas`, where `Param/Meas` is: $\frac{\sum x}{\sum y}$

Where `abs(Param/Meas)` is: $\left|\frac{\sum x}{\sum y}\right|$

Where `Param` is: $\sum x$

Where `1/Meas` is: $\frac{1}{\sum y}$

For example, when calculating an absolute general normalization, the 1st order regression main effect normalized value is:

$$sensitivity = \frac{\sum (xy) - \left(\sum x \times \sum \frac{y}{n}\right)}{\sum x^2 - \left(\frac{(\sum x)^2}{n}\right)} \times \left|\frac{\sum x}{\sum y}\right|$$

`[<sorted_by>]`

(Optional) The sorted values, which can be S, R^2, or R. For this function, you can only choose R.

`[<filter_for_s>]`

(Optional) The minimum value of the sensitivity filter.

Feedback

`[<filter_for _r2>]`

(Optional) The minimum value of the regression square filter.

`[<filter_for_r>]`

(Optional) The minimum value of the regression filter.

`[<histbar_plot>]`

(Optional) Organizes the result output by histogram plot, which can be 0 or 1. The default value is 1.

`[<scatter_plot>]`

(Optional) Organizes the result output by scatter plot, which can be 0 and 1. The default value is 0.

`[<text_window>]`

(Optional) Organizes the result output by text window, which can be 0 and 1. The default value is 0.

`[<text_file>]`

(Optional) Organizes the result output by text file, which can be the text file name or can be missing.

Errors

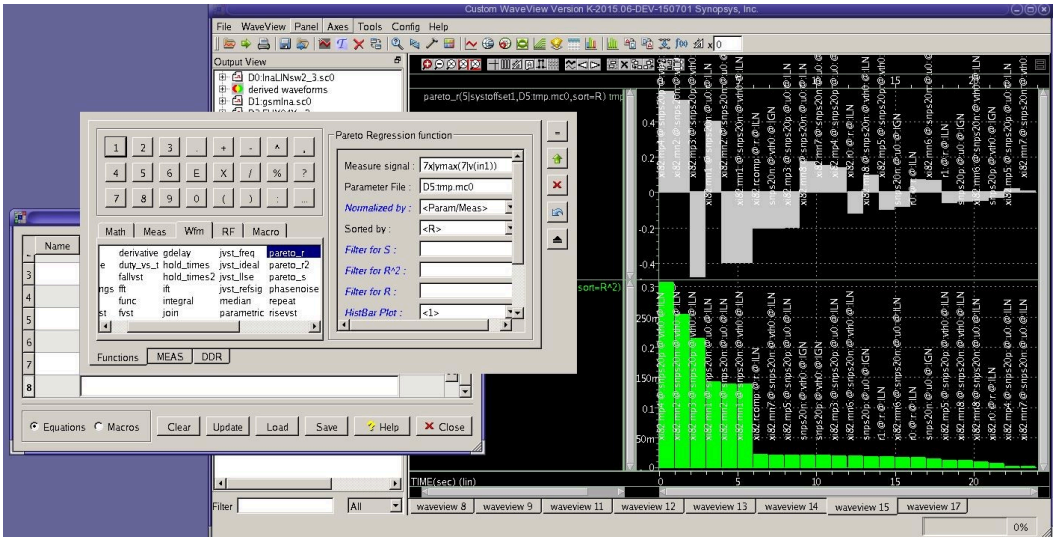
Possible error messages include:

- Signal not found
- Incorrect signal type
- Cannot open waveform file
- Param signal not specified
- Data count are not the same
- Sweep signal is not supported

Examples

```
pareto_r('0|v(out)','D0:tmp.mc0','Param/Meas','r',0,0,0,1,0,0)
pareto_r('0|v(out)','D0:tmp.mc0','Param/Meas','r',0,0,0,1,0,0)
```

Feedback



pareto_r2

Description

Calculates the Pareto regression square result for the s signal and the parameter file. The following equations for Pareto analysis calculations assume:

- x is the vector of parameter values
- y is the number of waveform values
- n is the number of points of the vectors

$$R = \frac{\sum(xy) - \left(\sum(x)\sum\frac{(y)}{n}\right)}{\sqrt{\left(\sum(x^2) - \frac{(\sum(x))^2}{n}\right)\left(\sum(y^2) - \frac{(\sum(y))^2}{n}\right)}} \quad R^2 = 1 - (1 - R \times R) \times \frac{n-1}{n-2}$$

Syntax

```
pareto_r2(<measure_signal>,<parameter_file>,
[<normalized_by>],[<sorted_by>],[<filter_for_s>],
[<filter_for_r2>],[<filter_for_r>],[<histbar_plot>],
[scatter_plot],[<text_window>],[<text_file>])
```

Returns

A waveform which can be a histbar, plot (default), scatter plot, text window, or text file.

Feedback

Arguments

`<measure_signal>`

The input signal to be measured (sweep signals are not supported).

`<parameter_file>`

The name of the file containing all of the parameters.

`[<normalized_by>]`

(Optional) For non-interactive regressions, the sensitivity values (effects) that the Pareto calculation results in can be normalized by multiplying a normalization factor. When the factor has a negative value, the sensitivity value signs are changed by the normalization. The normalization factor can be `Param/Meas`, `abs(Param/Meas)`, `Param`, `1/Meas`, or un-normalized.

The default normalization factor is `Param/Meas`, where `Param/Meas`: $\frac{\sum x}{\sum y}$

Where `abs(Param/Meas)` is: $\left|\frac{\sum x}{\sum y}\right|$

Where `Param` is: $\sum x$

Where `1/Meas` is: $\frac{1}{\sum y}$

For example, when calculating an absolute general normalization, the 1st order regression main effect normalized value is:

$$sensitivity = \frac{\sum (xy) - \left(\sum x \times \sum \frac{y}{n}\right)}{\sum x^2 - \left(\frac{(\sum x)^2}{n}\right)} \times \left|\frac{\sum x}{\sum y}\right|$$

`[<sorted_by>]`

(Optional) The sorted values, which can be S, R^2, or R. You can only choose R^2 for this function.

`[<filter_for_s>]`

(Optional) The minimum value of the sensitivity filter.

`[<filter_for_r2>]`

(Optional) The minimum value of the regression square filter.

`[<filter_for_r>]`

(Optional) The minimum value of the regression filter.

Feedback

`[<histbar_plot>]`

(Optional) Organizes the result output by histogram plot, which can be 0 or 1. The default value is 1.

`[<scatter_plot>]`

(Optional) Organizes the result output by scatter plot, which can be 0 and 1. The default value is 0.

`[<text_window>]`

(Optional) Organizes the result output by text window, which can be 0 and 1. The default value is 0.

`[<text_file>]`

(Optional) Organizes the result output by text file, which can be the text file name or can be missing.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Cannot open waveform file
- Param signal not specified
- Data count are not the same
- Sweep signal is not supported

Examples

```
pareto_r2('0|v(out)','D0:tmp.mc0','Param/Meas''r2')
pareto_r2('0|v(out)','D0:tmp.mc0','Param/Meas','r2',0,0,0,1,0,0)
```

---

pareto_s

Description

Calculates the Pareto sensitivity result for the s signal and the parameter file. The sensitivity values can be normalized by the multiplying factor. When the factor has a

Feedback

negative value, the signs of sensitivity values are changed by the normalization. The options for sensitivity normalizations are as follows:

- Multiply $\frac{\sum(x)}{\sum(y)}$
- ABS $\left(\frac{\sum(x)}{\sum(y)}\right)$
- When pseminormalized, multiply $\sum(x)$:
- When fseminormalized, multiply $\frac{1}{\sum(y)}$:
- un-normalized

For example, for absolute general normalization, the following equation applies:

$$sensitivity = \frac{\sum(x \times y) - \left(\sum x \times \sum \frac{y}{n}\right)}{\sum(x^2) - \frac{(\sum x)^2}{n}} \times \left|\frac{\sum x}{\sum y}\right|$$

Syntax

```
pareto_s(<measure_signal>,<parameter_file>,[<normalized_by>],[<sorted_by
>],[<filter_for_s>],[<filter_for_r2>],[<filter_for_r>],[<histbar_plot>],
 [scatter_plot],[<text_window>],[<text_file>])
```

Returns

A waveform which can be a histbar, plot (default), scatter plot, text window, or a text file.

Arguments

`<measure_signal>`

The input signal to be measured (sweep signals are not supported).

`<parameter_file>`

The name of the file containing all of the parameters.

`[<normalized_by>]`

(Optional) For non-interactive regressions, the sensitivity values (effects) that the Pareto calculation results in can be normalized by multiplying a normalization factor. When the factor has a negative value, the sensitivity value signs are changed by the normalization. The normalization factor can be `Param/Meas`, `abs(Param/Meas)`, `Param`, `1/Meas`, or un-normalized.

Feedback

The default normalization factor is `Param/Meas`, where `Param/Meas`: $\frac{\sum x}{\sum y}$

Where `abs(Param/Meas)` is: $\frac{\sum x}{\sum y}$

Where `Param` is: $\sum x$

Where `1/Meas` is: $\frac{1}{\sum y}$

For example, when calculating an absolute general normalization, the 1st order regression main effect normalized value is:

$$sensitivity = \frac{\sum (xy) - \left(\sum x \times \sum \frac{y}{n}\right)}{\sum x^2 - \left(\frac{(\sum x)^2}{n}\right)} \times \left|\frac{\sum x}{\sum y}\right|$$

`[<sorted_by>]`

(Optional) The sorted values, which can be S, R^2, or R. You can only choose R^2 for this function.

`[<filter_for_s>]`

(Optional). The minimum value of the sensitivity filter.

`[<filter_for_r2>]`

(Optional) The minimum value of the regression square filter.

`[<filter_for_r>]`

(Optional) The minimum value of the regression filter.

`[<histbar_plot>]`

(Optional) Organizes the result output by histogram plot, which can be 0 or 1. The default value is 1.

`[<scatter_plot>]`

(Optional) Organizes the result output by scatter plot, which can be 0 and 1. The default value is 0.

`[<text_window>]`

(Optional) Organizes the result output by text window, which can be 0 and 1. The default value is 0.

Feedback

`[<text_file>]`

(Optional) Organizes the result output by text file, which can be the text file name or can be missing.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Cannot open waveform file
- Param signal not specified
- Data count are not the same
- Sweep signal is not supported

Examples

```
pareto_s('0|v(out)','D0:tmp.mc0','Param/Meas','S')
pareto_s('0|v(out)','D0:tmp.mc0','Param/Meas','S',0,0,0,1,0,0)
```

---

pass

Returns 0 (specifying fail) or 1 (specifying pass) for a given expression defined in PrimeWave's output view. If the specification of the input expression name is pass, it returns 1, else it returns 0.

Syntax

```
pass( expr_name)
```

Returns

A scalar value of 0 or 1.

Arguments

`expr_name`

The expression name that is listed in PrimeWave's outputs tab. The expression name should also have the related defined specification in the specifications tab.

Feedback

pavg

Description

Calculates the average value of signal based on the given period. Applies the sample first, then performs the calculation.

Syntax

```
pavg(<signal>,<x_start>,<x_end>,[<period>],
[<sampling_factor>])
```

Returns

An analog signal.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The start of x value to be measured.

`<x_end>`

The end of x value to be measured.

`[<period>]`

(Optional) Specifies the period of the waveform. The default value is the range of x.

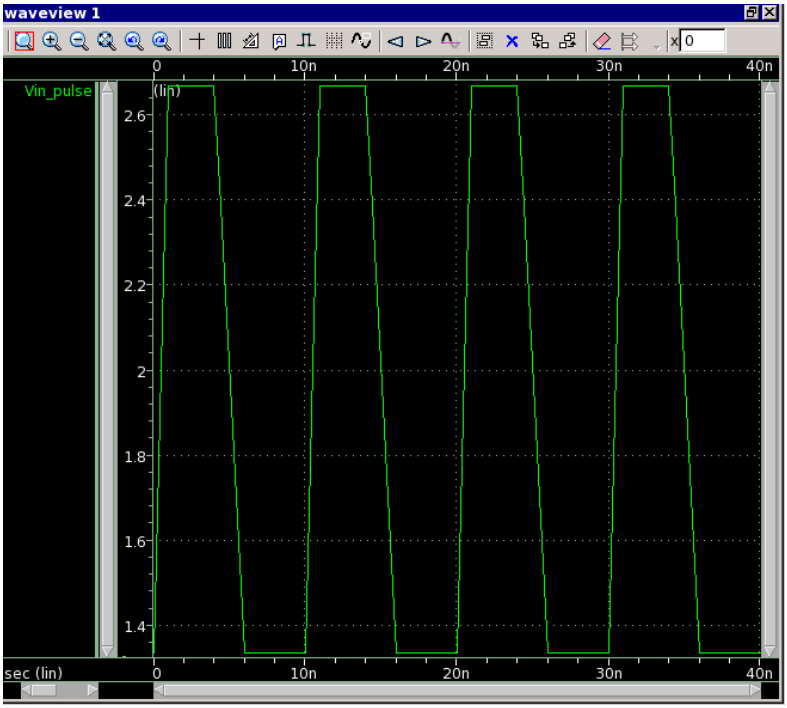
`[<sampling_factor>]`

(Optional) Specifies the value of the sampling factor. The larger the factor, the more points to be included. The value should be larger than 0. The default value is 1.

Feedback

Examples

The following example shows how the `pavg` function works on the input waveform.



The example inputs of the arguments are as follows:

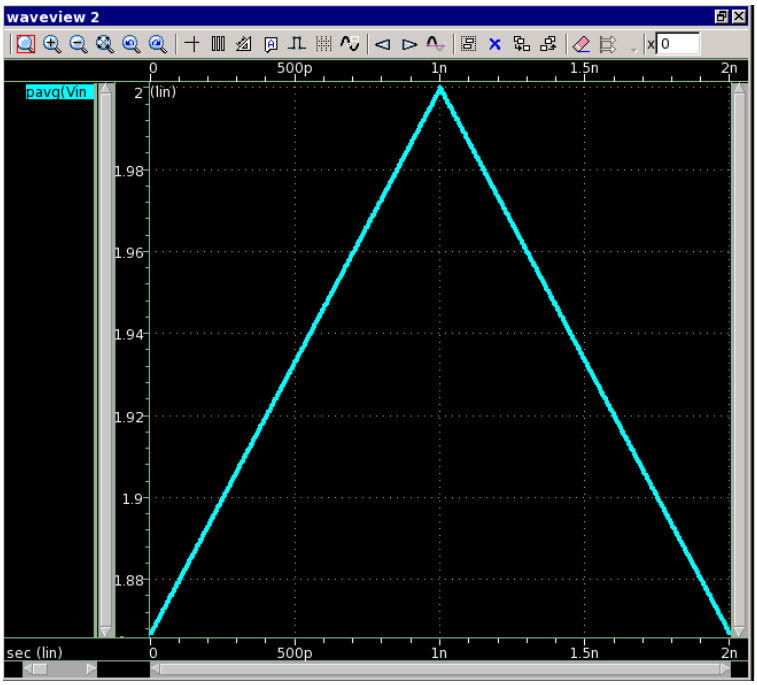
- signal: Vin_pulse
- x_start: 0
- x_end: 40n
- period: 2n
- sampling factor: 1

The function expression shows as follows:

```
pavg('Vin_pulse',0,40n,2n,1)
```

Feedback

with the output waveform being plotted as the following:



---

peak

Description

Calculates the peaks of a given signal.

Syntax

```
peak(<signal>,[<x_start>],[<x_end>],[<x_tolerance>],[<y_tolerance>],[<last>])
```

Returns

An analog signal.

Arguments

`<signal>`

The signal to be measured.

`[<x_start>]`

(Optional) The starting value of x. Default is first x point.

Feedback

`[<x_end>]`

(Optional) The ending value of x. Default is last x point.

`[<x_tolerance>]`

(Optional) Filters all peaks within the distance on the x-axis.

`[<y_tolerance>]`

(Optional) Filters all peaks within the distance on the y-axis.

`[<last>]`

(Optional) Determines whether to count the last point as a peak.

---

phasenoise

Calculates the single-sideband noise of a given signal.

Syntax

```
phasenoise(<signal>,<fundamental_freq>,[<x_start>],
[<x_stop>],[<sampling_freq>],[<min_offset_freq>],
[num_of_cycles],[<windowing_type>])
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured (complex signals are not supported).

`<fundamental_freq>`

The fundamental frequency.

`[<x_start>]`

(Optional) The starting time for the measured signal.

`[<x_stop>]`

(Optional) The ending time for the measured signal.

`[<sampling_freq>]`

(Optional) The sampling frequency. The default is 128*fundamental frequency.

`[<min_offset_freq>]`

(Optional) Minimum offset frequency. The default is frequency is 10.

Feedback

`[<num_of_cycles>]`

(Optional) Number of delay clock cycles. The default number is 0.

`[<windowing_type>]`

(Optional) The type of window in measuring the signal. The default is “rectangle.”

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Fundamental frequency must be > 0X value out of range

Examples

```
phasenoise('0|v(out)',2G)
phasenoise('0|v(out)',2G,0,70u,200,50,2,'rectangle')
```

---

prms

Description

Calculates the root-mean-square value of signal based on the given period. Applies the sample first then performs the calculation.

Syntax

```
prms(<signal>,<x_start>,<x_end>,[<period>],
[<sampling_factor>])
```

Returns

An analog signal.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The start of x value to be measured.

`<x_end>`

The end of x value to be measured.

Feedback

`[<period>]`

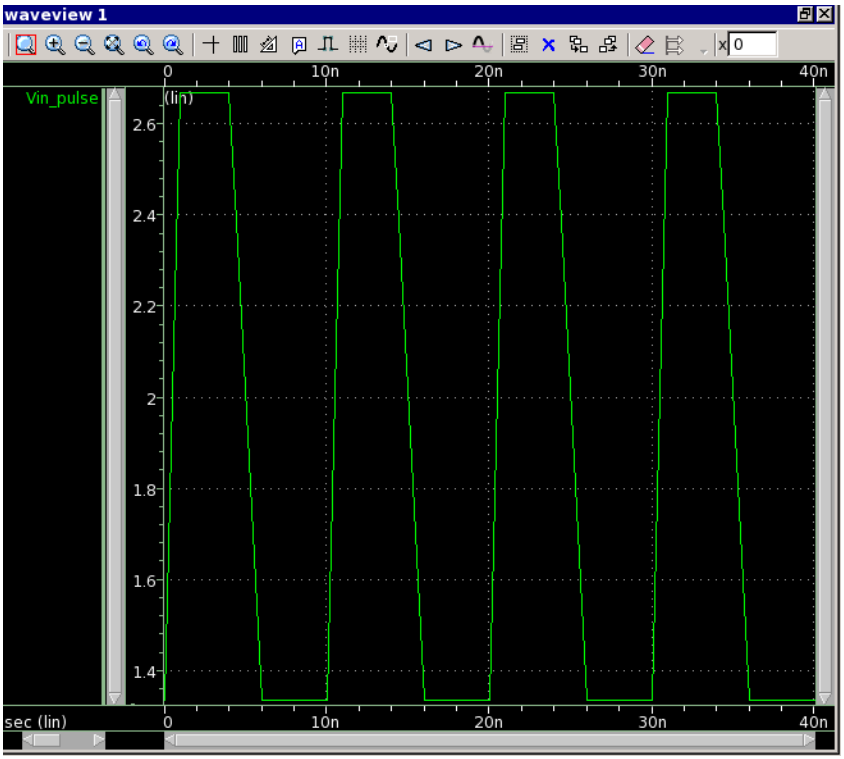
(Optional) Specifies the period of the waveform. The default value is the range of x.

`[<sampling_factor>]`

(Optional) Specifies the value of the sampling factor. The larger the factor, the more points to be included. The value should be larger than 0. The default value is 1.

Examples

The following example shows how the function works on the input waveform.



The example inputs of the arguments are as follows:

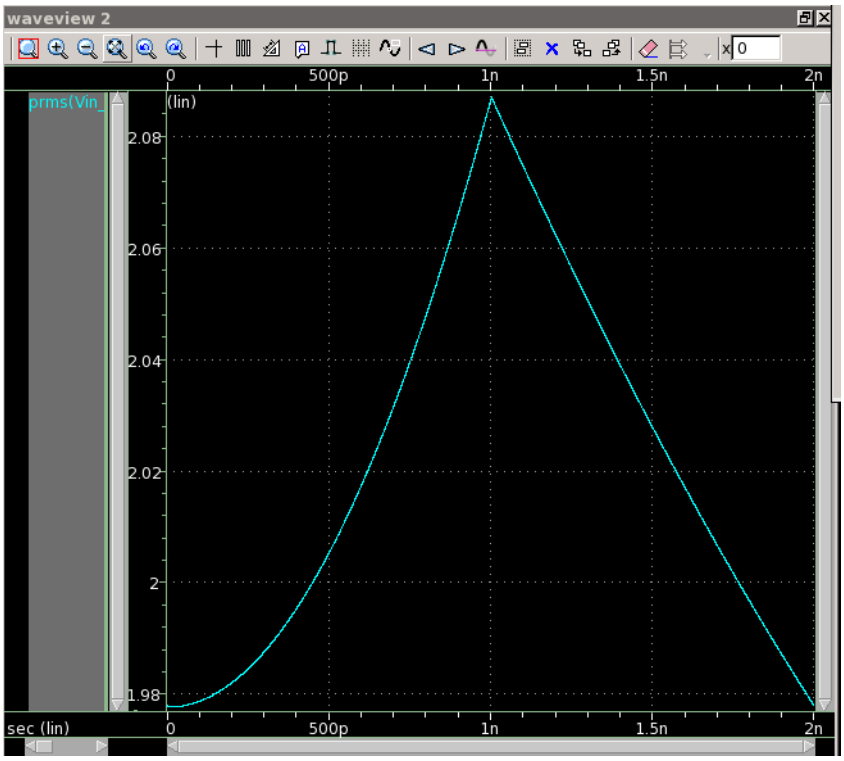
- signal: Vin_pulse
- x_start: 0
- x_end: 40n
- period: 2n
- sampling factor: 1

Feedback

The function expression shows as follows:

```
prms('Vin_pulse',0,40n,2n,1)
```

with the output waveform being plotted as the following:



psd

Description

Power spectral density. Describes how the power of a signal or time series is distributed with frequency. This function estimates Power Spectral Density using the overlapped segmented averaging of periodograms method. A periodogram is the DFT of one segment of the signal or time series.

Syntax

```
psd(<signal>,<x_start>,<x_stop>,<num_of_points>,
[<windowing_type>],[<window_alpha>],
[<window_size>],[<detrend_mode>],
[<coherent_gain>],[<coherent_gain_factor>])
```

Returns

An analog signal.

Feedback

Arguments

`<signal>`

The signal to be calculated.

`<x_start>`

Starting time for the spectral analysis interval.

`<x_stop>`

Ending time for the spectral analysis interval.

`<num_of_points>`

Number of time domain points to be used. Default value is 512.

`[<windowing_type>]`

(Optional) Specifies window to be used.

`[<windowing_alpha>]`

This argument is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 0 to 15. The value 0 implies no smoothing and is equivalent to a rectangular window. The default value is 1.

`[<window_size>]`

(Optional) Number of frequency domain points to use in the Fourier analysis.

`[<detrend_mode>]`

(Optional) Specifies the trend mode (linear, mean, none).

- linear: compute a least squares regression line for each segment and subtract that line from the data points of the segment
- mean: subtract the average value from the data points of the segment
- none (default): no trend removal

`[<coherent_gain>]`

(Optional) This argument specifies coherent gain factor type (none, default, magnitude, db20, db10). The coherent gain factor value is used to scale the power spectral density results.

- none: no scaling the results
- default: scaling the results by 1 if the window type is a rectangle. Else, scaling the results to the signal's amplitude.
- magnitude: scaling with coherent_gain_factor

Feedback

- dB20: scaling with pow(10,coherent_gain_fator/20)
- dB10: scaling with pow(10,coherent_gain_factor/10)

`[<coherent_gain_factor>]`

(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by 1/(Coherent Gain). Valid values are 0 < 1. Default is 0. You can use 1 if you do not want this scaling factor to be used.

---

psdbb

Description

Power spectral density baseband. Returns an estimate for the power spectral density of a waveform1+j * waveform2.

Syntax

```
psdbb(<signal1>,<signal2>,<x_start>,<x_stop>,
<num_of_points>,[<windowing_type>],[<window_alpha>],
[<window_size>],[<detrending_mode>],
[<coherent_gain>],[<coherent_gain_factor>])
```

Returns

An analog signal.

Arguments

`<signal1>`

First signal to be calculated.

`<signal2>`

Second signal to be calculated.

`<x_start>`

Starting time for the spectral analysis interval.

`<x_stop>`

Ending time for the spectral analysis interval.

`<num_of_points>`

Number of time domain points to be used. Default value is 512.

`[<windowing_type>]`

(Optional) Specifies window to be used.

Feedback

`[<windowing_alpha>]`

This argument is only applicable for Kaiser, Kaiser-bessel, and Gaussian window types. Accepts values from 0 to 15. The value 0 implies no smoothing and is equivalent to a rectangular window. The default value is 1.

`[<window_size>]`

(Optional) Specifies number of frequency domain points to use in the Fourier analysis. A larger window size results in an expectation operation over fewer samples, which leads to larger variations in the power spectral density. A small window size can smear out sharp steps in the power spectral density that might be present. Default value is 256.

`[<detrending_mode>]`

(Optional) Specifies the trend mode (linear, mean, none). If there is a deterministic trend to the underlying data, you might want to remove the trend before performing the spectral analysis. For example, consider analyzing phase noise in a VCO model. Without the noise, the phase increases more or less linearly with time, so it is appropriate to set the mode to linear.

To subtract an average value, set the to mean. If the spectrum of raw data is desired, set the mode to none. Default value is none.

`[<coherent_gain>]`

(Optional) Specifies the zero frequency gain (or the DC gain) of the window. When performing dft, the applied window type (aside from the rectangular type) changes the signal amplitude. Applying a coherent gain factor is a way to get consistent results regardless of the window type. Valid Values are none, default, magnitude, dB20, or dB10.

`[<coherent_gain_factor>]`

(Optional) Specifies the scaling factor. A non-zero value that scales the power spectral density by 1/(Coherent Gain). Valid values are 0 < 1. Default is 0. You can use 1 if you do not want this scaling factor to be used.

---

pstddev

Description

Calculates the standard deviation value of signal based on the given period. Applies the sample first then performs the calculation.

Syntax

```
pstddev(<signal>,<x_start>,<x_end>,[<period>],
[<sampling_factor>])
```

Feedback

Returns

An analog signal.

Arguments

`<signal>`

The signal to be measured.

`<x_start>`

The start of x value to be measured.

`<x_end>`

The end of x value to be measured.

`[<period>]`

(Optional) Specifies the period of the waveform. The default value is the range of x.

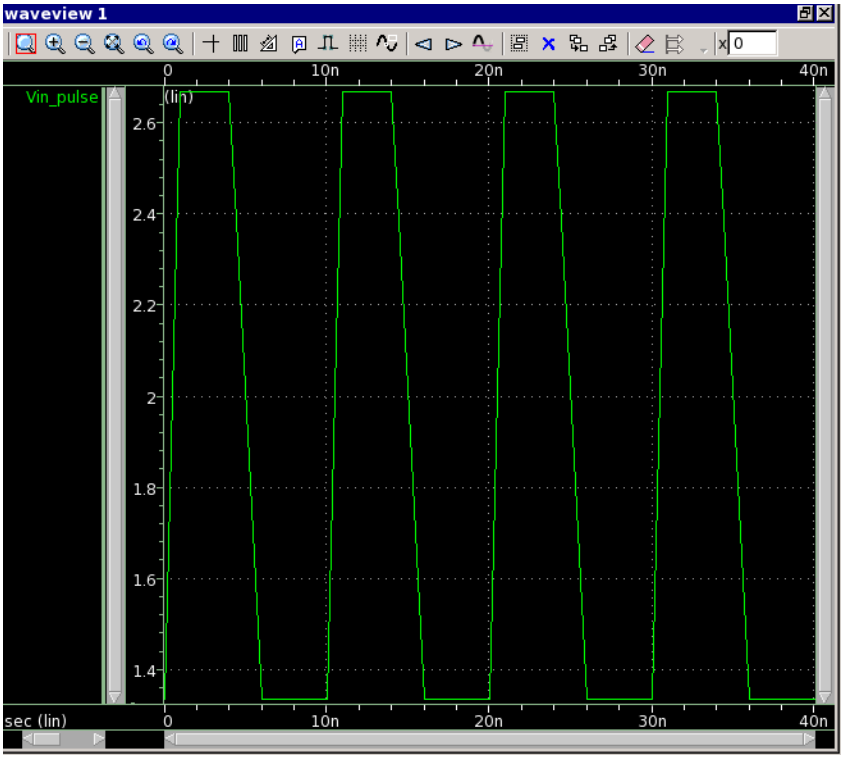
`[<sampling_factor>]`

(Optional) Specifies the value of the sampling factor. The larger the factor, the more points to be included. The value should be larger than 0. The default value is 1.

Feedback

Examples

The following example shows how the function works on the input waveform.



The example inputs of the arguments are as follows:

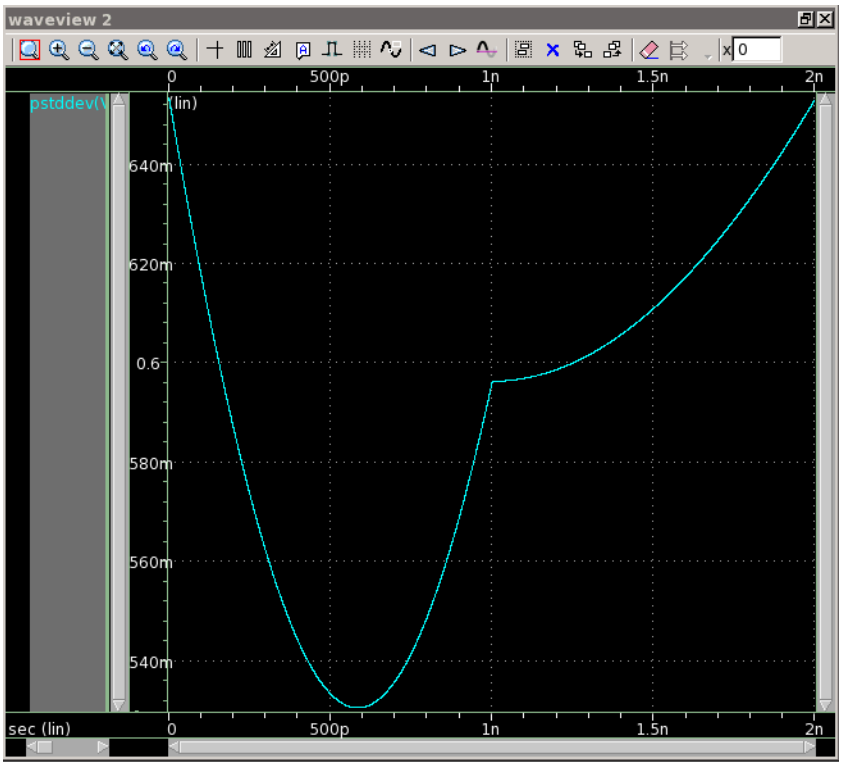
- signal: Vin_pulse
- x_start: 0
- x_end: 40n
- period: 2n
- sampling factor: 1

The function expression shows as follows:

```
pstddev('Vin_pulse',0,40n,2n,1)
```

Feedback

with the output waveform being plotted as the following:



pwc

Measures the piece-wise constant value of a signal.

Syntax

`pwc(<signal>)`

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

Feedback

Description

Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
pwc('0|v(out)')
```

---

pzFreqRealFilter

Description

Returns a filtered pole or zero signal from the input poles/zeros signal.

Syntax

```
pzFreqRealFilter(<signal>, [<Min Freq>], [<Max Freq>], [<Min abs(Real)>],
 [<Max abs(Real)>])
```

Arguments

`<signal>`

The input signal poles/zeros.

`<Min Freq>`

(Optional) The minimum frequency value.

`<Max Freq>`

(Optional) The maximum frequency value.

`<Min abs(Real)>`

(Optional) The minimum real value, abs(real).

`<Max abs(Real)>`

(Optional) The maximum real value, abs(real).

Errors

Possible error messages include:

- Signal not found or variable/type undefined.
- Incorrect signal type.
- Cannot add waveform data.

Feedback

Examples

```
pzFreqRealFilter(v(/out))

pzFreqRealFilter(v(/out), 5M, 10M)
```

The example inputs of the arguments shown are as follows:

- Signal: `v(/out)`
- Min Freq: 5M
- Max Freq: 10M

---

reflectionContour

Description

Constant reflection coefficient contour.

Syntax

```
reflectionContour(<current_signal>, <voltage_signal>, <harmonic_index>,
 [<current_load>], [<voltage_load>], [<max_power>], [<min_power>],
 [<num_contour>], [<resistance>], [<close>], [<modifier>])
```

Returns

A waveform.

Arguments

`<current_signal>`

The current used to get the reflection coefficient.

`<voltage_signal>`

The voltage used to get the reflection coefficient.

`<harmonic_index>`

Harmonic index of waves.

`[<current_load>]`

(Optional) The current used to get the reference reflection coefficient.

`[<voltage_load>]`

(Optional) The voltage used to get the reference reflection coefficient.

`[<max_ref>]`

(Optional) The maximum reflection coefficient.

Feedback

`[<min_ref>]`

(Optional) The minimum reflection coefficient.

`[<num_contour>]`

(Optional) The number of contours. Default is 8.

`[<resistance>]`

(Optional) Resistance used to get the reflection coefficient. Default is 50.

`[<close>]`

(Optional) The contours are closed contours (ON or OFF). Default is OFF.

---

repeat

Creates a new waveform by repeating the given period signal for a specified number of times.

Syntax

```
repeat(<signal>,<period>,<repeat_count>)
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

`<period>`

The period of the given signal.

`<repeat_count>`

The number of cycle times to be repeated.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Examples

```
repeat('0|v(out)',4n,5)
```

Feedback

risevst

Calculates the time difference versus the time of the given signal between `x_start` and `x_stop` from the low threshold crossing to the high threshold crossing.

Syntax

```
risevst(<signal>,<low_y_level>,<high_y_level>,[percent],
 [x_start],[x_stop])
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be calculated.

`<low_y_level>`

The low threshold level of y.

`<high_y_level>`

The high threshold level of y.

`[<percent>]`

(Optional) The value can be `0` and `1`. When specified as `1`, the y_threshold is considered a percentage value.

`[<x_start>]`

(Optional) The starting value of x.

`[<x_stop>]`

(Optional) The ending value of x.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Y value out of range
- X value out of range

Feedback

Examples

```
risevst('0|v(out)',1,4)
risevst('0|v(out)',1,4,1,2n,10n)
```

---

sample

Creates a new waveform which is a sample of a given signal starting from x with period for a given number of times.

Syntax

```
sample(<signal>,<x_start>,<period>,[<num_of_points>],
[<x_end>],[<type>])
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be calculated.

`<x_start>`

The starting value of x.

`<period>`

The period of a given signal.

`[<num_of_points>]`

(Optional) Specifies the number of sample times.

`[<x_end>]`

(Optional) Specifies the ending value of x. Disabled if `num_of_points` is set.

`[<type>]`

(Optional) Specifies scale of x-axis: linear or logarithmic. Default is linear.

Feedback

Errors

Possible error messages include:

- Signal not found
- X value out of range
- Sampling step is non-positive

Examples

```
sample('0|v(out)',2n,4n,4)
```

---

setup_times

Generates a new waveform containing all of the setup time data points.

Syntax

```
setup_times(<signal>,<clock_signal>,<clock_period>,
<y_threshold>,<signal_edge>,<clock_edge>,[<x_start>],
[<x_stop>],[<percent>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<clock_signal>`

The reference clock signal.

`<clock_period>`

The period of the reference `clock_signal`. If the input value for the `clock_signal` period is `0` or less than `0`, the `clock_period` is calculated using the `clock_signal`.

`<y_threshold>`

The threshold value of y for the measured signal.

`<signal_edge>`

The type of measured `signal_edge`. The valid `signal_edge` can be `rise`, `fall`, or `both`. The default `signal_edge` is `rise`.

Feedback

`<clock_edge>`

The type of reference `clock_signal` edge. The valid `signal_edge` can be `rise`, `fall`, or `both`. The default `signal_edge` is `rise`.

`[<x_start>]`

(Optional) The starting time of measurement.

`[<x_stop>]`

(Optional) The ending time of measurement.

`[<percent>]`

(Optional) The value can be `1` or `0`. When the value is `1`, the `y_threshold` is a percentage value. If the value is `0`, the `y_threshold` is not a percentage value.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Y value out of range
- X value out of range

Examples

```
setup_times('0|v(out)','0|v(out)',6n,'rise','rise')
setup_times('0|v(out)','0|v(out)',6n,'rise','rise',2n,10n,1)
```

---

setup_times2

Generates a new waveform that contains all of the setup time data points. `setup_times2` differs from `setup_times` in that it gives you the ability to specify a different threshold for the clock signal as well as the measured signal, whereas `setup_times` allows selecting the threshold of only the measured signal.

Syntax

```
setup_times2(<signal>,<clock_signal>,<clock_period>,
 <data_threshold>,<clock_threshold>,<signal_edge>,
 <clock_edge>,[<x_start>],[<x_stop>],[<percent>])
```

Returns

A waveform.

Feedback

Arguments

`<signal>`

The signal to be measured.

`<clock_signal>`

The reference clock signal.

`<clock_period>`

The period of the reference `clock_signal`. If the input value for the `clock_signal` period is `0` or less than `0`, the `clock_period` is calculated using the `clock_signal`.

`<data_threshold>`

The threshold value of y for the measured signal.

`<clock_threshold>`

The threshold value of y for the `clock_signal`.

`<signal_edge>`

The type of measured `signal_edge`. The valid `signal_edge` can be `rise`, `fall`, or `both`. The default `signal_edge` is `rise`.

`<clock_edge>`

The type of reference `clock_signal` edge. The valid `signal_edge` can be `rise`, `fall`, or `both`. The default `signal_edge` is `rise`.

`[<x_start>]`

(Optional) The starting time of measurement.

`[<x_stop>]`

(Optional) The ending time of measurement.

`[<percent>]`

(Optional) The value can be `1` or `0`. When the value is `1`, the `y_threshold` is a percentage value. If the value is `0`, the `y_threshold` is not a percentage value.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Feedback

- Y value out of range
- X value out of range

Examples

```
setup_times2('0|v(out)','0|v(out)',4n,2,3,'rise','rise')
setup_times2('0|v(out)','0|v(out)',4n,2,3,'rise','rise',2n
     6n,1)
```

---

signalAtFreq

Calculates the signal at frequency.

Syntax

```
signalAtFreq(<signal>,[<frequency>])
```

Arguments

`<signal>`

The input signal to be measured.

`<frequency>`

(Optional) The frequency.

---

signalcompare

Compares the waveform outputs of the simulation.

Syntax

```
signalcompare(<signal1>,<signal2>,[<absolute_tolerance>],[<relative_toler
ance>])
```

Arguments

`<signal1>`

The first signal to be measured.

`<signal2>`

The second signal to be measured.

`<absolute_tolerance>`

(Optional) The absolute tolerance.

`<relative_tolerance>`

(Optional) The relative tolerance.

Feedback

slewratevst

Calculates the slew rate versus time for the given signal within the `y_initial` and `y_final`.

Syntax

```
slewratevst(<signal>,<y_initial>,<y_final>,[<percent>],
 [<x_start>],[<x_stop>],[<trigger_type>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<y_initial>`

The initial threshold value of y.

`<y_final>`

The final threshold value of y.

`[<percent>]`

(Optional) The value can be `1` or `0`. If the value is `1`, the `y_threshold` is a percentage value. If the value is `0`, the `y_threshold` is not a percentage value.

`[<x_start>]`

(Optional) The starting time of measurement.

`[<x_stop>]`

(Optional) The ending time of measurement.

`[<trigger_type>]`

(Optional) The `trigger_type` of the signal within `y_initial` and `y_final`, which can be `rise`, `fall`, or `either`. The default `trigger_type` is `either`.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type

Feedback

- Y value out of range
- X value out of range

Examples

```
slewratevst('0|v(out)',1, 5 )
slewratevst('0|v(out)',1,5,1,2n,10n,'rise')
```

---

smooth

Calculates the smoothed waveform from the given signal. For each point of the input data of the signal, the average value from (x-x_window) to (x+x_window) is calculated to generate the smoothed output.

Syntax

```
smooth(<signal>,<x_window>)
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

`<x_window>`

The smoothing window in x value.

Description

Possible error messages include:

- Signal not found
- X value out of range

Examples

```
smooth('0|v(out)',2n)
```

---

spectralPower

Description

Calculates the spectral power for the specified current waveform and voltage waveform.

Feedback

Syntax

```
spectralPower(<current_signal>,<voltage_signal>)
```

Returns

An analog signal.

Arguments

`<current_signal>`

The current source.

`<voltage_signal>`

The voltage source.

---

sPowerDiffVI

Description

Calculates spectral power with two nodes voltage and current signals.

Syntax

```
sPowerDiffVI(<voltage_signal1>, <voltage_signal2>, <current_signal1>,
 <current_signal2>)
```

Returns

An analog signal.

Arguments

`<voltage_signal1>`

The input positive voltage signal.

`<voltage_signal2>`

The input negative voltage signal.

`<current_signal1>`

Current signal 1.

`<current_signal2>`

Current signal 2.

Feedback

sPowerDiffVR

Description

Calculates spectral power with two nodes voltage and resistance.

Syntax

```
sPowerDiffVR(<voltage_signal1>, <voltage_signal2>, <resistance>)
```

Returns

An analog signal.

Arguments

`<voltage_signal1>`

The input positive voltage signal.

`<voltage_signal2>`

The input negative voltage signal.

`<resistance>`

Resistance.

swapsweep

Description

Swaps the X-axis value with the specified sweep variable.

Syntax

```
swapsweep(<signal>,<parameter_name>,[<x_value>])
```

Returns

An analog signal.

Arguments

`<signal>`

The output waveform with the x-axis values to be swapped.

`<parameter_name>`

The sweep variable for which the x-axis of the given waveform is to be swapped. For example, CAP or temp.

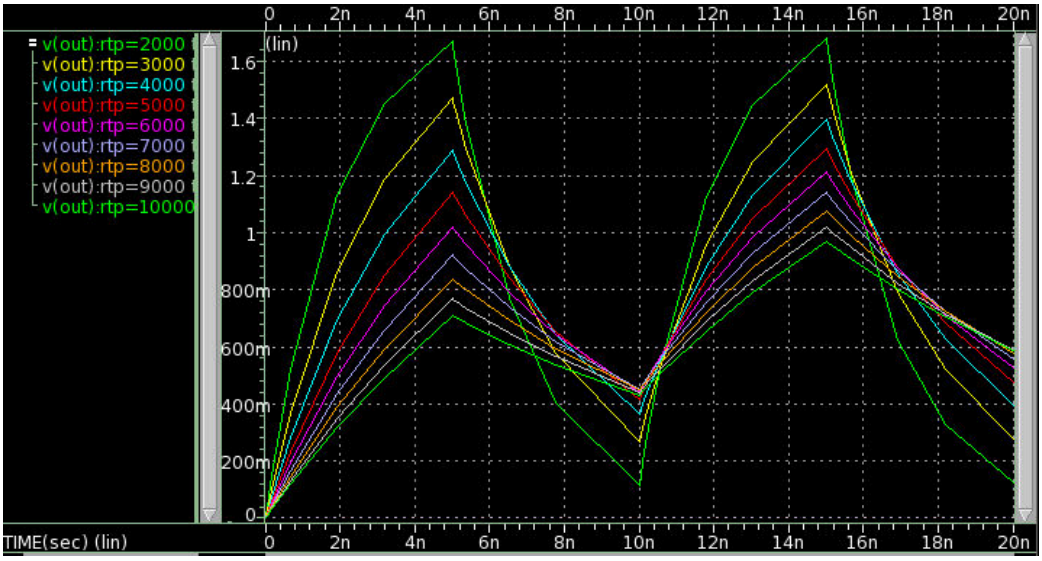
Feedback

`[<x_value>]`

(Optional) x-axis value at which the waveform is swapped with the specified sweep variable.

Examples

The following example shows how the function works on the input waveform “v(out)”:



The example inputs of the arguments show as follows:

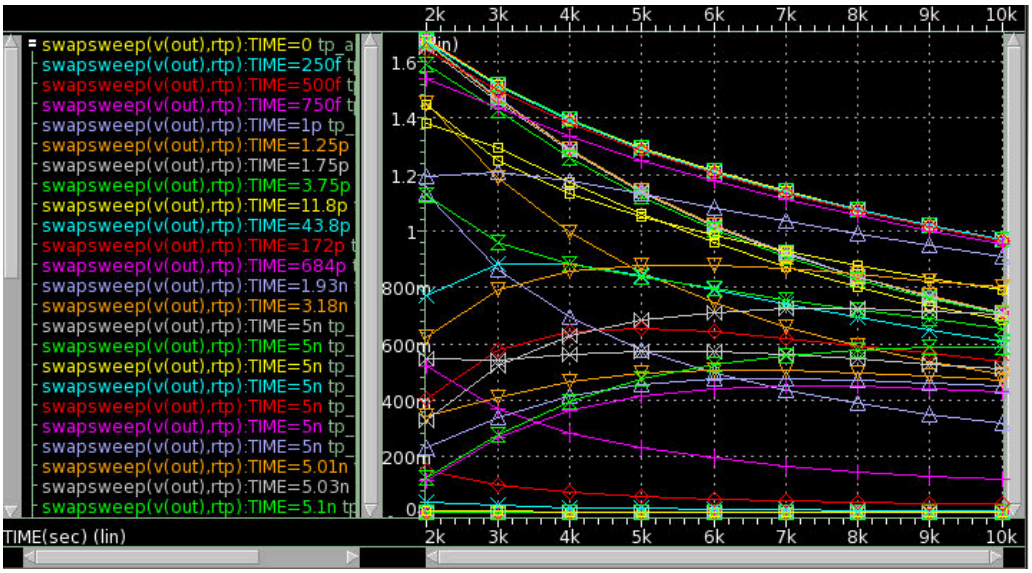
- signal: v(out)
- param name: rtp

The function expression shows as follows:

```
swapsweep('v(out)','rtp')
```

Feedback

The output waveform will be plotted in the following:



sweepavg

Calculates the average waveform of a multi-trace sweep waveform.

Syntax

```
sweepavg(<signal>)
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Mixed logic/analog-function equation not supported

Examples

```
sweepavg('0|v(out)')
```

Feedback

sweepmax

Calculates the maximum bound waveform of a multi-trace sweep.

Syntax

```
sweepmax(<signal>)
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Mixed logic/analog-function equation not supported

Examples

```
sweepmax('0|v(out)')
```

sweepmin

Calculates the minimum bound waveform of a multi-trace sweep.

Syntax

```
sweepmin(<signal>)
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

Feedback

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Mixed logic/analog-function equation not supported

Examples

```
sweepmin('0|v(out)')
```

---

tangent

Description

Calculates a line that passes through x and y coordinates and the specified slope.

Syntax

```
tangent(<signal>,[<x_value>],[y_value>],[<slope>])
```

Returns

An analog signal.

Arguments

`<signal>`

The target signal as the background.

`<x_value>`

Specifies the x-axis value. Default is x value of first point on the waveform.

`<y_value>`

Specifies y-axis value. Default is y of the specified x value or default x value.

`<slope>`

Specifies slope. Default is 1.0.

---

text2wave

Description

Reads specified columns from a text file and creates a signal.

Feedback

Syntax

```
text2wave(<path>,[<X_column>],[Y_column>],[<X_skip>],[<Y_skip>],[<X_label
>],[<Y_label>],[<X_unit>],[<Y_unit>])
```

Returns

A signal.

Arguments

`<path>`

Specifies the file path.

`<X_column>`

(Optional) Specifies the column of X values. Default is 1.

`<Y_column>`

(Optional) Specifies the column of Y values. Default is 2.

`<X_skip>`

(Optional) The number of lines skipped for reading X values.

`<Y_skip>`

(Optional) The number of lines skipped for reading Y values.

`<X_label>`

(Optional) Specifies the signal's X-axis label.

`<Y_label>`

(Optional) Specifies the signal's Y-axis label.

`<X_unit>`

(Optional) Specifies the signal's X-axis unit.

`<Y_unit>`

(Optional) Specifies the signal's Y-axis unit.

---

trace

Returns a trace from a multi-trace of signal s based on the parameter condition parm_con.

Syntax

```
trace(<signal>,<parameters>)
```

Feedback

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured. The signal must be a sweep signal with different parameters.

`<parameters>`

The parameter condition of the input signal.

Description

Possible error messages include:

- Signal not found
- Incorrect signal type
- Can’t find trace with matching conditions

Examples

```
trace('0|v(out)',"MONTE_CARLO=1")
```

---

v2todBc

Converts phase noise from V^2/Hz to dBc/Hz. This function only works for waveform files which contain carrier magnitude information; otherwise, the waveform viewer will report that carrier magnitude is not defined.

Syntax

```
v2todBc(<signal>)
```

Returns

N/A

Arguments

`<signal>`

The signal to be converted.

Examples

```
v2todBc(dBc2V2('PAC_L(f)'))
```

Feedback

window_average

Window average: calculates the average using step size between a start and a stop time.

Syntax

```
window_average(<signal> [<x_start>] [<x_stop>] [<time_step>]
 [<window_size>])
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be calculated. Analog signals are supported. The magnitude is automatically applied for complex signals.

`x_start`

(Optional) The start time (`x1`). Default: `xmin`.

`x_stop`

(Optional) The stop time (`x2`). Default: `xmax`.

`time_step`

(Optional) The step size (`step`). Default: (`x_stop-x_start`).

`window_size`

The window size (`size`). Default: `time_step`

Errors

Possible error messages include:

- Signal not found
- Incorrect signal type
- Input signal has no data point

x_at_y

Plots the crossing points of a given signal on a specified y_threshold.

Syntax

```
x_at_y(<signal>,<y_level>)
```

Feedback

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

`<y_level>`

The threshold value of y.

Errors

Possible error messages include:

- Signal not found
- Y value out of range

Examples

```
x_at_y('0|v(out)',3)
```

---

xscale

Plots a new waveform identical to the given signal with the x component multiplied by a scaling factor.

Syntax

```
xscale(<signal>,<scaling_factor>)
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

`<scaling_factor>`

The multiplier to cross the x value of a given signal.

Errors

Possible error messages include:

- Signal not found

Feedback

Examples

```
xscale('0|v(out)',2)
```

---

xshift

Plots a new waveform identical to the given signal but shifted to the right by x delay.

Syntax

```
xshift(<signal>,<x_delay>)
```

Returns

A waveform.

Arguments

`<signal>`

The signal to be measured.

`<x_delay>`

The x value by which the signal shifts to right.

Errors

Possible error messages include:

- Signal not found

Examples

```
xshift('0|v(out)',100u)
```

---

xval

Measures the x value of the given signal and creates a new waveform equivalent to y=x.

Syntax

```
xval(<signal>)
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

Feedback

Errors

Possible error messages include:

- Signal not found

Examples

```
xval('0|v(out)')
```

---

yvsi

Creates a new waveform with a y value versus a data point index signal.

Syntax

```
yvsi(<signal>)
```

Returns

A waveform.

Arguments

`<signal>`

The input signal to be measured.

Errors

Possible error messages include:

- Signal not found

Examples

```
yvsi('0|v(out)')
```

RF Functions

This topic contains information on how to use the supported RF functions.

The following table summarizes the supported RF functions. Complex mathematical functions can also operate on both scalars and waveforms.

Note:

The RF functions support S-parameter results from Spectre, ELDO-RF, and ADS.

Feedback

Function	Description
apgc	Calculates the available power gain circles (APGC) over S-parameter values and gain values at specified frequency values.
b1f	A 2-port stability factor B1.
ga	A 2-port available power gain.
gac_freq	The available power gain circles where the gain is fixed and frequency is swept.
gac_gain	The available power gain circles where the frequency is fixed and gain is swept.
gmax	A 2-port maximum available power gain.
gmin	The optimum noise reflection coefficient for nfmin.
gmsg	A 2-port max. stable power gain.
gp	A 2-port power gain.
gpc_freq	The operating power gain circles where the gain is fixed and frequency is swept.
gpc_gain	The operating power gain circles where the frequency is fixed and gain is swept.
gt	A 2-port transducer power gain.
gumx	A 2-port unilateral max. available power gain.
harmonic	Returns the signal value for a specific intermodulation product.
hp	2-port H-parameter, p1/p2 1-based port index. z0: port characteristic impedance.
imProd	Calculates the intermodulation produce of a given signal and frequency.
kf	A 2-port stability factor.
lsb	Returns the load stability circles.
lq	Calculates the Inductance value L and Quality factor Q.
mu1	U-test for stability circle derivation.
mu2	U-test for stability circle derivation.
nc_freq	Noise circles with fixed frequency.
nc_gain	Noise circles with fixed frequency.

Feedback

Function	Description
nf	The noise figure of the waveform file.
nfmin	The minimum noise figure.
noisec	Calculates the noise circle over input Noise Figure values at specified frequency values.
opgc	Calculates the Operating Power Gain Circles over S-parameter values and gain values at specified frequency values.
rn	Normalized equivalent noise resistance vs. frequency waveform.
Rn	Equivalent noise resistance vs. frequency waveform.
sp	S-parameter, p1 and p2 are 1-based port index.
spm	The differential S-parameter function (scattering parameters).
ssb	Returns the source stability circles.
stabc_in	Calculates the Input stability circles over S-parameters, (S11, S12, S21, and S22), at the specified frequency values.
stabc_out	Calculates the output stability circles over S-parameters, (S11, S12, S21 and S22), at the specified frequency values.
vswr	Measures the voltage standing wave ratio.
vswrc_in	Calculates the input voltage standing wave ratio cycle circles calculated over S-parameter values, as well as the input value of voltage standing wave ratios at specified frequency values.
vswrc_out	Calculates the output voltage standing wave ratio cycle circles over S-parameter values, as well as the output value of a voltage standing wave ratio at specified frequency values.
ym	Y parameter at port1 with all other ports matched.
yp	Y-parameter, p1 and p2 are 1-based port index. z0 is the port characteristic impedance.
zm	Z-parameter at port p1 with all other ports matched.
zp	Z-parameter, p1 and p2 are 1-based port index. z0 is the port characteristic impedance.

Feedback

apgc

Description

Calculates the available power gain circles (APGC) over S-parameter values and gain values at specified frequency values. The circles are implemented as sweep signals with `gain` as a parameter if gain sweep is the input, or with frequency as a parameter if multiple frequency values are input.

The result waveform can be one circle if the waveform file is not a sweep waveform file. One gain value is input, and only one frequency value is selected; otherwise, the resulting waveform can be multiple circles.

Syntax

```
apgc(<s11>,<s12>,<s21>,<s22>,<gain>,<freq_list>)
```

Returns

A set of circles.

Arguments

`<s11>`

Input S-parameter s11.

`<s12>`

Input signal S-parameter s12.

`<s21>`

Input signal S-parameter s21.

`<s22>`

Input signal S-parameter s22.

`<gain>`

The gain value in dB units, which can be one single value, two values separated by space in the format of “step number”, which means to sweep gain values from maximum gain value at a loss of “Step” (Gmax – Step) with “Number” times, or three values separated by a space in the format of “start stop step”. The maximum gain value is calculated with associated S-Parameter values at specified frequency values. For example, Gmax=0.6, Step=0.1, Number=2, gain values is {0.5 0.4}.

`<freq_list>`

Can be one single value or a list of frequency values separated by a space.

Feedback

Errors

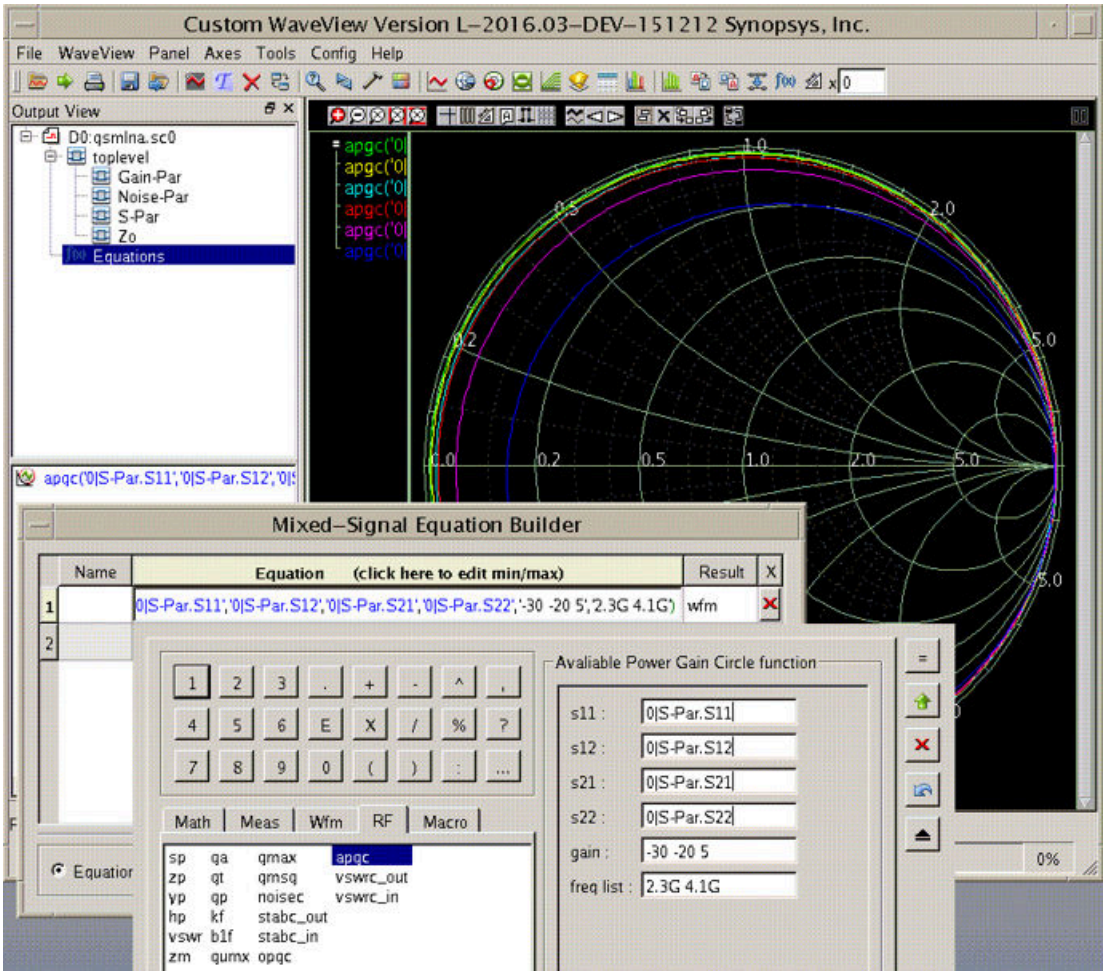
Possible error messages include:

- Ga value is greater than Gamax at trace#=’ ’, freq=’ ‘, Ga=’ ‘, Gamax=’ ’
- Ga value is greater than Gamax at freq=’ ‘, Ga=’ ‘, Gamax=’ ’
- Gamax can’t be defined, use Gmsg at trace#=’ ‘, freq=’ ‘, K=’ ‘, |delta|=’ ‘”
- Zero again, |S21| is too small at trace#=’ ‘, freq=’ ‘, S21=(‘ ‘, ‘ ‘)”
- Zero again, |S21| is too small at freq=’ ‘, S21=(‘ ‘, ‘ ‘)”
- Zero again, |S12| is too small at trace#=’ ‘, freq=’ ‘, S12=(‘ ‘, ‘ ‘)”
- Zero again, |S12| is too small at freq=’ ‘, S12=(‘ ‘, ‘ ‘)”
- All frequency values are not in the files
- Signal is not valid or cannot find the loaded signal

Examples

```
apgc('0|S-Par.S(1,1)','0|S-Par.S(1,2)','0|S-Par.S(2,1)',
     '0|S-Par.S(2,2)','-30 -20 5','2.3G 4.1G')
```

Feedback



b1f

A 2-port stability factor for B1.

Syntax

```
rf('b1f')
```

Returns

A waveform or a scalar value.

Feedback

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

Examples

```
rf('b1f')
```

---

ga

A 2-port available power gain. The algorithm for power gain is where P_{load} is the maximum time average for power to be delivered to the load and the maximization of power is over the load impedance, and P_{input} is the time averaged power entering the network:

$$G_p = \frac{P_{load}}{P_{input}}$$

Syntax

```
rf('ga',<load_impedance>)
```

Returns

A waveform or a scalar value.

Arguments

`<load_impedance>`

The characteristic load impedance.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

Examples

```
rf('ga',2)
```

Feedback

gac_freq

Description

The available power gain circles where the gain is fixed and frequency is swept.

Syntax

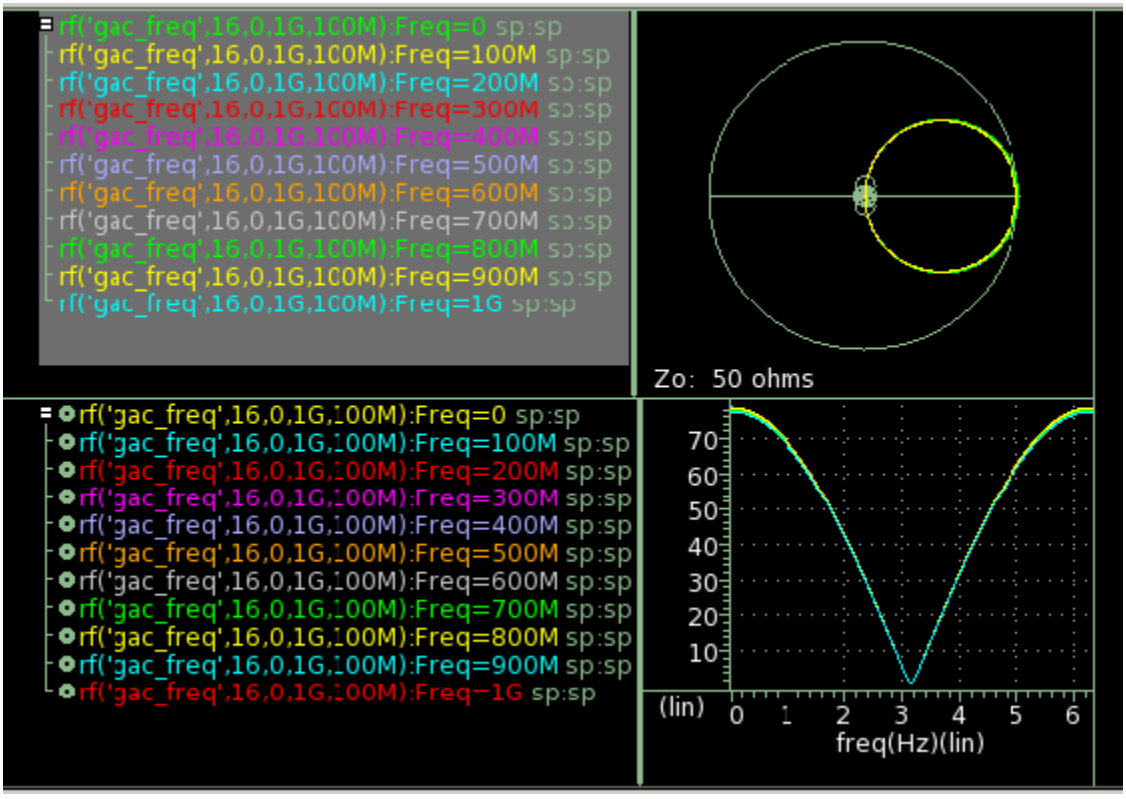
```
rf('gac_freq', level, start_freq, stop_freq, step_freq)
gac_freq(<file>, <level_db>, <start_freq>, <stop_freq>,
 <step_freq>)gac_freq(<file>, <level_db>, <start_freq>, <stop_freq>,
 <step_freq>
```

Returns

A set of circles.

Examples

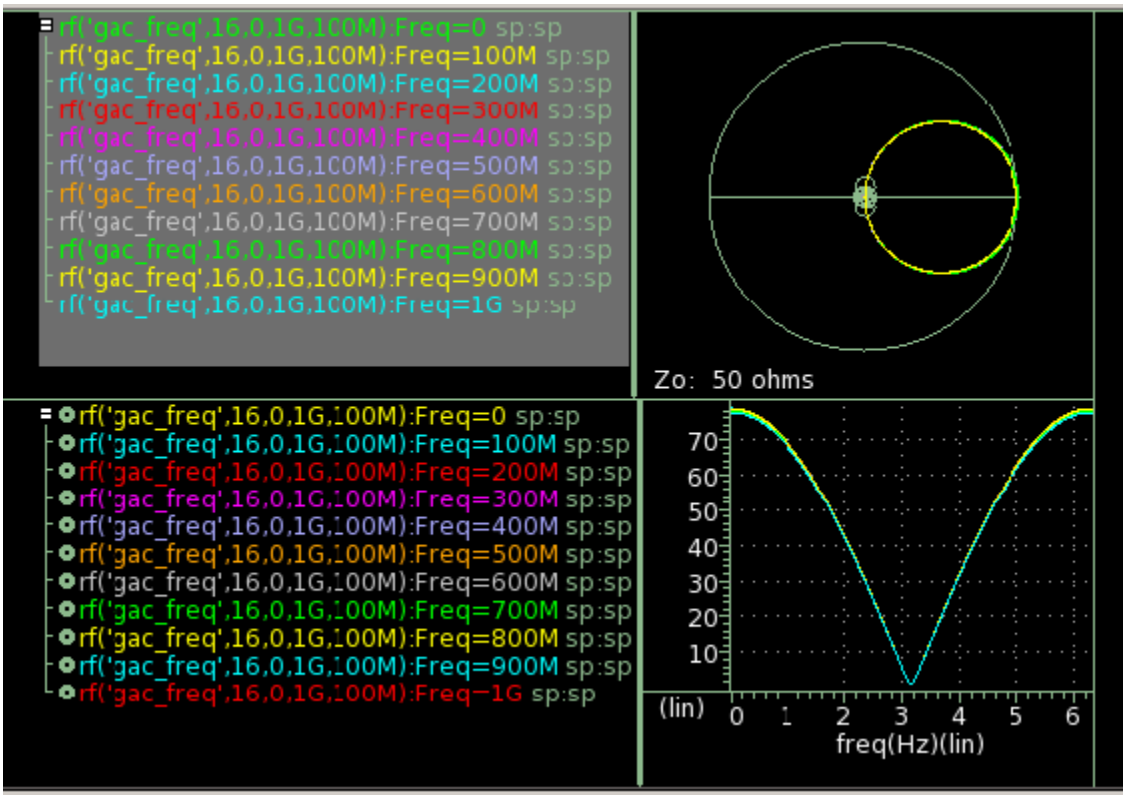
```
rf('gac_freq',16,0,1G,100M)
```



Examples

```
rf('gac_freq',16,0,1G,100M)
```

Feedback



gac_gain

Description

The available power gain circles where the frequency is fixed and gain is swept.

Syntax

```
rf('gac_gain', freq, start_gain, stop_gain, step_gain)
```

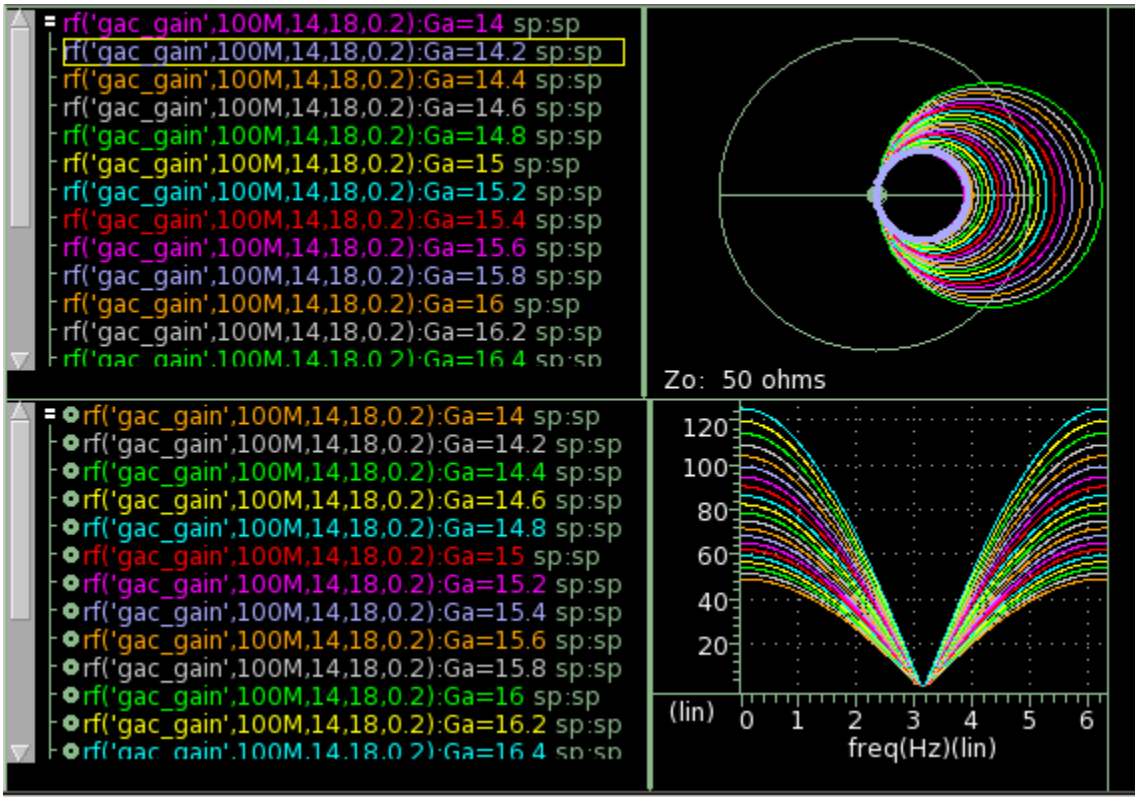
Returns

A set of circles.

Examples

```
rf('gac_gain',100M,14,18,0.2)
```

Feedback



gmax

A 2-port maximum available power gain.

Syntax

```
rf('gmax')
```

Returns

A waveform or a scalar value.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

Examples

```
rf('gmax')
```

Feedback

gmin

Description

The optimum noise reflection coefficient for `nfmin`.

Syntax

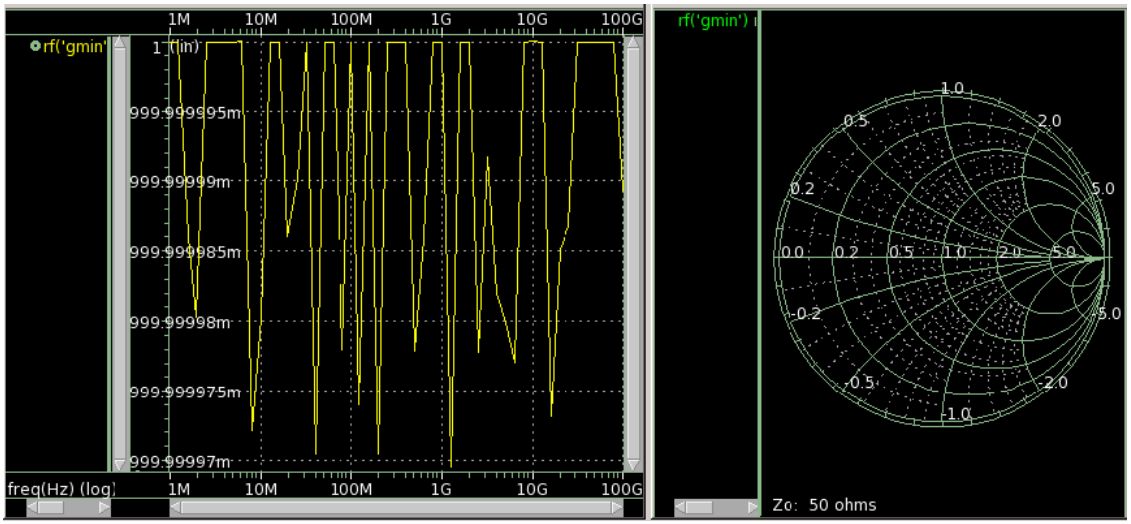
```
rf('gmin')
```

Returns

Returns a waveform or a scalar value.

Examples

```
rf('gmin')
```



gmsg

A 2-port maximum stable power gain.

Syntax

```
rf('gmsg')
```

Returns

A waveform or a scalar value.

Feedback

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

Examples

```
rf('gmsg')
```

---

gp

A 2-port power gain. The algorithm for power gain is where P_{load} is the average power delivered to the load and P_{source} is the average available power at the source:

$$G_p = \frac{P_{source}}{P_{load}}$$

Syntax

```
rf('gp',<source_impedance>,<load_impedance>)
```

Returns

A waveform or a scalar value.

Arguments

`<source_impedance>`

The characteristic source impedance.

`<load_impedance>`

The characteristic load impedance.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

Examples

```
rf('gp',3,2)
```

Feedback

gpc_freq

Description

The operating power gain circles where the gain is fixed and frequency is swept.

Syntax

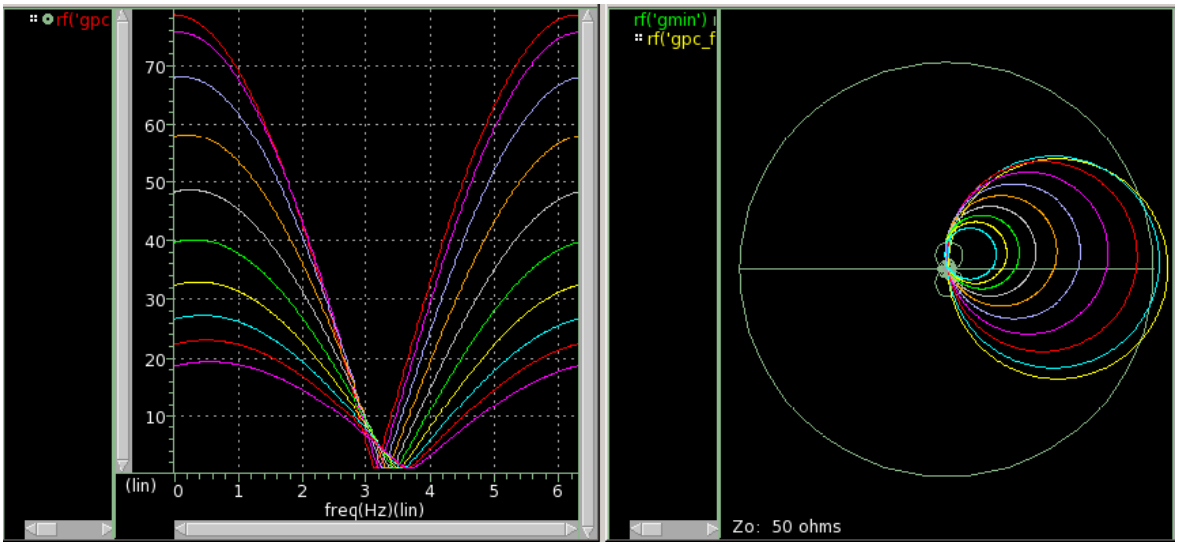
```
rf('gpc_freq', level, start_freq, stop_freq, step_freq)
```

Returns

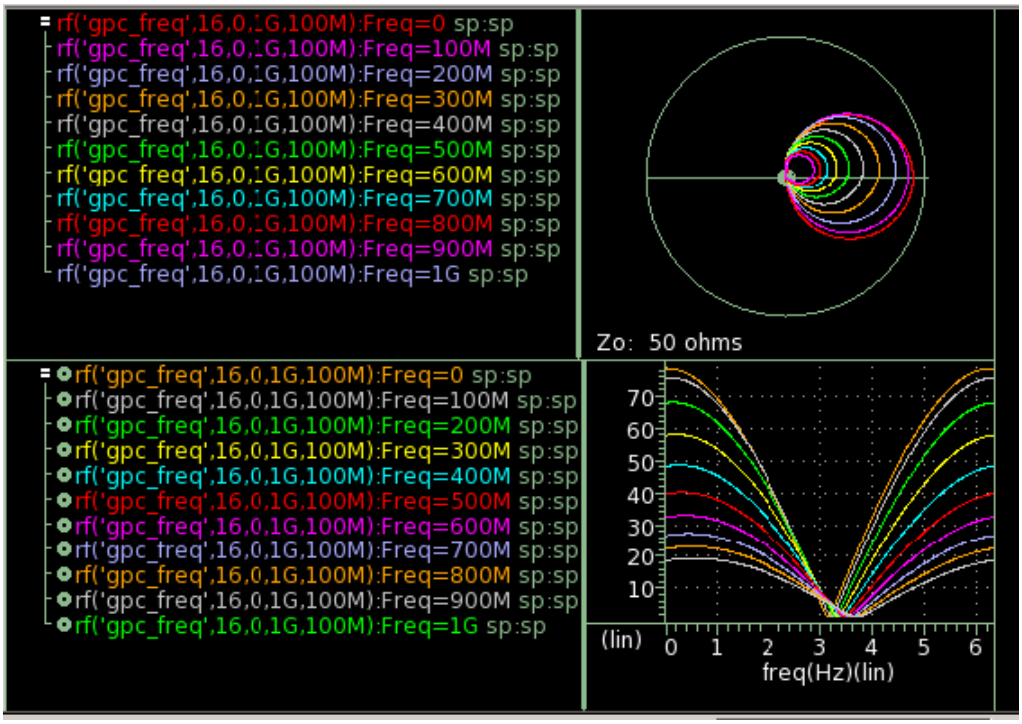
A set of circles.

Examples

```
rf('gpc_freq',16,0,1G,100M)
```



Feedback



gpc_gain

Description

The operating power gain circles where the frequency is fixed and gain is swept.

Syntax

```
rf('gpc_gain', freq, start_gain, stop_gain, step_gain)
```

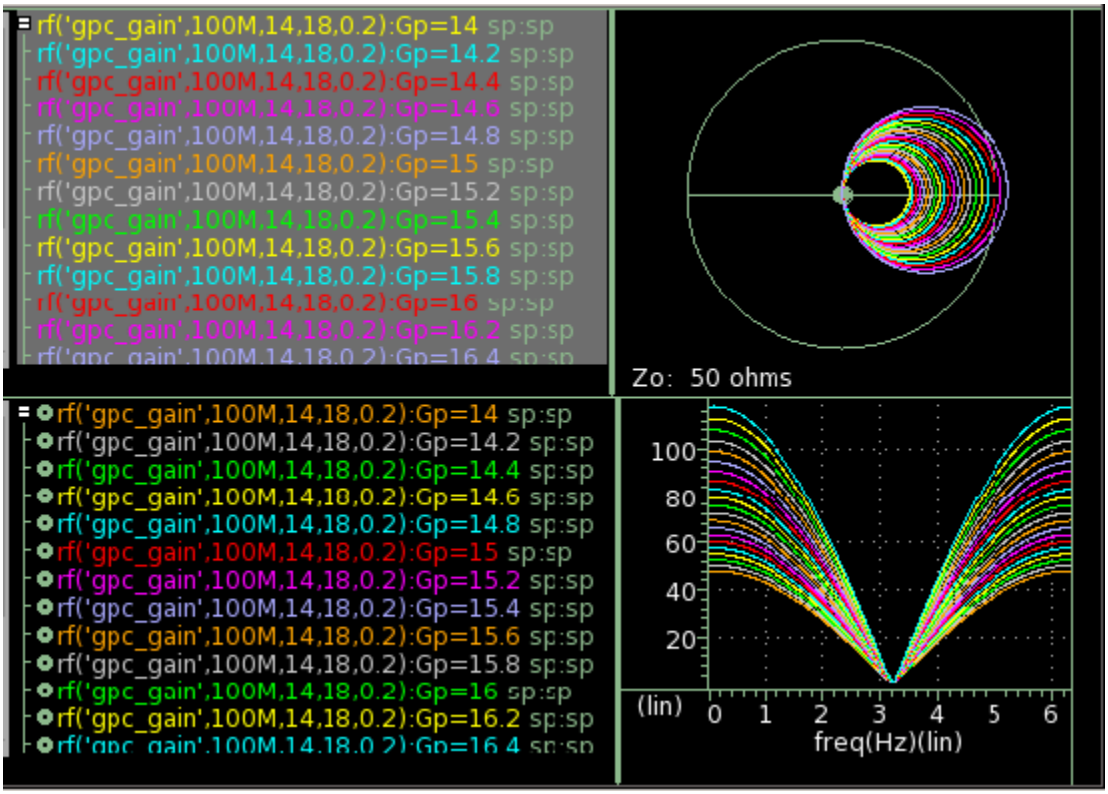
Returns

A set of circles.

Examples

```
rf('gpc_gain',100M,14,18,0.2)
```

Feedback



gt

A 2-port transducer power gain. The algorithm for power gain is where P_{load} is the average power delivered to the load and $P_{source\ max}$ is the maximum available power at the source:

$$G_p = \frac{P_{load}}{P_{sourcemax}}$$

Syntax

```
rf('gt',<source_impedance>)
```

Returns

A waveform or a scalar value.

Arguments

`<source_impedance>`

The characteristic source impedance.

Feedback

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

Examples

```
rf('gt',3)
```

---

gumx

A 2-port unilateral maximum power gain.

Syntax

```
rf('gumx')
```

Returns

A waveform or a scalar value.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

Examples

```
rf('gumx')
```

---

harmonic

Description

Returns the signal value for a specific intermodulation product.

Syntax

```
harmonic ([signal <string>]
     [, coeff1 <string>]
     [, coeff2 <string>]
     [, coeff3 <string>]
     [, coeff4 <string>]
     [, coeff5 <string>]
```

Feedback

```
[, coeff6 <string>]
[, coeff7 <string>]
[, coeff8 <string>]
[, coeff9 <string>]
[, coeff10 <string>]
[, xUnits <enum>]
[, resultType <string> | resultFile <string>]
[, resultsDir <string> | testbenchName <string>]
[, analysisName <string>]
[, params <string>])
```

Arguments

`signal`

(Optional) The signal expression that will be evaluated and reduced to a specific intermodulation product. For exampleFor snac and hbac data, if no coefficients are specified, the entire signal is reformatted such that a plot will display wavelets for each `sideband.v(/out)`. If a signal expression is not specified, the function returns the intermodulation frequency value.

`coeff1-coeff10`

(Optional) The frequency coefficients used to calculate the intermodulation frequency of the target harmonic. The number of coefficients specified must match the number of primary tones in the circuit.

`xUnits`

(Optional) Performs conversion of the xAxis units when not a frequency sweep. “asIs” means no conversion. “toW” means the xAxis sweep units in the data is in Dbm and will be converted to Watts. “toDbm” means the xAxis sweep units in the data is in Watts and will be converted to Dbm.

Valid values: asIs, isW, isDbm, toW, and toDbm. (Default: asIs)

`resultType`

(Optional) The result type.

`resultsDir`

(Optional) The directory where the results reside.

`resultFile`

(Optional) The result file.

`testbenchName`

(Optional) The testbench name.

`analysisName`

(Optional) The user-specified name for the analysis.

Feedback

`params`

(Optional) The parameter conditions.

---

hp

Description

2 port H parameter function (hybrid parameters) $\begin{bmatrix} V_1 \\ I_2 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} I_1 \\ V_2 \end{bmatrix}$

where: $h_{11} = \frac{V_1}{I_1}\Big|_{V_2=0}$, $h_{12} = \frac{V_1}{V_2}\Big|_{I_1=0}$, $h_{21} = \frac{I_2}{I_1}\Big|_{V_2=0}$, $Z_{22} = \frac{I_2}{V_2}\Big|_{I_1=0}$.

Syntax

`rf('hp',<port1>,<port2>,<impedance>)`

Returns

A waveform or a scalar value.

Arguments

`<port1>`

The first 1-base port index.

`<port2>`

The second 1-base port index.

`<impedance>`

The port characteristic impedance.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of H parameter size (no H parameter found)

Examples

```
rf('hp',2,1,40)
```

Feedback

imProd

Description

Calculates the intermodulation product of a given signal and frequency.

Syntax

```
imProd(<signal>, <freq>, [<param_name>])
```

Returns

An analog scalar value.

Arguments

`<signal>`

The signal to be calculated. Supports analog signals. For complex signals, the magnitude is automatically applied.

`<freq>`

The frequency value.

`<param_name>`

(Optional) The parameter condition of the input signal.

Examples

```
imProd('v(out)', 10M)
```

kf

A 2-port stability factor.

Syntax

```
rf('kf')
```

Returns

A waveform or a scalar value.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)

Feedback

Examples

```
rf('kf')
```

---

lsb

Description

Returns the load stability circles.

Syntax

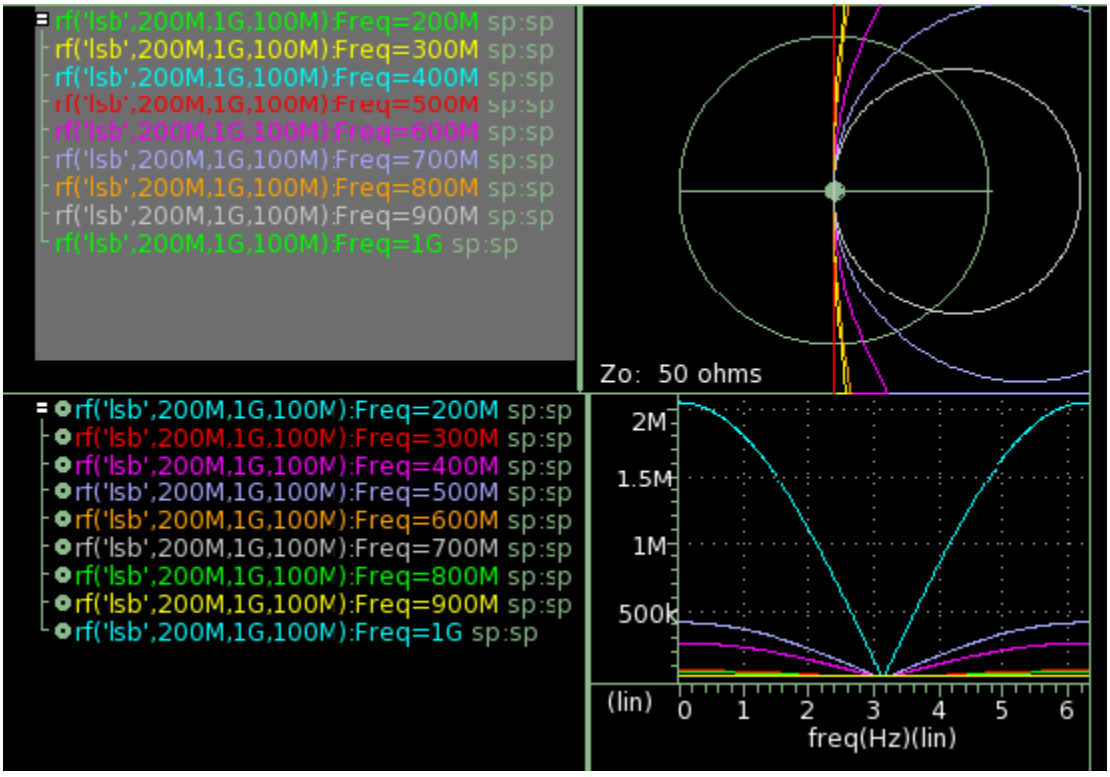
```
rf('lsb', start_freq, stop_freq, step_freq)
```

Returns

A set of circles.

Examples

```
rf('lsb',200M,1G,100M)
```



Feedback

lq

Description

Calculates the Inductance value L and Quality factor Q.

Syntax

```
lq(<file>,<type> [,<inductor_mode>][,<port>][,<port2_index>])
```

Returns

When type is L, returns the Inductance value L. When type is Q, returns the Quality factor value.

Arguments

`<file>`

The waveform file.

`<type>`

Valid types are L and Q. The default is L.

`<inductor mode>`

(Optional) The inductor mode.

`<port>`

(Optional) Port 1 index.

`<port2_index>`

(Optional) Port index.

Errors

Possible error messages include:

- `lq: unknown type '%s', expect SingleEnded/Differential/CommonFloating/CommonGND`: Indicates that the type is unexpected.
- `Input value of port is out the range of LQ size (no LQ found)`: Indicates that the port is unexpected.
- `Evaluation failed`: Indicates that the SP signal was not found.

Examples

```
lq(file(~/test.sp),L,SingleEnded)
```

Calculates the Inductance value in single-ended.

Feedback

```
lq(file(~/test.s3p),Q, Differential)
```

Calculates the Quality factor value of three port inductors in differential mode.

For an example of how to plot the inductance value L and Quality factor Q by S-parameter, see SolvNetPlus article, L&Q Plot by S-Parameter in the PrimeWave Design Environment".

---

mu1

U-test for stability circle derivation.

Syntax

```
rf('mu1')
```

Returns

A waveform or a scalar value.

---

mu2

U-test for stability circle derivation.

Syntax

```
rf('mu2')
```

Returns

A waveform or a scalar value.

---

nc_freq

Description

Noise circles with fixed gain.

Syntax

```
nc_freq(<level>,<start_freq>,<stop_freq>,<step_freq>)
```

Returns

A complex analog signal that can be plotted on Smith Chart.

Feedback

Arguments

`<level>`

Noise level in dB.

`<start_freq>`

Starting frequency.

`<stop_freq>`

Ending frequency.

`<step_freq>`

Frequency step value.

---

nc_gain

Description

Noise circles with fixed frequency.

Syntax

`nc_gain(<frequency>,<start_gain>,<stop_gain>,<step_gain>)`

Returns

A complex analog signal that can be plotted on Smith Chart.

Arguments

`<frequency>`

Noise level in Hz.

`<start_gain>`

Starting noise level.

`<stop_gain>`

Ending noise level.

`<step_gain>`

Noise level step value.

Feedback

nf

Description

The noise figure of the waveform file.

Syntax

```
rf('nf')
```

Returns

An analog signal.

nfmin

Description

The minimum noise figure.

Syntax

```
rf('nfmin')
```

Returns

An analog signal.

noisec

Description

Calculates the noise circle over input Noise Figure values at specified frequency values. The circles are implemented as a sweep signal with `noise_figure` as a parameter if multiple noise figures are specified. Frequency is another parameter if multiple frequency values are specified.

Other needed values include NFmin, Rn and Zopt. These values come from signals in the active waveform file (for example, `S-Par.NFmin`, `S-Par.Rn`, and `S-Par.Zopt`. If these signals are not located in a file, the function fails.

The resulting waveform can be one circle if the waveform file is not a sweep waveform file, one noise figure value is input and only one frequency value is input; otherwise, the resulting waveform can be multiple circles.

Syntax

```
rf('noisec',<noise_figures>,<freq_list>)
```

Feedback

Returns

A set of circles.

Arguments

`<noise_figures>`

A value in dB scale which can either be a single value or three values separated by space in the format of `start stop step`.

`<freq_list>`

A frequency list which can be one single value or a list of frequency values separated by space.

Errors

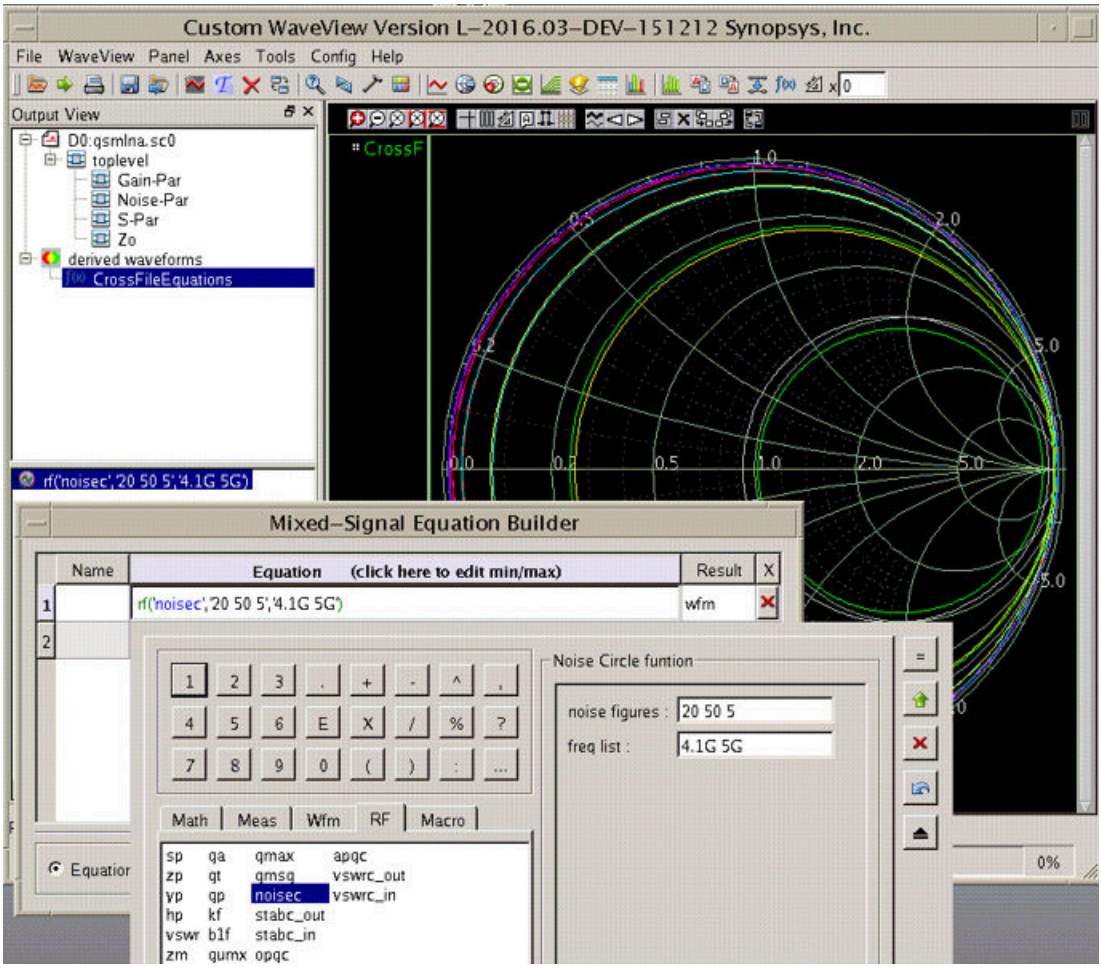
Possible error messages include:

- Signal not found
- Some frequency values are not in the file
- No noise parameter information in the file
- Imaginary radius values at trace#=’ ‘, freq=’ ‘, nf=’ ‘”
- No system impedance Zo in waveform file
- Invalid noise figure inputs

Examples

```
rf('noisc','20 50 5','4.1G 5G')
```

Feedback



opgc

Description

Calculates the Operating Power Gain Circles over S-parameter values and gain values at specified frequency values. The circles are implemented as sweep signals with `gain` as a parameter if gain sweep is input or frequency as a parameter if multiple frequency values are input.

The result waveform can be one circle if the waveform file is not a sweep waveform file. One gain value is input if only one frequency value is selected; otherwise, the result waveform can be multiple circles.

Syntax

```
opgc(<s11>,<s12>,<s21>,<s22>,<gain>,<freq_list>)
```

Feedback

Returns

A set of circles.

Arguments

`<s11>`

Input S-parameter s11.

`<s12>`

Input signal S-parameter s12.

`<s21>`

Input signal S-parameter s21.

`<s22>`

Input signal S-parameter s22.

`<gain>`

The gain value in dB scale, which can be a single value, two values separated by spaces in the “step number” format (sweep gain values from maximum gain value at a loss of “Step” (Gmax – Step) with “Number” times), or three values separated by a space in the format of “start stop step”. The maximum gain value is calculated with associated S-Parameter values at specified frequency values. For example, `Gmax=0.6`, `Step=0.1`, `Number=2`, gain values are `{0.5 0.4}`.

`<freq_list>`

Can be one single value or a list of frequency values separated by a space.

Errors

Possible error messages include:

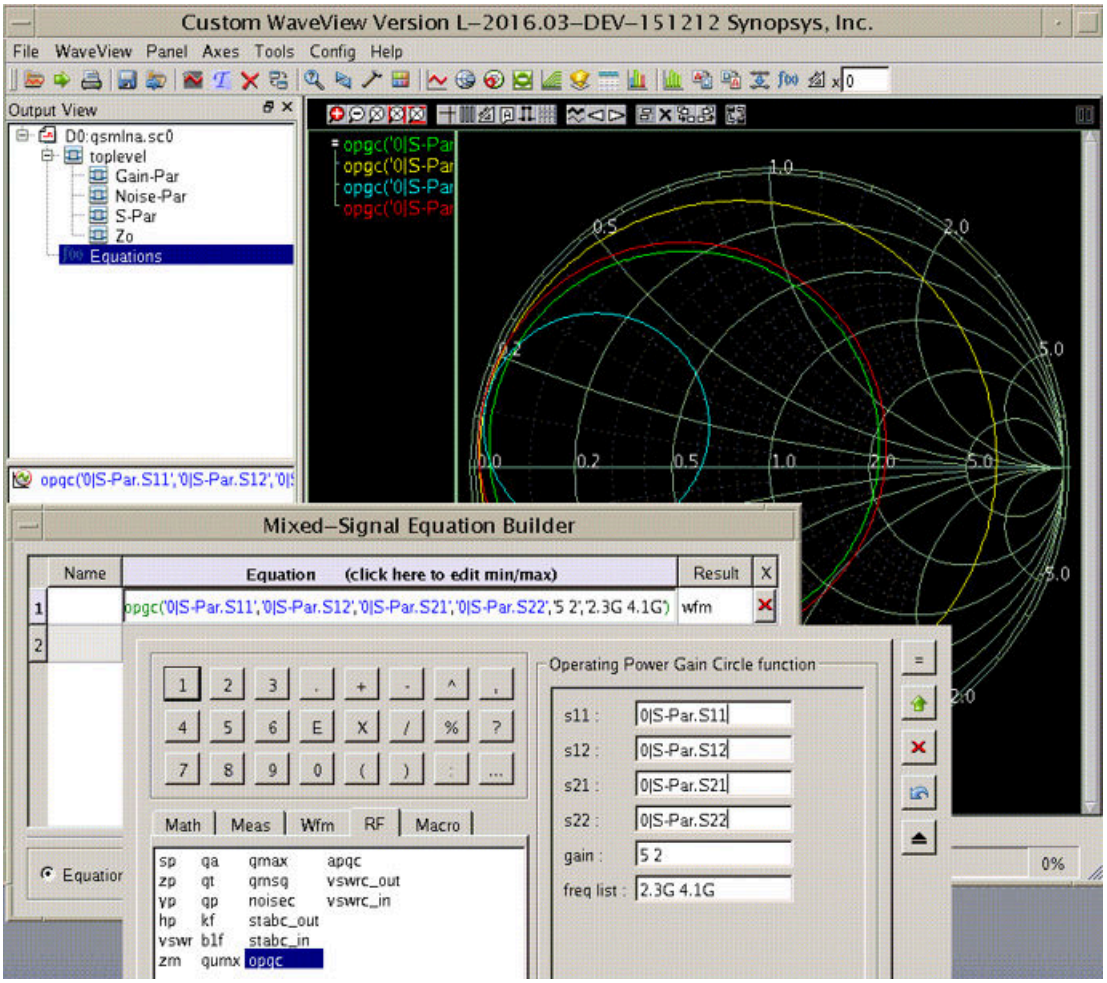
- Gp value is greater than Gpmax at trace#=’ ’, freq=’ ‘, Gp=’ ‘, Gpmax=’ ’
- Gpmax can’t be defined, use Gmsg at trace#=’ ‘, freq=’ ‘, K=’ ‘, |delta|=’ ‘”
- S11/S12/S21/S22 is not S-Parameter
- Zero again, |S21| is too small at trace#=’ ‘, freq=’ ‘, S21=(‘ ‘, ‘ ‘)”
- Zero again, |S21| is too small at freq=’ ‘, S21=(‘ ‘, ‘ ‘)”
- Zero again, |S12| is too small at trace#=’ ‘, freq=’ ‘, S12=(‘ ‘, ‘ ‘)”
- Zero again, |S12| is too small at freq=’ ‘, S12=(‘ ‘, ‘ ‘)”

Feedback

- All frequency values are not in the files
- Signal is not valid or cannot find the loaded signal

Examples

```
opgc('0|S-Par.S(1,1)','0|S-Par.S(1,2)','0|S-Par.S(2,1)','0|S-Par.S(2,2)'
,'5 2','2.3G 4.1G')
```



rn

Description

Normalized equivalent noise resistance vs. frequency waveform.

It is available on 2-port S-Parameter analysis.

Feedback

Syntax

```
rf('rn')
```

Returns

An analog signal.

Rn

Description

Equivalent noise resistance vs. frequency waveform. It is available on 2-port S-Parameter analysis.

Syntax

```
rf('Rn')
```

Returns

An analog signal.

---

sp

Description

The S-parameter function (scattering parameters) describes the electrical behavior of linear electrical networks when undergoing various steady state stimuli by electrical signals. For a generic multi-port network, the ports are numbered from 1 to N, where N is the total number of ports. For port n, the associated S-parameter definition is in terms of incident and reflected 'power waves', a_n and b_n respectively. The incident power wave for each port is as follows: $a = \frac{1}{2}k(V + Z_p I)$

The reflected wave for each port is defined as: $b = \frac{1}{2}k(V\text{-}Z_p I)$

Z_p is the diagonal matrix of the complex reference impedance for each port. Z_p^* is a conjugate of Z_p. V and I are the column vectors of the voltage and currents at each port.

An S-parameter matrix can be defined as where S is an NxN matrix containing elements that can be indexed using conventional matrix notation. $k = \left(\sqrt{|\Re\{Z_p\}|}\right)^{-1} b = Sa$

For two S-Parameters, the following S-parameter matrix $\begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} S_{11} & S_{12} \\ S_{21} & S_{22} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$ and $b_1 = S_{11}a_1 + S_{12}a_2; b_2 = S_{22}a_2$ equation applies.

Feedback

Syntax

```
rf('sp',<port1>,<port2>)
```

Returns

A waveform or a scalar value.

Arguments

`<port1>`

The first 1-base port index.

`<port2>`

The second 1-base port index.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of S parameter size (no S parameter found)

Examples

```
rf('sp',1,1)
```

---

spm

Description

The differential S-parameter function (scattering parameters).

Syntax

```
rf('spm',<port1>,<port2>,[<port1_instance>],[<port2_instance>])
```

Returns

A waveform or a scalar value.

Arguments

`<port1>`

The first 1-base port index.

Feedback

`<port2>`

The second 1-base port index.

`[<port1_instance>]`

(Optional) Type of S-parameter to be measured for the first port. Options are:

- `C`: Common
- `D`: Differential
- `S`: Single-ended

`[<port2_instance>]`

(Optional) Type of S-parameter to be measured for the second port. Options are:

- `C`: Common
- `D`: Differential
- `S`: Single-ended

---

ssb

Description

Returns the source stability circles.

Syntax

```
rf('ssb', start_freq, stop_freq, step_freq)
```

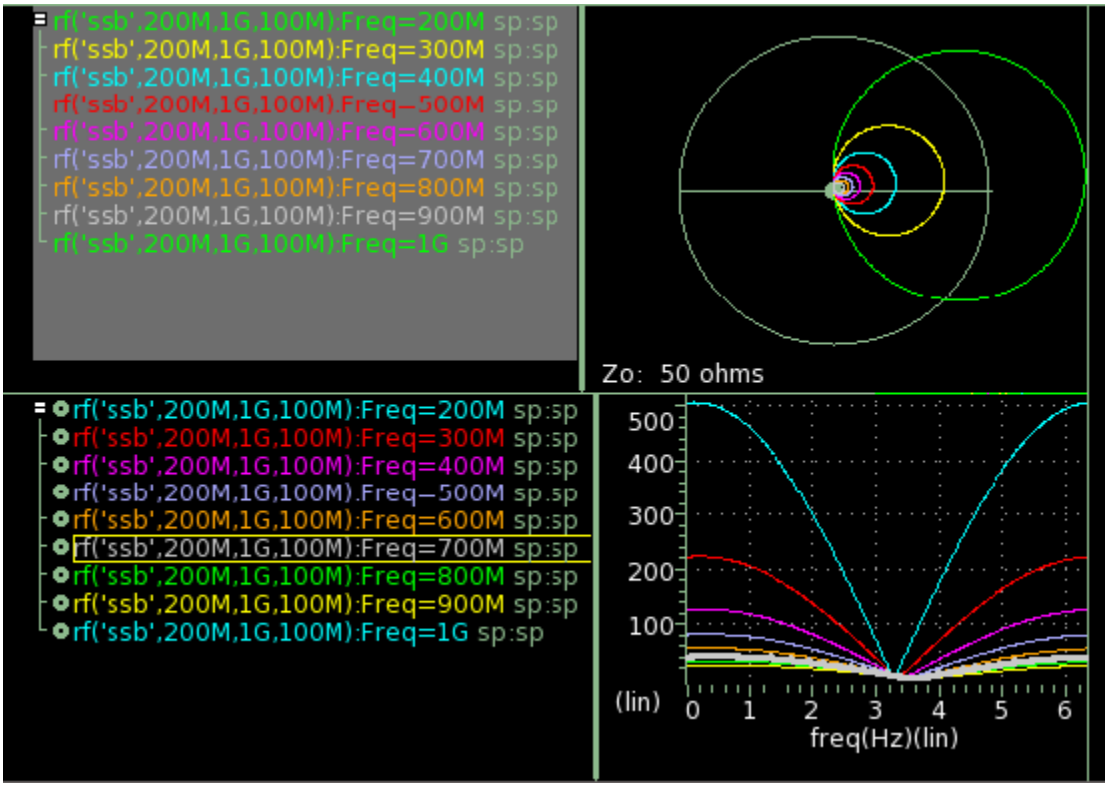
Returns

A set of circles.

Examples

```
rf('ssb',200M,1G,100M)
```

Feedback



stabc_in

Description

Calculates the Input stability circles over S-parameters, (S11, S12, S21, and S22), at the specified frequency values. The circles are implemented as a sweep signal with frequency as a parameter if multiple frequency values are specified.

The resulting waveform can be one circle if the waveform file is not a sweep waveform file and only one frequency value is input; otherwise, the resulting waveform can be multiple circles.

Syntax

```
stabc_in(<s11>,<s12>,<s21>,<s22>,<freq_list>)
```

Returns

A set of circles.

Feedback

Arguments

`<s11>`

Input S-parameter s11.

`<s12>`

Input signal S-parameter s12.

`<s21>`

Input signal S-parameter s21.

`<s22>`

Input signal S-parameter s22.

`<freq_list>`

Can be one single value or a list of frequency values separated by a space.

Errors

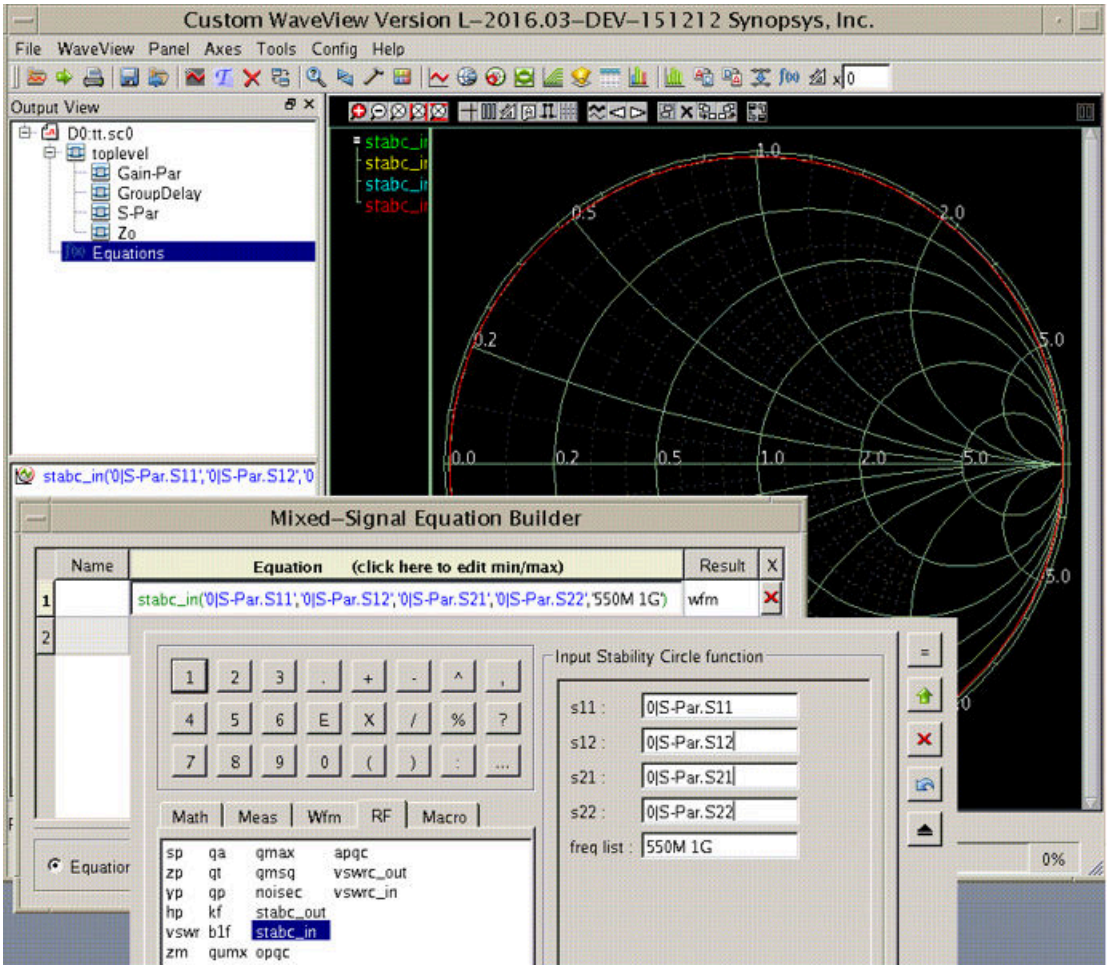
Possible error messages include:

- All frequency values are not in the file
- The circuit is unconditionally stable at trace#=", freq=' ', K=' ', |delta="|
- Signal is not valid or cannot find the loaded signal
- Invalid frequency inputs

Examples

```
stabc_in('0|S-Par.S(1,1)','0|S-Par.S(1,2)','0|S-Par.S(2,1)','0|S-Par.S(2,
2)','550M 1G')
```

Feedback



stabc_out

Description

Calculates the output stability circles over S-parameters, (S11, S12, S21 and S22), at the specified frequency values. The circles are implemented as a sweep signal with frequency as a parameter if multiple frequency values are specified.

The resulting waveform can be one circle if the waveform file is not a sweep waveform file and only one of the frequency values is input; otherwise, the resulting waveform can be multiple circles.

Syntax

```
stabc_out(<s11>,<s12>,<s21>,<s22>,<freq_list>)
```

Feedback

Returns

A set of circles.

Arguments

`<s11>`

Input S-parameter s11.

`<s12>`

Input signal S-parameter s12.

`<s21>`

Input signal S-parameter s21.

`<s22>`

Input signal S-parameter s22.

`<freq_list>`

Can be one single value or a list of frequency values separated by a space.

Errors

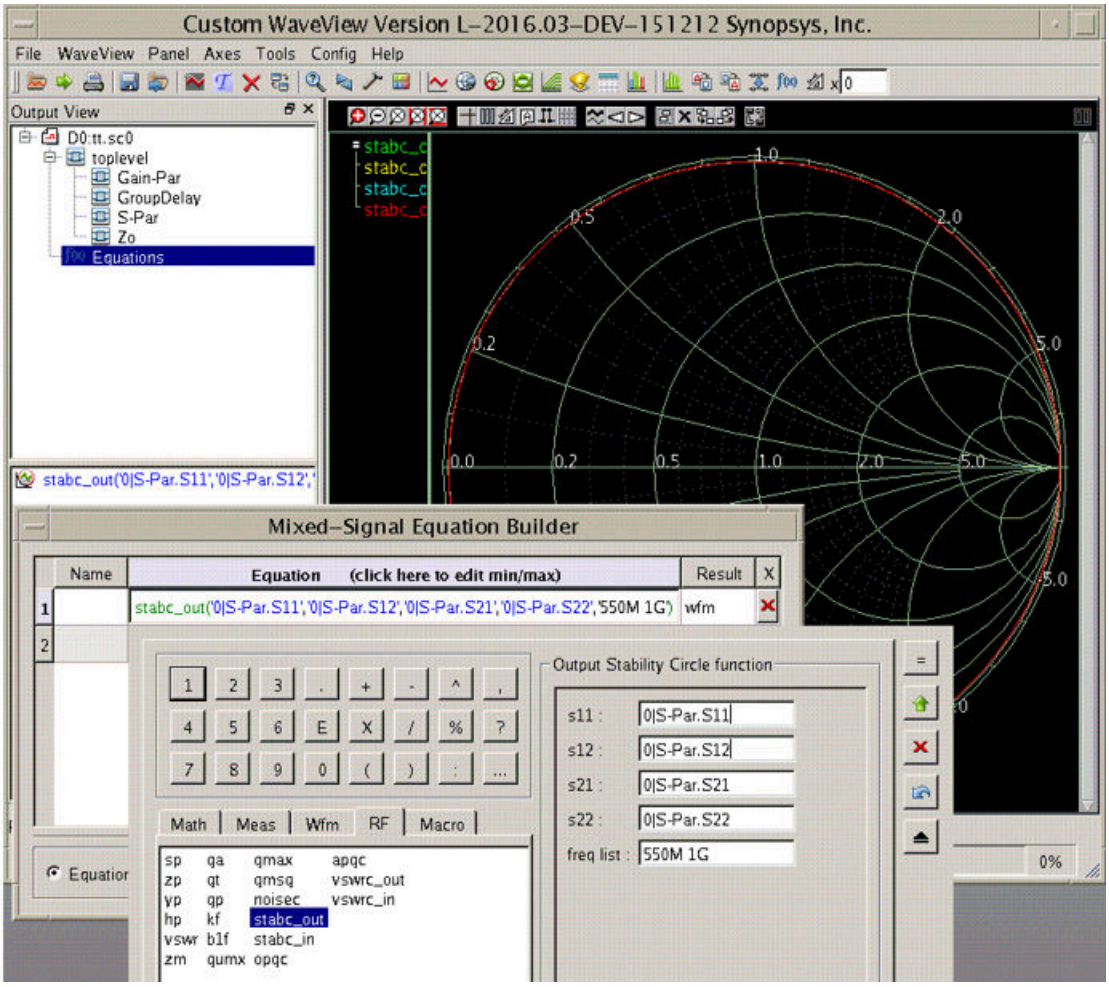
Possible error messages include:

- All frequency values are not in the file
- The circuit is unconditionally stable at trace#=", freq=' ', K=' ', |delta="|
- Signal is not valid or cannot find the loaded signal

Examples

```
stabc_out('0|S-Par.S(1,1)','0|S-Par.S(1,2)','0|S-Par.S(2,1)',
     '0|S-Par.S(2,2)','550M 1G')
```

Feedback



vswr

Description

Measures the voltage standing wave ratio. The algorithm is: $VSWR = \frac{1+|\Gamma|}{1-|\Gamma|}$

where Γis a reflection coefficient $\Gamma = \frac{V_r}{V_f}$.

V_r is a complex amplitude of the reflected wave and V_f is a complex amplitude of a forward wave.

Syntax

```
rf('vswr',<port1>)
```

Feedback

Returns

A waveform or a scalar value.

Arguments

`<port1>`

The first 1-base port index.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of VSWR size (no VSWR found)

Examples

```
rf('vswr',2)
```

---

vswrc_in

Calculates the Input voltage standing wave ratio cycle circles calculated over S-parameter values, as well as the input value of voltage standing wave ratios at specified frequency values. The circles are implemented as sweep signals with frequency as a parameter if multiple frequency values are input.

Syntax

```
vswrc_in(<s11>,<s12>,<s21>,<s22>,<input_vswr>, <freq_list>)
```

Returns

A set of circles.

Arguments

`<s11>`

Input S-parameter s11.

`<s12>`

Input signal S-parameter s12.

`<s21>`

Input signal S-parameter s21.

Feedback

`<s22>`

Input signal S-parameter s22.

`<input_vswr>`

The input value of voltage standing wave ratio.

`<freq_list>`

Can be one single value or a list of frequency values separated by a space.

Errors

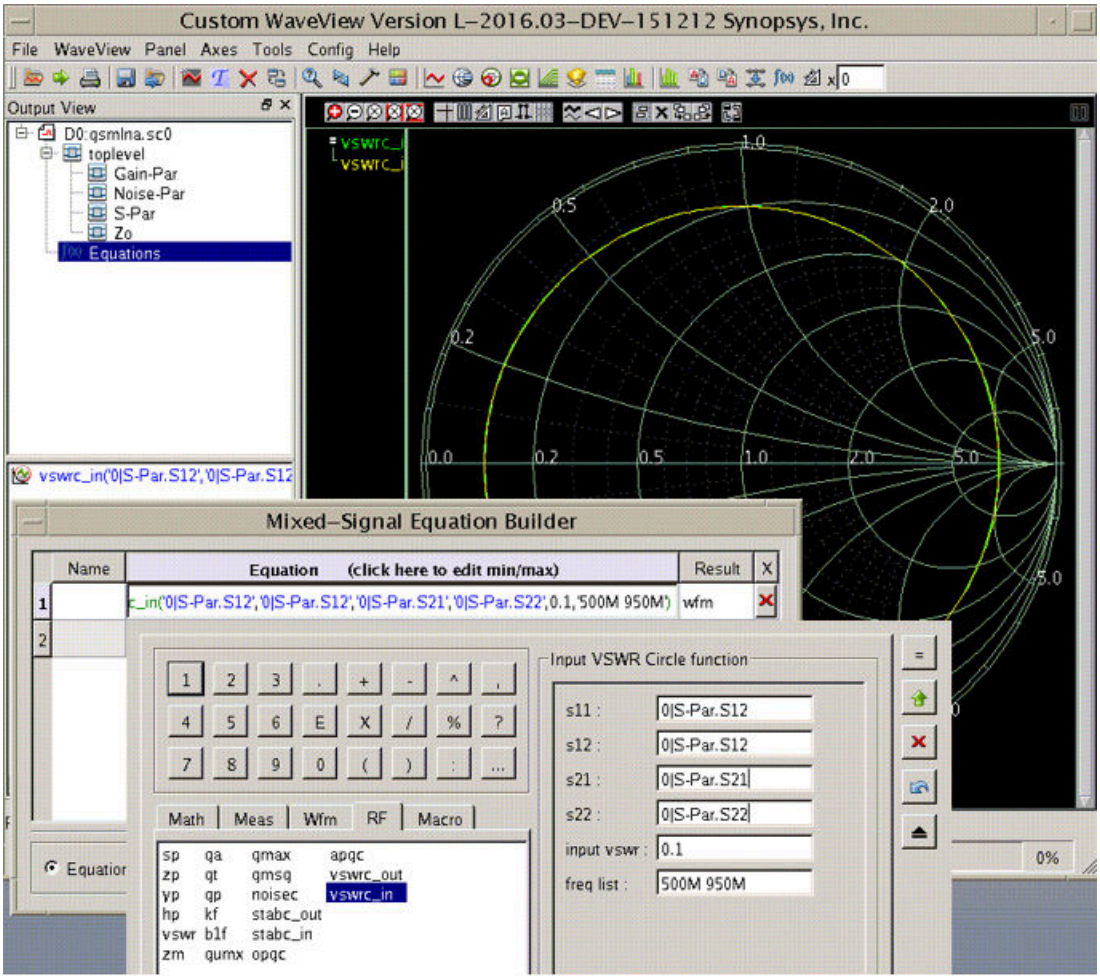
Possible error messages include:

- All frequency values are not in the file

Examples

```
vswrc_in('0|S-Par.S(1,1)','0|S-Par.S(1,2)','0|S-Par.S(2,1)','0|S-Par.S(2,
2)',0.1,'500M 950M')
```

Feedback



vswrc_out

Calculates the output voltage standing wave ratio cycle circles over S-parameter values, as well as the output value of a voltage standing wave ratio at specified frequency values. The circles are implemented as sweep signals with frequency as parameter if multiple frequency values are input.

Syntax

```
vswrc_out(<s11>,<s12>,<s21>,<s22>,<output_vswr>, <freq_list>)
```

Returns

A set of circles.

Feedback

Arguments

`<s11>`

Input S-parameter s11.

`<s12>`

Input signal S-parameter s12.

`<s21>`

Input signal S-parameter s21.

`<s22>`

Input signal S-parameter s22.

`<output_vswr>`

The output value of voltage standing wave ratio.

`<freq_list>`

Can be one single value or a list of frequency values separated by a space.

Errors

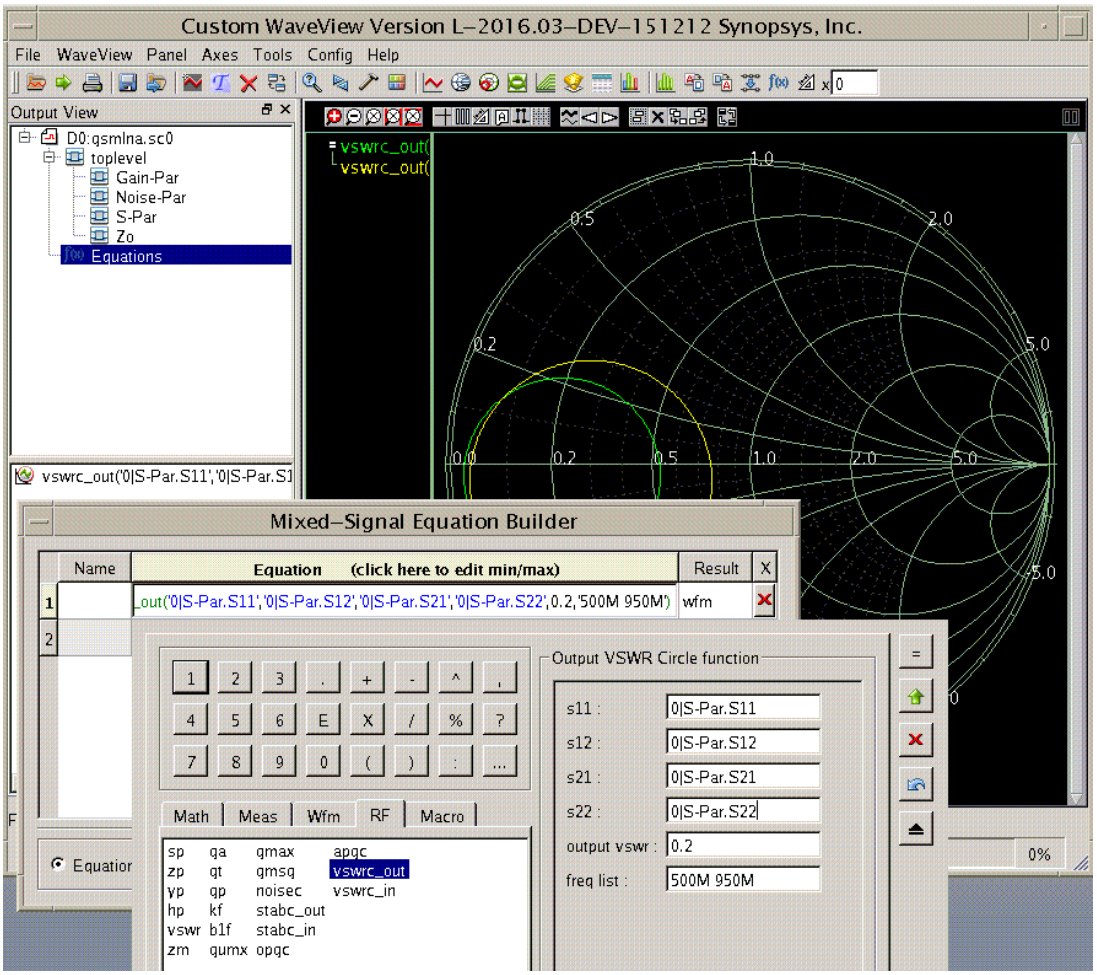
Possible error messages include:

- All frequency values are not in the file

Examples

```
vswrc_out('0|S-Par.S(1,1)','0|S-Par.S(1,2)',
     '0|S-Par.S(2,1)','0|S-Par.S(2,2)',0.2,'500M 950M')
```

Feedback



ym

Y parameter at port1 with all other ports matched. The formula is as follows: `ym=(1-s)/(1+s)/Z0`

Syntax

```
rf('ym',<port1>)
```

Returns

A waveform or a scalar value.

Arguments

`<port1>`

The 1-base port index.

Feedback

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of YM size (no YM found)

Examples

```
rf('ym',2)
```

---

yp

Description

The Y parameter function (admittance parameters) are used in electrical engineering, electronic engineering, and communication systems engineering. This function describes the electrical behavior of linear electrical networks and also describes the small-signal response of non-linear networks.

Two-port networks, the relationship between the port currents, port voltages, and the Z-parameter matrix is given by: $\begin{bmatrix} I_1 \\ I_2 \end{bmatrix} = \begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \end{bmatrix}$

where $Y_{11} = \frac{I_1}{V_1}\Big|_{V_2=0}$, $Y_{12} = \frac{I_1}{V_2}\Big|_{V_1=0}$, $Y_{21} = \frac{I_2}{V_1}\Big|_{V_2=0}$, $Y_{22} = \frac{I_2}{V_2}\Big|_{V_1=0}$.

Relation to S-parameters (two port): $Y_{11} = \frac{((1-S_{11})(1+S_{22})+S_{12}S_{21})}{\Delta s} Y_0 \quad Y_{12} = \frac{-2S_{12}}{\Delta s} Y_0 \quad Y_{21} = \frac{-2S_{12}}{\Delta s} Y_0 \quad Y_{22} = \frac{((1+S_{11})(1-S_{22}$

where $\Delta s = (1-S_{11})(1-S_{22})-S_{12}S_{21}$

Syntax

```
rf('yp'<port1>,<port2>,<impedance>)
```

Returns

A waveform or scalar value.

Arguments

`<port1>`

The first 1-base port index.

Feedback

`<port2>`

The second 1-base port index.

`<impedance>`

The port characteristic impedance.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of Y parameter size (no Y parameter found)

Examples

```
rf ('yp',2, 1, 40
```

---

zm

Z parameter at port1 with all other ports matched.

Syntax

`rf('zm',<port1>)`

Returns

A waveform or a scalar value.

Arguments

`<port1>`

The 1-base port index.

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of ZM size (no ZM found)

Feedback

Examples

```
rf('zm',2)
```

---

zp

Description

The Z parameter function (impedance parameters) are used in electrical engineering, electronic engineering, and communication systems engineering. This function describes the electrical behavior of linear electrical networks and also describes the small-signal response of non-linear networks.

Two-port networks, the relationship between the port currents, port voltages, and the Z-parameter matrix is given by: $\begin{bmatrix} V_1 \\ V_2 \end{bmatrix} = \begin{bmatrix} Z_{11} & Z_{12} \\ Z_{21} & Z_{22} \end{bmatrix} \begin{bmatrix} I_1 \\ I_2 \end{bmatrix}$

where $Z_{11} = \frac{V_1}{I_1}\Big|_{I_2=0}, Z_{12} = \frac{V_1}{I_2}\Big|_{I_1=0}, Z_{21} = \frac{V_2}{I_1}\Big|_{I_2=0}, Z_{22} = \frac{V_2}{I_2}\Big|_{I_1=0}.$

Relation to S-parameters (two port): $Z_{11} = \frac{((1+S_{11})(1-S_{22})+S_{12}S_{21})}{\Delta s} Z_0 Z_{12} = \frac{2S_{12}}{\Delta s} Z_0 Z_{12} = \frac{2S_{12}}{\Delta s} Z_0 Z_{22} = \frac{((1+S_{11})(1-S_{22})}{\Delta s}$

where $\Delta s = (1-S_{11})(1-S_{22})-S_{12}S_{21}$

Syntax

```
rf('zp'<port1>,<port2>,<impedance>)
```

Returns

A waveform or a scalar value.

Arguments

`<port1>`

The first 1-base port index.

`<port2>`

The second 1-base port index.

`<impedance>`

The port characteristic impedance.

Feedback

Errors

Possible error messages include:

- Data file has no data point
- The waveform file type is invalid (the file type can only be PSB, PSA, COU, JWDB, ADS, Table)
- Input value of port is out the range of Z parameter size (no Z parameter found)

Examples

```
rf('zp',2,1,50)
```

8

Working With Designs

This chapter details working with designs in the WaveView tool.

Note:

A WaveView ADV license is required for many of the features accessible from the **Design** tab.

This section contains information on the following topics:

- Importing a Design
- Applying RC Reduction to a Design
- Using DesignView
- Using the Current Debug Tool
- Using SourceView
- Using ConnectionView
- Using SPICE Lint
- Annotating a Netlist
- Cosim

Importing a Design

Note:

A WaveView ADV license is required for this feature.

Designs can be imported in the GUI or from the command line.

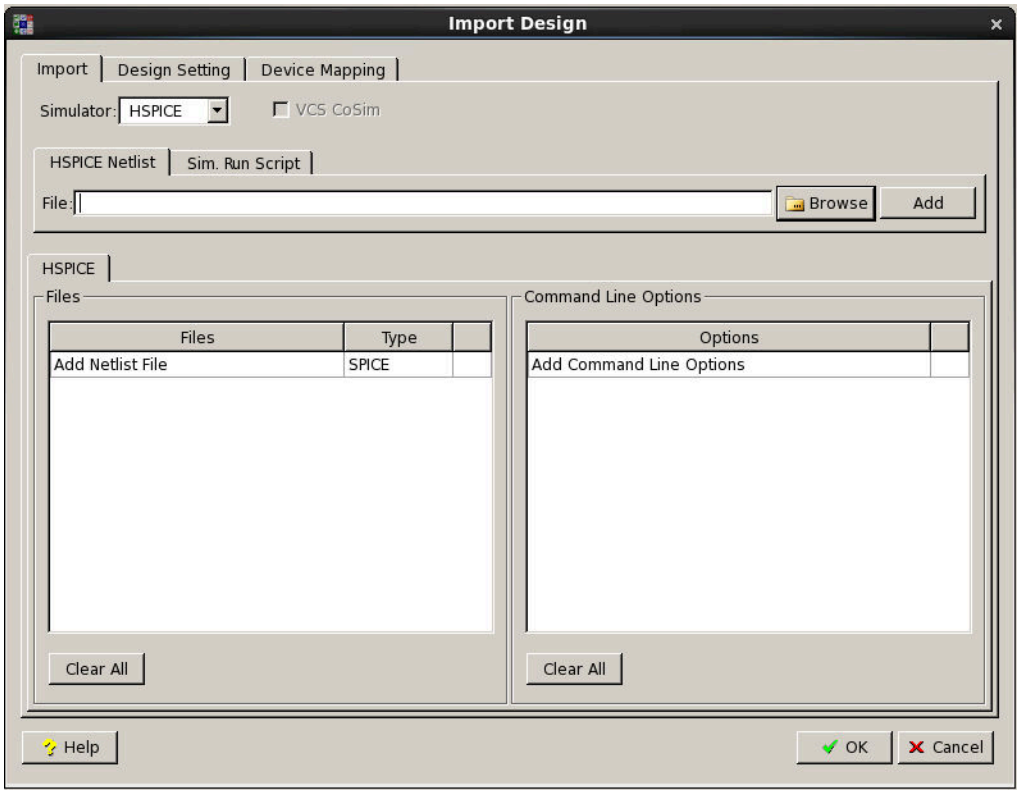
To import your design from the command line, see Invoking the WaveView Tool With a Design or Netlist.

Feedback

To import your design from the GUI:

1. Choose **Import Design** from the **File** or **Design** tabs.

   The Import Design window opens.



2. Choose the simulator from which your design originates.

   Note:

   While Spectre® is a supported simulator, it is not displayed in the Simulator menu by default. To add Spectre to the Simulator menu, add the following line to your .cxinit file:

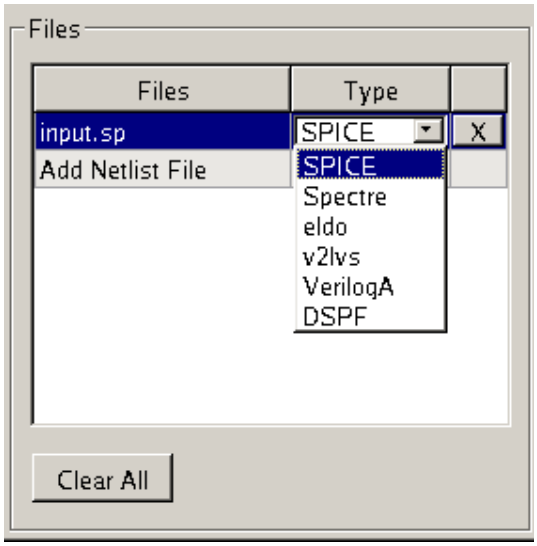
   ```
   cx_simulator_list -add spectre
   ```

3. Add netlist files.

   To add a netlist file, enter the path name of the netlist file in the File text box and click **Add**, or click **Browse** to locate one. The file is added to the list of netlist files.

   If necessary, you can change the netlist type by clicking in the Type column next to a netlist file name and choosing from the menu of file types that appear.

Feedback



4. Add simulator setup files.

   To add a simulator setup file, click the Sim. Run Script tab, and either enter the path name of the setup file or browse to its location. Click **Import** to import the setup file.

5. Add any command-line options in the Command Line Options table.

   See Supported Command-Line Options for a list of available command-line options.

6. Use the **Design Settings** tab to set:
   - Power and ground signals
   - RC Reduction
   - Recognize CMOS gates
   - Merge parallel MOS

7. Use the **Device Mapping** tab to open and save map files or load subciruits from a file.

8. Click **OK** to import your design.

Importing Scripts for VCS 3 Step Flow

For VCS 3 step flow, use the **Import Design** dialog to import both .compile and .elaborate scripts. First, import vcs.compile, then import vcs.elaborate.

Feedback

Updating a Design

To update a design when imported netlist files are modified outside of the WaveView tool, select the name of a design, and choose **Update** from the **Design** tab. The design is recompiled, and any tests are updated.

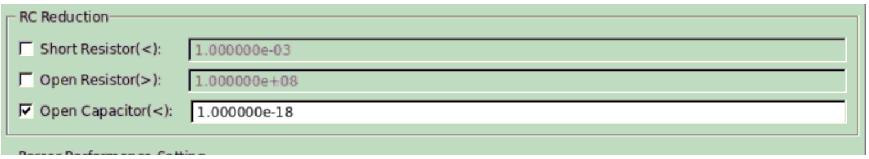
Removing a Design

To remove a design, click the design name, and then click **Remove** on the **Design** tab.

Applying RC Reduction to a Design

To apply RC reduction to your design:

1. Click **Import Design** in the **Design** section of the **Design** tab, then select the **Design Settings** tab.



2. Click the **Short Resistor (<)**, **Open Resistor (>)**, or **Open Capacitor (<)** check boxes to set RC reduction values for those types, then enter the reduction value in the associated text field.
3. Click **OK** to save your settings.

   The RC reduction you specify is applied to your design.

Using DesignView

This section contains information on the following topics:

- Opening DesignView
- Traversing Designs
- Searching Designs
- Generating Reports
- Generating a Flattened Netlist
- Extracting Measure Statements

Feedback

- Extracting PWL Source
- Converting Stimuli

Opening DesignView

To open the DesignView window, click the name of a netlist or design in the Project Hierarchy pane of the Design Import window, and click **DesignView** from the **Design** tab. You can also right-click the name of a netlist or design in the Project Hierarchy pane, and choose **Open DesignView** from the menu that opens.

Traversing Designs

This section contains information on the following topics:

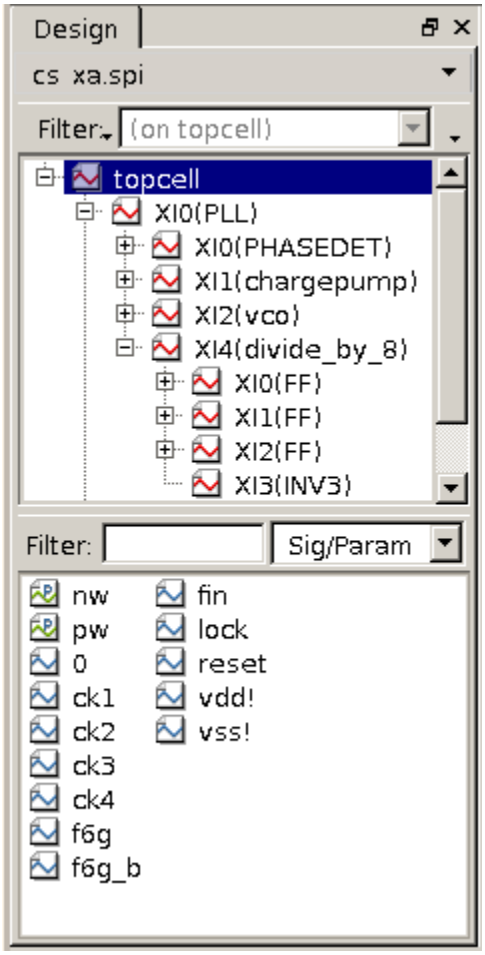
- Viewing the Design Browser
- Design Display Settings
- Viewing Design Hierarchy
- Viewing Design Cells
- Viewing Design Files
- Filtering Signals, Elements, and Files

Viewing the Design Browser

When you click on a cell in the Design Browser, which is located in the DesignView window, the following corresponding design elements are displayed:

- Any associated signals, which are listed the bottom half of the Design Browser under the Signal heading
- The associated design netlist with the selected cell portion positioned at the top of the SourceView window
- The subcircuit connections for the selected cell in the ConnectionView pane
- Double-clicking a cell shows the full connection of the cell in the current view (Full Connection mode is activated)

Feedback



Design Display Settings

The **Browser** section of the **Design** tab selects which parts of your design to display in the File Browser:

- **Hierarchy**

  Displays the full hierarchy of your design.

- **Cells**

  Displays the cells of your design.

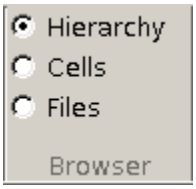
- **Files**

  Displays the files associated with your design.

Feedback

Viewing Design Hierarchy

To view the design hierarchy, click the **Hierarchy** option on the ribbon. The Design Hierarchy Browser displays the hierarchy tree.



Changing the Design Hierarchy Name Display

To change the design hierarchy name display:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.

2. Click the **DesignView** category on the left.

3. Choose one of the following display types from the **DesignView Hierarchy Name Display** section, which controls how instances appear in the Design Hierarchy:

   - **Instance(Module) Type**
   - **Instance(Module)**
   - **Instance**

4. Click **Apply** to save your settings for just this session, or click **Apply-Save** to save your preference settings for future sessions.

Filtering Design Hierarchy Instances

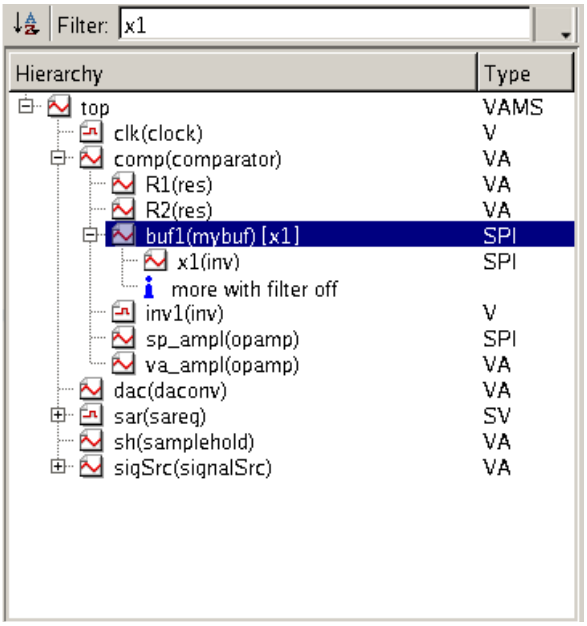
Before you filter any instances in the Design Hierarchy Browser, consider the following filter rules:

- A new filter always overwrites the previous filter for the same scope.
- Filter strings cannot be used to search within a scope; the text string is treated as a filter string for the currently selected scope.
- Filter strings are not case-sensitive.
- Asterisks (*) and question marks (?) can be used as a wildcard symbol as part of a filter string.

Feedback

To filter instances in the Design Hierarchy Browser:

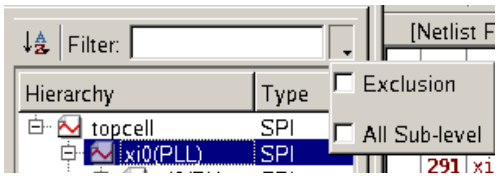
1. Click a scope level in the hierarchy tree.
2. Type the string you want to filter out in the Filter text box.



Other instances are filtered out of the current scope. The filtered name is displayed in brackets, which shows that a particular scope is filtered.

If you do not want to filter instances for a scope with a filtered tag, click that scope in the hierarchy list. The filtered string of the selected scope is displayed and modified or removed.

3. (Optional) Click the down arrow next to the Filter text box to change how the filter works.



Clicking the **Exclusion** check box filters out all instances that contain inputted text instead of displaying them.

Clicking the **All Sub-level** check box filters all sublevels of a selected node and adds tags for all sublevel instances in the hierarchy tree.

Feedback

Tracing Through all Hierarchy (in Full Connection Mode)

User tracing is allowed in Full Connection mode. Select a signal in the current view and click the **Trace Through All Hierarchy** button  to open a new side-by-side view and trace the selected signal's full hierarchy connection in the new view.

Viewing Design Cells

To view all the elements for the currently selected cells in a design, click the Cells tab, which is located at the bottom of the Design Browser in the DesignView window.

The Cell List displays all subcircuits and modules in design. Unused cells are listed, but library cells for digital designs are not. The element names and corresponding type are displayed in the Element List.

Viewing Design Files

To view the files that are associated with your design, click the Files tab, which is located at the far bottom of the Design Browser in the DesignView window. All files are listed in the Design Browser, and all cells in a selected file are listed in the Signal Browser.

The top area of the File Browser displays the imported file structure, and the bottom area displays cells that are defined in the currently selected file.

Signal tracing in the File Browser is limited to the scope of current module.

Filtering Signals, Elements, and Files

To find a signal, element, or file within the Signal, Element, or File Browser, enter the first few characters of the name in the Filter text box just above the Signal, Element, and File Browser. The full list of signals, elements, or files are filtered, and just those items that match the entered characters are displayed.

Searching Designs

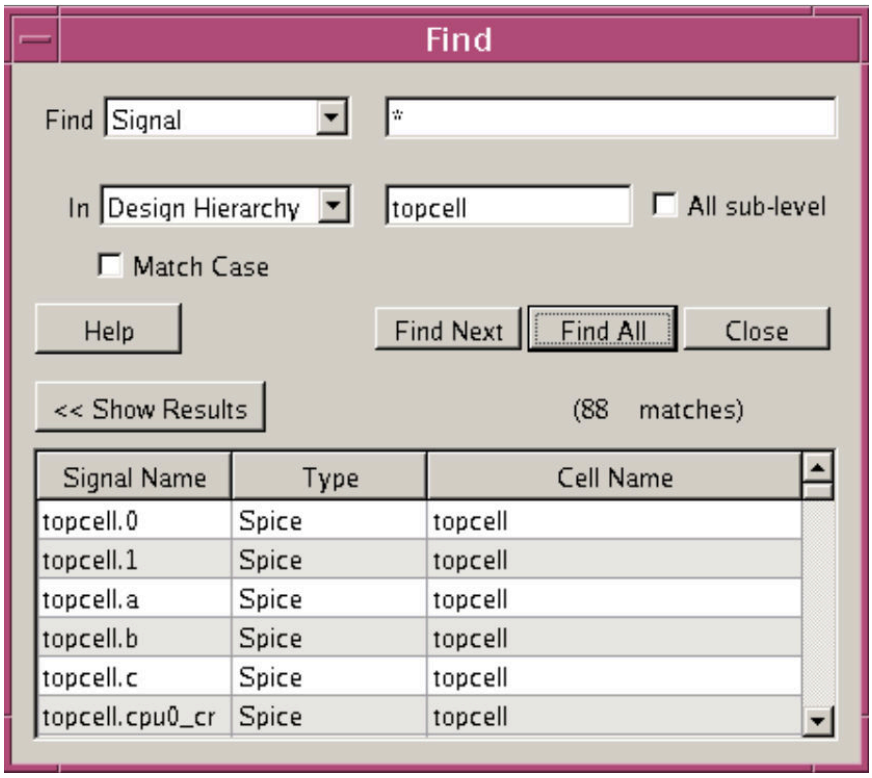
To search for a signal, instance, cell, or string in the DesignView:

1. Click the binoculars icon in the DesignView toolbar, or click **Search** on the **Design** tab.



The **Find** dialog box opens.

Feedback



2. Choose the type of item you want to search from the **Find** menu, and enter a search term in the text box.
3. Choose where you want to search from the **In** menu.
4. If you choose **Cell** or **String** from the **Find** menu, **All Imported Cells** or **All Imported Files** is selected by default in the **In** menu, respectively. Skip to the next step.

   Otherwise, If you choose **Signal**, **Instance**, or **Instance by Cell**, enter the signal or instance you want to find into the text box just to the right of the **In** menu. `toplevel` is the default.

   Also, if you are searching in the **Design Hierarchy**, you can click the **All sub-level** check box to find a match in the current level and all sublevels of the design hierarchy.
5. Click the **Find Next** button to find the next occurrence of the search term.

   The next match to the search term is highlighted in the DesignView and added to the search results.

   If you click **Find All**, all the matched search terms are added to the search results.

Feedback

6. (Optional) Click the **Show Results** button to display a list of search results.

   Each time a search term is matched, its location is added to the list of results. Clicking a search result also highlights the result in the DesignView.

Generating Reports

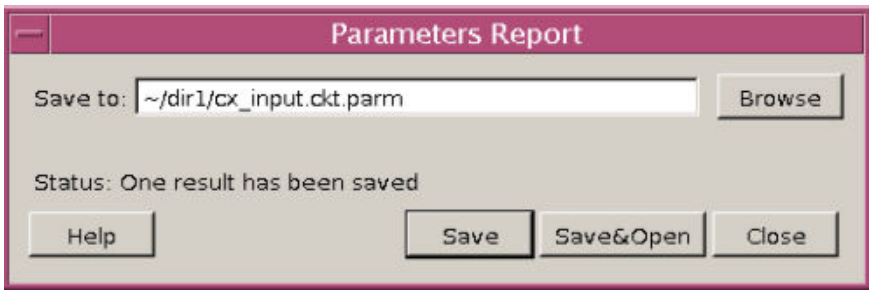
This section contains information on the following topics:

- Param Report
- W/L Report

Param Report

To generate a parameter report for all hierarchy levels:

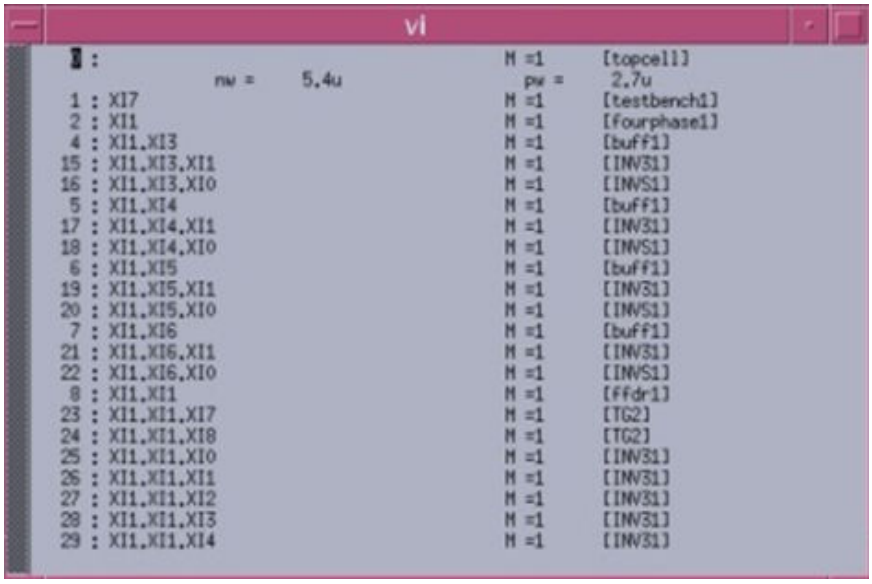
1. Ensure you have a DesignView open (click **DesignView** on the **Design** tab).
2. Choose **Param Report** from the menu in the **Utilities** section of the **Design** tab.



   The **Parameters Report** dialog box opens.

3. Enter the path to where you want to save the parameters report in the **Save To** text box, or browse to a location.
4. Click **Save** to create a parameter report in the selected location.

   You can also click **Save & Open**, which creates the parameter report and opens it in the VI editor (or other specified text editor).



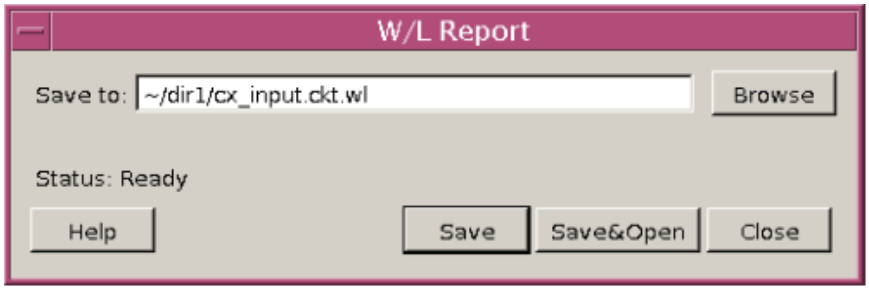
The following table explains the information each column contains in this sample parameter report file:

Report Column	Description
1	Index of instances.
2	Name of the full scope.
3	Parameters and corresponding values. For example, where `nw=5.4u` and `pw=2.7u` in the parameters report sample, `M=1` represents the count of the instance.
4	Subcircuit name of the instance.

W/L Report

To create a W/L report for your design:

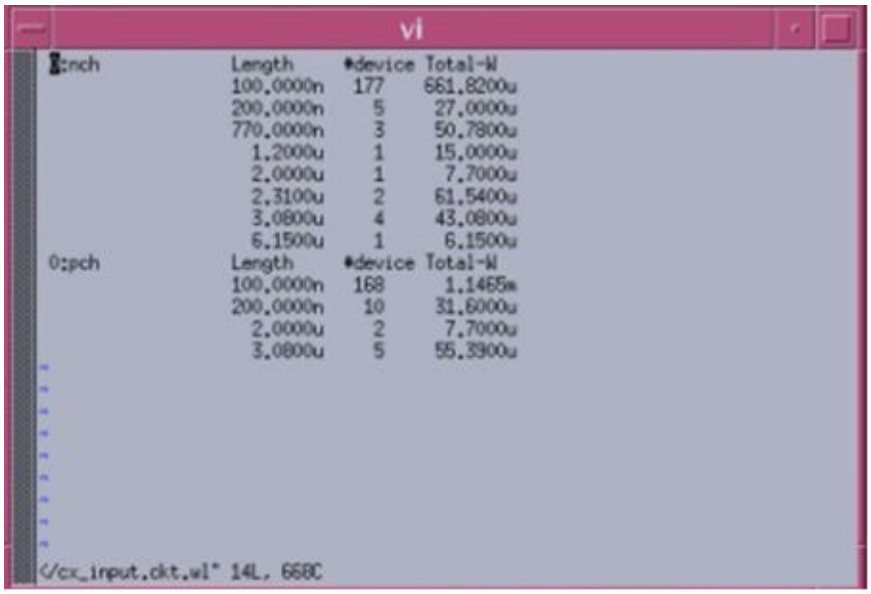
1. Ensure you have a DesignView open (click **DesignView** on the **Design** tab).
2. Choose **W/L Report** from the menu in the **Utilities** section of the **Design** tab.



The **W/L Report** dialog box opens.

3. Enter the path to where you want to save the W/L report in the **Save To** text box, or browse to a location.
4. Click **Save** to create a W/L report in the selected location.

   You can also click **Save & Open**, which creates the W/L report and opens it in the VI editor (or specified text editor).



The following table explains the information each column contains in this sample W/L report file:

Report Column	Description
1	Model name.
2	Various lengths.

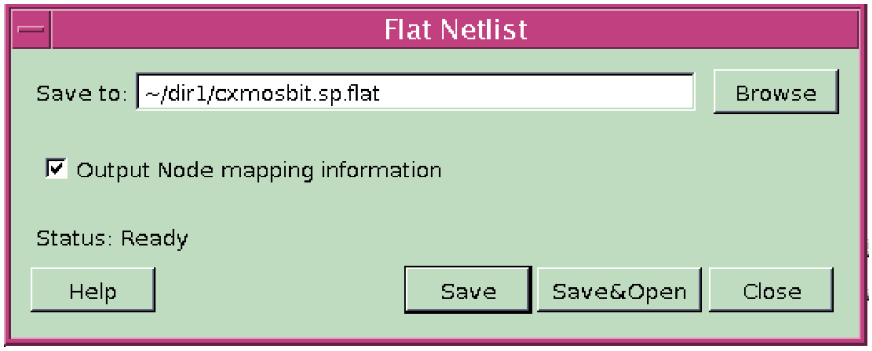
Feedback

Report Column	Description
3	Device numbers of each length.
4	Total width of each length.

Generating a Flattened Netlist

To generate a flat netlist from an original netlist:

1. Ensure you have a DesignView open (click **DesignView** on the **Design** tab).
2. Hold the **Shift** key, and drag the selection rectangle over the parts of the design you want to include in the flat netlist.
3. Choose **Top-Level Flat Netlist** from the menu in the **Utilities** section of the **Design** tab.



The **Flat Netlist** dialog box opens.

4. Enter the path to the original netlist in the Save To text box, or browse to its location.
5. (Optional) Unckeck the **Output Node mapping information** check box if you do not want to include output node mapping information in your flat netlist.

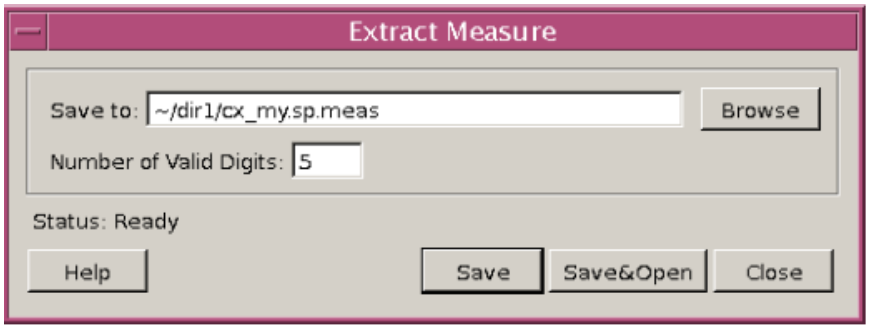
   Output node mapping information is included by default.
6. Click **Save** to flatten the netlist.

Extracting Measure Statements

To extract .MEASURE statements from a netlist file:

1. Ensure you have a DesignView open (click **DesignView** on the **Design** tab).
2. Choose **Extract Measure** from the menu in the **Utilities** section of the **Design** tab.

Feedback



The **Extract Measure** dialog box opens.

3. Enter the path to where you want to save the extracted .MEASURE statements in the Save To text box, or browse to a location.
4. (Optional) Enter the number of valid digits in the Number of Valid Digits text box. The default is 5.

   For example, if the value of parameter A is `3.2` and the number of valid digits is `5`, then parameter A is `3.2000` in the exported file.
5. Click **Save** to create a W/L report in the selected location.

   You can also click **Save & Open**, which creates the list and opens it in the VI editor.

   The extracted .MEASURE statements are exported to a text file, which you can apply to the corresponding simulation files. If the input netlist is a hierarchical netlist with .MEASURE statements embedded inside subcircuits, any parameter or output variable in these .MEASURE statements are resolved before being exported.

   The exported netlist file also contains all the necessary .PROBE statements for generating simulation output waveforms. These .PROBE statements are commented out in the extracted file.

   For example, when extracting .MEASURE statements from the following netlist:

```
.subckt sub1 1 3
R1 1 2 1K
R2 2 3 1K
.MEASURE avg_value AVG PAR('V(2)*A')
.ends
X1 1 0 S1
.param A=3
```

   The result is:

```
.MEASURE X1.avg_value AVG PAR('V(x1.2)*3')
*.PROBE V(X1.2)
```

Feedback

The extracted netlist no longer depends on any parameters since all parameters values are resolved in the exported netlist. If an output variable does not exist in the input netlist, the exported netlist uses its original variable name. For example, when the following netlist is exported:

```
* VDD does not exist in the netlist
.MEASURE avg_cur AVG I(VDD)
```

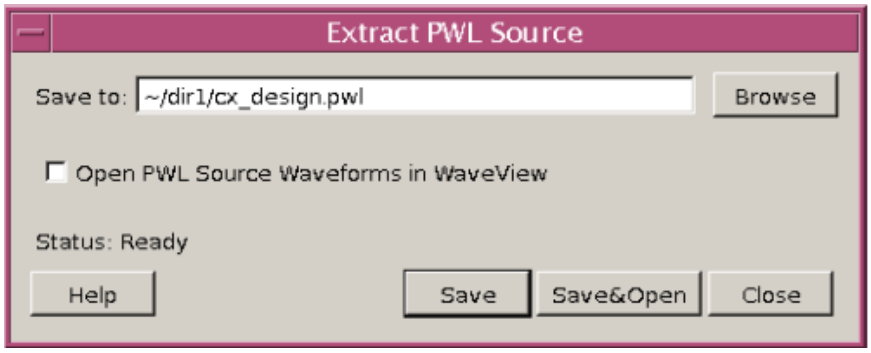
The result is:

```
.MEASURE avg_cur AVG I(VDD)
```

Extracting PWL Source

To extract and save the PWL source from your design:

1. Ensure you have a DesignView open (click **DesignView** on the **Design** tab).
2. Choose **Extract PWL Source** from the menu in the **Utilities** section of the **Design** tab.



The **Extract PWL Source** dialog box opens.

3. Enter the location for the PWL data to be saved, or browse to and select a directory.

   PWL data is saved in your working directory by default.

4. (Optional) Click the **Open PWL Source Waveforms in WaveView** check box to also view the PWL source in the WaveView tool.
5. Click **Save** to save the PWL data, or click **Save & Open** to both save and view the PWL data.

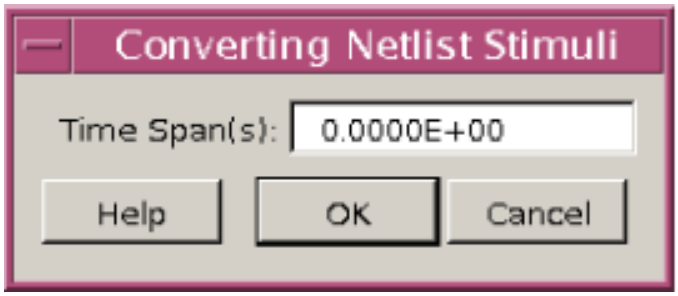
   The PWL data is saved in the specified location, and the data is displayed in the WaveView tool if the **Open PWL Source Waveforms in WaveView** check box is checked.

Feedback

Converting Stimuli

To convert stimuli in a netlist to a special waveform file, which contains waveforms of all of the input voltage or current sources:

1. Ensure you have a DesignView open (click **DesignView** on the **Design** tab).
2. Choose **Convert Stimuli** from the menu in the **Utilities** section of the **Design** tab.



The **Converting Netlist Stimuli** dialog box opens.

3. Enter the simulation Time Span value, which specifies the maximum time.
4. Click **OK** to convert the netlist stimuli.

The output file contains PWL syntax. For example:

```
V1 V PWL DC=NA
+0 400m
+1n 2.8
+7.5n 2.8
+8.5n 400m
+15n 400m R=0 TD=0
```

Using the Current Debug Tool

For each signal in DesignView, the Current Debug tool can be invoked to display a dialog for the current of all of the signal's connected ports. In this dialog, you can set time or time range, sort, and filter results.

The following sections detail using the Current Debug tool:

- Invoking the Current Debug Tool
- Viewing the Current Debug Dialog Box
- Single Time Point Current vs. Average Current

Feedback

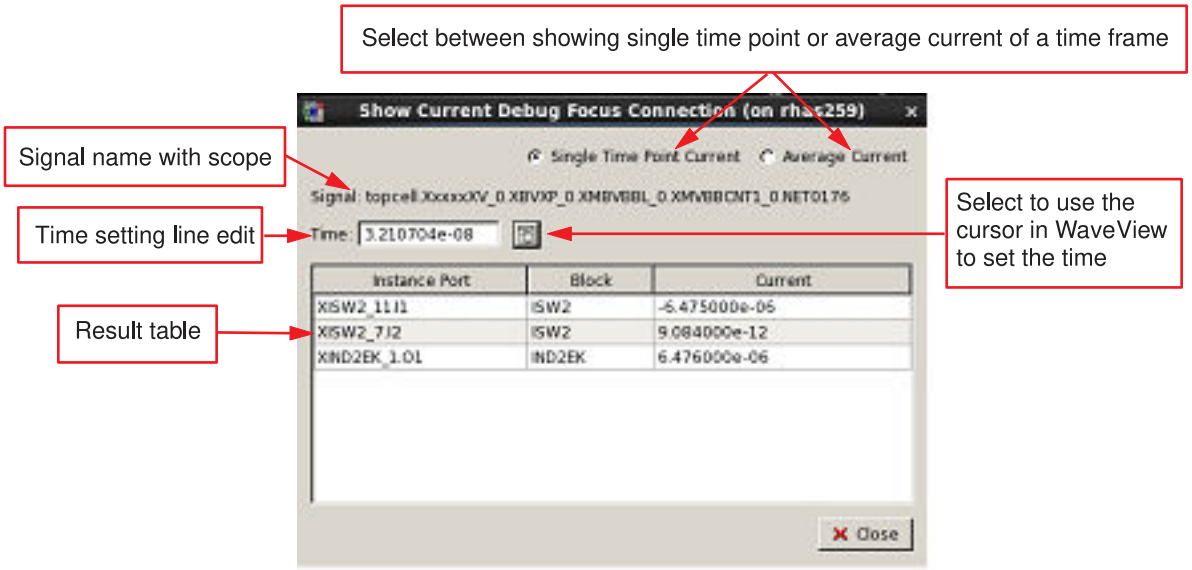
- Setting Time
- Additional Options

Invoking the Current Debug Tool

From Connection View, right-click on a net or port and select **Current Debug on This Signal**.

Viewing the Current Debug Dialog Box

The following figure details the Current Debug dialog box:

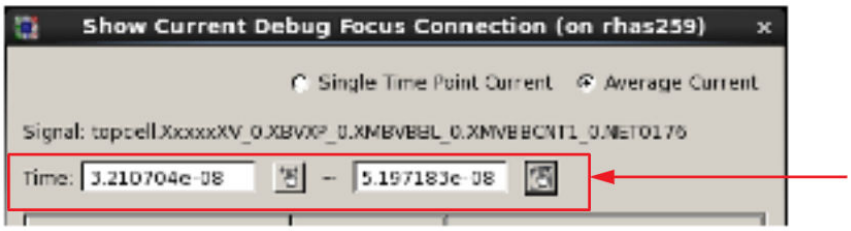


Single Time Point Current vs. Average Current

You can choose to show one time point current or time range average current by selecting the corresponding radio box at the top of the dialog window. When choosing **Average**

Feedback

Current, an additional Time setting line and **cursor selection** button are shown, as follows:



Setting Time

You can set time in the dialog window in one of the following ways:

- Input time manually — input the time value in the Time field and press the **Enter** key to update the results.
- Select time with the cursor in WaveView — click the **cursor selection** button and in WaveView, either select or create a new cursor to set the time value. The selected cursor is bound to this time setting; if the cursor is moved, the results are updated.

Additional Options

The following options are also available in the Current Debug dialog window:

- Sort the result table — right-click on a column head in the results table and choose **Sort by Ascend** or **Sort by Descend**.
- Filter the result table by current value — right-click on the current value column and select **Filter** to open a new dialog box where you can input the threshold value. Any current items smaller than this value are removed from the results table.
- Sync highlight port in Connection View — if you select one port in the table, the Connection View will automatically focus to this port and highlight it in the graph. If the port does not exist in the current Connection View, the tool will open a new DesignView to show this port's instance and highlight this port.
- Sync signal from Connection View — when you select another signal or port from the current Connection View, the Current Debug dialog is automatically updated to show new signal's connection result.

Using SourceView

Check the **SourceView** option in the **Show** section of the **Design** tab to display the SourceView pane.

Feedback

The following topics are available in this section:

- Using Netlists
- Finding Signal Connections
- Traversing Connections
- Tracing Connections
- Displaying Trace Connections
- Suppressing Out of Scope Warnings

Using Netlists

This section contains information on the following topics:

- Using Bookmarks in your Netlist
- Searching Netlists
- Hiding Netlist Line Numbers

Using Bookmarks in your Netlist

You can set bookmarks in your netlist so that you can quickly refer back to specific parts.

This following topics are available in this section:

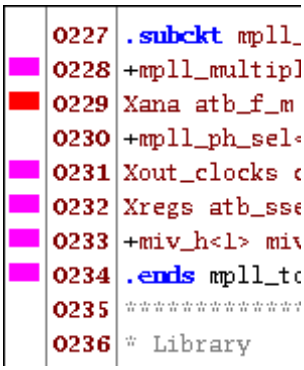
- Setting Bookmarks
- Traversing Bookmarks
- Removing Bookmarks

Setting Bookmarks

To set a bookmark, click in the column to the left of the lines in a netlist.

Feedback

A red rectangle appears at the beginning of a line that is bookmarked and currently selected. All other lines that contain that a bookmark, but are not currently selected, are marked with a pink rectangle. For example:



Traversing Bookmarks

To move from one set bookmark to the next, click the **Previous Bookmark** or **Next Bookmark** buttons, which are located in the SourceView button toolbar.

Removing Bookmarks

To remove a bookmark, click the red or pink rectangle bookmark symbol next to a bookmarked netlist line.

Searching Netlists

To search the netlist within the currently selected scope, click the binoculars button in the SourceView button toolbar. Search matches are highlighted in the netlist.

Hiding Netlist Line Numbers

To hide netlist line numbers in the SourceView:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.

2. Click the **SourceView** category in the left hand column.
3. Click the **Line Number Off** check box in the **Preference Settings** window.
4. Click **Apply** to save your settings for just this session, or click **Apply-Save** to save your preference settings for future sessions.

Finding Signal Connections

To set a trace, double-click the name of a signal in the netlist.

Feedback

A symbol appears at the beginning of a line that contains a currently selected trace signal. All other lines that contain that trace, but are not currently selected, are marked with a symbol.

Traversing Connections

To traverse through the connection traces you have set in your netlist, click the up or down buttons just above the netlist pane. When a trace is selected in the SourceView, the corresponding connection is highlighted in red in the ConnectionView.

You can also double-click the name of a signal in the SourceView or choose a specific connection trace from the Trace menu located just below the DesignView window menu bar.



Tracing Connections

To display the connections for a trace, right-click a signal, and choose **Trace Connectivity** from the menu that opens. Blocks that contain the connection are highlighted as turquoise text in the Design Browser.

Displaying Trace Connections

To display the connections associated with a trace, right-click a signal, and choose **Trace Connectivity** from the menu that opens. The connections appear in the ConnectionView with the trace node highlighted.

The **Trace Connectivity** menu options are:

- **New View**: Always opens a new view to show its connection.
- **Append**: Adds the signal connection to the current view.
- **Replace**: Replaces the current view's connection with the signal's connection.

You can also trace multi-net signals by double-clicking a net in the ConnectionView. The connecting net is added to the ConnectionView display.

Feedback

Suppressing Out of Scope Warnings

To suppress out of scope warnings:

1. Click **Settings** on the **Configuration** tab.

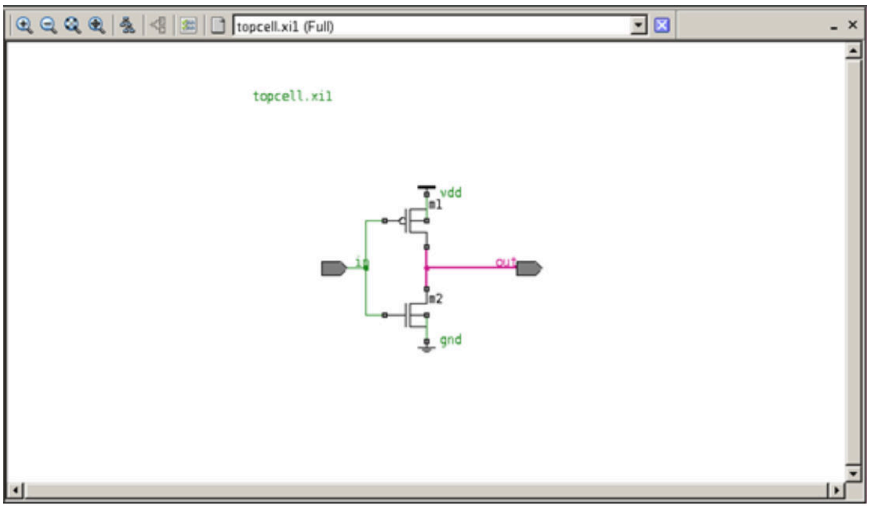
   The **Preference Settings** window opens.

2. Click the **DesignView** category on the left.
3. Click the **Suppress "Out of Scope" Warning for Signal Tracing** check box.
4. Click **Apply** to save your settings for just this session, or click **Apply-Save** to save your preference settings for future sessions.

Using ConnectionView

This section explains how to view and trace circuit connections in your design in the ConnectionView. To access the ConnectionView, open a DesignView (click **DesignView** on the **Design** tab), and then click the ConnectionView check box in the **Show** section of the **Design** tab.

The ConnectionView is the following pane at the bottom of the DesignView window:



This section contains information on the following topics:

- Selecting and Deselecting Items
- Zooming In and Out
- Viewing Full and Current Hierarchy Connections

Feedback

- Viewing the Total Load for Nets
- Hiding and Showing Instances, Nodes, or Terminals
- Showing Full and All Sub-level Full Connections
- Expanding All Sub-Level Instances
- Showing Single Connections
- Showing Connections Through All Scopes
- Showing Lumped Capacitance
- Working With Hierarchical Labels for Instances or Nodes
- Calculating Resistance
- Setting Global VDD and GND Signals
- Setting the Maximum Displayed String Length
- Setting Parasitic RC Filters
- Enabling Warnings for Traced VDD or GND Signals
- Enabling Large Resistors to Be Opened
- Setting Highlighting Style for Selected Instances or Nodes
- Merging Parallel Instances
- Changing ConnectionView Display Settings
- Displaying Key Bindings

Selecting and Deselecting Items

To select an item, click your mouse button on an item. When you click on another item when one is already selected, any previously selected ones are deselected.

If you want to select more than one item, you can either click an item one at a time while holding the **Ctrl** key, or you can hold the **Ctrl** key while drawing a selection rectangle around items you want to select.

To deselect an individual item when multiple items are selected, hold the **Ctrl** key and click your mouse button on each item you want to deselect. Otherwise, you can deselect multiple items by clicking on one item.

Feedback

Zooming In and Out

To zoom into the display of your circuit connection, click the **Zoom In** button  in the ConnectionView toolbar, or roll your mouse wheel forward. To zoom into a specific area, hold the **Shift** key while dragging your mouse pointer to select the section of connections you want to zoom.

To zoom out, click the **Zoom Out** button  , or roll your mouse wheel backward.

Viewing Full and Current Hierarchy Connections

The ConnectionView toggles between displaying the full connection hierarchy or the current connection hierarchy.

To view the full hierarchy connection when you are viewing the current hierarchy connection, click the  button in the upper left corner of the ConnectionView pane.

To view the current hierarchy when you are viewing the full hierarchy connection, click the  button.

Viewing the Total Load for Nets

To view the total load for a net, right-click a net in the ConnectionView and choose **Signal Loads** form the menu that opens. A dialog box opens with the name of the net and total load value displayed.

Hiding and Showing Instances, Nodes, or Terminals

To hide an instance, node, or terminal, right-click an item in the ConnectionView, and choose **Hide Objects** from the menu that opens.

To show hidden items, click any open space in the ConnectionView, and choose **Show Hidden Objects** from the menu that opens.

Showing Full and All Sub-level Full Connections

To show full connections, select one or more instances, then right-click your mouse and choose **Show Full Connections** from the menu that opens. The full connections for the selected instances are displayed.

Feedback

To show all sublevel full connections, select one or more instances, then right-click your mouse and choose **Show All Sub-level Full Connections** from the menu that opens. All instance sublevel connections are displayed.

Expanding All Sub-Level Instances

To expand all sublevel connections in the selected instance, right-click your mouse and choose **Expand All Sub-Level Instances** from the menu that opens.

Showing Single Connections

To show a single connection, right click a connection, and choose **Show This Connection Only** from the menu that opens.

Showing Connections Through All Scopes

To show connections through all scopes, choose **Show Connection Through All Scopes** from the menu that opens.

To trace all ports up to the parent scope in the current view, click the Up arrow after the scope name (or press **B**). The connection of the subscope is hidden in the instance box. If there is no port, a warning is displayed.

Showing Lumped Capacitance

Right-click on a signal and select **Show Lumped Capacitance** to view the signal's lumped capacitance (the sum of all Cg values connected to this signal).

This is also supported for DSPF. When showing lumped capacitance on the ideal signal (dashed line, in ConnectionView), the tool sums all Cg values inside this ideal signal.

Working With Hierarchical Labels for Instances or Nodes

You can add hierarchical labels for instances and nodes in the ConnectionView.

This section contains information on the following topics:

- Adding Labels
- Moving Labels
- Deleting Labels

Feedback

Adding Labels

To add hierarchical labels for instances or nodes, right-click an instance or a node in the ConnectionView, and choose **Label Full Hierarchy Name** from the menu that opens. A label appears next to the instance or node name with either the full hierarchy or shortened text, which you can set in the preferences.

When you click a label, the instance or node name is also displayed in line editor added in NO.2. If you click a label, both the label and node are highlighted. In the same way, if you click a node, the label is also highlighted.

Note:

The label display is only for the current schematic sheet. If you change to another sheet, you need to add a new label to the new sheet.

Moving Labels

To move a label, right-click a label in the ConnectionView, and choose **Move** from the menu that opens. Move your mouse pointer to the location where you want the label, the click your mouse. The label is moved to the new location.

Deleting Labels

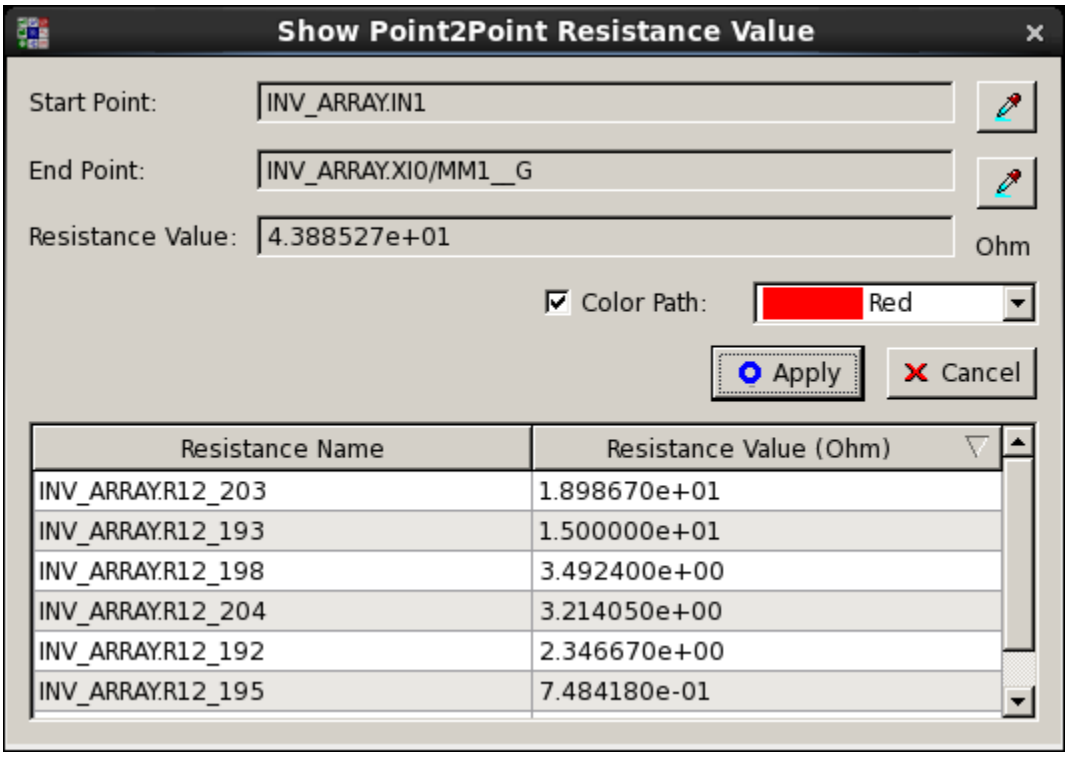
To delete a label, right-click a label in the ConnectionView, and choose **Delete** from the menu that opens. The label is removed.

Calculating Resistance

To calculate resistance and view resistance results in ConnectionView, right-click on a signal or empty space in the ConnectionView window and select **Show Point2Point Resistance Value** from the context menu.

Feedback

The Show Point2Point Resistance Value dialog box appears, as follows:



In the Show Point2Point Resistance Value dialog box:

1. Use the pick buttons to set **Start Point** and **End Point** from a signal or port in ConnectionView. If you used a signal to open the dialog, that signal is automatically populated as the startpoint.
2. Check **Color Path** and select a color to mark the signal and resistance in ConnectionView after calculating. To remove (or reset) the color, uncheck the **Color Path** option (to reset the color, uncheck it and check it again).
3. Click **Apply**.

   If the calculation is a success (has a path), all resistance in the path and its resistance value are listed in a table in the bottom of dialog. The resistances are sorted descending as default. Right-click the column header to change sorting method.

Setting Global VDD and GND Signals

To set global VDD and GND signals in the ConnectionView:

1. Click **Settings** on the **Configuration** tab.

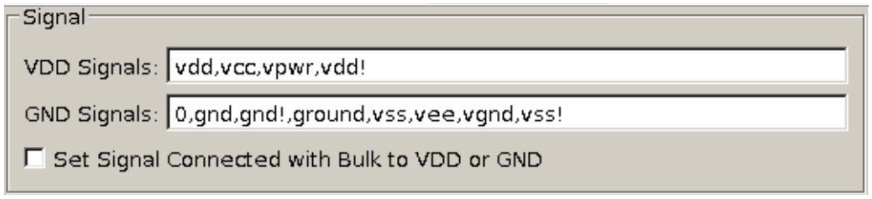
   The **Preference Settings** window opens.

Feedback

2. Click the **ConnectionView** preference category on the left.

   The ConnectionView preferences appear.

3. Enter the types of VDD and GND signals in the **VDD Signals** and **GND Signals** text boxes.



4. (Optional) Click the **Set Signal Connected with Bulk to VDD or GND** check box to set the signals that are connected with bulk to the VDD or GND signals.

5. Click **Apply** to save your settings for just this session, or click **Apply-Save** to save your preference settings for future sessions.

Setting the Maximum Displayed String Length

To set the maximum displayed string length in the ConnectionView:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.

2. Click the **ConnectionView** preference category on the left.

   The ConnectionView preferences appear.

3. Enter the maximum number of characters you want to display for stings in the **Max Displayed String Length** text box.

   A 20 character maximum is specified by default.

4. Click **Apply** to save your settings for just this session, or click **Apply-Save** to save your preference settings for future sessions.

Setting Parasitic RC Filters

To set a parasitic RC filter in the ConnectionView:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.

2. Click the **ConnectionView** preference category on the left.

Feedback

The ConnectionView preferences appear.

3. Click the **Parasitic RC Filter** check box.
4. Enter values for the parasitic resistor and capacitor thresholds in the **Parasitic Resistor Threshold** and **Parasitic Capacitor Threshold** text boxes, respectively.
5. Click **Apply** to save your settings for just this session, or click **Apply-Save** to save your preference settings for future sessions.

Enabling Warnings for Traced VDD or GND Signals

To enable a popup warning for traced VDD or GND signals in the ConnectionView:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.
2. Click the **ConnectionView** preference category on the left.

   The ConnectionView preferences appear.
3. Click the **Popup Warning Dialog for Tracing VDD or GND Signals** check box.

   This preference is enabled by default.
4. Click **Apply** to save your settings for just this session, or click **Apply-Save** to save your preference settings for future sessions.

Enabling Large Resistors to Be Opened

To enable large resistors (larger than 1e120hm) to be opened in the ConnectionView:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.
2. Click the **ConnectionView** preference category on the left.

   The ConnectionView preferences appear.
3. Click the **Open Large Resistors (R > 1e120hm)** check box.
4. Click **Apply** to save your settings for just this session, or click **Apply-Save** to save your preference settings for future sessions.

Feedback

Setting Highlighting Style for Selected Instances or Nodes

To set the color and style for instances or nodes you select in the ConnectionView:

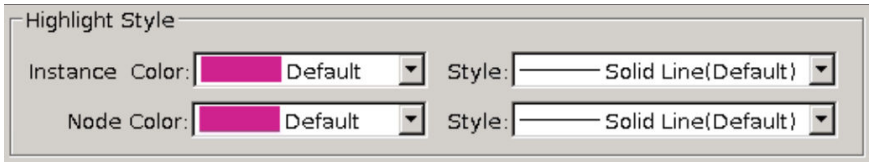
1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.

2. Click the **ConnectionView** preference category on the left.

   The ConnectionView preferences appear.

3. Select a color and line style for instances and nodes in the **Highlight Style** section at the bottom of the page. The same color is applied while traversing up or down the design hierarchy.



   The magenta color and solid line is selected by default.

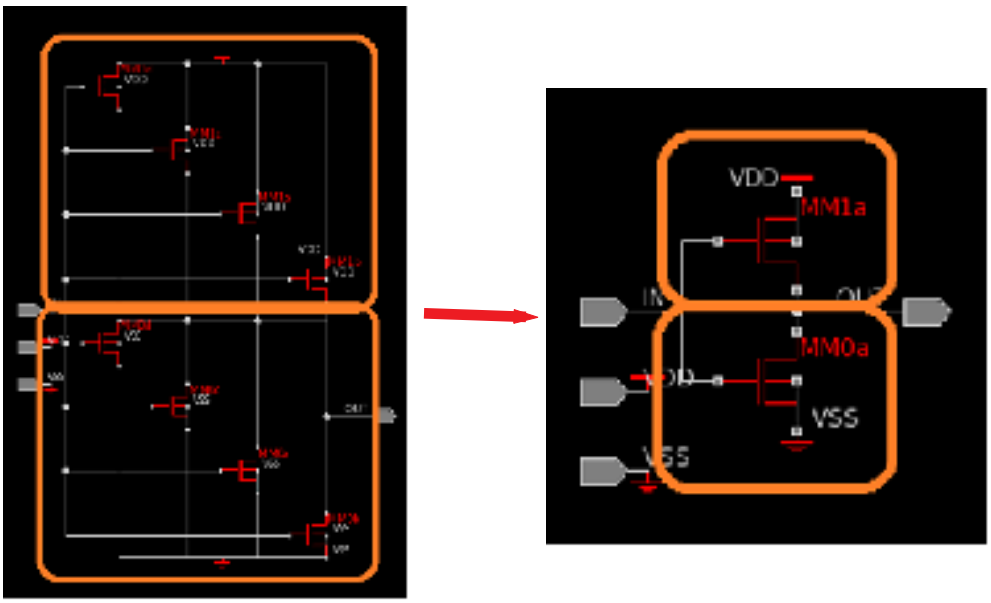
4. Click **Apply** to save your settings for just this session, or click **Apply-Save** to save your preference settings for future sessions.

Merging Parallel Instances

Enable the preference **Merge Parallel MOS** to have all instance parameters of the parallel instances merged. The M (multiple) instance parameter is updated with the number of parallel instances.

Feedback

For example:



Changing ConnectionView Display Settings

You can change how terminals and instances are displayed in the ConnectionView. When you change the display options, all schematic sheets in the ConnectionView are updated.

Note:

Changes you make to the ConnectionView display are not saved by default—you must save your session or design file to save any changes.

Changing Subcircuit Display

To change how subcircuits are displayed in the ConnectionView:

1. Click in an empty part of the ConnectionView, and choose **ConnectionView Display Options** from the menu that opens.

   The ConnectionView Display Options window opens.

2. Click the plus symbol next to the **Show connected terminals only per subcircuit** text to display the list of subcircuits in your design.

3. (Optional) To filter specific subcircuits out from the list, enter a filter string, and click **Filter**.

   The asterisk (*) and question mark (?) wildcards are supported.

Feedback

4. Click the check box next to a subcircuit name to select a terminals you want to display.

   Selected terminals are only displayed when the connection node of a terminal is traced. Other terminals are not displayed in the ConnectionView. Clicking the check box in the heading of the column selects or deselects all subcircuit terminals.

5. Click **Apply** to apply the display changes to the ConnectionView.

Changing Instance Visibility Display

To change the visibility of instances in the ConnectionView:

1. Click in an empty part of the ConnectionView, and choose **ConnectionView Display Options** from the menu that opens.

   The ConnectionView Display Options window opens.

2. Click the plus symbol next to the **Instance visibility display** text to display the list of subcircuits in your design.

3. Click the check box next to an instance name you want to display.

4. Click **Apply** to apply the display changes to the ConnectionView.

Displaying Key Bindings

Right-click in ConnectionView and select **Show Key Binding Information** to display the GUI key bind information.

Using SPICE Lint

To open the SPICELint window, click **SPICELint** on the **Design** tab.

SPICELint screens SPICE netlist files for several problem areas, which can include:

- Syntax errors
- Floating nodes
- Shorted terminals

Netlist properties are also highlighted, which can potentially cause inconsistent simulation behaviors in various simulators.

This section contains information on the following topics:

- Selecting Linter Rules and Running the SPICELinter
- Viewing Linter Options

Feedback

- Viewing Linter Results
- Saving SPICELint Results
- Displaying the LintView on Startup

Selecting Linter Rules and Running the SPICELinter

To select the rules you want the Linter to use when checking your design and then run the Linter:

1. Click **SPICELint** on the **Design** tab.

   The **Linter** window opens.

2. Click the **Syntax** category in the **Lint Options** pane.

   All Syntax rules are displayed in the **Rules** table.

3. Select or deselect any rules that you want or do not want to include.

4. Enable or disable any rules for the Connection, ERC Checks, and Parameters rule categories.

5. Click **Run** to run the Linter.

   The Linter closes after the run is complete and the status bar reaches 100%.

   Results are displayed in the LintView pane in the DesignView window. See Viewing Linter Results for more information on reading the results.

You can also invoke the Linter from the command line by entering the following line:

```
cx -spicelint <design_netlist_file>
```

Viewing Linter Options

To view the linter options currently set in the Linter, click **SPICELint** on the **Design** tab. The Linter window opens with the options for checking Syntax, Connectivity, ERC Check, or Parameters displayed.

The following SPICELint option categories are available:

- SPICELint Syntax Options
- SPICELint Connectivity Options
- SPICELint ERC Checks Options
- SPICELint Parameter Options

Feedback

See Selecting Linter Rules and Running the SPICELinter for information on how to set the Linter rules.

SPICELint Syntax Options

You can choose from the following SPICELint Syntax options:

Table 1 Linter Syntax options

Rule	Description	Type
SERR	Syntax error	Error
TLE	Command ignored in title line	Warning
ESM	.ENDS/.EOS is missing.	Warning
SOC	Redundant .ENDS	Warning
ENE	Premature .END deck	Warning
NED	.END deck is missing	Warning
LMQ	Line has missing quote	Warning
NCR	Line has no carriage return	Warning
RST	Redundant slash token	Warning
FNF	File not found	Warning
RLP	Relative library/included file path	Warning
FRD	File read denied	Error
FRI	File recursively included	Error
LEM	Nested library	Warning
UEV	Undefined environment variable in path	Warning
SNP	Subcircuit has no port	Warning
SPR	Subcircuit port redefined	Error
SRD	Subcircuit is redefined	Warning
UET	Unknown device type	Error
NMD	Model device name missing	Error
NMP	Model name missing	Error

Feedback

Table 1 Linter Syntax options (Continued)

Rule	Description	Type
MRD	Model redefined	Warning
PRD	Parameter redefined	Warning
FDX	User function in X-instance	Warning
CPN	Parameter name conflicts with reserved words	Warning
CFN	User function conflicts with built-in functions	Warning
FNQ	Function value without quote	Warning
OVM	Option value missing	Warning
VIT	Variables in .temp	Warning
UDM	Undefined model	Error
IHC	Name contains illegal character	Warning
DNN	Device node not defined	Error
TSD	Device terminals shorted	Warning
NGN	Ground node missing	Warning
NUL	No device in topcell	Error
SNE	Subcircuit has no device	Warning
FDA	Duplicated argument	Error
MCM	Model card missing	Error
SGP	Global subcircuit port	Warning
ICE	Incomplete device syntax	Error
NRD	Name redeclared	Warning
IXN	Invalid signal name	Warning
PNN	Problematic signal name	Warning
FBK	Bulk node note connected	Warning
CEH	Parameter CUR in non-G elements	Warning
VGF	Parameter VOL in non-E elements	Warning

Feedback

Table 1 Linter Syntax options (Continued)

Rule	Description	Type
PRU	Resistance undefined	Warning
PCU	Capacitance undefined	Warning
PIU	Inductance undefined	Warning
PKU	Coupling coefficient undefined	Warning
PLU	Length undefined	Warning
PWU	Width undefined	Warning
PIR	Device parameter is redefined	Warning
MTS	Missing transient parameters	Warning
SND	Missing source DC value	Warning
PVM	Parameter value is missing	Warning
NIF	.IF statement is missing	Error
IAT	Invalid ".ALTER" command	Warning
USC	Unknown SPICE command	Warning

SPICELint Connectivity Options

You can choose from the following SPICELint Connectivity options:

Table 2 Linter Connection options

Rule	Description	Type
IPC	Port number mismatch	Error
RSC	Too deep module instantiation	Error
SNR	Subcircuit is not used	Warning
SPF	Subcircuit port not connected	Warning
UDS	Master module of an instance is missing	Error
SSP	Subcircuit instance/device has single port	Warning
DGN	Dangling node	Warning

Feedback

Table 2 Linter Connection options (Continued)

Rule	Description	Type
DFN	DC floating node	Warning
DFB	DC floating block	Warning
LGN	Global node not powered	Warning
ETF	Device terminals floated	Warning
FTS	Unconnected transient source	Warning
LOV	Loop of v-source/inductor	Error
ETS	Device terminals shorted	Warning
MCP	MOS-cap device	Warning
FXG	Pass gate floating end	Warning

SPICELint ERC Checks Options

You can choose from the following SPICELint ERC Checks options:

Table 3 Linter ERC Check options

Rule	Description	Type
EDC	Static DC conducting device	Warning
ELC	Static DC leakage device	Warning
EDP	Static DC conducting path	Warning
ELP	Static DC leakage path	Warning
EHZ	Static hi-Z node	Warning

SPICELint Parameter Options

You can choose from the following SPICELint Parameter options:

Table 4 Linter Parameter options

Rule	Description	Type
INF	Invalid number format	Error

Feedback

Table 4 Linter Parameter options (Continued)

Rule	Description	Type
ESE	Equation syntax error	Error
UDP	Undefined parameter	Error
EDE	Equation domain error	Warning
NPM	Zero/negative multiplier	Warning
CSV	Compound scale value	Warning
USP	Undefined parameter in value	Error
PMC	Measure name conflicts with parameter name	Warning
PNS	Parameter name is reserved or builtin parameter name	Warning
PUD	Parameter referenced before definition	Warning
RPD	Recursive parameter definition	Warning
NMS	No matched model	Warning
PWS	MOSFET’s width too small	Warning
PWL	MOSFET’s width too large	Warning
PNW	Non-positive MOSFET’s width	Warning
PLS	MOSFET’s length too small	Warning
PLL	MOSFET’s length too large	Warning
PNL	Non-positive MOSFET’s length	Warning
PCL	Capacitance is too large	Warning
PNC	Non-positive capacitance	Warning
PRS	Resistance too small	Warning
PNR	Non-positive resistance	Warning
TRV	PWL time not increased	Warning
NSC	Non-positive scale	Warning
NSM	Non-positive scalm	Warning
OXS	Option delmax too small	Warning

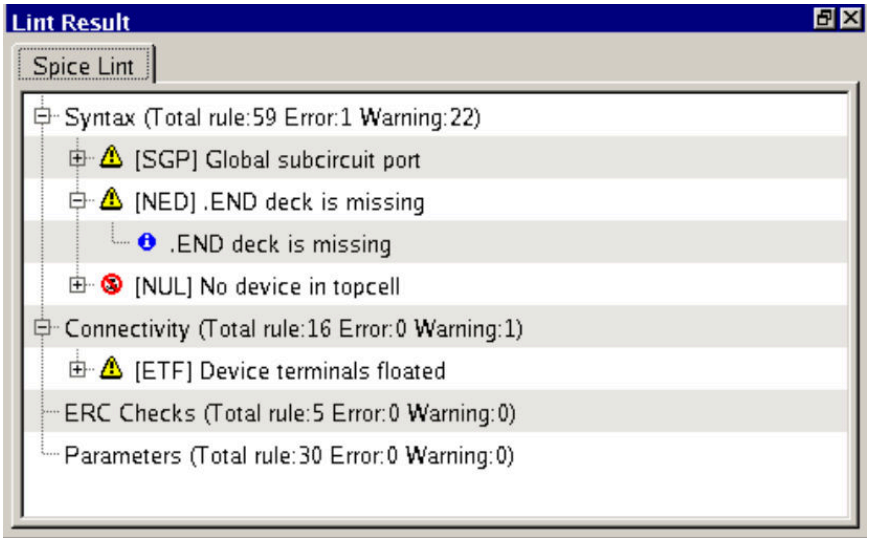
Feedback

Table 4 Linter Parameter options (Continued)

Rule	Description	Type
OGL	Option gmin too large	Warning
SBS	Pulse slew less than zero	Warning
TSP	Pulse period is too short	Warning
TDC	Initial transient != dc value	Warning

Viewing Linter Results

Once you run the Linter, the results are displayed in the SPICELinter pane. Rule categories that do not contain any violations are greyed out.



The following table shows what the SPICELinter results icons signify:

Icon	Description
	Linter rule category with violations.
	Warning issued for a rule.
	Error issued for a rule.
	Specific information regarding the violation.

Feedback

Saving SPICELint Results

When the WaveView tool is launched with the option `-dump_splint` with the `wv` command, results are saved after running SPICELint in the file wv_splint_result.txt under the current working directory.

For example:

```
wv -dump_splint
```

Displaying the LintView on Startup

To display the LintView pane as part of the DesignView window every time you run WaveView:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.

2. Click the **DesignView** category on the left.

3. Click the **Show Lint View on Startup** check box.

4. Click **Apply** to save your settings for just this session, or click **Apply-Save** to save your preference settings for future sessions.

Annotating a Netlist

This section contains information on the following items in the **Annotation** section of the **Design** tab:

- On/Off
- Setup
- Binding Waveforms

On/Off

To annotate your netlist with linked waveform data:

1. Click **On/Off** in the **Annotation** section of the **Design** tab.

   The green plus sign is displayed in the icon for the On/Off annotation when it not yet on; a red minus sign is displayed when annotation is on.

   If you already set up annotations, the annotation values from the linked waveform data appear below each signal in the SourceView. You can skip the remaining steps.

Feedback

If you have not set up annotations, click **Setup** in the **Annotation** section of the **Design** tab. The **Annotation Setup** window opens.

Continue on to the next step.

2. Choose what type of time you want to annotate:
   - **Use Target Signal Change**

     If the target signal is analog, you can choose **Value Change**, which jumps to the next or previous time point when the Y-value time point changes (for example, when you click the **Previous** or **Next** button when viewing waveform data). **Value Change** is the default.

     If the target signal is digital, you can choose **Logic Rise** or **Logic Fall**, which jumps to the next or previous time point depending on the rising or falling edge of the signals (for example, when you click the **Previous** or **Next** button when viewing waveform data).
   - **Use Reference Signal**

     By default, the referenced signal is the selected signal, but you can also choose another signal as the reference signal.

     If the target signal is analog, you can choose **Value Change**, which jumps to the next or previous time point when the Y-value time point changes (for example, when you click the **Previous** or **Next** button when viewing waveform data). **Value Change** is the default.

     If the target signal is digital, you can choose **Logic Rise** or **Logic Fall**, which jumps to the next or previous time point depending on the rising or falling edge of the signals (for example, when you click the **Previous** or **Next** button when viewing waveform data).
   - **Use Fixed Time Step**

     Uses the specified value as a time step instead of the time points in waveform files.
   - **To Time**

     Jumps directly to the specified time.
3. (Optional) Click the **Use WV Cursor Value** check box to change the X-value based on the position of a cursor in the WaveView tool.

Feedback

4. Choose the maximum string width for annotation values, which specifies the number of values that are displayed in the SourceView and ConnectionView:
   - **User Specified**

     The default logic value is 8, and the default analog value is 8.
   - **Same as Signal Name String**
5. Enter a value in the Analog Precision Value text box.

   The default is 2.
6. Click **OK** to save the settings.

   Annotation values from the linked waveform data appear just below each signal in the SourceView.

Setup

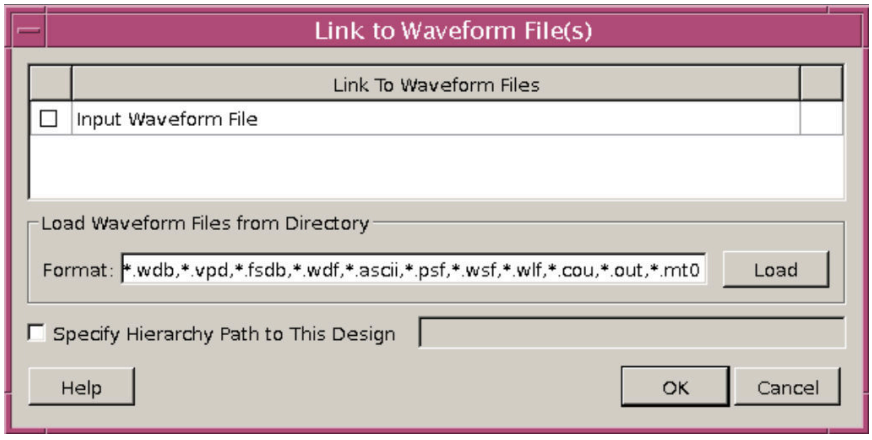
To change annotations, click **Setup** in the **Annotation** section of the **Design** tab.

For information on an annotation, see Step 2 in On/Off.

Binding Waveforms

To bind your design to a waveform file:

1. Right-click a design in the Design Browser, and choose **Bind Waveform** from the menu that opens.



   The Link to Waveform File(s) window opens.
2. Click the `Input Waveform File` text.

Feedback

3. Enter the path name of a waveform file, or click the **Browser** button to browse to the required waveform file.

   You can also load multiple waveform files from a directory by clicking the **Load** button and browsing to a directory that contains all the waveform files you want to load. Only waveform files with the following file extensions are loaded by default, although you can add additional formats in the Format text box:

   - *.vcd
   - *.tr0
   - *.wdb
   - *.vpd
   - *.fsdb
   - *.wdf
   - *.ascii
   - *.psf
   - *.wsf
   - *.wlf
   - *.cou
   - *.out
   - *.mt0

4. Click the check box next to the name of the waveform files you want to use for linking.
5. (Optional) Click the **Specify Hierarchy Path to This Design** check box to specify the hierarchical path to a waveform that is part of your design.
6. Click **OK** to save any changes.

   All cells that contain the linked waveform data appear as yellow icons in the Hierarchy and Signal browsers.

Enabling and Disabling Linked Waveform Data

To enable or disable linked waveform data:

1. Choose **Design > Link to Waveform File(s)** from the DesignView window menu bar.

   The Link to Waveform File(s) window opens.

2. Click the check box next to the waveform file you want to enable or disable.

Feedback

A checked box means that the waveform file is enabled, and an unchecked box means that the waveform file is not enabled.

3. Click **OK** to save your changes.

Removing Linked Waveform Data

To remove linked waveform data:

1. Choose **Design > Link to Waveform File(s)** from the DesignView window menu bar.

   The **Link to Waveform File(s)** window opens.

2. Click the **X** button to the right of the waveform file you want to remove.

   The waveform data file disappears from the table.

3. Click **OK** to save your changes.

Cosim

This section contains information on the following topics:

- Config
- Link A/D Report

Config

To set up your cosimulation configuration, click **Config** in the **Cosim** section of the **Design** tab. The **New Configuration** window opens.

Link A/D Report

To link an A/D report, click **Link A/D Report** in the **Cosim** section of the **Design** tab. The **New Configuration** window opens. The Link to A/D Report window opens.

9

Using WaveView Tools

This chapter details how to use WaveView editors and tools.

This chapter contains information on the following topics:

- Using the PWL Editor
- Using the ADC Toolbox
- Using the Coherent Sample Only (CSO) ADC Toolbox
- Using the CCK Viewer
- Using CSA (Channel Simulation Analysis)
- Using ADE Post-Process Integration

Using the PWL Editor

To open the PWL Editor, click **PWL Editor** in the **Tools** tab.

The PWL Source Editor is used to construct SPICE PWL input stimuli for circuit simulations. The editor supports interactive editing functions via direct point dragging. Multiple sources can be edited or overlapped simultaneously. You can store the edit result in the text PWL storage files (.pwl) or exported to SPICE netlist files.

The buttons are grouped into three groups:

- File Operations
- Source Operations
- Point Operations

File Operations

Open a prestored PWL editing task with the **Open** button. Save an editing task with the **Save** button. The save function automatically appends file extension `.pwl` to the user-

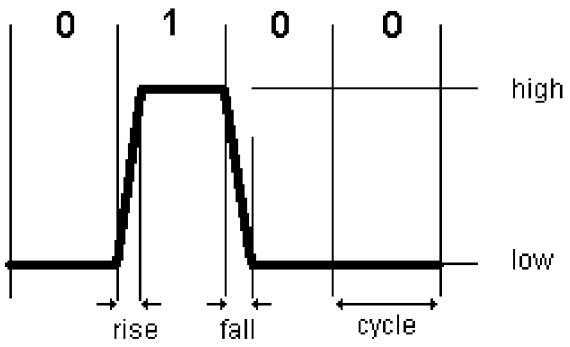
Feedback

specified file name if the `.pwl` extension is not used. Click the **Check** button to check errors in the PWL sources such as overlapping time points.

Click **Close** to close the PWL editor window. All PWL sources are not removed when the PWL editor window is closed.

Source Operations

Click the **Add New Source** button to add a single PWL source or a bus pattern that contains multiple PWL sources. For bus pattern, the usage of high or low levels, cycle time, rise time, and fall time are indicated in the following figure:



The source list window keeps a list of all PWL sources. The selected PWL source is the active source highlighted with thick line. Click **Modify a Source** to modify the name and parameters of the active source. Click **Display Source** to toggle the active source on and off.

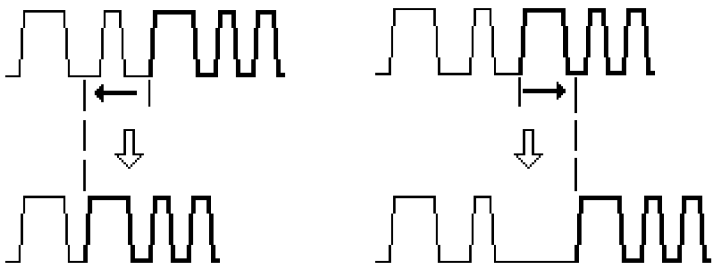
Delete the active source with the **Delete a Source** button, or clear all sources with the **Delete All** button.

Point Operations

A newly added source has five points. Click on a point to select the point as the active point. You can insert a new point after the current active point using the **Insert a Point** button, or insert a pattern using the **Insert Pattern** button. A point can also be directly added with the Time and Value text fields and the **Add** button. The newly added point becomes the active point highlighted with a solid box. With multiple points in a source, click the **Next** or the **Prev** button to jump the active point of a source.

Feedback

To shift the entire PWL pattern forward or backward after the active point, click the **Shift Pattern** button, and drag the active point horizontally to shift the pattern. For example:



The Time and Value fields are updated when a new active point is selected. To modify the values of the active point, enter new values and click the **Modify** button to make the change. You can also change the values of any point by dragging it directly. Dragging a point also selects it as the active point.

Click the **Delete a Point** button to delete the active point.

To assist point placement, snap functions can be enabled along the horizontal and vertical direction. Enter the preferred snap distance and check the corresponding option button to enable the snap function.

To zoom in an area, click and hold the mouse and move the mouse pointer to select the zoom area. Click the **Zoomout** button to zoom out or the **Full** button to reset to full viewing range.

Using the ADC Toolbox

To open the **ADC Toolbox**, choose **ADC Toolbox** from the menu located in the **Utilities** section of the **Waveform** tab. Simplified SNR/THD/SNDR/ENOB calculation functions, for sine input signals, are also provided in the FFT dialog box and the equation builder.

The ADC Toolbox is a built-in tool for detailed performance analysis of A-to-D converter designs. Based on simulation output waveform data or measured data, the ADC Toolbox reports the following performance merits:

- DC static characteristics—Histogram, INL/DNL min/max values, and their standard deviations.
- AC dynamic characteristics—Noise parameters SNR, THD, SNDR, SFDR and ENOB and the frequency-domain power spectrum.

Feedback

This section contains information on the following topics:

- Generic Versus Coherent ADC Toolbox
- Overview of the ADC ToolBox
- Input Signal of ADC Toolbox
- DC DNL/INL Analysis
- Sine Input
- Ramp Input
- Number of Sample Points and INL/DNL
- Sampling Frequency and INL/DNL
- AC Analysis: Coherent Sampling Versus Window Sampling
- Coherent Sampling
- Window Sampling
- Exporting the DNL/INL/FFT Results as Waveform Data
- Saving or Loading an Analysis Setup
- Dumping the Contents of the ADC Toolbox
- ADC Toolbox Display Controls
- Sampled Signal Window
- Histogram, DNL, and INL Windows
- Power Spectrum (Sine Input Only)
- AC Dynamic Characteristics
- DC Static Characteristics
- Running Batch-mode ADC Analyses

Generic Versus Coherent ADC Toolbox

Before WaveView version 2004.4, the ADC Toolbox only supports the Coherent sampling method for sine input waveform and the input waveform have their signal amplitude in code values (signal values range from 0 to 2N-1, where N is the number of bits of the design).

Feedback

From release 2004.4, a new generic ADC Toolbox is introduced. The new ADC toolbox accepts input signal in arbitrary amplitude, and a more generic Window sampling method is added to eliminate the need to sample the input sine waveform over integer cycles, which is a requirement of the Coherent sampling method.

To use the original Coherent-only ADC Toolbox, set the `SW_SX_ORG_ADC` environment variable to 1 in your shell:

```
setenv SW_SX_ORG_ADC 1
```

Without setting the `SW_SX_ORG_ADC` environment variable, the new generic ADC Toolbox is invoked as the default mode.

The new generic ADC Toolbox also allows the AC and the DC analysis to be carried out separately.

Overview of the ADC ToolBox

The ADC Toolbox window is divided into three tabs: the Input Parameters tab, DC Analysis Result tab, and AC Analysis Result tab.

The Input Parameter tab acquires input parameters from users for an ADC analysis task. The input parameters are divided into four categories: General input signal properties and data sampling parameters, DC analysis parameters, AC analysis parameters, and AC Coherent sampling parameters.

The DC Analysis Result tab displays sampled waveform and DC result including histogram, DNL and INL plots/distribution.

The AC Analysis Result tab displays the FFT spectrum of the sine input waveform and AC figure of merits including SNR/THD/SNDR/SFDR/ENOB.

Input Signal of ADC Toolbox

The ADC Toolbox accepts the following input signals:

- Sine: Both AC FFT analysis and INL/DNL analysis are available for the sine input type.
- Ramp: Only the INL/DNL analysis is available for the ramp input type.

The input signal of the ADC Toolbox can be an analog signal with arbitrary amplitude ranges, or a digital bus signal representing the digital output code of an ADC design. For digital bus input signal, the bus signal must be displayed in a waveview panel. Drag an analog signal from the Waveform Browser, or a bus signal from a panel to the Signal field in the ADC Toolbox.

The input signal is sampled based on one group of the following three sampling parameters: Start Time, Stop Time, and number of points, or Start Time, Sampling

Feedback

Frequency (Fs), and number of points. When the stop time is used (in the first group), the last sample point is at:

```
stop_time - (stop_time - start_time)/number_of_points
```

For sine input type, the input frequency (Fin) also needs to be specified.

You can input analog bus signals directly by entering bus named delimied by semicolons (;). For example:

```
ADCOUT_3;ADCOUT_2;ADCOUT_1;ADCOUT_0
```

When you enter analog bus signals directly, the following requirements must be satisfied:

- Signals must be analog.
- The number of signals must be the same as the number of bits.
- The order bus signals must be MSB ; .... ; LSB.
- All signals in the bus must come from the same file.
- No more than 32 signals can be entered at a time.

Once the input signals are accepted, the median signal value among all signals is used as the A2D conversion threshold. For example:

```
0.5 * (ymax_of_all_bit + ymin_of_all_bits)
```

The analog bus signals are then converted to a logic bus whose code values are then used as the input to the ADC Toolbox. Analog bus signals are also supported during .INI file save/load operations.

For measured data that was sampled at known Fs, the data (x,y) value pair can use integer index as the x values. If this is the case, select the **Measured data** option and the ADC Toolbox processes the input as if the data is sampled at Fs. When the **Measured Data** option is selected, the Start T/P value is the starting index (0-based) of the measured data.

If you have a measurement data with only single column, (that is, the data is measured at fixed known sampling rate so the data file does not contain x-axis sample time information), edit the file header to add the following lines:

```
#format table data_only

name_of_data

measure data 1

measure data 2

…..
```

Feedback

With the keywords in the header line, the WaveView tool can load such a text data file as a regular waveform file and automatically assign integer index to the x-axis. You can then apply the **Measured Data** option and use the measurement data in the ADC Toolbox.

DC DNL/INL Analysis

To calculate DNL/INL, an analog input signal is first scaled into full code range based on the **Code Range** parameters in the DC Input Parameters section. If the **Number of Bits** value is used, the ADC Toolbox assumes that the input signal covers full code range from 0 to 2N-1. If the **User-defined Code Range** option is selected, the ADC Toolbox assumes that the input signal swings from the **Min** code value to the **Max** code value. Digital input bus signal is not subject to this amplitude scaling, instead the bus direct code values are used.

If the analog input signal value is already converted to the A/D converter output bus values, select the **Signal Value=Output Code** option.

Because the ADC Toolbox always automatically scales the analog input signal, PWL sampling is always used in sampling analog input signal.

After entering the necessary DC analysis parameters, click **Evaluate DC** to carry out a DC analysis.

Sine Input

For sine input signal, based on the sampled data and the Fin parameters, the ADC Toolbox needs to find the ideal sine wave to calculate INL and DNL. The ADC Toolbox uses two methods to calculate the ideal sine waveform:

- FFT fitting: If the input sine waveform data is not clipped. The amplitude and phase of the strongest FFT spectrum component of the input sine signal is used as the ideal sine waveform.
- LMS (least-mean-square) fitting: If the input sine waveform data is clipped. This is often seen with measure data. The ADC Toolbox uses the sample data within the Clip-Min and Clip-Max code values, and LMS fitting method to find the original ideal sine waveform. The offset and the amplitude of the fitted ideal sine waveform is displayed in the Sampled Data plot.

With the LMS fitting, since the clipped sampled data might be heavily distorted, FFT AC analysis is disabled.

Ramp Input

For the ramp input type, ADC Toolbox always assumes that the ideal ramp signal evenly covers the full code range. Based on this assumption, multiple complete cycles of ramp (up or/and down) are accepted.

Number of Sample Points and INL/DNL

In order to produce meaningful INL/DNL result, the number of sample points must be large enough so that there is at least one sample point in each code value. For example, for a 10-bit design, ramp input, at least 1024 points (210) sample points are needed. Sine input type needs even more points because the sample points do not distribute evenly over different code values.

Since the ADC Toolbox scales the original analog input signal and uses PWL method to sample the scaled input signal, it is the user's responsibility to make sure that the measurement/simulation generates enough true data points at each code value.

Sampling Frequency and INL/DNL

For sine input type, if multiple periods of the input sine waveform are sampled, the sampling frequency Fs and input frequency Fin must be carefully selected to avoid sampling the same code values in each period. For example, if a 5 MHz input sine waveform (6-bit) is sampled at 40 MHz over 128 cycles, the 8 sampled code values from each cycle are always the same. As a result, only 8 out of the 64 code values are available in the sampled data. If Fin is at 4.99 MHz, with the sampling frequency, the sampled data has different code values from cycle-to-cycle, and the INL/DNL result would be more meaningful.

AC Analysis: Coherent Sampling Versus Window Sampling

With the sine input type, the ADC Toolbox supports FFT AC analysis using two different types of sampling method:

- Coherent Sampling
- Window Sampling

Coherent Sampling

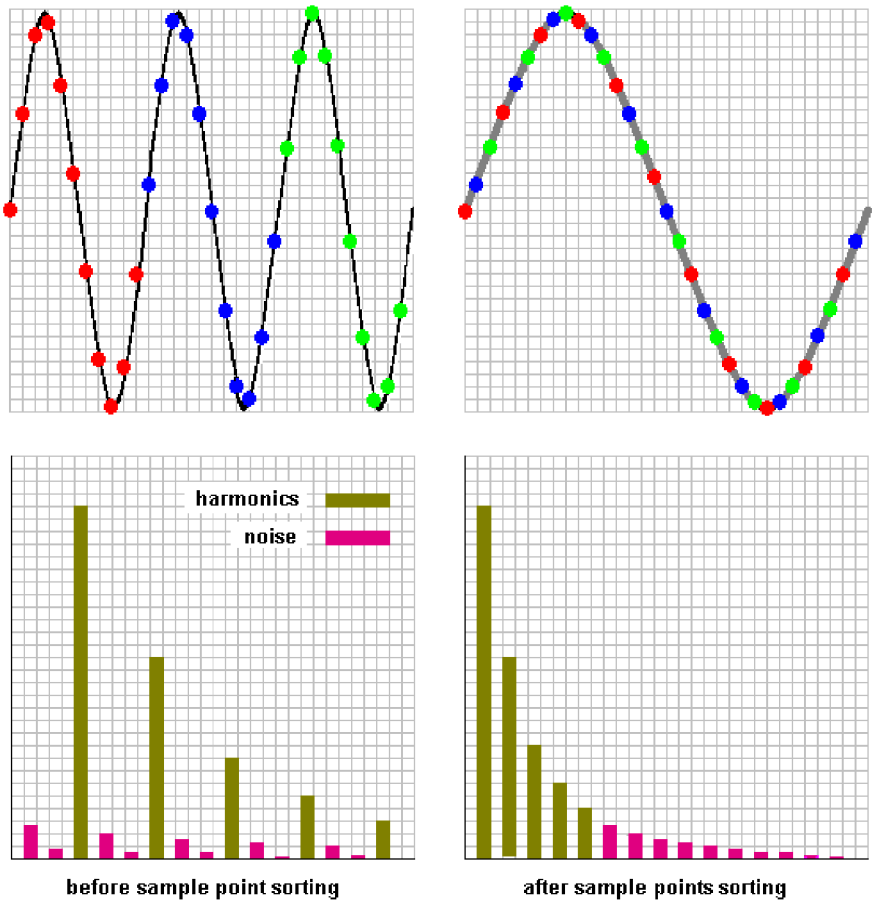
With coherent sampling, exact integer multiple cycles (FinNUM) of the input sine wave are sampled. The sampled data is then sorted into a single cycle of sine wave before performing the dynamic performance analysis using FFT.

Feedback

Advantages of the coherent sampling method include:

- It allows an input signal at higher frequency to be sampled using a lower sampling rate. In the real world, physical measurement equipment might sometimes have limited sampling performance to meet the Nyquist criterion required by the bandwidth of the input signal.
- Harmonics and noise are readily separated in the power spectrum using coherent sampling.
- FFT windowing functions are not required because integer cycles of the input signal are sampled.

Figure 4 Coherent sine wave sampling



However, coherent sampling method does require the exact frequency (Fin) of the input sine wave to be known prior to running an ADC simulation. To use the coherent sampling method in the ADC Toolbox, you need to enter the preferred Precision, the estimated Fin and Fclk, and Sample Pts. Then, click **Evaluate AC** to find out the exact values for Fin.

Feedback

Figure 4 is a coherent sampling example with FinNUM = 3, N=32. Coherent sampling requires that N x Fclk = FinNUM x Fin. In order to sort sampled points of multiple cycles into a single sine wave cycle, N and FinNUM cannot have any common factor. Since N is always power of 2, an odd number for FinNUM is always a valid choice. In the ADC Toolbox, the FinNUM value can be automatically determined based on user-defined Fin and Fclk values.

As depicted in the above graph, the power spectrum of the sorted sine wave is also a sorted version of the original multi-cycle sine waveform's power spectrum.

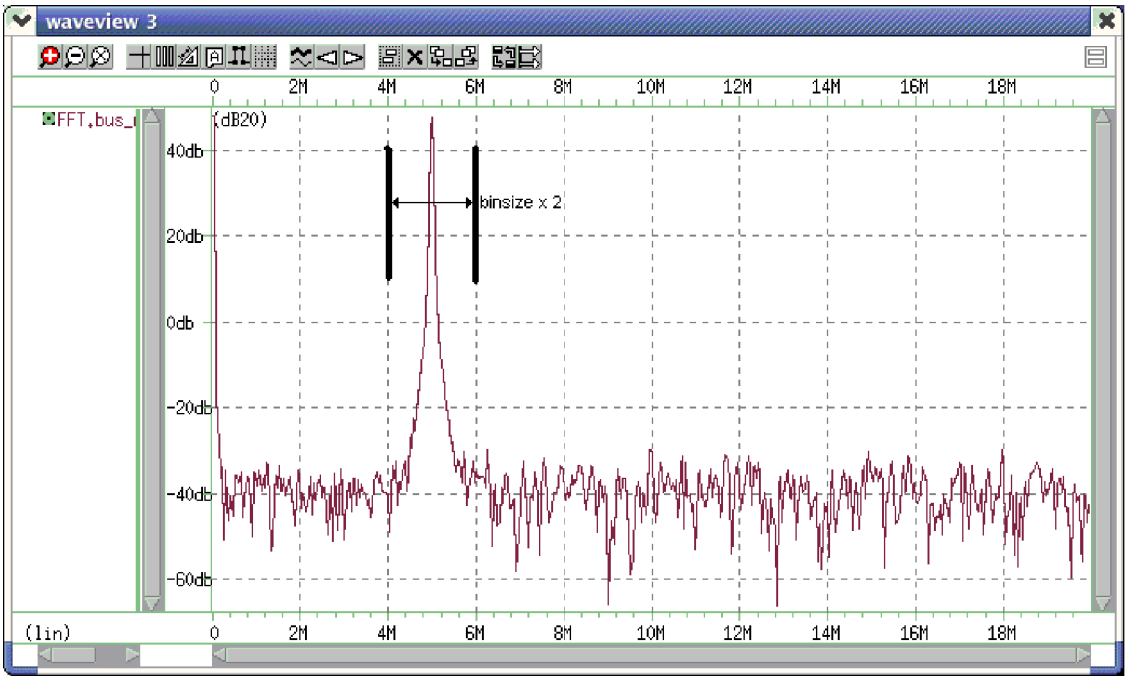
The sorted single-cycle sine wave is sometime also referred to as the unfolded sine wave.

Refer to Using the Coherent Sample Only (CSO) ADC Toolbox for details about entering parameters for Coherent Sample.

Window Sampling

The window sampling method does not require integer number of cycles of the input sine signal to be sampled. However, to overcome the spectral leakage problem from not sampling over an integer cycles of the input waveform, a FFT windowing function is required. Figure 5 shows the FFT spectrum of a 4.99 MHz sine signal sampled over 127.75 cycles at 40 MHz.

Figure 5 FFT Spectrum With Window Functions



Feedback

As depicted in the above spectrum, the power at the fundamental frequency (4.99 MHz) gets spread into neighboring spectral components due to spectral leakage. For this reason, a binsize parameter is required to include all neighboring components within +/- binsize spectral range as part of the original fundamental (or harmonic) components. The default of BinSize is 10.

After entering the necessary AC analysis parameters, click **Evaluate AC** to start an AC analysis.

Exporting the DNL/INL/FFT Results as Waveform Data

The ADC Toolbox result, including the sampled waveform data, histogram, INL/DNL and the FFT power spectrum, can be exported to a waveview window for further viewing purpose. Click **Waveform** next to the plot window to display the data into a new waveview window.

Saving or Loading an Analysis Setup

The ADC Toolbox setup, including the input parameters, sampled data, and the analysis result can be saved into data files. To restore a previously saved setup, load from the .INI (Input parameter) file.

Dumping the Contents of the ADC Toolbox

To output the contents of the ADC Toolbox to a graphic file (PNG, JPG, or BMP), click the **Dump** button on the ADC Toolbox window.

ADC Toolbox Display Controls

The following window displays are included in the ADC toolbox:

Sampled Signal Window

The Sampled Signal window displays the sampled data waveform based on the input parameters. The y-axis is in code value, with min-code value at the bottom and max-code value at the top.

Ideal/Acquired/Error/All switch:

- Ideal: Displays the ideal signal used for INL/DNL calculation.
- Acquired: Displays the sampled waveform data.

- Error: Displays the error between ideal signal and the sampled data.
- All: Displays all of the above.

Sin/Cos/Original (sine input only):

- Sin: Adjusts the phase of the displayed waveform into Sin phase (starts from zero).
- Cos: Adjusts the phase of the displayed waveform into Cos phase (starts from the maximum value).
- Original: No phase adjust for sampled waveform display.

Exp/Integer:

- Exp: Displays the y-axis value ranges using the engineering format.
- Integer: Displays the y-axis value ranges using the integer code values.

Histogram, DNL, and INL Windows

These windows display the histogram, DNLb, and INL plots of the sampled signal. Aligned with the y-axis of the Sampled signal plot, the y-axes of all three plots correspond to the code value. The x-axis is the histogram / DNL / INL value in LSB.

Power Spectrum (Sine Input Only)

The power spectrum window displays the noise spectrum of the sampled signal.

Line/Bar/Both

- Line: Plots the power spectrum using line chart.
- Bar: Plots the power spectrum using bar chart.
- Both: Plots the power spectrum using both line and bar chart.

Index/Frequency

- Index: Displays the spectrum x-axis value range using harmonic index.
- Frequency: Displays the spectrum x-axis value range using frequency.

Feedback

AC Dynamic Characteristics

The AC Characteristics window displays the spectrum harmonic strength and the following values:

- SNR: The signal-to-noise ratio.
- THD: The total harmonic distortion.
- ENOB: The effective number of bits.
- SNDR: The signal-to-noise and distortion ratio.
- SFDR: The spurious free dynamic range. The harmonic index of the second strongest harmonic component is also displayed together with the SFDR value (for example, SFDR index_after_sorting (index_before_sorting)).

The strength of the power spectrum is in db relative to the strongest harmonic component (0 db for the strongest harmonic component). The AC characteristic window lists the value of each of the harmonic component. Use the **up** and **down** buttons to scroll up/down to view the spectrum values at different harmonic range.

Select the toggle check box next to a harmonic component to either exclude the harmonic for the THD calculation or include the harmonic for the SNR calculation.

Click **Evaluate AC** to update the AC noise analysis result (SNR/THD/ENOB/SNDR/SFDR) based on the new harmonic selection.

DC Static Characteristics

The DC Characteristics window displays the min/max values and the standard deviation of the INL/DNL distribution. The unit is 1 LSB.

Running Batch-mode ADC Analyses

ADC Toolbox analysis can be run in batch mode with the -adcin, -adcout,

-adcwdf, and -adccsv command line option. The usage is:

```
wv -adcin adc.INI -adcout out_sw -adcwdf wdf_sw
```

The adc.INI is a previously saved .INI file, out_sw is the _ (underscore) delimited type switch that controls the output text files; wdf_sw is the _ (underscore) delimited type switch that controls the output WDF waveform files. The -adccsv option sets the output text file in the CSV (comma separated values) format.

Feedback

One of the -adcout and -adcwdf options must be specified or the batch mode operation is terminated. Supported type strings are INL, DNL, FFT, TLA, HIS, IND, ALL, and DAT (-adcout only). FFT and TLA have no effect on ramp input signals. For example:

```
wv -adcin adc.INI -adcout INL_DNL -adcwdf TLA_FFT -adccsv
```

Using the Coherent Sample Only (CSO) ADC Toolbox

To start the ADC Toolbox in the CSO mode, set the `SW_SX_ORG_ADC` shell environment variable.

This section contains information on the following topics:

- Input Signal Requirements
- Preparing the Input Waveform
- Sampling the Input Waveform
- Preparing Test Benches for the CSO ADC Toolbox
- Input Signal Strength
- Input Sine Wave Frequency
- CSO ADC Toolbox User Interface
- Input Parameters
- Selecting Sampling Parameters
- Sine Input
- Ramp Input
- Exporting DNL/INL/FFT Results as Waveform Data
- Saving or Loading an Analysis Setup
- ADC Toolbox Display Controls
- Sampled Signal Window
- Histogram, DNL, and INL Windows
- Power Spectrum (Sine Input Only)
- AC Dynamic Characteristics
- DC Static Characteristics

Feedback

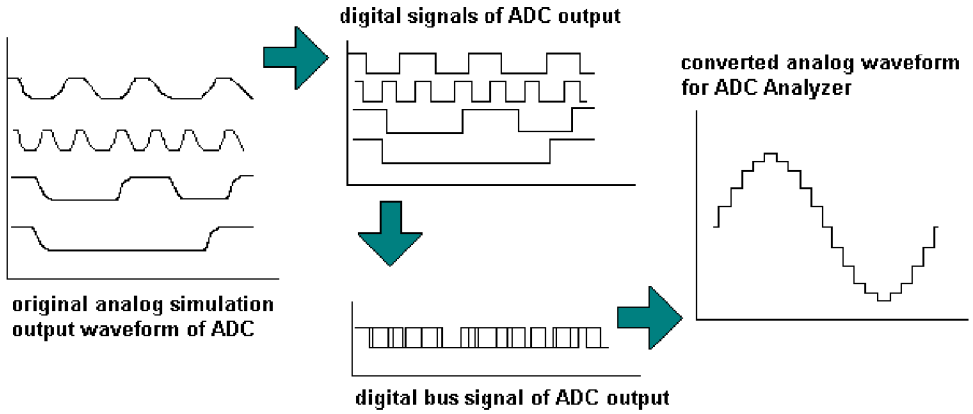
Input Signal Requirements

The input waveform for the CSO ADC Toolbox must have its signal strength in integer code values (for example, 0 to 63 for a 6-bit ADC). Run a simulation on an ADC design first and then convert the ADC output codes into an acceptable waveform.

Preparing the Input Waveform

As depicted in the flowchart below, the following steps demonstrate how to use the WaveView tool to convert the simulated ADC output code (in analog waveforms) to an acceptable input signal for the CSO ADC Toolbox.

Figure 6 Preparing an input signal for the ADC toolbox



To convert the simulated ADC output code to an acceptable input signal for the CSO ADC Toolbox:

1. Identify the analog output waveforms that represent the ADC output code, for example, "ad0" to "ad5" for a 6-bit design.
2. Display these signals in a waveview window and then highlight them by highlighting the panels these signals reside in.
3. Use the A/D converter tool in the WaveView single-bit mode to convert each of the highlighted analog waveforms to a digital waveform. Name the converted digital waveforms "AD0" - "AD5", and ensure proper threshold value is used for this A to D conversion step.
4. Group the converted logic signal "AD0" - "AD5" into a digital bus signal. Make sure that the signals are ordered (sorted) correctly so "AD5" is the MSB and "AD0" is the LSB.
5. Rename the grouped bus signal to "AD". You can use this logic bus signal directly as the input to the ADC Toolbox, or you can continue to Step 6.

Feedback

To use the logic bus signal as the input, the bus signal must be displayed in a waveview window, and you must drag and drop the logic bus signal from the display panel.

6. Use the D/A convertor tool in the WaveView tool to convert the grouped converted digital bus "AD", in multi-bit mode, into an analog waveform.
7. Ensure that the **Use logic bus value** option is selected so that the converted waveform has strength in code value.
8. Rename the final analog waveform "ad".

   The analog waveform "ad" is now ready for use with the ADC Toolbox. Skip Step 1 to Step 3 if your simulator generates logic output waveforms directly. If your simulator directly generates a logic bus waveform, skip Step 1 to Step 5.

To specify the input signal, drag and drop the converted analog signal from the Waveform Browser to the Signal field in the ADC Toolbox.

Sampling the Input Waveform

The CSO ADC Toolbox samples the input signal waveform based on the following equation:

$$N \cdot Fin = FinNUM \cdot Fclk$$

where:

- *N* is the number of sampling points (power of 2)
- *Fin* is the input frequency
- *FinNUM* is the total number of cycles of a sine signal input used for coherent sampling.
- *Fclk* is the sampling frequency

Note:

N and *FinNUM* must not have any common factors.

For sine wave input, the number of sample points N must to be power of 2 because FFT is used internally to calculate the ADC dynamic performance. It is important to choose a value for N, which is large enough to produce at least one sample point for each output code value. In the ADC Toolbox, PWC sampling is always used to sample the input signal waveform.

Feedback

Preparing Test Benches for the CSO ADC Toolbox

To maximize the effectiveness of the CSO ADC Toolbox, the following guidelines can be used to prepare the input test tone for simulating an ADC design.

Input Signal Strength

For both ramp and sine input type, the ideal input signal strength drives the ADC output over a full swing output code from 0 to 2^BIT-1, where BIT is the number of bits. On the other hand, the input signal cannot be too strong, because it can overdrive the ADC outside its operating range. Proper input signal strength results in more accurate INL/DNL analysis results.

Input Sine Wave Frequency

In general, depending on the speed performance of the ADC design, the input test tone frequency might be low enough so that the digital output does not skip codes. (All output codes can be exercised by the input test tone.) For coherent sine wave sampling, the exact input test tone frequency Fin must be determined before running the ADC simulation.

CSO ADC Toolbox User Interface

The CSO ADC Toolbox accepts user input parameters and generates graphic and textual analysis report.

After completing an analysis based on user input parameters, the CSO ADC Toolbox allows you to perform the following actions:

- Display analysis results in regular waveview windows
- Export analysis result to text files
- Save/load the analysis parameters setup

Input Parameters

The following table summarizes the input parameters required by the CSO ADC Toolbox.

Input Signal Type	Sine	Ramp
Start Time	mandatory	mandatory
Precision	mandatory	not required

Input Signal Type	Sine	Ramp
Decimation	mandatory	not required
Sample Points	mandatory	mandatory
Fclk User	mandatory	mandatory
Fin User	mandatory	not required
Fin #	optional	not required
Fclk	optional	not required
Fin	optional	not required
Bits	mandatory	mandatory
THD	mandatory	not required
SNR	mandatory	not required

- Start Time: The time at which the ADC Toolbox starts sampling the input signal.
- Precision: Number of digits after the decimal point for the internal Fclk, Fin and Fin# calculation. For the sine wave coherent sampling method, small error in Fclk and Fin can result in inaccurate analysis result. If the **Use user Fclk/Fin** option is not selected, the ADC Toolbox recalculates the precise values internally for Fclk, Fin, and Fin#.
- Decimation: Integer decimation-in-time. Default is 1. The decimation is used to mimic the sampling speed limitation of real measurement equipment by slowing down the sampling clock by the integer decimation factor. The actual sampling clock frequency is Fclk/decimation. When the ADC Toolbox is used on simulation data, set the value to 1.
- Sample Points: Number of sample points. Since the ADC Toolbox uses FFT for noise analysis, the number of sample points must be power of 2.
- Fclk user: The frequency of the sampling clock.
- Fin user: The frequency of the input sine wave. Ignored for ramp input.
- FinNUM: The total integer number of cycles of input sine signal for coherent sampling. Required only when the **Use user Fclk/Fin** option is selected. It is otherwise calculated internally based on other input parameters. Ignored for ramp input.

  Note:

  If the **Use user Fclk/Fin** option is selected, the Fclk-user value is used directly as the sampling frequency for acquiring data from the input signal. The input data is sampled for a total span of Sample-points / Fclk-user.

Feedback

If the **Use user Fclk/Fin** option is not selected, the ADC Toolbox uses Precision, Fin-User, Fclk-User, Sample Points, and Decimation to calculate the precise internal Fclk, Fin, and FinNUM. In this case, you only need to provide approximate Fclk and Fin values. The final internally calculated Fclk, Fin, and FinNUM values are displayed in corresponding fields.

- Bits/Min/Max: Number of bits of the AD converter design, and the minimal and maximal code value (MIN/MAX). The MIN/MAX values, instead of the Bits value, are used if the **Use Code Value Range** option is selected. If the **Use Code Value Range** option is not selected, the Bits value is used and the code value is assumed to range from 0 to 2^Bits - 1. Any sampled code values that are outside this range are clipped at the MIN/MAX values.
- THD: The highest harmonic order that is included as harmonic for the total harmonic distortion calculation.
- SNR: The highest harmonic order that is excluded from noise for the signal -to-noise ratio calculation.
- Indexed Data: If the input data has been sampled at a fixed rate without the x-axis time information for each of the data points (for example, measurement data), select this option and the ADC Toolbox assumes that the data is sampled at Fclk.

  After entering all the required input parameters and the options, click **Evaluate** to complete the analysis.

Selecting Sampling Parameters

The following sampling parameters are available:

- Sine Input
- Ramp Input

Sine Input

To select a sine input sampling parameter:

1. Select the Sine option.
2. Enter a Sample Pts value.
3. If the **Use user Fclk/Fin** option is selected, enter proper Fclk, Fin, and FinNUM values. These values are used directly by the waveform sampling process.
4. If the **Use user Fclk/Fin** option is selected, enter proper Fclk, Fin, and FinNUM values. These values are used directly by the waveform sampling process.

Feedback

5. If the **Use user Fclk/Fin** option is not selected, enter the preferred Precision value and estimated Fclk and Fin values. The FinNUM value and the exact Fclk and Fin values are calculated automatically.
6. Select the **Sort Sampled Points** option to view the unfolded power spectrum of the sorted sampled waveform, or de-select the **Sort Sampled Points** option to view the folded power spectrum of the unsorted sampled waveform. You need to click **Evaluate** every time the **Sort Sampled Points** option is changed.

Ramp Input

Select the **Ramp** option and enter the Start, Sample-Pts, and Fclk values.

Exporting DNL/INL/FFT Results as Waveform Data

The ADC Toolbox result, including the sampled waveform data, histogram, INL/DNL and the FFT power spectrum, can be exported to a waveview window for further viewing purpose. Select the preferred data type switches in the ADC Toolbox and click **Waveform** to display the data into a new waveview window.

Saving or Loading an Analysis Setup

The ADC Toolbox setup, including the input parameters, sampled data and the analysis result can be saved into data files. To restore a previously saved setup, load from the .INI (Input parameter) file.

ADC Toolbox Display Controls

The following ADC toolbox display controls are available:

- Sampled Signal Window
- Histogram, DNL, and INL Windows
- Power Spectrum (Sine Input Only)

Sampled Signal Window

The Sampled Signal window displays the sampled data waveform based on the input parameters. The y-axis is in code value, with min-code value at the bottom and max-code value at the top.

Feedback

Ideal/Acquired/Error/All switch:

- Ideal: Display the ideal signal used for INL/DNL calculation.
- Acquired: Display the sampled waveform data.
- Error: Display the error between ideal signal and the sampled data.
- All: Display all of the above.

Sin/Cos/Original (sine input only)

- Sin: Adjust the phase of the displayed waveform into Sin phase (starts from zero).
- Cos: Adjust the phase of the displayed waveform into Cos phase (starts from the maximum value).
- Original: No phase adjust for sampled waveform display.

Exp/Integer

- Exp: Display the y-axis value ranges using the engineering format
- Integer: Display the y-axis value ranges using the integer code values

Histogram, DNL, and INL Windows

Theses windows display the histogram, DNL, and INL plots of the sampled signal. Aligned with the y-axis of the Sampled Signal plot, the y-axes of all three plots correspond to the code value. The x-axis is the histogram / DNL / INL value in LSB.

Power Spectrum (Sine Input Only)

The power spectrum window displays the noise spectrum of the sampled signal.

Line/Bar/Both

- Line: Plot the power spectrum using line chart.
- Bar: Plot the power spectrum using bar chart.
- Both: Plot the power spectrum using both line and bar chart.

Index/Frequency

- Index: Display the spectrum x-axis value range using harmonic index.
- Frequency: Display the spectrum x-axis value range using frequency.

AC Dynamic Characteristics

The AC Characteristics window displays the spectrum harmonic strength and the following values:

- SNR: The signal-to-noise ratio
- THD: The total harmonic distortion
- ENOB: The effective number of bits
- SNDR: The signal to noise and distortion ratio
- SFDR: The spurious free dynamic range. The harmonic index of the second strongest harmonic component is also displayed together with the SFDR value (for example, SFDR index_after_sorting (index_before_sorting)).

The strength of the power spectrum is in db relative to the strongest harmonic component (for example, 0 db for the strongest harmonic component). The AC characteristic window lists the value of each of the harmonic component. Use the **up** or **down** buttons to scroll up/down to view the spectrum values at different harmonic range. Select the toggle check box next to a harmonic component to exclude the harmonic for the THD calculation or include the harmonic for the SNR calculation.

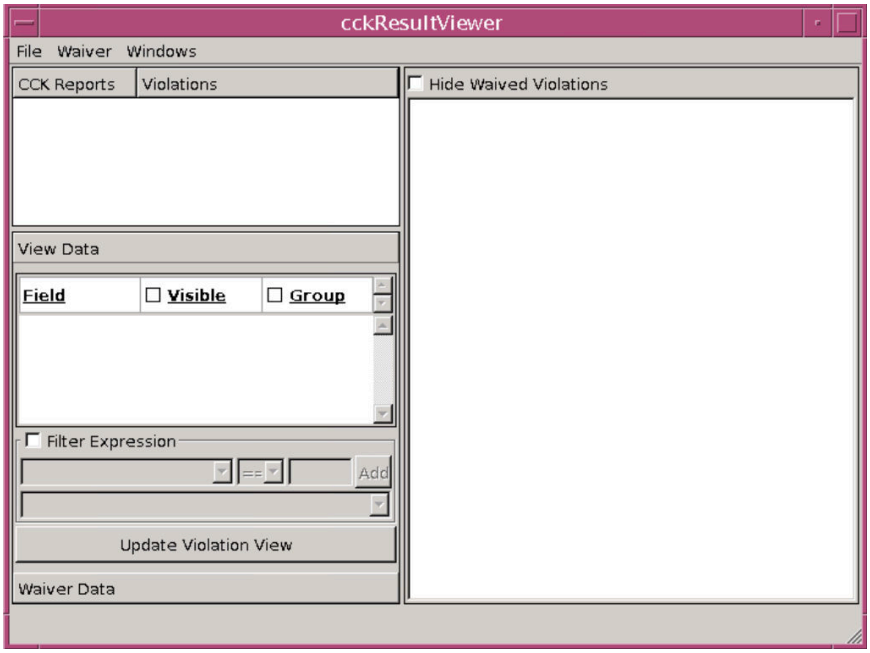
Click **Evaluate** to update the AC noise analysis result (SNR/THD/ENOB/SNDR/SFDR) based on the new harmonic selection.

DC Static Characteristics

The DC Characteristics window displays the min/max values and the standard deviation of the INL/DNL distribution. The value is in LSB units.

Using the CCK Viewer

To load, view, and export CircuitCheck error reports, click **CCK Viewer** on the **Tools** tab.



The CCK Result Viewer window opens. For more information on CCK Results, see the *CustomSim Circuit Check Reference Manual*.

This section contains information on the following topics:

- Loading CircuitCheck Error Reports
- Exporting CircuitCheck Error Reports

Loading CircuitCheck Error Reports

To load a CircuitCheck error report:

1. Ensure the **CCK Result Viewer** is open (click **CCK Viewer** on the **Tools** tab).
2. Choose **File > Import CCK Result File** from the CCK Result Viewer menu bar.

   The **Import CCK Error Report** window opens.
3. Browse to the location of the result file, and click **Open**.

   The error report is displayed in the CCK Result Viewer.

Exporting CircuitCheck Error Reports

To export an error report:

1. In the **CCK Result Viewer** window, choose **File > Export CCK Result File**.

   The **Export CCK Report Result File** window opens.

2. In the **Files of type** field, select the file format for the exported report.
3. Choose a name for the exported report, and click **OK**.

   The error report is exported and saved to the location you selected.

Using CSA (Channel Simulation Analysis)

Note:

This feature requires a WaveView ADV license.

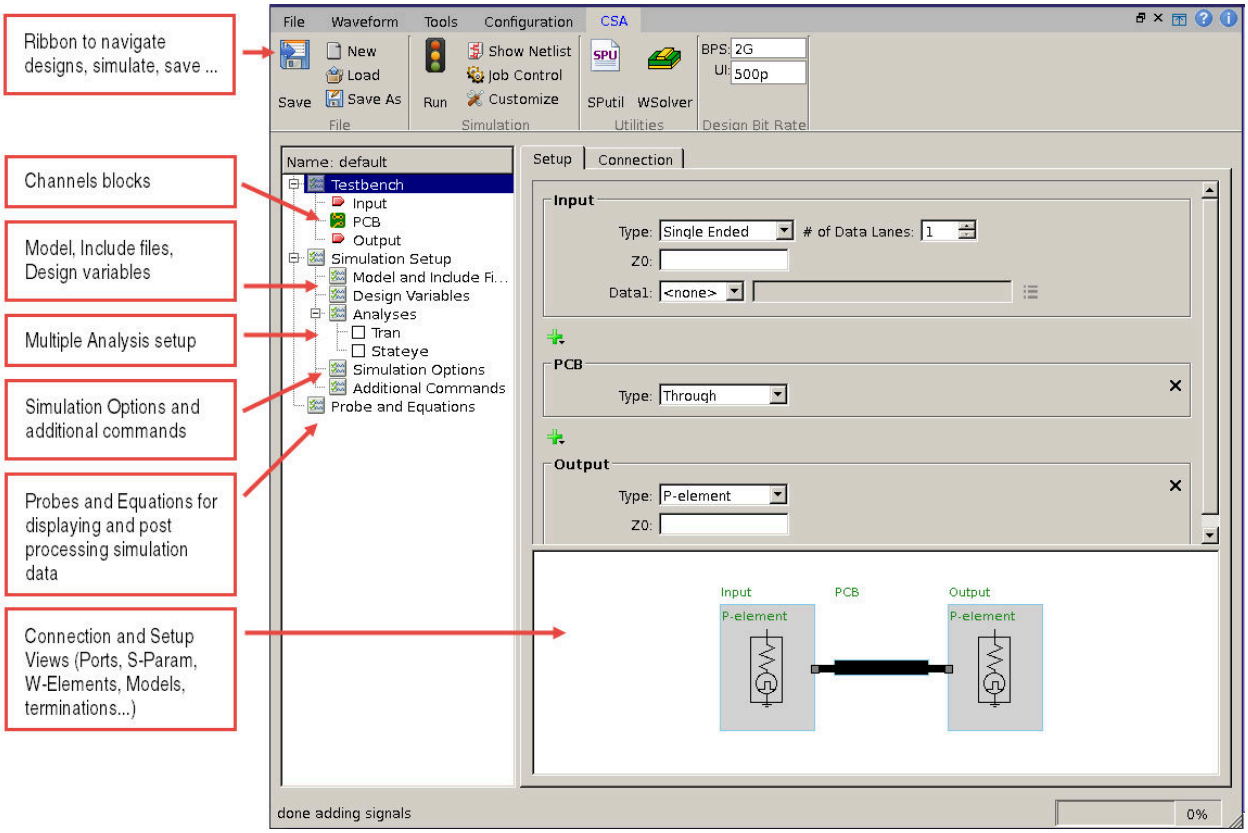
The CSA environment provides an environment for simulating and analyzing PrimeSim HSPICE channel simulation (CS) designs. In this environment you can set up CS testbenches, run PrimeSim HSPICE simulations, and analyze CS results.

- Getting Started With the CSA Environment
- Setting Up CS Testbenches
- Setting Up CS Simulations
- Adding CS Probes and Equations
- Adding CS Corners
- Show Netlist
- SPUTIL
- Run

Getting Started With the CSA Environment

To start the CSA Environment, click **CSA** on the **Tools** tab.

Feedback



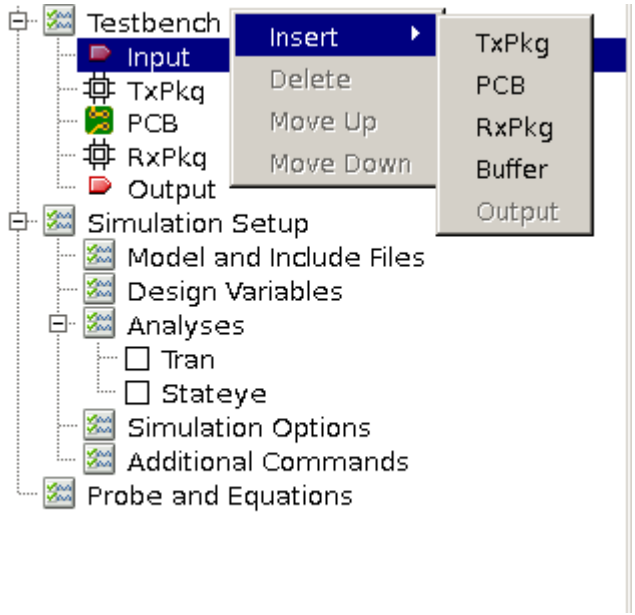
The **Channel Simulation and Analysis** window opens.

The CSA environment window contains the following parts:

- **Navigation panel**

  The Navigation panel displays the categories of all testbench, simulation, and probe/ equation setup.

Feedback



The **Navigation** panel tree contains the following items:

Item Name		Description
Testbench		General settings for the Input, TxPkg, PCB, RxPkg and Output. See Setting Up CS Testbenches for information on setting up these testbench stages.
	Input Element Options	Detailed settings for input elements, which includes general settings and additional options.
	TxPkg Options	Detailed settings for TxPkg, which includes general settings and additional options. Visible if you right-click and choose **Insert** (as above).
	RxPkg Options	Detailed settings for RxPkg, which includes general settings and additional options. Visible if you right-click and choose **Insert**.
	Output Element Options	Detailed settings for output elements, which includes general settings and additional options.
Simulation Setup		Simulation executable path and options. See Setting Up CS Simulations for information on setting up these simulation options.
	Model and Include Files	Model file, include file, and vec file setup.

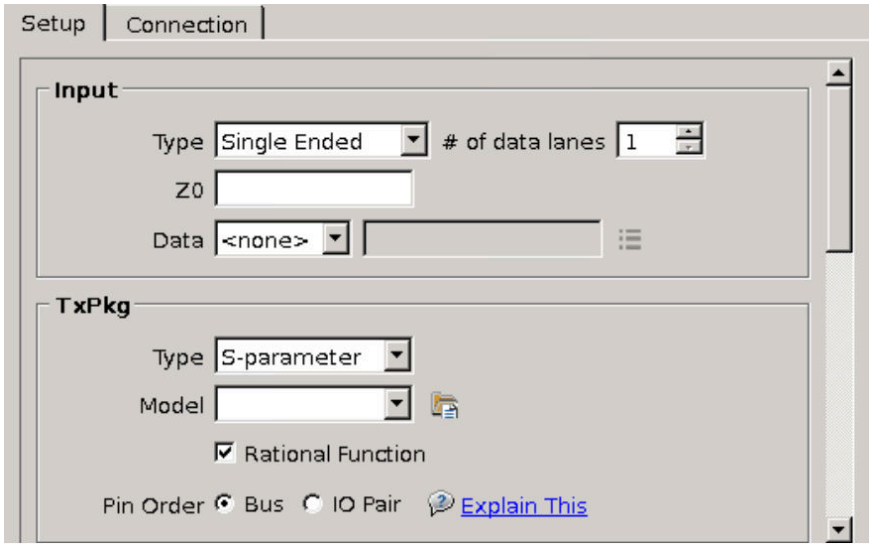
Item Name		Description
	Design Variables	Design variables and temperature setup.
	Analyses	Simulation analyses setup, which includes OP, AC, DC, Tran, Noise, Lin, and Stateye analyses.
	Sweep Data	Sweep data. **Note:** This setup option is not displayed in the Navigation panel by default. See Customize for more information on customizing the list testbench and simulation settings listed in the Navigation panel.
	Simulation Options	SPICE .option commands setup.
	Additional Commands	SPICE command in addition to built-in analyses commands.
Probe and Equations		Probe and equation setup. See Adding CS Probes and Equations for information on setting up probes and equations.
Corners		Corners setup. **Note:** This setup option is not displayed in the Navigation panel by default. See Customize for more information on customizing the list testbench and simulation settings listed in the Navigation panel.

Note:

Some setup items in the Navigation panel might not be visible. See Customize for information on controlling which items you want to include in the Navigation panel.

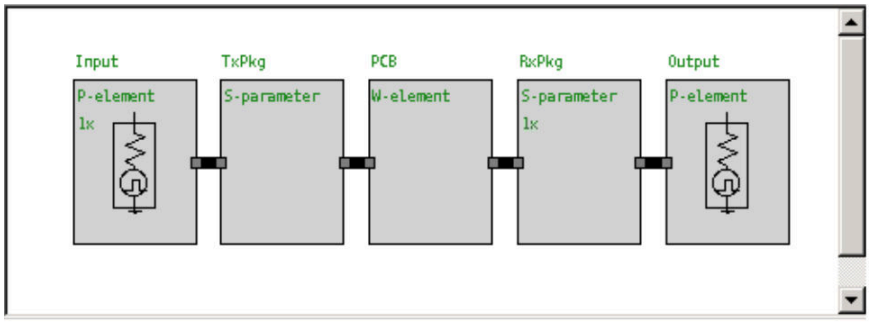
- **Setup panel**

The **Setup** panel contains testbench setup details for the testbench setup item you choose in the **Navigation** panel.



- **Stage Diagram Viewer**

  The Stage Diagram Viewer panel is a visual representation of the CS testbench you are setting up.



  As you set up your CS testbench, each part of the setup is added to this panel as a new stage. You can click on each stage to view the testbench setup details, which are displayed in the Setup panel.

  Note:

  If you do not see the Stage Diagram Viewer panel, click the **Testbench** category in the Navigation panel.

Customize

To customize the list of CS setup options that are visible in the **Navigation** pane:

1. Click **Customize** in the **Simulation** section of the **CSA** tab.

   The **Setup Customize** dialog box opens.

2. Click the check box next to each setup option that you want to access in the **Navigation** pane.
3. Click **OK** to save your changes.

   The setup options you choose are displayed in the **Navigation** pane.

Setting the Initial Design Bit Rate

To set the initial design bit rate:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.
2. Click the **Channel Sim. & Analysis** preference category in the Preferences navigation pane.

   The Channel Simulation and Analysis preferences are displayed.
3. Enter an initial design bit rate value in the **Initial Design Bit Rate (bps)** text box.

   `2G` is the default initial design bitrate.
4. Click the **Apply** button to save your changes.

Setting the Initial Resistance for Terminating Pins

To set the initial resistance for terminating pins:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.
2. Click the **Channel Sim. & Analysis** preference category in the Preferences navigation pane.

   The Channel Simulation and Analysis preferences are displayed.
3. Enter an initial resistance value for terminating pins in the **Initial Appropriate Resistance for Terminating Pins (ohms)** text box.

   `50` is the default initial resistance value.
4. Click the **Apply** button to save your changes.

Saving Testbench Stages When Exiting CSA

To save the current testbench stages to an initial file when you exit CSA:

1. Click **Settings** on the **Configuration** tab.

   The **Preference Settings** window opens.

2. Click the **Channel Sim. & Analysis** preference category in the Preferences navigation pane.

   The Channel Simulation and Analysis preferences are displayed.

3. Click the **Save Current Series of Testbench Stages to Initial File When Exiting CSA** check box to enable the option.

   This option is disabled by default.

4. Click the **Apply** button to save your changes.

   The current series of testbench stages are now saved to your `.cxinit` file when CSA is closed using the following format:

   ```
   cx_init_stages stage1 stage2 … stageN
   ```

   For example:

   ```
   cx_init_stages Input TxPkg PCB RxPkg Output
   ```

Setting Up CS Testbenches

This section contains information on the following topics:

- Working With Testbench Stages
- Setting Up Input Element Options
- Setting Up TxPkg Options
- Setting Up PCB Options
- Setting Up RxPkg Options
- Setting Up Buffer Options
- Setting Up Output Element Options
- Working With the CS Connection View

Working With Testbench Stages

When you set up testbench stages, you can also configure the order in which they are simulated.

Feedback

You must adhere to the following rules when setting up testbench stages:

- The `Input` stage is always the first stage, and you cannot delete or move the `Input` stage.
- The `Output` stage is always the last stage, and you cannot delete or move the `Output` stage.
- No more than one `Input` stage and one `Input` can exist in one testbench setup.

This section contains information on the following topics:

- Adding and Deleting Stages
- Changing Stage Order
- Renaming Stages

Adding and Deleting Stages

To add a testbench stage:

1. Click `Testbench` in the navigation pane.

   The **Setup** tab is displayed with all the currently enabled stages in the order of simulation.

2. Click the **Add** button  in the upper-left corner of a stage section, and choose a stage type from the menu that appears.

   A stage is added just before the **Add** button you use, and the stage name is also added to the list of stages in the Navigation pane. A block diagram of the new channel connection is also displayed.

   Note:

   The **Output** stage is not available if you already have an Output stage included in your testbench.

To delete a stage, click the  button at the right-hand side of a stage section. The stage is removed from the **Navigation** pane, and the block diagram is updated.

Changing Stage Order

To change the order of stages, right-click a stage name in the **Navigation** pane, and choose **Move Up** or **Move Down** from the menu that opens. The stage moves one position up or down, respectively.

Note:

The stage just after the `Input` stage cannot be moved up, and the stage just before the `Output` stage cannot be moved down.

Feedback

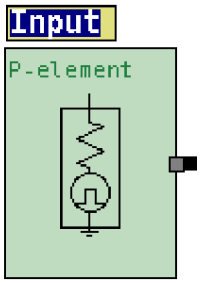
Renaming Stages

To rename a stage:

1. Click `Testbench` in the navigation pane.

   The **Setup** tab is displayed with all the currently enabled stages in the order of simulation.

2. Double-click a stage name in the block diagram.



   The stage name is highlighted, and a text box opens around the stage name.

   Note:

   If you choose a stage name that is too long to display in the ConnectionView, only the beginning and ending characters of the stage name are displayed with an ellipsis in the middle. Hover your mouse pointer over the shortened stage name to view the full stage name.

3. Enter a new name, and press the **Return** key.

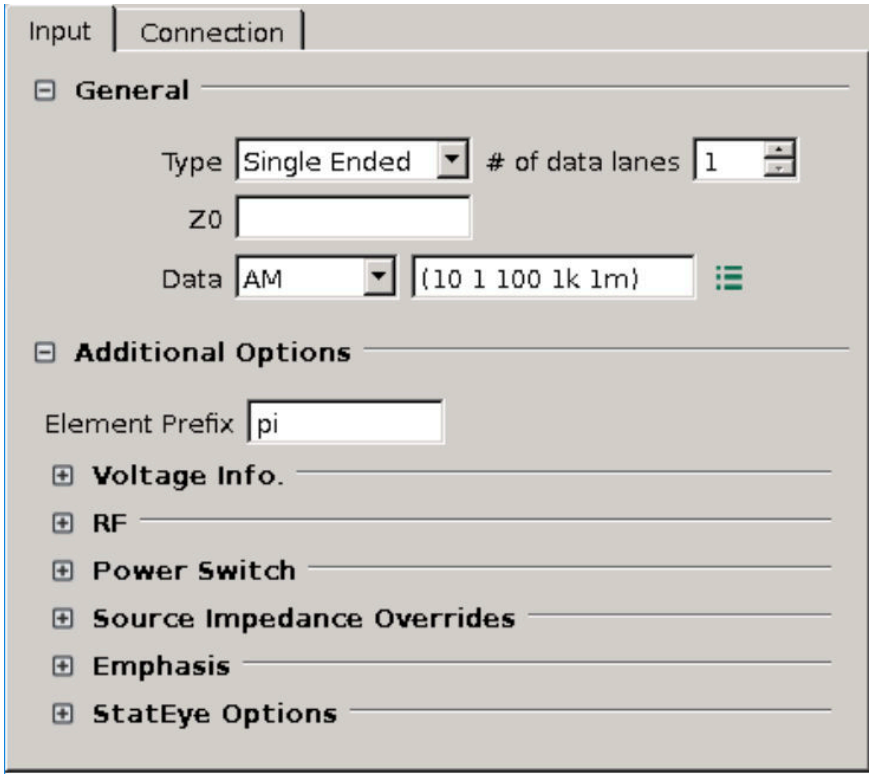
   The new stage name is saved.

Feedback

Setting Up Input Element Options

To set up input element options:

1. Click **Input Element Options** in the Navigation panel.



   The Input tab page is displayed.

   Note:

   You can click on the Connection tab at any time to view and change connections. See Working With the CS Connection View for more information.

2. In the General section, choose an input element type from the **Type** menu.

   If you choose **Single Ended** or **Differential Pair**, continue to the next step.

   If you choose **Subckt**, skip to Step 7.

3. Choose the number of data lanes from the **# of data lanes menu**
4. Enter a Z0 value in the **Z0** text field.
5. Choose the type of data from the **Data** menu.

Feedback

Default values are automatically entered for each data type. If you want to change any of those values, click the  icon, and enter new values.

6. Enter any additional options under any of the sub categories under the Additional Options category.

   See the *HSPICE® User Guide: Signal Integrity Modeling and Analysis* for more information on these additional options.

   Skip to Step 9

7. Choose the number of data lanes from the **# of data lanes menu**

8. Choose a subcircuit from the **Subckt** menu, or click the  icon to browse to and load a subcircuit file.

9. The setup for the Input Element Options is complete.

Move on to the next testbench setup section, Setting Up TxPkg Options, or jump to one of the following other testbench setup sections:

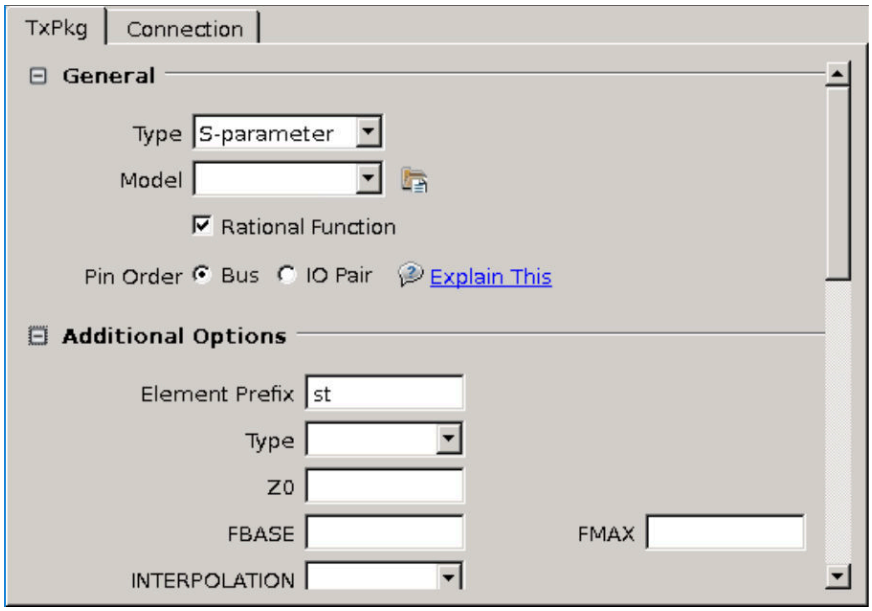
If you are done with your CS testbench setup, see Setting Up CS Simulations or Adding CS Probes and Equations sections.

Feedback

Setting Up TxPkg Options

To set up TxPkg options:

1. Right-click **Input** in the navigation panel, and choose **Insert > TxPkg** from the menu that opens.



   The TxPkg tab page is displayed.

   Note:

   You can click on the Connection tab at any time to view and change connections. See Working With the CS Connection View for more information.

2. Choose a package type from the **Type** menu.

   If you choose **S Parameter**, continue to the next step.

   If you choose **Subckt**, skip to Step 7.

   If you choose **Through**, skip to Step 9.

3. Choose a model from the **Model** menu, or click the  icon to browse to and load a model file.

4. (Optional) Click the **Rational Function** check box if this a rational function.

   This options in enabled by default.

5. Choose a **Pin Order**.

Feedback

6. Enter any additional options under any of the sub categories under the Additional Options category.

   See the *HSPICE® User Guide: Signal Integrity Modeling and Analysis* for more information on these additional options.

   Skip to Step 9.

7. Choose a subcircuit from the **Subckt** menu, or click the  icon to browse to and load a subcircuit file.
8. Choose a **Pin Order**.
9. The setup for the TxPkg Options is complete.

Move on to the next testbench setup section, Setting Up PCB Options, or jump to one of the following other testbench setup sections:

- Setting Up Input Element Options
- Setting Up RxPkg Options
- Setting Up Output Element Options

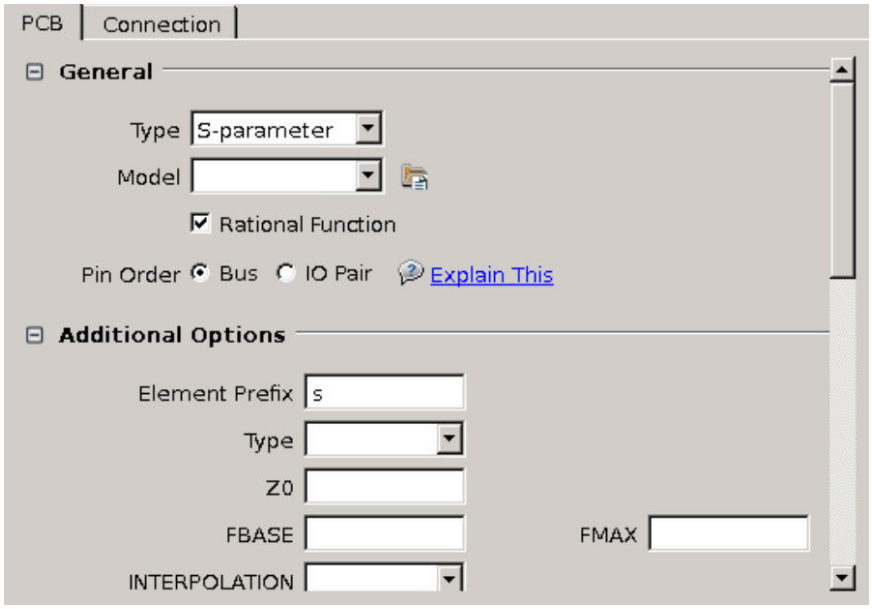
If you are done with your CS testbench setup, see Setting Up CS Simulations or Adding CS Probes and Equations sections.

Feedback

Setting Up PCB Options

To set up PCB options:

1. Click **PCB Options** in the Navigation panel.



   The PCB tab page is displayed.

   Note:

   You can click on the Connection tab at any time to view and change connections. See Working With the CS Connection View for more information.

2. Choose a PCB type from the **Type** menu.

   If you choose **W Element**, continue to the next step.

   If you choose **S Parameter**, skip to Step 6.

   If you choose **Subckt**, skip to Step 7.

   If you choose **Through**, skip to Step 12.

3. Enter the length value in the **Length** text field.

   Default is 2.

4. Choose a model from the **Model** menu, or click the  icon to browse to and load a model file.

Feedback

5. Choose a **Pin Order**.

   Skip to Step 12.

6. Choose a model from the **Model** menu, or click the  icon to browse to and load a model file.

7. (Optional) Click the **Rational Function** check box if this a rational function.

   This options in enabled by default.

8. Choose a **Pin Order**.

9. Enter any additional options under any of the sub categories under the Additional Options category.

   See the *HSPICE® User Guide: Signal Integrity Modeling and Analysis* for more information on these additional options.

   Skip to Step 12.

10. Choose a subcircuit from the **Subckt** menu, or click the  icon to browse to and load a subcircuit file.

11. Choose a **Pin Order**.

12. The setup for the PCB Options is complete.

Move on to the next testbench setup section, Setting Up RxPkg Options, or jump to one of the following other testbench setup sections:

- Setting Up Input Element Options
- Setting Up TxPkg Options
- Setting Up PCB Options
- Setting Up Output Element Options

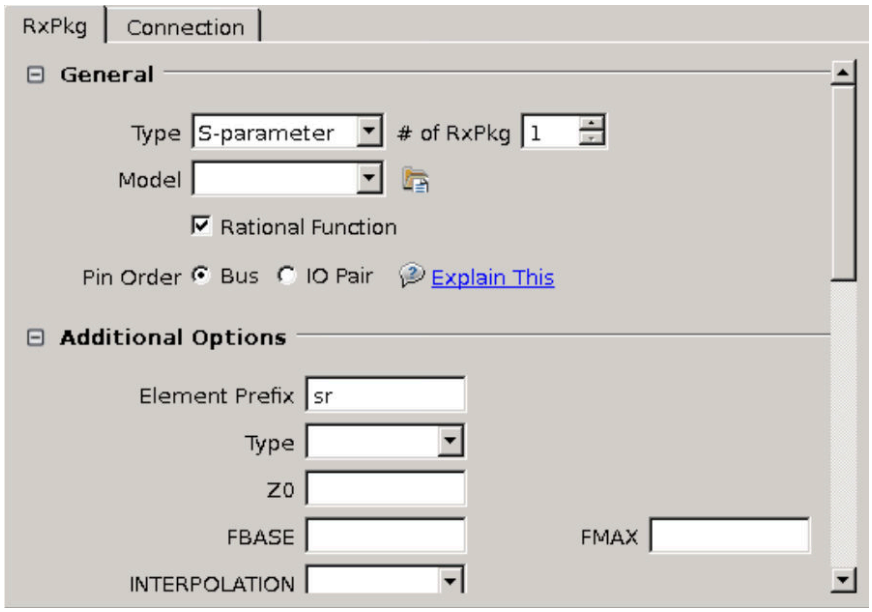
If you are done with your CS testbench setup, see Setting Up CS Simulations or Adding CS Probes and Equations sections.

Feedback

Setting Up RxPkg Options

To set up RxPkg options:

1. Right-click **PCB** in the navigation panel, and choose **Insert > RxPkg** from the menu that opens.



   The RxPkg tab page is displayed.

   Note:

   You can click on the Connection tab at any time to view and change connections. See Working With the CS Connection View for more information.

2. Choose a package type from the **Type** menu.

   If you choose **S Parameter**, continue to the next step.

   If you choose **Subckt**, skip to Step 8.

   If you choose **Through**, skip to Step 10.

3. Choose the number of packages from the **# of Rx Pkg** menu.

4. Choose a model from the **Model** menu, or click the  icon to browse to and load a model file.

5. (Optional) Click the **Rational Function** check box if this a rational function.

   This options in enabled by default.

Feedback

6. Choose a **Pin Order**.
7. Enter any additional options under any of the sub categories under the Additional Options category.

   See the *HSPICE® User Guide: Signal Integrity Modeling and Analysis* for more information on these additional options.

   Skip to Step 10.
8. Choose a subcircuit from the **Subckt** menu, or click the  icon to browse to and load a subcircuit file.
9. Choose a **Pin Order**.
10. The setup for the RxPkg Options is complete.

Move on to the next testbench setup section, Setting Up Output Element Options, or jump to one of the following other testbench setup sections:

- Setting Up TxPkg Options
- Setting Up PCB Options
- Setting Up RxPkg Options

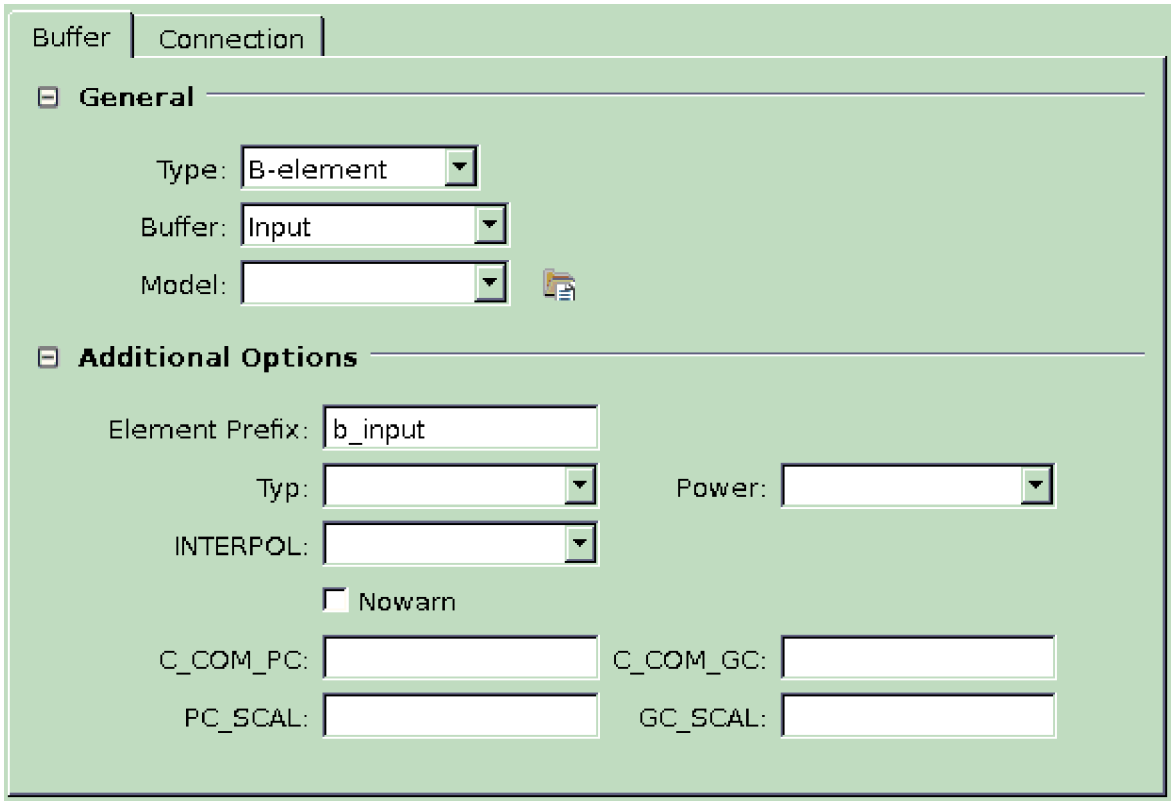
If you are done with your CS testbench setup, see Setting Up CS Simulations or Adding CS Probes and Equations sections.

Feedback

Setting Up Buffer Options

To set up Buffer options:

1. Click **Buffer** in the Navigation panel.



   The Buffer tab page is displayed.

   Note:

   You can click on the Connection tab at any time to view and change connections. See Working With the CS Connection View for more information.

2. Choose an output element type from the **Type** menu.

   If you choose **B Element**, continue to the next step.

   If you choose **Subckt**, skip to Step 7.

   If you choose **Through**, skip to Step 8.

3. Choose a buffer from the **Buffer** menu.

Feedback

4. Choose a model from the **Model** menu, or click the  icon to browse to and load a model file.
5. Enter any additional options under any of the sub categories under the **Additional Options** section.

   See the *HSPICE® User Guide: Signal Integrity Modeling and Analysis* for more information on these additional options.

   Skip to Step 8.
6. Enter an element prefix in the **Element Prefix** text field.

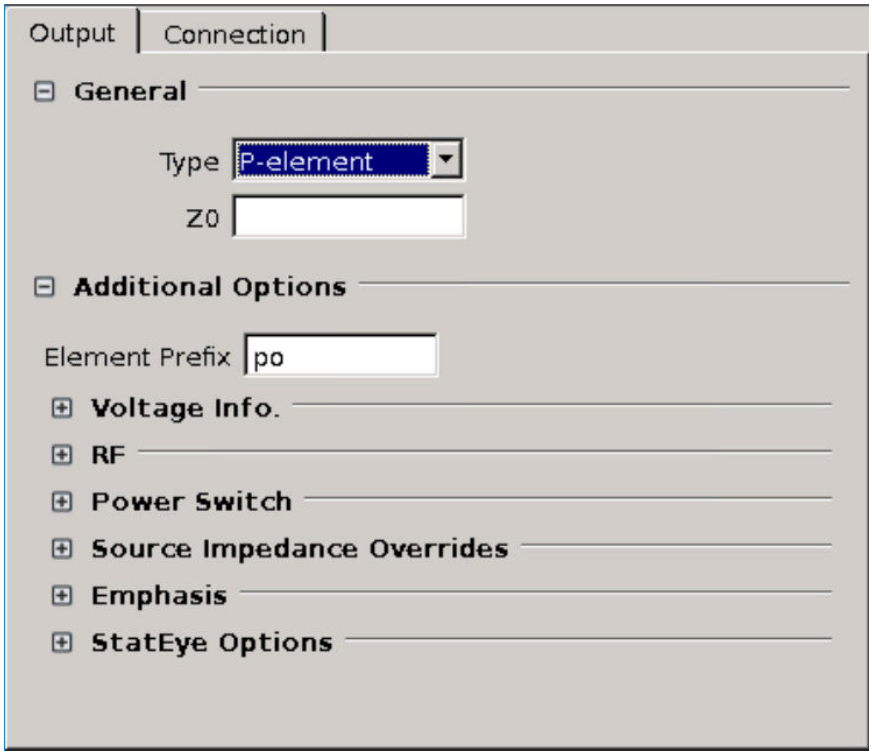
   The `b_input` text is the Element Prefix by default.
7. Choose a subcircuit from the **Subckt** menu, or click the  icon to browse to and load a subcircuit file.
8. The setup for the Buffer Options is complete.

Feedback

Setting Up Output Element Options

To set up Output Element options:

1. Click **Output Element Options** in the Navigation panel.



   The Output Elements tab page is displayed.

   Note:

   You can click on the Connection tab at any time to view and change connections. See Working With the CS Connection View for more information.

2. Choose an output element type from the **Type** menu.

   If you choose **P Element**, continue to the next step.

   If you choose **Resistor Load**, skip to Step 5.

   If you choose **Subckt**, skip to Step 6.

3. Enter a Z0 value in the **Z0** text field.

4. Enter any additional options under any of the sub categories under the **Additional Options** section.

Feedback

See the *HSPICE® User Guide: Signal Integrity Modeling and Analysis* for more information on these additional options.

Skip to Step 7.

5. Enter a resistor load value into the **Value** text field.

   Skip to Step 7.

6. Choose a subcircuit from the **Subckt** menu, or click the  icon to browse to and load a subcircuit file.

   Skip to Step 7.

7. The setup for the Output Element Options is complete.

Move on to the next testbench setup section, Setting Up PCB Options, or jump to one of the following other testbench setup sections:

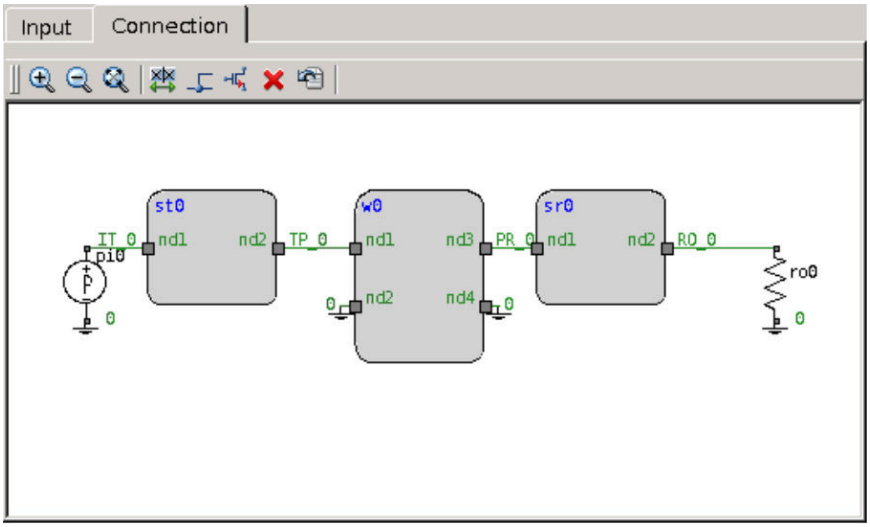
- Setting Up Input Element Options
- Setting Up RxPkg Options
- Setting Up Output Element Options

If you are done with your CS testbench setup, see Setting Up CS Simulations or Adding CS Probes and Equations sections.

Working With the CS Connection View

The CS Connection View displays the current connections for which you are setting up an CS testbench. You can access the CS Connection View via the Connection tab from the Setup panel for any testbench setup option in the Navigation panel.

Feedback



This section contains information on the following topics:

- Changing Pin Direction
- Adding Connections
- Inserting Devices
- Connecting a Device to Ground
- Inserting Dummy Ports
- Terminating Pins
- Deleting Devices or Pins
- Viewing the Device Setup
- Zooming In and Out
- Zooming to Fit
- Resetting the CS Connection View Display

Changing Pin Direction

To change the direction of a pin in the CS Connection View, right-click a pin, and choose **Change Pin Direction** from the menu that opens.

Adding Connections

To add a connection in the CS Connection View, right-click a pin, and choose **Add Connection** from the menu that opens. Left-click another pin to complete the connection.

Inserting Devices

To insert a device in the CS Connection View:

1. Right-click anywhere in the Connection View pane, and choose **Insert Device** from the menu that opens.

   The Insert Device dialog box opens.

2. Choose an element from the **Element** menu.

3. Enter a name for the device in the **Name** text field.

4. Choose a device category from the **Category** menu.

   The category varies depending on what kind of element you choose.

   For information on the elements or categories, see the *HSPICE® User Guide: Signal Integrity Modeling and Analysis*.

5. Click **OK** to save your settings.

   The device is added to the CS Connection View.

Connecting a Device to Ground

To connect a device to ground in the CS Connection View, right-click a device, and choose **Connect to GND** from the menu that opens.

Inserting Dummy Ports

To insert a dummy Port in the CS Connection View, right-click a connection, and choose **Insert Dummy Port** from the menu that opens. A dummy port is added to the Connection View.

You can also add dummy ports via stateye analysis setup.

Terminating Pins

To terminate a pin, right-click a pin, and either choose **Terminate Pin** from the menu that opens or click the **Terminate Pin** button  in the button toolbar.

Deleting Devices or Pins

To delete a pin or device in the CS Connection View, right-click a pin or device, and choose **Delete** from the menu that opens.

Viewing the Device Setup

To view the settings for a device in the CS Connection View, right-click a device, and choose **Device Setup** from the menu that opens. The Setup panel opens with the device settings for the selected device.

Feedback

Zooming In and Out

To zoom into a connection in the CS Connection View, click the  button in the button toolbar.

To zoom out, click the  button.

Zooming to Fit

To zoom the display to fit the size of your connections in the CS Connection View, click the  button in the button toolbar.

Resetting the CS Connection View Display

To reset the CS Connection View display, click the  button in the button toolbar. All devices and connections revert back to their original state.

Setting Up CS Simulations

This section contains information on the following topics:

- Specifying General CS Simulation Settings
- Adding CS Model Files
- Adding CS Include Files
- Adding CS VEC Files
- Adding CS Design Variables
- Adding CS Analyses
- Adding CS Sweep Data
- Adding CS Simulation Options
- Adding Additional CS Commands

Specifying General CS Simulation Settings

To start setting up your CS simulation:

1. Click the **Simulation Setup** category in the **Navigation** panel.

   The **Setup** panel displays the general simulation settings.

Feedback

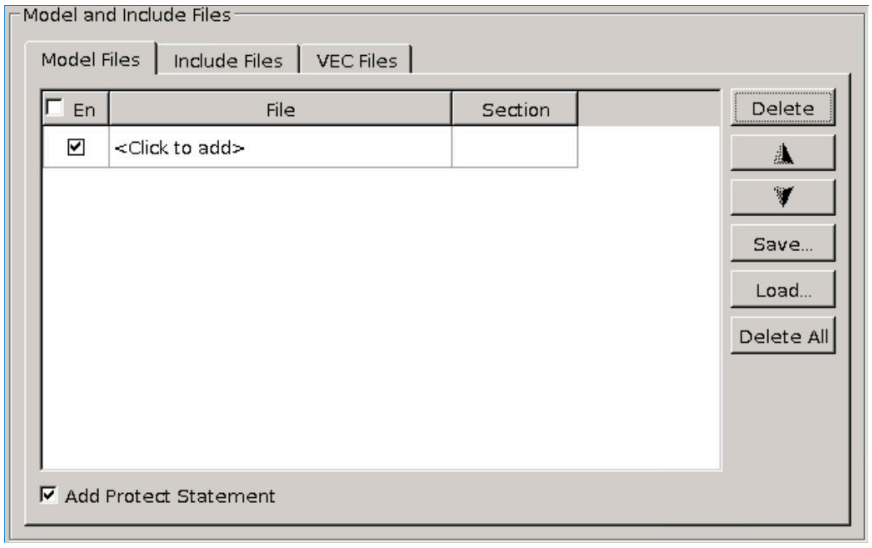
2. Enter the path to a simulator executable file in the **Executable** text box if different configuration settings exist for the simulator, or click the  button to browse to and select one.
3. Enter any extra command arguments for running a simulation in the **Options** text box.
4. Enter any comments you want to include in the netlist in the **Comments** text box.

Adding CS Model Files

To add CS model files:

1. Click **Model and Include Files** in the Navigation panel.

   The Model and Include files tab page is displayed.



2. Click the **Model Files** tab.

   The Model Files tab page is displayed.

3. Double-click the "Click to add" text, and either enter the path to a model file, or click the  button to browse to and select a model file.

   If the model file you select contains separate sections, such as .LIB sections in PrimeSim HSPICE, the sections are listed in a menu in the Section column for that row.

4. Ensure the check box in the **En** column is clicked for model files you want to enable.

   Uncheck the check boxes next to any model files you want to disable.

5. If the model file you choose contains sections, choose a section you want to use in the menu that appears in the **Section** column.

Feedback

If multiple sections exist that need to be included in a single file, you need to include the file multiple times.

6. (Optional) Click the **Add Protect Statement** check box to add the protect statement to your model file.

   This option is enabled by default.

7. Add any additional model files as necessary.

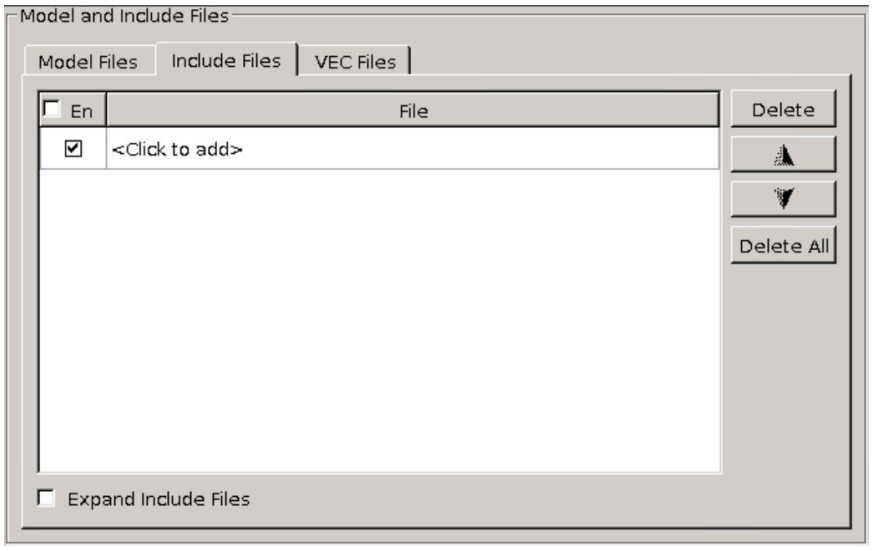
   Your model files are now set up.

Adding CS Include Files

To add CS include files:

1. Click **Model and Include Files** in the Navigation panel.

   The Model and Include files tab page is displayed.



2. Click the **Include Files** tab.

   The Model Files tab page is displayed.

3. Double-click the "Click to add" text, and either enter the path to an include file, or click the  button to browse to and select an include file.

4. Ensure the check box in the **En** column is clicked for include files you want to enable.

   Uncheck the check boxes next to any include files you want to disable.

5. (Optional) Click the **Expand Include Files** check box to expand the include files.

Feedback

6. Add any additional include files as necessary.

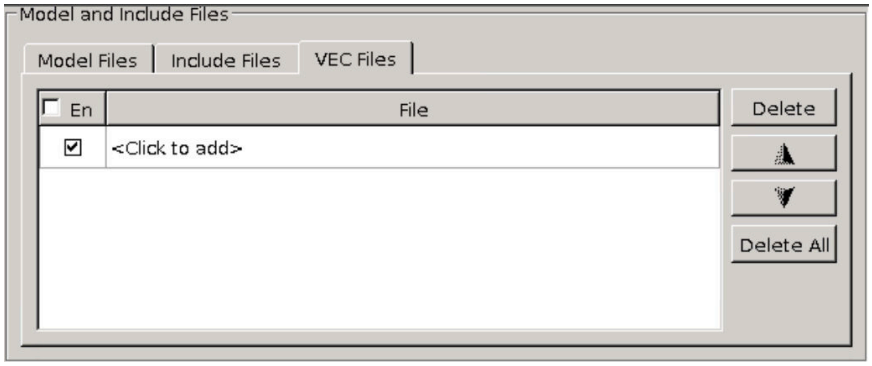
   Your include files are now set up.

Adding CS VEC Files

To add CS vector files:

1. Click **Model and Include Files** in the Navigation panel.

   The Model and Include files tab page is displayed.



2. Click the **VEC Files** tab.

   The VEC Files tab page is displayed.

3. Double-click the "Click to add" text, and either enter the path to a vector file, or click the  button to browse to and select a vector file.

4. Ensure the check box in the **En** column is clicked for vector files you want to enable.

   Uncheck the check boxes next to any vector files you want to disable.

5. Add any additional vector files as necessary.

   Your vector file setup is complete.

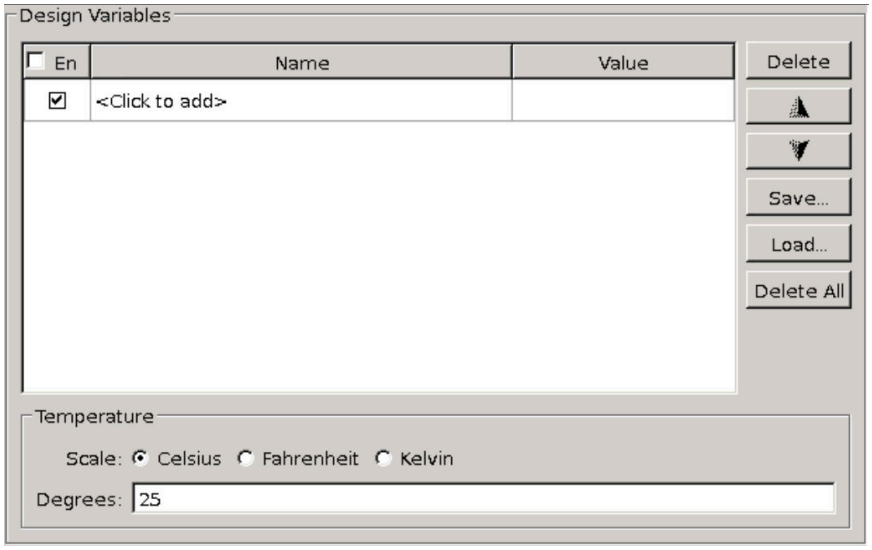
Adding CS Design Variables

To add CS design variables:

1. Click **Design Variables** in the Navigation panel.

   The Design Variables page is displayed.

Feedback



2. Double-click the "Click to add" text, and enter the name of a design variable.
3. Double-click in the **Value** cell next to the design variable name you create, and enter a value for the design variable.
4. Ensure the check box in the **En** column is clicked for model files you want to enable.

   Uncheck the check boxes next to any model files you want to disable.
5. Choose temperature **Scale**, and enter a temperature value in the **Degrees** text box.
6. Add any additional design variables as necessary.

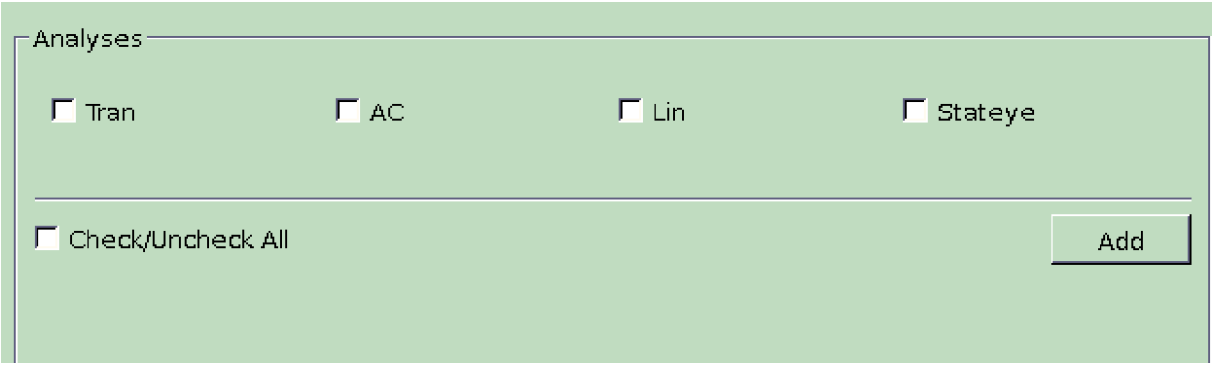
   Your design variables are now set up.

Adding CS Analyses

To add CS analyses:

1. Click **Analyses** in the Navigation panel.

   The Analyses page is displayed.



2. Click the check box next to each analysis you want to include in your simulation setup.

   The **Tran** and **Stateye** analyses are selected by default.

   Note:

   WaveView supports only TRAN section of the PSF-XL format for corners, MC, and sweeps. For details, see SolvNetPlus article 000040866, “PSFXL Format Support in WaveView”.

3. Click **Add** to add the selected analyses.

   The selected analyses are displayed in the Navigation panel.

4. Click the name of an analysis in the Navigation panel.

   The analysis setup details are displayed in the Setup panel. For information on analysis setup details, see the *HSPICE Reference Manual: Commands and Control Options*.

5. Once all analyses are set up, click the check box next to each analysis name in the Navigation panel to enable it.

   Your CS analyses are set up.

Adding CS Sweep Data

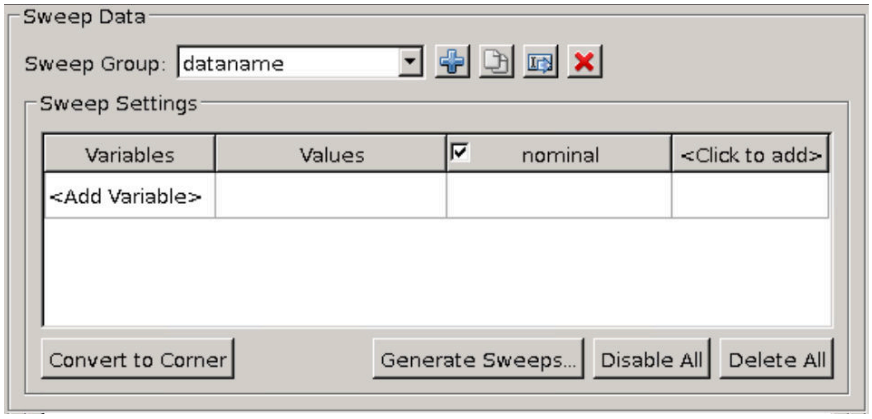
Note:

By default, all added sweeps are added to the default `dataname` Sweep Group. To create a new sweep group, see Creating New CS Sweep Groups.

To add CS sweep data:

1. Click **Sweep Data** in the Navigation panel.

   The Sweep Data page is displayed.



2. Double-click the "Add Variable" text, and choose a design variable you already have as part of your simulation setup from the menu that opens.

   The value for the selected design variable appears in the nominal column.

3. Double-click in the **Values** column next to the design variable name you select, and enter one or more values for additional sweeps separated by commands.

   New sweeps are automatically created in new columns.

4. Double-click the "Click to add" column heading to add additional sweeps.

   A new column is created with the Sweep <n> heading for each new sweep.

5. Double-click a table cell in the Sweep <n> column for that sweep, and enter a value.

6. Create any additional sweeps as needed, then ensure the check box in the column heading for each sweep name is selected for each sweep you want to enable.

   All created sweeps are enabled by default.

   Your sweeps are now set up.

This section contains information on the following topics:

- Creating New CS Sweep Groups
- Deleting CS Sweep Groups
- Creating Copies of CS Sweep Groups
- Renaming CS Sweep Groups
- Generating CS Sweeps
- Converting CS Sweeps to CS Corners

Feedback

Creating New CS Sweep Groups

To create a new sweep group:

1. Click the  button.

   The Add Sweep Group dialog box opens.

2. Enter a name for the new sweep group.
3. Click **OK**.

   A new sweep group is created with the name you specify.

Deleting CS Sweep Groups

To deleted a sweep group:

1. Select the name of the sweep group you want to rename from the **Sweep Group** menu.
2. Click the  button.

   A delete confirmation dialog box opens.

3. Click **OK** to delete the sweep group you choose.

   The sweep group is deleted and removed from the list of available sweep groups.

Creating Copies of CS Sweep Groups

To create a copy sweep group:

1. Select the name of the sweep group you want to copy from the **Sweep Group** menu.
2. Click the  button.

   The Copy Sweep Group dialog box opens.

3. Enter a new name for the sweep group, and click **OK**.

   A copy of the specified sweep group is created and displayed in the men of available sweep groups.

Renaming CS Sweep Groups

To rename a sweep group:

1. Select the name of the sweep group you want to rename from the **Sweep Group** menu.
2. Click the  button.

Feedback

The Rename Sweep group dialog box opens.

3. Enter a new name for the sweep group, and click **OK**.

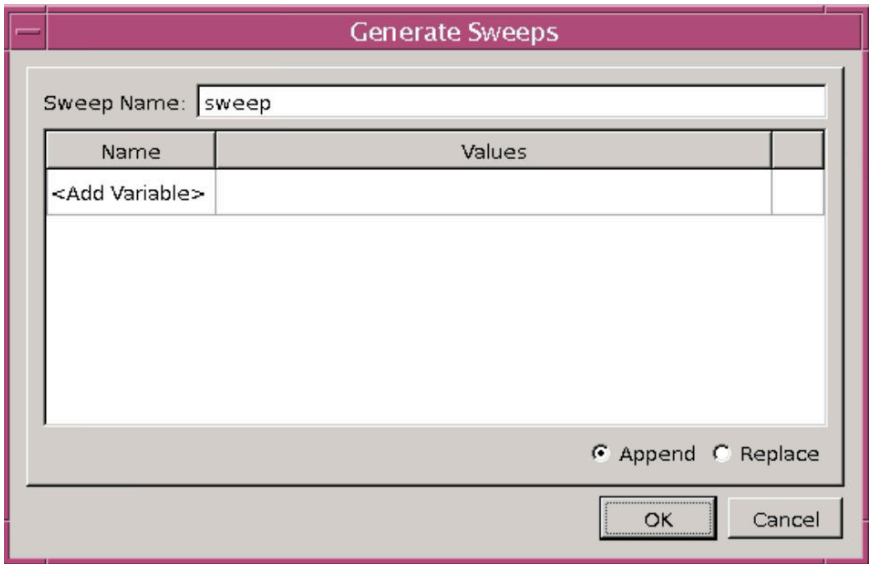
   The sweep group name is updated with the new name you specify.

Generating CS Sweeps

To generate a sweep:

1. Click the **Generate Sweeps** button on the Sweep Data setup page.

   The Generate Sweeps dialog box opens.



2. Double-click the "Add Variable" text, and choose a design variable you already have as part of your simulation setup from the menu that opens.
3. Click the [x,y] button to specify a range for the sweep.

   The Specify Range dialog box opens.

4. Enter the **Start**, **Stop**, **Step** , and **POI** values for the sweep.
5. Click **OK** to save the sweep values.

   The sweep values are displayed in the Value column of the Generate Sweeps dialog box.

6. Click either **Append** or **Replace** to append the generated sweep settings to the sweep you choose or to replace the sweep settings.

Feedback

7. Click **OK** to generate a sweep with the specified values.

   The generated sweep and sweep values are displayed in the Sweep Data setup page.

Converting CS Sweeps to CS Corners

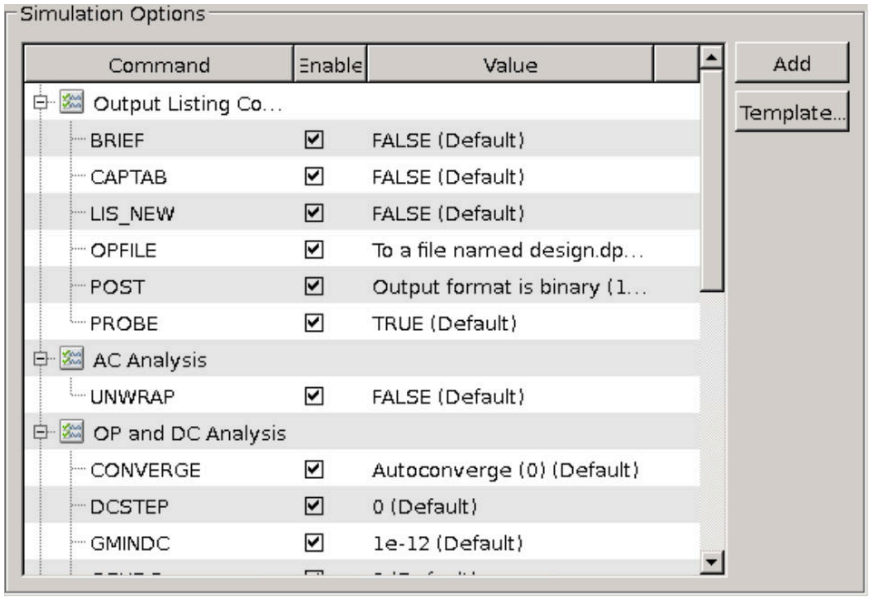
To convert a sweep to a corner, click the name of a sweep, and then click the **Convert to Corner** button. The selected sweep is converted to a corner and is displayed in the **Corners** setup in the Navigation panel.

Adding CS Simulation Options

To add CS simulation options:

1. Click **Simulation Options** in the Navigation panel.

   The Simulation Options page is displayed.



2. Click the check box next to each simulation option you want to include in your simulation setup.

   For information on simulation setup details, see the *HSPICE User Guide: Simulation and Analysis*.

3. If you do not see a simulation option that you want to use, click the **Add** button, enter the name of the simulation option, and press the **Enter** key.

   The simulation option you specify is added to the list in the Simulation Options Setup panel.

Feedback

4. (Optional) If you want to set up a simulation options template, click the **Template** button.

   The Simulation Options Template window opens. Complete it to set a template.

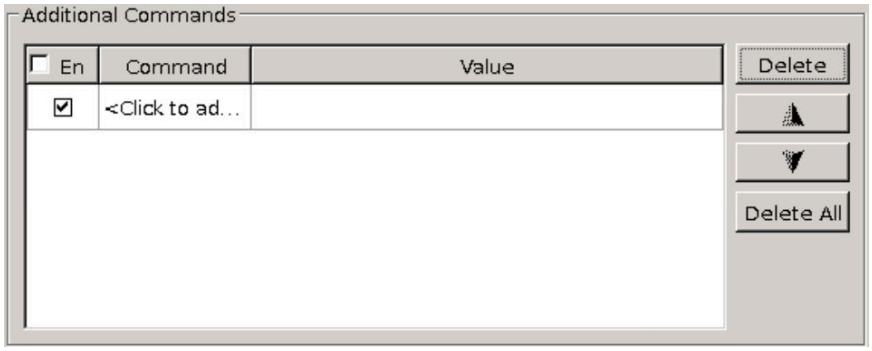
   Your simulation options are now set up.

Adding Additional CS Commands

To additional CS commands:

1. Click **Additional Commands** in the Navigation panel.

   The Additional Commands page is displayed.



2. Double-click the "Click to add" text, and enter the a command name.
3. Double-click in the **Value** column of the command you add, and enter a value for that command.
4. Ensure the check box in the **En** column is clicked for command you want to enable.

   Uncheck the check boxes next to any commands you want to disable.

5. Add any additional commands as necessary.

   Your additional command setup is complete.

Adding CS Probes and Equations

This section contains information on the following topics:

- Adding CS Probes
- Adding CS Equations
- Plotting Result Lists

Feedback

Adding CS Probes

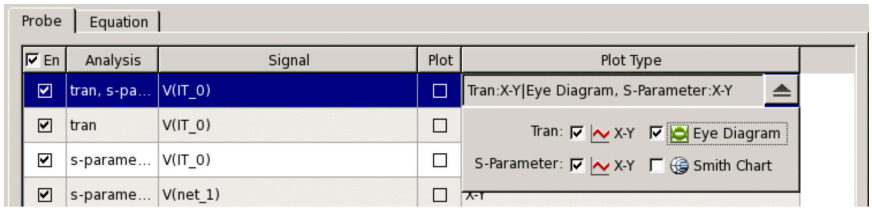
To add a probe:

1. Click the **Probe and Equations** category in the Navigation panel.

   The Probe and Equations page opens with either the **Probe** tab page or **Equation** tab page visible.

2. Click the **Probe** tab.

   The **Probe** tab page opens.



3. Click the `Click to add` text in the probes table.

   A text box appears in the table, and the **Type** and **Variable** text boxes appear as well.

4. Choose a probe type from the **Type** menu.

5. Enter a variable name in the **Variable** text box (or **Variable 1** if you select **Differential Voltage** as the probe type), or click the  button to select an object from the CS Connection View.

6. If you select **Differential Voltage**, enter a second variable name in the **Variable 2** text box, or click the  button to select an object from the CS Connection View.

7. Enter a signal name to probe, and click **Enter**.

   If the signal can be probed, the signal name is added to the table

8. (Optional) Click the check box in the **Plot** column to plot results for a probe.

   The signal is plotted on completion of a successful simulation run.

9. (Optional) Use **Plot Type** to set one or multiple plot types with different analysis types.

   When one probe has multiple analysis types, the plot type setting field shows available analysis types as check boxes. Different plot types are displayed under different waveform tab titles when plotted.

10. Click **Apply** to save your changes.

Feedback

Saving and Loading Probe Measure Files

To save a probe measure file, click the **Save** button in the lower right-hand corner of the **Probe** tab (accessible from the **Probe & Equation** tab in the Channel Simulation and Analysis window). The probe measure file is saved.

To load a probe measure file, click the **Load** button in the lower right-hand corner of the **Probe** tab (accessible from the **Probe & Equation** tab in the Channel Simulation and Analysis window). The probe measure file is loaded.

Removing Probes

To remove a probe:

1. Ensure the **Probe** tab page is open (accessible from the **Probe & Equation** tab in the Channel Simulation and Analysis window).
2. Click the  button at the end of the table row for the probe you want to remove.

   Note:

   If you accidentally delete one or more probes, click **Reset** in the lower right-hand corner to restore all probes setup to the previously saved version.
3. Click **Apply** to save your changes.

Enabling and Disabling Probes

To enable a probe in your setup (accessible from the **Probe** tab page on the Probe & Equation tab), click the check box next to a probe name, and click **Apply**.

To disable a probe, uncheck the check box to the left of a probe name, and click **Apply**. Any disabled probes are not included as part of your design when it is simulated.

Loading Probes

To load a probe measure file, click **Load** on the **Probe** tab page (accessible from the **Probe & Equation** tab).

Saving Probes

To save a probe measure file, click **Save** on the on the **Probe** tab page (accessible from the **Probe & Equation** tab).

Adding CS Equations

To add an equation:

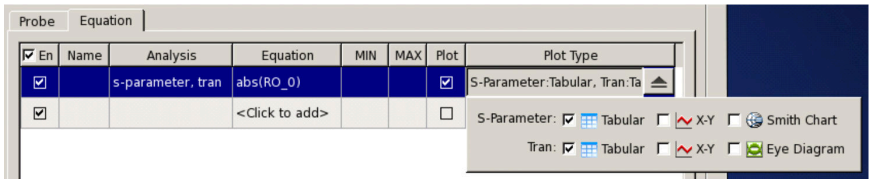
1. Click the **Probe and Equations** category in the Navigation panel.

   The Probe and Equations page opens with either the **Probe** tab page or **Equation** tab page visible.

Feedback

2. Click the **Equation** tab.

   The **Equation** tab page opens.



3. Click `Click to add` in the Equation column, and enter a signal (or expression) directly into the text box that appears, or create an equation from the calculator that opens.

   The function is added to your equation. Moving your pointer over any of the functions gives you descriptions of the function arguments.

4. Enter a name in the **Name** column for the equation you create.

5. Choose an analysis in the **Analysis** column from any analyses you set up.

6. (Optional) Enter minimum and maximum values in the Min and Max columns for the equation you are creating.

7. (Optional) Click the check box in the **Plot** column to plot the results of your equation after the simulation is complete.

8. (Optional) Use **Plot Type** to set one or multiple plot types with different analysis types.

   When one probe has multiple analysis types, the plot type setting field shows available analysis types as check boxes. Different plot types are displayed under different waveform tab titles when plotted

9. Click **Apply** to add the equation.

For more information on the calculator and calculator functions, see Working With the Equation Builder.

Working With the Equation Builder

The Equation Builder provides a collection of functions used to analyze simulation data. To access the Equation Builder, click the Probe and Equation tab in the Channel Simulation and Analysis window, then click the Equation tab.

This section contains information on the following topics:

Note:

When you create an equation, any probes that are required to satisfy the equation are created automatically.

Using the Equation Builder Functions

See the "Using the Equation Builder" chapter in the *WaveView User Guide* for more information on the Equation Builder functions.

Removing Equations

To remove an equation:

1. Ensure the **Equation** tab page is open (accessible from the **Probe & Equation** tab in the Channel Simulation and Analysis window).
2. Click the X button at the end of the table row for the equation you want to remove.

   Note:

   If you accidentally delete one or more equations, click **Reset** in the lower right-hand corner to restore all equations setup to the previously saved version.
3. Click **Apply** to save your changes.

Enabling and Disabling Equations

To enable an equation in your setup (accessible from the **Equation** tab page on the **Probe & Equation** tab), click the check box to the left of an equation, and click **Apply**.

To disable an equation, uncheck the check box next to an equation, and click **Apply**. Any disabled equations are not included as part of your design when it is simulated.

Loading Equations

To load a probe measure file (which contains equations), click **Load** on the **Equation** tab page (accessible from the **Probe & Equation** tab).

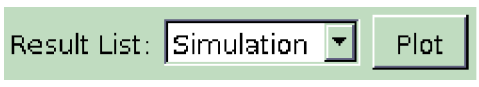
Saving Equations

To save a probe measure file (which contains equations), click **Save** on the on the **Equation** tab page (accessible from the **Probe & Equation** tab).

Plotting Result Lists

To plot a list of results from a simulation run:

1. Click on the **Probe** or **Equation** tab (accessible from the **Probe and Equations** page).
2. Choose a list of results from the Result List menu in the lower right-hand corner of the **Probe and Equations** page.



3. Click **Plot** to plot the chosen Result List.

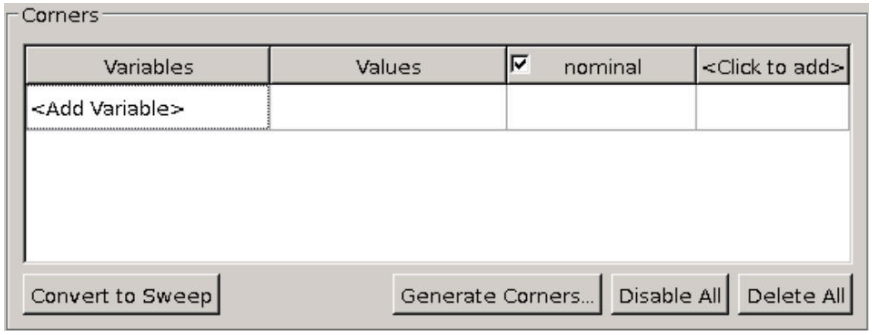
   The WaveView tool opens with your results plotted.

Adding CS Corners

To add corners:

1. Click **Corners** in the Navigation panel.

   The **Corners** page is displayed.



2. Double-click the `Add Variable` text, and choose a design variable you already have as part of your simulation setup or choose **Model File** to browse to and select a model file.

   The value for the selected design variable appears in the nominal column.

3. Double-click in the **Values** column next to the design variable name you select, and enter one or more values for additional corners separated by commands.

   New corners are automatically created in new columns.

4. Double-click the `Click to add` column heading to add additional corners.

   A new column is created for each corner with Corner <n> heading.

5. Double-click a table cell in the Corner <n> column for that corner, and enter a value.

6. Create any additional corners as needed, then ensure the check box in the column heading for each corner name is selected for each corner you want to enable.

   All created corners are enabled by default.

   Your corners are now set up.

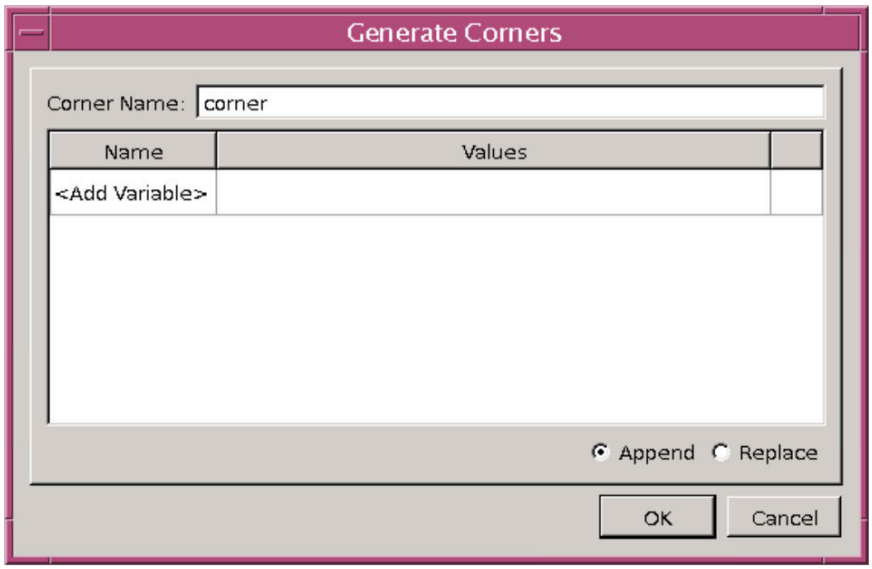
Feedback

Generating CS Corners

To generate a corner:

1. Click the **Generate Corners** button on the Corners setup page.

   The Generate Corners dialog box opens.



2. Double-click the "Add Variable" text, and choose a design variable you already have as part of your simulation setup, or choose **Model File** to browse to and select a model file.

   The value for the selected design variable appears in the nominal column.

3. Click the  button to specify a range for the corner.

   The Specify Range dialog box opens.

4. Enter the **Start**, **Stop**, **Step** , and **POI** values for the corner.

5. Click **OK** to save the corner values.

   The corner values are displayed in the Value column of the Generate Corners dialog box.

6. Click either **Append** or **Replace** to append the generated corner settings to the corner you choose or to replace the corner settings.

7. Click **OK** to generate a corner with the specified values.

   The generated corner and corner values are displayed in the Corner setup page.

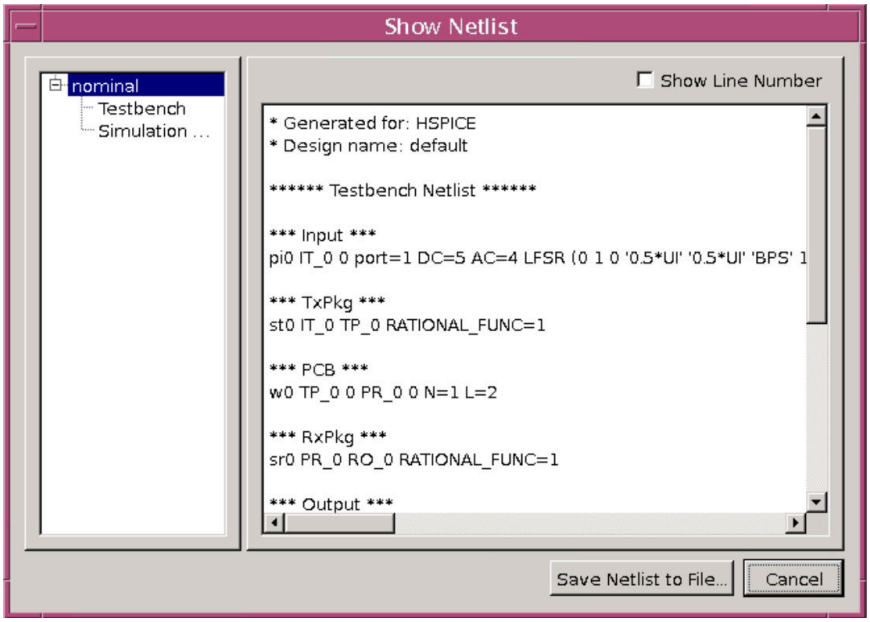
Feedback

Converting CS Corners to CS Sweeps

To convert a corner to a sweep, click the name of a corner, and then click the **Convert to Sweep** button. The selected corner is converted to a sweep and is displayed in the **Sweep Data** setup in the Navigation panel.

Show Netlist

To display your CS netlist, click **Show Netlist** in the **Simulation** section of the **CSA** tab.

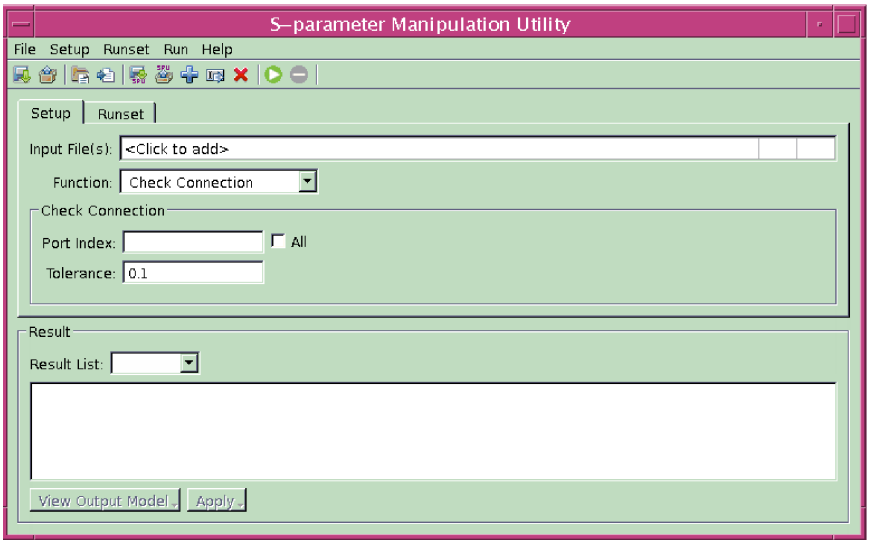


The Show Netlist window opens with a netlist of all the settings for your CS testbench and simulation. If you want to save the netlist, click **Save Netlist to File**.

SPUTIL

The S-parameter Manipulation Utility (SPUTIL) is a standalone executable, which gives you access to the PrimeSim HSPICE S-element features without invoking a PrimeSim HSPICE simulation or additional utility functions. You can also use the tool to predict and verify the behavior of given S-parameters in advance of PrimeSim HSPICE simulation runs.

To open this utility, click **SPUTIL** in the **Utilities** section of the **Waveform** tab.



The **S-Parameter Manipulation Utility** window opens. The **Runset** pane is where you set S-parameter commands, and the **Result** pane is for checking output results after running the SPUTIL.

This section contains information on the following topics:

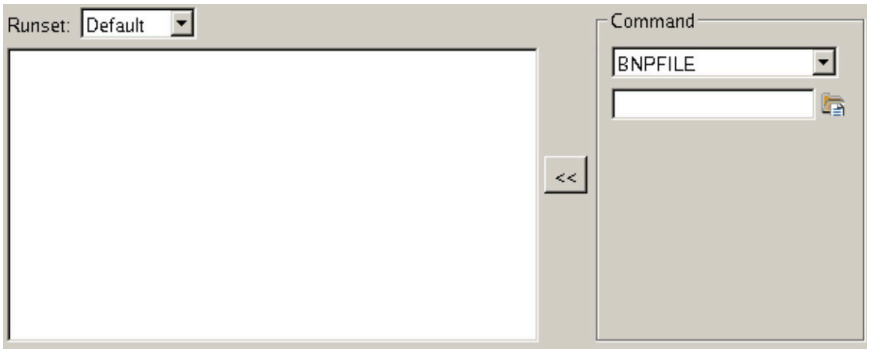
- Adding and Editing S-Parameter Commands
- Working With Runsets
- Setting S-Parameter Simulation Options
- Running and Stopping S-Parameter Simulations
- Saving and Loading SPUTIL Files
- Working With Results

Adding and Editing S-Parameter Commands

Note:

If you want to rename the **Default** runset after you add commands, see Renaming Runsets for more information.

To add an S-parameter command, choose a command from the **Command** menu, (which is at the right-hand side of the **Runset** pane, then click the double arrow button  .



The commands you add to your script in the **Runset** text box is known as a "runset." You can edit your script as necessary in the **Runset** text box.

See the *HSPICE* ® *User Guide: Signal Integrity Modeling and Analysis* for more information on S-parameter commands and command options.

Working With Runsets

A runset contains all the S-parameter commands and setup for an S-parameter simulation.

This section contains information on the following topics:

- Creating and Deleting Runsets
- Renaming Runsets
- Predefining Runsets in the WaveView Initialization File

Creating and Deleting Runsets

To create a new runset, ensure the **S-Parameter Manipulation Tool** window is open (click **SPUTIL** in the **Utilities** section of the **Waveform** tab), then click the **Add** button in the button toolbar. New runsets are added to the **Runset** menu.

To delete a runset, choose a runset from the **Runset** menu. Then, choose **Runset > Delete** from the **S-Parameter Manipulation Utility** window menu bar or click the **Delete** button in the button toolbar. Deleted runsets are removed from the **Runset** menu.

Renaming Runsets

To rename a runset:

1. Ensure the **S-Parameter Manipulation Tool** window is open (click **SPUTIL** in the **Utilities** section of the **Waveform** tab).
2. Choose a runset from the **Runset** menu that you want to rename.
3. Choose **Runset > Rename** from the **S-Parameter Manipulation Utility** window menu bar or click the **Rename** button in the button toolbar.

Feedback

The **Rename Runset** dialog box opens.

4. Enter a new name, and click **OK** to save your changes.

   The runset name is updated in the **Runset** menu.

Predefining Runsets in the WaveView Initialization File

You can predefine which runsets you want to automatically load each time you open the SPUTIL.

To define a runset in your `.cxinit` file, open the `.cxinit` file, and add a definition for a runset using the following syntax:

```
si_load_spumacro {"runset_name" "SPU_file_path"}
```

For example, to define a runset named `ct2`, which is in the `/remote/sputil/ct2.spu` location, add the following line to your `.cxinit` file:

```
si_load_spumacro {"c2t" "/remote/sputil/c2t.spu"}
```

Add runset multiple definitions for additional runsets you want to predefine.

Setting S-Parameter Simulation Options

To set S-parameter simulation options, choose **Run > Options** from the **S-Parameter Manipulation Utility** window menu bar.

The **SPutil Run Options** dialog box opens with the following options:

- **Executable** text box

  This option defines the full path of the `sputil` executable, which is located from the environment settings.

- **Run Directory** text box

  The option defines where the input file (`*.spu`) and output log file (`*.lis`) is generated. The default directory is `$WorkingDir/sputil`.

Running and Stopping S-Parameter Simulations

To run an S-parameter simulation, choose **Run > Run** from the **S-Parameter Manipulation Utility** window menu bar, or click the **Run** button  in the button toolbar. If the simulation is successful, the results are displayed in the **Result** pane.

To stop an S-parameter simulation, choose **Run > Stop** from the **S-Parameter Manipulation Utility** window menu bar, or click the **Stop** button  in the button toolbar.

Feedback

Saving and Loading SPUTIL Files

To save an S-Parameter Manipulation Utility file, which contains all the setup information for an S-parameter simulation, choose **File > Save** from the **S-Parameter Manipulation Utility** window menu bar. SPUTIL files are saved with the `*.spu` file extension.

To load an S-Parameter Manipulation Utility file, choose **File > Load** from the **S-Parameter Manipulation Utility** window menu bar.

Working With Results

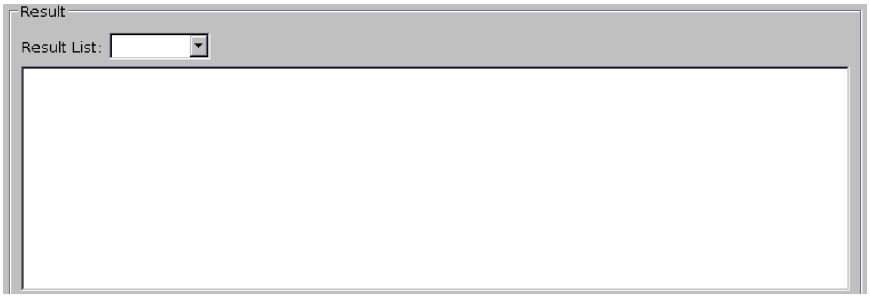
Once you complete a successful S-parameter simulation, you can view and apply the results to your simulation.

This section contains information on the following topics:

- Viewing S-Parameter Simulation Results
- Viewing the Output Model File
- Applying Output Model Files to Stages

Viewing S-Parameter Simulation Results

The results from a successful S-parameter simulation are displayed in the **Result** pane text box of the **S-Parameter Manipulation Utility** window.



The **Result List** menu contains a list of runset simulations that are complete. Choose a runset to display its results. You can also view its output model file or apply its model file to a simulation stage.

Viewing the Output Model File

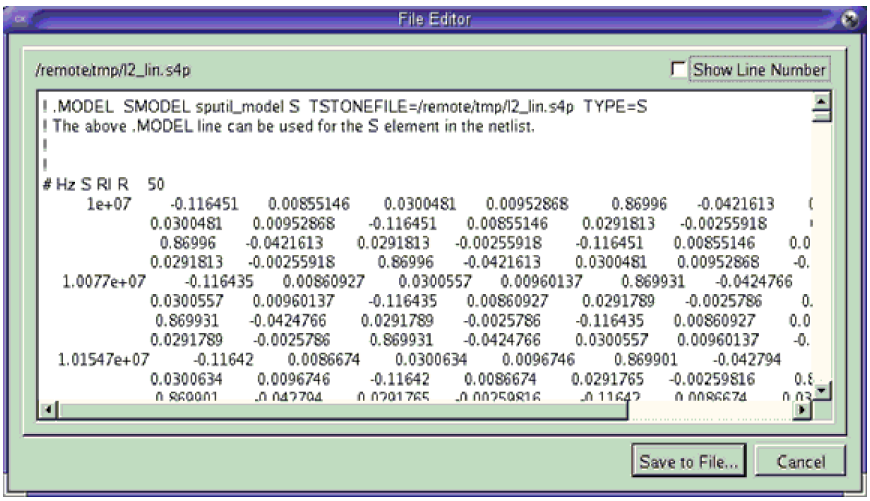
Once a successful simulation is complete, you can view the output model file in the WaveView tool or a file editor.



Feedback

To view the output model file from a Touchstone file in the WaveView tool, click **WaveView**, and then click **Open**. The WaveView tool opens with your output model file loaded.

To view the output model file in a file editor, click **File Editor**, and then click **Open**. The **File Editor** window opens with your output model file loaded. You can edit this file as necessary and save its contents to a file by clicking the **Save to File** button.



Applying Output Model Files to Stages

To apply an output model file to an S-parameter stage after a successful simulation, choose a stage from the **Apply Output Model to Stage** menu, and click **Apply**.

The testbench stage is displayed with the output model file as part of the stage, and the stage **Type** is changed to **S Parameter**.

Run

Once you have your CS testbench, simulation, and probes/equations setup, you can run a simulation. To run a simulation, click the **Run** button in the **Simulation** section of the **CSA** tab.

The simulation is run with the Channel Simulation Job settings that are currently set. See Job Control for more information on changing these job control settings.

Job Control

To change the channel simulation job control options:

1. Click **Job Control** from the **Simulation** section of the **CSA** tab, or click the **Job Control** button  in the button icon menu bar.

   The **Simulation Job Control** window opens.

2. (Optional) Enter a name for your simulation run in the **Run Name** text field.

   The `Simulation` name is used by default.

3. (Optional) Click **Browse** to choose a Run Directory.

   You can also enter a run directory name in the **Run Directory** text field. The `cx_simulation` directory is created for you by default.

4. In the **Test List** table, enable or disable any corners you want to include in or exclude from your simulation.

5. (Optional) Click **Run Options** to specify where to run the simulations and how many simulations to run in parallel.

   See Setting Run Options for more information on setting run options.

6. Click **Start** to run the simulations.

   As the simulation runs, the status is displayed in the Log tab page. Any errors that occur are also displayed here.

   Click the **Stop All** button if you need to abort a simulation while it is running.

Setting Run Options

The job control Run Options specify where to run your simulations (local or remote server) and how many parallel simulations to run. By default, simulations are run on a local server, and two jobs are run in parallel.

To set the simulation run options:

1. Click the **Run Options** button, which is located in the **Simulation Job Control** window (click **Job Control** from the **Simulation** section on the **CSA** tab).

   The **Run Options** window opens.

2. Click the **Job Distribution** tab.

   The Job Distribution tab page is displayed.

3. If you want to run simulations locally, click **Local Server**.

Feedback

Otherwise, skip to Step 5.

4. Enter a the number of simulation jobs you want to run in parallel.

   By default, two simulation jobs are run in parallel. Skip to step Step 11.

5. Click **Remote Server** to run your simulation jobs on a remote server.
6. Choose the type of network on which the jobs are run: **GRID**, **LSF**, or **Other**.
7. Enter any options you want include when running jobs.
8. Enter the number of jobs to run in parallel on the remote server.

   Three simulation jobs are run in parallel by default.

9. Click the **Result Handling** tab.

   The **Result Handling** tab page is displayed.

10. Enable or disable the results handling options, as well as how you want signals plotted after a simulation is complete.

    Note:

    If you enable the **Result History** option, a date and time stamp is appended to any additional results that come before the current results. For example:

    ```
    Simulation.1-Dec04-1157.tr0 (1st run result)
    Simulation.2-Dec04-1200.tr0 (2nd run result)
    Simulation.tr0 (current result)
    ```

11. Click **OK** to save your changes.

Using ADE Post-Process Integration

The ADE postprocess flow provides a Results Viewer for plotting and analyzing results for the ADE integration flow. The flow uses a runObject file as basic result file, when possible. You can group signals, separating voltage and currents. Results are displayed in different views based on the analysis type.

This section contains information on the following topics:

- Using the Results Viewer
- Basic Analysis Results
- Advanced Analysis Results

Feedback

- Device Aging Results
- Advanced Plots

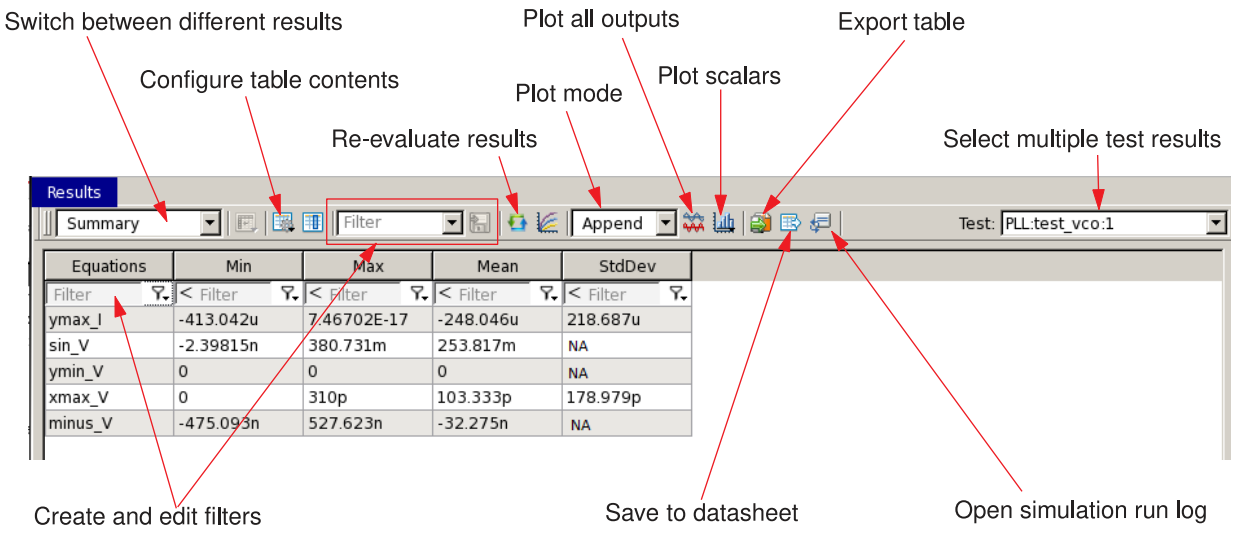
Using the Results Viewer

The Results Viewer can only be invoked with the ADE interface, and opens automatically after simulation. It appears as a Results tab in the canvas. The following sections describe using the Results Viewer:

- Viewing the Results Viewer GUI
- Configuring and Filtering Results and Table Content
- Exporting Results

Viewing the Results Viewer GUI

The Results Viewer normally appears as follows. Some tables may differ.



Configuring and Filtering Results and Table Content

Use the Results Viewer toolbar to configure results and content as follows:

- To switch between different results, click the left drop-down and select to view either Summary, Detail, or OP Report results.
- To configure table contents, click the  button to view a list of selections. Different tables have different selections. Unavailable items will be grey.

Feedback

- To show/hide table columns, click the  button to view a list of columns. Filtering is supported.
- To view multiple test results, click the right drop-down to switch between tests. Click **X** to delete a test.
- To filter results:
  - Use the filter text field above each column to filter the row results. There are text filters and value filters. You can have multiple filters for each column.
  - To save these filters, enter a name in the **Filter** text field in the toolbar and click the **Save** button next to it.

Exporting Results

You can select the content to export.

To export results as a Web page (.CSV or html), click the  button.

To save results as a datasheet, click the  button.

Basic Analysis Results

The Result Viewer shows the following results information for all results:

- Summary Table
- Detailed Table

Summary Table

This table contains the following summary data for each equation:

- Specification
- Violations
- Values of Min, Max, Mean, and StdDev

Detailed Table

This table shows more detailed information, such as:

- Design variable value
- Corner trace
- Monte index

Feedback

- Wave expression
- Probe signal
- Scalar value per run
- Specification of expression
- Pass/fail status of each value

For example:

Equations	Min	Max	nom	C0	C1
Filter	< Filter	< Filter	< Filter	< Filter	< Filter
			tune = 0.1	tune = 0.1	tune = 0.1
ymax_I	-413.042u	7.46702E-17	-413.042u	-331.095u	7.46702E-17
sin_V	-2.39815n	380.731m			
ymin_V	0	0			
xmax_V	0	310p	0	310p	0
minus_V	-475.093n	527.623n			

Advanced Analysis Results

The following advanced analysis results contain additional data and features:

- Corner and Sweep Analysis
- Monte Carlo Analysis

Corner and Sweep Analysis

For corner and sweep analysis, the following features are available:

- Parametric Reduction
- Rotate Variable Table

Parametric Reduction

Rotate Variable Table

After parametric reduction, the Result Viewer and signal plotting are updated by corner selection. Unchecked corners are hidden.

You can rotate the detailed table according to Corner Name, Monte, Equation Name, or None.

Feedback

For example:

Equations	Specification	Corner1	Corner2	Corner3	Corner4
Filter	< Filter	< Filter	< Filter	< Filter	< Filter
		res1=1.1k	res1=600	res1=900	res1=1.1k
		res2=100	res2=100	res2=100	res2=200
osc	<8	-723.664m	7.50787	3.15293	3.11934
meas1		3.29994	3.30001	3.29999	14.635u

Monte Carlo Analysis

For Monte Carlo analysis, the following features are available:

- Statistical Results
- Variation Contribution Results
- Corner Summary Results
- Advanced Plots

Statistical Results

The statistical results appear as follows:

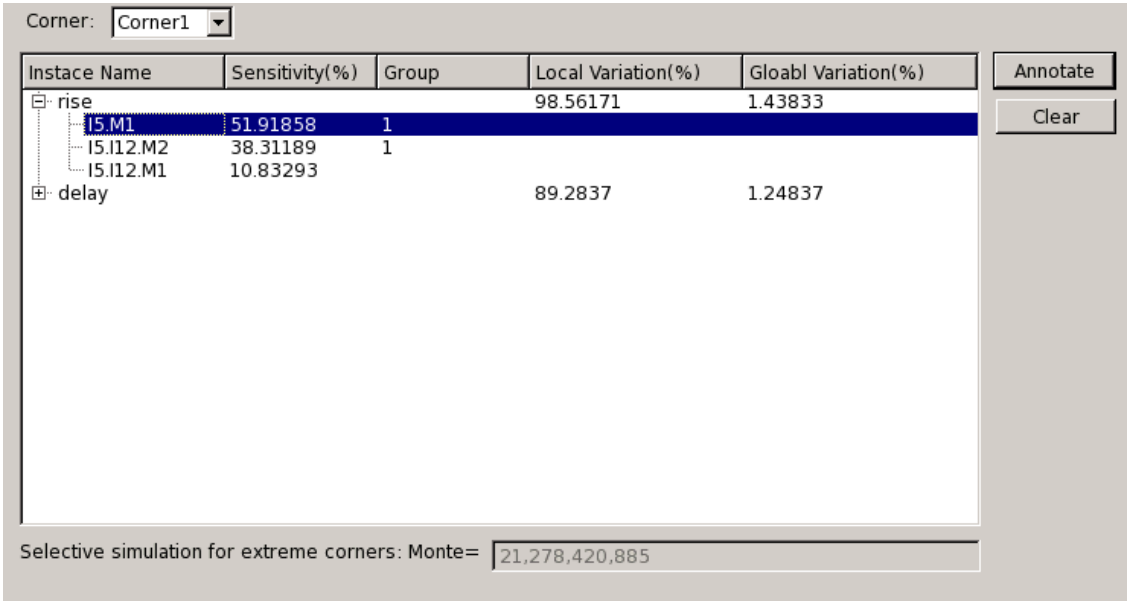
Equations	Mean	StdDev	Mad	Median	Skewness	Kurtosis
r1	1.005e+01	9.864e-01	9.687e-01	1.009e+01	-9.554e-02	3.029e+00
r2	1.005e+01	9.864e-01	9.687e-01	1.009e+01	-9.554e-02	3.029e+00
v1	4.014e+01	2.959e+00	2.906e+00	4.026e+01	-9.554e-02	3.029e+00

Variation Contribution Results

These results allow annotating results to schematic, and filtering measurements from the context menu.

Feedback

For example:



Corner Summary Results

For Monte Carlo with corner analysis, there are two additional result tables for corner summary.

Advanced Plots

For information on pair plot and QQ plot results, see Advanced Plots.

Device Aging Results

For device aging, the following features are available:

- Aging Time
- Device Degradation Results

Feedback

Aging Time

The detailed table contains Aging Time data for each run, as follows:

Aging Time (years)	avg	meas1
< Multiple Selected	< Filter	< Filter
0	1.32132	3.3
2	1.32132	3.29997
4	1.32132	3.29996
5	1.32132	3.3

You can rotate the table for device aging results:

Equation	Aging Time=0 yr	Aging Time =2 yr	Aging Time=4 yr	Aging Time = 5 yr
< Filter	< Filter	< Filter	< Filter	< Filter
avg	1.32132	3.3	1.32132	3.3
meas1	1.32132	3.29997	1.32132	3.29997

Device Degradation Results

The device degradation is displayed for aging analysis results:

- MOSRA results
- TMI results (right-click and select **Configuration** to show/hide HCI, BTI, HCI + BTI categories)

Advanced Plots

Advanced plots can be used for multiple runs and trace results, and can be invoked from the toolbar. Right-click an item in the Equation column to quick plot. The following advanced plots are available:

- Histogram
- Scatter
- Pair Plot
- QQ Plot

Histogram

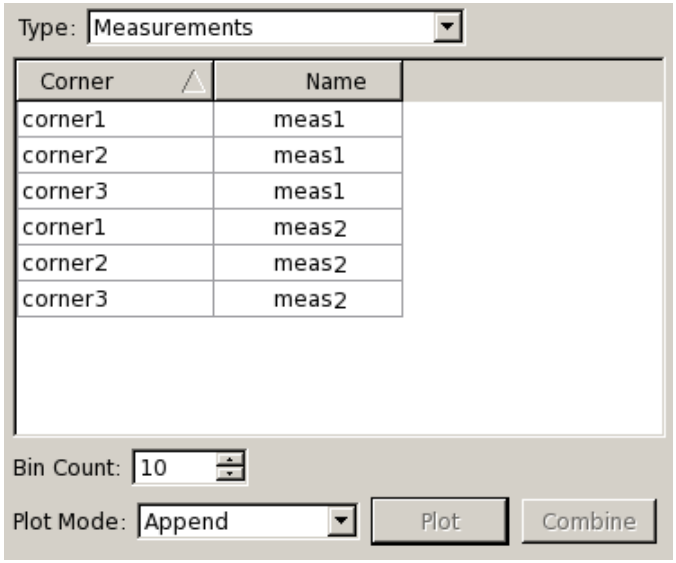
The histogram plot has the following features:

- Two types: Measurements and Parameters
- Three plot modes: Append, New Waveview, Replace

Feedback

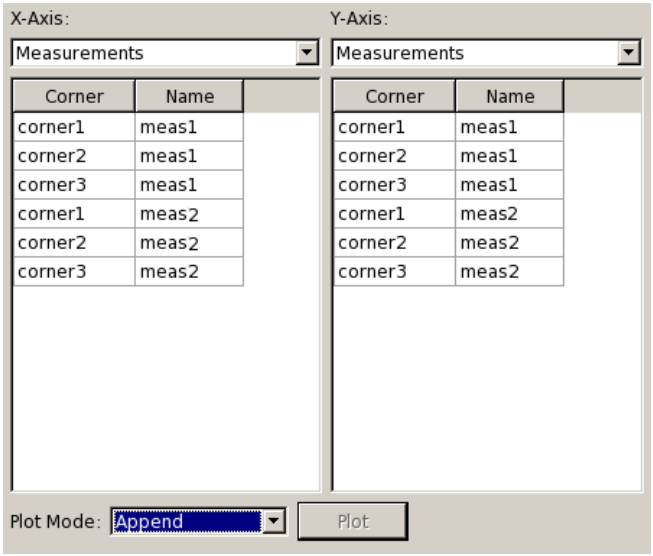
- Use the **Combine** button to combine different results

  For example:



Scatter

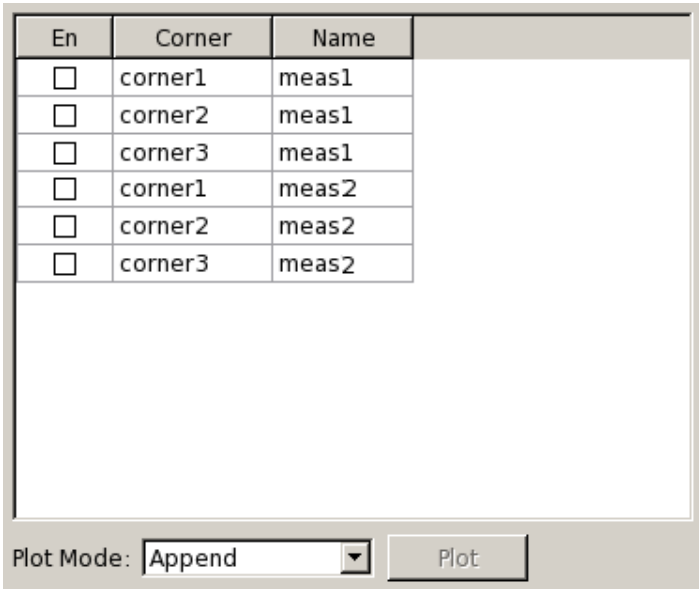
The scatter plot options are as follows:



Feedback

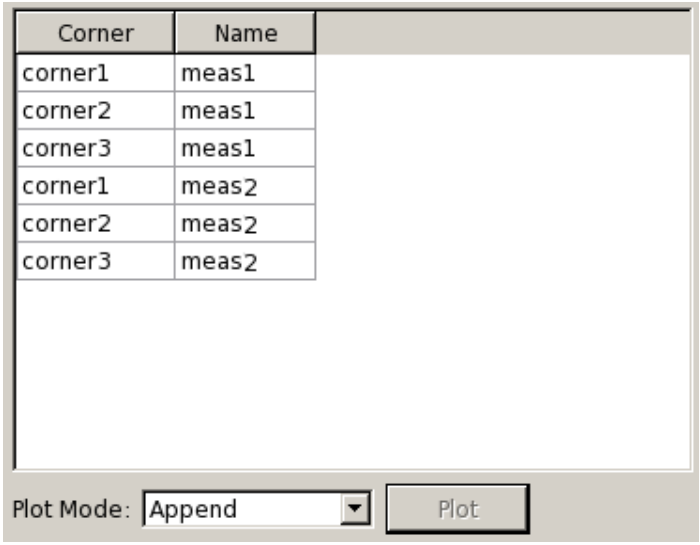
Pair Plot

The pair plot is available only for Monte Carlo analysis. Options are as follows:



QQ Plot

The QQ plot is available only for Monte Carlo analysis. Options are as follows:



10

Customizing the WaveView Tool

This chapter details how to customize options in the WaveView tool.

This chapter contains information on the following topics:

- Setting Key Bindings (Shortcut Keys)
- Setting Preferences
- Mapping Bus Values

Setting Key Bindings (Shortcut Keys)

To open the shortcut key settings, click **Settings** on the **Waveform** tab, and choose **Key Bindings** from the menu that opens. The BindKey Setting Finder window opens.

This section describes the following topics:

- Customizing Shortcut Keys
- Top Menu-Operated Shortcut Key Functions
- WaveView Toolbar Shortcut Key Functions
- Other Shortcut Key Functions

Customizing Shortcut Keys

Shortcut key settings can be customized for key action in the WaveView windows.

The following shortcut keys are allowed:

- A to Z
- 0 - 9
- Ctrl+A to Ctrl+Z
- Ctrl+0 to Ctrl+9

- Shift
- Esc
- Delete
- Backspace
- Up Arrow
- Down Arrow

Except for the five function keys (Eec, Delete, Backspace, Up, and Down), a key-down action and a key-up action can be associated with each key.

The available actions can be categorized into the following types:

- menuFunctionName: Equivalent to the top menu actions.
- wctrlFunctionName: Equivalent to the waveview control button actions.
- wcmdFunctionName: Special waveview command actions.

The Bind-Key dialog box automatically flags warnings if the selected action is not allowed for a key event. Select your auto-repeat and action preferences and click **Apply** to apply the change or click **Save** to apply and save the change.

Reading the Key Binding File

The key binding file must be named `.spxkey`. The WaveView tool can read this file from these different locations, with the following precedence:

- User-specified file name
- `-spxkey` option usage
- `current_working_directory/.spxkey`
- `$(SX_HOME)/.spxkey` (if `SX_HOME` is defined)
- `$(HOME)/.spxkey` (if `HOME` is defined)

The WaveView tool can also read the `.sae.spxkey` file with the following precedence:

- `$(SX_HOME)/sae.spxkey` (if `SX_HOME` is defined)
- `$(HOME)/sae.spxkey` (if `HOME` is defined)

`SX_HOME` is a user-defined environment variable; `HOME` is a shell variable which defaults to a user's home directory.

Settings read first are overridden by those read later in the sequence.

Top Menu-Operated Shortcut Key Functions

These functions can be also invoked from the top menu. The current shortcut key setting is indicated at the right end in the associated menu box.

`menuLoadWaveformFiles`

Opens the **Open: Waveform Files** dialog box.

`menuUpdateAll`

Updates all loaded waveform files.

`menuPrintWV`

Opens the **Print Setup** dialog box.

`menuExportWV`

Opens the **Waveform Export Parameters** dialog box.

`menuAbortLoading`

Aborts the current waveform file loading process.

`menuShowDirTable`

Opens the **Directory/File Table** dialog box.

`menuQuitApp`

Terminates the current WaveView process.

`menuLoadSession`

Opens the **Load Session** dialog box.

`menuSaveSession`

Opens the **Save Session** dialog box.

`menuNewWaveView`

Adds a new waveview window.

`menuUndoWaveView`

Undoes the previous waveview operation.

`menuDeleteWaveView`

Deletes the active waveview window.

`menuDeleteAllWaveView`

Deletes all waveview windows.

Feedback

`menuRenameWaveView`

Renames the active waveview window.

`menuRefreshWaveView`

Refreshes the content of the active waveview.

`menuToggleWVDocking`

Toggles waveview between dock/undock modes.

`menuToggleBrowser`

Shows/hides the signal browser.

`menuAddCrossCursor`

Adds a cross-cursor. The default shortcut key is **CTRL+g**.

`menuAddPanelLinear menuAddPanelSmith menuAddPanelPolar menuAddPanelEyeDiagram menuAddPanel2Dsweep menuAddPanel3Dsweep menuAddPanelHistogram`

Adds an empty panel in various types.

`menuDeletePanels`

Deletes all selected panels.

`menuUngroupPanels menuGroupPanels`

Ungroups/groups all selected panels.

`menuSelectAllPanels menuUnselectAllPanels`

Selects/unselects all panels in the active waveview.

`menuFitAllPanels`

Fits all selected panels to waveview height.

`menuDefHAllPanels`

Resets all selected panels to default height.

`menuSetRadixBinary menuSetRadixOctal menuSetRadixHex menuSetRadixDecimal menuSetRadixASCII`

Sets radix of all selected panels to various mode.

`menuShowShortVector menuShowFullVector`

Sets logic values of all selected panels to the shortest mode or the full bus width mode.

Feedback

`MenuShowAlias menuShowOrgName`

Sets signal names in all selected panels to using alias or original signal name.

`menuSetStyleVCD menuSetStyleRTF`

Sets logic value style to the standard VCD logic or the Synopsys/Nassda HANEX RTF-style logic.

`menuUndoZoom menuRedoZoom`

Undoes/redoes the previous zoom/unzoom operation.

`menuSetLinearXAxis menuSetLogXAxis menuSetDBXAxis (db20)`
`menuSetLinearYAxis menuSetLogYAxis menuSetDBYAxis (db20)`

Sets the x/y axis type of selected panels in the active waveview.

`menuSetZoomRange`

Opens the **Zoom Settings** dialog box for manual zoom.

`menuResetZoom`

Fully unzooms selected panels in the active waveview.

`menuConfigSelectedYAxis`

Opens the **Setting Y Axis Full Range** dialog box.

`menuConfigYAxisDefault`

Opens the **Default Y Axis Range** dialog box.

`menuConfigXAxisVariable`

Opens the **Setting X Variable** dialog box.

`menuJumpCursorForward menuJumpCursorBackward`

Jumps the main cursor forward/backward.

`menuCallSignalFindermenuCallFFT menuCallD2A menuCallA2D`
`menuCallDataReduction menuCallCalculator` (calculator) `menuCallDynamicMeter` (measurement tool) `menuCallPWLEditor` (PWL editor)

Opens various analysis tools.

`menuShowSelectedSignals menuHideSelectedSignals`

Shows/hides all highlighted signals in selected panels of the active waveview.

WaveView Toolbar Shortcut Key Functions

These functions can be also invoked from the mini toolbar in each WaveView window.

`wbtnScanNext wbtnScanPrevious`

Scans signals in selected panels of the active waveview to the next/previous signal in the same waveform file.

`wbtnToggleDataPoints`

Toggles on/off the data point markers.

`wbtnToggleGrids`

Toggles on/off the X/Y grid lines.

`wbtnAddCursor`

Adds a new cursor in selected panels.

`wbtnAddCursorL`

Moves a left waveform cursor to the closest point on the waveform to the mouse cursor.

`wbtnAddCursorR`

Moves a right waveform cursor to the closest point on the waveform to the mouse cursor.

`wbtnDelete`

Deletes the selected panels.

`wbtnSwitchMainCursor`

Switches the main cursor.

`wbtnAddMonitor`

Adds a new monitor.

`wbtnAddTextLabel`

Adds a text label.

Other Shortcut Key Functions

These functions can be invoked using shortcut keys.

`wcmdAddDiffDMeter`

Adds a Difference measurement in the active waveform.

`wcmdAddToEquation`

Adds the selected signal to the Calculator.

Feedback

`wcmdAddXYDMeter`

Adds a Data(X,Y) measurement in the active waveform.

`wcmdCopyLastDMeter`

Makes a copy of the last dynamic measurement added in a waveview.

`wcmdRepeatScript`

Repeats the last ACE script executed.

`wcmdTogglePanelFit`

Toggles all selected panels in the active waveview between fit/default panel height.

`wcmdToggleSyncMode`

Toggles the active WaveView between sync/unsync modes.

`wcmdArmBoxZoom wcmdUnarmBoxZoom`

Key-activated box-zoom operations. The Arm and the Unarm functions must be assigned to the key-down and the key-up actions respectively of the same bindkey.

`wcmdArmVertZoom wcmdUnarmVertZoom`

Key-activated vertical zoom operations. The Arm and the Unarm functions must be assigned to the key-down and the key-up actions, respectively, of the same shortcut key.

`wcmdArmHorzZoom wcmdUnarmHorzZoom`

Key-activated horizontal zoom operations. The Arm and the Unarm functions must be assigned to the key-down and the key-up actions, respectively, of the same shortcut key.

`wcmdArmSignalMode wcmdUnarmSignalMode`

Key-activated signal mode operations. The Arm and the Unarm functions must be assigned to the key-down and the key-up actions, respectively, of the same shortcut key.

`wcmdCopyPanels wcmdCutPanels wcmdPastePanels`

Copies, cuts, or pastes panels (or signals) in the active waveview.

`wcmdScrollWVUp wcmdScrollWVDown`

Scrolls the panel stack in the active waveview (stack-mode waveview only) up and down.

Feedback

`wcmdXPanRight wcmdXPanLeft`

Pans the X-range in the active waveview (stack-mode waveview only) right and left.

`wcmdYPanUp wcmdYPanDown`

Pans the Y-range of all selected panels in the active waveview up and down.

`wcmdSetHorzZoom() wcmdSetVertZoom()`

Sets the zoom direction without zooming out the design.

`wcmdReloadSession`

Reloads the last session file used.

`wcmdQuitZoom`

Aborts a zoom operation during mouse drag.

`wcmdSortSignals`

Sorts signals order (by their names) in all selected panels.

`wcmdDumpScreen`

Dumps the content of the active waveview. Opens the **Save Screen Dump** dialog box.

`wcmdJumpCursorBackwardwcmdJumpCursorForward wcmdResetZoom wcmdUndoZoom wcmdRedoZoom`

These functions are duplicates of the same 'menu' shortcut key functions. They are supported only for backward compatibility.

`wcmdToggleLayoutMode`

Switches between Stack and Row/Column layout modes.

`wcmdSigAdd wcmdSigDiff wcmdSigMultiply wcmdSigDivide`

Performs Add (+), Differential (-), Multiply (* - represented as Shift+8 under KEY in the BindKey Settings Finder), and Divide (/) operations on selected single or multiple signals.

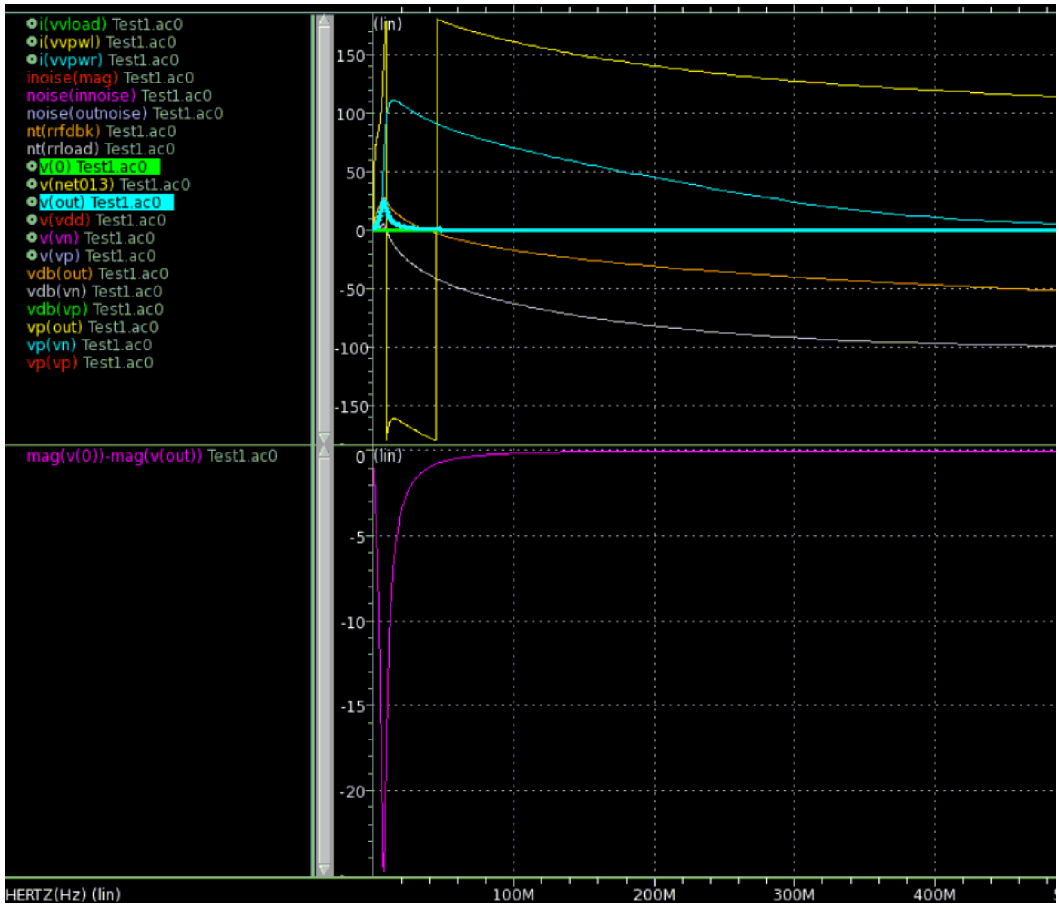
Behavior of operations is as follows:

- When a single signal is selected:
  - **Add**, **Multiply** — show the original signal.
  - **Differential** — show the data of `-signal`.
  - **Divide** — show the data of `/signal`.

Feedback

- When multiple signals are selected:
  - **Add** — add the signals (order of signals does not matter).
  - **Multiply** — multiply the signals (order of signals does not matter).
  - **Differential** — perform the operation on selected signals in the order top-to-down as they appear in the Signal List.
  - **Divide** — perform the operation on selected signals in the order top-to-down as they appear in the Signal List.

  For example, selecting v(0) and v(out) and selecting **Differential** (or the shortcut key **-**) calculates the differential operation "v(0) - v(out)".



Feedback

Setting Preferences

This section contains information on the following topics:

- General Settings
- Readers Settings
- Waveview Settings
- Panel Settings
- Signals Settings
- Axis Settings
- Color Settings
- Threshold Settings
- CDS Link Settings
- Saving Preference Files
- Loading Preference Files

General Settings

The following general settings are available:

Option Name in GUI	Option Name	Value (Default)	Description
GUI Font	gui_font	large/(small)	Sets preferred GUI font size.
GUI Colors	gui_color	(bright)/dark	Select default GUI color tone.
Quick Bar Position		left/(right)	Select the position of the Quick Bar.
Canvas Tab Position	waveview_TabPos	(top)/bottom	Controls the position of waveview tabs.
Canvas Background	waveview_background	(black)/white	Select canvas background color.
Display Modeless Dialog	wv_iconizable	raise by default/(always on top)	Controls whether or not modeless windows are displayed on top.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Calculator Error/Warning Message	show_eqn_err_msg	0, (1), 2	0: Not show 1: Show in message window 2: Show in terminal
Use Native Qt File Dialog	use_qtfsb	true/(false)	Control the file loading dialog box style. When enabled, the native Qt dialog box is used, which is the native Windows dialog on Windows platforms.
Hide Ribbon Menu	hide_ribbon	true/(false)	Hides the tool ribbon menu on startup.
Show Console Window	show_console	true/(false)	Displays the Console window.
Suppress Canvas/Panel Delete Confirmation	delete_confirm	(true)/false	Set to false to disable confirm dialog box for waveview/panel delete action.
Disable Application Quit Confirm	exit_confirm	true/(false)	Set to false to disable confirm dialog when quitting.
Archive data files with waveform data files with logfiles on crash	archive_data_on_crash	true/(false)	Archives the `sx_crash.sx`, `sx_crash.log`, and `sxcmd.log` waveform data files to a zip file called `data_on_crash.tar.gz`. The archive file is created in the working directory
Enable Dynamic Filtering	dynamic_filter	(true)/false	Enables or disables dynamic filtering in the Signal List.
Keep Searching String and Type When Switching Hierarchy	keep_signalsearchstring	(true)/false	Keeps the filter string and type when changing hierarchy.
Include Sub-level Signals in Signal List	include_sub_sig	true/(false)	Includes sub-level signals in the signal list. When the signals have a different hierarchy, it includes signals with full hierarchy path at the top level.
Suppress Function Completion for Building Equation	suppress_eqn_func_completion	true/(false)	Turns off auto-completion for functions in Equation Builder.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Disable Expanding Hierarchy Tree with Key-in '*'	disable_star_expand_tree	true/(false)	Turns off expanding the whole hierarchy of the sub-level signal when pressing the "shift+*" and selecting the top level.
Disable Auto-Copy to Clipboard when Selecting Item	disable_autocopy_clipboard	true/(false)	Turns off automatically copying selected items to the clipboard for pasting.
Disable Auto Strip Off Zeros	disable_autostrip0	true/(false)	Turns off automatically removing zeroes.
Number of Significant Digits	precision_digits	integer_value (3)	Sets the number of valid digits for all numeric values. ([3,10]).
Replay Command Delay (ms)	replay_delay	integer_value (1)	Controls how fast the log is replayed.
Default Printout Title	print_title	string	Specifies the title displayed at the top of printouts.
Printer Devices	printer_device	printer_name (null)	Sets the default printer device in the Print Setup dialog box.
Linux Print Command	print_command	print command	Sets the user-defined print command (Linux only).
Waveform File Filter	wdf_fsb_filter	"label:pattern"	Adds a user-defined filter pattern to the WDF loading dialog box. This option can be used multiple times to add multiple filter patterns.
Netlist File Filter	spice_fsb_filter	Spice filename extension:*.sp, *.sp0, *.spi, *.scs (NULL)	Sets filter of file type when importing design from PrimeSim HSPICE Netlist.
Disable Auto Close Hint Message		true/(false)	Enables the pop-up hint window listing which signals could not be found in the FSDB due to changed names.
Default Browser Width	browser_width	integer_value (250)	Sets window width for the browsers. (All five left browsers).
Use width last saved in .spxlast	sx_tv_width	true/(false)	Uses the browser width saved in your .spxlast file.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Default WaveView Geometry	wv_geom	width*height+X+Y	Sets the size of the WaveView GUI. The next time you open WaveView, it will be of size width*height at (x,y). **Note:** Ensure the **Use Geometry Last Saved in .spxlast** preference is not selected. For example: 800x600+5+10

Readers Settings

The following readers settings are available:

Option Name in GUI	Option Name	Value (Default)	Description
Always Preload Waveforms to RAM	load_data_at_open	true/(false)	Sets WDF data loading schedule. If set to true, all waveform data are loaded when a file is initially open.
Sort Hierarchy Names by Length in OutputView	smart_hsort	true/(false)	Sorts the hierarchy names by length in the OutputView.
Automatically Connect to Subsequent Split Files	auto_connect_split_files	true/(false)	Automatically connects subsequent split files.
Disable Waveform Reloading Confirmation	disable_reload_wdf_confirm	true/(false)	Disables the waveform reloading confirmation dialog box.
Merge Continuous Identical Logic Values	merge_digital_time	true/(false)	Merges continuous identical logic values.
Session File			
Suppress Popup Warning Messages when Loading Session File	suppress_load_session_msg	true/(false)	Suppress warnings for missing signal when loading a session file.
HSPICE Formats			

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Always Remove Duplicated Signals in tr0 Files	remove_dupsig	true/(false)	Removes duplicated signal names in the signal browser.
Case Sensitive HSPICE Waveform Files	case_sens_tro	(false)/true	Specifies case sensitivity for signal names in PrimeSim HSPICE tr0 files.
Convert S to Y/Z parameters in sc0 and TouchStone Files	convert_s_to_yz	(true)/false	Converts S parameters to Y/Z parameters in sc0 and TouchStone files.
Use S## naming convention when port number < 10 in sc0 and TouchStone files	use_snn	true/(false)	Uses the S## naming convention for files that have less than 10 port.
Use Signal Leaf Name for HSPICE and PSF Files	use_leaf_name	true/(false)	Uses the leaf name in the Signal List Browser.
FSDB Format			
Always Add toplevel for FSDB File	add_toplevel_for_fsdb	true/(false)	Adds toplevel to FSDB files. When disabled, the toplevel is added for FSDB files based on the signals it contains. If signal exists without a scope, a toplevel is added; otherwise, a toplevel is not added.
Add a point to extend waveform data to simulation max time for EPIC fsdb files	extend_waveform	(false)/true	Extends the EPIC-type waveforms in fsdb file to the maximum simulation time.
Filter Out Data Points Whose X Value Is Greater Than Simulation Max Time for FSDB File	filter_byxrange	true/(false)	When enabled, filters out the data points whose x value is out of range for FSDB files. When saving a .spxrc file, the preference uses the variable `filter_byxrange`, which can be set to false (default) or true.
Display Logic Real Type Signals in PWC by Default for FSDB File	logic_real_pwc_fsdb	(true)/false	Display logic type signals in PWC attribute by default.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Load Scope on Demand When Scope Counts Over	read_top_hier read_top_hier_threshold	(true)/false default is: 5 (million)	Controls FSDB file loading.
Display REAL/AMS WReal Type Signals in PWC by Default for FSDB File	amsreal_pwc_fsdb	true/(false)	When enabled, displays Real/AMS WReal Type signals in FSDB files as PWC.
Other Formats			
Flatten ai_pl File data segments when each segment has only one point	flatten_flags	(true)/false	Flattens the sweep signal in ai_pl files when each signal trace has only 1 point.
Turn off visibility of VHDL processes in VPD file in Signal Browser	not_show_vhdl_processes	(true)/false	Displays a confirmation dialog box when reloading. Otherwise, a confirmation dialog box is displayed first before reloading.
.meas File index X Values Start From 1 instead of 0		true/(false)	X index values start from 1 when reading measurement result files.

Waveview Settings

The following waveview settings are available:

Option Name in GUI	Option Name	Value (Default)	Description
Grid Brightness	grid_brightness	(normal)/low	Sets the background grid brightness for waveviews.
Cycle Color/Marker Per	cycle_color_by	panel/waveview/(panel waveview)/waveform_file	Selects color cycle mode. In the panel mode, the default line color restarts at color 0 for each panel. In the Waveform File mode, the waveform color is cycled when a waveform is from a different waveform file. Waveforms from the same waveform file are plotted in the same color.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Left Button Default	left_button_action	(zoom)/cursor	Sets default action for left mouse button (LMB). In zoom mode, box-zoom operation is the default action when LMB is pressed and dragged. If LMB is released without any drag action, the main cursor (if it exists) is moved to the mouse pointer location. In cursor mode, pressing LMB moves the main cursor to the pointer location. In both modes, if LMB is pressed near a cursor, the cursor is grabbed and becomes the main cursor.
XY-Panel Drop Mode	init_xy_mode	(hvsel)/vonly/honly	Selects the default orientation for XY-panels.
Inf Value Plotting	show_inf_data	(skip)/use value	Control the Inf value display and sets the value for Inf when plotting to WaveView.
Show Testbench Name in Waveview Tab Name	show_tbname_in_wvtab	true/(false)	Shows the testbench name in the waveview tab name. This is useful, for example, when two similarly named analyses are plotted from PrimeWave.
Cursor Operations			
Default Cursor Type	cursor_type	vertical/horizontal/cross	Set the cursor type for the cursor created by the **Add Cursor** toolbar button and shortcut keys [and].
Move Cursor to Mouse Click Position	click_move_cursor	(true)/false	Moves cursor to x-axis where canvas is clicked.
Enable Cursor Snap to Logic Value Change	logic_cursor_snap	(true)/false	Enables cursor snap function in logic panels if set to true.
Display Value Labels at Crossing Points by Default	show_cross_label	true/(false)	Shows crossing labels along cursor.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Hide X Values on the Value Labels at V-Cursor Crossing Points by Default	hide_cross_label_x_value	true/(false)	Hides the X values on the value labels at V-cursor crossing points by default if set to true.
Hide All Distances between Cursors	hide_cursor_distance	true/(false)	Hides distance lines and value boxes between reference and other cursors.
Show Reference Cursor Crossing Value	ref_cursor_value_indv	individually/Along with Target Cursor	Individually: P1 value is shown along reference cursor, P2 value and delta value are shown along target cursor. Along with Target Cursor: P1 value, P2 value and delta value are shown along with target cursor.
Canvas Operations			
Realtime Waveform Window Panning	real_time_scroll	(true)/false	Enables real-time scrolling of a zoomed panel if set to true.
Limit Side-Bar Grip to Bar Region	bar_grip_scroll	true/(false)	Limits slide-bar grip to the bar region if set to true.
Limit Phase of Complex Value Between (-180, 180)	clamp_ac_phase	(true)/false	Controls phase range of complex values. Phase is limited between -180/180 if set to true.
Extend Analog Signal Waveform to Fill X Range	auto_x_extend	true/(false)	Extends analog waveforms to fill the x-axis range.
Extend Logic Signal Waveform to Fill X Range	auto_x_extend_dig	(true)/false	Extends logic waveforms to fill the x-axis range.
Show Waveform X-Y Value Near Mouse Pointer	snap_marker	(true)/false	Enables waveform values mouse tracing cursor, which will snap to waveform data points and show the value box (will not be shown when not near a data point).
Display Logic Glitch (time-overlapping changes)	logic_glitch_highlight	(true)/false	Enables glitch highlight in logic panels if set to true.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Switch Mouse Left and Right Button Actions in Canvas	switch_mouse_button	true/(false)	When enabled, the left and right mouse button actions will be switched, meaning the left mouse button will bring up the context menu and the right mouse button is for zoom and selection.
Enable Mouse Right Button Drag to Zoom in	right_mouse_zoom	true/(false)	When enabled, you can use right click + mouse drag to zoom-in in the waveform canvas.
Enable VNC Display Acceleration Mode	fast_vnc_mode	true/(false)	Shows performance enhancement in the VNC display.
Enable Line Drag-n-Drop on Canvas		true/(false)	Enables drag and drop of a waveform or a group of waveforms to a new window or sub-window.
Unit Scale Factor For Monitor	unit_scale	(auto)/f/p/n/u/m/none/k/M/G/T	Sets the scale factor for Monitor values.
Disable Waveform Marching Reload Confirmation	reload_confirm	true/(false)	Disables the waveform marching reloading confirmation dialog box.
Do Not Update Loaded Signals When Plotting New Signal	disable_update_signal	(true)/false	When enabled, the WaveView tool does not update loaded signals automatically; to update loaded signals, you must click the **Update** button.
Disable Equation Update when Reloading Signals		true/(false)	When selected, equations are not updated automatically when signals are updated; to reevaluate an equation, you must click the **Update** button and then click **Evaluate**.
Synchronize Signal List with Equation Builder	sync_eqnbuilder	true/(false)	If an equation is deleted in the Calculator, WaveView deletes the equation result on the canvas and in the browser tree.

Feedback

Panel Settings

The following panel settings are available:

Option Name in GUI	Option Name	Value (Default)	Description
Set Grid Default to Off	grid_default	true/(false)	Sets default grid on/off mode.
Automatically Switch to Row/Column Mode when Plotting Signals with Different X Ranges	auto_switch_horz	true/(false)	Switches modes when plotting two different signals with different X ranges (units) in the same waveview, to help in telling them apart.
Limit Zoom Out Range to Axis Full Scale Range	limit_zoom_range	true/(false)	Limits zooming out to, but not beyond, the full scale range.
Zoom Direction Default	def_zoom_dir	(xy_zoom)/x_zoom/y_zoom	Specifies the default zoom mode for panel.
Double-Click to plot signal at	plotsig_bottompanel	(new_panel)/bottom_panel	Specifies where a signal is plotted when double-clicked in the signal list.
Stack-Mode Panel Height			
Automatically Fit Analog Panels to Waveview Height	stack_auto_fit	(true)/false	Enables panel height auto-adjustment for non-logic panels in vertical waveviews.
Hide Dividers Between Stacked Panels	hide_panel_div	true/(false)	Hides the dividing lines between panels in stack mode.
Max. # of Visible Panels	max_visible_panels	integer_value (0)	Sets the maximum number of visible XY panels of vertical stack waveviews.
Minimum Analog Panel Height	min_panel_height	integer_value (20)	Sets minimum height for non-logic panels (in screen pixels) in vertical waveviews. [20,1000].
Analog Panel Height	analog_panel_height	integer_value (120)	Sets analog panel height for non-auto-height-fit mode.
Logic Panel Height	logic_panel_height	integer_value (24)	Sets logic panel height.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
General Panel Settings			
Row/Column Mode Default Panels Per Row	default_panels_perrow	integer_value (4)	Specifies the default number of panels per row in row/column mode.
Row/Column Mode Minimum Panel Height	horz_panel_height	integer_value (80)	Specifies the minimum panel height in row/column mode.
Default Histogram Bin Size	default_hist_binsize	integer_value (10)	Sets the default bin size of histograms.
Show Single Histogram of All Signals by Default	default_hist_one	true/(false)	Drags multiple singles and shows histogram as one signal by default.
Signals Per Panel (d-n-d)	lines_per_panel	integer_value (0)	Sets number of lines per panel for signal allocation after the initial signal drag-and-drop operations. The default setting 0 causes all signals to be placed in one new panel.
Maximum Log Scale Span in Decades	max_log_span	(12)	Defines maximum log scale span in decades.
Zoom In Ratio (%)	zoom_in_ratio	(50)	Sets ratio for zoom.
Smith Chart Characteristic Impedance	smith_char_impedance	real_value(50.0)	Sets the default characteristic impedance of Smith charts.
Use Fixed Y Full Range	user_fixed_yrange	true/(false)	Uses the user-defined values for vmin (default 0), vmax (default 3.3), imin (default 0), and imax (default 1m).
Displaying Margin for Y Axis of Full Scale Panels	auto_y_margin panel_fs_margin_xy	on/(off) integer_value(0)	Allows WaveView to determine the good Y axis display margin. Sets the display margin for y-axis when **Fixed Size** is selected.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Displaying Margin for X Axis of Full Scale Panels	auto_x_margin panel_fs_margin_x	on/(off) integer_value(0)	Allows WaveView to determine the good X axis display margin. Sets the display margin for x-axis when the **Fixed Size** is selected.
Smith Chart Value Type	smithchart_valuetype	(mag/phase) / real/imaginary	Specifies the type of values that are displayed in the Smith chart.
Polar Value Type	polar_valuetype	(mag/phase) / real/imaginary	Specifies the type of values that are displayed in the Polar plot.

Signals Settings

The following signals settings are available:

Option Name in GUI	Option Name	Value (Default)	Description
Synchronize Signal Alias Between Browser and Display	sync_alias	(true)/false	Synchronizes the signal name alias between the signal browser and the name column in the canvas.
Add Brackets [ ] for Signal Alias Names	alias_bracket	(true)/false	Adds bracket to user-defined signal alias name.
Alias Signal Name Only	alias_signal_name_only	true/(false)	Sets the alias to replace leaf node names only.
Always Name Sweep Family Signals as Individual Traces	sweep_expand	true/(false)	Treats the sweep signal lines as an individual or a family of lines.
Plot Single Point Sweep Family Signals on 2D Panel by Default	plot_single_point_sweep_on_2d	true/(false)	View abstract line when enabled, and detailed line when disabled. If the y value is too small to show, disable to see the detail of the signal.
Flatten Single Point Sweep Signals on 2D Panel	flatten_single_point_sweep	(true)/false	Flattens sweep signals that have only 1 point in each trace when plotting on a 2D panel.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Display Signal Name in Tooltip Popup	signame_popup	true/(false)	Enables signal name popup tooltip.
Display Sweep Trace Parameters in Tooltip Popup		true/(false)	Enables sweep trace parameters popup tooltip.
Display Complex Plotting Type as Modifier in Signal Name	show_MP_prefix	true/(false)	Sets the plotting type as the signal name modifier when plotting complex signals.
Use Simplified PWL Algorithm in rmsval Calculation	simplified_rms_calc	true/(false)	Uses simplified RMS calculation algorithm.
Use little-endian format to display bus value	use_little_endian	true/(false)	Specifies little endian or big endian for bus value display.
Show Toggle Button in Toolbar for Signal Selection Mode and Panel Selection Mode	show_SM_button	true/(false)	Displays the **Signal/Panel Mode Selection** button in the WaveView toolbar
Set Panel Mode as Default	show_panel_mode	true/(false)	Sets Panel mode as the default mode.
Highlighting Signal when Mouse Hovers on its Name	enable_wv_highlight	(true)/false	Controls the automatic highlighting of signals when hovering the mouse over it.
Support Multiple Selection for Signals and Panels with Single Click	enable_mul_click_items	true/(false)	Supports selecting multiple signals with a single click.
Signal Search Options	signal_filter_option	(Glob pattern)/Regular expression	Specifies the string pattern used to search against waveform names is a glob-style or a regular expression pattern in the Signal Finder.
Support Substring Search	support_substr_search	true/(false)	Matches the substrings of waveform names with the specified searching pattern.
Signal Name Width	signame_width	widest/(fixed size)	Specifies the width of signal names.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Signal Name Alignment	signame_align	(left)/right	Sets signal name alignment.
Signal Name Content	signal_name	netname/leafname/fullname/(fullname+file)/(fullname+path+file)/(netname+file)/(netname+path+file) file	Sets signal name mode in signal name column.
Signal Hierarchy/Name Order	signal_path	(hierachy first)/name first	Sets signal hierarchy path style.
Order Signals from Grouped Files	group_display_order	(per file)/per signal	Sets the signals (from grouped files) order during the drag-and-drop display operations.
Signal Name Letter Spacing	signame_spacing	(normal)/large	Controls the letter spacing in signal names.
Signal Mode Drag-n-Drop	signal_dnd_mode	(move)/copy	Moves or copies a signal when dragged and dropped.
Default Logic Value Type	logic_style_default	(vcd)/rtf	Specifies the text style for logic signal values.
Default AC Signal Display	def_ac_stack	(complex)/magphase/realimag/mag+phase	Sets the default display mode for AC signals.
Signal Name Font	signal_font	(normal)/large	Controls the font size of signal names.
Default AC Signal	def_acx_type	(linear)/log/db20	Sets the default X-axis type for AC signals.
Default AC Signal	def_acy_type	(linear)/log/db20/db10	Sets the default Y-axis type for AC signals.
Default TR Signal	def_trx_type	(linear)/log/db20	Sets the default X-axis type for real type signals.
Default TR Signal	det_try_type	(linear)/log/db20/db10	Sets the default Y-axis type for real type signals.
Default FFT Signal	def_fft_xscale	linear/(log)/db20	Sets the default X-axis type for FFT signals.
Default FFT Signal	def_fft_yscale	linear/log/(db20)/db10	Sets the default Y-axis type for FFT signals.

Feedback

Axis Settings

The following axis settings are available:

Option Name in GUI	Option Name	Value (Default)	Description
Automatic Dual Y-Axes (X-Y Panels)	auto_y_axes	on/(off)	Sets automatic dual Y-axes for mixed I/V signals.
Display Y-Axis Unit Label	show_ytype	(true)/false	Displays the y-axis unit labels on a WaveView window.
Display Prefix in Axis Unit Labels	show_unit_prefix	true/(false)	Displays the unit prefix axis labels on a WaveView window.
Place Label in the Middle and Parallel to Axes	center_axis_label	(true)/false	When enabled, places units and labels in the middle of and parallel to the x/y-axis, which can enhance clarity.
Show Abstract Line for Small Y Range (Difference at >9th Digit)	adjust_small_y_range	(true)/false	When Y Range is very small, the difference at or after the 9th significant digits, the tool shows a straight abstract line.
Use Default X Variable while Setting X Variable		(true)/false	The default X-variable as the x-axis of the plot. This is useful when multiple parameter sweeps are present and enables user to plot across original x-axis instead of the first sweep parameter.
Y-Axis Label Width	auto_y_width	automatic/(fixed)	Automatically sets the y-axis width based on the length of labels, or set to a fixed width.
X-Axis Pan Step (%)	horizontal_slider_step	(50)	Controls how fast the display pans left or right via the left or right keys or the WaveView slider arrows.
Font Size for Axis	axis_font	(Normal)/Large/Gothic/Times/Roman/Courier	Sets the default font size for axis labels.
X-Axis Auto Grid	xaxis_auto_grid	(true)/false	Justify and show the grid in x-axis direction.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Left Y-Axis Auto Grid	yaxis_auto_grid	(true)/false	Justify and show the grid in y-axis direction on the left side.
Right-Y-Axis Auto Grid	y2axis_auto_grid	(true)/false	Justify and show the grid in y-axis direction on the right side.

Color Settings

The following color settings are available:

Option Name in GUI	Option Name	Value (Default)	Description
Waveform Colors	user_colors	(Pre-Defined)User-Defined/Predefined and User-Defined	Selects pre-defined colors, user-defined colors, or a combination of both.
Lock Auto Selected Line Color	lock_color_sweep	true/(false)	Retains the color of a plotted signal when moving signals (for example, dragging and dropping a signal).
Display Sweep Signal Traces With Same Color	same_color_sweep	true/(false)	Plots all sweep signal traces with the same color.
Display Sweep Signal Traces With Looped Line Styles by Default	loop_linestyles_sweep	true/(false)	Plots sweep signals with looped line styles by default.
Display Signal Labels with White Color	white_label_color	true/(false)	When enabled, Waveview will always show the signal label with white color. If disabled, Waveview will show the signal label with same color as the trace color.
Default Waveform Line Thickness	line_thickness	1 to 7	Selects thick line as the default waveform line width.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
User-Defined Waveform Colors	user_rgb	R:G:B	A user-defined color. Select **Clear Colors** to remove old colors. Select **Add Color** or **Change Color** to bring up a color dialog box where you can select or set colors (including true colors).

Threshold Settings

The following threshold settings are available:

Option Name in GUI	Option Name	Value (Default)	Description
D2A Convert X State	d2a_xtol	to high/(to low)	Selects D/A conversion method for the X/Z logic states.
D2A X-Axis Range	d2a_fullext	line range/(waveview range)	Selects the converted signals' x-axis range for D/A conversion.
D2A Threshold Levels	d2a_high_value	real_value(3.300E+00)	Defines the analog level of the high logic state for D/A conversion.
D2A Threshold Levels	d2a_low_value	real_value(0)	Defines the analog level of the low logic state for D/A conversion.
D2A Rise/Fall Time	d2a_rise_slew	real_value(10ps)	Defines the rise slew for the D/A conversion process.
D2A Rise/Fall Time	d2a_fall_slew	real_value(10ps)	Defines the fall slew for the D/A conversion process.
A2D Threshold	a2d_single_threshold	high-low/(single)	Selects default A/D conversion method.
A2D Center Threshold	a2d_center_threshold	real_value(2.5)	Defines center threshold for the A/D conversion process.
A2D Threshold Levels	a2d_high_threshold	Real_value (4.0)	Defines high threshold for the A/D conversion process.
A2D Threshold Levels	a2d_low_threshold	real_value (1.0)	Defines low threshold for the A/D conversion process.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Meter level/margin scale changes according to panel scale	SX_DM_DMScale	true/(false)	Specifies if the level and margin values for dynamic measurements depend on the axis scale (for example, dB, log, linear).
Meter Results Use Panel Scale	SX_DM_ResultPanelScale	true/(false)	Uses the panel scale for measurement results.
Move label Independently on Difference meter	dm_diff_movelabel	true/(false)	Moves label Independently on Difference meter
Display Unit in Value Labels	show_value_unit	true/(false)	Displays unit in value labels
Count floor noise components in SNR/SNDR calculation	SNR_inc_floor_noise	true/(false)	Counts the floor noise components in SNR and SNDR calculations.
Scalar Result Format	calc_eng	true/(false)	Sets scalar result format in the Equation Builder. True sets to engineering notation. Default is scientific notation.

CDS Link Settings

The following CDS Link settings are available:

Option Name in GUI	Option Name	Value (Default)	Description
CDS-Link Plot Mode	cds_link_plot_mode	(replace)/append	Replaces or appends a plotted signal. This preference is only available when the WaveView tool is started in CDS-Link mode.
Use ADE Append/Replace Plotting Mode	use_ade_append_replace	replace/(append)	WaveView will plot signals according to the plotting mode in ADE and ignore the plot mode in WaveView.
CDS-Link Update Mode	cds_link_update_mode	replace/(append)	Replaces or appends a plotted signal when updating waveform files. This preference is only available when the WaveView tool is started in CDS-Link mode.

Feedback

Option Name in GUI	Option Name	Value (Default)	Description
Signal Crossing Probing Color Scheme	x-probing_color_schem	(Use Schematic Color Probe)/Use WaveView Signal Color	Specify to use Schematic Probe Color or to use WaveView Signal Color as doing cross probing between WaveView and another program.
Use Different Color for Bus Signal	bus_diff_color	true/(false)	WaveView will plot all bus signals in the same color by default. If the user selects this preference, WaveView will plot signal in different color.

Saving Preference Files

To save preferences into a file:

1. Ensure the **Preference Settings** window is open (click **Preferences** on the **Configuration** tab).
2. Click the **Apply-Save** button.

   The **Preference Save Location** dialog box opens.
3. Choose a location, either in your home directory, the working directory, or in a path you enter by typing or choosing from the file browser.

   Note:

   Select **Save as PrimeWave Design Environment Preference Name** to save the preference settings in PrimeWave preference format.

   For example, if you have **Number of Significant Digits** set to 13 in the **Preference Settings** window, this will be saved as precision_digits 5 by default, but if you select **Save as PrimeWave Design Environment Preference Name**, it will be saved as wfWVPrecisionDigits 5.
4. Click **OK** to save the file.

Feedback

Loading Preference Files

To load an existing preference file:

1. Ensure the **Preference Settings** window is open (click **Preferences** on the **Configuration** tab).
2. Click the **Load** button.

   The **Load Preference File** dialog box opens.
3. Browse to and select your preference file, and then click **OK**.

   Note:

   The WaveView tool can read WaveView-related preferences from the `synopsys_custom.spxrc` file.

   Your preference file is now loaded into the WaveView tool.

Mapping Bus Values

To open a bus mnemonic map file, click **Settings** on the **Waveform** tab, and choose **Bus Mnemonic Map** from the menu that opens.

Bus values can be mapped to predefined text strings. You can provide a translation table in an external file using the following syntax:

Syntax

```
# comment lines start with '#'
value1 name1
value2 name2
…
```

The values are the bus values to be mapped.

The following formats are supported:

- 2400
- 0xAF32
- 'hAF32
- 'b1001_0100_0101
- 'd2400

Feedback

- 'b110110111100
- 'o371

The names are mapped to text strings. A text string must be quoted if the name contains spaces.

Wildcards are supported in Bus Mnemonic expressions. The ? character is a single bit for wildcards. It works only on ’b, ’o, and ’h.

For example:

```
2'b1?
3'o1?1
4'h1?1?
```

11

Troubleshooting

This chapter details troubleshooting the WaveView tool.

The following topics are available in this chapter:

- Linux Platforms
- X-Window Font Warnings
- XmTextField Font Warning
- Cannot Change Flexlm License File

Linux Platforms

The following troubleshooting categories are available for Linux platforms:

- Screen Refresh
- Alternative Methods for Backingstore Setup
- With GNOME
- Program Fails During Startup on Linux Platforms

Screen Refresh

When running in Linux 7.0 or later, waveview screen content is not refreshed after being blocked by other windows.

The problem is related to the backing store setting of your X server. Waveview windows in the WaveView tool rely on the backing store function of X server to refresh screen content.

You have two options to solve this problem: Turn on the backing store setting of your X-window server, or turn on the **Internal Screen Refresh** option in the Preference Settings.

To check the backing store setting of your X server, run the following:

```
xdpyinfo
```

Feedback

If the following output appears, the backing store function of your X server is turned off:

```
options: backing-store NO, save-unders NO
```

Enable backing store of your X server by inserting the following line in the Screen section of your X server configuration file (normally `/etc/X11/XF86Config or /etc/X11/XF86Config-4`, but `backing_store` also works):

```
Option "backingstore"
```

Alternatively, you can run your X server with the option +bs to obtain the same result without editing your configuration file. Restart your X server, then run the `xdpyinfo` command again to verify that backing store is enabled. Recent Red Hat Linux distributions (7.0 and 7.1) include XFree86 4.x binaries with backing store disabled by default.

The backingstore option might not work properly if overlay mode is enabled.

Alternative Methods for Backingstore Setup

To change the backingstore attribute, you must have root privileges.

Locate the XServers File

This file can be in a variety of locations. Most Linux distributions place the file in /etc/X11/xdm/Xservers. You might have to look around for your Xserver configuration files.

Enable Backing-store for Your Xserver

The following example is of a /var/X11/xdm/Xservers file:

```
:0 /usr/bin/X11/X +bs
```

The `+bs` server command-line option turns on the backing store.

Restart Your Xserver to Enable the Changes

This varies system to system, but usually logging out and back in again makes the program that starts the Xserver read in the Xservers file again. On Linux you can also restart the Xserver by pressing **Control + Alt + Backspace**

With GNOME

If you are using GNOME, modify /etc/X11/gdm/gdm.conf and change the following lines from:

```
[servers]
```

```
0=/usr/bin/X11/X
```

Feedback

to

```
[servers]

0=/usr/bin/X11/X +bs
```

Then, restart your Xserver.

Program Fails During Startup on Linux Platforms

This problem might happen if the executable and Linux OS version are not compatible. The fail occurs in the license initialization code used by the Flexlm license manager. Linux is known to have a wide range of kernel variations; fails might occur if the system calls in the kernel are not compatible with the system calls required by the license manager code.

The current release includes binary for Linux 6, 7, and 8 OS. Lower OS executable might fail on a higher OS. For example, the Linux 6 binary might fail on Linux 7/8. Make sure that you select a proper executable for your specific Linux OS.

X-Window Font Warnings

When the WaveView tool is started on a remote host and displayed on a local X-Windows server, the application might start with some Motif Font warning messages if the default character set cannot be found on the local display. The following warning messages are typical:

- Warning: Missing charsets in String to FontSet conversion
- Warning: Cannot convert string "-dt-interface system-medium-r-normal-l*-*-*-*-*-*-*-*-*-*" to type FontSet
- Warning: Missing charsets in String to FontSet conversion
- Warning: Unable to load any usable fontset
- Warning:

  Name: FONTLIST_DEFAULT_TAG_STRING

  Class: XmRendition

  Conversion failed. Cannot load font.
- Warning:

  Name: FONTLIST_DEFAULT_TAG_STRING

Feedback

Class: XmRendition

Conversion failed. Cannot load font.

As a result, text messages in some of the dialog boxes might disappear. To solve this problem, follow these steps to modify the X font resource settings.

1. Save the X setting by running:

   ```
   xrdb -query > xrdb_org
   ```

2. Edit file "xrdb_org" to change the values of the following resources to use "fixed" (or 9x15) as the default font:

   ```
   *FontList: fixed

   *buttonFontList: fixed

   *labelFontList: fixed

   *textFontList: fixed
   ```

3. After editing, run the following line to apply the change to your local X-Windows server:

   ```
   xrdb -load xrdb_org
   ```

XmTextField Font Warning

Symptom: Cannot type text in "text field" entry in dialog boxes.

This problem is a Motif X-Server problem. The following warning message is issued in a non-English X-Windows environment:

Warning:

```
Name: td_text

Class: XmTextField

Character '\123' not supported in font. Discarded.
```

This warning message is usually issued multiple times when the file loading dialog box (or any other dialog box with a text field) is open. In the case of the file loading dialog box, file name text string is not automatically filled into the File Name field when a file entry is selected.

To solve this problem, the following Linux environment variables need to be set:

```
setenv LANG C (or setenv LANG en_US.iso88591)

setenv LC_CTYPE C
```

Feedback

Cannot Change Flexlm License File

The Flexlm license manager stores the location of the license file after the application successfully starts. The stored license file is used as the default license file the next time the application starts.

The stored setting needs to be removed when installing a new license file to a different location.

To remove the default setting on Linux, edit the file .flexlmrc in your home directory to remove the line starting with SANDWORK_LICENSE_FILE.

To remove the default setting on Windows, click the **Start** button, choose **Run**, and enter `regedit` to edit the Windows Registry. Browse to **HKEY_LOCAL_MACHINE > SOFTWARE > FLEXlm License Manager** and delete the `SANDWORK_LICENSE_FILE` entry.